KODEKLOUD

Course Objectives

- Core Concepts
  - Cluster Architecture
  - API Primitives
  - Services & Other Network Primitives
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Networking
- Installation, Configuration & Validation
- Troubleshooting

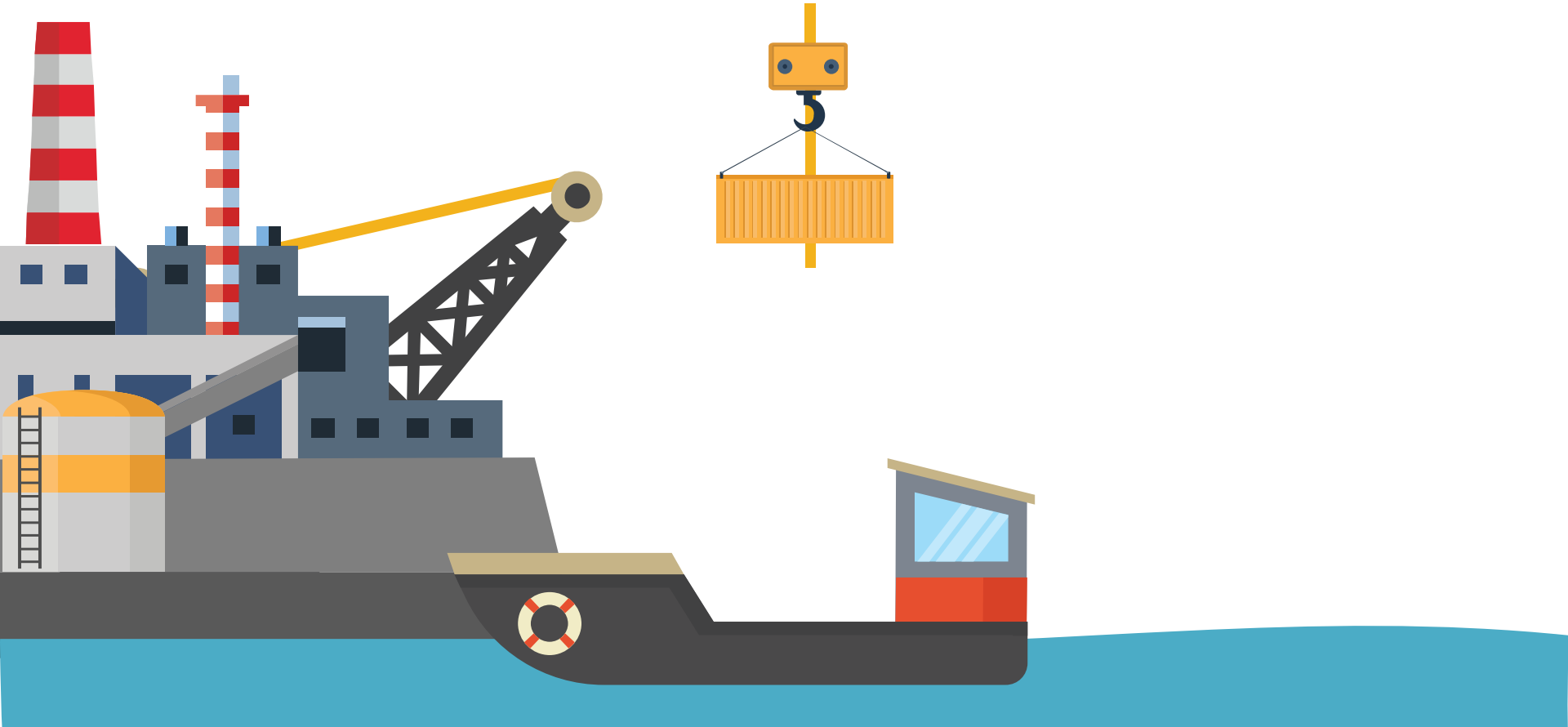
KODEKLOUD

Cluster Architecture

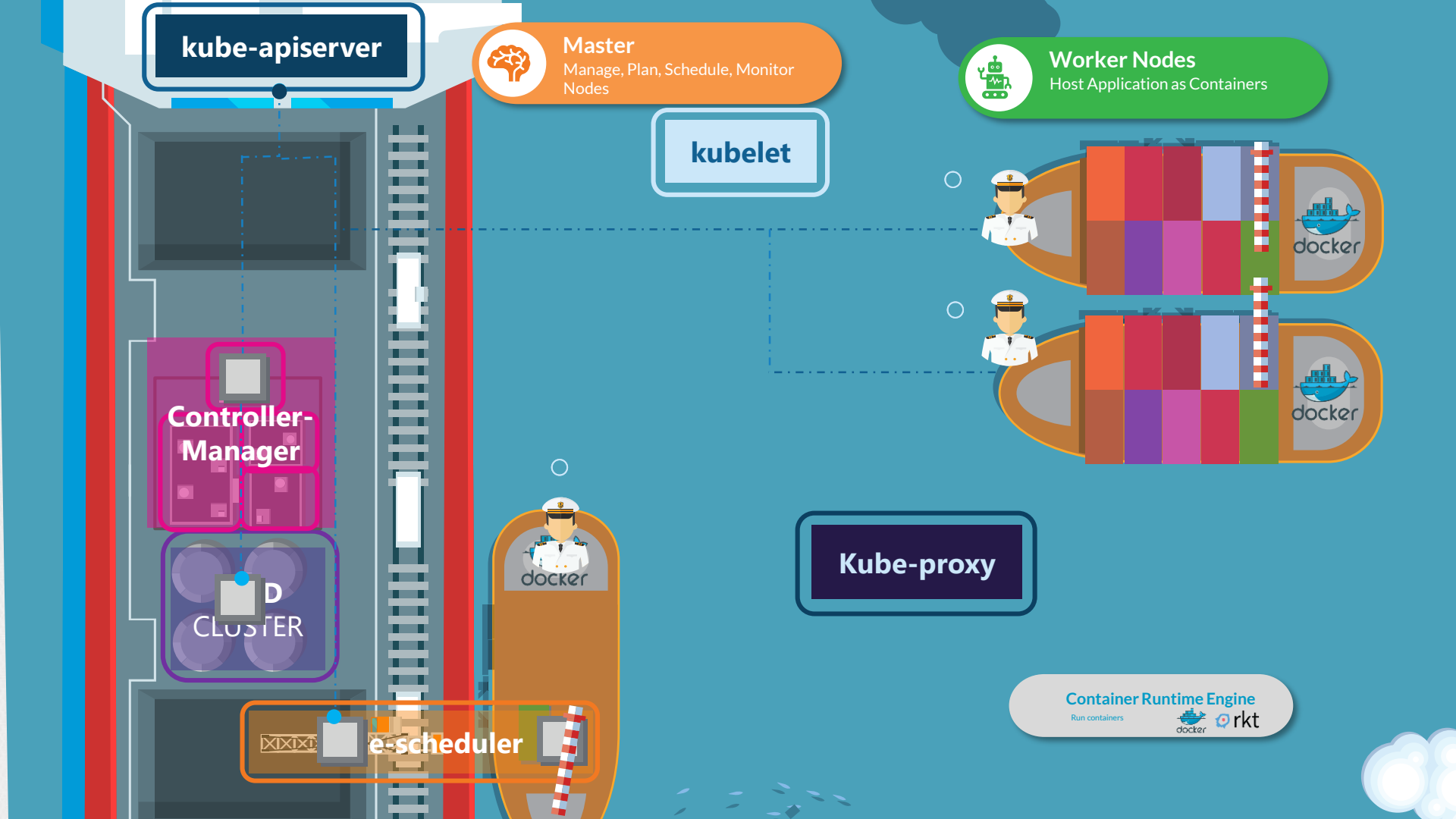
- ❑ Kubernetes Architecture
- ❑ ETCD For Beginners
- ❑ ETCD in Kubernetes
- ❑ Kube-API Server
- ❑ Controller Managers
- ❑ Kube Scheduler
- ❑ Kubelet
- ❑ Kube Proxy

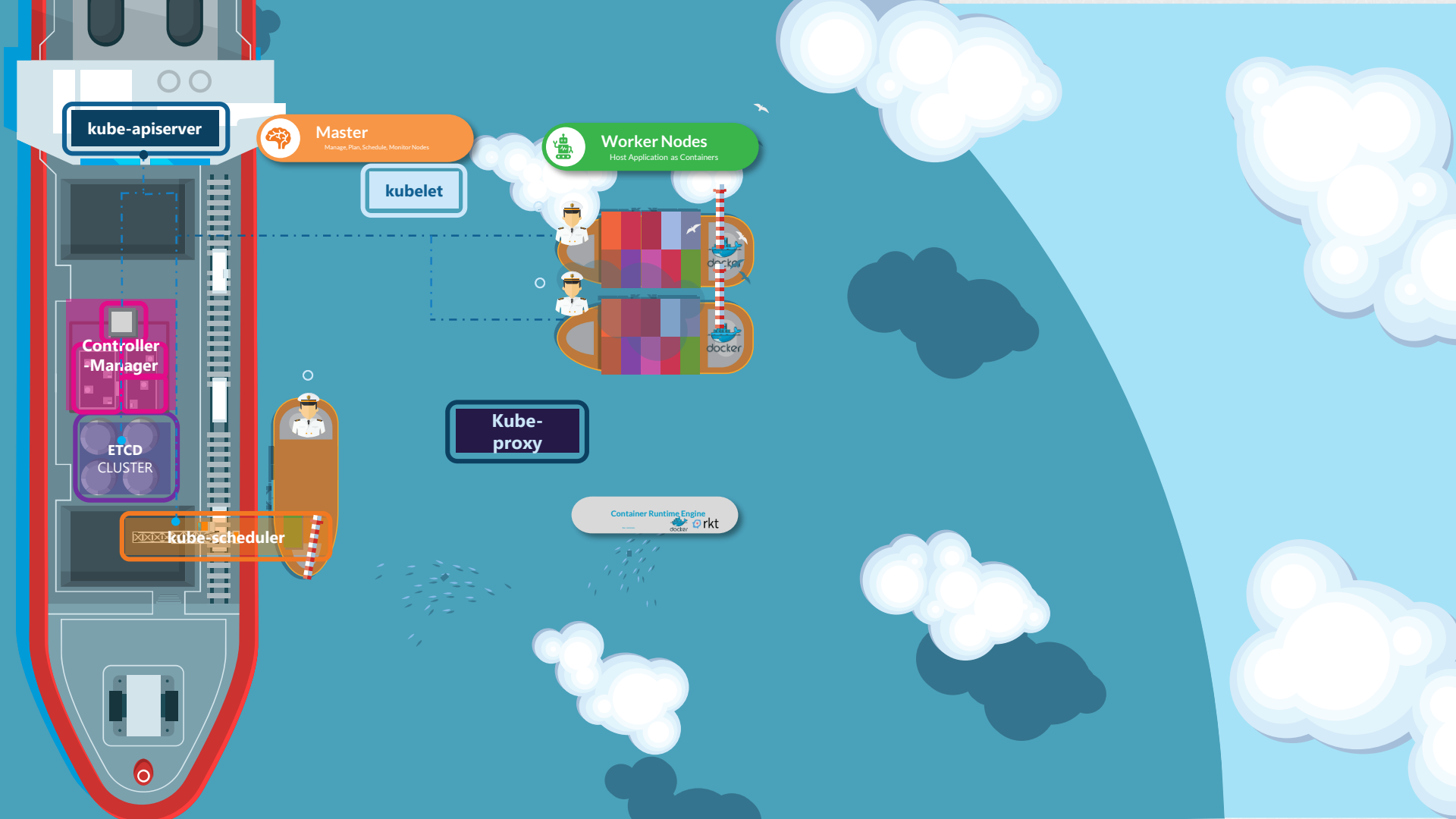
KODEKLOUD

KUBERNETES ARCHITECTURE

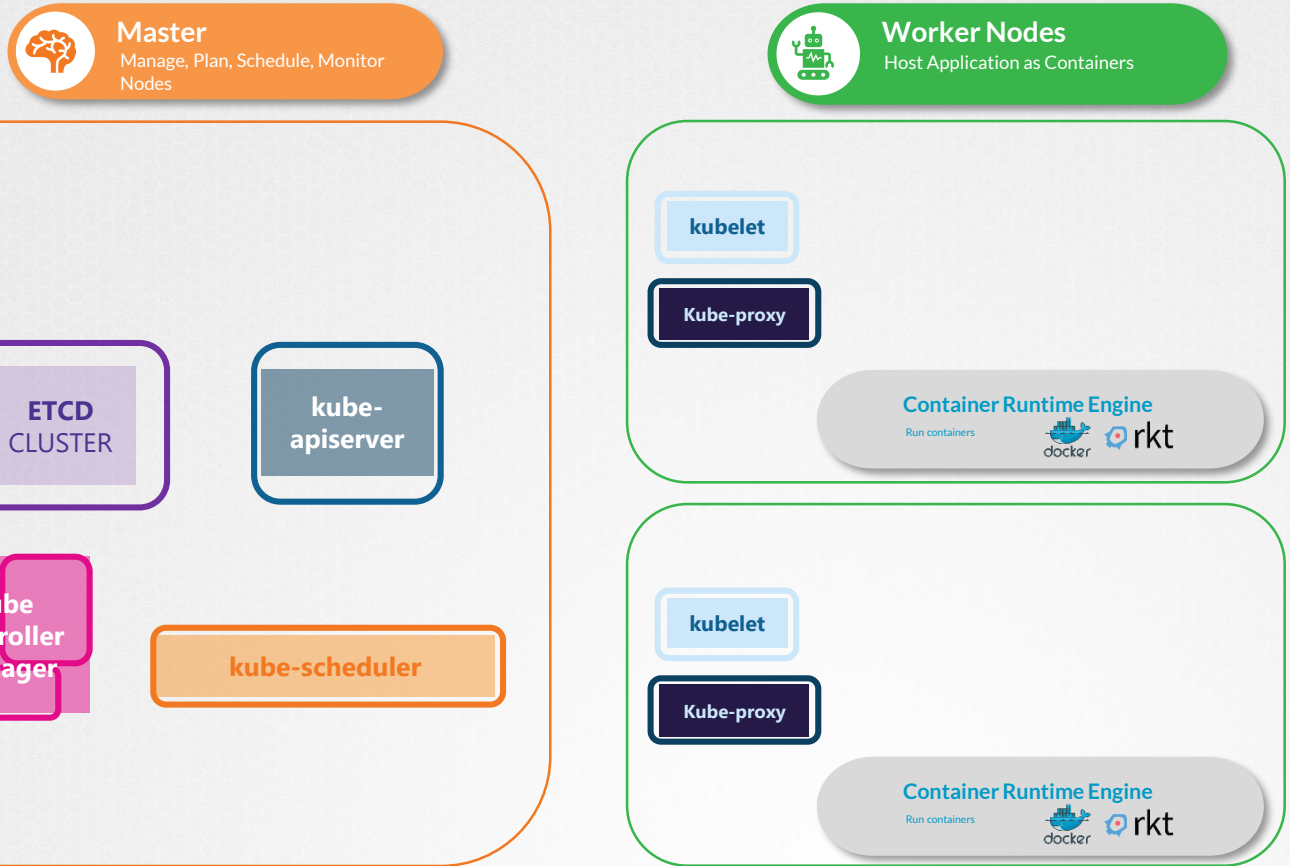


Master
Manage, Plan, Schedule, Monitor Nodes
Worker Nodes
Host Application as Containers


kube-apiserver
Master
Manage, Plan, Schedule, Monitor Nodes
Worker Nodes
Host Application as Containers
kubelet
Controller-Manager
D
CLUSTER
e-scheduler
docker
docker
docker
Kube-proxy
Container Runtime Engine
Run containers
docker
rkt


kube-apiserver
Master
Manage, Plan, Schedule, Monitor Nodes
kubelet
Worker Nodes
Host Application as Containers
Controller -Manager
ETCD CLUSTER
kube-scheduler
Kube-proxy
Container Runtime Engine
docker
rkt

Kubernetes Architecture



KODEKLOUD

KODEKLOUD

ETCD

FOR BEGINNERS

Objectives

- What is ETCD?
- What is a Key-Value Store?
- How to get started quickly?
- How to operate ETCD?
- What is a distributed system?
- How ETCD Operates
- RAFT Protocol
- Best practices on number of nodes

KODEKLOUD

ETCD is a distributed reliable key-value store that is Simple, Secure & Fast

key-value store

Tabular/Relational Databases

Name	Age	Location	Salary	Grade
John Doe	45	New York	5000	
Dave Smith	34	New York	4000	
Aryan Kumar	10	New York		A
Lauren Rob	13	Bangalore		C
Lily Oliver	15	Bangalore		B

KODEKLOUD

key-value store

Key	Value
Name	John Doe
Age	45
Location	New York
Salary	5000

Key	Value
Name	Dave Smith
Age	34
Location	New York
Salary	4000
Organization	ACME

Key	Value
Name	Aryan Kumar
Age	10
Location	New York
Grade	A

Key	Value
Name	Lauren Rob
Age	13
Location	Bangalore
Grade	C

Key	Value
Name	Lily Oliver
Age	15
Location	Bangalore
Grade	B

KODEKLOUD

key-value store

```
{
  "name": "John Doe",
  "age": 45,
  "location": "New York",
  "salary": 5000
}
```

```
{
  "name": "Dave Smith",
  "age": 34,
  "location": "New York",
  "salary": 4000,
  "organization": "ACME"
}
```

```
{
  "name": "Aryan Kumar",
  "age": 10,
  "location": "New York",
  "Grade": "A"
}
```

```
{
  "name": "Lily Oliver",
  "age": 15,
  "location": "Bangalore",
  "Grade": "B"
}
```

```
{
  "name": "Lauren Rob",
  "age": 13,
  "location": "Bangalore",
  "Grade": "C"
}
```

KODEKLOUD

Install ETCD

1. Download Binaries

```
curl -L https://github.com/etcd-io/etcd/releases/download/v3.3.11/etcd-v3.3.11-linux-amd64.tar.gz -o etcd-v3.3.11-linux-amd64.tar.gz
```

2. Extract

```
tar xzvf etcd-v3.3.11-linux-amd64.tar.gz
```

3. Run ETCD Service

```
./etcd
```

KODEKLOUD

Operate ETCD

3. Run ETCD Service

```
./etcd
```

```
./etcdctl set key1 value1
```

```
./etcdctl get key1
value1
```

```
./etcdctl
NAME:
   etcdctl - A simple command line client for etcd.

COMMANDS:
     backup          backup an etcd directory
     cluster-health  check the health of the etcd cluster
     mk              make a new key with a given value
     mkdir           make a new directory
     rm              remove a key or a directory
     rmdir           removes the key if it is an empty directory or a key-value pair
     get             retrieve the value of a key
```

KODEKLOUD

KODEKLOUD

Course Objectives

- Core Concepts
  - Cluster Architecture
  - API Primitives
  - Services & Other Network Primitives
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Networking
- Installation, Configuration & Validation
- Troubleshooting

KODEKLOUD

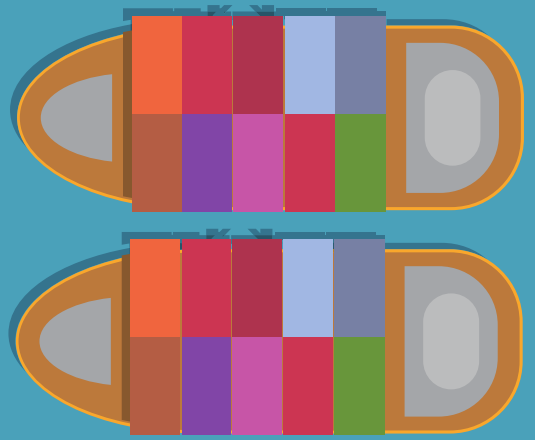
ETCD

In Kubernetes

Master
Manage, Plan, Schedule, Monitor Nodes

- Nodes
- PODs
- Configs
- Secrets
- Accounts
- Roles
- Bindings
- Others

ETCD
CLUSTER



Setup - Manual

```
wget -q --https-only \
    "https://github.com/coreos/etcd/releases/download/v3.3.9/etcd-v3.3.9-linux-amd64.tar.gz"
```

etcd.service

```
ExecStart=/usr/local/bin/etcd \\
  --name ${ETCD_NAME} \\
  --cert-file=/etc/etcd/kubernetes.pem \\
  --key-file=/etc/etcd/kubernetes-key.pem \\
  --peer-cert-file=/etc/etcd/kubernetes.pem \\
  --peer-key-file=/etc/etcd/kubernetes-key.pem \\
  --trusted-ca-file=/etc/etcd/ca.pem \\
  --peer-trusted-ca-file=/etc/etcd/ca.pem \\
  --peer-client-cert-auth \\
  --client-cert-auth \\
  --initial-advertise-peer-urls https://${INTERNAL_IP}:2380 \\
  --listen-peer-urls https://${INTERNAL_IP}:2380 \\
  --listen-client-urls https://${INTERNAL_IP}:2379,https://127.0.0.1:2379 \\
  --advertise-client-urls https://${INTERNAL_IP}:2379 \\
  --initial-cluster-token etcd-cluster-0 \\
  --initial-cluster controller-0=https://${CONTROLLER0_IP}:2380,controller-1=https://${CONTROLLER1_IP}:2380 \\
  --initial-cluster-state new \\
  --data-dir=/var/lib/etcd
```

KODEKLOUD

Setup - kubeadm

```
kubectl get pods -n kube-system
```

```
NAMESPACE     NAME                              READY     STATUS    RESTARTS   AGE
kube-system   coredns-78fcdf6894-prwvl          1/1       Running   0          1h
kube-system   coredns-78fcdf6894-vqd9w          1/1       Running   0          1h
kube-system   etcd-master                       1/1       Running   0          1h
kube-system   kube-apiserver-master             1/1       Running   0          1h
kube-system   kube-controller-manager-master    1/1       Running   0          1h
kube-system   kube-proxy-f6k26                  1/1       Running   0          1h
kube-system   kube-proxy-hnzsw                  1/1       Running   0          1h
kube-system   kube-scheduler-master             1/1       Running   0          1h
kube-system   weave-net-924k8                   2/2       Running   1          1h
kube-system   weave-net-hzfcz                   2/2       Running   1          1h
```

```
kubectl exec etcd-master -n kube-system etcdctl get / --prefix -keys-only
```

```
/registry/apiregistration.k8s.io/apiservices/v1.
/registry/apiregistration.k8s.io/apiservices/v1.apps
/registry/apiregistration.k8s.io/apiservices/v1.authentication.k8s.io
/registry/apiregistration.k8s.io/apiservices/v1.authorization.k8s.io
/registry/apiregistration.k8s.io/apiservices/v1.autoscaling
/registry/apiregistration.k8s.io/apiservices/v1.batch
/registry/apiregistration.k8s.io/apiservices/v1.networking.k8s.io
/registry/apiregistration.k8s.io/apiservices/v1.rbac.authorization.k8s.io
/registry/apiregistration.k8s.io/apiservices/v1.storage.k8s.io
/registry/apiregistration.k8s.io/apiservices/v1beta1.admissionregistration.k8s.io
```

Run inside the etcd-master POD

KODEKLOUD

Explore ETCD

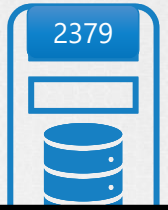
```
kubectl exec etcd-master -n kube-system etcdctl get / --prefix -keys-only
/registry/apiregistration.k8s.io/apiservices/v1.
/registry/apiregistration.k8s.io/apiservices/v1.apps
/registry/apiregistration.k8s.io/apiservices/v1.authentication.k8s.io
/registry/apiregistration.k8s.io/apiservices/v1.authorization.k8s.io
/registry/apiregistration.k8s.io/apiservices/v1.autoscaling
/registry/apiregistration.k8s.io/apiservices/v1.batch
/registry/apiregistration.k8s.io/apiservices/v1.networking.k8s.io
/registry/apiregistration.k8s.io/apiservices/v1.rbac.authorization.k8s.io
/registry/apiregistration.k8s.io/apiservices/v1.storage.k8s.io
/registry/apiregistration.k8s.io/apiservices/v1beta1.admissionregistration.k8s.io
```

Run inside the etcd-master POD

- Registry
  - minions
  - pods
  - replicasets
  - deployments
  - roles
  - secrets

KODEKLOUD

ETCD in HA Environment



etcd.service

```
ExecStart=/usr/local/bin/etcd \\
  --name ${ETCD_NAME} \\
  --cert-file=/etc/etcd/kubernetes.pem \\
  --key-file=/etc/etcd/kubernetes-key.pem \\
  --peer-cert-file=/etc/etcd/kubernetes.pem \\
  --peer-key-file=/etc/etcd/kubernetes-key.pem \\
  --trusted-ca-file=/etc/etcd/ca.pem \\
  --peer-trusted-ca-file=/etc/etcd/ca.pem \\
  --peer-client-cert-auth \\
  --client-cert-auth \\
  --initial-advertise-peer-urls https://${INTERNAL_IP}:2380 \\
  --listen-peer-urls https://${INTERNAL_IP}:2380 \\
  --listen-client-urls https://${INTERNAL_IP}:2379,https://127.0.0.1:2379 \\
  --advertise-client-urls https://${INTERNAL_IP}:2379 \\
  --initial-cluster-token etcd-cluster-0 \\
  --initial-cluster controller-0=https://${CONTROLLER0_IP}:2380,controller-1=https://${CONTROLLER1_IP}:2380 \\
  --initial-cluster-state new \\
  --data-dir=/var/lib/etcd
```

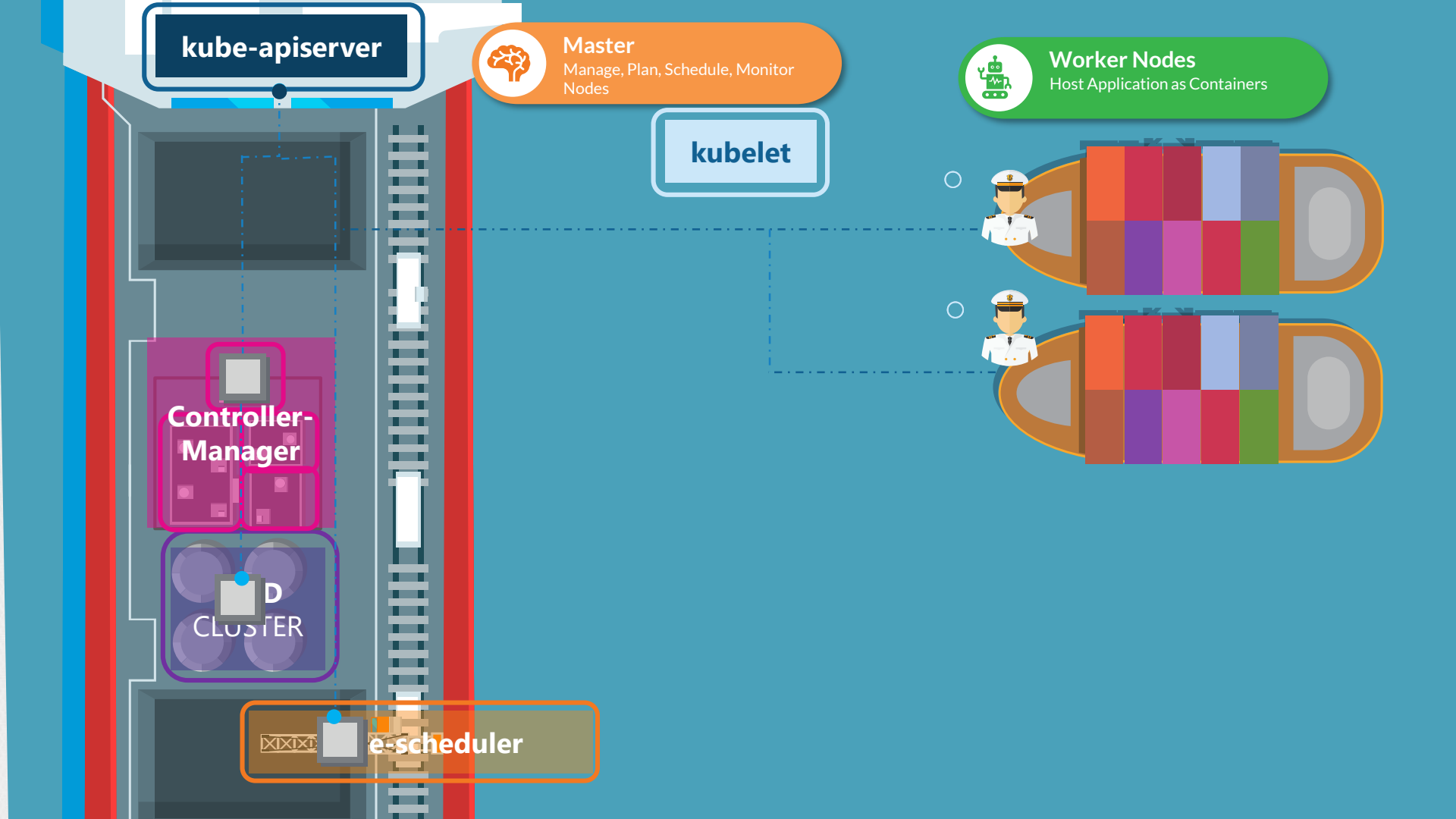
KODEKLOUD

Course Objectives

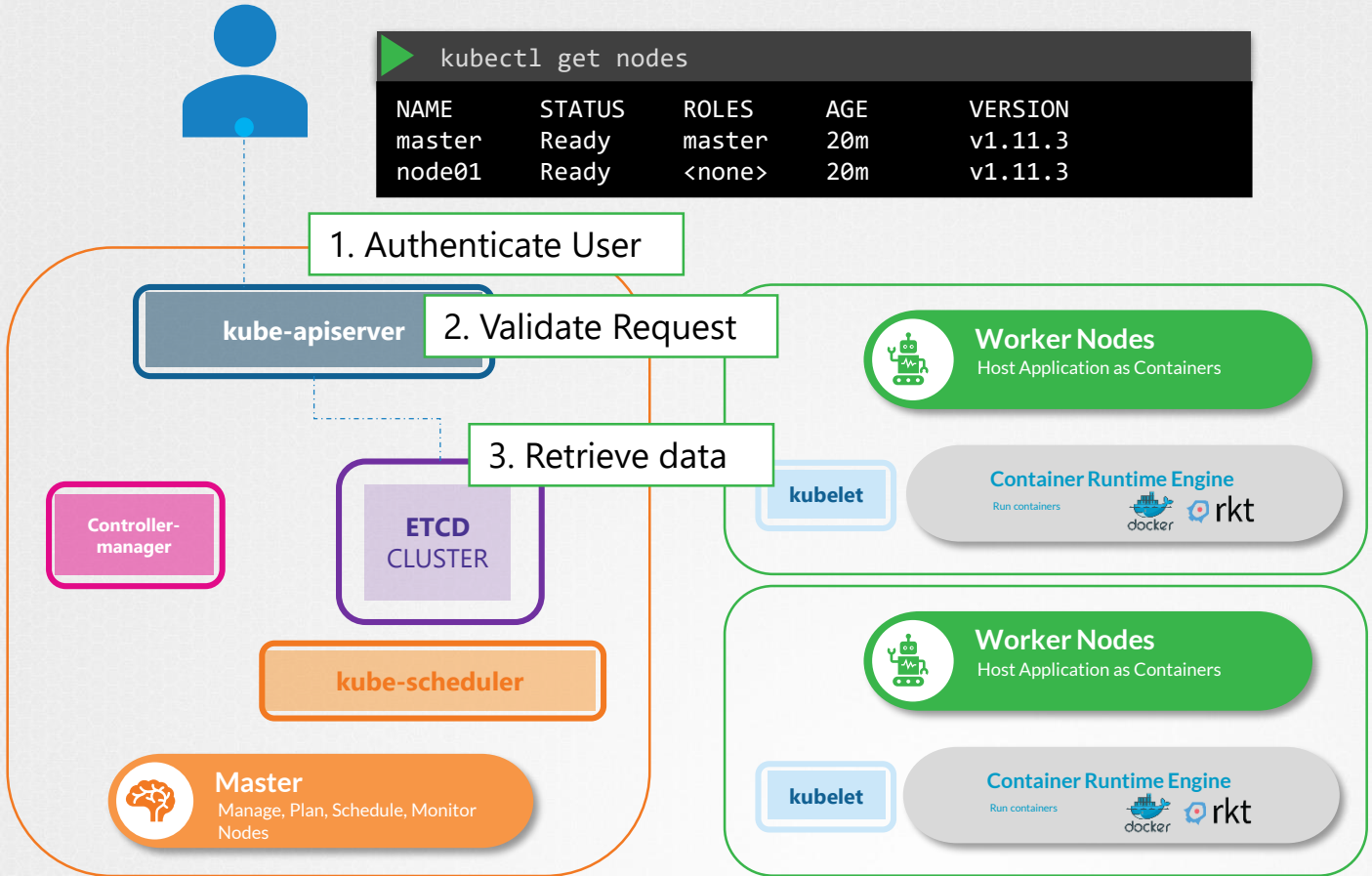
- Core Concepts
  - Cluster Architecture
  - API Primitives
  - Services & Other Network Primitives
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Networking
- Installation, Configuration & Validation
- Troubleshooting

KODEKLOUD

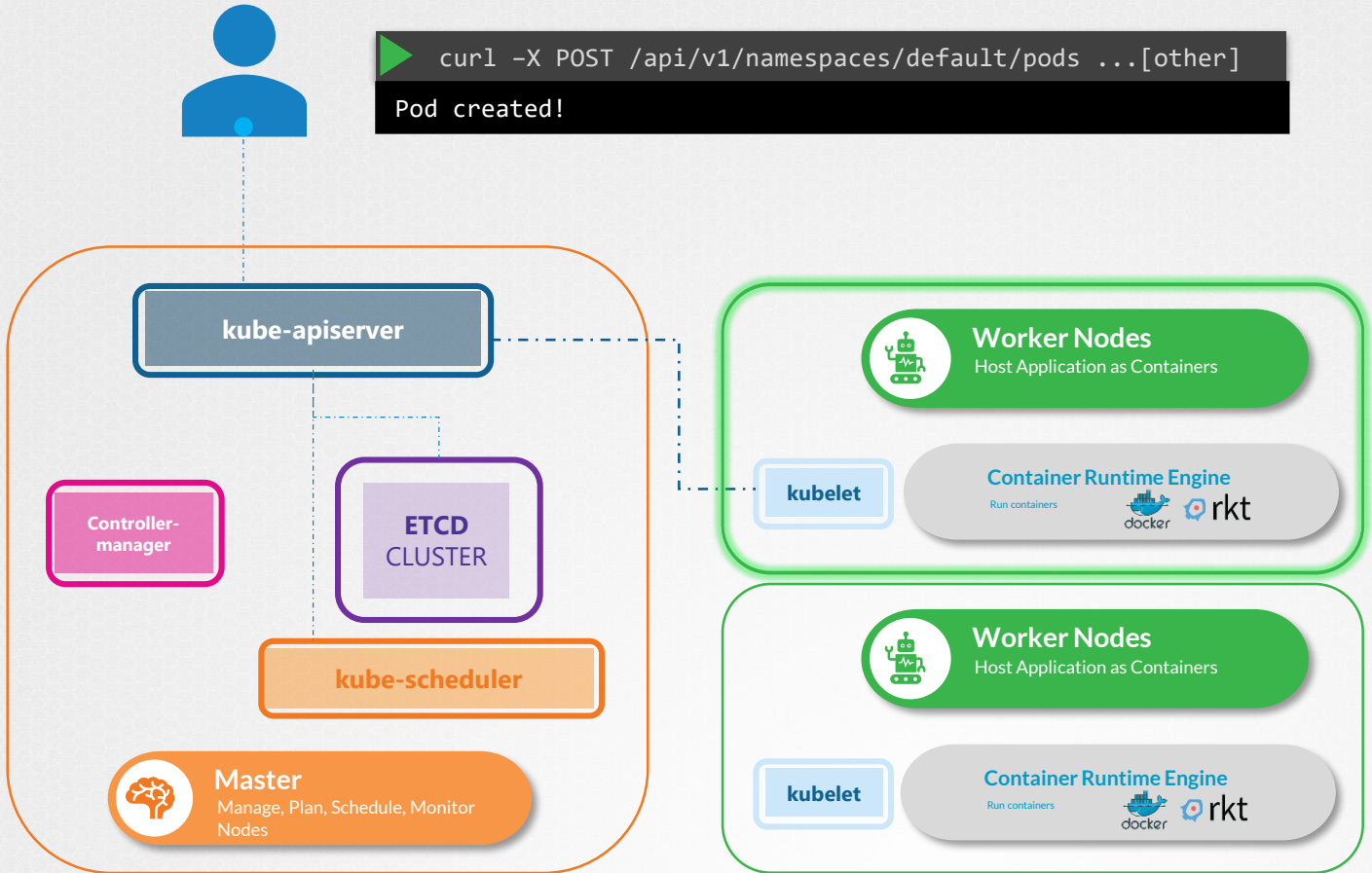
kube-api server


kube-apiserver
Master
Manage, Plan, Schedule, Monitor Nodes
kubelet
Worker Nodes
Host Application as Containers
Controller-Manager
D
CLUSTER
e-scheduler

Kubernetes Architecture



Kubernetes Architecture



1. Authenticate User
2. Validate Request
3. Retrieve data
4. Update ETCD
5. Scheduler
6. Kubelet

Kube-api Server

1. Authenticate User
2. Validate Request
3. Retrieve data
4. Update ETCD
5. Scheduler
6. Kubelet

KODEKLOUD

Installing kube-api server

```
wget https://storage.googleapis.com/kubernetes-release/release/v1.13.0/bin/linux/amd64/kube-apiserver
```

kube-apiserver.service

```
ExecStart=/usr/local/bin/kube-apiserver \\
  --advertise-address=${INTERNAL_IP} \\
  --allow-privileged=true \\
  --apiserver-count=3 \\
  --authorization-mode=Node,RBAC \\
  --bind-address=0.0.0.0 \\
  --enable-admission-
plugins=Initializers,NamespaceLifecycle,NodeRestriction,LimitRanger,ServiceAccount,DefaultStorageClass,Reso
urceQuota \\
  --enable-swagger-ui=true \\
  --etcd-servers=https://127.0.0.1:2379 \\
  --event-ttl=1h \\
  --experimental-encryption-provider-config=/var/lib/kubernetes/encryption-config.yaml \\
  --runtime-config=api/all \\
  --service-account-key-file=/var/lib/kubernetes/service-account.pem \\
  --service-cluster-ip-range=10.32.0.0/24 \\
  --service-node-port-range=30000-32767 \\
  --v=2
```

KLOUD

View api-server - kubeadm

```
kubectl get pods -n kube-system
```

```
NAMESPACE     NAME                              READY     STATUS    RESTARTS   AGE
kube-system   coredns-78fcdf6894-hwrq9          1/1       Running   0          16m
kube-system   coredns-78fcdf6894-rzhjr          1/1       Running   0          16m
kube-system   etcd-master                       1/1       Running   0          15m
kube-system   kube-apiserver-master             1/1       Running   0          15m
kube-system   kube-controller-manager-master    1/1       Running   0          15m
kube-system   kube-proxy-lzt6f                  1/1       Running   0          16m
kube-system   kube-proxy-zm5qd                  1/1       Running   0          16m
kube-system   kube-scheduler-master             1/1       Running   0          15m
kube-system   weave-net-29z42                   2/2       Running   1          16m
kube-system   weave-net-snmdl                   2/2       Running   1          16m    -
```

KODEKLOUD

View api-server options - kubeadm

```
cat /etc/kubernetes/manifests/kube-apiserver.yaml
```

```
spec:
  containers:
  - command:
    - kube-apiserver
    - --authorization-mode=Node,RBAC
    - --advertise-address=172.17.0.32
    - --allow-privileged=true
    - --client-ca-file=/etc/kubernetes/pki/ca.crt
    - --disable-admission-plugins=PersistentVolumeLabel
    - --enable-admission-plugins=NodeRestriction
    - --enable-bootstrap-token-auth=true
    - --etcd-cafile=/etc/kubernetes/pki/etcd/ca.crt
    - --etcd-certfile=/etc/kubernetes/pki/apiserver-etcd-client.crt
    - --etcd-keyfile=/etc/kubernetes/pki/apiserver-etcd-client.key
    - --etcd-servers=https://127.0.0.1:2379
    - --insecure-port=0
    - --kubelet-client-certificate=/etc/kubernetes/pki/apiserver-kubelet-client.crt
    - --kubelet-client-key=/etc/kubernetes/pki/apiserver-kubelet-client.key
    - --kubelet-preferred-address-types=InternalIP,ExternalIP,Hostname
    - --proxy-client-cert-file=/etc/kubernetes/pki/front-proxy-client.crt
    - --proxy-client-key-file=/etc/kubernetes/pki/front-proxy-client.key
    - --requestheader-allowed-names=front-proxy-client
    - --requestheader-client-ca-file=/etc/kubernetes/pki/front-proxy-ca.crt
    - --requestheader-extra-headers-prefix=X-Remote-Extra-
    - --requestheader-group-headers=X-Remote-Group
    - --requestheader-username-headers=X-Remote-User
```

KODEKLOUD

View api-server options

```
cat /etc/systemd/system/kube-apiserver.service
```

```
[Service]
ExecStart=/usr/local/bin/kube-apiserver \\
  --advertise-address=${INTERNAL_IP} \\
  --allow-privileged=true \\
  --apiserver-count=3 \\
  --audit-log-maxage=30 \\
  --audit-log-maxbackup=3 \\
  --audit-log-maxsize=100 \\
  --audit-log-path=/var/log/audit.log \\
  --authorization-mode=Node,RBAC \\
  --bind-address=0.0.0.0 \\
  --client-ca-file=/var/lib/kubernetes/ca.pem \\
  --enable-admission-
plugins=Initializers,NamespaceLifecycle,NodeRestriction,LimitRanger,ServiceAccount,Defa
ultStorageClass,ResourceQuota \\
  --enable-swagger-ui=true \\
  --etcd-cafile=/var/lib/kubernetes/ca.pem \\
  --etcd-certfile=/var/lib/kubernetes/kubernetes.pem \\
  --etcd-keyfile=/var/lib/kubernetes/kubernetes-key.pem \\
  --etcd-
servers=https://10.240.0.10:2379,https://10.240.0.11:2379,https://10.240.0.12:2379 \\
  --event-ttl=1h \\
  --experimental-encryption-provider-config=/var/lib/kubernetes/encryption-config.yaml
\\
  --kubelet-certificate-authority=/var/lib/kubernetes/ca.pem \\
  --kubelet-client-certificate=/var/lib/kubernetes/kubernetes.pem \\
```

KODEKLOUD

View api-server options

```
ps -aux | grep kube-apiserver
```

```
root      2348  3.3 15.4 399040 315604 ?       Ssl  15:46   1:22 kube-apiserver --authorization-mode=Node,RBAC --advertise-address=172.17.0.32 --allow-privileged=true --client-ca-file=/etc/kubernetes/pki/ca.crt --disable-admission-plugins=PersistentVolumeLabel --enable-admission-plugins=NodeRestriction--enable-bootstrap-token-auth=true --etcd-cafile=/etc/kubernetes/pki/etcd/ca.crt --etcd-certfile=/etc/kubernetes/pki/apiserver-etcd-client.crt --etcd-keyfile=/etc/kubernetes/pki/apiserver-etcd-client.key --etcd-servers=https://127.0.0.1:2379 --insecure-port=0 --kubelet-client-certificate=/etc/kubernetes/pki/apiserver-kubelet-client.crt --kubelet-client-key=/etc/kubernetes/pki/apiserver-kubelet-client.key --kubelet-preferred-address-types=InternalIP,ExternalIP,Hostname --proxy-client-cert-file=/etc/kubernetes/pki/front-proxy-client.crt --proxy-client-key-file=/etc/kubernetes/pki/front-proxy-client.key--requestheader-allowed-names=front-proxy-client --requestheader-client-ca-file=/etc/kubernetes/pki/front-proxy-ca.crt --requestheader-extra-headers-prefix=X-Remote-Extra- --requestheader-group-headers=X-Remote-Group --requestheader-username-headers=X-Remote-User --secure-port=6443 --service-account-key-file=/etc/kubernetes/pki/sa.pub --service-cluster-ip-range=10.96.0.0/12 --tls-cert-file=/etc/kubernetes/pki/apiserver.crt --tls-private-key-file=/etc/kubernetes/pki/apiserver.key
```

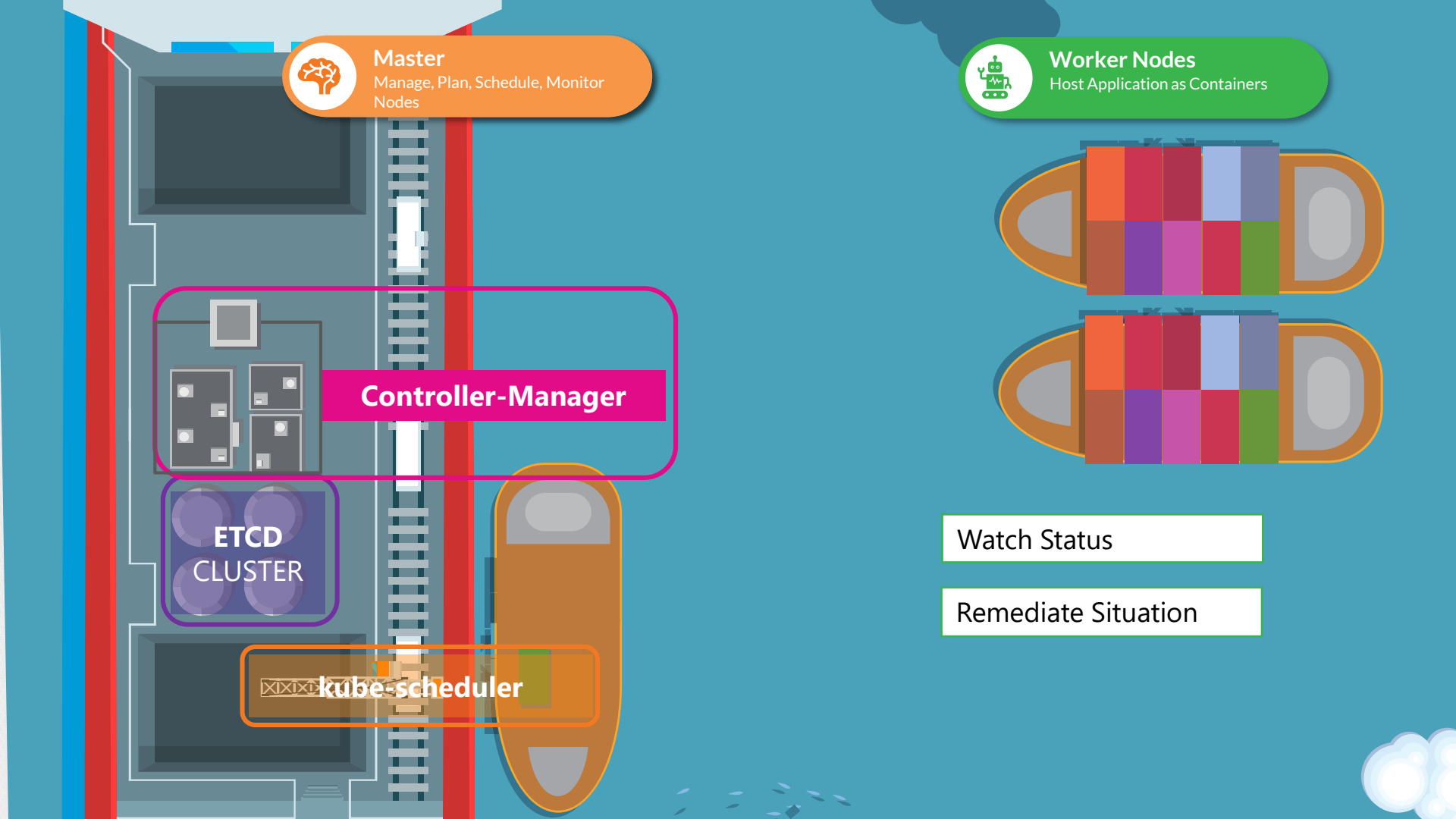
KODEKLOUD

KODEKLOUD

Course Objectives

- Core Concepts
  - Cluster Architecture
  - API Primitives
  - Services & Other Network Primitives
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Networking
- Installation, Configuration & Validation
- Troubleshooting

Kube Controller Manager


Master
Manage, Plan, Schedule, Monitor Nodes
Worker Nodes
Host Application as Containers
Controller-Manager
ETCD
CLUSTER
kube-scheduler
Watch Status
Remediate Situation

Controller

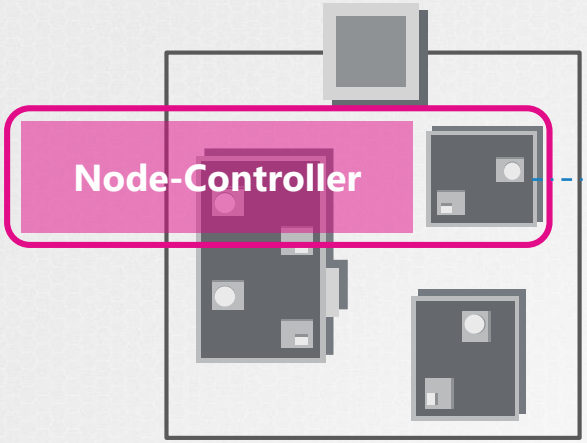
Watch Status

Remediate Situation

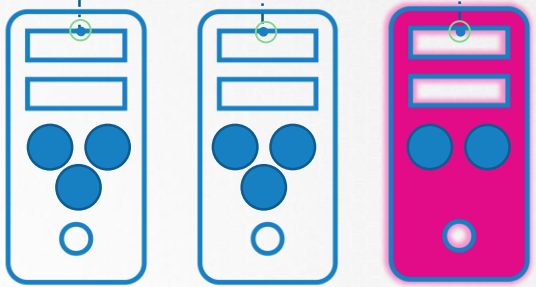
Node Monitor Period = 5s

Node Monitor Grace Period = 40s

POD Eviction Timeout = 5m



kube-apiserver

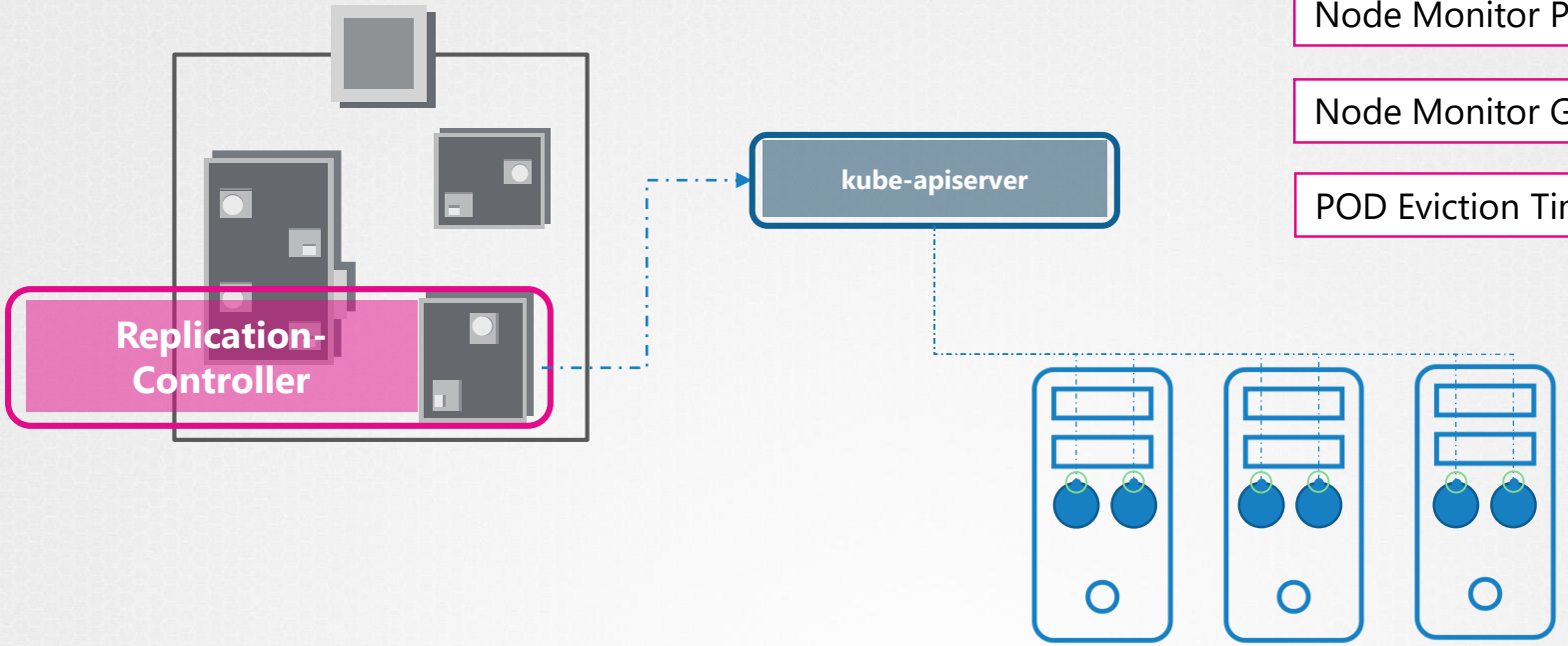


UNREACHABLE

```
kubectl get nodes
NAME       STATUS     ROLES    AGE   VERSION
worker-1   Ready      <none>   8d    v1.13.0
worker-2   NotReady   <none>   8d    v1.13.0
```

KODEKLOUD

Controller



Watch Status

Remediate Situation

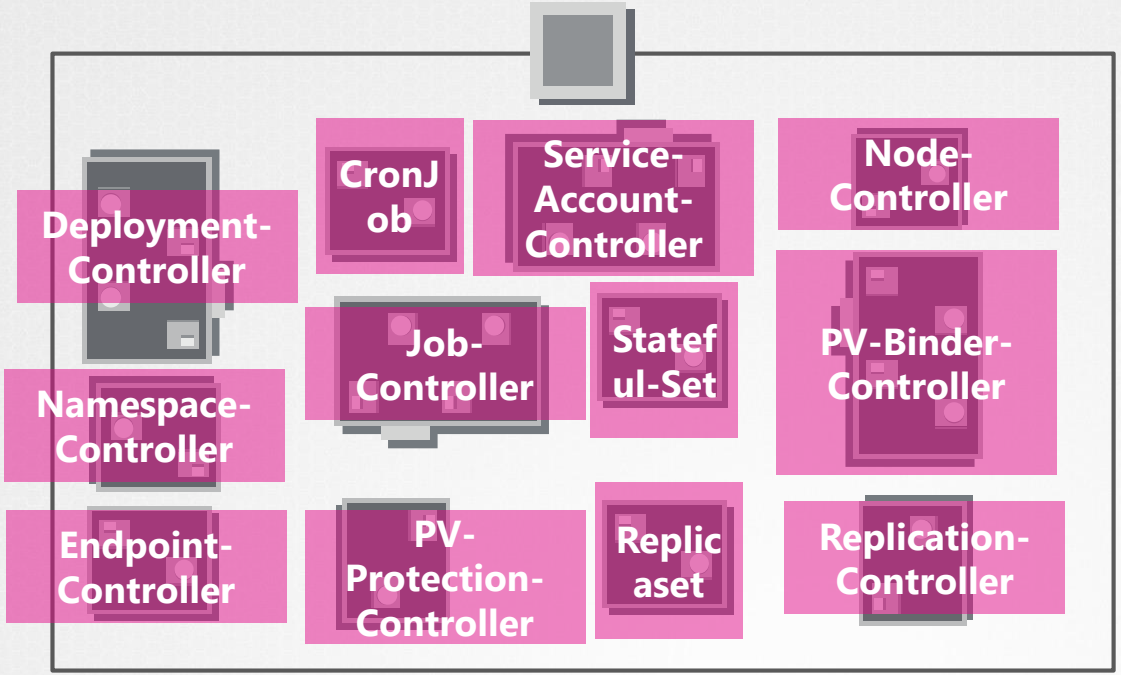
Node Monitor Period = 5s

Node Monitor Grace Period = 40s

POD Eviction Timeout = 5m

KODEKLOUD

Controller



Watch Status

Remediate Situation

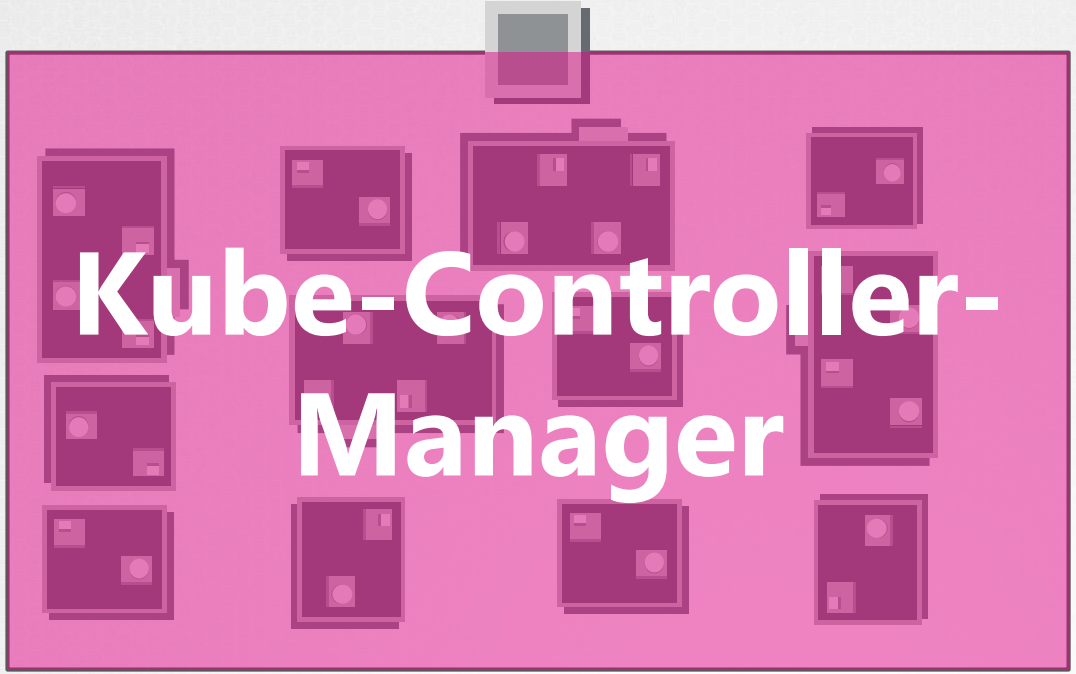
Node Monitor Period = 5s

Node Monitor Grace Period = 40s

POD Eviction Timeout = 5m

KODEKLOUD

Controller



- Watch Status
- Remediate Situation
- Node Monitor Period = 5s
- Node Monitor Grace Period = 40s
- POD Eviction Timeout = 5m

KODEKLOUD

Installing kube-controller-manager

```
wget https://storage.googleapis.com/kubernetes-release/release/v1.13.0/bin/linux/amd64/kube-controller-manager
```

kube-controller-manager.service

```
ExecStart=/usr/local/bin/kube-controller-manager \\
  --address=0.0.0.0 \\
  --cluster-cidr=10.200.0.0/16 \\
  --cluster-name=kubernetes \\
  --cluster-signing-cert-file=/var/lib/kubernetes/ca.pem \\
  --cluster-signing-key-file=/var/lib/kubernetes/ca-key.pem \\
  --kubeconfig=/var/lib/kubernetes/kube-controller-manager.kubeconfig \\
  --leader-elect=true \\
  --root-ca-file=/var/lib/kubernetes/ca.pem \\
  --service-account-private-key-file=/var/lib/kubernetes/service-account-key.pem \\
  --service-cluster-ip-range=10.32.0.0/24 \\
  --use-service-account-credentials=true \\
  --v=2
```

```
N --node-monitor-period=5s
```

```
d=40s
```

```
s
```

```
--controllers stringSlice     Default: [*]
A list of controllers to enable. '*' enables all on-by-default controllers, 'foo' enables the controller
named 'foo', '-foo' disables the controller named 'foo'.
All controllers: attachdetach, bootstrapsigner, clusterrole-aggregation, cronjob, csrapproving,
csrcleaner, csrsigning, daemonset, deployment, disruption, endpoint, garbagecollector,
horizontalpodautoscaling, job, namespace, nodeipam, nodelifecycle, persistentvolume-binder,
persistentvolume-expander, podgc, pv-protection, pvc-protection, replicaset, replicationcontroller,
```

KLOUD

Installing kube-controller-manager

```
--controllers stringSlice     Default: [*]
A list of controllers to enable. '*' enables all on-by-default controllers, 'foo' enables the controller
named 'foo', '-foo' disables the controller named 'foo'.
All controllers: attachdetach, bootstrapsigner, clusterrole-aggregation, cronjob, csrapproving,
csrcleaner, csrsigning, daemonset, deployment, disruption, endpoint, garbagecollector,
horizontalpodautoscaling, job, namespace, nodeipam, nodelifecycle, persistentvolume-binder,
persistentvolume-expander, podgc, pv-protection, pvc-protection, replicaset, replicationcontroller,
resourcequota, root-ca-cert-publisher, route, service, serviceaccount, serviceaccount-token, statefulset,
tokencleaner, ttl, ttl-after-finished
Disabled-by-default controllers: bootstrapsigner, tokencleaner
```

KODEKLOUD

View kube-controller-manager - kubeadm

```
kubectl get pods -n kube-system
```

```
NAMESPACE     NAME                              READY     STATUS    RESTARTS   AGE
kube-system   coredns-78fcdf6894-hwrq9          1/1       Running   0          16m
kube-system   coredns-78fcdf6894-rzhjr          1/1       Running   0          16m
kube-system   etcd-master                       1/1       Running   0          15m
kube-system   kube-apiserver-master             1/1       Running   0          15m
kube-system   kube-controller-manager-master    1/1       Running   0          15m
kube-system   kube-proxy-lzt6f                  1/1       Running   0          16m
kube-system   kube-proxy-zm5qd                  1/1       Running   0          16m
kube-system   kube-scheduler-master             1/1       Running   0          15m
kube-system   weave-net-29z42                   2/2       Running   1          16m
kube-system   weave-net-snmdl                   2/2       Running   1          16m    -
```

KODEKLOUD

View kube-controller-manager options - kubeadm

```
cat /etc/kubernetes/manifests/kube-controller-manager.yaml
```

```
spec:
  containers:
  - command:
    - kube-controller-manager
    - --address=127.0.0.1
    - --cluster-signing-cert-file=/etc/kubernetes/pki/ca.crt
    - --cluster-signing-key-file=/etc/kubernetes/pki/ca.key
    - --controllers=*,bootstrapsigner,tokencleaner
    - --kubeconfig=/etc/kubernetes/controller-manager.conf
    - --leader-elect=true
    - --root-ca-file=/etc/kubernetes/pki/ca.crt
    - --service-account-private-key-file=/etc/kubernetes/pki/sa.key
    - --use-service-account-credentials=true
```

KODEKLOUD

View controller-manager options

```
cat /etc/systemd/system/kube-controller-manager.service
```

```
[Service]
ExecStart=/usr/local/bin/kube-controller-manager \\
  --address=0.0.0.0 \\
  --cluster-cidr=10.200.0.0/16 \\
  --cluster-name=kubernetes \\
  --cluster-signing-cert-file=/var/lib/kubernetes/ca.pem \\
  --cluster-signing-key-file=/var/lib/kubernetes/ca-key.pem \\
  --kubeconfig=/var/lib/kubernetes/kube-controller-manager.kubeconfig \\
  --leader-elect=true \\
  --root-ca-file=/var/lib/kubernetes/ca.pem \\
  --service-account-private-key-file=/var/lib/kubernetes/service-account-key.pem \\
  --service-cluster-ip-range=10.32.0.0/24 \\
  --use-service-account-credentials=true \\
  --v=2
Restart=on-failure
RestartSec=5
```

KODEKLOUD

View controller-manager options

```
ps -aux | grep kube-controller-manager
```

```
root      1994  2.7  5.1 154360 105024 ?       Ssl  06:45   1:25 kube-controller-manager --address=127.0.0.1 --cluster-signing-cert-file=/etc/kubernetes/pki/ca.crt --cluster-signing-key-file=/etc/kubernetes/pki/ca.key --controllers=*,bootstrapsigner,tokencleaner --kubeconfig=/etc/kubernetes/controller-manager.conf --leader-elect=true --root-ca-file=/etc/kubernetes/pki/ca.crt --service-account-private-key-file=/etc/kubernetes/pki/sa.key --use-service-account-credentials=true
```

KODEKLOUD

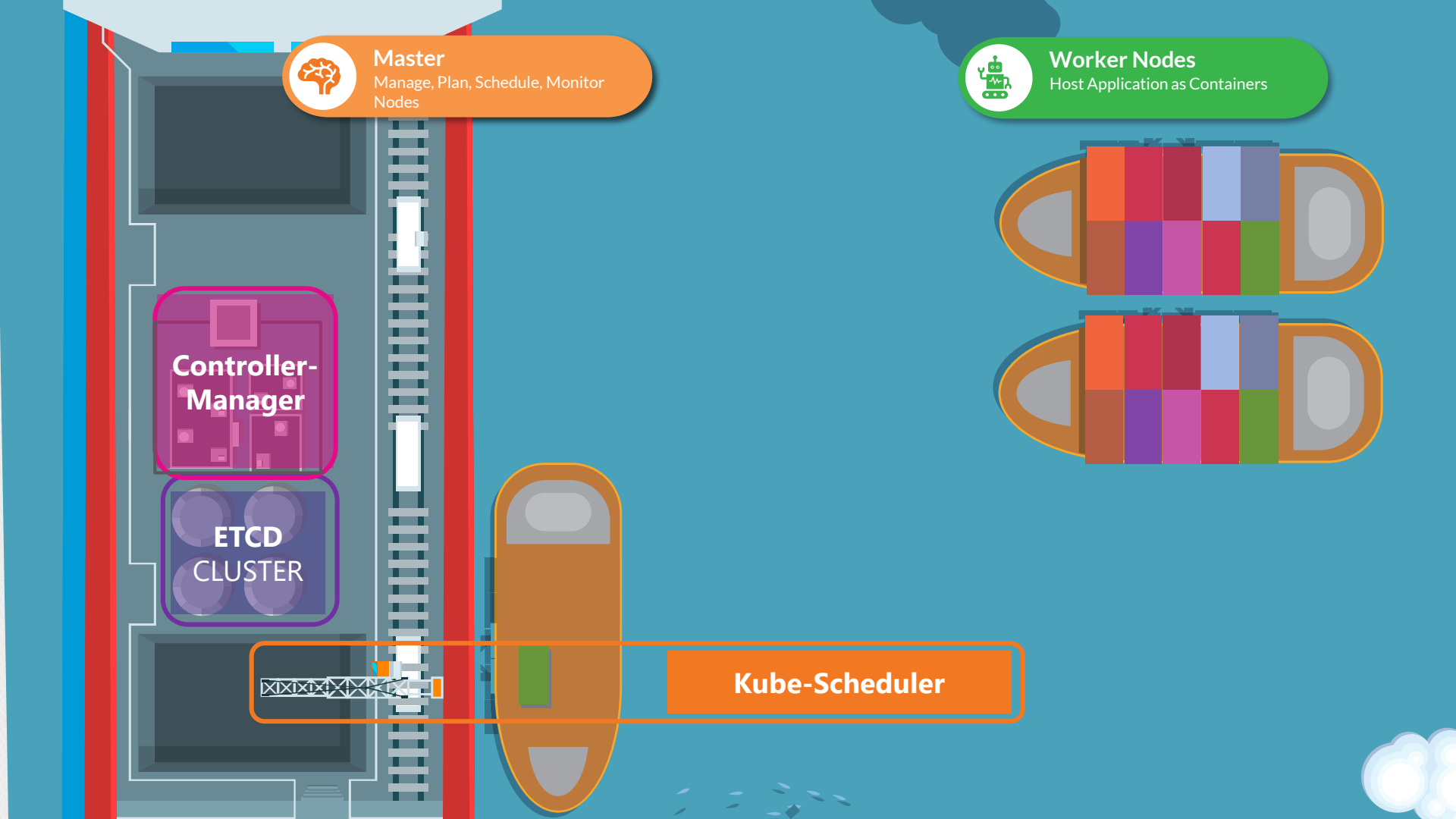
KODEKLOUD

Course Objectives

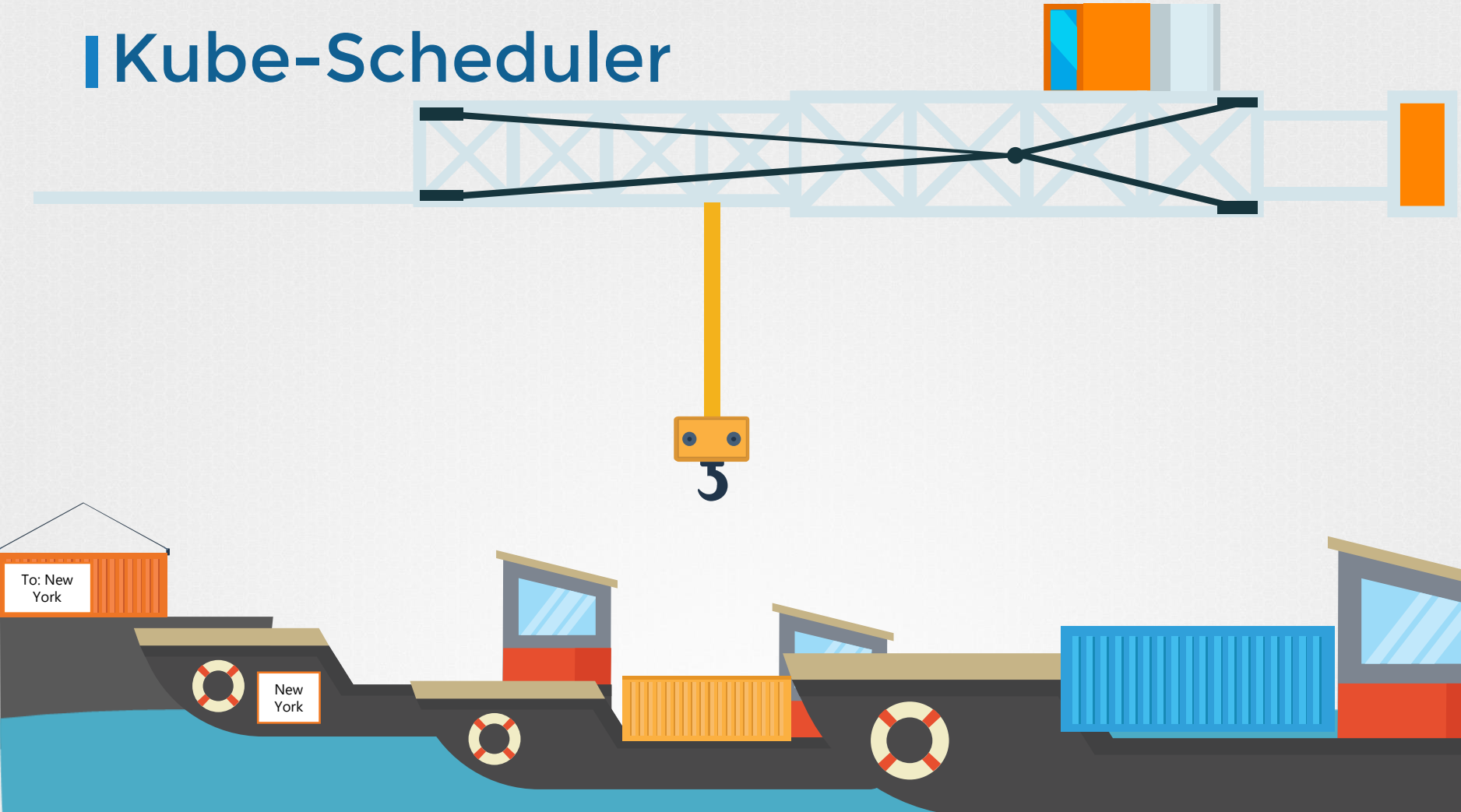
- Core Concepts
  - Cluster Architecture
  - API Primitives
  - Services & Other Network Primitives
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Networking
- Installation, Configuration & Validation
- Troubleshooting

KODEKLOUD

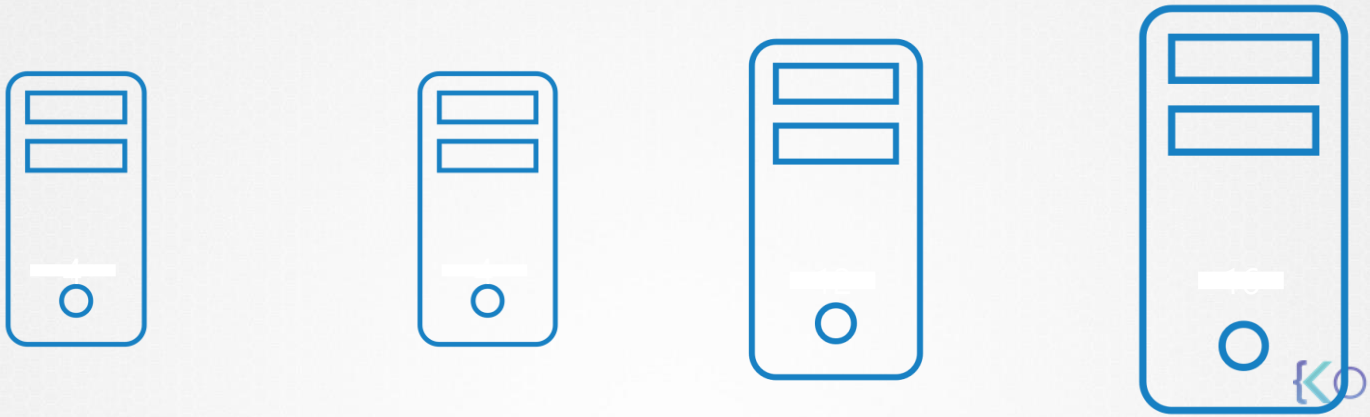
Kube Scheduler


Master
Manage, Plan, Schedule, Monitor Nodes
Worker Nodes
Host Application as Containers
Controller-Manager
ETCD CLUSTER
Kube-Scheduler

Kube-Scheduler

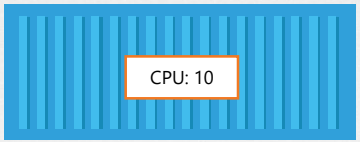


Kube-Scheduler



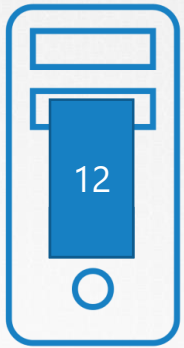
KODEKLOUD

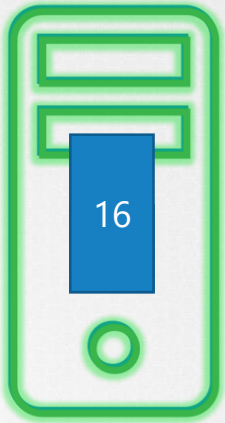
Kube-Scheduler



1. Filter Nodes

2. Rank Nodes





KODEKLOUD

More Later...

- Resource Requirements and Limits
- Taints and Tolerations
- Node Selectors/Affinity

KODEKLOUD

Course Objectives

- Scheduling
  - Labels & Selectors
  - Resource Limits
  - Manual Scheduling
  - Daemon Sets
  - Multiple Schedulers
  - Scheduler Events
  - Configure Kubernetes Scheduler
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Troubleshooting

KODEKLOUD

Installing kube-scheduler

```
wget https://storage.googleapis.com/kubernetes-release/release/v1.13.0/bin/linux/amd64/kube-scheduler
```

kube-scheduler.service

```
ExecStart=/usr/local/bin/kube-scheduler \\
  --config=/etc/kubernetes/config/kube-scheduler.yaml \\
  --v=2
```

KODEKLOUD

View kube-scheduler options - kubeadm

```
cat /etc/kubernetes/manifests/kube-scheduler.yaml
```

```
spec:
  containers:
  - command:
    - kube-scheduler
    - --address=127.0.0.1
    - --kubeconfig=/etc/kubernetes/scheduler.conf
    - --leader-elect=true
```

KODEKLOUD

View kube-scheduler options

```
ps -aux | grep kube-scheduler
```

```
root      2477  0.8  1.6  48524 34044 ?        Ssl  17:31   0:08 kube-scheduler --
address=127.0.0.1 --kubeconfig=/etc/kubernetes/scheduler.conf --leader-elect=true
```

KODEKLOUD

KODEKLOUD

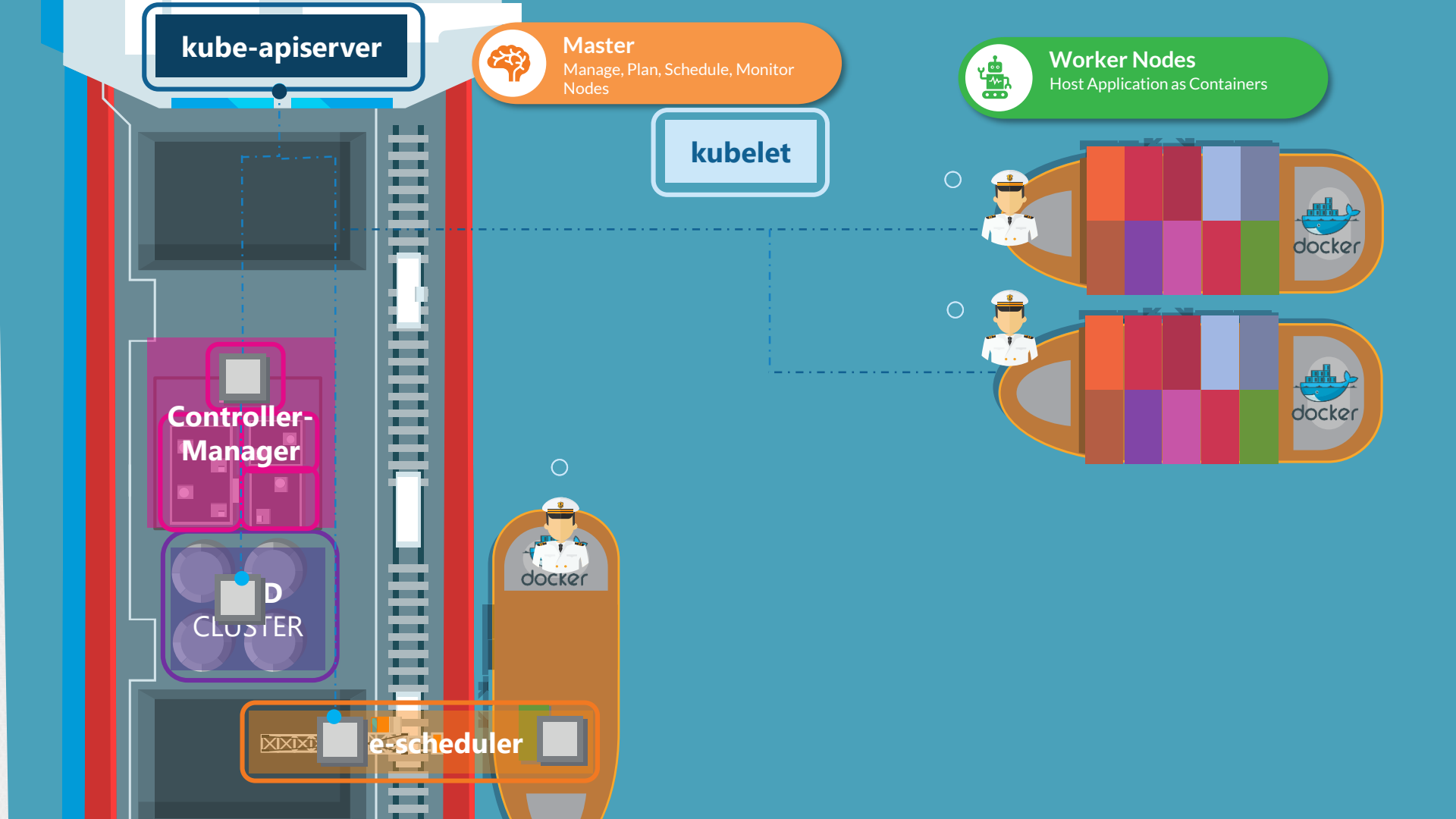
Course Objectives

- Core Concepts
  - Cluster Architecture
  - API Primitives
  - Services & Other Network Primitives
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Networking
- Installation, Configuration & Validation
- Troubleshooting

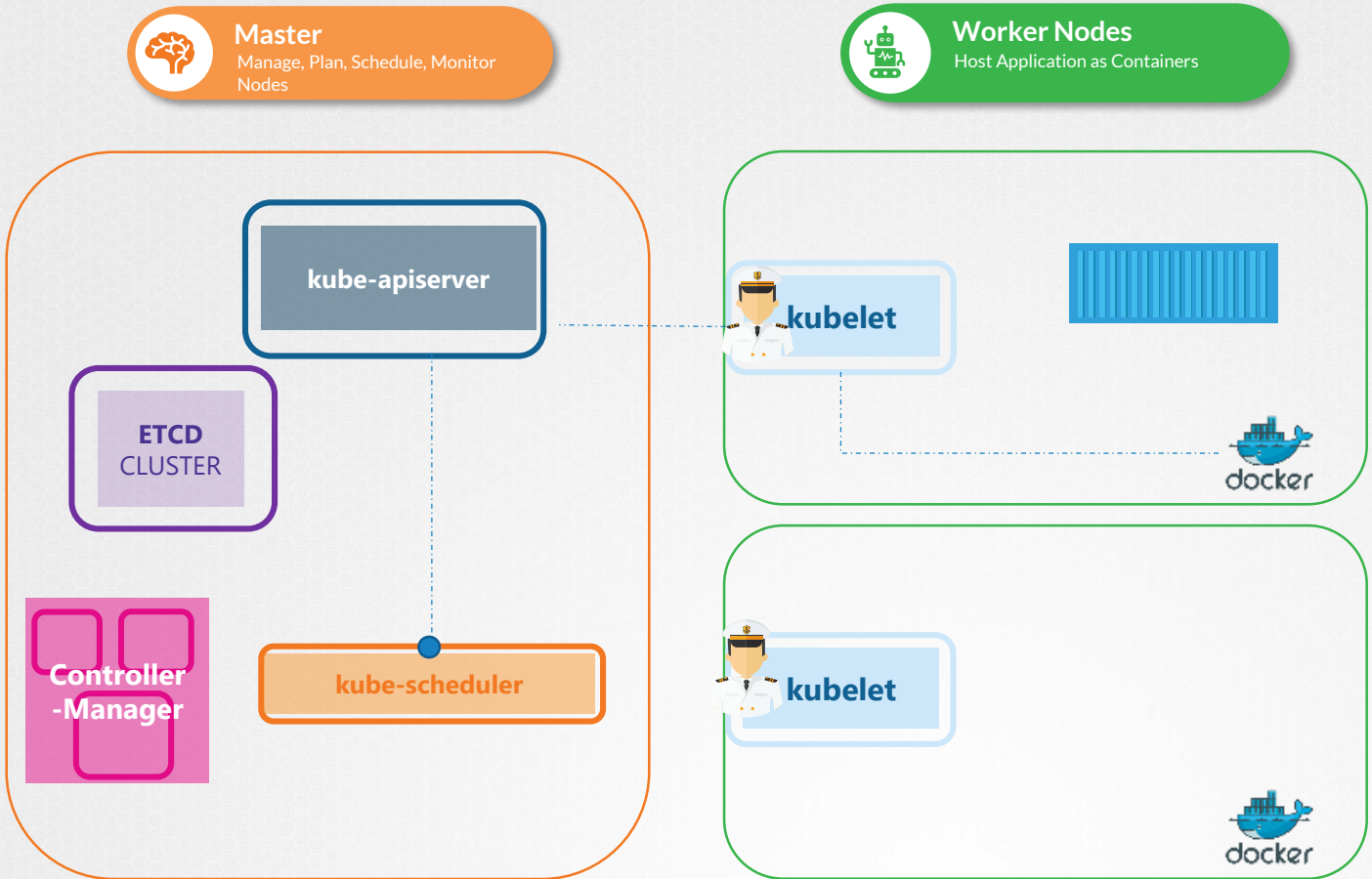
KODEKLOUD

Kubelet

KODEKLOUD


kube-apiserver
Master
Manage, Plan, Schedule, Monitor Nodes
kubelet
Worker Nodes
Host Application as Containers
docker
docker
Controller-Manager
D
CLUSTER
docker
e-scheduler

Kubernetes Architecture



Register Node

Create PODs

Monitor Node & PODs

KODEKLOUD

Installing kubelet

```
wget https://storage.googleapis.com/kubernetes-release/release/v1.13.0/bin/linux/amd64/kubelet
```

kubelet.service

```
ExecStart=/usr/local/bin/kubelet \\
  --config=/var/lib/kubelet/kubelet-config.yaml \\
  --container-runtime=remote \\
  --container-runtime-endpoint=unix:///var/run/containerd/containerd.sock \\
  --image-pull-progress-deadline=2m \\
  --kubeconfig=/var/lib/kubelet/kubeconfig \\
  --network-plugin=cni \\
  --register-node=true \\
  --v=2
```

Kubeadm does not deploy Kubelets

KODEKLOUD

View kubelet options

```
ps -aux | grep kubelet
```

```
root      2095  1.8  2.4 960676 98788 ?        Ssl  02:32   0:36 /usr/bin/kubelet --bootstrap-kubeconfig=/etc/kubernetes/bootstrap-kubelet.conf --kubeconfig=/etc/kubernetes/kubelet.conf --config=/var/lib/kubelet/config.yaml --cgroup-driver=cgroupfs --cni-bin-dir=/opt/cni/bin --cni-conf-dir=/etc/cni/net.d --network-plugin=cni
```

KODEKLOUD

KODEKLOUD

Course Objectives

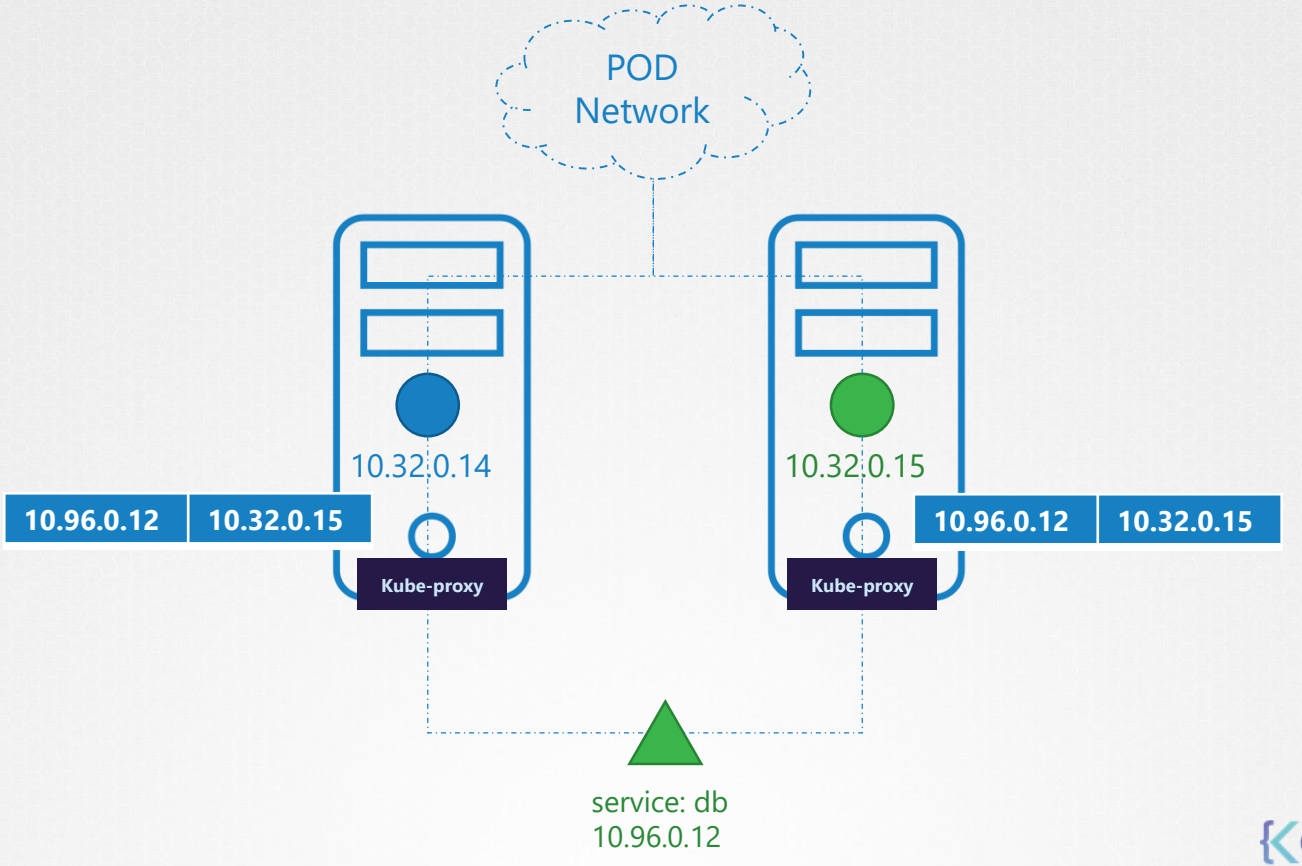
- Core Concepts
  - Cluster Architecture
  - API Primitives
  - Services & Other Network Primitives
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Networking
- Installation, Configuration & Validation
- Troubleshooting

KODEKLOUD

Kube-proxy

KODEKLOUD

Kube-proxy



KODEKLOUD

Installing kube-proxy

```
wget https://storage.googleapis.com/kubernetes-release/release/v1.13.0/bin/linux/amd64/kube-proxy
```

kube-proxy.service

```
ExecStart=/usr/local/bin/kube-proxy \\
  --config=/var/lib/kube-proxy/kube-proxy-config.yaml
Restart=on-failure
RestartSec=5
```

KODEKLOUD

View kube-proxy - kubeadm

```
kubectl get pods -n kube-system
```

```
NAMESPACE     NAME                              READY     STATUS    RESTARTS   AGE
kube-system   coredns-78fcdf6894-hwrq9          1/1       Running   0          16m
kube-system   coredns-78fcdf6894-rzhjr          1/1       Running   0          16m
kube-system   etcd-master                       1/1       Running   0          15m
kube-system   kube-apiserver-master             1/1       Running   0          15m
kube-system   kube-controller-manager-master    1/1       Running   0          15m
kube-system   kube-proxy-lzt6f                  1/1       Running   0          16m
kube-system   kube-proxy-zm5qd                  1/1       Running   0          16m
kube-system   kube-scheduler-master             1/1       Running   0          15m
kube-system   weave-net-29z42                   2/2       Running   1          16m
kube-system   weave-net-snmdl                   2/2       Running   1          16m    -
```

```
kubectl get daemonset -n kube-system
```

```
NAME         DESIRED   CURRENT   READY     UP-TO-DATE   AVAILABLE   NODE SELECTOR                   AGE
kube-proxy   2         2         2         2            2           beta.kubernetes.io/arch=amd64   1h
```

KODEKLOUD

KODEKLOUD

KODEKLOUD

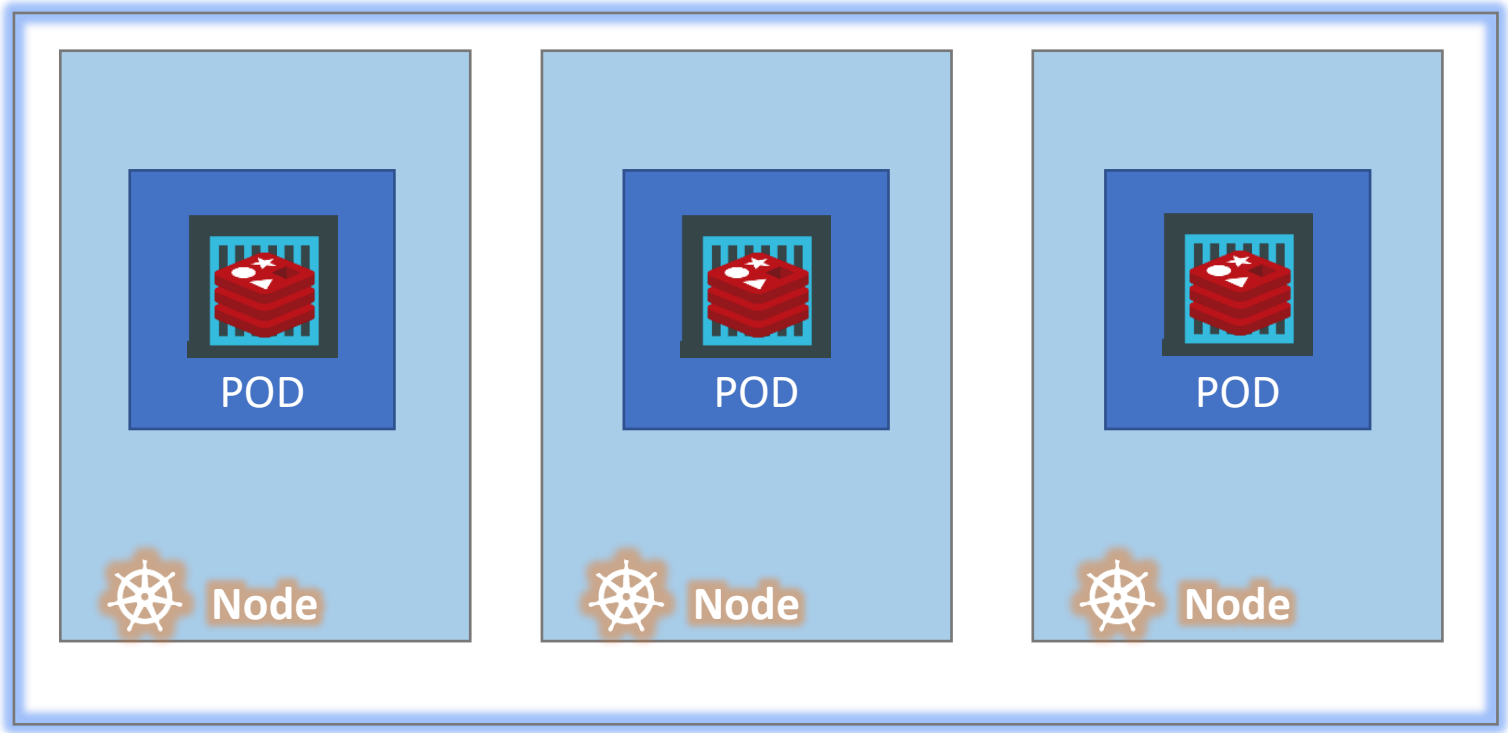
POD

Assumptions

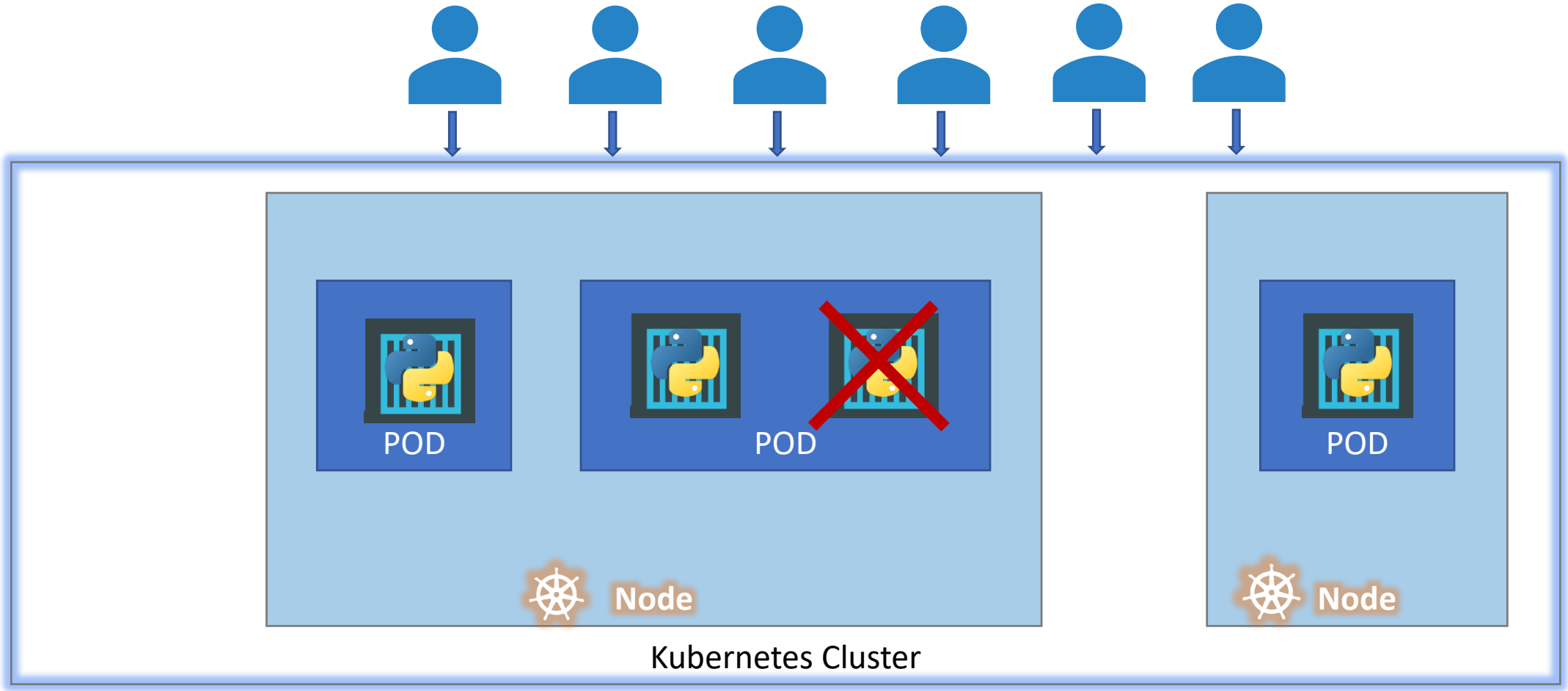
Docker Image

Kubernetes Cluster

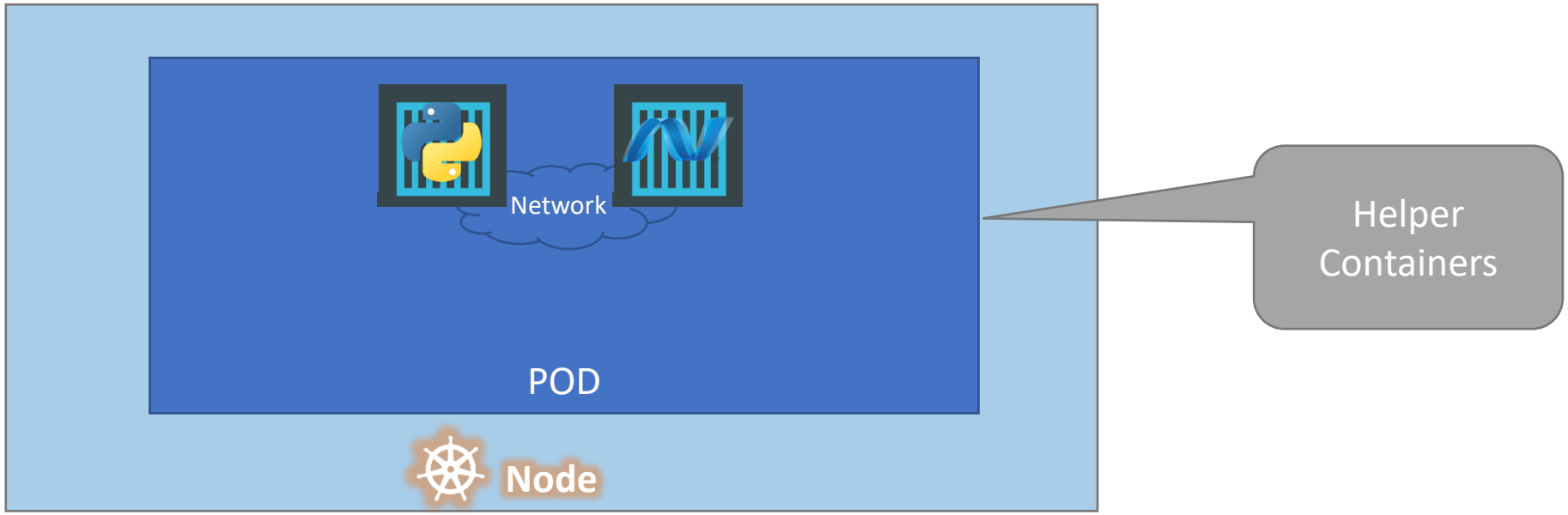
POD



POD



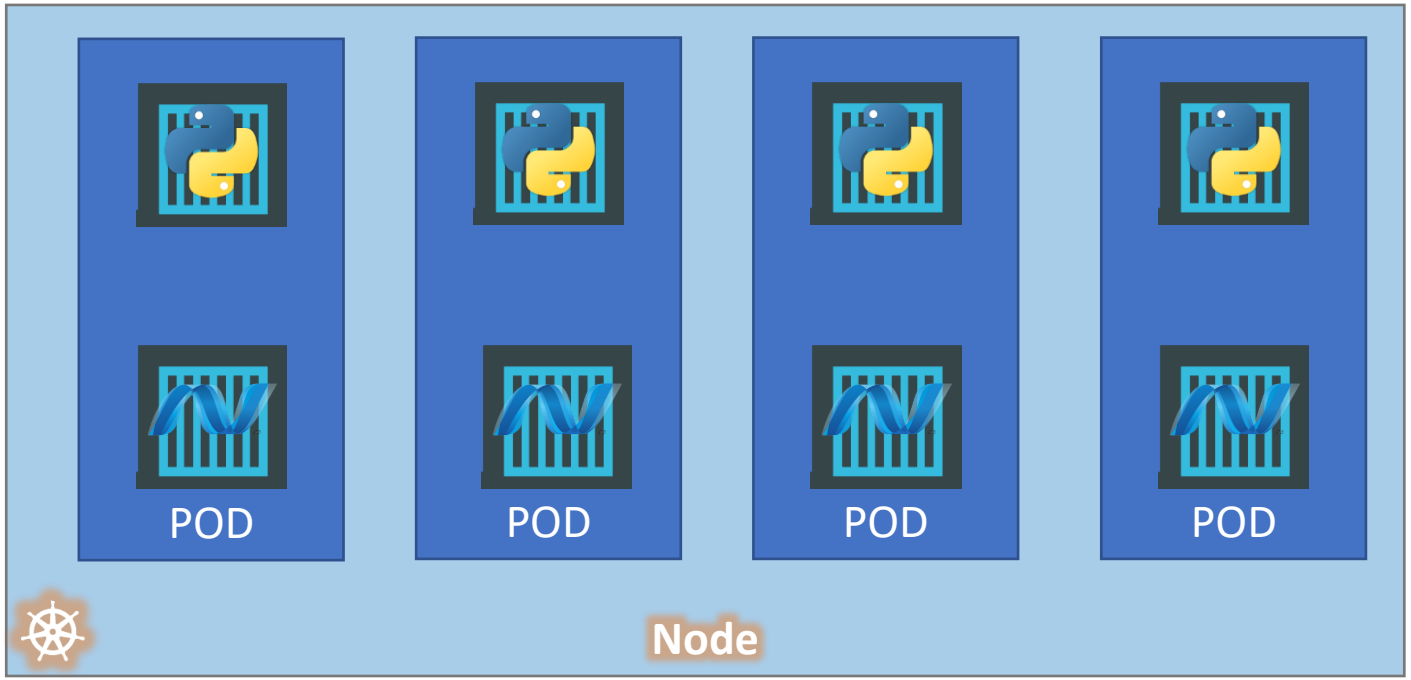
Multi-Container PODs



PODs Again!

```
docker run python-app
docker run python-app
docker run python-app
docker run python-app

docker run helper –link app1
docker run helper –link app2
docker run helper –link app3

docker run helper –link app4
```

App	Helper	Volume
Python1	App1	Vol1
Python2	App2	Vol2



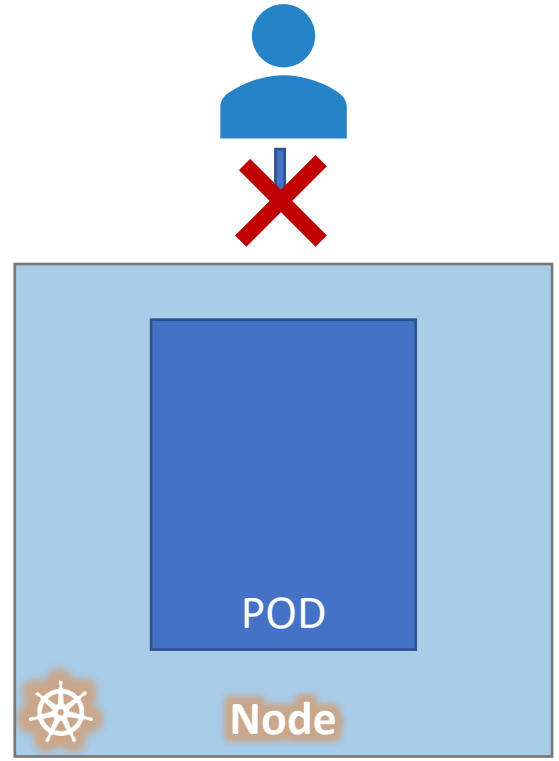
Note: I am avoiding networking and load balancing details to keep explanation simple.

kubectl

- `kubectl run nginx --image nginx`

```
kubectl get pods
NAME    READY   STATUS              RESTARTS   AGE
nginx   0/1     ContainerCreating   0          6s
```

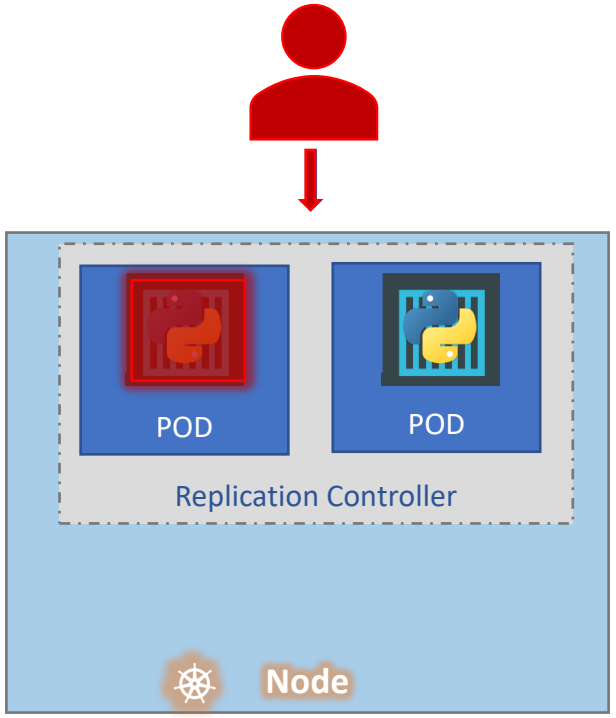
```
NAME    READY   STATUS    RESTARTS   AGE
nginx   1/1     Running   0          34s
```

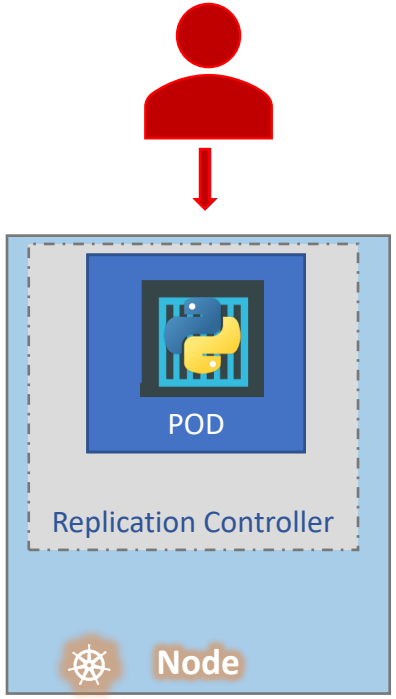


KODEKLOUD

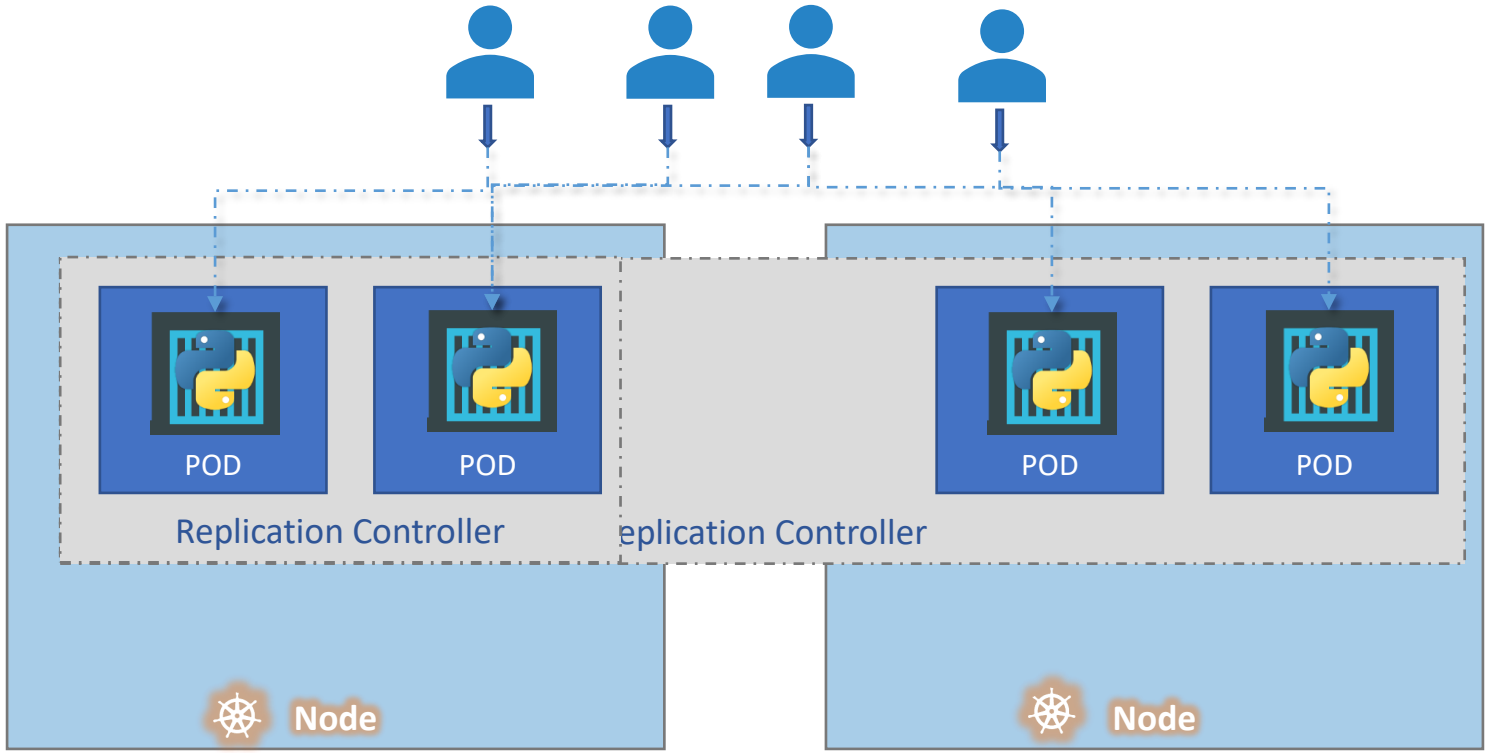
Replication Controller

High Availability





Load Balancing & Scaling



- Replication Controller

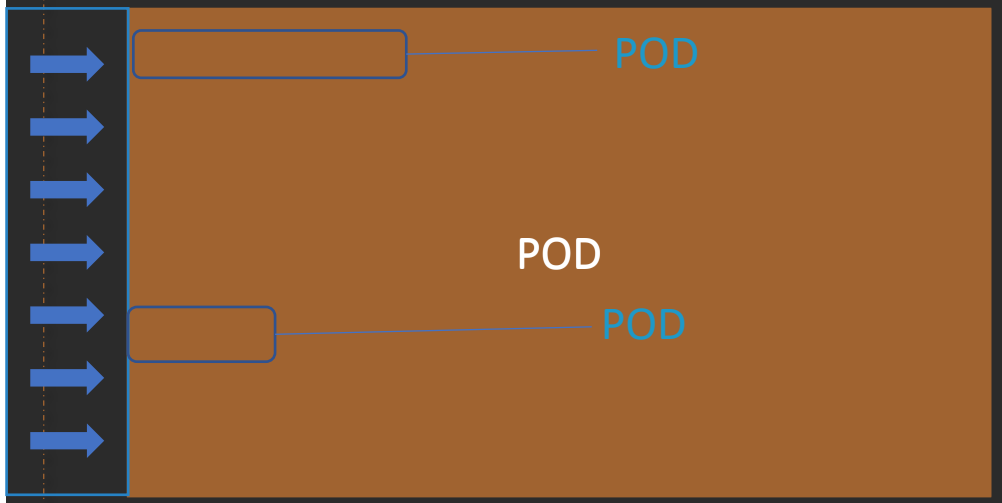
Replica Set

rc-definition.yml

```yaml
apiVersion: v1
kind: ReplicationController
metadata:
  name: myapp-rc
  labels:
      app: myapp
      type: front-end
spec:
  template:
    replicas: 3
```

Replication Controller

Replication Controller



pod-definition.yml

```yaml
apiVersion: v1
kind: Pod

metadata:
 name: myapp-pod
 labels:
    app: myapp
    type: front-end
spec:
  containers:
  - name: nginx-container
    image: nginx
```

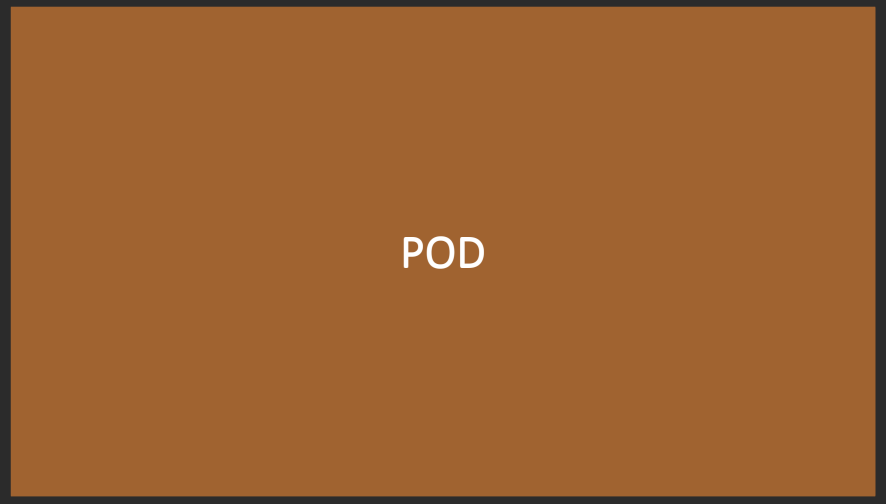
KODEKLOUD

```
> kubectl create -f rc-definition.yml
replicationcontroller "myapp-rc" created
```

```
> kubectl get replicationcontroller
NAME       DESIRED   CURRENT   READY     AGE
myapp-rc   3         3         3         19s
```

```
> kubectl get pods
NAME             READY     STATUS    RESTARTS   AGE
myapp-rc-4lvk9   1/1       Running   0          20s
myapp-rc-mc2mf   1/1       Running   0          20s
myapp-rc-px9pz   1/1       Running   0          20s
```

replicaset-definition.yml

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: myapp-repl
  labels:
      app: myapp
      type: front-end
spec:
  template:
      
  replicas: 3
  selector:
     matchLabels:
        type: front-end
```



```
error: unable to recognize "replicaset-
definition.yml": no matches for /, Kind=ReplicaSet
```

pod-definition.yml

```yaml
apiVersion: v1
kind: Pod
 labels:
    app: myapp
    type: front-end
spec:
  containers:
  - name: nginx-container
    image: nginx
```

- > kubectl create –f replicaset-definition.yml

```
replicaset "myapp-replicaset" created
```

> kubectl get replicaset

```
NAME            DESIRED   CURRENT   READY     AGE
myapp-replicaset   3         3         3         19s
```

> kubectl get pods

```
NAME                    READY     STATUS    RESTARTS   AGE
myapp-replicaset-9ddl9   1/1       Running   0          45s
myapp-replicaset-9jtpx   1/1       Running   0          45s
myapp-replicaset-hq84m   1/1       Running   0          45s
```

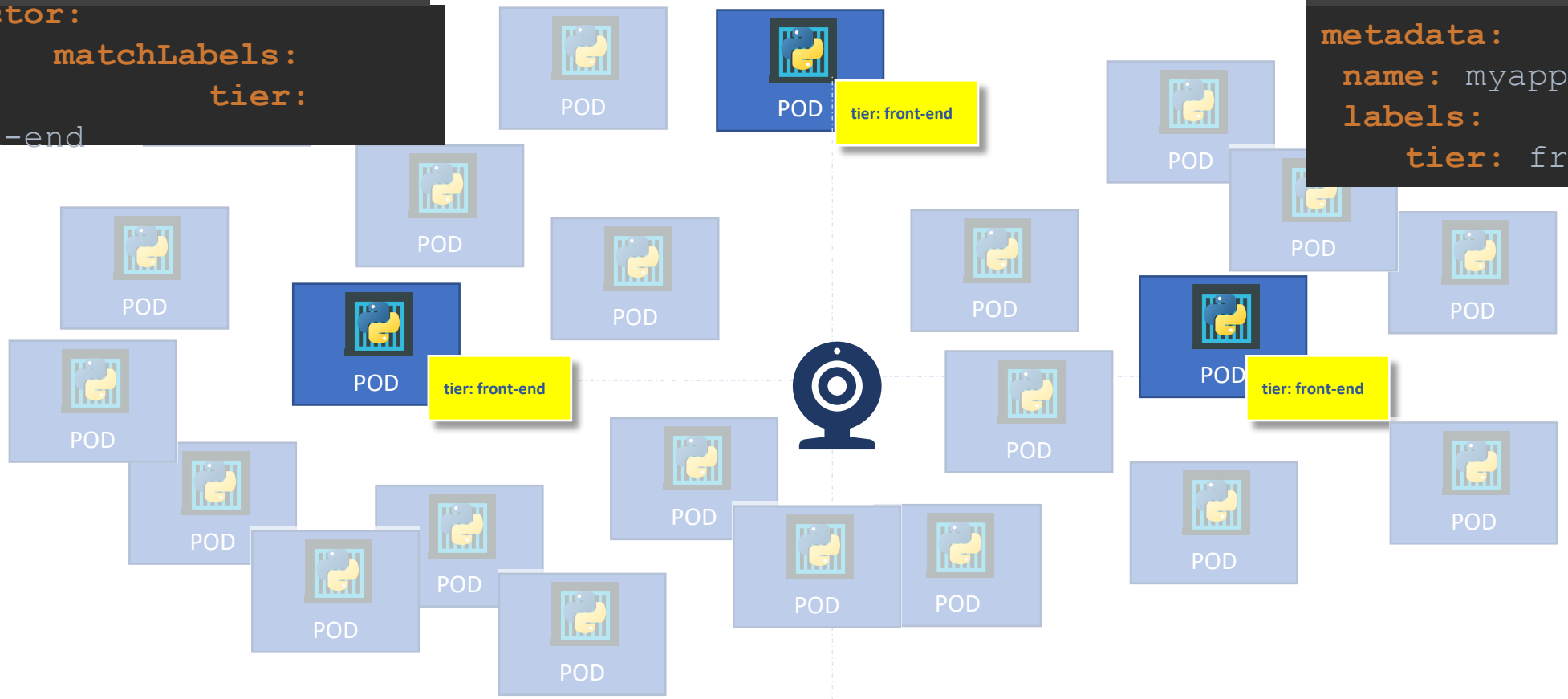
Labels and Selectors

replicaset-definition.yml

```
selector:
        matchLabels:
                tier:
front-end
```

pod-definition.yml

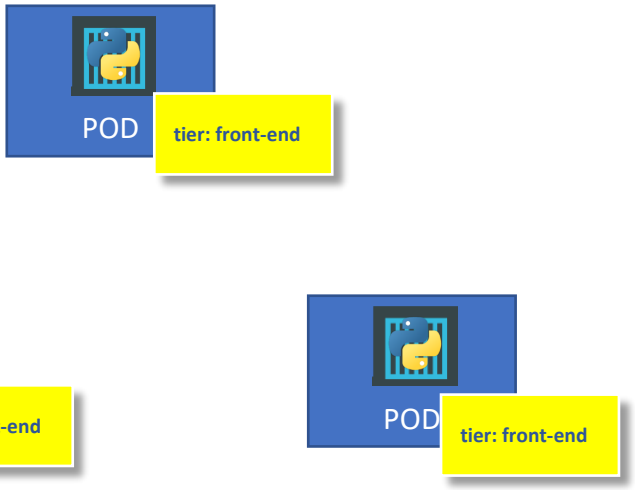
```
metadata:
  name: myapp-pod
  labels:
     tier: front-end
```


POD
POD
tier: front-end
POD
POD
POD

replicaset-definition.yml

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: myapp-replicaset
  labels:
      app: myapp
      type: front-end
spec:
  template:
    metadata:
      name: myapp-pod
      labels:
        app: myapp
        type: front-end
    spec:
      containers:
      - name: nginx-container
        image: nginx
  replicas: 3
  selector:
    matchLabels:
      type: front-end
```

Template



Scale

```
> kubectl replace -f replicaset-definition.yml
```

```
> kubectl scale --replicas=6 -f replicaset-definition.yml
```

```
> kubectl scale --replicas=6 replicaset myapp-replicaset
```

TYPE NAME

replicaset-definition.yml

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: myapp-replicaset
  labels:
      app: myapp
      type: front-end
spec:
  template:
    metadata:
      name: myapp-pod
      labels:
        app: myapp
        type: front-end
    spec:
      containers:
      - name: nginx-container
        image: nginx
  replicas: 6
  selector:
    matchLabels:
      type: front-end
```

commands

```
> kubectl create –f replicaset-definition.yml
```

```
> kubectl get replicaset
```

```
> kubectl delete replicaset myapp-replicaset
```

*Also deletes all underlying PODs

```
> kubectl replace -f replicaset-definition.yml
```

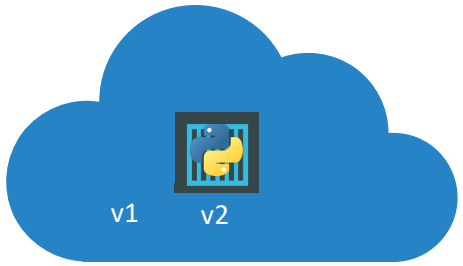
```
> kubectl scale –replicas=6 -f replicaset-definition.yml
```

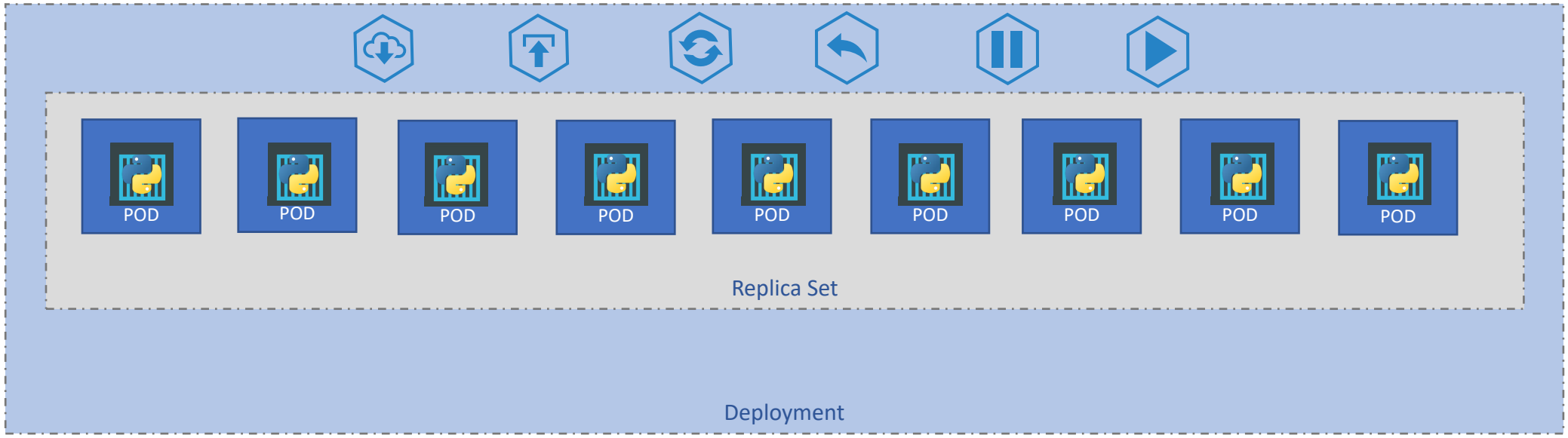
KODEKLOUD

KODEKLOUD

Deployment

Deployment


v1
v2


POD
POD
POD
POD
POD
POD
POD
POD
POD
Replica Set
Deployment

Definition

```
> kubectl create -f deployment-definition.yml
deployment "myapp-deployment" created
```

```
> kubectl get deployments
NAME              DESIRED   CURRENT   UP-TO-DATE   AVAILABLE   AGE
myapp-deployment  3         3         3            3           21s
```

```
> kubectl get replicaset
NAME                        DESIRED   CURRENT   READY     AGE
myapp-deployment-6795844b58 3         3         3         2m
```

```
> kubectl get pods
NAME                              READY     STATUS    RESTARTS   AGE
myapp-deployment-6795844b58-5rbjl 1/1       Running   0          2m
myapp-deployment-6795844b58-h4w55 1/1       Running   0          2m
myapp-deployment-6795844b58-lfjhv 1/1       Running   0          2m
```

deployment-definition.yml

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: myapp-deployment
  labels:
      app: myapp
      type: front-end
spec:
  template:
    metadata:
      name: myapp-pod
      labels:
         app: myapp
         type: front-end
    spec:
      containers:
      - name: nginx-container
        image: nginx

  replicas: 3
  selector:
    matchLabels:
       type: front-end
```

commands

```
> kubectl get all
NAME                     DESIRED   CURRENT   UP-TO-DATE   AVAILABLE   AGE
deploy/myapp-deployment   3         3         3            3           9h

NAME                               DESIRED   CURRENT   READY     AGE
rs/myapp-deployment-6795844b58   3         3         3         9h

NAME                                    READY     STATUS    RESTARTS   AGE
po/myapp-deployment-6795844b58-5rbjl   1/1       Running   0          9h
po/myapp-deployment-6795844b58-h4w55   1/1       Running   0          9h
po/myapp-deployment-6795844b58-lfjhv   1/1       Running   0          9h
```

KODEKLOUD

KODEKLOUD

Deployment

Updates and Rollback

Rollout and Versioning

Revision 1

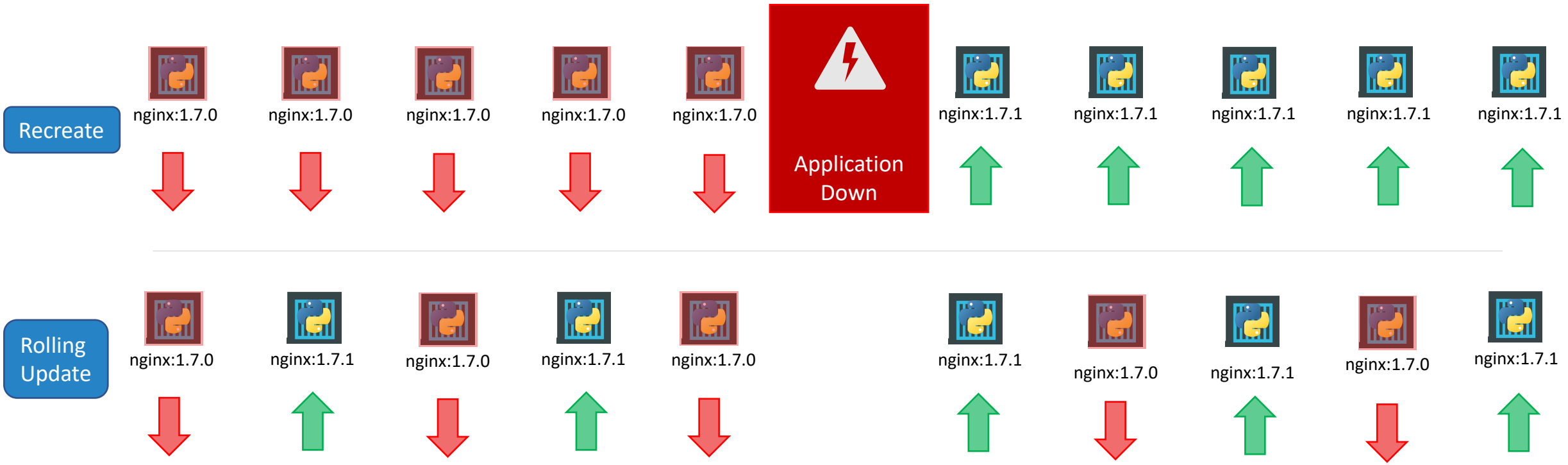
Revision 2

Rollout Command

```
> kubectl rollout status deployment/myapp-deployment
Waiting for rollout to finish: 0 of 10 updated replicas are available...
Waiting for rollout to finish: 1 of 10 updated replicas are available...
Waiting for rollout to finish: 2 of 10 updated replicas are available...
Waiting for rollout to finish: 3 of 10 updated replicas are available...
Waiting for rollout to finish: 4 of 10 updated replicas are available...
Waiting for rollout to finish: 5 of 10 updated replicas are available...
Waiting for rollout to finish: 6 of 10 updated replicas are available...
Waiting for rollout to finish: 7 of 10 updated replicas are available...
Waiting for rollout to finish: 8 of 10 updated replicas are available...
Waiting for rollout to finish: 9 of 10 updated replicas are available...
deployment "myapp-deployment" successfully rolled out
```

```
> kubectl rollout history deployment/myapp-deployment
deployments "myapp-deployment"
REVISION  CHANGE-CAUSE
1         <none>
2         kubectl apply --filename=deployment-definition.yml --record=true
```

Deployment Strategy



Kubectl apply

```
> kubectl apply -f deployment-definition.yml
```

```
deployment "myapp-deployment" configured
```

```
> kubectl set image deployment/myapp-deployment \
                  nginx=nginx:1.9.1
```

```
deployment "myapp-deployment" image is updated
```

deployment-definition.yml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-deployment
  labels:
      app: myapp
      type: front-end
spec:
  template:
    metadata:
      name: myapp-pod
      labels:
         app: myapp
         type: front-end
    spec:
      containers:
      - name: nginx-container
        image: nginx:1.7.1
  replicas: 3
  selector:
    matchLabels:
       type: front-end
```

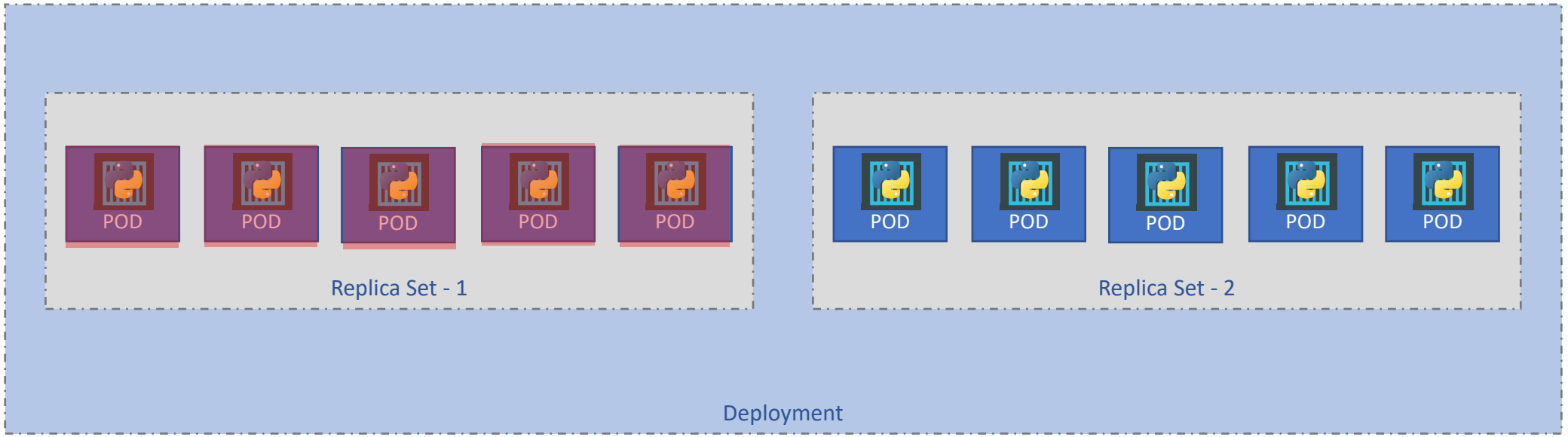
```
C:\Kubernetes>kubectl describe deployment myapp-deployment
Name:                   myapp-deployment
Namespace:              default
CreationTimestamp:      Sat, 03 Mar 2018 17:01:55 +0800
Labels:                 app=myapp
                        type=front-end
Annotations:            deployment.kubernetes.io/revision=2
                        kubectl.kubernetes.io/last-applied-configuration={"apiVersion":"apps/v1","kind":"Deployment","me
s\\Google...
                        kubernetes.io/change-cause=kubectl apply --filename=d:\Mumshad Files\Google Drive\Udemy\Kubernet
Selector:               type=front-end
Replicas:               5 desired | 5 updated | 5 total | 5 available | 0 unavailable
StrategyType:           Recreate
MinReadySeconds:        0
Pod Template:
  Labels:  app=myapp
           type=front-end
  Containers:
   nginx-container:
    Image:        nginx:1.7.1
    Port:         <none>
    Environment:  <none>
    Mounts:       <none>
  Volumes:        <none>
Conditions:
  Type           Status  Reason
  ----           ------  ------
  Available      True    MinimumReplicasAvailable
  Progressing    True    NewReplicaSetAvailable
OldReplicaSets:  <none>
NewReplicaSet:   myapp-deployment-54c7d6ccc (5/5 replicas created)
Events:
  Type    Reason             Age   From                   Message
  ----    ------             ----  ----                   -------
  Normal  ScalingReplicaSet  11m   deployment-controller  Scaled up replica set myapp-deployment-6795844b58 to 5
  Normal  ScalingReplicaSet  1m    deployment-controller  Scaled down replica set myapp-deployment-6795844b58 to 0
  Normal  ScalingReplicaSet  56s   deployment-controller  Scaled up replica set myapp-deployment-54c7d6ccc to 5
```

Recreate

```
C:\Kubernetes>kubectl describe deployment myapp-deployment
Name:                   myapp-deployment
Namespace:              default
CreationTimestamp:      Sat, 03 Mar 2018 17:16:53 +0800
Labels:                 app=myapp
                        type=front-end
Annotations:            deployment.kubernetes.io/revision=2
                        kubectl.kubernetes.io/last-applied-configuration={"apiVersion":"apps/v1","kind":"Deployment","metadata
Files\\Google...
                        kubernetes.io/change-cause=kubectl apply --filename=d:\Mumshad Files\Google Drive\Udemy\Kubernetes\De
Selector:               type=front-end
Replicas:               5 desired | 5 updated | 6 total | 4 available | 2 unavailable
StrategyType:           RollingUpdate
MinReadySeconds:        0
RollingUpdateStrategy:  25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=myapp
           type=front-end
  Containers:
   nginx-container:
    Image:        nginx
    Port:         <none>
    Environment:  <none>
    Mounts:       <none>
  Volumes:        <none>
Conditions:
  Type           Status  Reason
  ----           ------  ------
  Available      True    MinimumReplicasAvailable
  Progressing    True    ReplicaSetUpdated
OldReplicaSets:  myapp-deployment-67c749c58c (1/1 replicas created)
NewReplicaSet:   myapp-deployment-7d57dbdb8d (5/5 replicas created)
Events:
  Type    Reason             Age   From                   Message
  ----    ------             ----  ----                   -------
  Normal  ScalingReplicaSet  1m    deployment-controller  Scaled up replica set myapp-deployment-67c749c58c to 5
  Normal  ScalingReplicaSet  1s    deployment-controller  Scaled up replica set myapp-deployment-7d57dbdb8d to 2
  Normal  ScalingReplicaSet  1s    deployment-controller  Scaled down replica set myapp-deployment-67c749c58c to 4
  Normal  ScalingReplicaSet  1s    deployment-controller  Scaled up replica set myapp-deployment-7d57dbdb8d to 3
  Normal  ScalingReplicaSet  0s    deployment-controller  Scaled down replica set myapp-deployment-67c749c58c to 3
  Normal  ScalingReplicaSet  0s    deployment-controller  Scaled up replica set myapp-deployment-7d57dbdb8d to 4
  Normal  ScalingReplicaSet  0s    deployment-controller  Scaled down replica set myapp-deployment-67c749c58c to 2
  Normal  ScalingReplicaSet  0s    deployment-controller  Scaled up replica set myapp-deployment-7d57dbdb8d to 5
  Normal  ScalingReplicaSet  0s    deployment-controller  Scaled down replica set myapp-deployment-67c749c58c to 1
```

RollingUpdate

Upgrades

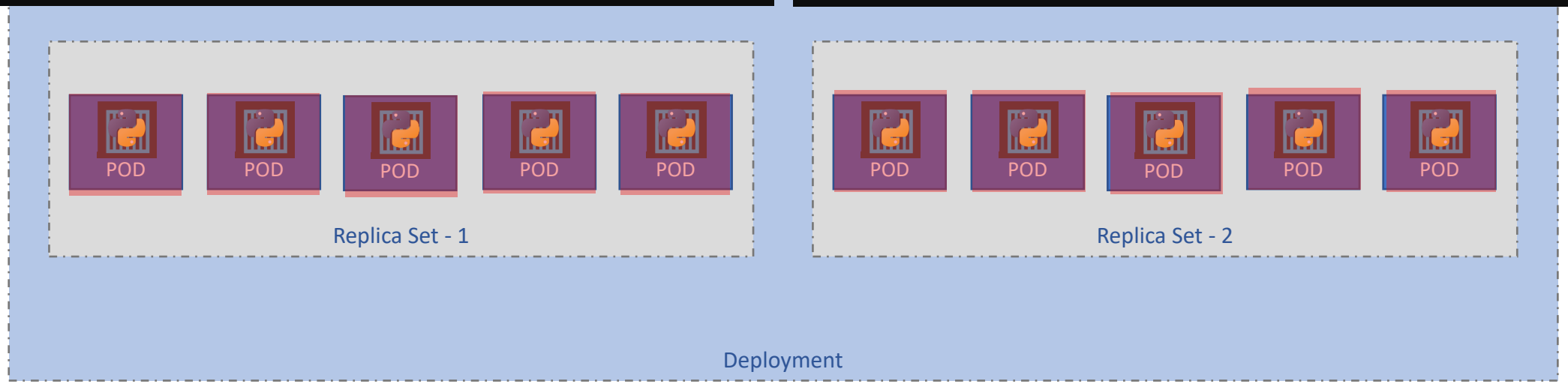



```
> kubectl get replicasets
NAME                          DESIRED   CURRENT   READY     AGE
myapp-deployment-67c749c58c   0         0         0         22m
myapp-deployment-7d57dbdb8d   5         5         5         20m
```

Rollback

```
> kubectl get replicasets
NAME                          DESIRED   CURRENT   READY   AGE
myapp-deployment-67c749c58c   0         0         0       22m
myapp-deployment-7d57dbdb8d   5         5         5       20m
```

```
> kubectl get replicasets
NAME                          DESIRED   CURRENT   READY   AGE
myapp-deployment-67c749c58c   5         5         5       22m
myapp-deployment-7d57dbdb8d   0         0         0       20m
```




```
> kubectl rollout undo deployment/myapp-deployment
deployment “myapp-deployment” rolled back
```

kubectl run

```
> kubectl run nginx --image=nginx
deployment "nginx" created
```

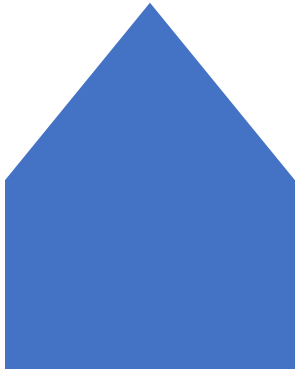
Summarize Commands

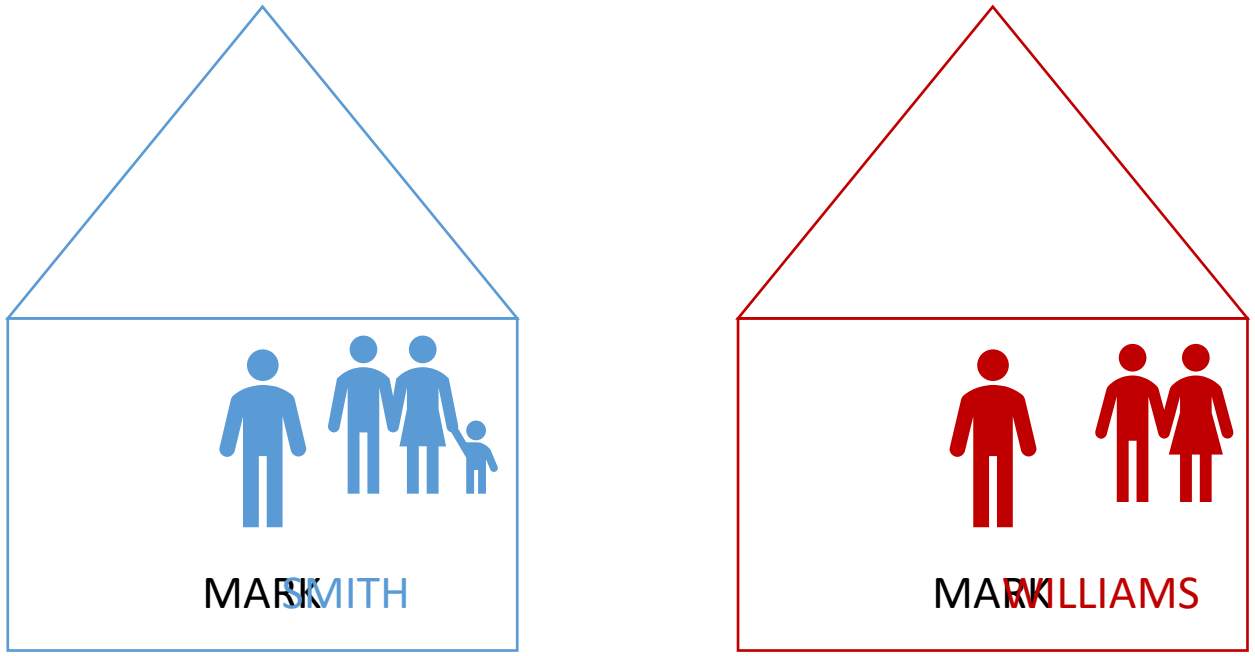
Create	`> kubectl create –f deployment-definition.yml`
Get	`> kubectl get deployments`
Update	`> kubectl apply –f deployment-definition.yml`
	`> kubectl set image deployment/myapp-deployment nginx=nginx:1.9.1`
Status	`> kubectl rollout status deployment/myapp-deployment`
	`> kubectl rollout history deployment/myapp-deployment`
Rollback	`> kubectl rollout undo deployment/myapp-deployment`

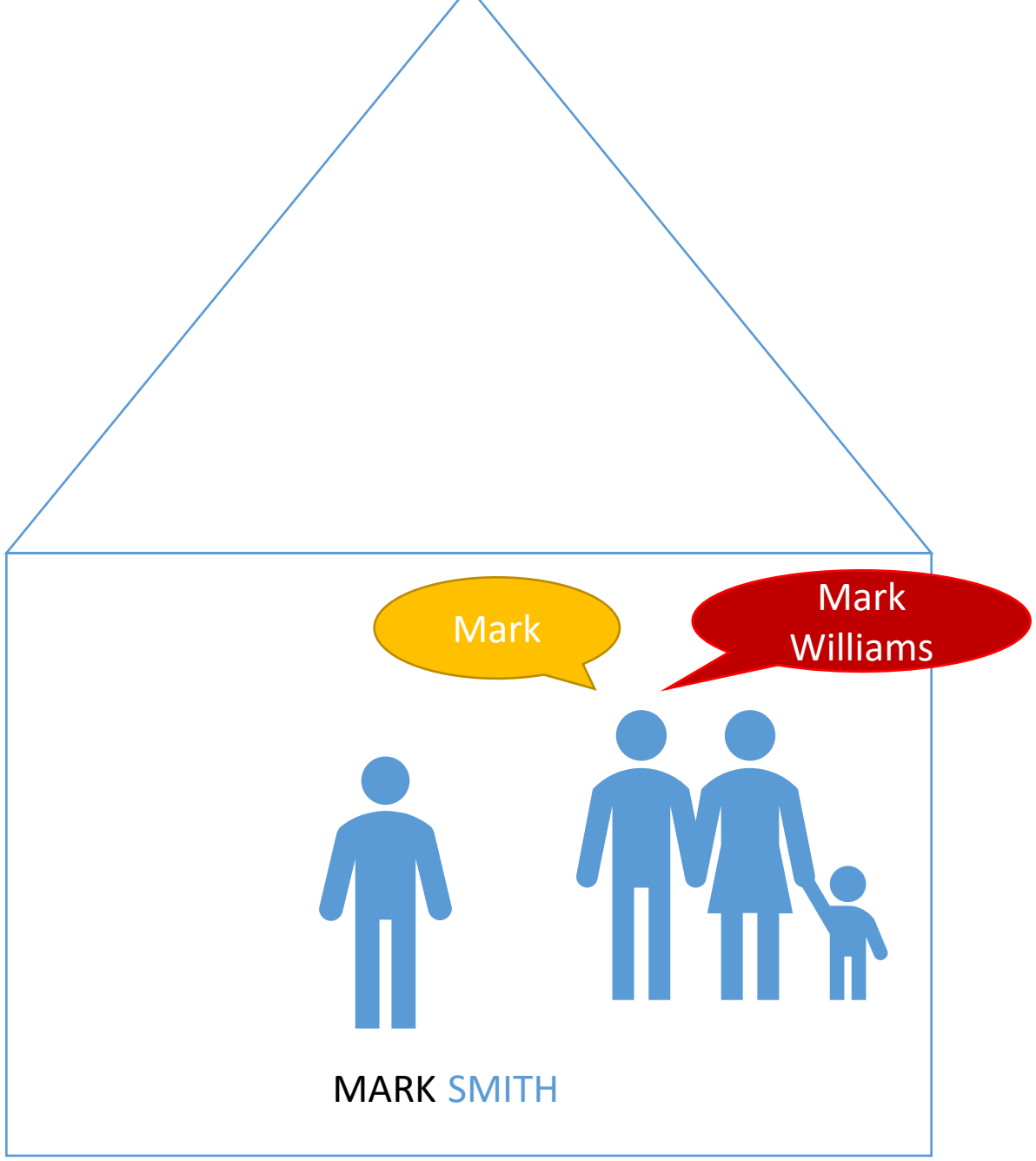
KODEKLOUD

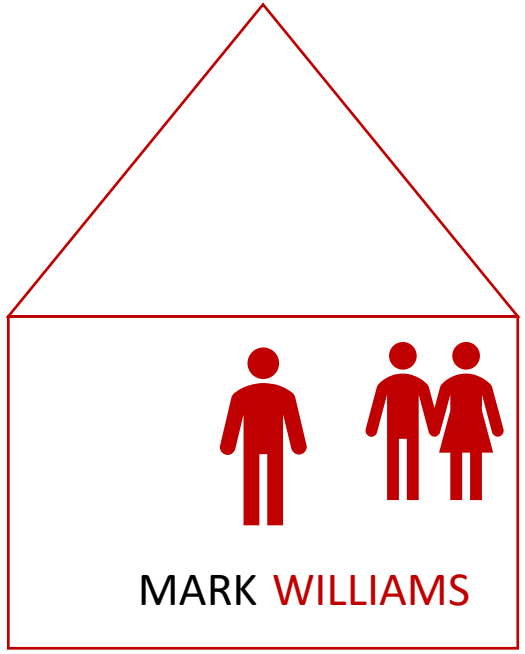
KODEKLOUD

Namespaces

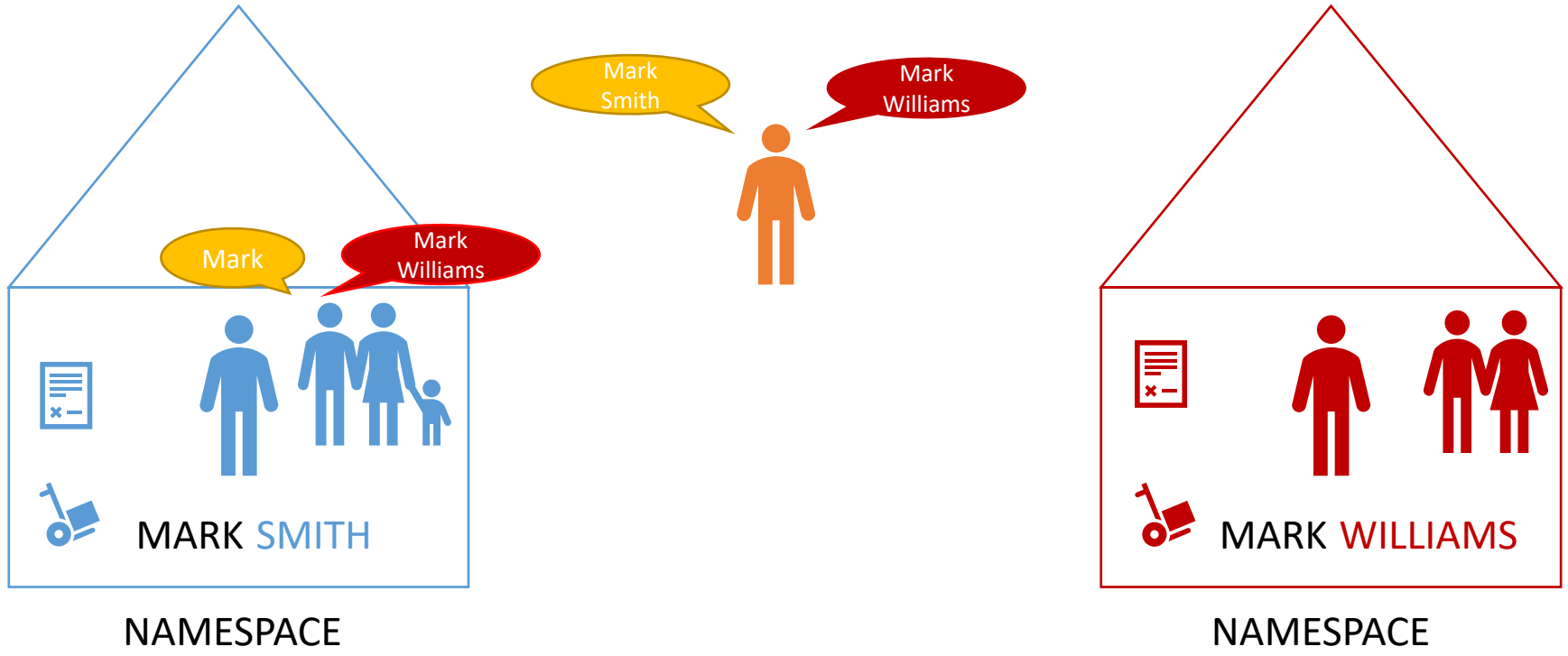



MARK SMITH
MARK WILLIAMS

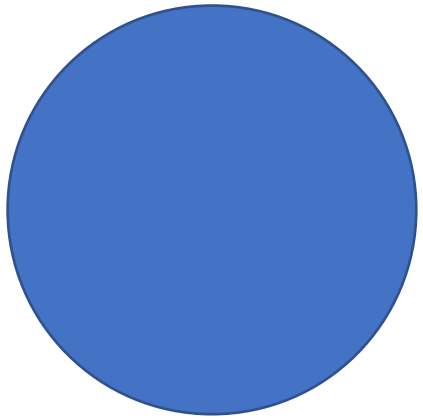

Mark
Mark
Williams
MARK SMITH

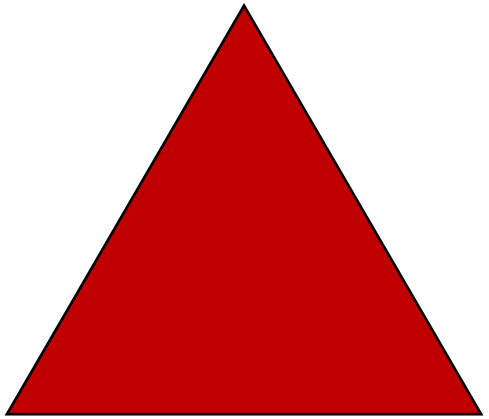

MARK WILLIAMS

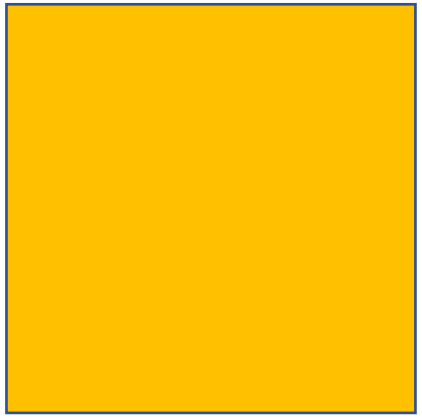
KODEKLOUD

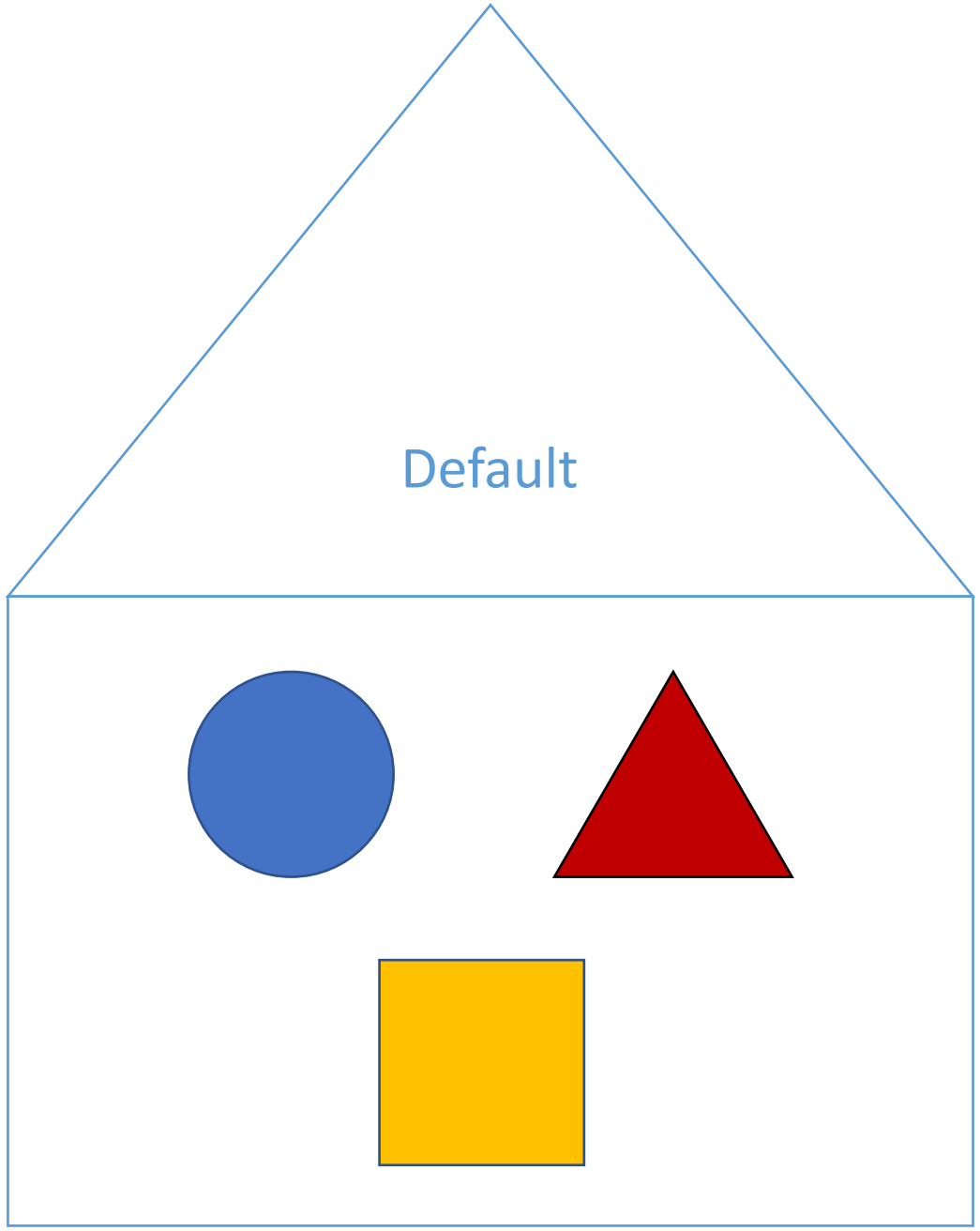

Mark
Mark Williams
MARK SMITH
NAMESPACE
Mark Smith
Mark Williams
MARK WILLIAMS
NAMESPACE

KODEKLOUD

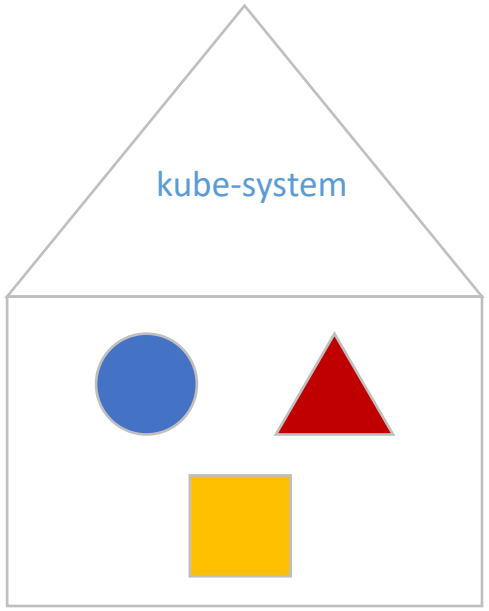


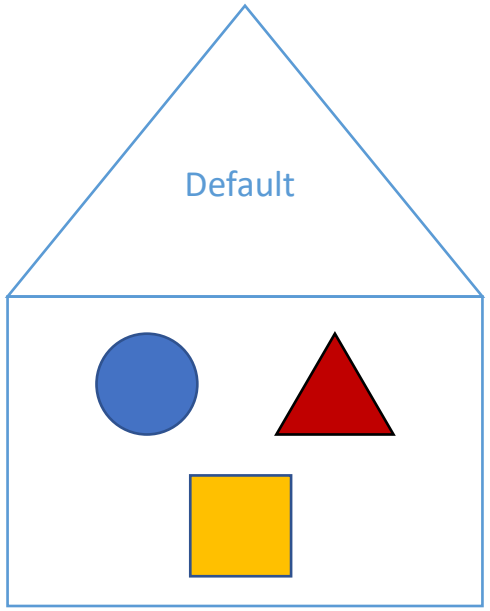


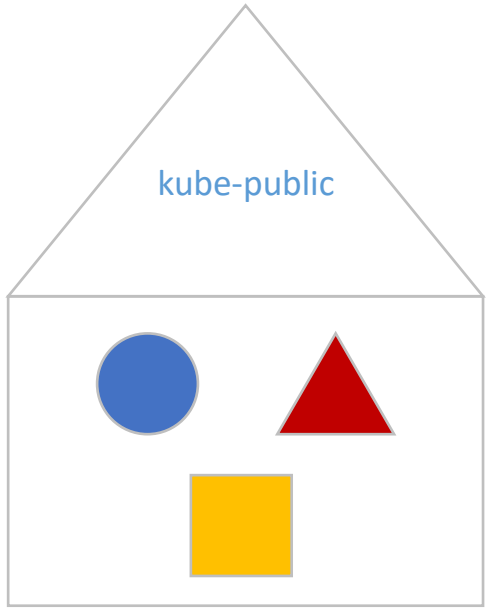



Default

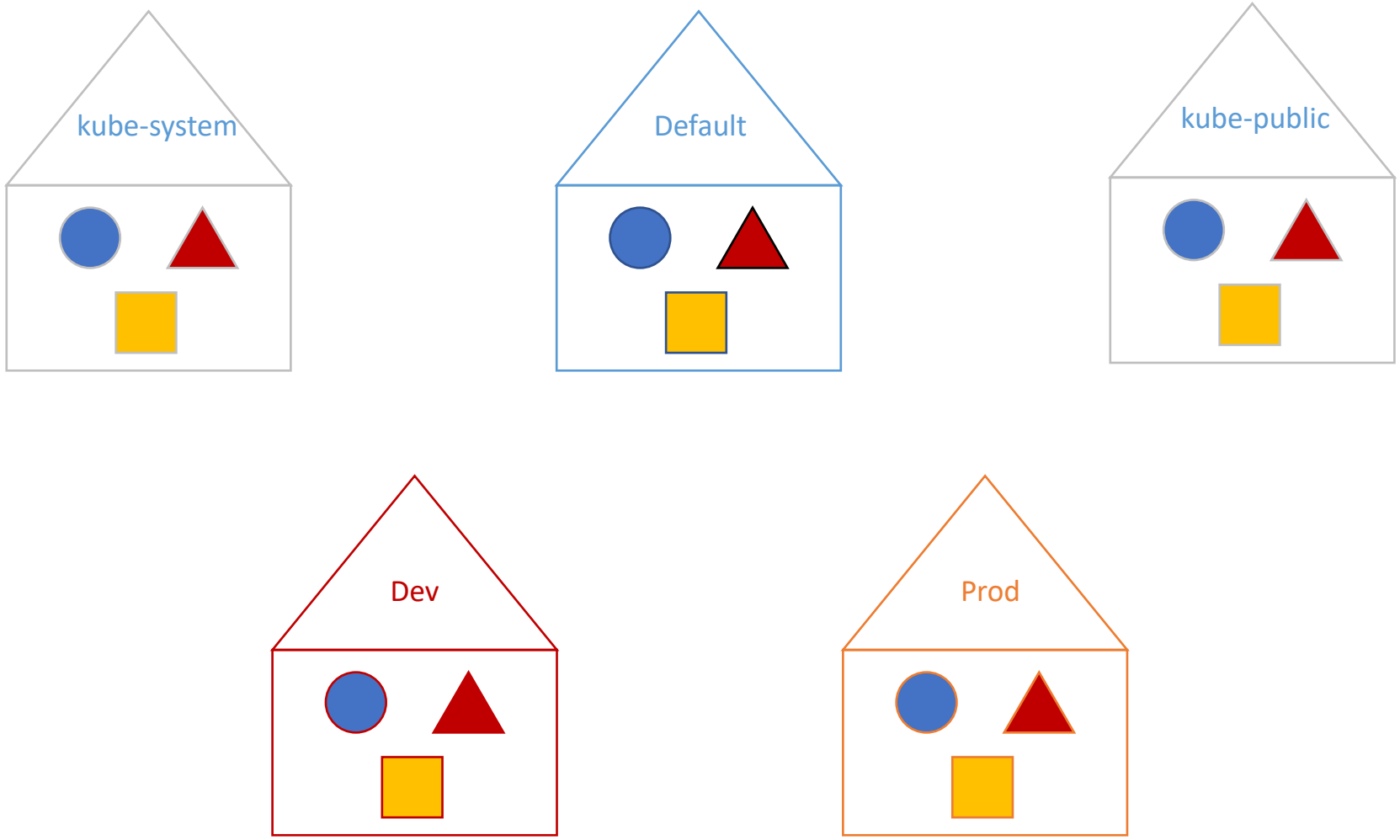
KODEKLOUD



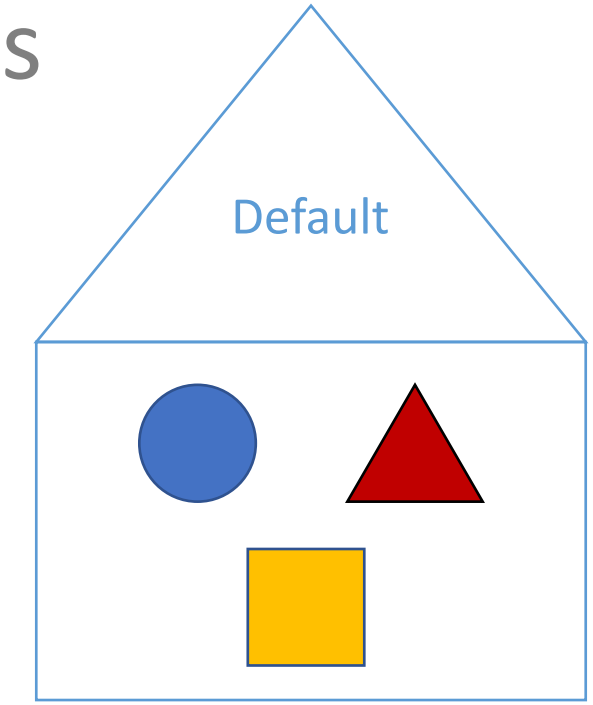


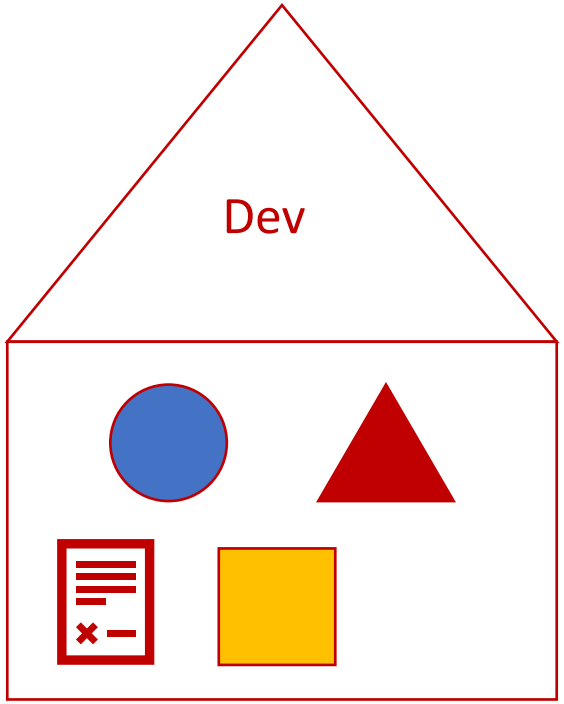


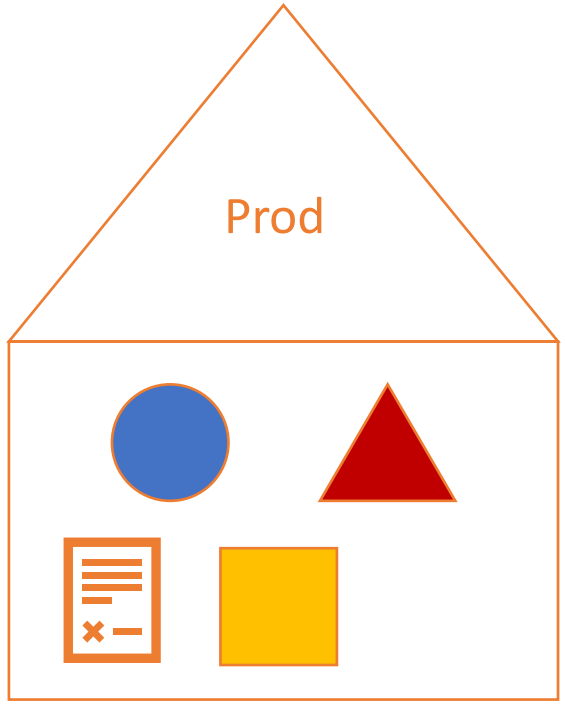
Namespace - Isolation



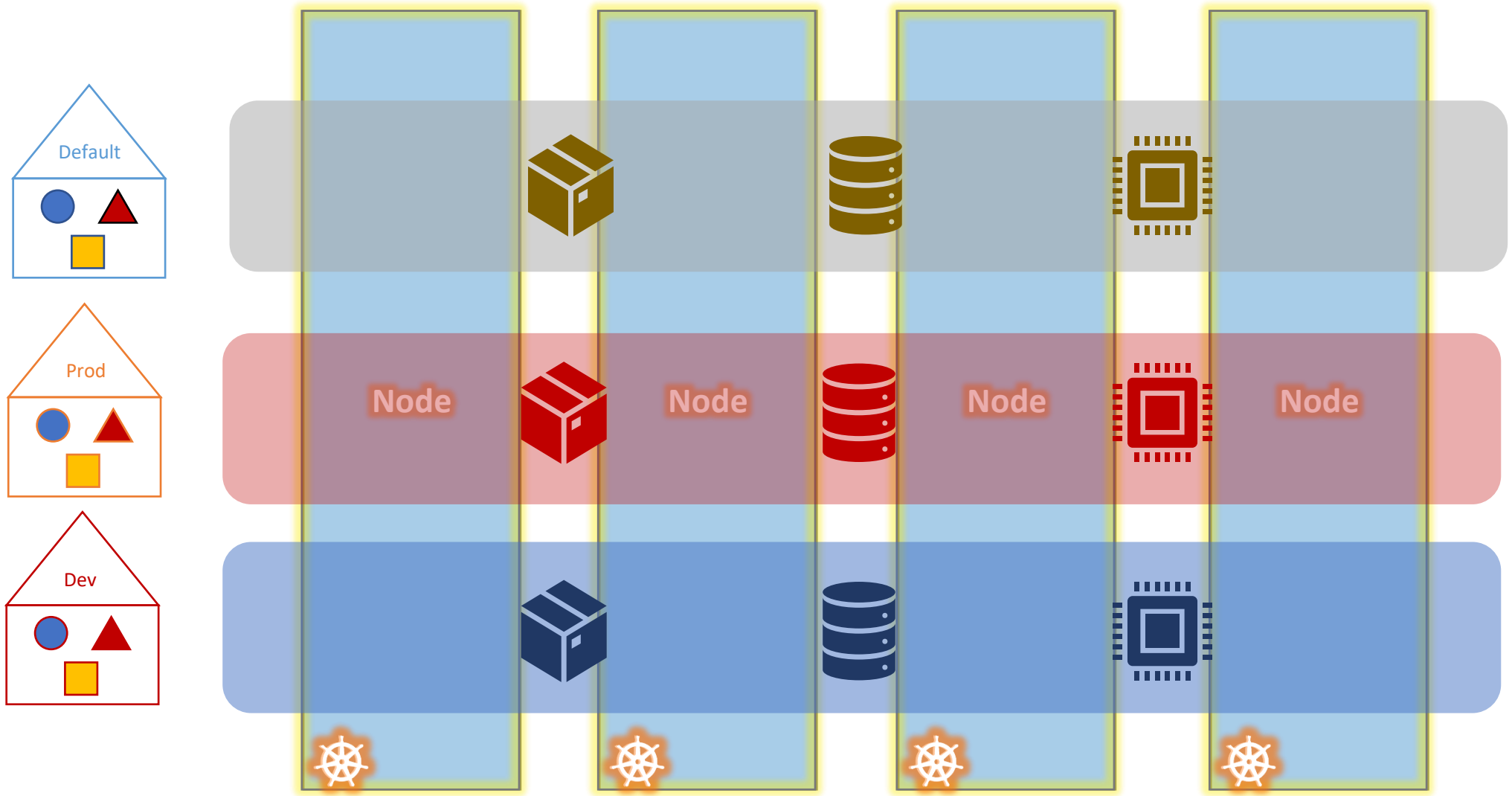
Namespace - Policies



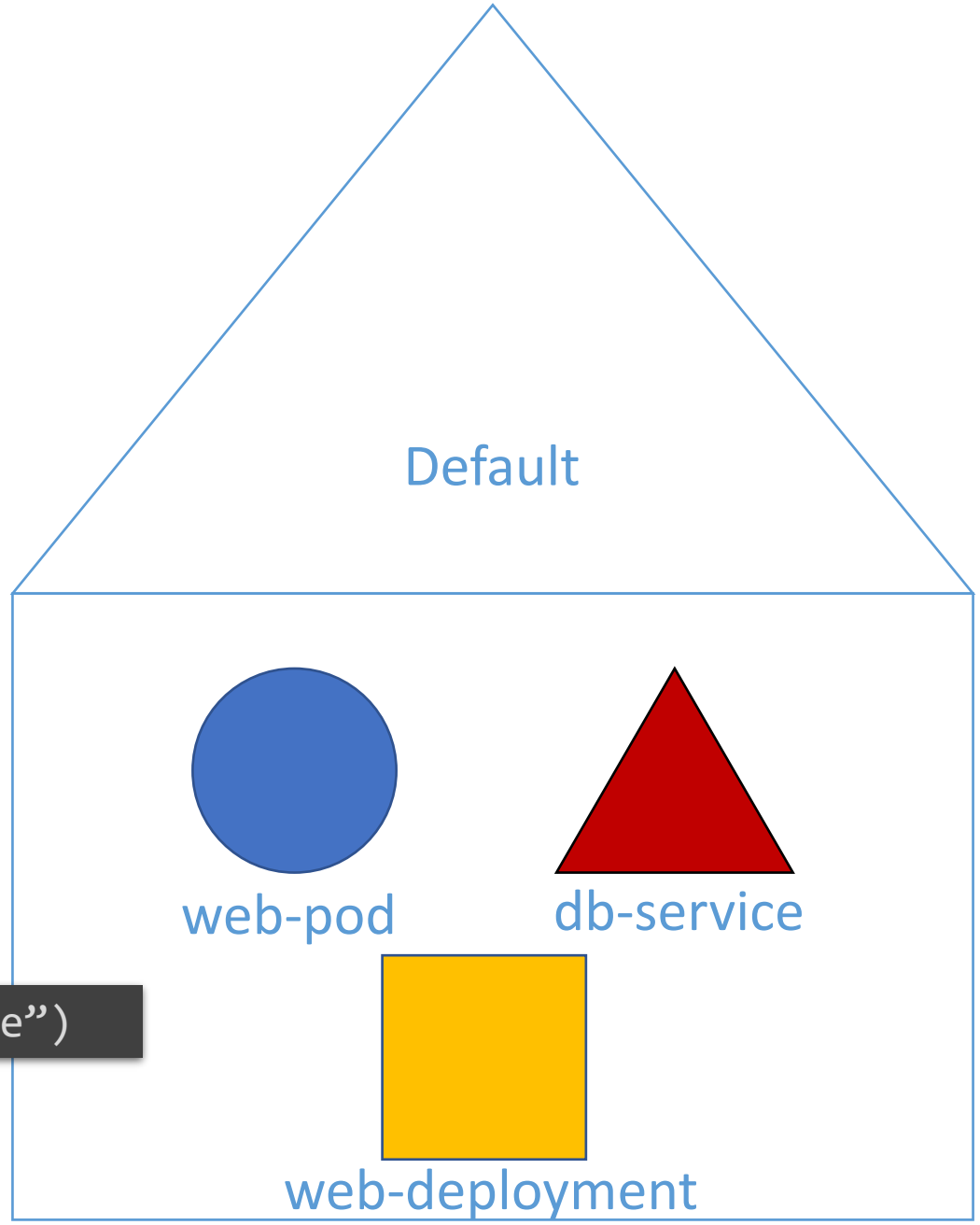




Namespace – Resource Limits

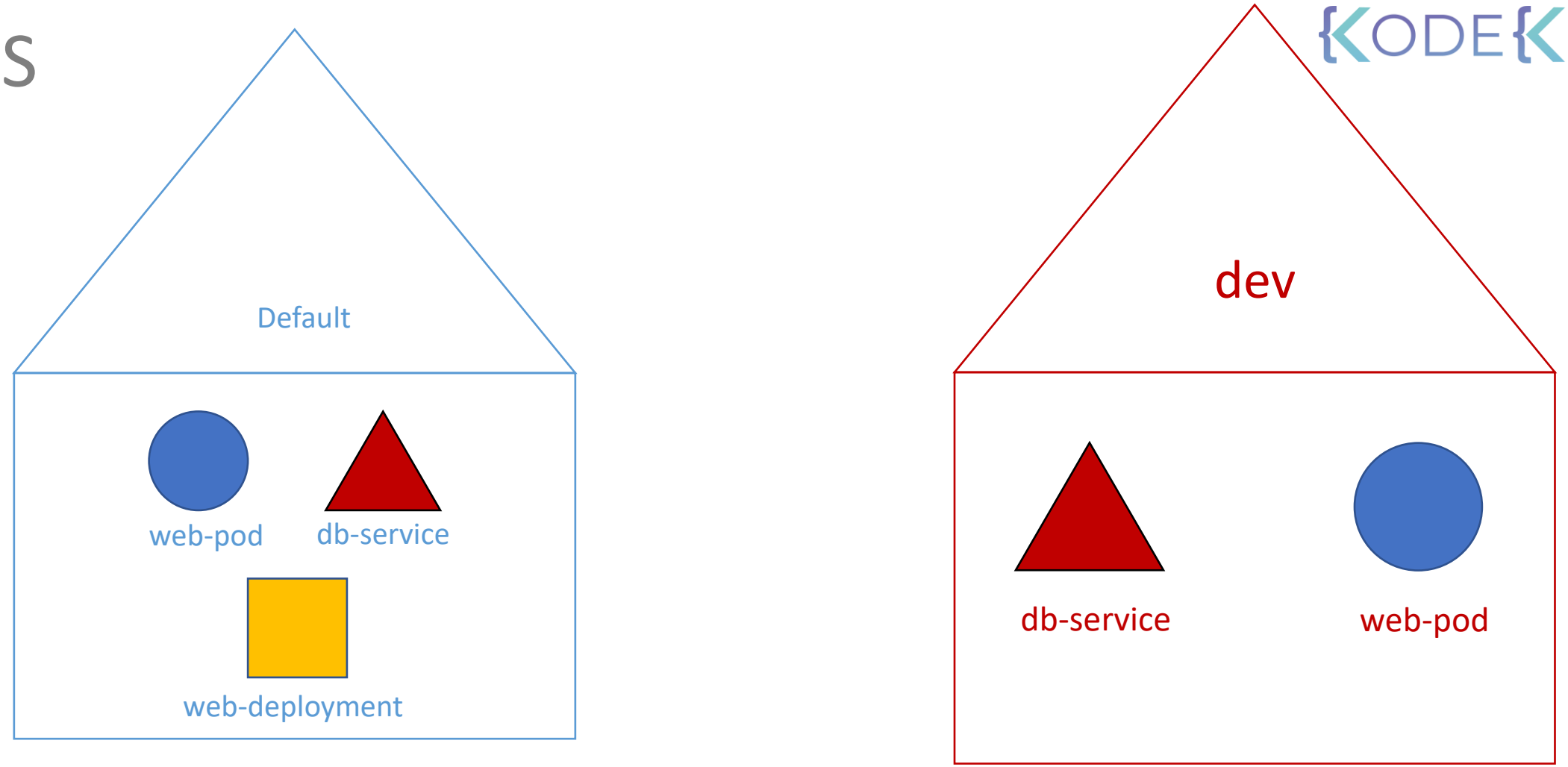


DNS




```
mysql.connect("db-service")
```

DNS




```
mysql.connect("db-service")
```

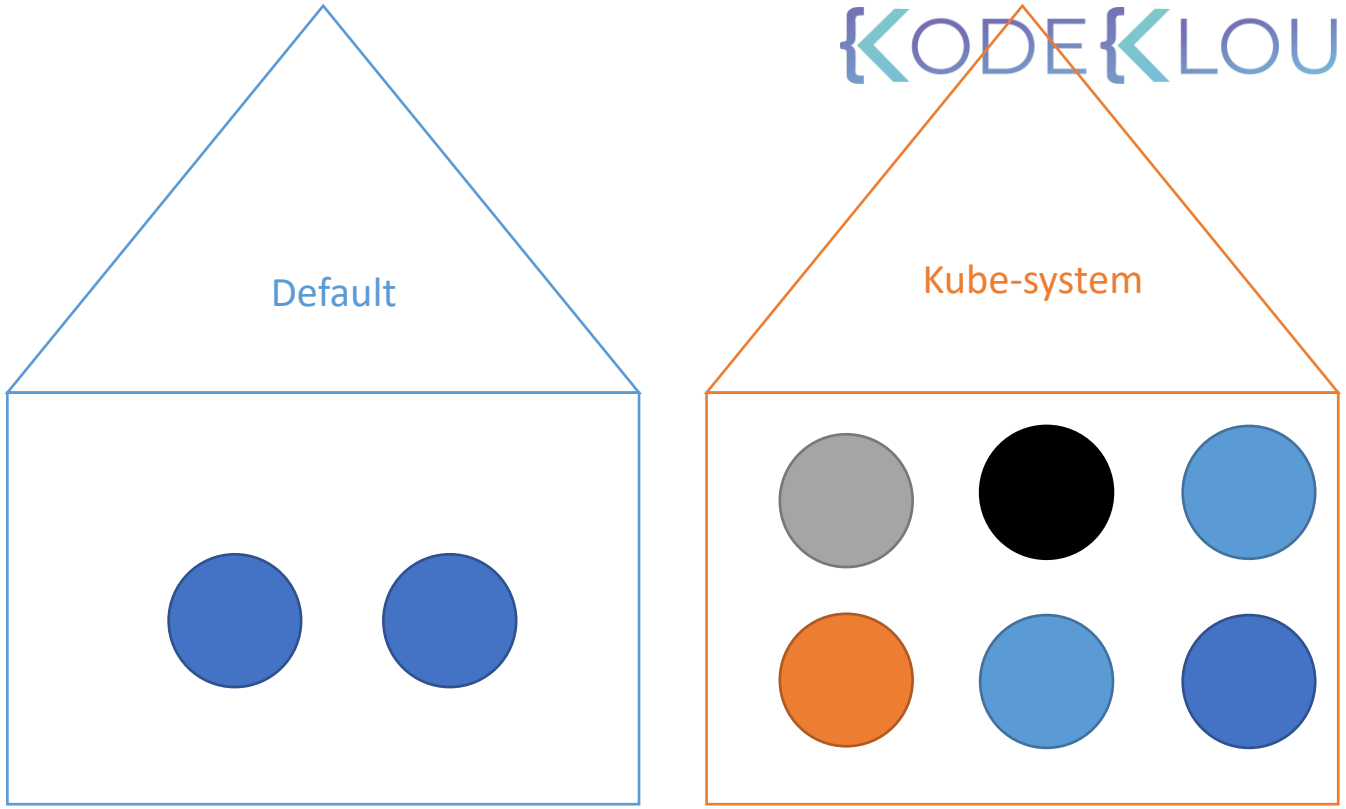
```
mysql.connect("db-service.dev.svc.cluster.local")
```

DNS

```
mysql.connect("db-service.dev.svc.cluster.local")
```

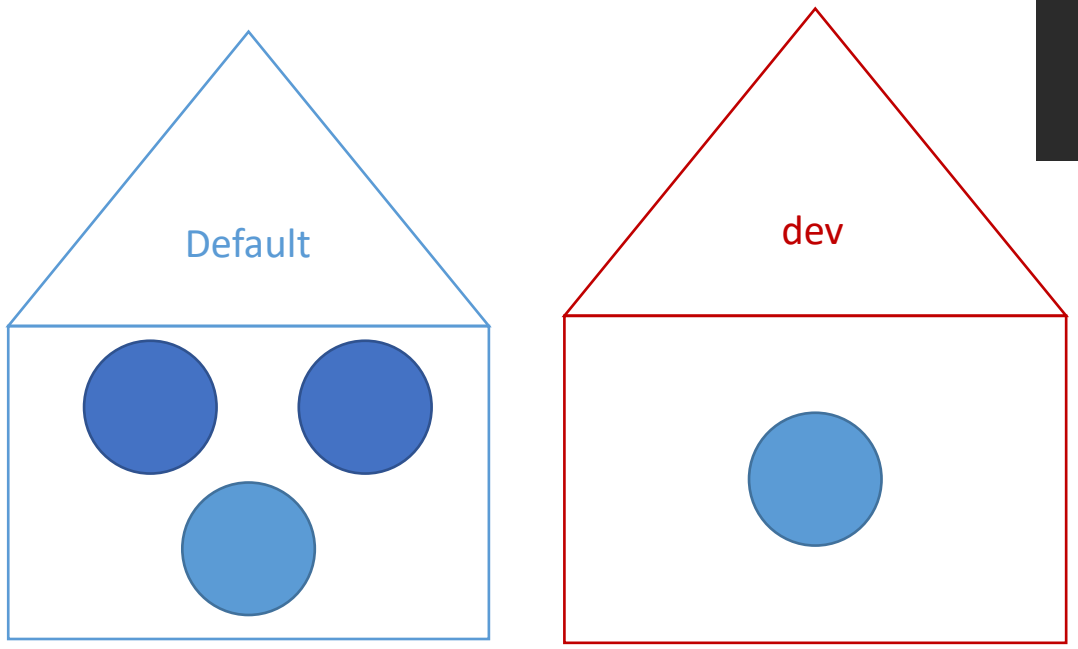
```
> kubectl get pods
NAME      READY     STATUS    RESTARTS   AGE
Pod-1     1/1       Running   0          3d
Pod-2     1/1       Running   0          3d
```

```
> kubectl get pods --namespace=kube-system
NAME                              READY     STATUS    RESTAR
coredns-78fcdf6894-92d52          1/1       Running   7
coredns-78fcdf6894-jx25g          1/1       Running   7
etcd-master                       1/1       Running   7
kube-apiserver-master             1/1       Running   7
kube-controller-manager-master    1/1       Running   7
kube-flannel-ds-amd64-hz4cf       1/1       Running   14
kube-proxy-4b8tn                  1/1       Running   7
kube-proxy-98db4                  1/1       Running   7
kube-proxy-jjrbs                  1/1       Running   7
kube-scheduler-master             1/1       Running   7
```



```
> kubectl create -f pod-definition.yml
pod/myapp-pod created
```

```
> kubectl create -f pod-definition.yml --namespace=dev
pod/myapp-pod created
```

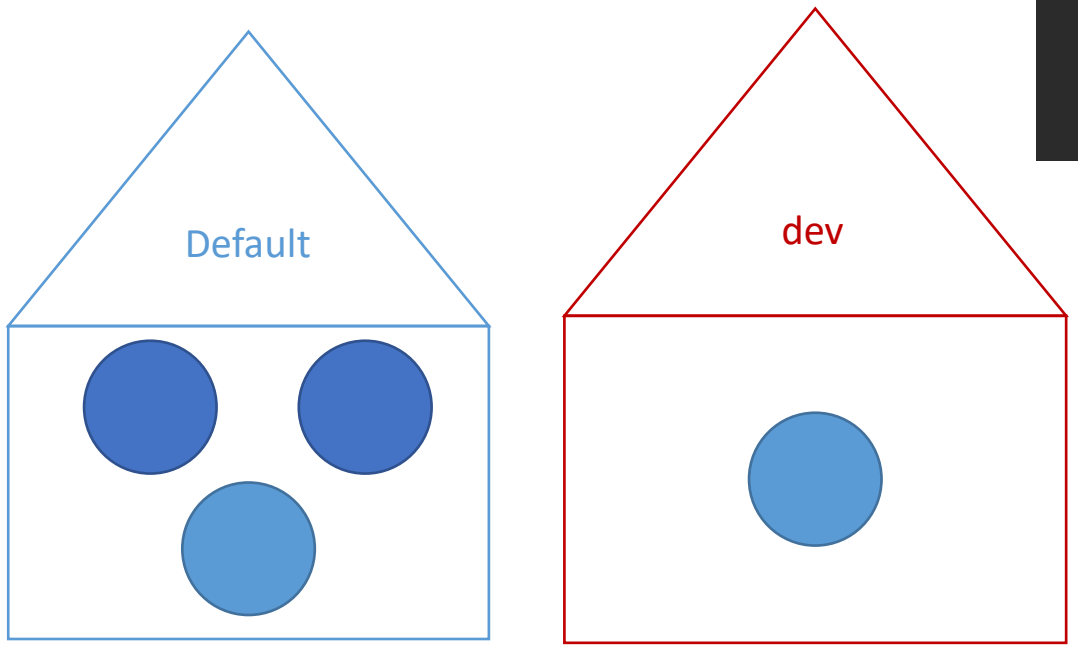


pod-definition.yml

```yaml
apiVersion: v1
kind: Pod

metadata:
 name: myapp-pod
 labels:
    app: myapp
    type: front-end
spec:
  containers:
  - name: nginx-container
    image: nginx
```

```
> kubectl create -f pod-definition.yml
pod/myapp-pod created
```

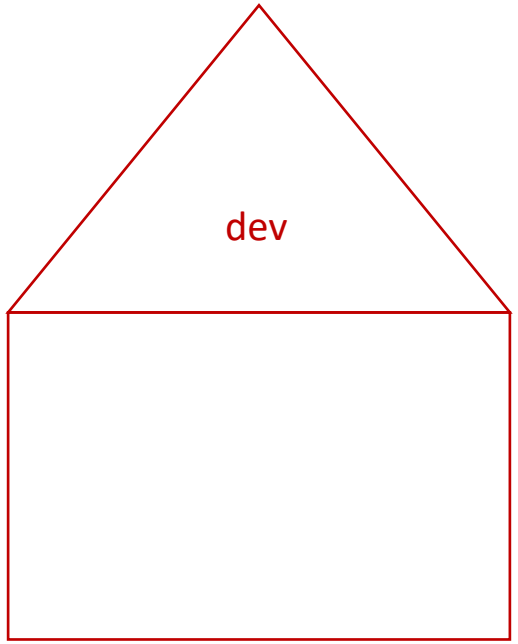
```
> kubectl create -f pod-definition.yml --namespace=dev
pod/myapp-pod created
```



pod-definition.yml

```yaml
apiVersion: v1
kind: Pod

metadata:
 name: myapp-pod
 namespace: dev

 labels:
    app: myapp
    type: front-end
spec:
  containers:
  - name: nginx-container
    image: nginx
```

Create Namespace



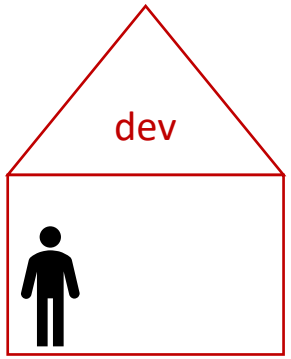
namespace-dev.yml

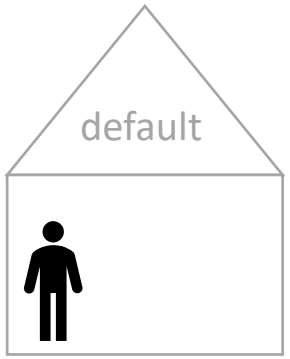
```yaml
apiVersion: v1
kind: Namespace
metadata:
    name: dev
```

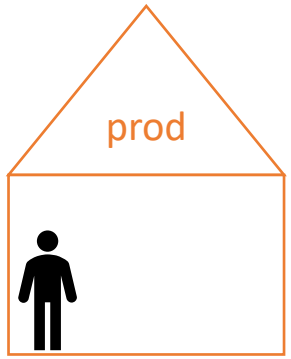
```
> kubectl create -f namespace-dev.yml
namespace/dev created
```

```
> kubectl create namespace dev
namespace/dev created
```

Switch








```
> kubectl get pods --namespace=dev
```

```
> kubectl get pods
```

```
> kubectl get pods --namespace=prod
```

```
> kubectl config set-context $(kubectl config current-context) --namespace=dev
```

```
> kubectl get pods
```

```
> kubectl get pods --namespace=default
```

```
> kubectl get pods --namespace=prod
```

```
> kubectl config set-context $(kubectl config current-context) --namespace=prod
```

```
> kubectl get pods --namespace=dev
```

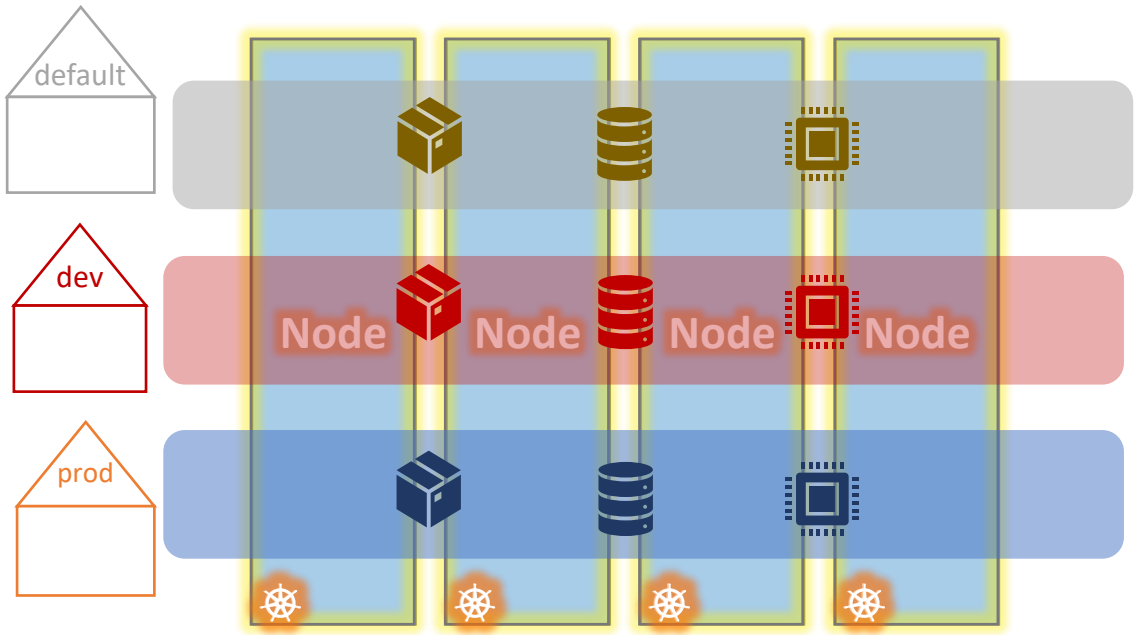
```
> kubectl get pods --namespace=default
```

```
> kubectl get pods
```

```
> kubectl get pods --all-namespaces
```

```
> kubectl config set-context $(kubectl config current-context) --namespace=dev
```

Resource Quota



Compute-quota.yaml

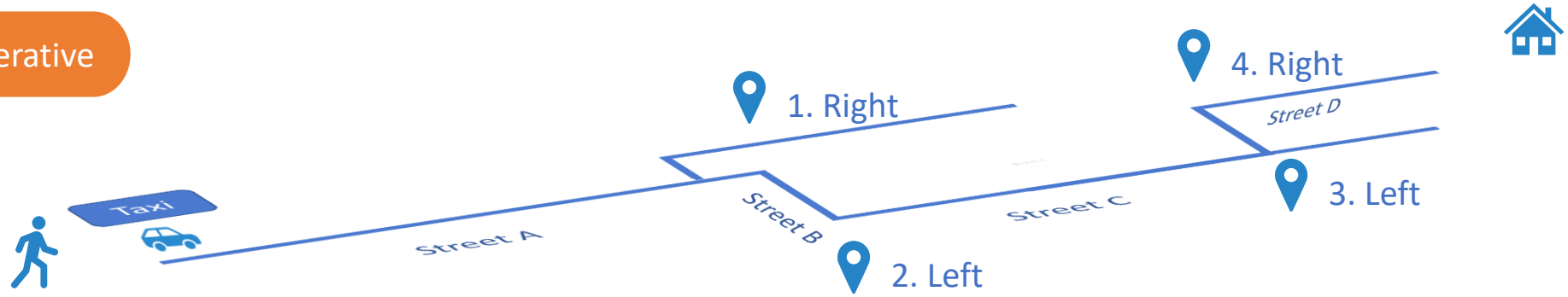
```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
    name: compute-quota
    namespace: dev

spec:
  hard:
    pods: "10"
    requests.cpu: "4"
    requests.memory: 5Gi
    limits.cpu: "10"
    limits.memory: 10Gi
```

```
> kubectl create -f compute-quota.yaml
```

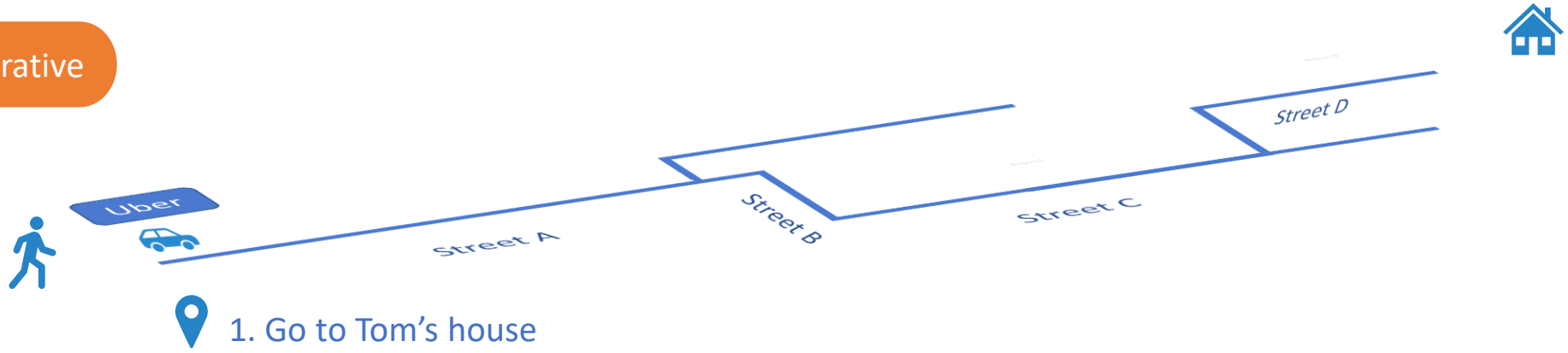
KODEKLOUD

Imperative vs Declarative

Imperative



Declarative



Infrastructure as Code

KODEKLOUD

Imperative

```
1. Provision a VM by the name ‘web-server’
2. Install NGINX Software on it
3. Edit configuration file to use port ‘8080’
4. Edit configuration file to web path ‘/var/www/nginx’
5. Load web pages to ‘/var/www/nginx’ from GIT Repo - X
6. Start NGINX server
```

Declarative

```
VM Name: web-server
Package: nnginx:1.18
Port: 8080
Path: /var/www/nginx
Code: GIT Repo - X
```

Kubernetes

KODEKLOUD

Imperative

```
> kubectl run --image=nginx nginx
```

```
> kubectl create deployment --image=nginx nginx
```

```
> kubectl expose deployment nginx --port 80
```

```
> kubectl edit deployment nginx
```

```
> kubectl scale deployment nginx --replicas=5
```

```
> kubectl set image deployment nginx nginx=nginx:1.18
```

```
> kubectl create –f nginx.yaml
```

```
> kubectl replace –f nginx.yaml
```

```
> kubectl delete –f nginx.yaml
```

Declarative

```
> kubectl apply –f nginx.yaml
```

Imperative Commands

KODEKLOUD

Create Objects

```
> kubectl run --image=nginx nginx
```

```
> kubectl create deployment --image=nginx nginx
```

```
> kubectl expose deployment nginx --port 80
```

Update Objects

```
> kubectl edit deployment nginx
```

```
> kubectl scale deployment nginx --replicas=5
```

```
> kubectl set image deployment nginx nginx=nginx:1.18
```

Imperative Object Configuration Files

Create Objects

```
> kubectl create -f nginx.yaml
```

Update Objects

```
> kubectl edit deployment nginx
```

nginx.yaml

```yaml
apiVersion: v1
kind: Pod

metadata:
  name: myapp-pod
  labels:
    app: myapp
    type: front-end
spec:
  containers:
  - name: nginx-container
    image: nginx
```

Imperative Object Configuration Files

Create Objects

```
> kubectl create -f nginx.yaml
```

Update Objects

```
> kubectl edit deployment nginx
```

nginx.yaml

```yaml
apiVersion: v1
kind: Pod

metadata:
 name: myapp-pod
 labels:
    app: myapp
    type: front-end
spec:
  containers:
  - name: nginx-container
    image: nginx
```

pod-definition

```yaml
apiVersion: v1
kind: Pod

metadata:
 name: myapp-pod
 labels:
    app: myapp
    type: front-end
spec:
  containers:
  - name: nginx-container
    image: nginx:1.18
status:
  conditions:
  - lastProbeTime: null
    status: "True"
    type: Initialized
```

Local file

Kubernetes Memory

Imperative Object Configuration Files

KODEKLOUD

Create Objects

```
> kubectl create -f nginx.yaml
```

nginx.yaml

```yaml
apiVersion: v1
kind: Pod

metadata:
 name: myapp-pod
 labels:
    app: myapp
    type: front-end-service
spec:
  containers:
  - name: nginx-container
    image: nginx:1.18
```

Update Objects

```
> kubectl edit deployment nginx
```

```
> kubectl replace -f nginx.yaml
```

```
> kubectl replace --force -f nginx.yaml
```

```
> kubectl create -f nginx.yaml
Error from server (AlreadyExists): error when creating "nginx.yaml": pods ”myapp-pod" already exists
```

```
> kubectl replace -f nginx.yaml
Error from server (Conflict): error when replacing "nginx.yaml": Operation cannot be fulfilled on pods
”myapp-pod"
```

Declarative

KODEKLOUD

Create Objects

```
> kubectl apply -f nginx.yaml
```

```
> kubectl apply -f /path/to/config-files
```

Update Objects

```
> kubectl apply -f nginx.yaml
```

nginx.yaml

```yaml
apiVersion: v1
kind: Pod

metadata:
  name: myapp-pod
  labels:
    app: myapp
    type: front-end-service
spec:
  containers:
  - name: nginx-container
    image: nginx:1.18
```

Exam Tips

KODEKLOUD

Create Objects

```
> kubectl apply –f nginx.yaml
```

```
> kubectl run --image=nginx nginx
```

```
> kubectl create deployment --image=nginx nginx
```

```
> kubectl expose deployment nginx --port 80
```

Update Objects

```
> kubectl apply –f nginx.yaml
```

```
> kubectl edit deployment nginx
```

```
> kubectl scale deployment nginx --replicas=5
```

```
> kubectl set image deployment nginx nginx=nginx:1.18
```

Search

Home
Getting started
Concepts
Tasks
Install Tools
Administer a Cluster
Configure Pods and Containers
Manage Kubernetes Objects
Declarative Management of Kubernetes Objects Using Configuration Files
Declarative Management of Kubernetes Objects Using Kustomize

Manage Kubernetes Objects

Declarative and imperative paradigms for interacting with the Kubernetes API.

Declarative Management of Kubernetes Objects Using Configuration Files

Declarative Management of Kubernetes Objects Using Kustomize

Managing Kubernetes Objects Using Imperative Commands

Imperative Management of Kubernetes Objects Using Configuration Files

Update API Objects in Place Using kubectl patch
Use kubectl patch to update Kubernetes API objects in place. Do a strategic merge patch or a JSON merge patch.

KODEKLOUD

KODEKLOUD

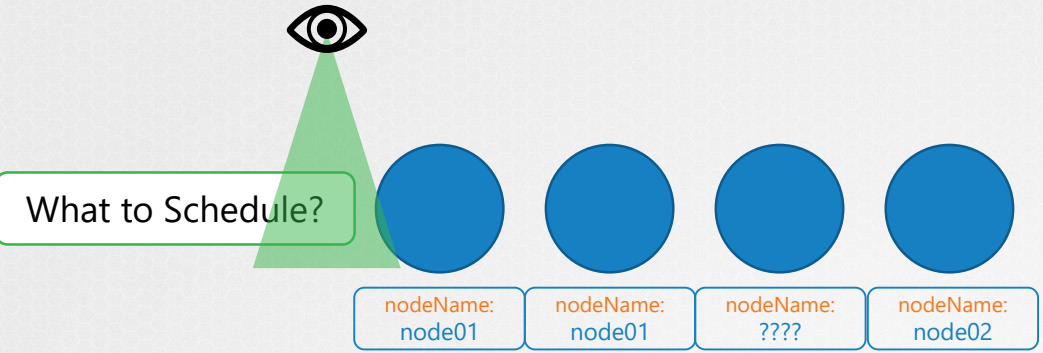
Course Objectives

- Scheduling
  - Manual Scheduling
  - Labels & Selectors
  - Resource Limits
  - daemon Sets
  - Multiple Schedulers
  - Scheduler Events
  - Configure Kubernetes Scheduler
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Troubleshooting

KODEKLOUD

MANUAL SCHEDULING

How scheduling works



Which node to schedule?

(Schedule)Bind Pod to Node

nodeName:
node02

pod-definition.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    name: nginx
spec:
  containers:
  - name: nginx
    image: nginx
    ports:
      - containerPort: 8080

  nodeName: node02
```

KODEKLOUD

No Scheduler!

```
kubectl get pods
```

```
NAME      READY     STATUS              RESTARTS   AGE
nginx     0/1       Pending             0          3s
```

```
kubectl get pods
```

```
NAME      READY     STATUS    RESTARTS   AGE       IP           NODE
nginx     1/1       Running   0          9s        10.40.0.4    node02
```

pod-definition.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    name: nginx
spec:
  containers:
  - name: nginx
    image: nginx
    ports:
      - containerPort: 8080

  nodeName: node02
```

KODEKLOUD

No Scheduler!

Pod-bind-definition.yaml

```yaml
apiVersion: v1
kind: Binding
metadata:
  name: nginx
target:
  apiVersion: v1
  kind: Node
  name:
```

'{"apiVersion":"v1", "kind": "Binding“ …. }

pod-definition.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    name: nginx
spec:
  containers:
  - name: nginx
    image: nginx
    ports:
      - containerPort: 8080

  nodeName: node02
```

```
curl --header "Content-Type:application/json" --request POST --data
http://$SERVER/api/v1/namespaces/default/pods/$PODNAME/binding/
```

KODEKLOUD

KODEKLOUD

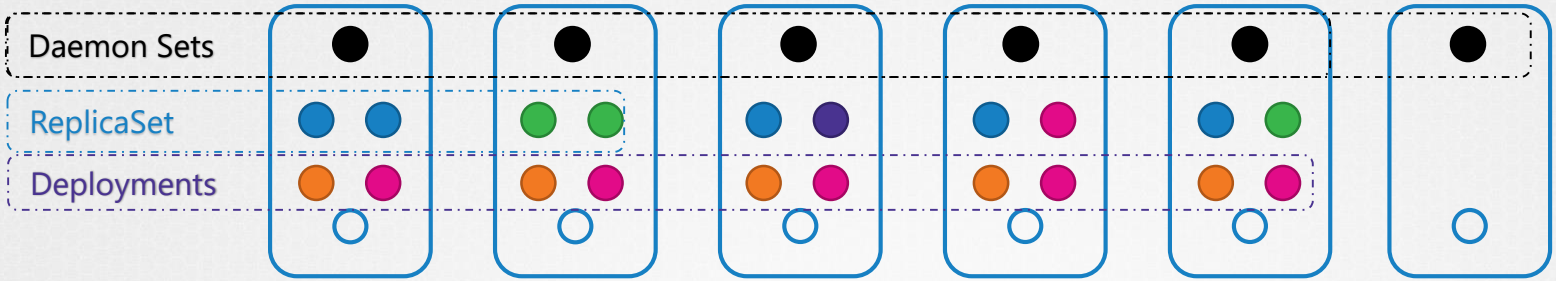
Course Objectives

- Scheduling
  - Manual Scheduling
  - Labels & Selectors
  - Resource Limits
  - daemon Sets
  - Multiple Schedulers
  - Scheduler Events
  - Configure Kubernetes Scheduler
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Troubleshooting

KODEKLOUD

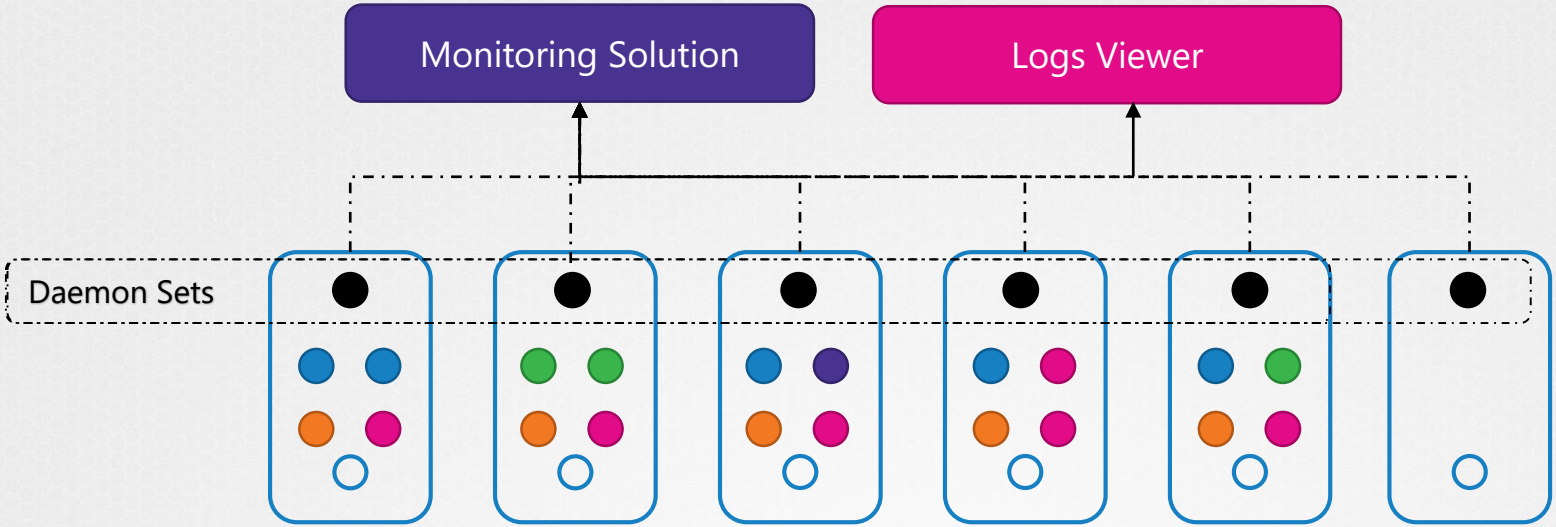
Daemon Sets

Daemon Sets



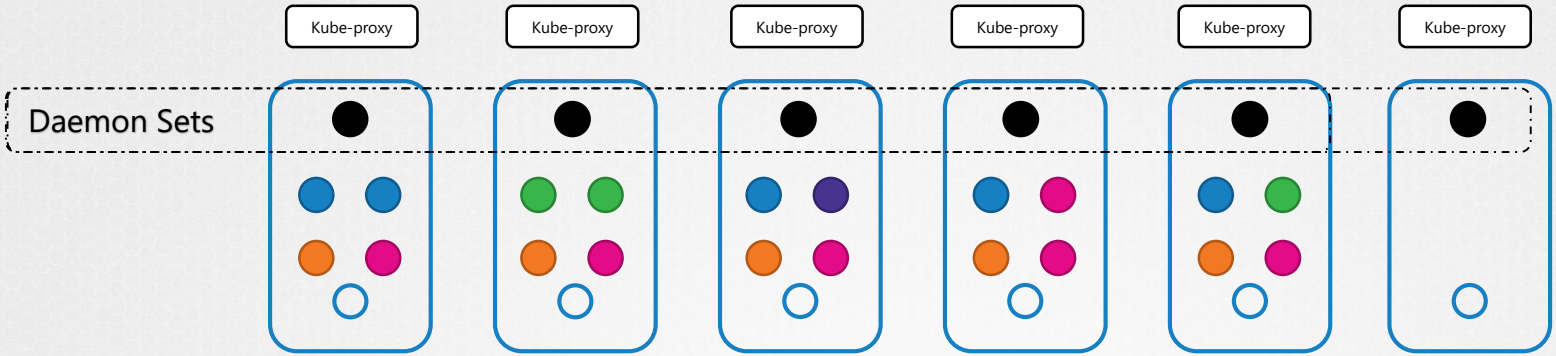
KODEKLOUD

Daemon Sets – UseCase



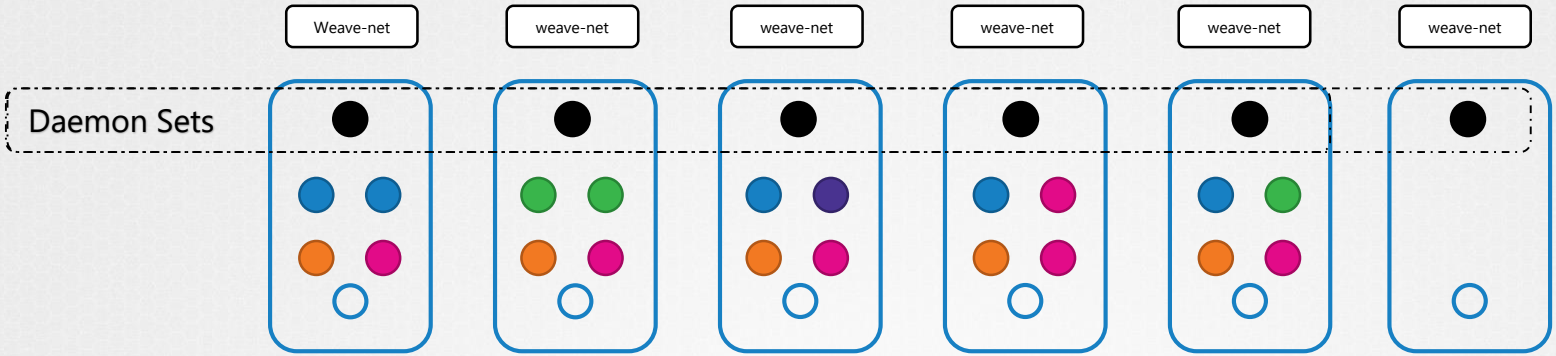
KODEKLOUD

Daemon Sets – UseCase – kube-proxy



KODEKLOUD

Daemon Sets – UseCase – Networking



KODEKLOUD

DaemonSet Definition

daemon-set-definition.yaml

```yaml
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: monitoring-daemon
spec:
  selector:
    matchLabels:
      app: monitoring-agent
  template:
    metadata:
      labels:
        app: monitoring-agent
    spec:
      containers:
      - name: monitoring-agent
        image: monitoring-agent
```

replicaset-definition.yaml

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: monitoring-daemon
spec:
  selector:
    matchLabels:
      app: monitoring-agent
  template:
    metadata:
      labels:
        app: monitoring-agent
    spec:
      containers:
      - name: monitoring-agent
        image: monitoring-agent
```

```
kubectl create -f daemon-set-definition.yaml
```

```
daemon-set Created
```

KODEKLOUD

View DaemonSets

```
kubectl get daemonsets
```

```
      NAME         DESIRED   CURRENT   READY     UP-TO-DATE   AVAILABLE   AGE
monitoring-daemon   1         1         1         1            1           41
```

```
kubectl describe daemonsets monitoring-daemon
```

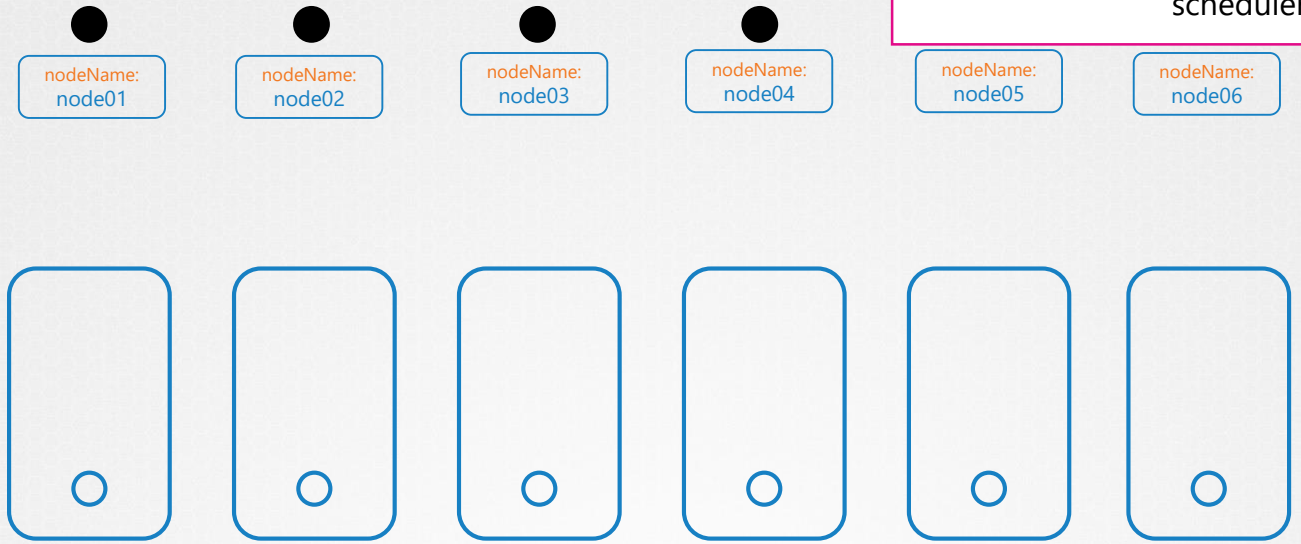
```
Name:           monitoring-daemon
Selector:       name=monitoring-daemon
Node-Selector:  <none>
Labels:         name=monitoring-daemon
Desired Number of Nodes Scheduled: 2
Current Number of Nodes Scheduled: 2
Number of Nodes Scheduled with Up-to-date Pods: 2
Number of Nodes Scheduled with Available Pods: 1
Number of Nodes Misscheduled: 0
Pods Status:  2 Running / 0 Waiting / 0 Succeeded / 0 Failed
Pod Template:
  Labels:           app=monitoring-agent
  Containers:
```

KODEKLOUD

How does it work?

Default Behavior till v1.12

From v1.12 - uses NodeAffinity and default scheduler



KODEKLOUD

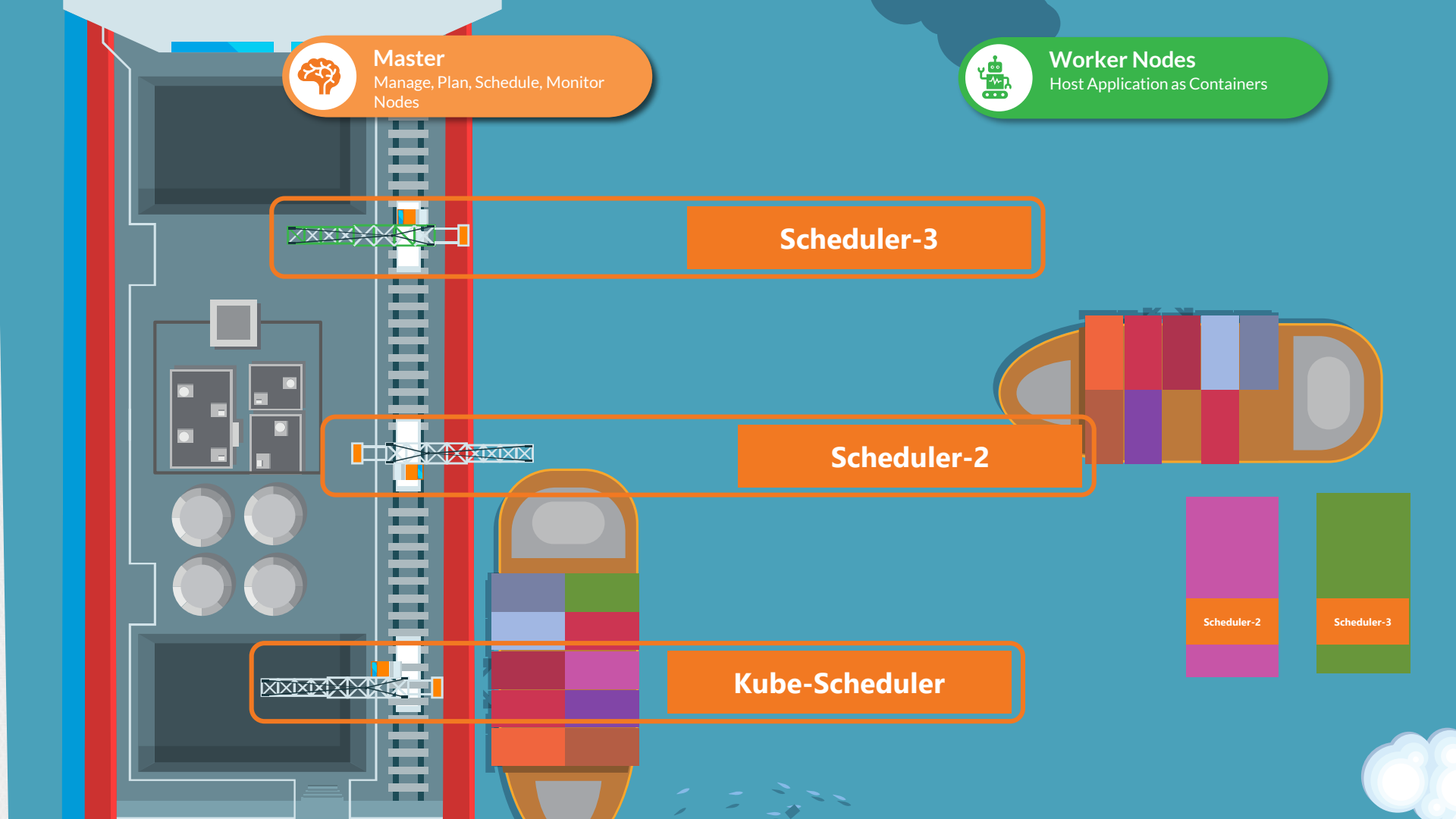
KODEKLOUD

Course Objectives

- Scheduling
  - Manual Scheduling
  - Labels & Selectors
  - Resource Limits
  - daemon Sets
  - Multiple Schedulers
  - Scheduler Events
  - Configure Kubernetes Scheduler
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Troubleshooting

KODEKLOUD

MULTIPLE SCHEDULERS


Master
Manage, Plan, Schedule, Monitor Nodes
Worker Nodes
Host Application as Containers
Scheduler-3
Scheduler-2
Kube-Scheduler
Scheduler-2
Scheduler-3

Deploy Additional Scheduler

```
wget https://storage.googleapis.com/kubernetes-release/release/v1.12.0/bin/linux/amd64/kube-scheduler
```

kube-scheduler.service

```
ExecStart=/usr/local/bin/kube-scheduler \\
  --config=/etc/kubernetes/config/kube-scheduler.yaml \\
  --scheduler-name= default-scheduler
```

my-custom-scheduler.service

```
ExecStart=/usr/local/bin/kube-scheduler \\
  --config=/etc/kubernetes/config/kube-scheduler.yaml \\
  --scheduler-name= my-custom-scheduler
```

KODEKLOUD

Deploy Additional Scheduler - kubeadm

/etc/kubernetes/manifests/kube-scheduler.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: kube-scheduler
  namespace: kube-system
spec:
  containers:
  - command:
    - kube-scheduler
    - --address=127.0.0.1
    - --kubeconfig=/etc/kubernetes/scheduler.conf
    - --leader-elect=true
    image: k8s.gcr.io/kube-scheduler-amd64:v1.11.3
    name: kube-scheduler
```

my-custom-scheduler.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-custom-scheduler
  namespace: kube-system
spec:
  containers:
  - command:
    - kube-scheduler
    - --address=127.0.0.1
    - --kubeconfig=/etc/kubernetes/scheduler.conf
    - --leader-elect=true
    image: k8s.gcr.io/kube-scheduler-amd64:v1.11.3
    name: kube-scheduler
```

KODEKLOUD

View Schedulers

```
kubectl get pods --namespace=kube-system
```

```
NAME                              READY     STATUS    RESTARTS   AGE
coredns-78fcdf6894-bk4ml          1/1       Running   0          1h
coredns-78fcdf6894-ppr6m          1/1       Running   0          1h
etcd-master                       1/1       Running   0          1h
kube-apiserver-master             1/1       Running   0          1h
kube-controller-manager-master    1/1       Running   0          1h
kube-proxy-dgbgv                  1/1       Running   0          1h
kube-proxy-fptbr                  1/1       Running   0          1h
kube-scheduler-master             1/1       Running   0          1h
my-custom-scheduler               1/1       Running   0          9s
weave-net-4tfpt                   2/2       Running   1          1h
weave-net-6j6zs                   2/2       Running   1          1h
```

KODEKLOUD

Use Custom Scheduler

```
kubectl get pods --namespace=kube-system
```

```
NAME                                READY     STATUS    RESTARTS   AGE
coredns-78fcdf6894-bk4ml            1/1       Running   0          1h
coredns-78fcdf6894-ppr6m            1/1       Running   0          1h
etcd-master                         1/1       Running   0          1h
kube-apiserver-master               1/1       Running   0          1h
kube-controller-manager-master      1/1       Running   0          1h
kube-proxy-dgbgv                    1/1       Running   0          1h
kube-proxy-fptbr                    1/1       Running   0          1h
kube-scheduler-master               1/1       Running   0          1h
my-custom-scheduler                 1/1       Running   0          9s
weave-net-4tfpt                     2/2       Running   1          1h
weave-net-6j6zs                     2/2       Running   1          1h
```

pod-definition.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  -   image: nginx
      name: nginx

   schedulerName:
```

```
kubectl create -f pod-definition.yaml
```

```
kubectl get pods
```

```
NAME      READY     STATUS    RESTARTS   AGE
nginx     0/1       Pending   0          6s
```

```
kubectl get pods
```

```
NAME      READY     STATUS    RESTARTS   AGE
nginx     1/1       Running   0          6s
```

View Events

```
kubectl get events
```

```
LAST SEEN   COUNT     NAME          KIND  TYPE      REASON      SOURCE                 MESSAGE
9s          1         nginx.15      Pod   Normal    Scheduled   my-custom-scheduler    Successfully assigned default/nginx to node01
8s          1         nginx.15      Pod   Normal    Pulling     kubelet, node01        pulling image "nginx"
2s          1         nginx.15      Pod   Normal    Pulled      kubelet, node01        Successfully pulled image "nginx"
2s          1         nginx.15      Pod   Normal    Created     kubelet, node01        Created container
2s          1         nginx.15      Pod   Normal    Started     kubelet, node01        Started container
```

KODEKLOUD

View Scheduler Logs

```
kubectl logs my-custom-scheduler --name-space=kube-system
```

```
I0204 09:42:25.819338       1 server.go:126] Version: v1.11.3
W0204 09:42:25.822720       1 authorization.go:47] Authorization is disabled
W0204 09:42:25.822745       1 authentication.go:55] Authentication is disabled
I0204 09:42:25.822801       1 insecure_serving.go:47] Serving healthz insecurely on 127.0.0.1:10251
I0204 09:45:14.725407       1 controller_utils.go:1025] Waiting for caches to sync for scheduler controller
I0204 09:45:14.825634       1 controller_utils.go:1032] Caches are synced for scheduler controller
I0204 09:45:14.825814       1 leaderelection.go:185] attempting to acquire leader lease  kube-system/my-custom-scheduler...
I0204 09:45:14.834953       1 leaderelection.go:194] successfully acquired lease kube-system/my-custom-scheduler
```

KODEKLOUD

KODEKLOUD

Course Objectives

- Scheduling
  - Labels & Selectors
  - Resource Limits
  - Manual Scheduling
  - daemon Sets
  - Multiple Schedulers
  - Scheduler Events
  - Configure Kubernetes Scheduler
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Troubleshooting

KODEKLOUD

CONFIGURING SCHEDULER

Deploy Additional Scheduler

```
wget https://storage.googleapis.com/kubernetes-release/release/v1.12.0/bin/linux/amd64/kube-scheduler
```

kube-scheduler.service

```
ExecStart=/usr/local/bin/kube-scheduler \\
  --config=/etc/kubernetes/config/kube-scheduler.yaml \\
  --scheduler-name= default-scheduler
```

my-custom-scheduler.service

```
ExecStart=/usr/local/bin/kube-scheduler \\
  --config=/etc/kubernetes/config/kube-scheduler.yaml \\
  --scheduler-name= my-custom-scheduler
```

KODEKLOUD

Deploy Additional Scheduler - kubeadm

/etc/kubernetes/manifests/kube-scheduler.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: kube-scheduler
  namespace: kube-system
spec:
  containers:
  - command:
    - kube-scheduler
    - --address=127.0.0.1
    - --kubeconfig=/etc/kubernetes/scheduler.conf
    - --leader-elect=true
    image: k8s.gcr.io/kube-scheduler-amd64:v1.11.3
    name: kube-scheduler
```

my-custom-scheduler.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-custom-scheduler
  namespace: kube-system
spec:
  containers:
  - command:
    - kube-scheduler
    - --address=127.0.0.1
    - --kubeconfig=/etc/kubernetes/scheduler.conf
    - --leader-elect=true
    image: k8s.gcr.io/kube-scheduler-amd64:v1.11.3
    name: kube-scheduler
    - --lock-object-name=my-custom-scheduler
```

KODEKLOUD

KODEKLOUD

Course Objectives

- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
  - Authentication & Authorization
  - Kubernetes Security
  - Network Policies
- Storage
- Troubleshooting

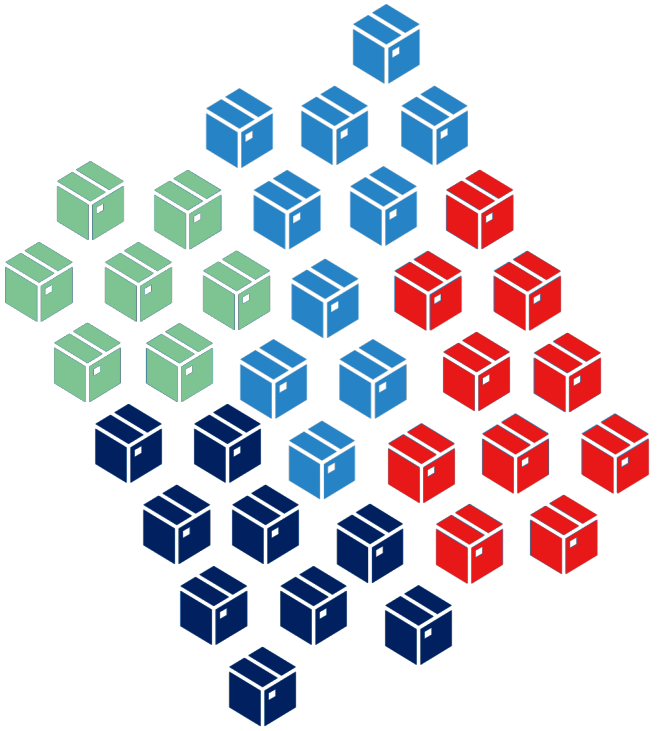
Secrets

- TLS Certificates for Cluster Components
- Images Securely
- Security Contexts
- Secure Persistent Key Value Store

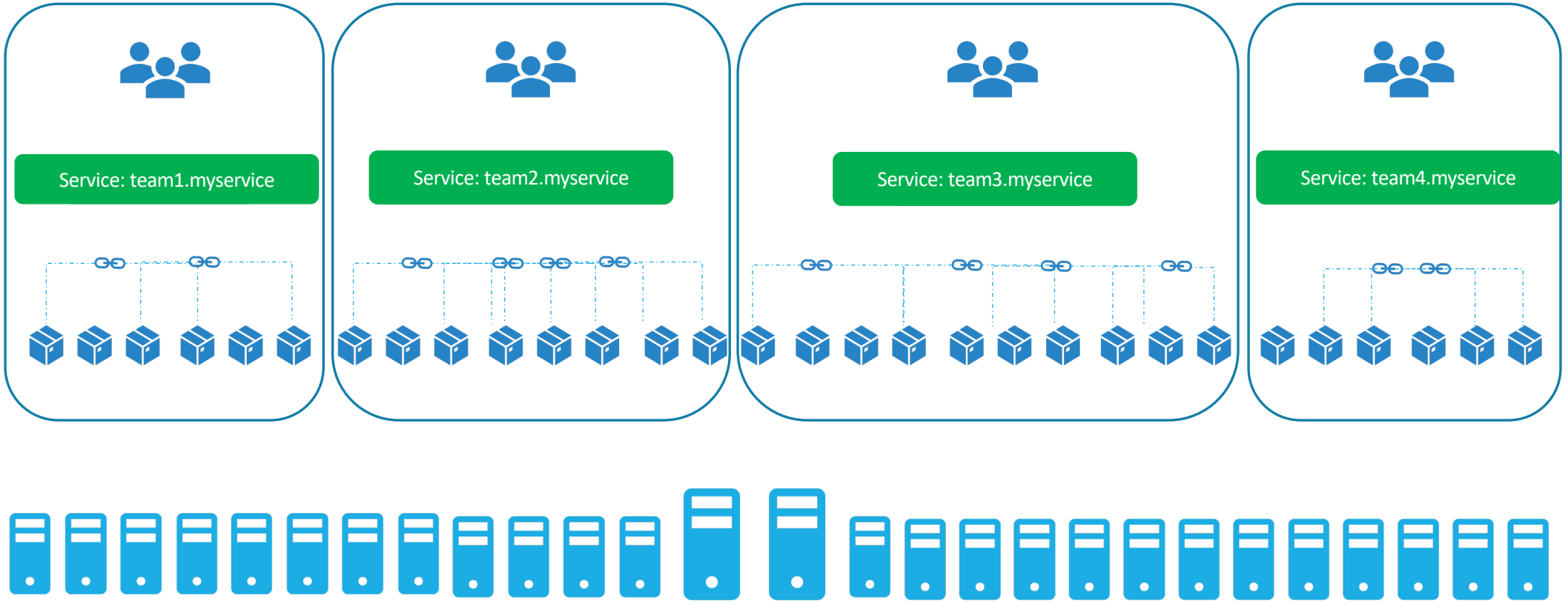
KODEKLOUD

AUTHENTICATION

Namespace



Namespace


Service: team1.myservice
Service: team2.myservice
Service: team3.myservice
Service: team4.myservice

View Namespaces

```
> kubectl get namespaces
NAME          STATUS   AGE
default       Active   20m
kube-system   Active   20m
```

View Resources in a Namespace

```
> kubectl get pods
NAME            READY     STATUS    RESTARTS   AGE
myapp-pod       1/1       Running   0          20s
```

```
> kubectl get pods --namespace=kube-system
NAME                                READY     STATUS    RESTARTS   AGE
etcd-kubemaster                     1/1       Running   2          14d
kube-apiserver-kubemaster           1/1       Running   4          14d
kube-controller-manager-kubemaster  1/1       Running   4          14d
kube-dns-86f4d74b45-zgh8p           3/3       Running   10         14d
kube-flannel-ds-gmg5r               1/1       Running   4          14d
kube-flannel-ds-kt74d               1/1       Running   4          14d
kube-flannel-ds-qtlg8               1/1       Running   4          14d
kube-proxy-4nkb6                    1/1       Running   3          14d
kube-proxy-b6wnm                    1/1       Running   3          14d
kube-proxy-rph7b                    1/1       Running   2          14d
kube-scheduler-kubemaster           1/1       Running   3          14d
metrics-server-6fbfb84cdd-jt5q9     1/1       Running   0          3d
```

Create Namespace

namespace-definition.yml

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: team1

spec:
```

```
kubectl create -f namespace-definition.yml
```

```
namespace “team1” created
```

Assign Objects to Namespace

```
kubectl create -f pod-definition.yml  --namespace=team1
pod "myapp-pod" created
```

```
kubectl create -f pod-definition.yml
pod "myapp-pod" created
```

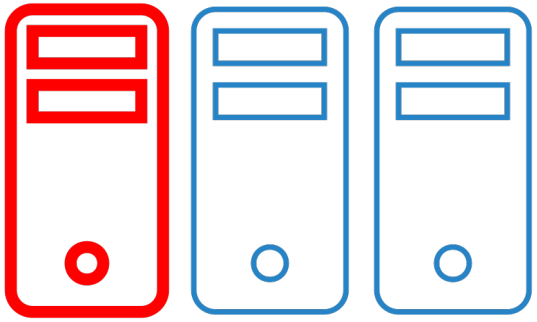
pod-definition.yml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  namespace: team1
  labels:
    app: myapp
    type: front-end
spec:
  containers:
  - name: nginx-container
    image: nginx
```

Demo

Namespace

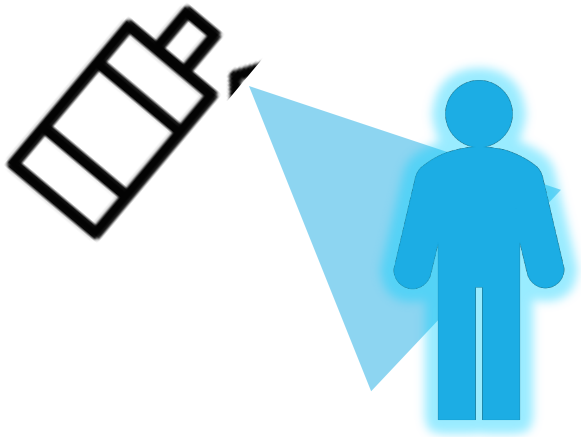
Taints And Tolerations



2

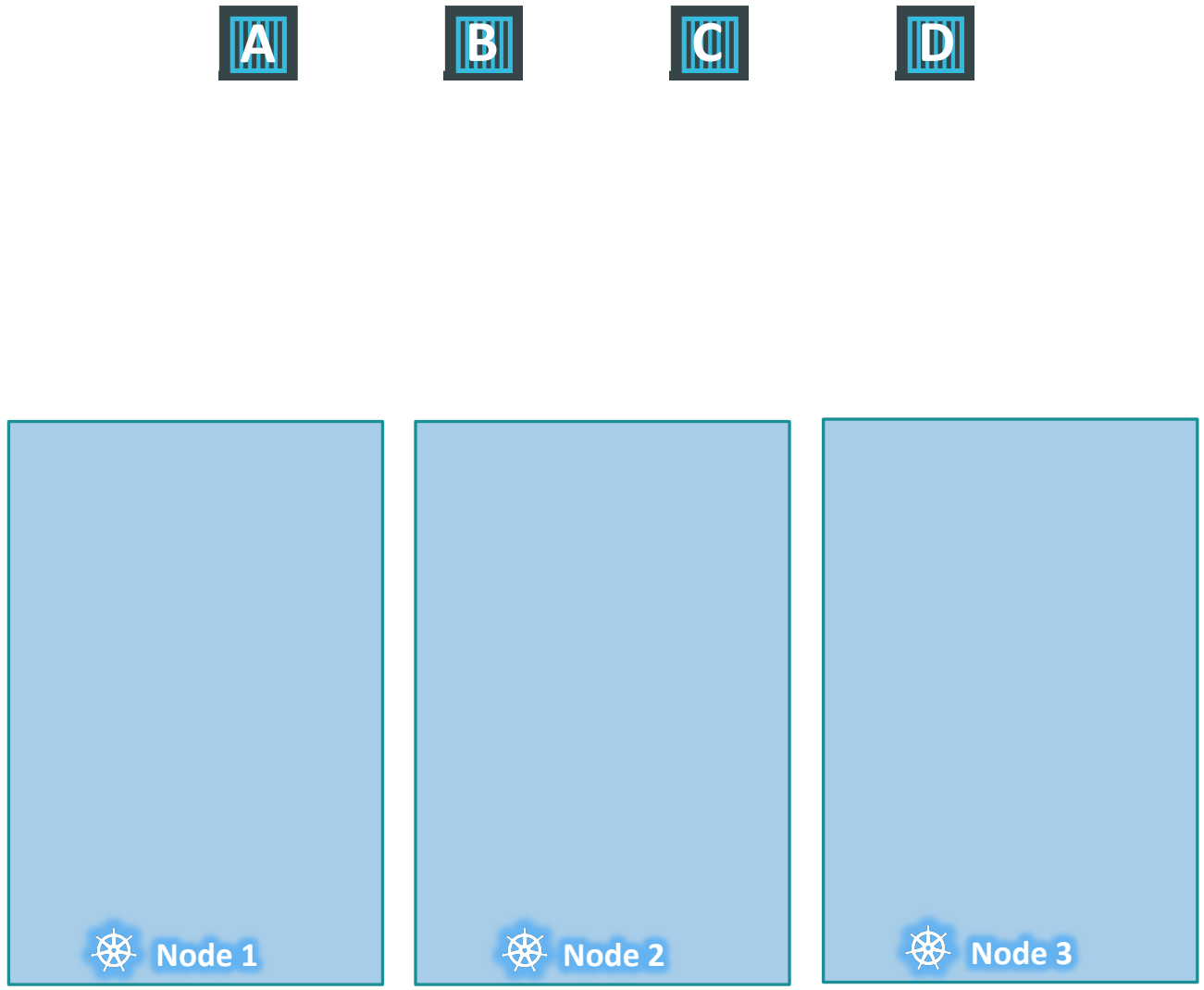
Intolerant

Tolerant

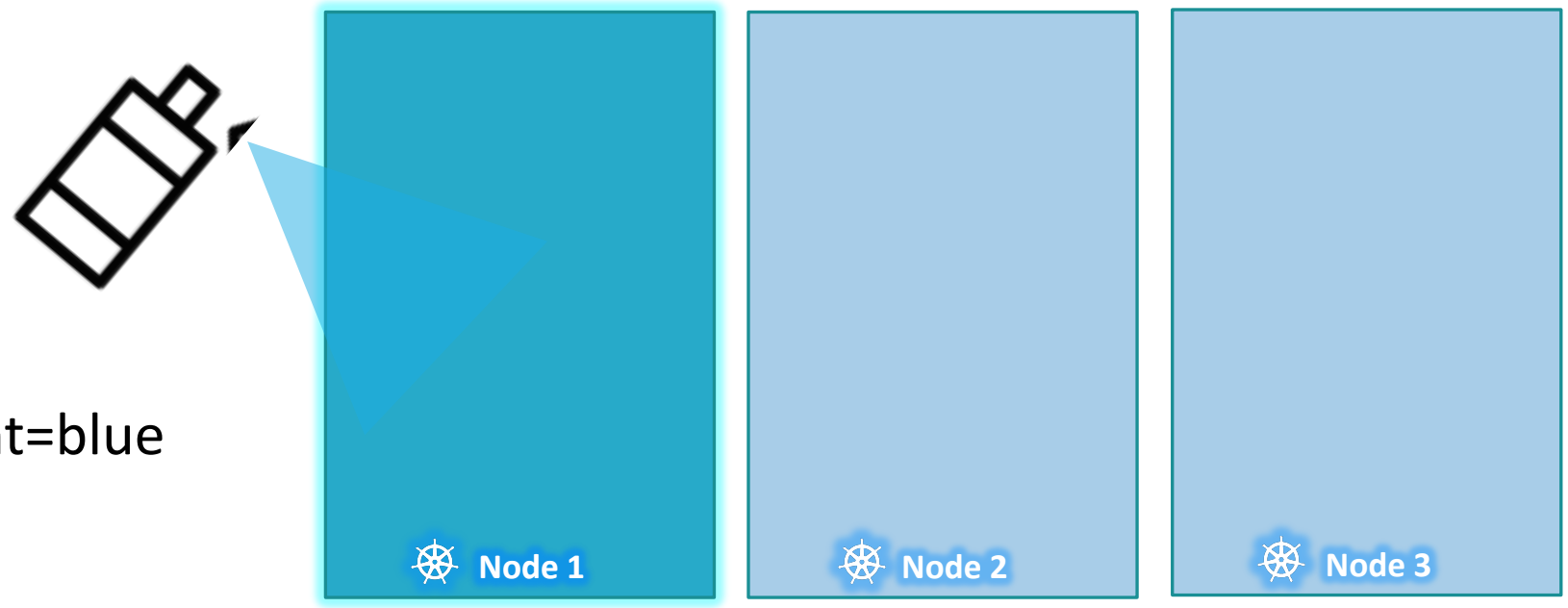


1

Taint


A
B
C
D
Node 1
Node 2
Node 3

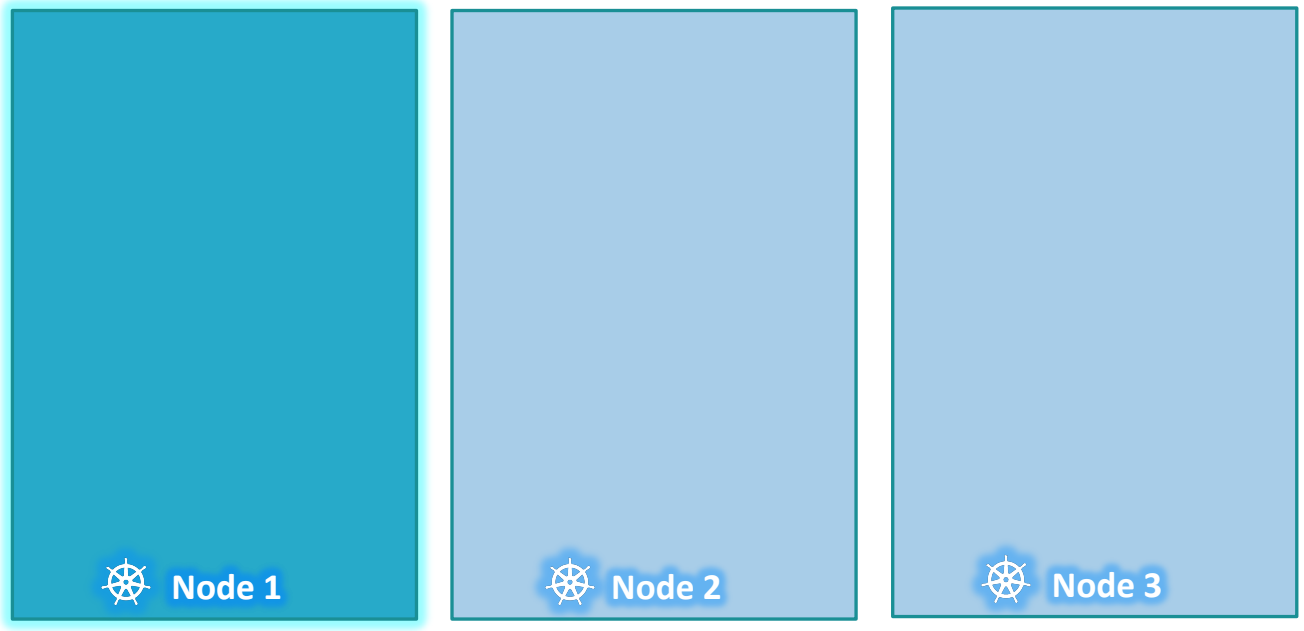
2



Taint=blue

1

Taint=blue



Taints - Node

```
kubectl taint nodes node-name key=value:taint-effect
```

```
NoSchedule | PreferNoSchedule | NoExecute
```

What happens to PODs that do not tolerate this taint?

```
kubectl taint nodes node1 app=myapp:NoSchedule
```

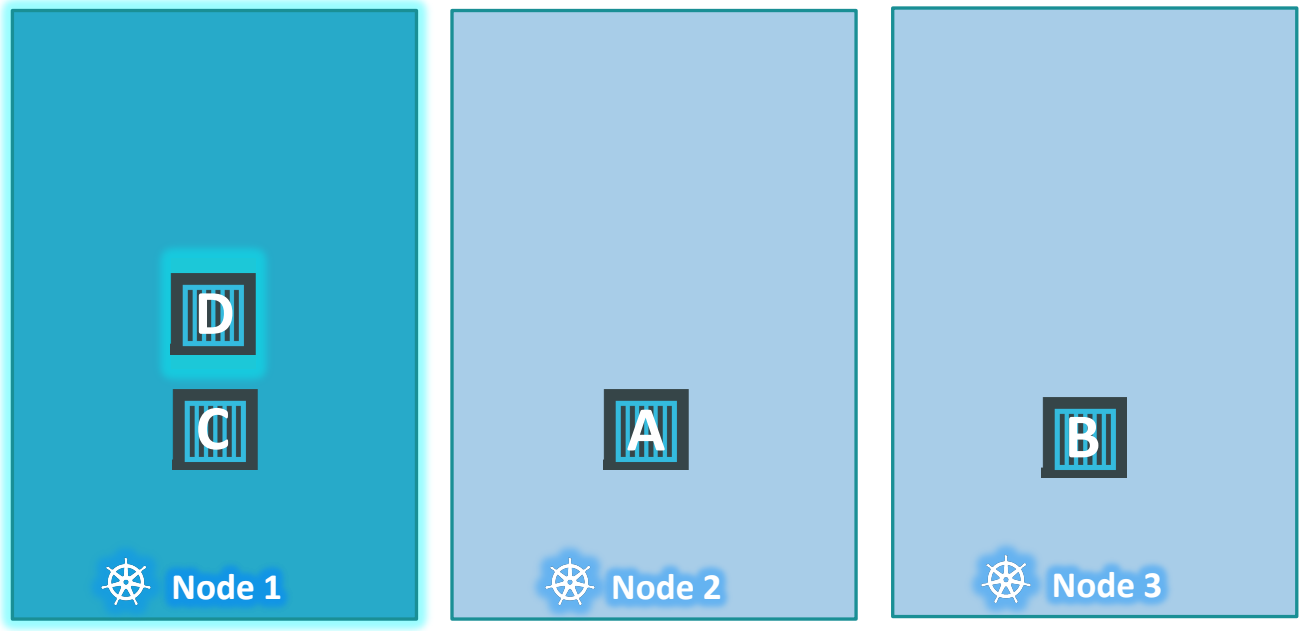
Tolerations - PODs

```
kubectl taint nodes node1 app=myapp:NoSchedule
```

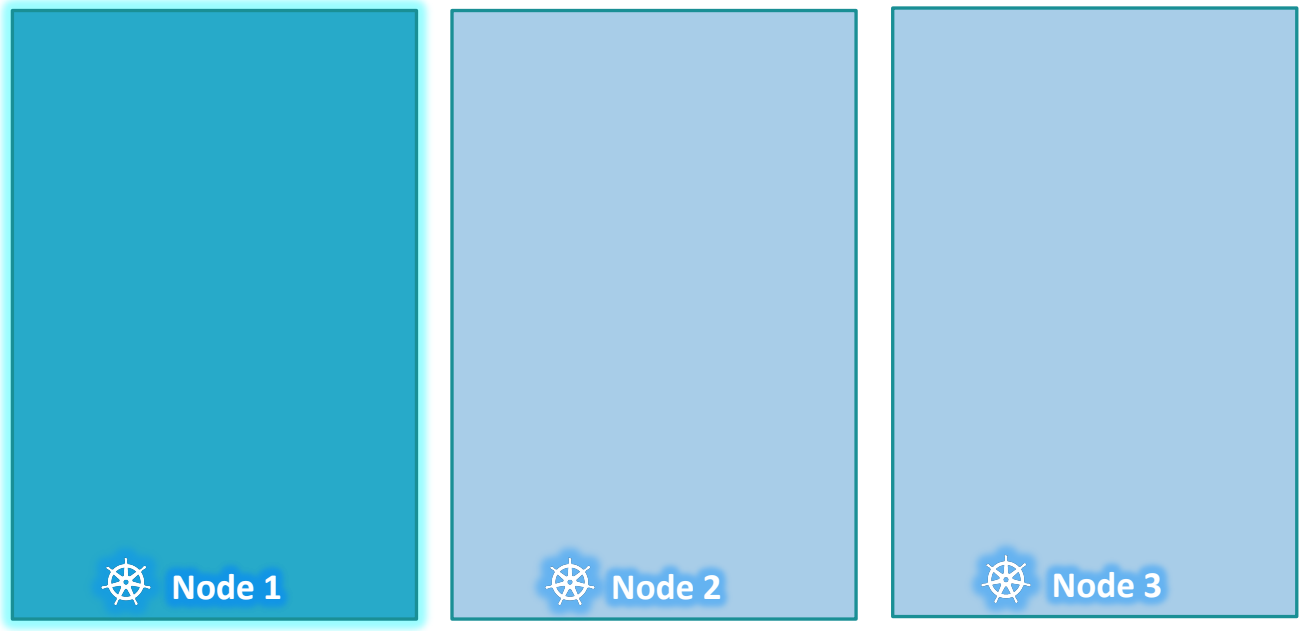
pod-definition.yml

```
apiVersion:
kind: Pod
metadata:
 name: myapp-pod
spec:
  containers:
  - name: nginx-container
    image: nginx

  tolerations:
  - key:"    "
    operator:"Equal"
    value:"    "
    effect:"          "
```

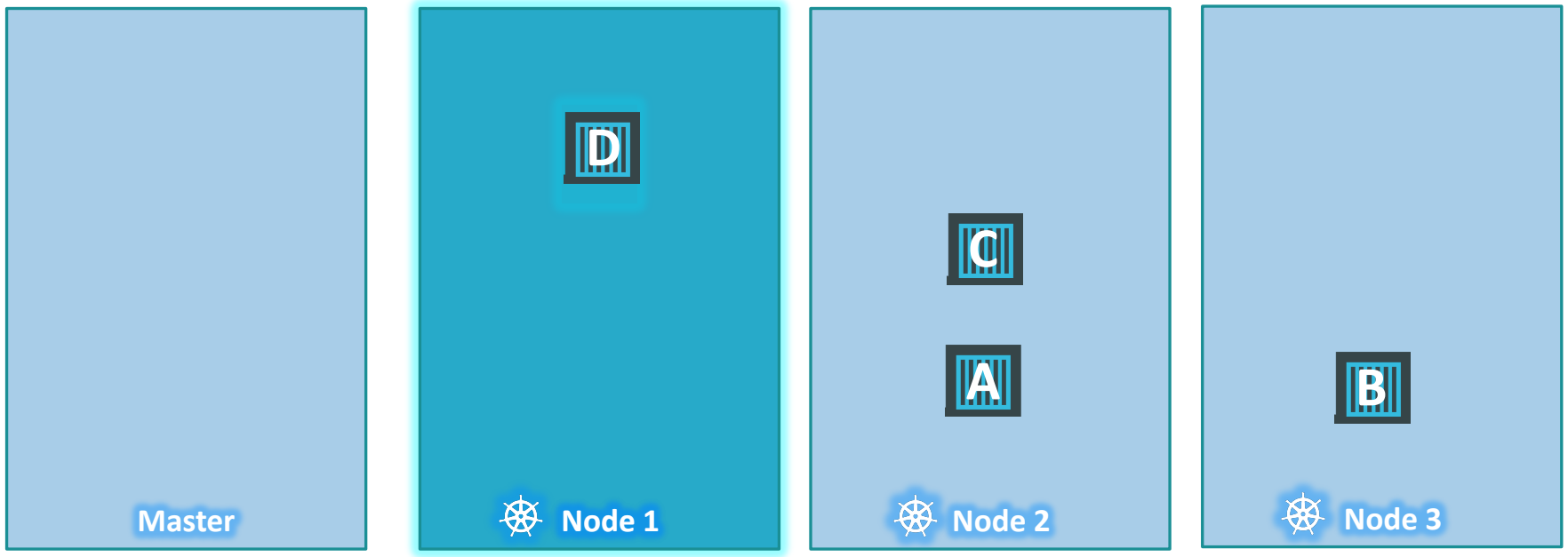
Taint - NoExecute



Taint=blue



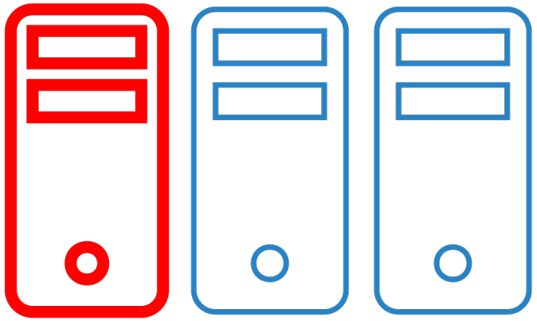
```
kubectl describe node kubemaster | grep Taint
Taints:             node-role.kubernetes.io/master:NoSchedule
```

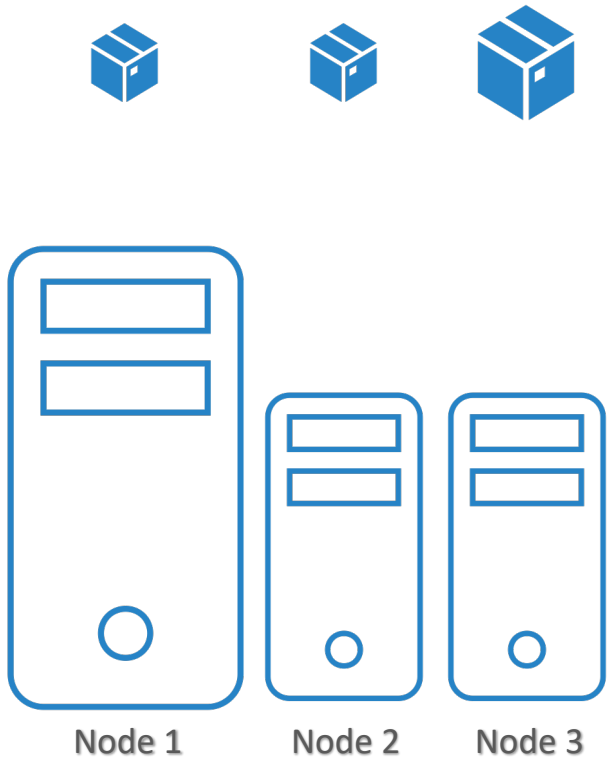


Demo

Taints and Tolerations

Node Selectors

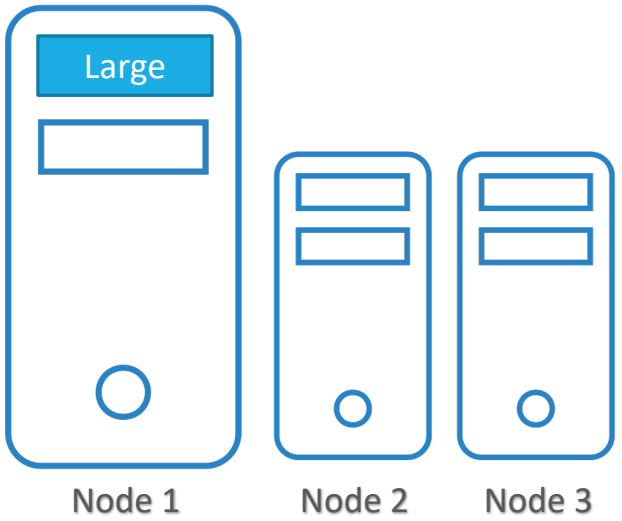



Node 1
Node 2
Node 3

Node Selectors

pod-definition.yml

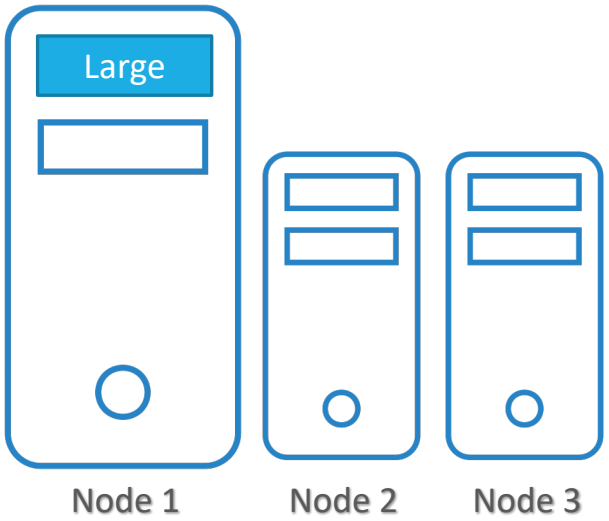
```
apiVersion:
kind: Pod
metadata:
  name: myapp-pod
spec:
  containers:
  - name: data-processor
    image: data-processor

  nodeSelector:
    size: Large
```



Label Nodes

```
kubectl label nodes <node-name> <label-key>=<label-value>
```

```
kubectl label nodes node-1 size=Large
```



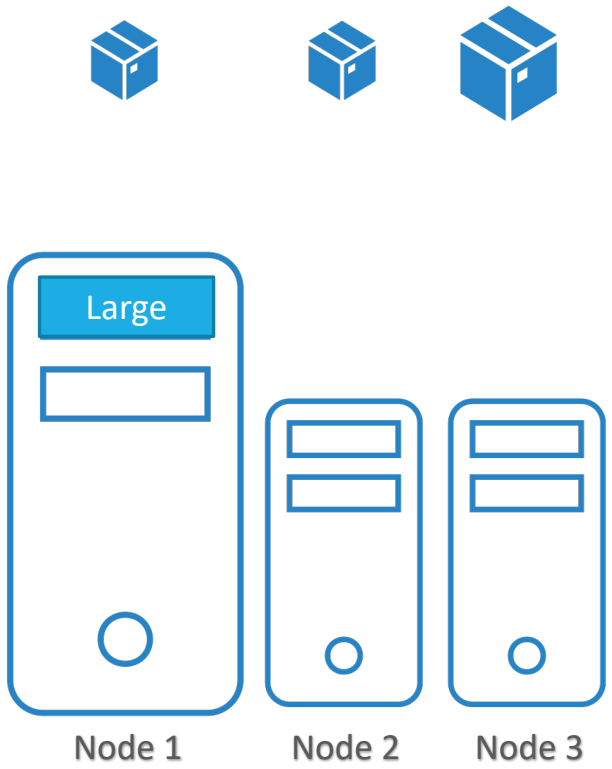
Node Selector

pod-definition.yml

```yaml
apiVersion:
kind: Pod
metadata:
 name: myapp-pod
spec:
  containers:
  - name: data-processor
    image: data-processor

  nodeSelector:
    size: Large
```

```
kubectl create -f pod-definition.yml
```

Node Selector

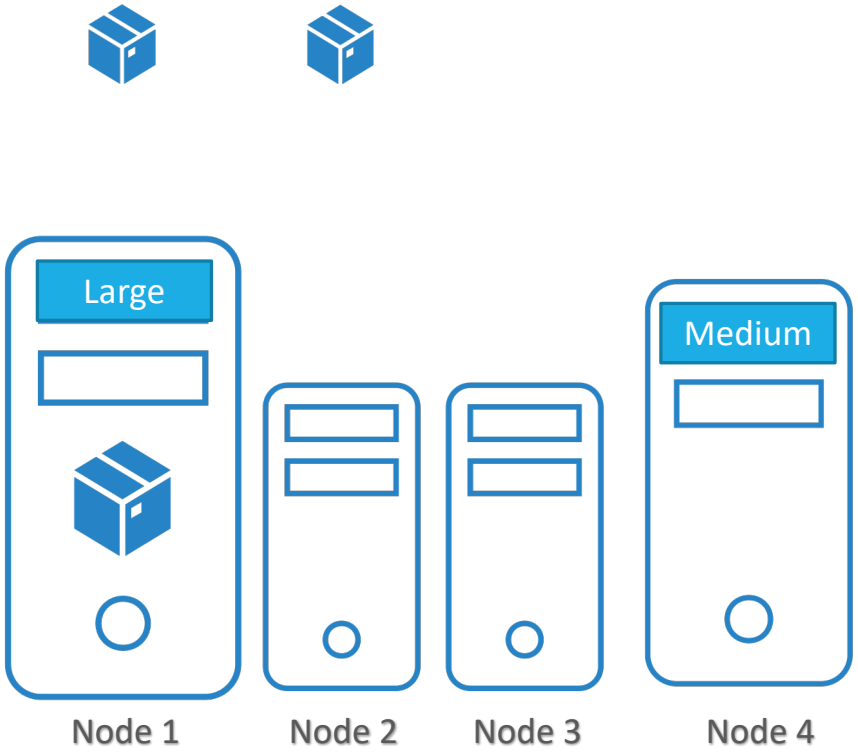


pod-definition.yml

```yaml
apiVersion:
kind: Pod
metadata:
  name: myapp-pod
spec:
  containers:
  - name: data-processor
    image: data-processor

  nodeSelector:
    size: Large
```

```
kubectl create -f pod-definition.yml
```

Node Selector - Limitations

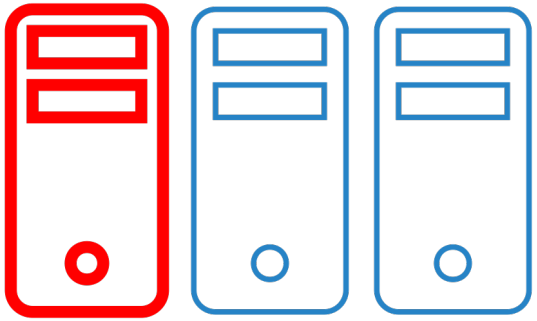


- Large OR Medium?
- NOT Small

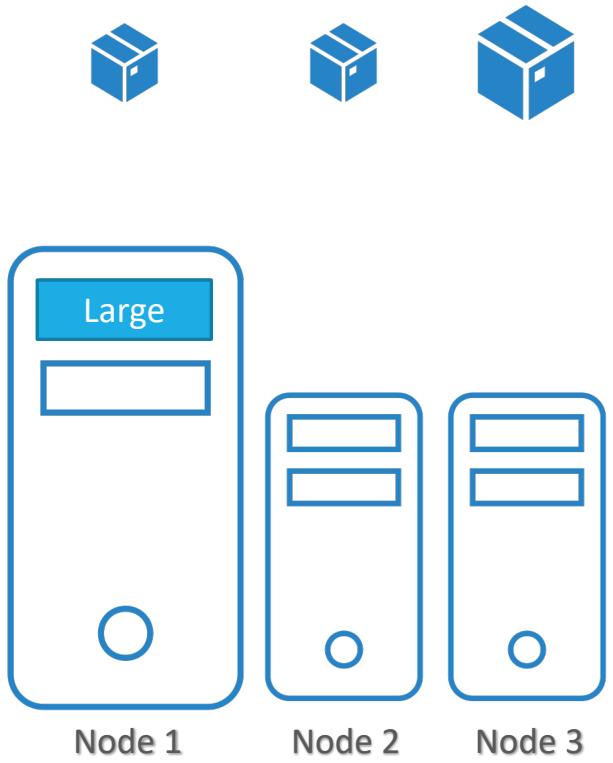
Demo

Taints and Tolerations

Node Affinity



PODs to Nodes



Node Selectors

pod-definition.yml

```
apiVersion:
kind: Pod
metadata:
 name: myapp-pod
spec:
  containers:
  - name: data-processor
    image: data-processor

  nodeSelector:
    size: Large
```

- Large OR Medium?
- NOT Small

Node Affinity

pod-definition.yml

```yaml
apiVersion:
kind: Pod

metadata:
 name: myapp-pod
spec:

  containers:
  - name: data-processor
    image: data-processor

  nodeSelector:
    size: Large
```

pod-definition.yml

```yaml
apiVersion:
kind:

metadata:
 name: myapp-pod
spec:

 containers:
  - name: data-processor
    image: data-processor

 affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
      - matchExpressions:
        - key: size
          operator: In
          values:
          - Large
```

Node Affinity

pod-definition.yml

```yaml
apiVersion:
kind:

metadata:
 name: myapp-pod
spec:

 containers:
  - name: data-processor
    image: data-processor
 affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
      - matchExpressions:

        - key: size
          operator: Exists
```

Node Affinity Types

Available:

`requiredDuringSchedulingIgnoredDuringExecution`

`preferredDuringSchedulingIgnoredDuringExecution`

Planned:

`requiredDuringSchedulingRequiredDuringExecution`

Node Affinity Types

Available:

requiredDuringSchedulingIgnoredDuringExecution

preferredDuringSchedulingIgnoredDuringExecution

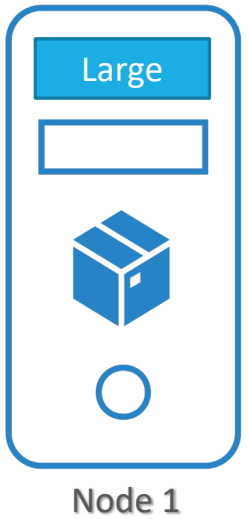
	DuringScheduling	DuringExecution
Type 1	Required	Ignored
Type 2	Preferred	Ignored

Node Affinity Types

Planned:

requiredDuringSchedulingRequiredDuringExecution

	DuringScheduling	DuringExecution
Type 1	Required	Ignored
Type 2	Preferred	Ignored
Type 3	Required	Required



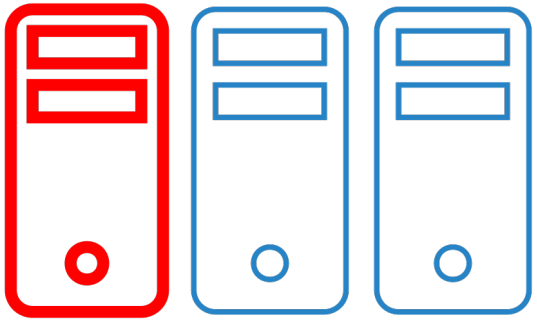
Demo

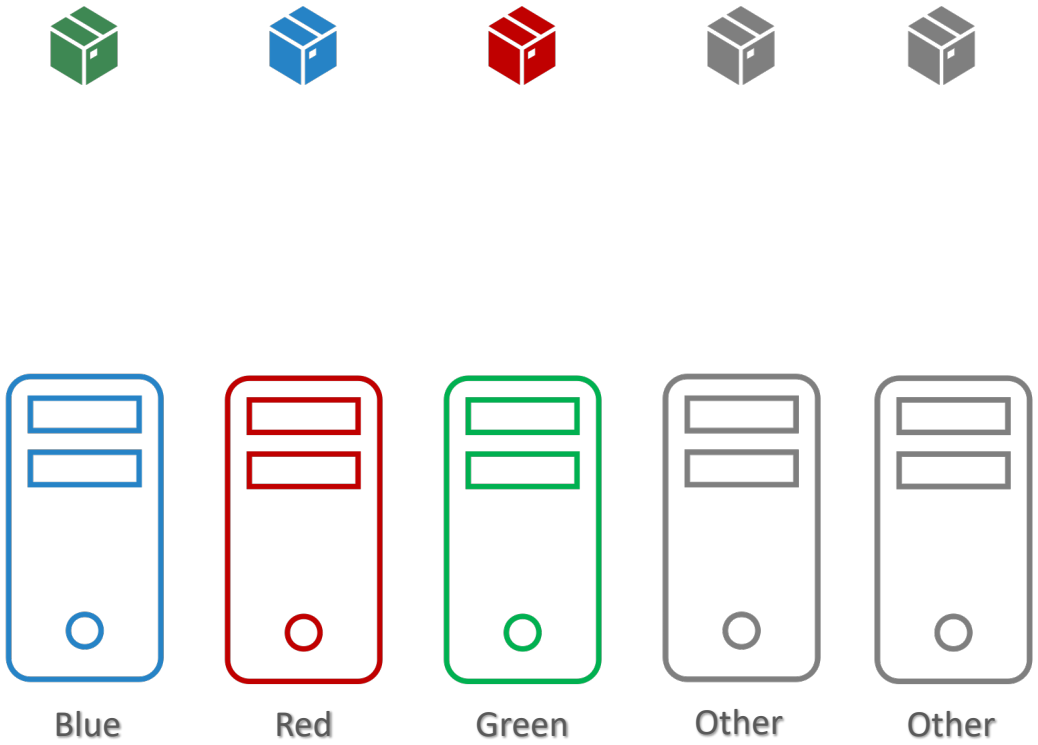
Taints and Tolerations

References

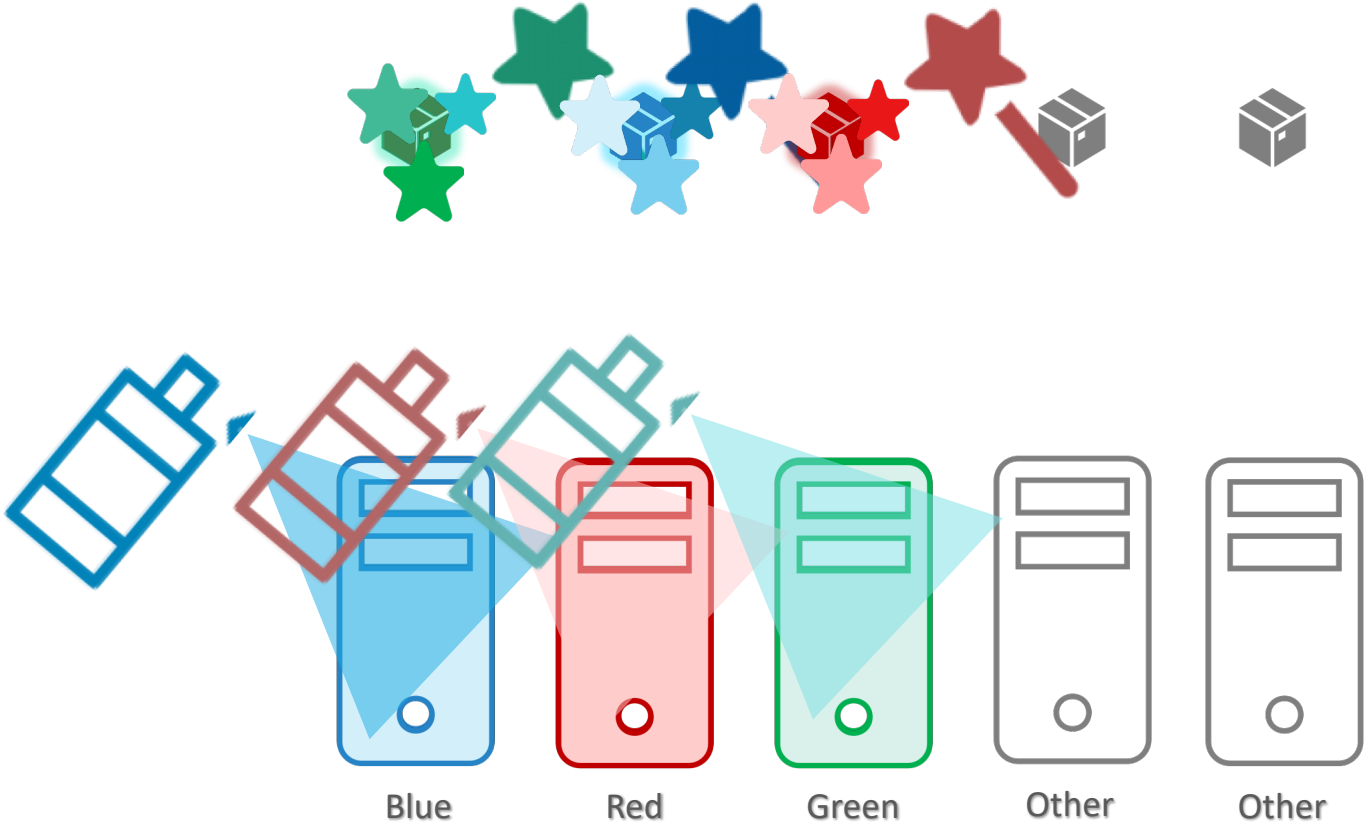
https://kubernetes.io/docs/concepts/configuration/assign-pod-node/

Node Affinity vs Taints and Tolerations

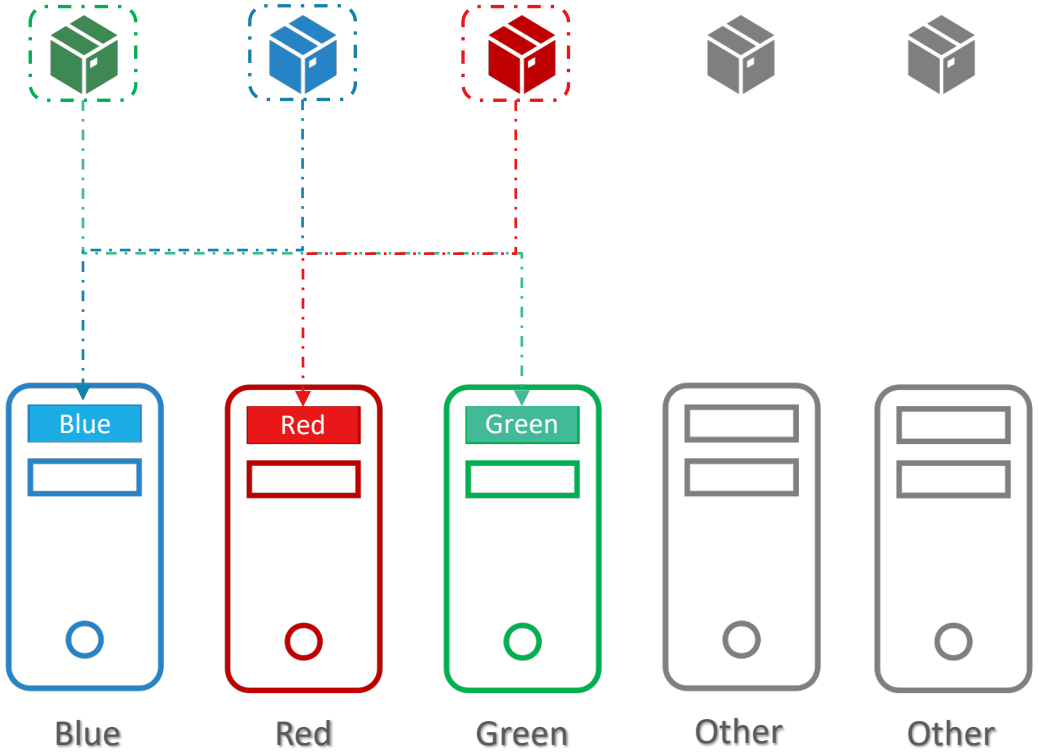



Blue
Red
Green
Other
Other

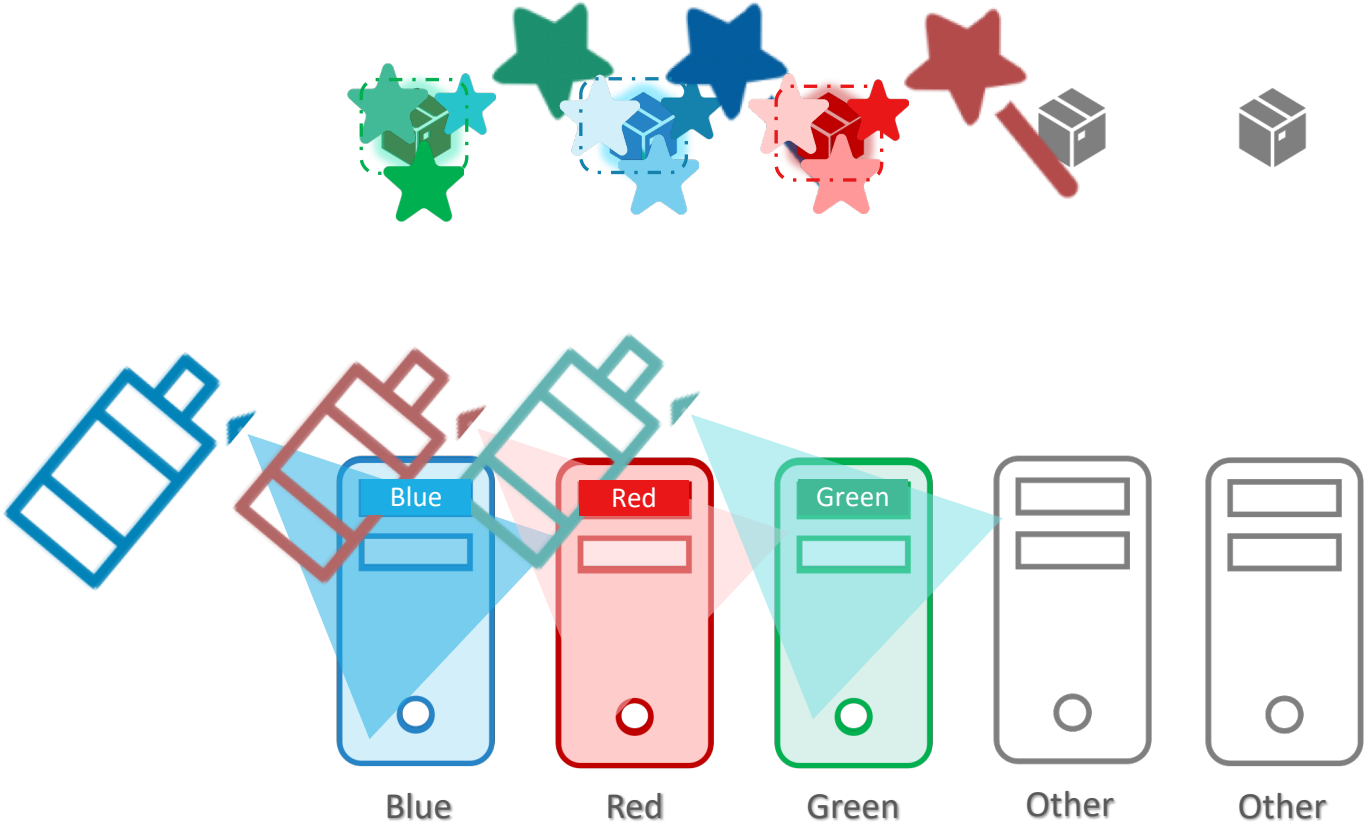
Taints and Tolerations



Node Affinity

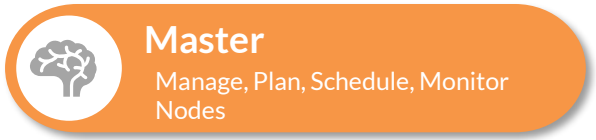


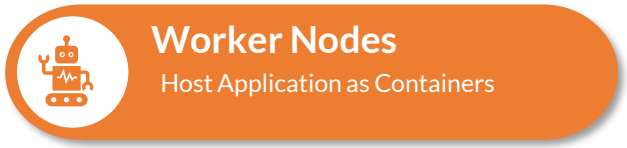
Taints/Tolerations and Node Affinity

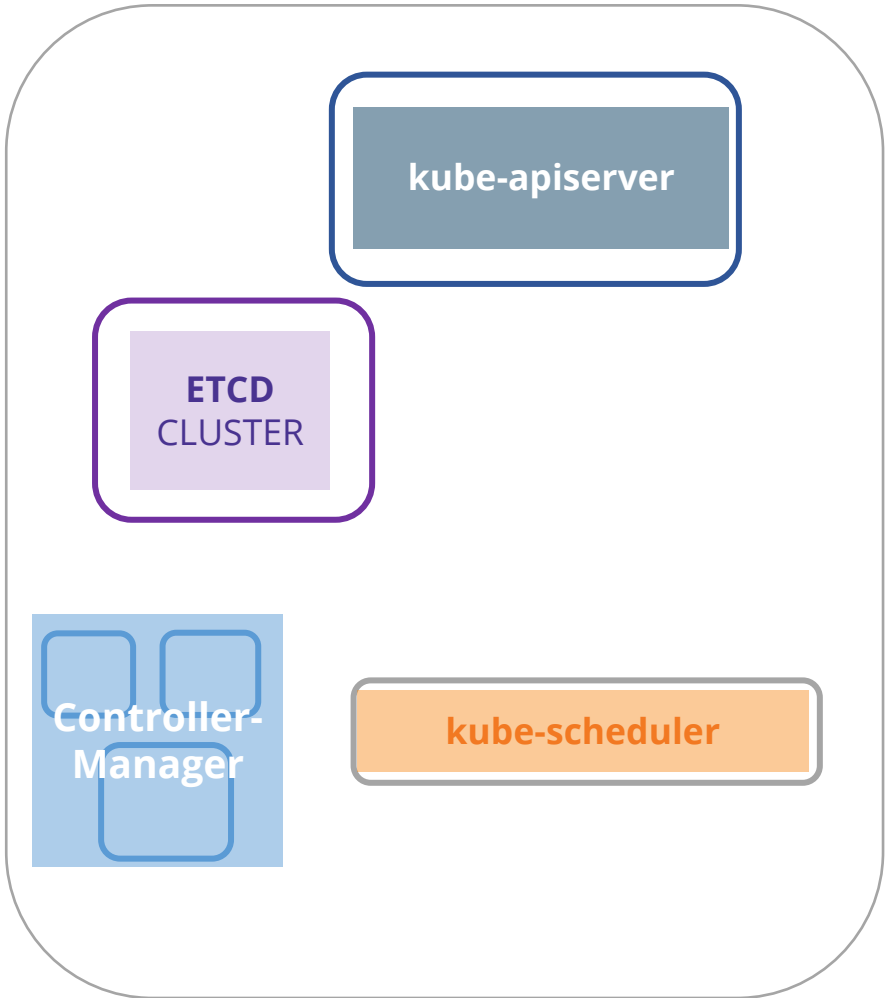


Static PODs

Kubernetes Architecture

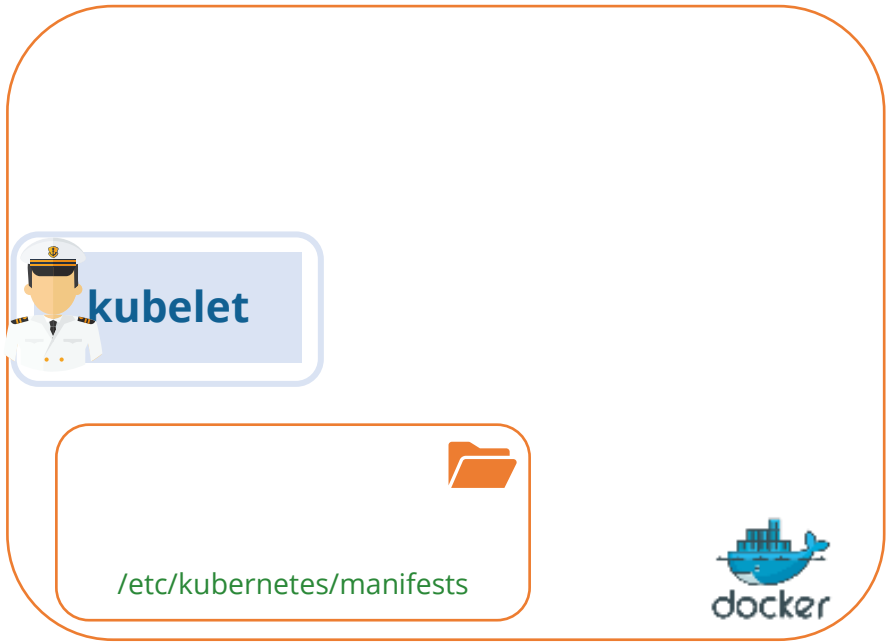






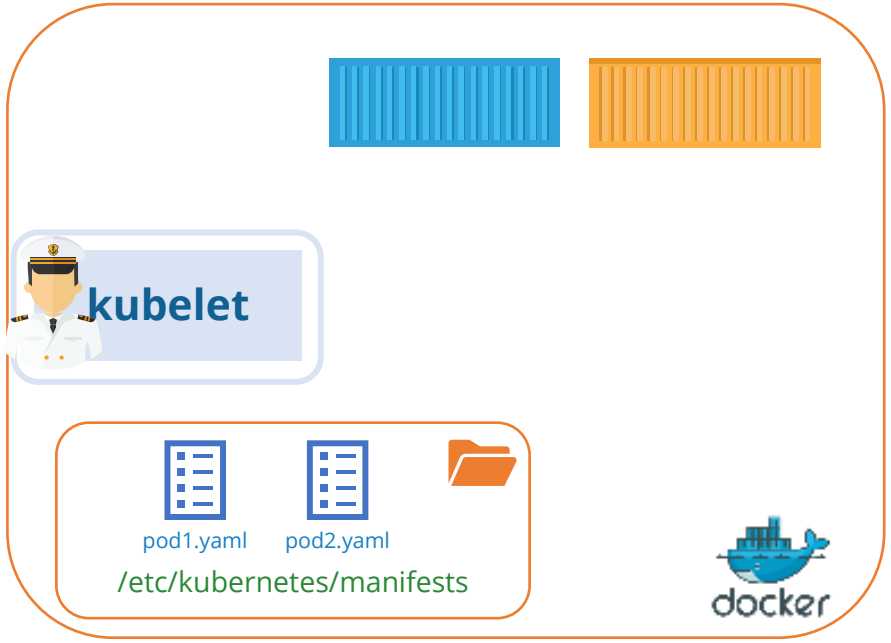
KodeKloud

pod1.yaml
pod2.yaml


kubelet
/etc/kubernetes/manifests
docker

KodeKloud

Static PODs

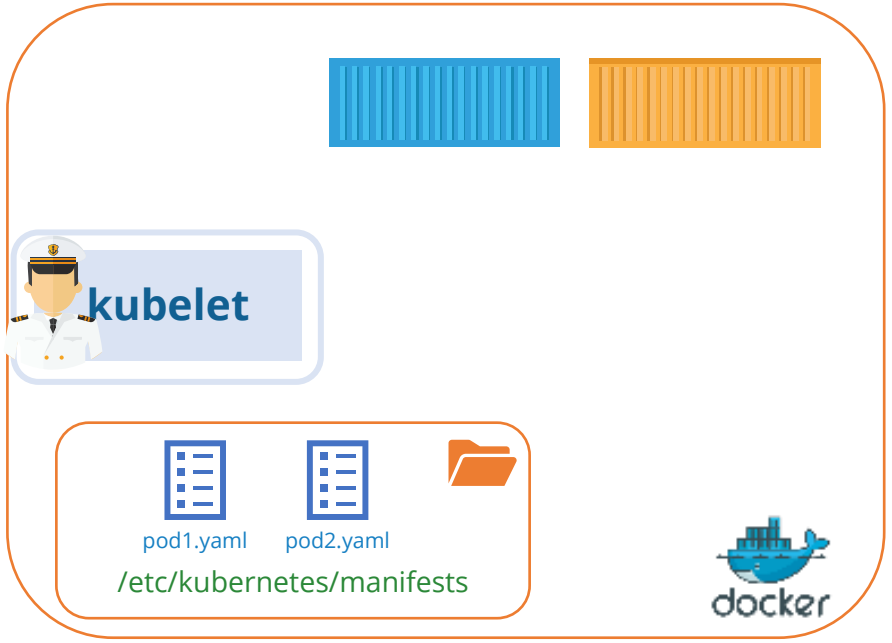


kubelet.service

```
ExecStart=/usr/local/bin/kubelet \\
  --container-runtime=remote \\
  --container-runtime-endpoint=unix:///var/run/containerd/containerd.sock \\
  --pod-manifest-path=/etc/Kubernetes/manifests \\
  --kubeconfig=/var/lib/kubelet/kubeconfig \\
  --network-plugin=cni \\
  --register-node=true \\
  --v=2
```

KodeKloud

Static PODs



kubelet.service

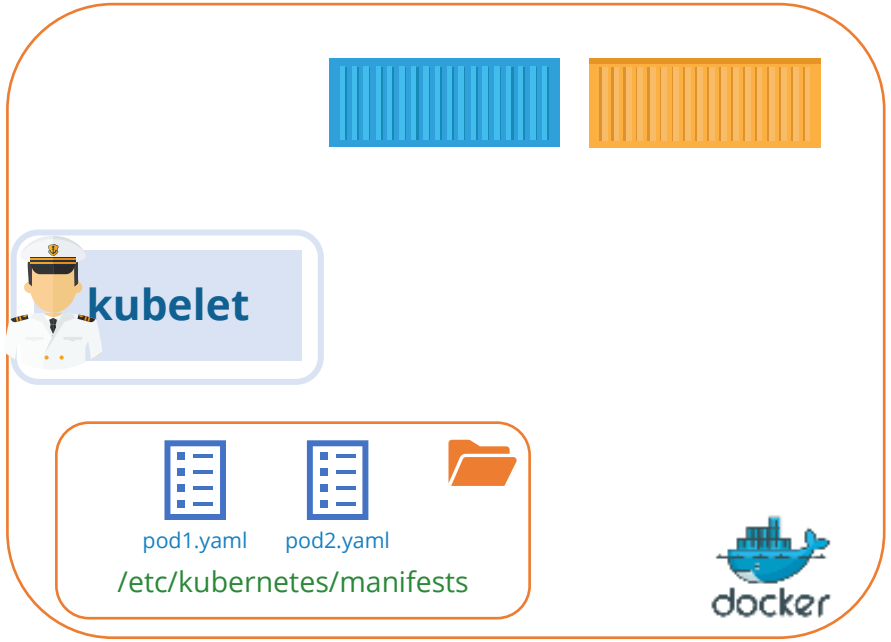
```
ExecStart=/usr/local/bin/kubelet \\
  --container-runtime=remote \\
  --container-runtime-endpoint=unix:///var/run/containerd/containerd.sock \\
  --config=kubeconfig.yaml \\
  --kubeconfig=/var/lib/kubelet/kubeconfig \\
  --network-plugin=cni \\
  --register-node=true \\
  --v=2
```

kubeconfig.yaml

```
staticPodPath: /etc/kubernetes/manifest
```

KodeKloud

Static PODs

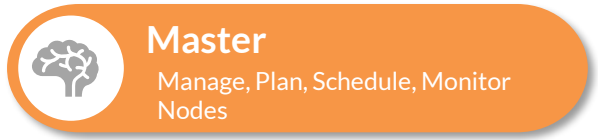


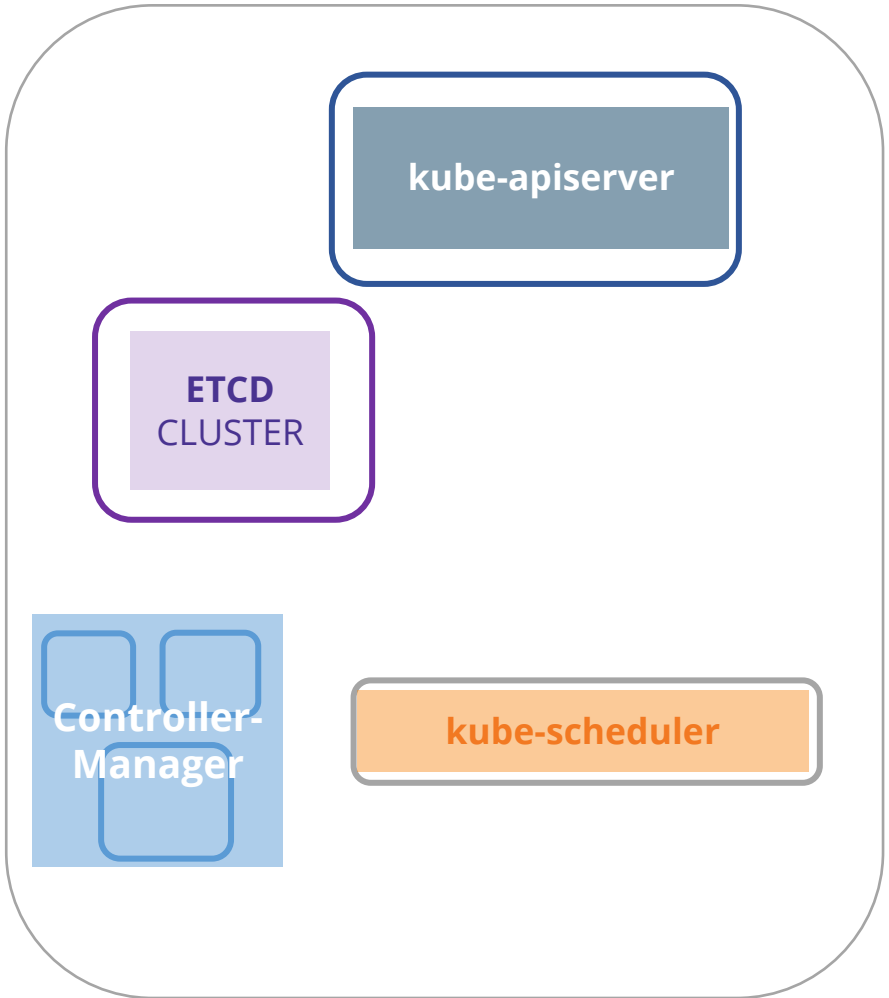

```
docker ps
```

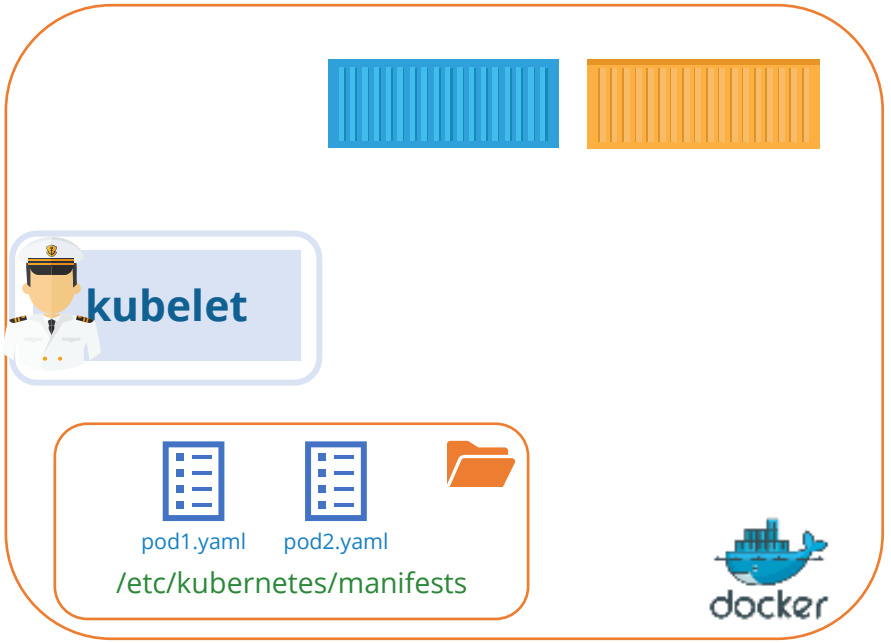
```
CONTAINER ID        IMAGE                  COMMAND                  CREATED             STATUS
PORTS               NAMES
8e5d4c4db7b6        busybox                "sh -c 'echo Hello K…"   20 seconds ago      Up 20 seconds
k8s_myapp-container_myapp-pod-host01_default_48e37fb432f2e06350e76786bd0bac66_0
f6737e1149cb        k8s.gcr.io/pause:3.1   "/pause"                 24 seconds ago      Up 23 seconds
k8s_POD_myapp-pod-host01_default_48e37fb432f2e06350e76786bd0bac66_0
```

KodeKloud

Static PODs



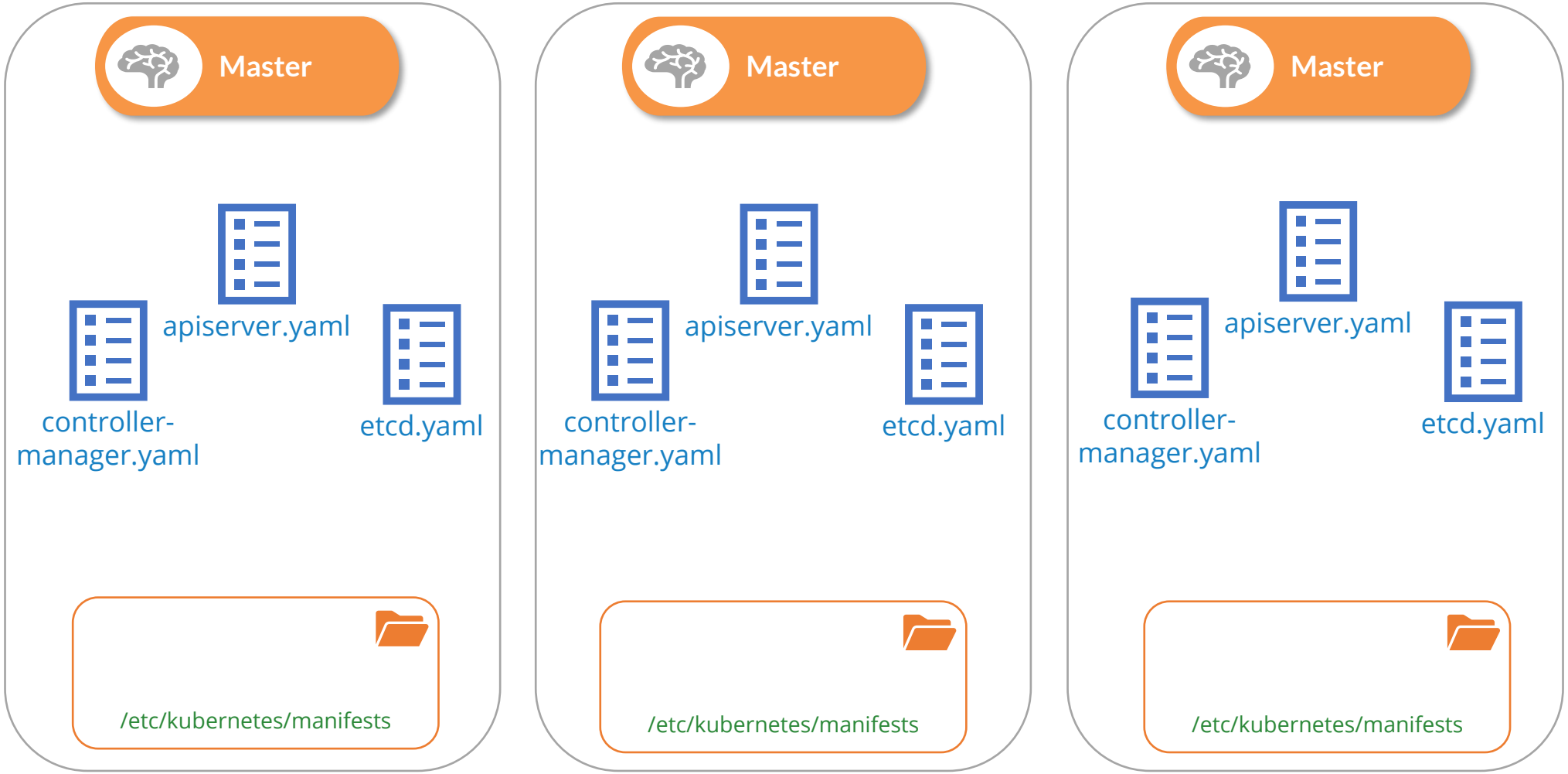





```
kubectl get pods
NAME                READY   STATUS              RESTARTS   AGE
static-web-node01   0/1     ContainerCreating   0          29s
```

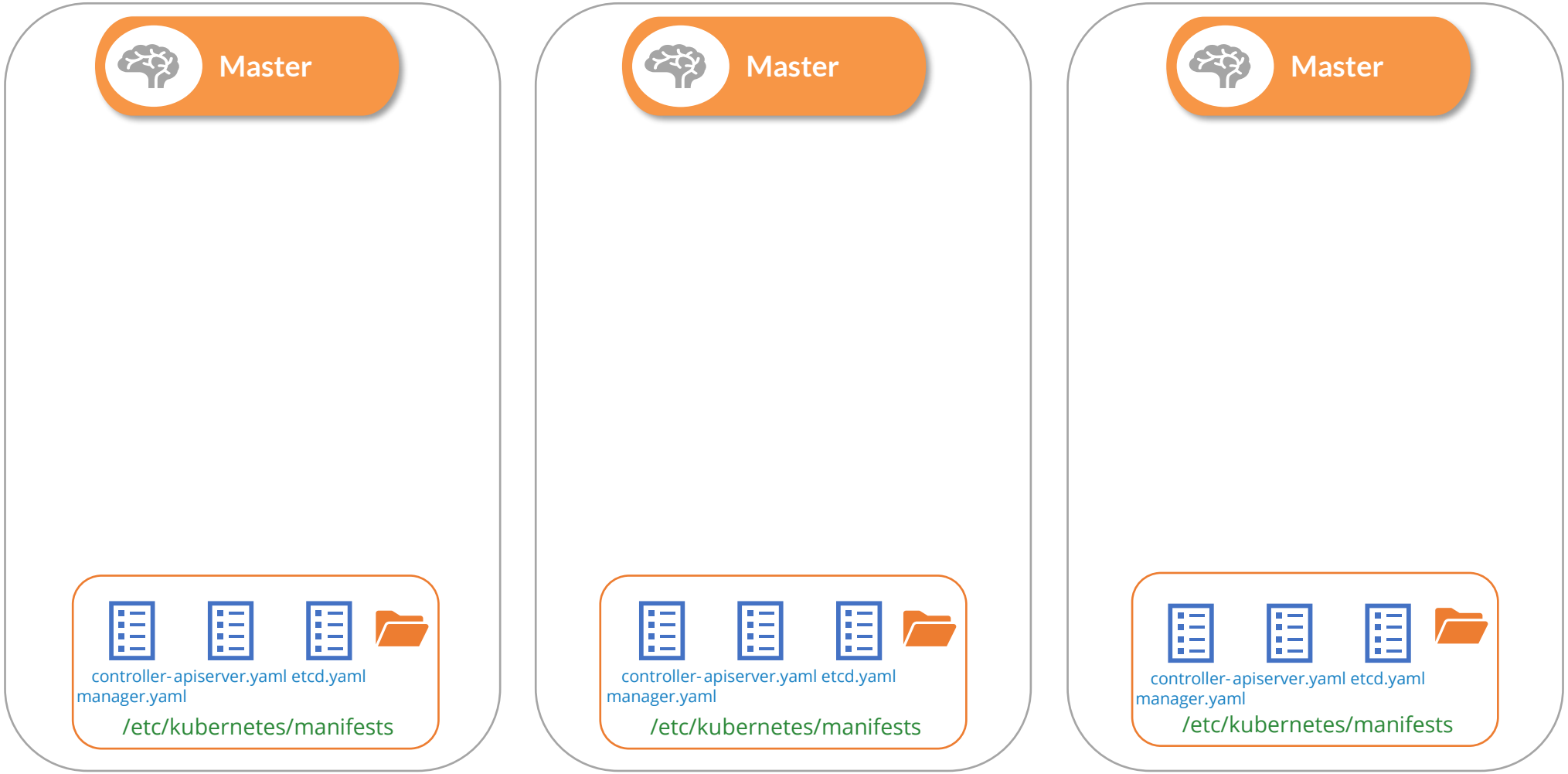
KodeKloud

Use Case



KodeKloud

Use Case



KodeKloud

Use Case

```
kubectl get pods -n kube-system
```

```
NAMESPACE     NAME                                READY     STATUS    RESTARTS   AGE
kube-system   coredns-78fcdf6894-hwrq9            1/1       Running   0          16m
kube-system   coredns-78fcdf6894-rzhjr            1/1       Running   0          16m
kube-system   etcd-master                         1/1       Running   0          15m
kube-system   kube-apiserver-master               1/1       Running   0          15m
kube-system   kube-controller-manager-master      1/1       Running   0          15m
kube-system   kube-proxy-lzt6f                    1/1       Running   0          16m
kube-system   kube-proxy-zm5qd                    1/1       Running   0          16m
kube-system   kube-scheduler-master               1/1       Running   0          15m
kube-system   weave-net-29z42                     2/2       Running   1          16m
kube-system   weave-net-snmdl                     2/2       Running   1          16m    -
```

KodeKloud

Static PODs vs DaemonSets

Static PODs	DaemonSets
Created by the Kubelet	Created by Kube-API server (DaemonSet Controller)
Deploy Control Plane components as Static Pods	Deploy Monitoring Agents, Logging Agents on nodes
Ignored by the Kube-Scheduler	

KodeKloud

KODEKLOUD

Container Logging

Logs - Docker

```
docker run kodekloud/event-simulator
```

```
2018-10-06 15:57:15,937 - root - INFO - USER1 logged in
2018-10-06 15:57:16,943 - root - INFO - USER2 logged out
2018-10-06 15:57:17,944 - root - INFO - USER2 is viewing page2
2018-10-06 15:57:18,951 - root - INFO - USER3 is viewing page3
2018-10-06 15:57:19,954 - root - INFO - USER4 is viewing page1
2018-10-06 15:57:20,955 - root - INFO - USER2 logged out
2018-10-06 15:57:21,956 - root - INFO - USER1 logged in
2018-10-06 15:57:22,957 - root - INFO - USER3 is viewing page2
2018-10-06 15:57:23,959 - root - INFO - USER1 logged out
2018-10-06 15:57:24,959 - root - INFO - USER2 is viewing page2
2018-10-06 15:57:25,961 - root - INFO - USER1 logged in
2018-10-06 15:57:26,965 - root - INFO - USER4 is viewing page3
2018-10-06 15:57:27,965 - root - INFO - USER4 is viewing page3
2018-10-06 15:57:28,967 - root - INFO - USER2 is viewing page1
2018-10-06 15:57:29,967 - root - INFO - USER3 logged out
2018-10-06 15:57:30,972 - root - INFO - USER1 is viewing page2
2018-10-06 15:57:31,972 - root - INFO - USER4 logged out
2018-10-06 15:57:32,973 - root - INFO - USER1 logged in
2018-10-06 15:57:33,974 - root - INFO - USER1 is viewing page3
```

Logs - Docker

```
docker run -d kodekloud/event-simulator
```

```
docker logs -f ecf
```

```
2018-10-06 15:57:15,937 - root - INFO - USER1 logged in
2018-10-06 15:57:16,943 - root - INFO - USER2 logged out
2018-10-06 15:57:17,944 - root - INFO - USER2 is viewing page2
2018-10-06 15:57:18,951 - root - INFO - USER3 is viewing page3
2018-10-06 15:57:19,954 - root - INFO - USER4 is viewing page1
2018-10-06 15:57:20,955 - root - INFO - USER2 logged out
2018-10-06 15:57:21,956 - root - INFO - USER1 logged in
2018-10-06 15:57:22,957 - root - INFO - USER3 is viewing page2
2018-10-06 15:57:23,959 - root - INFO - USER1 logged out
2018-10-06 15:57:24,959 - root - INFO - USER2 is viewing page2
2018-10-06 15:57:25,961 - root - INFO - USER1 logged in
2018-10-06 15:57:26,965 - root - INFO - USER4 is viewing page3
2018-10-06 15:57:27,965 - root - INFO - USER4 is viewing page3
2018-10-06 15:57:28,967 - root - INFO - USER2 is viewing page1
2018-10-06 15:57:29,967 - root - INFO - USER3 logged out
2018-10-06 15:57:30,972 - root - INFO - USER1 is viewing page2
2018-10-06 15:57:31,972 - root - INFO - USER4 logged out
2018-10-06 15:57:32,973 - root - INFO - USER1 logged in
2018-10-06 15:57:33,974 - root - INFO - USER1 is viewing page3
```

Logs - Kubernetes

```
kubectl create -f event-simulator.yaml
```

```
kubectl logs -f event-simulator-pod
```

```
2018-10-06 15:57:15,937 - root - INFO - USER1 logged in
2018-10-06 15:57:16,943 - root - INFO - USER2 logged out
2018-10-06 15:57:17,944 - root - INFO - USER2 is viewing page2
2018-10-06 15:57:18,951 - root - INFO - USER3 is viewing page3
2018-10-06 15:57:19,954 - root - INFO - USER4 is viewing page1
2018-10-06 15:57:20,955 - root - INFO - USER2 logged out
2018-10-06 15:57:21,956 - root - INFO - USER1 logged in
2018-10-06 15:57:22,957 - root - INFO - USER3 is viewing page2
2018-10-06 15:57:23,959 - root - INFO - USER1 logged out
2018-10-06 15:57:24,959 - root - INFO - USER2 is viewing page2
2018-10-06 15:57:25,961 - root - INFO - USER1 logged in
2018-10-06 15:57:26,965 - root - INFO - USER4 is viewing page3
2018-10-06 15:57:27,965 - root - INFO - USER4 is viewing page3
2018-10-06 15:57:28,967 - root - INFO - USER2 is viewing page1
2018-10-06 15:57:29,967 - root - INFO - USER3 logged out
2018-10-06 15:57:30,972 - root - INFO - USER1 is viewing page2
2018-10-06 15:57:31,972 - root - INFO - USER4 logged out
2018-10-06 15:57:32,973 - root - INFO - USER1 logged in
2018-10-06 15:57:33,974 - root - INFO - USER1 is viewing page3
```

event-simulator.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: event-simulator-pod
spec:
  containers:
  - name: event-simulator
    image: kodekloud/event-simulator
```

Logs - Kubernetes

```
kubectl logs -f event-simulator-pod event-simulator
```

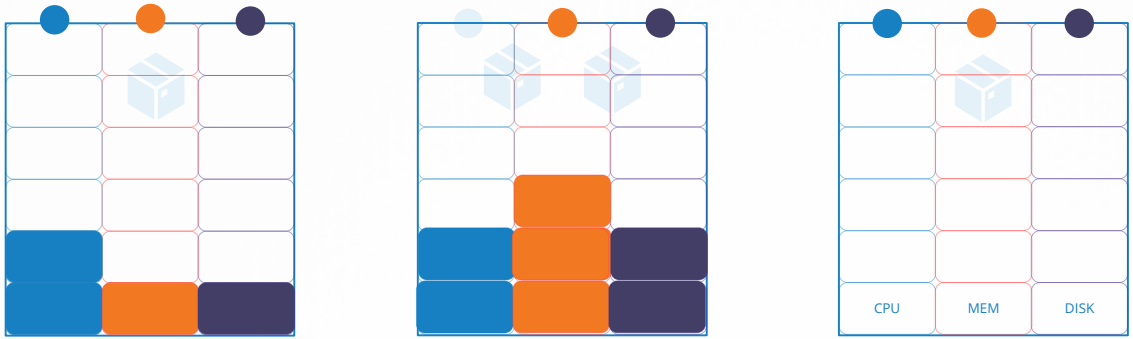
```
2018-10-06 15:57:15,937 - root - INFO - USER1 logged in
2018-10-06 15:57:16,943 - root - INFO - USER2 logged out
2018-10-06 15:57:17,944 - root - INFO - USER2 is viewing page2
2018-10-06 15:57:18,951 - root - INFO - USER3 is viewing page3
2018-10-06 15:57:19,954 - root - INFO - USER4 is viewing page1
2018-10-06 15:57:20,955 - root - INFO - USER2 logged out
2018-10-06 15:57:21,956 - root - INFO - USER1 logged in
2018-10-06 15:57:22,957 - root - INFO - USER3 is viewing page2
2018-10-06 15:57:23,959 - root - INFO - USER1 logged out
2018-10-06 15:57:24,959 - root - INFO - USER2 is viewing page2
2018-10-06 15:57:25,961 - root - INFO - USER1 logged in
2018-10-06 15:57:26,965 - root - INFO - USER4 is viewing page3
2018-10-06 15:57:27,965 - root - INFO - USER4 is viewing page3
2018-10-06 15:57:28,967 - root - INFO - USER2 is viewing page1
2018-10-06 15:57:29,967 - root - INFO - USER3 logged out
2018-10-06 15:57:30,972 - root - INFO - USER1 is viewing page2
2018-10-06 15:57:31,972 - root - INFO - USER4 logged out
2018-10-06 15:57:32,973 - root - INFO - USER1 logged in
2018-10-06 15:57:33,974 - root - INFO - USER1 is viewing page3
```

event-simulator.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: event-simulator-pod
spec:
  containers:
  - name: event-simulator
    image: kodekloud/event-simulator
  - name: image-processor
    image: some-image-processor
```

Monitoring Kubernetes

Monitor



Monitor

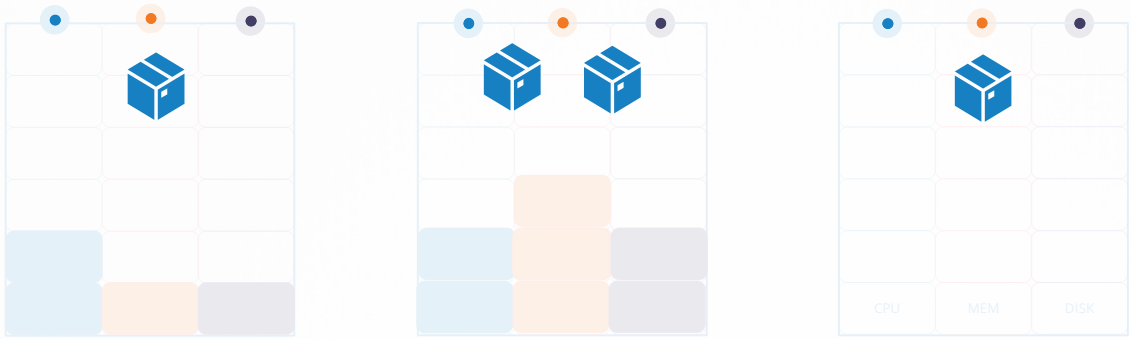
METRICS SERVER

Prometheus

Elastic Stack

DATADOG

dynatrace



Monitor

METRICS SERVER

Prometheus

Elastic Stack

DATADOG

dynatrace

Monitor

METRICS SERVER

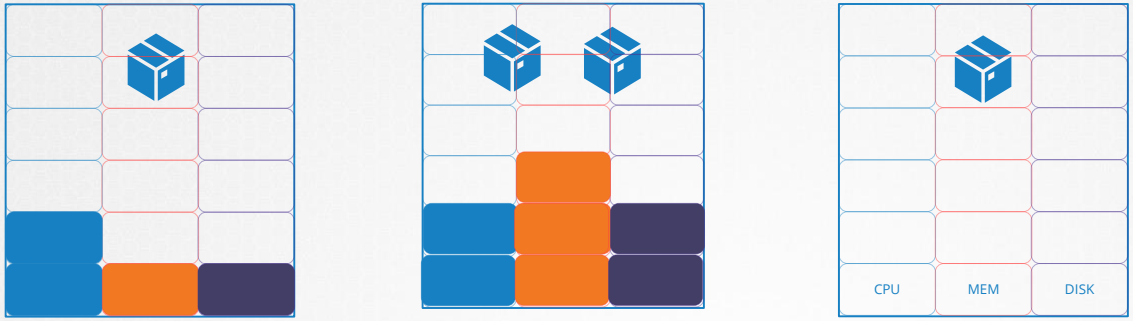
Prometheus

Elastic Stack

Heapster vs Metrics Server

Metrics Server

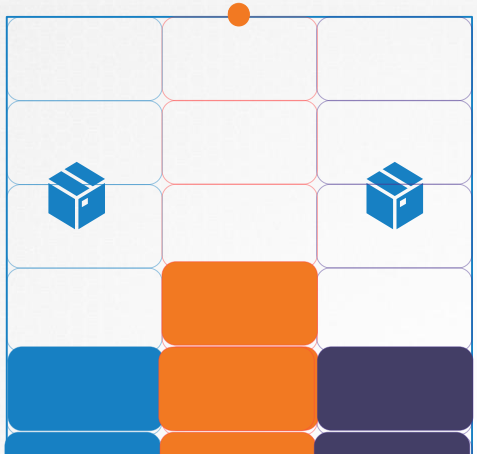
METRICS SERVER



Metrics Server

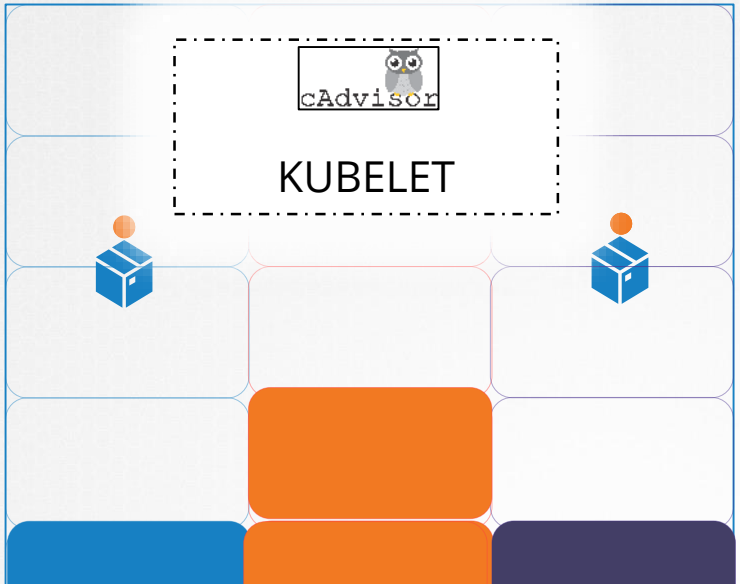
METRICS SERVER

IN-MEMORY



Metrics Server

METRICS SERVER



Metrics Server – Getting Started

```
minikube addons enable metrics-server
```

others

```
git clone https://github.com/kubernetes-incubator/metrics-server.git
```

```
kubectl create –f deploy/1.8+/
clusterrolebinding "metrics-server:system:auth-delegator" created
rolebinding "metrics-server-auth-reader" created
apiservice "v1beta1.metrics.k8s.io" created
serviceaccount "metrics-server" created
deployment "metrics-server" created
service "metrics-server" created
clusterrole "system:metrics-server" created
clusterrolebinding "system:metrics-server" created
```

View

```
kubectl top node
```

```
NAME         CPU(cores)   CPU%      MEMORY(bytes)   MEMORY%
kubemaster   166m         8%        1337Mi          70%
kubenode1    36m          1%        1046Mi          55%
kubenode2    39m          1%        1048Mi          55%
```

```
kubectl top pod
```

```
NAME         CPU(cores)   CPU%      MEMORY(bytes)   MEMORY%
nginx        166m         8%        1337Mi          70%
redis        36m          1%        1046Mi          55%
```

Notes

- Play-with-k8s is on version 1.8, so it still requires heapster. Does not work with metrics-server.

References

- https://kubernetes.io/docs/tasks/debug-application-cluster/core-metrics-pipeline/
- https://kubernetes.io/docs/tasks/debug-application-cluster/resource-usage-monitoring/
- https://kubernetes.io/docs/tasks/administer-cluster/manage-resources/

KODEKLOUD

Rolling Updates & Rollbacks

Rollout and Versioning

Revision 1

nginx:1.7.0

nginx:1.7.0

nginx:1.7.0

nginx:1.7.0

nginx:1.7.0

nginx:1.7.0

nginx:1.7.0

nginx:1.7.0

nginx:1.7.0

Revision 2

nginx:1.7.1

nginx:1.7.1

nginx:1.7.1

nginx:1.7.1

nginx:1.7.1

nginx:1.7.1

nginx:1.7.1

nginx:1.7.1

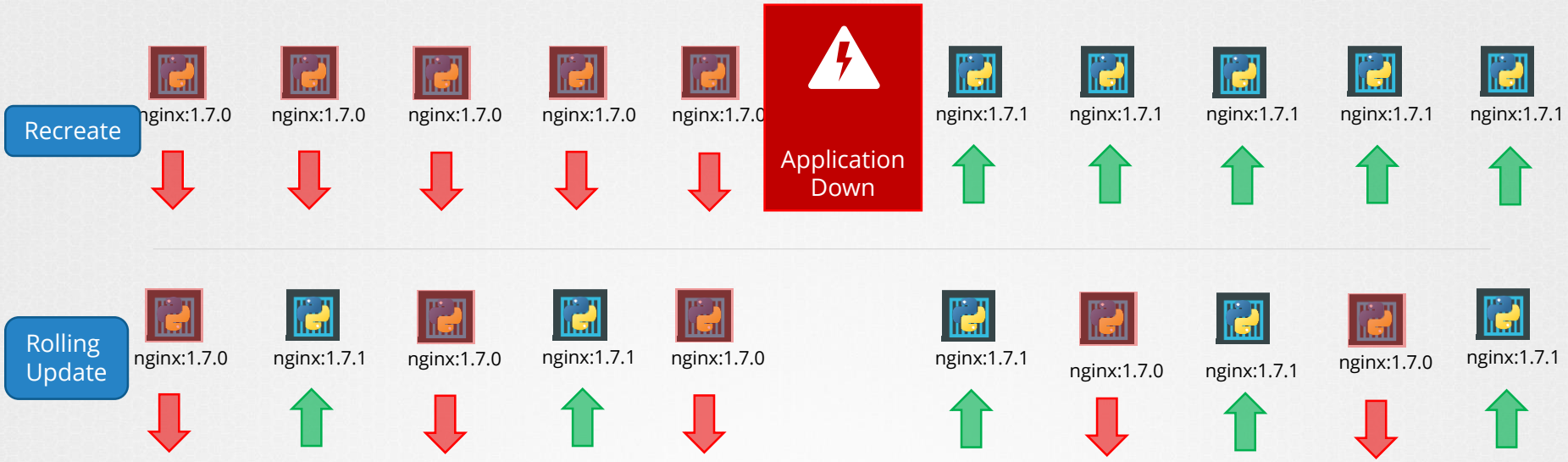
nginx:1.7.1

Rollout Command

```
> kubectl rollout status deployment/myapp-deployment
Waiting for rollout to finish: 0 of 10 updated replicas are available...
Waiting for rollout to finish: 1 of 10 updated replicas are available...
Waiting for rollout to finish: 2 of 10 updated replicas are available...
Waiting for rollout to finish: 3 of 10 updated replicas are available...
Waiting for rollout to finish: 4 of 10 updated replicas are available...
Waiting for rollout to finish: 5 of 10 updated replicas are available...
Waiting for rollout to finish: 6 of 10 updated replicas are available...
Waiting for rollout to finish: 7 of 10 updated replicas are available...
Waiting for rollout to finish: 8 of 10 updated replicas are available...
Waiting for rollout to finish: 9 of 10 updated replicas are available...
deployment "myapp-deployment" successfully rolled out
```

```
> kubectl rollout history deployment/myapp-deployment
deployments "myapp-deployment"
REVISION  CHANGE-CAUSE
1         <none>
2         kubectl apply --filename=deployment-definition.yml --record=true
```

Deployment Strategy



Kubectl apply

```
> kubectl apply -f deployment-definition.yml
deployment "myapp-deployment" configured
```

```
> kubectl set image deployment/myapp-deployment \
                    nginx=nginx:1.9.1
deployment "myapp-deployment" image is updated
```

deployment-definition.yml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-deployment
  labels:
      app: myapp
      type: front-end
spec:
  template:
    metadata:
      name: myapp-pod
      labels:
         app: myapp
         type: front-end
    spec:
      containers:
      - name: nginx-container
        image: nginx:1.7.1
  replicas: 3
  selector:
    matchLabels:
       type: front-end
```

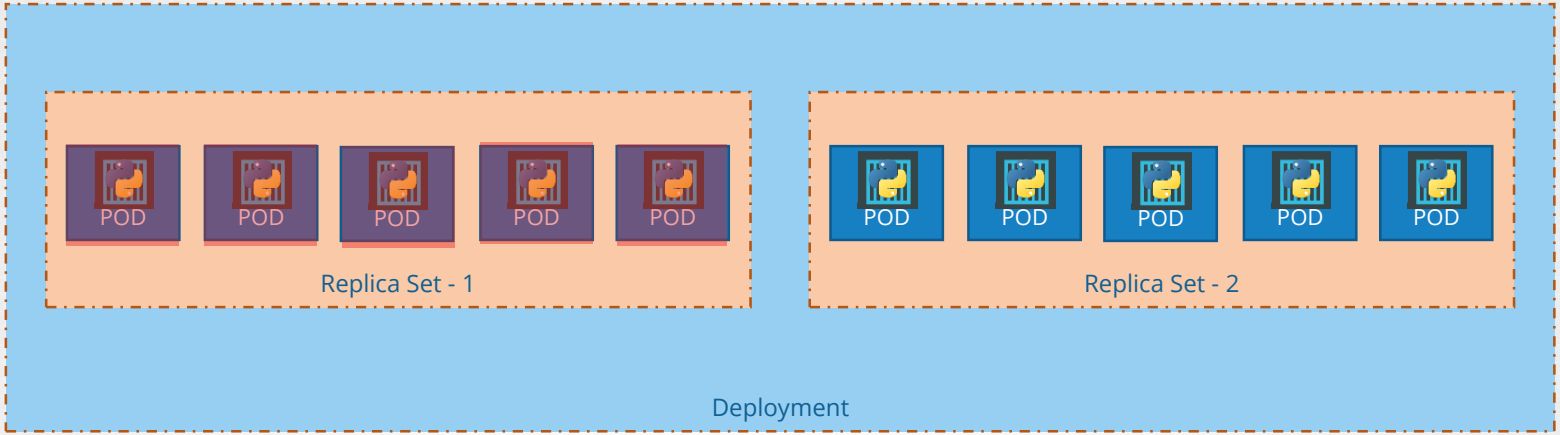
```
C:\Kubernetes>kubectl describe deployment myapp-deployment
Name:                   myapp-deployment
Namespace:              default
CreationTimestamp:      Sat, 03 Mar 2018 17:01:55 +0800
Labels:                 app=myapp
                        type=front-end
Annotations:            deployment.kubernetes.io/revision=2
                        kubectl.kubernetes.io/last-applied-configuration={"apiVersion":"apps/v1","kind":"Deployment","me
s\\Google...
                        kubernetes.io/change-cause=kubectl apply --filename=d:\Mumshad Files\Google Drive\Udemy\Kubernet
Selector:               type=front-end
Replicas:               5 desired | 5 updated | 5 total | 5 available | 0 unavailable
StrategyType:           Recreate
MinReadySeconds:        0
Pod Template:
  Labels:  app=myapp
           type=front-end
  Containers:
   nginx-container:
    Image:        nginx:1.7.1
    Port:         <none>
    Environment:  <none>
    Mounts:       <none>
  Volumes:        <none>
Conditions:
  Type           Status  Reason
  ----           ------  ------
  Available      True    MinimumReplicasAvailable
  Progressing    True    NewReplicaSetAvailable
OldReplicaSets:  <none>
NewReplicaSet:   myapp-deployment-54c7d6ccc (5/5 replicas created)
Events:
  Type    Reason             Age   From                   Message
  ----    ------             ----  ----                   -------
  Normal  ScalingReplicaSet  11m   deployment-controller  Scaled up replica set myapp-deployment-6795844b58 to 5
  Normal  ScalingReplicaSet  1m    deployment-controller  Scaled down replica set myapp-deployment-6795844b58 to 0
  Normal  ScalingReplicaSet  56s   deployment-controller  Scaled up replica set myapp-deployment-54c7d6ccc to 5
```

Recreate

```
C:\Kubernetes>kubectl describe deployment myapp-deployment
Name:                   myapp-deployment
Namespace:              default
CreationTimestamp:      Sat, 03 Mar 2018 17:16:53 +0800
Labels:                 app=myapp
                        type=front-end
Annotations:            deployment.kubernetes.io/revision=2
                        kubectl.kubernetes.io/last-applied-configuration={"apiVersion":"apps/v1","kind":"Deployment","metadata
Files\\Google...
                        kubernetes.io/change-cause=kubectl apply --filename=d:\Mumshad Files\Google Drive\Udemy\Kubernetes\Dem
Selector:               type=front-end
Replicas:               5 desired | 5 updated | 6 total | 4 available | 2 unavailable
StrategyType:           RollingUpdate
MinReadySeconds:        0
RollingUpdateStrategy:  25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=myapp
           type=front-end
  Containers:
   nginx-container:
    Image:        nginx
    Port:         <none>
    Environment:  <none>
    Mounts:       <none>
  Volumes:        <none>
Conditions:
  Type           Status  Reason
  ----           ------  ------
  Available      True    MinimumReplicasAvailable
  Progressing    True    ReplicaSetUpdated
OldReplicaSets:  myapp-deployment-67c749c58c (1/1 replicas created)
NewReplicaSet:   myapp-deployment-7d57dbdb8d (5/5 replicas created)
Events:
  Type    Reason             Age   From                   Message
  ----    ------             ----  ----                   -------
  Normal  ScalingReplicaSet  1m    deployment-controller  Scaled up replica set myapp-deployment-67c749c58c to 5
  Normal  ScalingReplicaSet  1s    deployment-controller  Scaled up replica set myapp-deployment-7d57dbdb8d to 2
  Normal  ScalingReplicaSet  1s    deployment-controller  Scaled down replica set myapp-deployment-67c749c58c to 4
  Normal  ScalingReplicaSet  1s    deployment-controller  Scaled up replica set myapp-deployment-7d57dbdb8d to 3
  Normal  ScalingReplicaSet  0s    deployment-controller  Scaled down replica set myapp-deployment-67c749c58c to 3
  Normal  ScalingReplicaSet  0s    deployment-controller  Scaled up replica set myapp-deployment-7d57dbdb8d to 4
  Normal  ScalingReplicaSet  0s    deployment-controller  Scaled down replica set myapp-deployment-67c749c58c to 2
  Normal  ScalingReplicaSet  0s    deployment-controller  Scaled up replica set myapp-deployment-7d57dbdb8d to 5
  Normal  ScalingReplicaSet  0s    deployment-controller  Scaled down replica set myapp-deployment-67c749c58c to 1
```

RollingUpdate

Upgrades

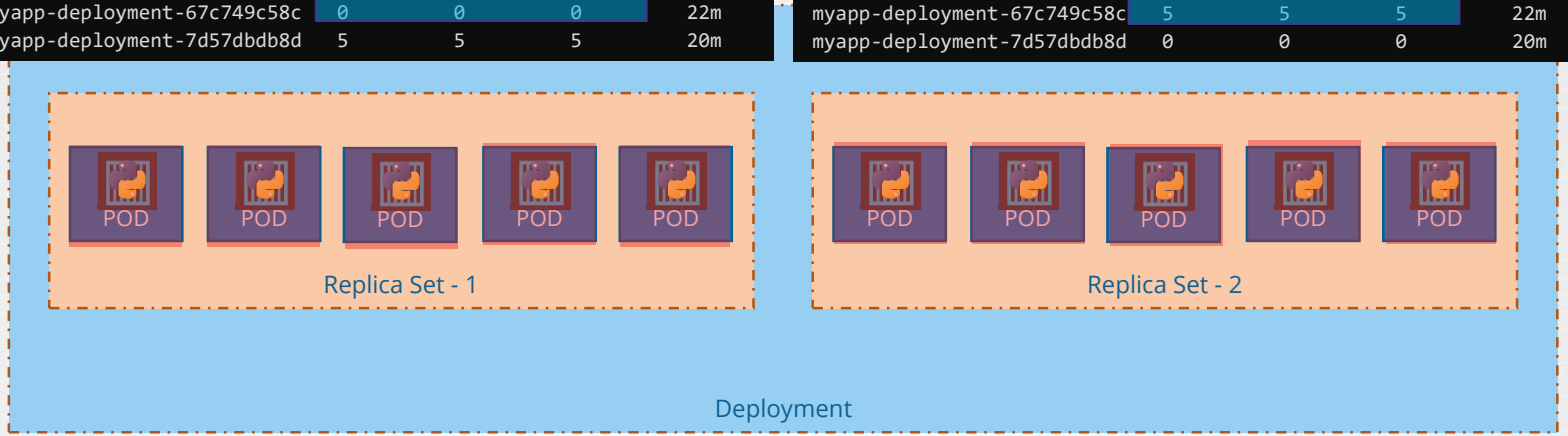



```
> kubectl get replicasets
NAME                          DESIRED   CURRENT   READY     AGE
myapp-deployment-67c749c58c   0         0         0         22m
myapp-deployment-7d57dbdb8d   5         5         5         20m
```

Rollback

```
> kubectl get replicasets
NAME                          DESIRED   CURRENT   READY   AGE
myapp-deployment-67c749c58c   0         0         0       22m
myapp-deployment-7d57dbdb8d   5         5         5       20m
```

```
> kubectl get replicasets
NAME                          DESIRED   CURRENT   READY   AGE
myapp-deployment-67c749c58c   5         5         5       22m
myapp-deployment-7d57dbdb8d   0         0         0       20m
```




```
> kubectl rollout undo deployment/myapp-deployment
deployment “myapp-deployment” rolled back
```

kubectl run

```
> kubectl run nginx --image=nginx
deployment "nginx" created
```

Summarize Commands

Create

```
> kubectl create –f deployment-definition.yml
```

Get

```
> kubectl get deployments
```

Update

```
> kubectl apply –f deployment-definition.yml
```

```
> kubectl set image deployment/myapp-deployment nginx=nginx:1.9.1
```

Status

```
> kubectl rollout status deployment/myapp-deployment
```

```
> kubectl rollout history deployment/myapp-deployment
```

Rollback

```
> kubectl rollout undo deployment/myapp-deployment
```

COMMANDS & ARGUMENTS

```
docker run ubuntu
```

```
docker ps
```

```
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS
```

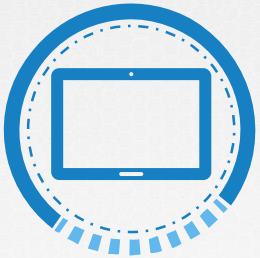
```
docker ps -a
```

```
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS                      PORTS
45aacca36850        ubuntu              "/bin/bash"         43 seconds ago      Exited (0) 41 seconds ago
```

```
# Install Nginx.
RUN \
  add-apt-repository -y ppa:nginx/stable && \
  apt-get update && \
  apt-get install -y nginx && \
  rm -rf /var/lib/apt/lists/* && \
  echo "\ndaemon off;" >> /etc/nginx/nginx.conf && \
  chown -R www-data:www-data /var/lib/nginx

# Define mountable directories.
VOLUME ["/etc/nginx/sites-enabled", "/etc/nginx/certs", "/etc/nginx/cor

# Define working directory.
WORKDIR /etc/nginx

# Define default command.
CMD ["nginx"]
```

```
ARG MYSQL_SERVER_PACKAGE_URL=https://repo.mysql.com/yum/mysql-8.0-community/docker/x86_
ARG MYSQL_SHELL_PACKAGE_URL=https://repo.mysql.com/yum/mysql-tools-community/el/7/x86_6

# Install server
RUN rpmkeys --import https://repo.mysql.com/RPM-GPG-KEY-mysql \
  && yum install -y $MYSQL_SERVER_PACKAGE_URL $MYSQL_SHELL_PACKAGE_URL libpwquality \
  && yum clean all \
  && mkdir /docker-entrypoint-initdb.d

VOLUME /var/lib/mysql

COPY docker-entrypoint.sh /entrypoint.sh
COPY healthcheck.sh /healthcheck.sh
ENTRYPOINT ["/entrypoint.sh"]
HEALTHCHECK CMD /healthcheck.sh
EXPOSE 3306 33060
CMD ["mysqld"]
```

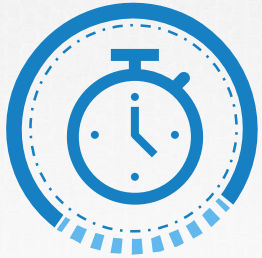
```
# Pull base image.
FROM ubuntu:14.04

# Install.
RUN \
  sed -i 's/# \(.*multiverse$\)/\1/g' /etc/apt/sources.list && \
  apt-get update && \
  apt-get -y upgrade && \
  apt-get install -y build-essential && \
  apt-get install -y software-properties-common && \
  apt-get install -y byobu curl git htop man unzip vim wget && \
  rm -rf /var/lib/apt/lists/*

# Add files.
ADD root/.bashrc /root/.bashrc
ADD root/.gitconfig /root/.gitconfig
ADD root/.scripts /root/.scripts

# Set environment variables.
ENV HOME /root

# Define working directory.
WORKDIR /root

# Define default command.
CMD ["bash"]
```



??

```
docker run ubuntu [COMMAND]
```

```
docker run ubuntu sleep 5
```



```
FROM Ubuntu

CMD sleep 5
```

CMD command param1

CMD [“command”, “param1”]

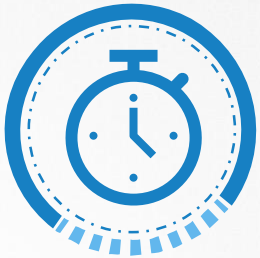
CMD sleep 5

CMD [“sleep”, “5”]

CMD [“sleep 5”]

```
docker build -t ubuntu-sleeper .
```

```
docker run ubuntu-sleeper
```



```
FROM Ubuntu

CMD sleep 5
```

```
docker run ubuntu-sleeper sleep 10
```

Command at Startup: sleep 10

```
FROM Ubuntu

ENTRYPOINT ["sleep"]
```

```
docker run ubuntu-sleeper 10
```

Command at Startup:

```
docker run ubuntu-sleeper
sleep: missing operand
Try 'sleep --help' for more information.
```

Command at Startup:

```
FROM Ubuntu

ENTRYPOINT ["sleep"]

CMD ["5"]
```

```
docker run ubuntu-sleeper
sleep: missing operand
Try 'sleep --help' for more information.
```

Command at Startup:

```
docker run ubuntu-sleeper 10
```

Command at Startup:

```
docker run --entrypoint sleep2.0 ubuntu-sleeper 10
```

Command at Startup:

```
docker run --name ubuntu-sleeper ubuntu-sleeper
```

```
docker run --name ubuntu-sleeper ubuntu-sleeper 10
```

pod-definition.yml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: ubuntu-sleeper-pod
spec:
  containers:
    - name:
      image:
      args:
```

```
kubectl create -f pod-definition.yml
```

```
FROM Ubuntu

ENTRYPOINT ["sleep"]

CMD ["5"]
```

```
docker run --name ubuntu-sleeper \
    --entrypoint ["sleep2.0"]
    ubuntu-sleeper 10
```

pod-definition.yml

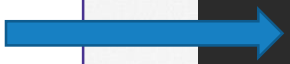
```
apiVersion: v1
kind: Pod
metadata:
  name: ubuntu-sleeper-pod
spec:
  containers:
    - name: ubuntu-sleeper
      image: ubuntu-sleeper
      command: ["10"]
```

```
kubectl create -f pod-definition.yml
```

FROM Ubuntu

ENTRYPOINT [“sleep”]

CMD [“5”]



pod-definition.yml

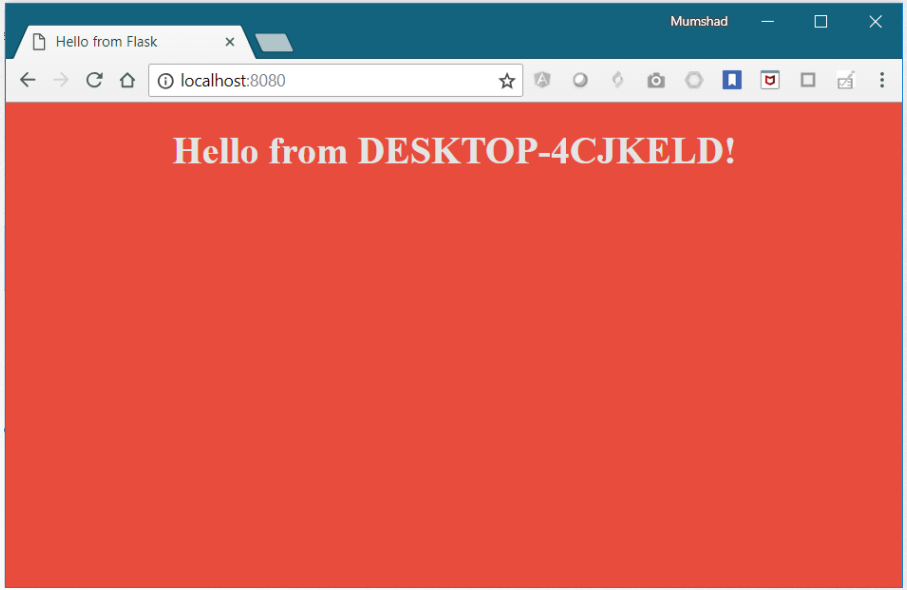
```
apiVersion: v1
kind: Pod
metadata:
  name: ubuntu-sleeper-pod
spec:
  containers:
    - name: ubuntu-sleeper
      image: ubuntu-sleeper
      command:[“sleep2.0”]
      args: [“10”]
```

ENVIRONMENT VARIABLES

Environment Variables

app.py

```python
import os
from flask import Flask

app = Flask(__name__)

…
…

color = "red"

@app.route("/")
def main():
    print(color)
    return render_template('hello.html', color=color)


if __name__ == "__main__":
    app.run(host="0.0.0.0", port="8080")
```




```
python app.py
```

Environment Variables

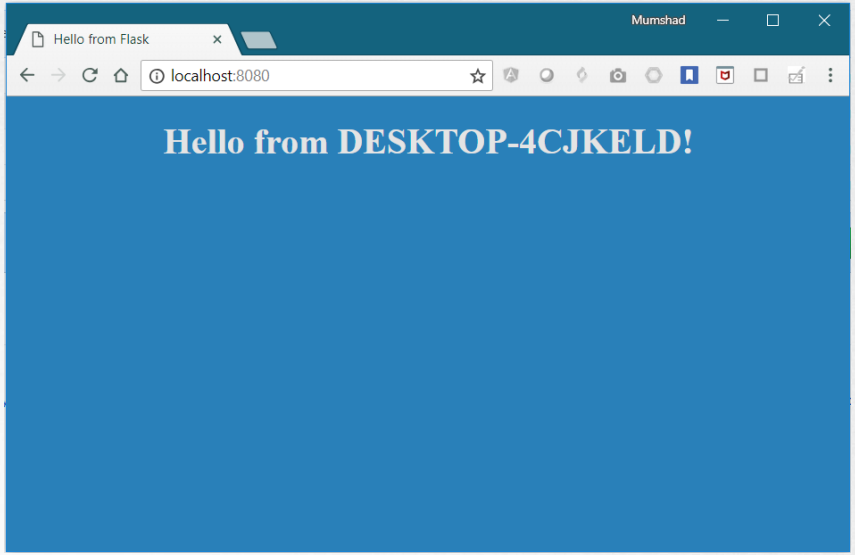
app.py

```
import os
from flask import Flask

app = Flask(__name__)

…
…

color = "red"

@app.route("/")
def main():
    print(color)
    return render_template('hello.html', color=color)


if __name__ == "__main__":
    app.run(host="0.0.0.0", port="8080")
```

Environment Variables

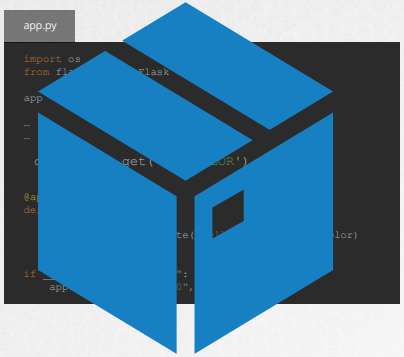
app.py

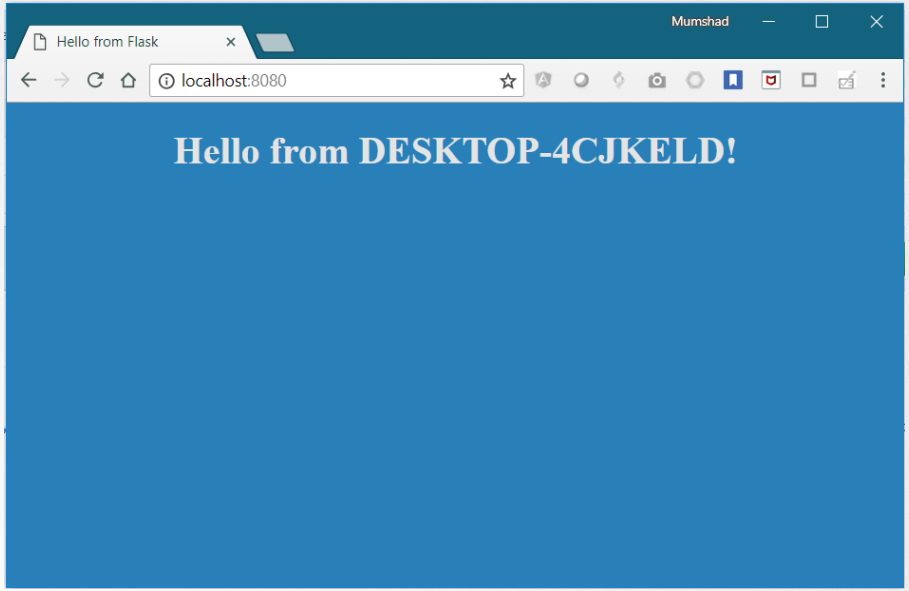
```
import os
from flask import Flask

app = Flask(__name__)

…
…

color = os.environ.get('APP_COLOR')

@app.route("/")
def main():
    print(color)
    return render_template('hello.html', color=color)


if __name__ == "__main__":
    app.run(host="0.0.0.0", port="8080")
```




```
export APP_COLOR=blue; python app.py
```

ENV Variables in Docker

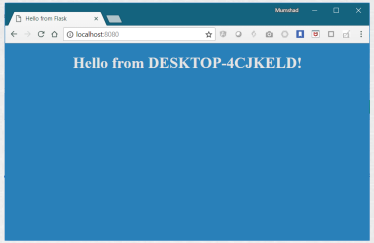




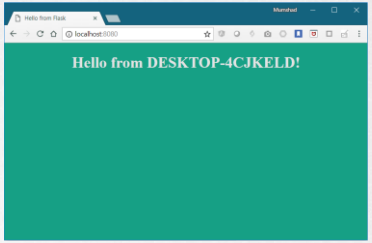

```
docker run -e APP_COLOR=blue simple-webapp-color
```

ENV Variables in Docker

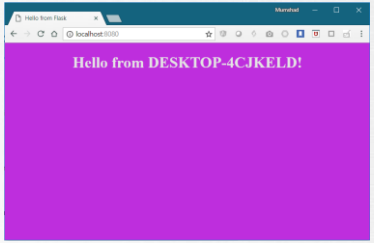
```
docker run  -e APP_COLOR=blue    simple-webapp-color
```




```
docker run  -e APP_COLOR=green simple-webapp-color
```




```
docker run  -e APP_COLOR=pink simple-webapp-color
```

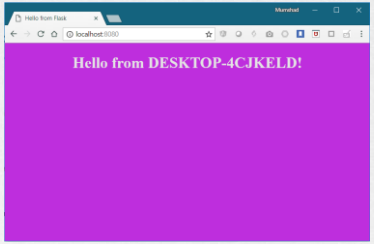


ENV Variables in Kubernetes

```
docker run  -e APP_COLOR=pink simple-webapp-color
```

pod-definition.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: simple-webapp-color
spec:
  containers:
  - name: simple-webapp-color
    image: simple-webapp-color
    ports:
      - containerPort: 8080

    env:
      - name: APP_COLOR
        value: pink
```



ENV Value Types

```
env:
  - name: APP_COLOR
    value: pink
```

```
env:
  - name: APP_COLOR
    valueFrom:
      configMapKeyRef:
```

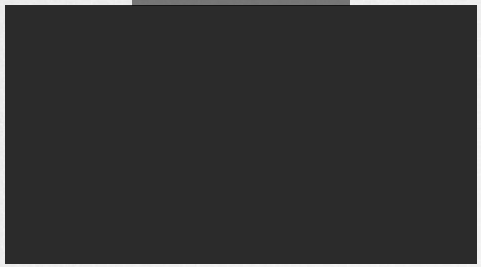
```
env:
  - name: APP_COLOR
    valueFrom:
      secretKeyRef:
```

Course Objectives

- Core Concepts
- Configuration
  - ConfigMaps
  - SecurityContexts
  - Resource Requirements
  - Secrets
  - ServiceAccounts
- Multi-Container Pods
- Observability
- Pod Design
- Services & Networking
- State Persistence

ConfigMaps

ConfigMap



pod-definition.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: simple-webapp-color
spec:
  containers:
  - name: simple-webapp-color
    image: simple-webapp-color
    ports:
      - containerPort: 8080

    env:
      - name: APP_COLOR
        value: blue
      - name: APP_MODE
        value: prod
```

ConfigMaps

ConfigMap

```
APP_COLOR: blue
APP_MODE: prod
```

pod-definition.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: simple-webapp-color
spec:
  containers:
  - name: simple-webapp-color
    image: simple-webapp-color
    ports:
      - containerPort: 8080

    envFrom:
    - configMapRef:
        name: app-config
```

Create ConfigMap

Inject into Pod

Create ConfigMaps

ConfigMap

```
APP_COLOR: blue
APP_MODE: prod
```

Imperative

```
kubectl create configmap
```

Declarative

```
kubectl create -f
```

Create ConfigMap

Create ConfigMaps

ConfigMap

```
APP_COLOR: blue
APP_MODE: prod
```

Create ConfigMap

Imperative

```
kubectl create configmap
    <config-name> --from-literal=<key>=<value>
```

```
kubectl create configmap \
    app-config --from-literal=APP_COLOR=blue \
               --from-literal=APP_MOD=prod
```

```
kubectl create configmap
    <config-name> --from-file=<path-to-file>
```

```
kubectl create configmap \
    app-config --from-file=app_config.properties
```

Create ConfigMaps

ConfigMap

```
APP_COLOR: blue
APP_MODE: prod
```

Declarative

```
kubectl create -f
```

config-map.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_COLOR: blue
  APP_MODE: prod
```

Create ConfigMap

```
kubectl create -f config-map.yaml
```

Create ConfigMaps

app-config

```
APP_COLOR: blue
APP_MODE: prod
```

mysql-config

```
port: 3306
max_allowed_packet: 128M
```

redis-config

```
port: 6379
rdb-compression: yes
```

Create ConfigMap

View ConfigMaps

```
kubectl get configmaps
```

```
NAME         DATA      AGE
app-config   2         3s
```

```
kubectl describe configmaps
```

```
Name:         app-config
Namespace:    default
Labels:       <none>
Annotations:  <none>

Data
====
APP_COLOR:
----
blue
APP_MODE:
----
prod
Events:  <none>
```

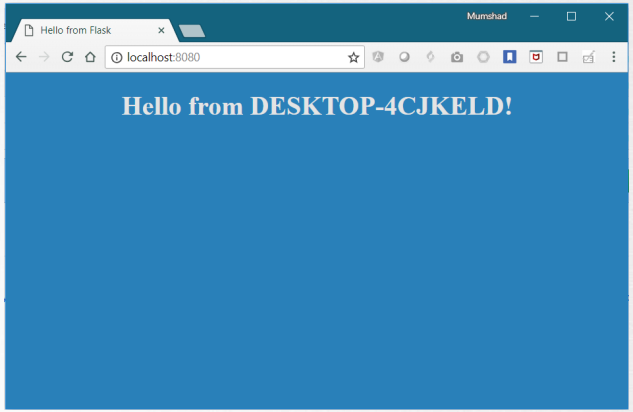
ConfigMap in Pods

pod-definition.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: simple-webapp-color
  labels:
    name: simple-webapp-color
spec:
  containers:
  - name: simple-webapp-color
    image: simple-webapp-color
    ports:
      - containerPort: 8080
    envFrom:
      - configMapRef:
          name: app-config
```

config-map.yaml

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_COLOR: blue
  APP_MODE: prod
```




```
kubectl create -f pod-definition.yaml
```

ConfigMap in Pods

```
envFrom:
  - configMapRef:
      name: app-config
```

ENV

SINGLE ENV

```
env:
  - name: APP_COLOR
    valueFrom:
      configMapKeyRef:
        name: app-config
        key: APP_COLOR
```

```
volumes:
- name: app-config-volume
  configMap:
    name: app-config
```

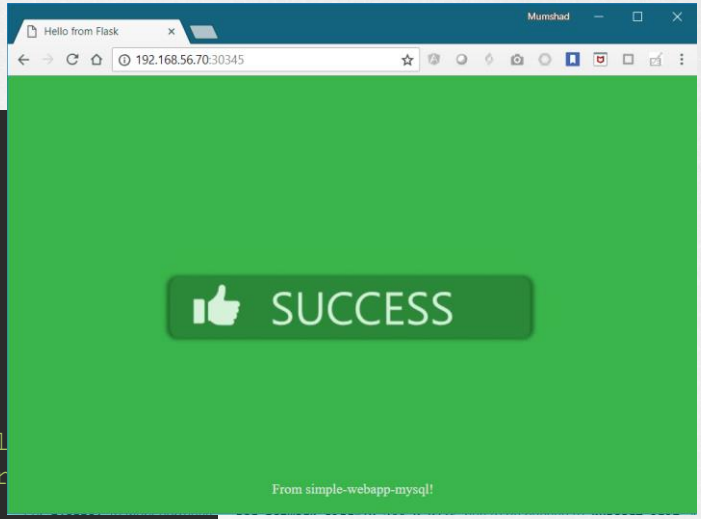
VOLUME

Kubernetes Secrets

Web-MySQL Application

app.py

```
import os
from flask import Flask

app = Flask(__name__)


@app.route("/")
def main():

    mysql.connector.connect(host='mysql', database='mysql
                            user='root', password='paswr

    return render_template('hello.html', color=fetchcolor())


if __name__ == "__main__":
    app.run(host="0.0.0.0", port="8080")
```



Web-MySQL Application

app.py

```
import os
from flask import Flask

app = Flask(__name__)


@app.route("/")
def main():

     mysql.connector.connect(host='mysql', database='mysql',
                             user='root', password='paswrd')

    return render_template('hello.html', color=fetchcolor())


if __name__ == "__main__":
    app.run(host="0.0.0.0", port="8080")
```

Web-MySQL Application

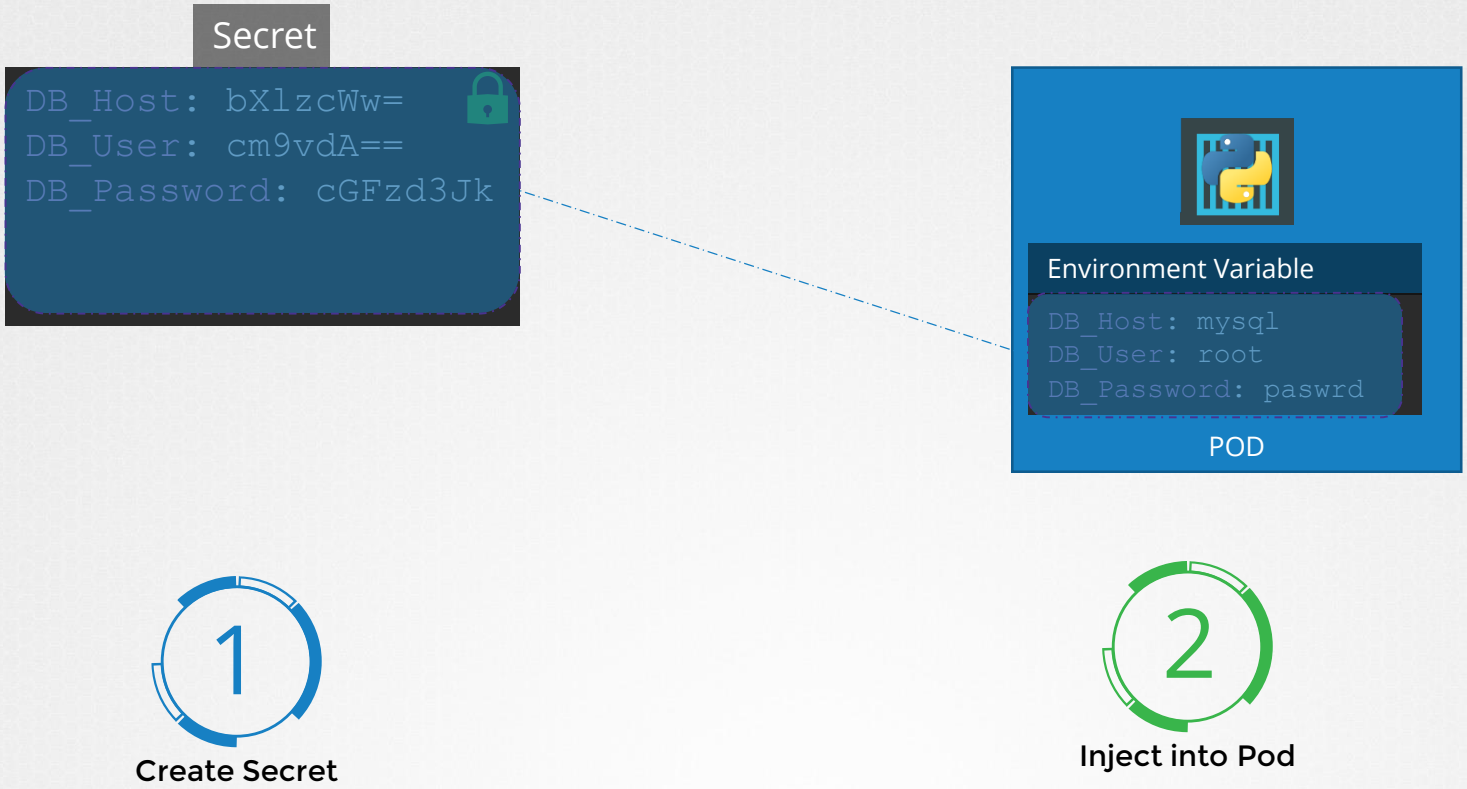
app.py

```
import os
from flask import Flask

app = Flask(__name__)


@app.route("/")
def main():

    mysql.connector.connect(host='mysql', database='mysql',
                            user='root', password='paswrd')

    return render_template('hello.html', color=fetchcolor())


if __name__ == "__main__":
    app.run(host="0.0.0.0", port="8080")
```

config-map.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  DB_Host: mysql
  DB_User: root
  DB_Password: paswrd
```

Secret


Secret
DB_Host: bXlzcWw=
DB_User: cm9vdA==
DB_Password: cGFzd3Jk
Environment Variable
DB_Host: mysql
DB_User: root
DB_Password: paswrd
POD
1
Create Secret
2
Inject into Pod

Create Secrets

Secret

```
DB_Host: mysql
DB_User: root
DB_Password: paswrd
```

Imperative

```
kubectl create secret generic
```

Declarative

```
kubectl create -f
```

Create Secret

Create Secrets

Secret

```
DB_Host: mysql
DB_User: root
DB_Password: paswrd
```

Imperative

```
kubectl create secret generic
    <secret-name> --from-literal=<key>=<value>
```

```
kubectl create secret generic \
    app-secret --from-literal=DB_Host=mysql      \
               --from-literal=DB_User=root
               --from-literal=DB_Password=paswrd
```

```
kubectl create secret generic
    <secret-name> --from-file=<path-to-file>
```

```
kubectl create secret generic \
    app-secret --from-file=app_secret.properties
```

Create Secret

Create Secrets

Secret

```
DB_Host: mysql
DB_User: root
DB_Password: paswrd
```

Create Secret

Declarative

```
kubectl create -f
```

secret-data.yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
data:
  DB_Host: bXlzcWw=
  DB_User: cm9vdA==
  DB_Password: cGFzd3Jk
```

```
kubectl create -f secret-data.yaml
```

Encode Secrets

```
DB_Host: mysql
DB_User: root
DB_Password: paswrd
```

```
DB_Host: bXlzcWw=
DB_User: cm9vdA==
DB_Password: cGFzd3Jk
```

```
echo -n 'mysql' | base64
bXlzcWw=
```

```
echo -n 'root' | base64
cm9vdA==
```

```
echo -n 'paswrd' | base64
cGFzd3Jk
```

View Secrets

```
kubectl get secrets
```

```
NAME                  TYPE                                  DATA      AGE
app-secret            Opaque                                3         10m
default-token-mvtkv   kubernetes.io/service-account-token   3         2h
```

```
kubectl describe secrets
```

```
Name:         app-secret
Namespace:    default
Labels:       <none>
Annotations:  <none>

Type:  Opaque

Data
====
DB_Host:      10 bytes
DB_Password:  6 bytes
DB_User:      4 bytes
```

```
kubectl get secret app-secret -o yaml
```

```
apiVersion: v1
data:
  DB_Host: bXlzcWw=
  DB_Password: cGFzd3Jk
  DB_User: cm9vdA==
kind: Secret
metadata:
  creationTimestamp: 2018-10-18T10:01:12Z
  labels:
    name: app-secret
  name: app-secret
  namespace: default
uid: be96e989-d2bc-11e8-a545-080027931072
type: Opaque
```

Decode Secrets

```
DB_Host: mysql
DB_User: root
DB_Password: paswrd
```

```
DB_Host: bXlzcWw=
DB_User: cm9vdA==
DB_Password: cGFzd3Jk
```

```
echo -n 'bXlzcWw=' | base64 --decode
mysql
```

```
echo -n 'cm9vdA==' | base64 --decode
root
```

```
echo -n 'cGFzd3Jk' | base64 --decode
paswrd
```

Secrets in Pods

pod-definition.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: simple-webapp-color
  labels:
    name: simple-webapp-color
spec:
  containers:
  - name: simple-webapp-color
    image: simple-webapp-color
    ports:
      - containerPort: 8080
    envFrom:
      - secretRef:
          name: app-secret
```

secret-data.yaml

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
data:
  DB_Host: bXlzcWw=
  DB_User: cm9vdA==
  DB_Password: cGFzd3Jk
```

Inject into Pod

```
kubectl create -f pod-definition.yaml
```

Secrets in Pods

```
envFrom:
  - secretRef:
      name: app-config
```

ENV

SINGLE ENV

```
env:
  - name: DB_Password
    valueFrom:
      secretKeyRef:
        name: app-secret
        key: DB_Password
```

```
volumes:
- name: app-secret-volume
  secret:
    secretName: app-secret
```

VOLUME

Secrets in Pods as Volumes

```
volumes:
- name: app-secret-volume
  secret:
    secretName: app-secret
```

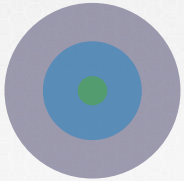
VOLUME

```
ls /opt/app-secret-volumes
DB_Host      DB_Password  DB_User
```

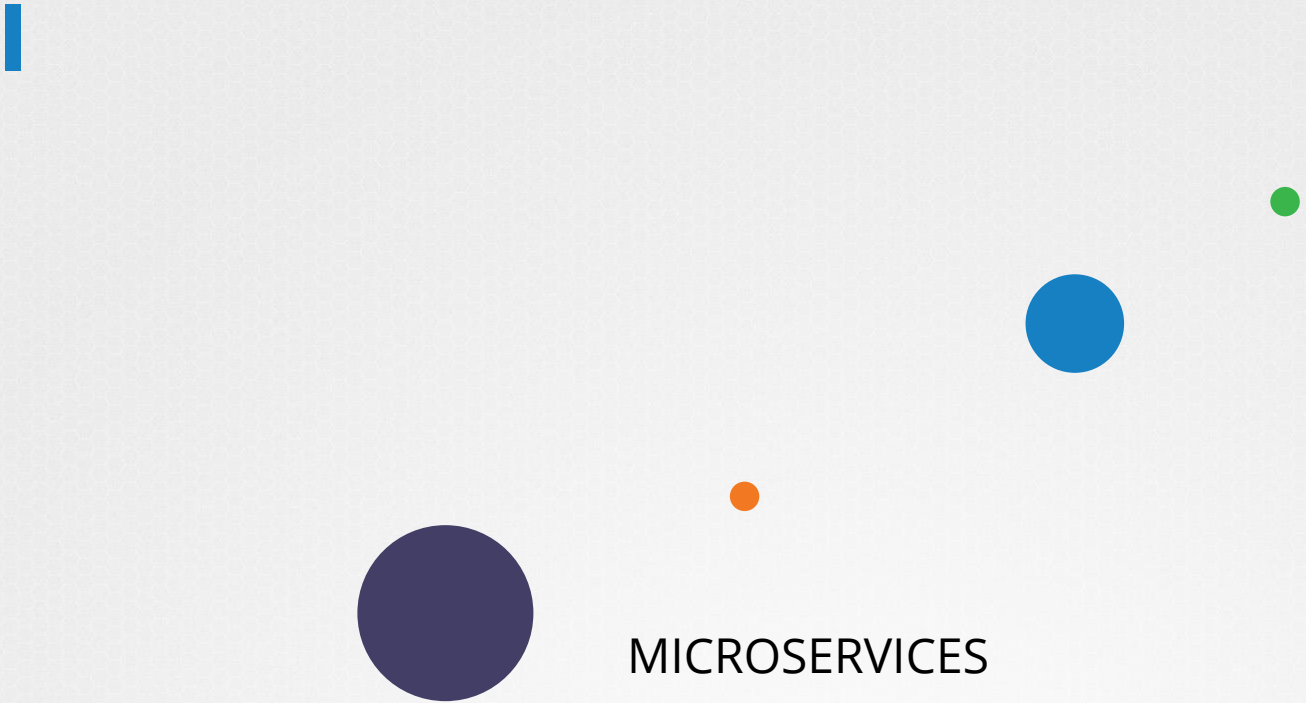
```
cat /opt/app-secret-volumes/DB_Password
paswrd
```

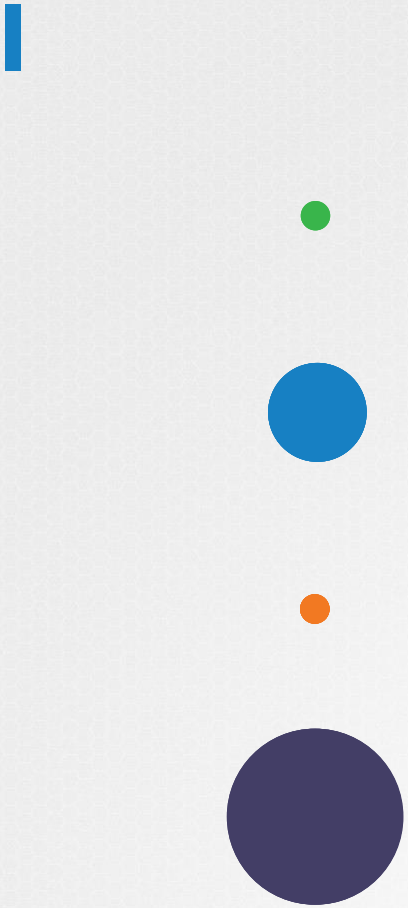
Inside the Container

Kubernetes Multi-Container PODs

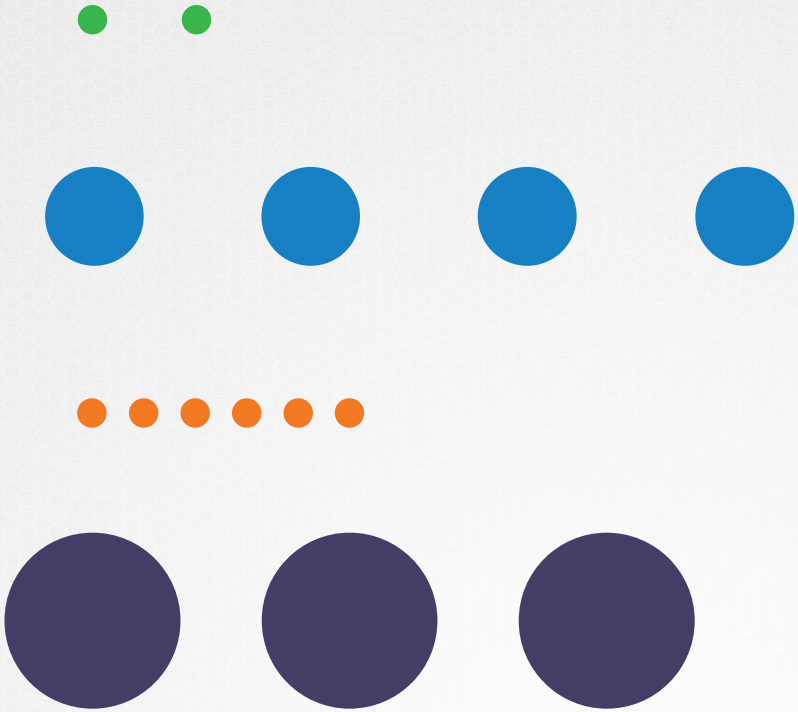


MONOLITH

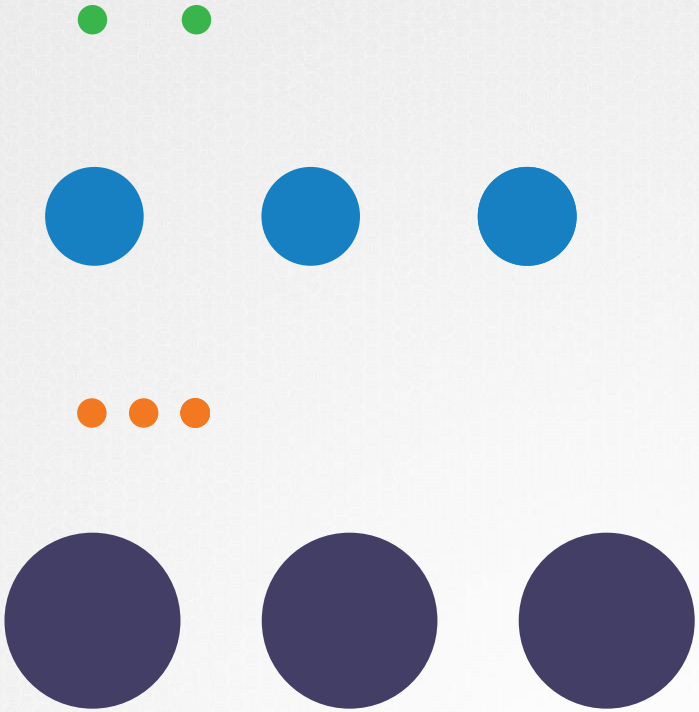

MICROSERVICES



MICROSERVICES



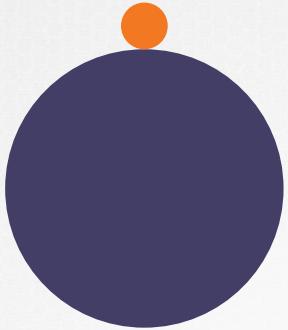
MICROSERVICES



MICROSERVICES

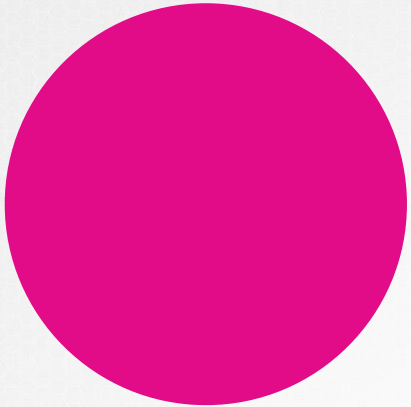
LOG Agent

WEB Server



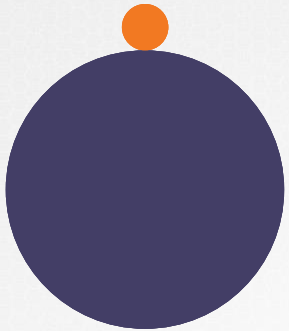
LOG Agent

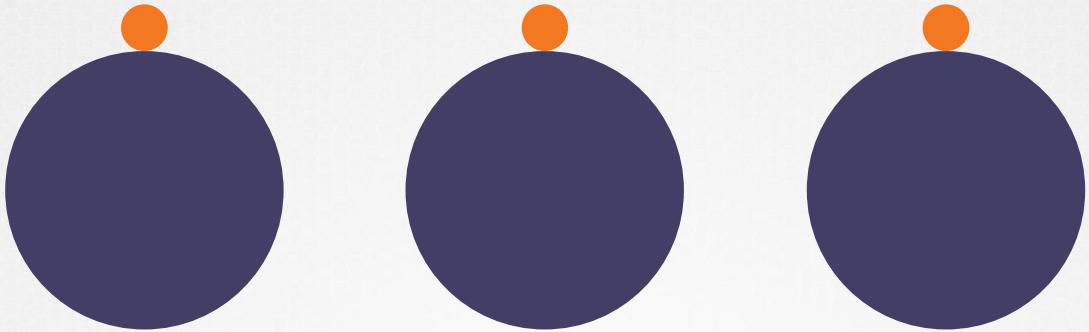
WEB Server



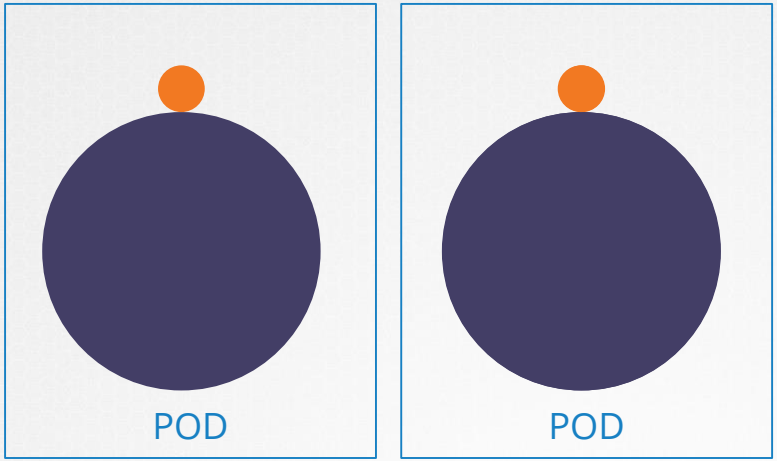
LOG Agent

WEB Server



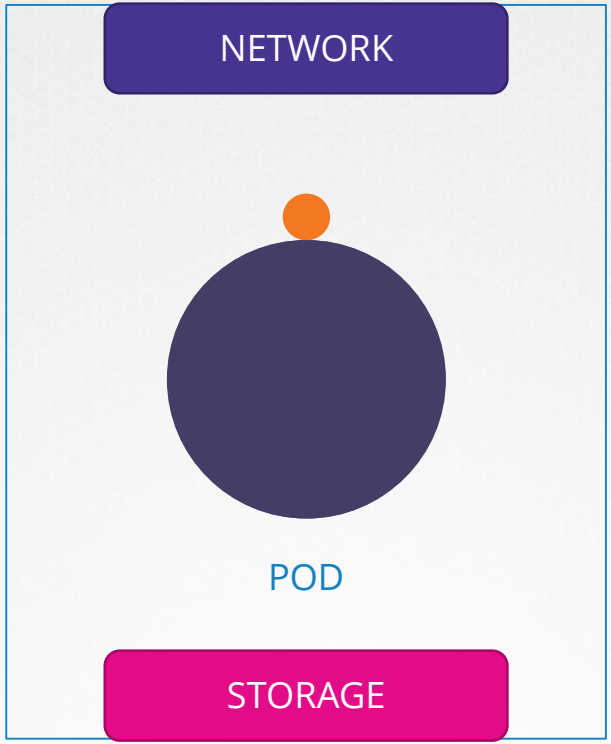


Multi-Container PODs



Multi-Container PODs

LIFECYCLE



Create

pod-definition.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: simple-webapp
  labels:
    name: simple-webapp
spec:
  containers:
  - name: simple-webapp
    image: simple-webapp
    ports:
      - containerPort: 8080

  - name: log-agent
    image: log-agent
```

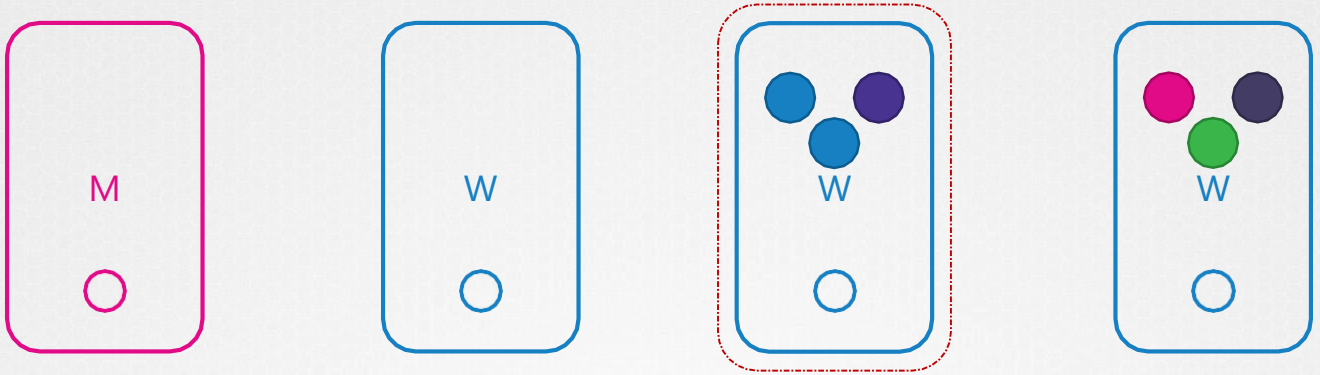
KODEKLOUD

Course Objectives

- Core Concepts
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
  - Operating System Upgrades
  - Kubernetes Releases/Versions
  - Cluster Upgrade Process
  - Backup and Restore Methodologies
- Security
- Storage
- Networking
- Installation, Configuration & Validation
- Troubleshooting

KODEKLOUD

Operating System Upgrade




```
kubectl drain node-1
```

```
kubectl cordon node-2
```

```
kubectl uncordon node-1
```

KODEKLOUD

KODEKLOUD

Course Objectives

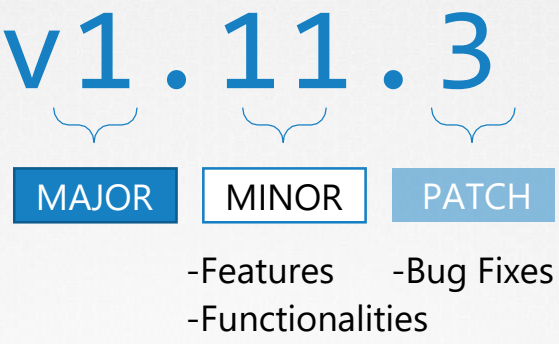
- Core Concepts
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
  - Operating System Upgrades
  - Kubernetes Releases/Versions
  - Cluster Upgrade Process
  - Backup and Restore Methodologies
- Security
- Storage
- Networking
- Installation, Configuration & Validation
- Troubleshooting

KODEKLOUD

Kubernetes Releases

```
kubectl get nodes
NAME      STATUS    ROLES     AGE       VERSION
master  Ready  master node-    1d        v1.11.3
1    Ready    <none>  node-2    1d        v1.11.3
Ready  <none>                   1d        v1.11.3
```

KODEKLOUD


v1.11.3
MAJOR
MINOR
PATCH
-Features
-Functionalities
-Bug Fixes

KODEKLOUD

v0.20 June 2015

Mar 2015 v0.12

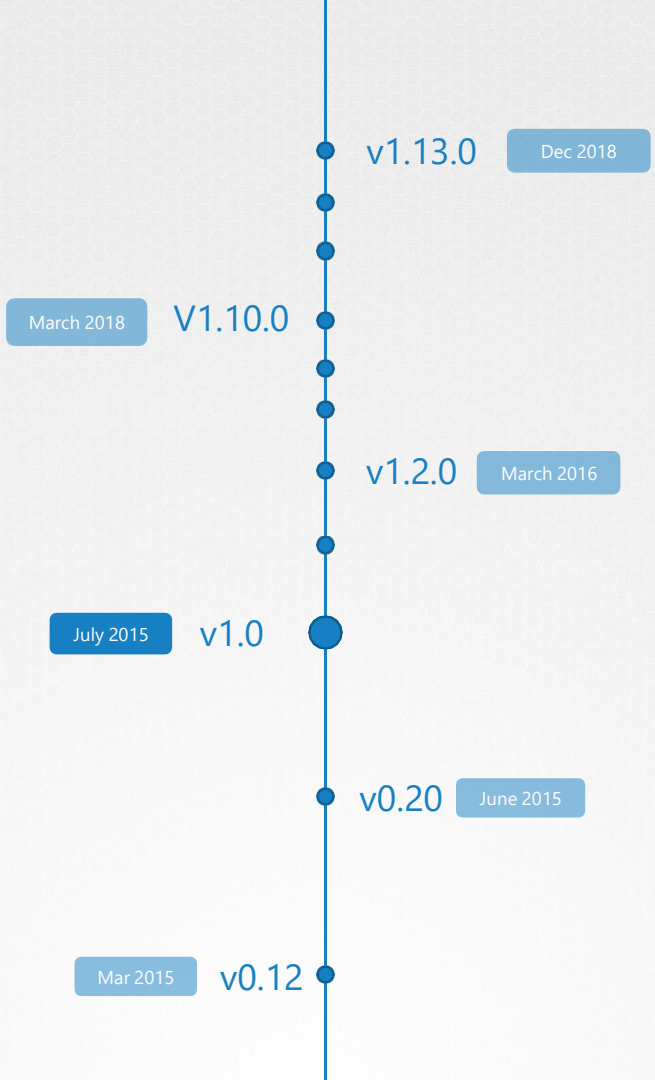
v0.10 Feb 2015

Jan 2015 v0.8

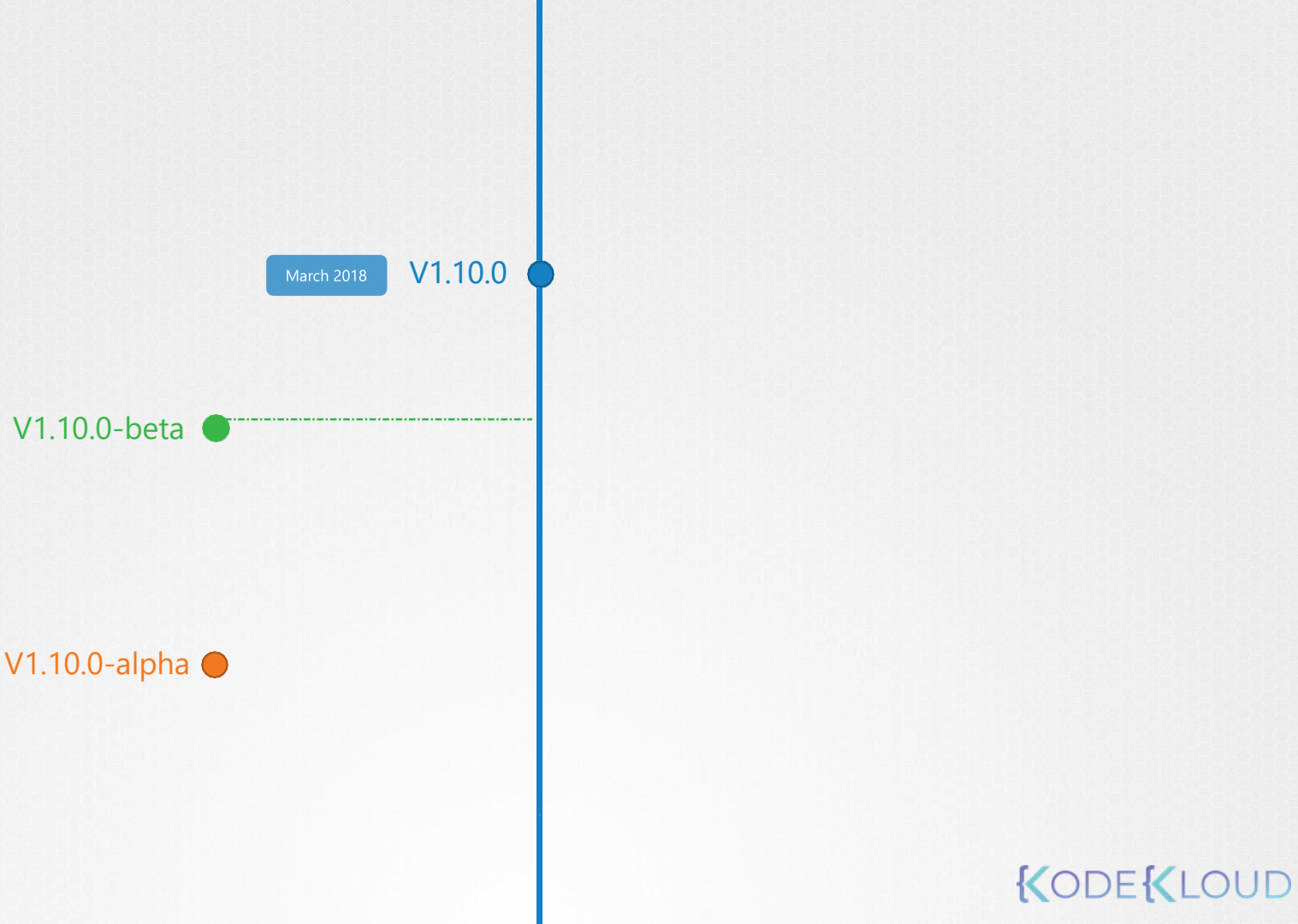
v0.6 Dec 2014

Oct 2014 v0.4

KODEKLOUD


v1.13.0
Dec 2018
March 2018
V1.10.0
v1.2.0
March 2016
July 2015
v1.0
v0.20
June 2015
Mar 2015
v0.12

KODEKLOUD



KODEKLOUD

kubernetes / kubernetes

Watch 2,918 | Unstar 49,224 | Fork 17,023

Code | Issues 2,151 | Pull requests 992 | Projects 11 | Insights

Releases | Tags

8 days ago

v1.13.5-beta.0 ...

9cb83c5 zip tar.gz

v1.13.4

c27b913

v1.13.4

k8s-release-robot released this 8 days ago · 8 commits to release-1.13 since this release

See kubernetes-announce@ and CHANGELOG-1.13.md for details.

SHA512 for `kubernetes.tar.gz`:

```
591cd3f4f479744a1d4754490281735032 1c63f8c37ad771d559e293bcdbc421e89d62663300a6739c667d34e1e24bb080dd735
62dc29713381db079ba6e9223
```

Additional binary downloads are linked in the CHANGELOG-1.13.md.

Assets 3

kubernetes.tar.gz	1.85 MB
Source code (zip)	
Source code (tar.gz)	

https://github.com/kubernetes/kubernetes/releases

LOUD

v1.13.4

kube-apiserver	Controller-manager	kube-scheduler	kubelet	kube-proxy	kubectl
v1.13.4	v1.13.4	v1.13.4	v1.13.4	v1.13.4	v1.13.4

ETCD CLUSTER	CoreDNS
v3.2.18	v1.1.3

KODEKLOUD

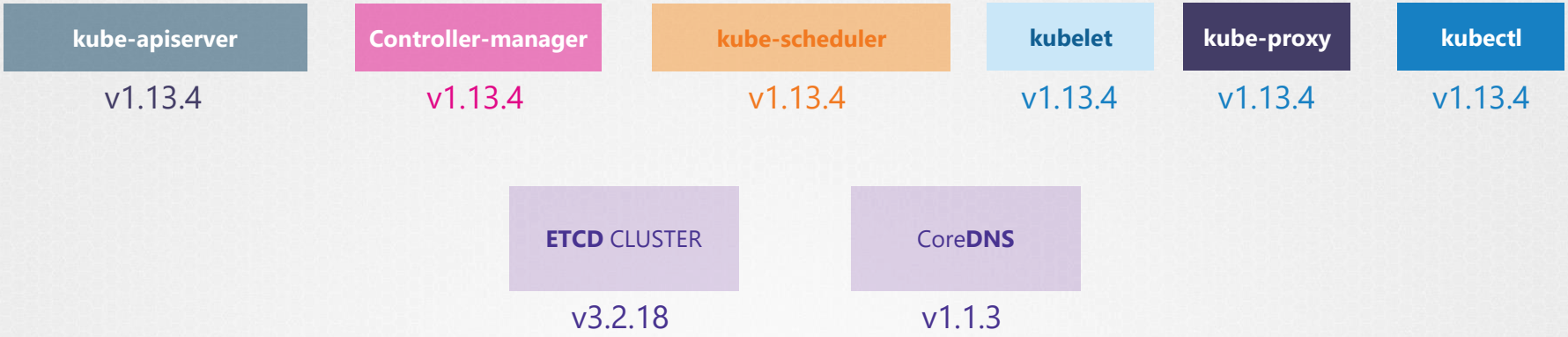
KODEKLOUD

Course Objectives

- Core Concepts
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
  - Operating System Upgrades
  - Kubernetes Releases/Versions
  - Cluster Upgrade Process
  - Backup and Restore Methodologies
- Security
- Storage
- Networking
- Installation, Configuration & Validation
- Troubleshooting

KODEKLOUD

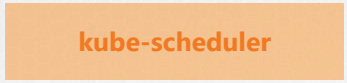
Cluster Upgrade Process


kube-apiserver
v1.13.4
Controller-manager
v1.13.4
kube-scheduler
v1.13.4
kubelet
v1.13.4
kube-proxy
v1.13.4
kubectl
v1.13.4
ETCD CLUSTER
v3.2.18
CoreDNS
v1.1.3

KODEKLOUD

X v1.10

X-1
v1.9 or v1.10



X-1
v.19 or v1.10

X+1 > X-1

X-2

v1.8 or v1.9 or v.110

X-2

V1.8 or v1.9 or v1.10

KODEKLOUD

kube-apiserver

v1.10

Controller-manager

v1.10

kube-scheduler

v1.10

kubectl

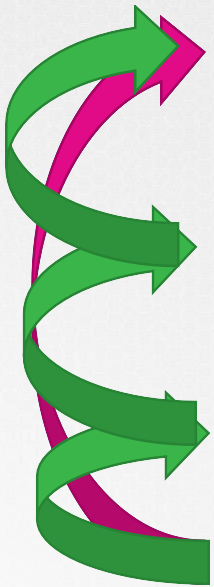
v1.10

kubelet

v1.10

kube-proxy

v1.10



v1.13 Supported

v1.12 Supported

v1.11 Supported

v1.10 Un-Supported

KODEKLOUD

standard-cluster-1

Details Storage Nodes

Cluster

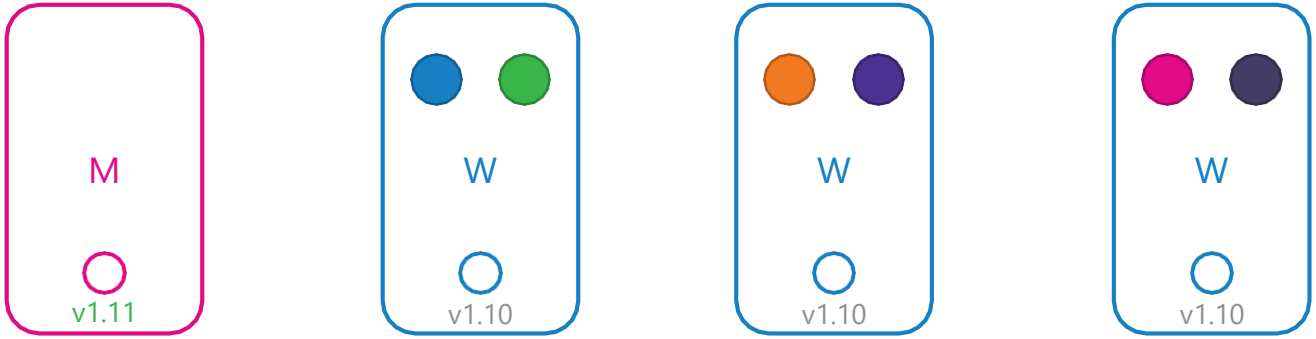
Master version	1.10.12-gke.7	Upgrade available
Endpoint	35.238.15.143	Show credentials
Client certificate	Enabled	
Binary authorisation	Disabled	
Kubernetes alpha features	Disabled	
Total size	3	
Master zone	us-central1-a	
Node zones	us-central1-a	
Network	default	

kubeadm

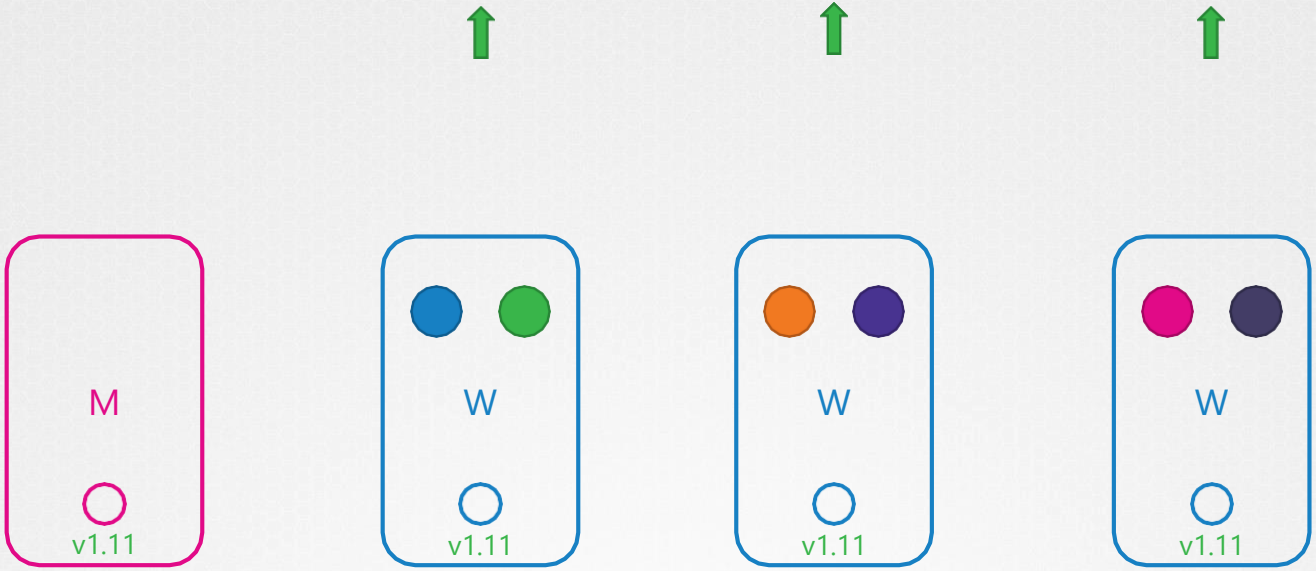
```
kubeadm upgrade plan
```

```
kubeadm upgrade apply
```

“The hard way”


M
v1.11
W
v1.10
W
v1.10
W
v1.10

Strategy - 1

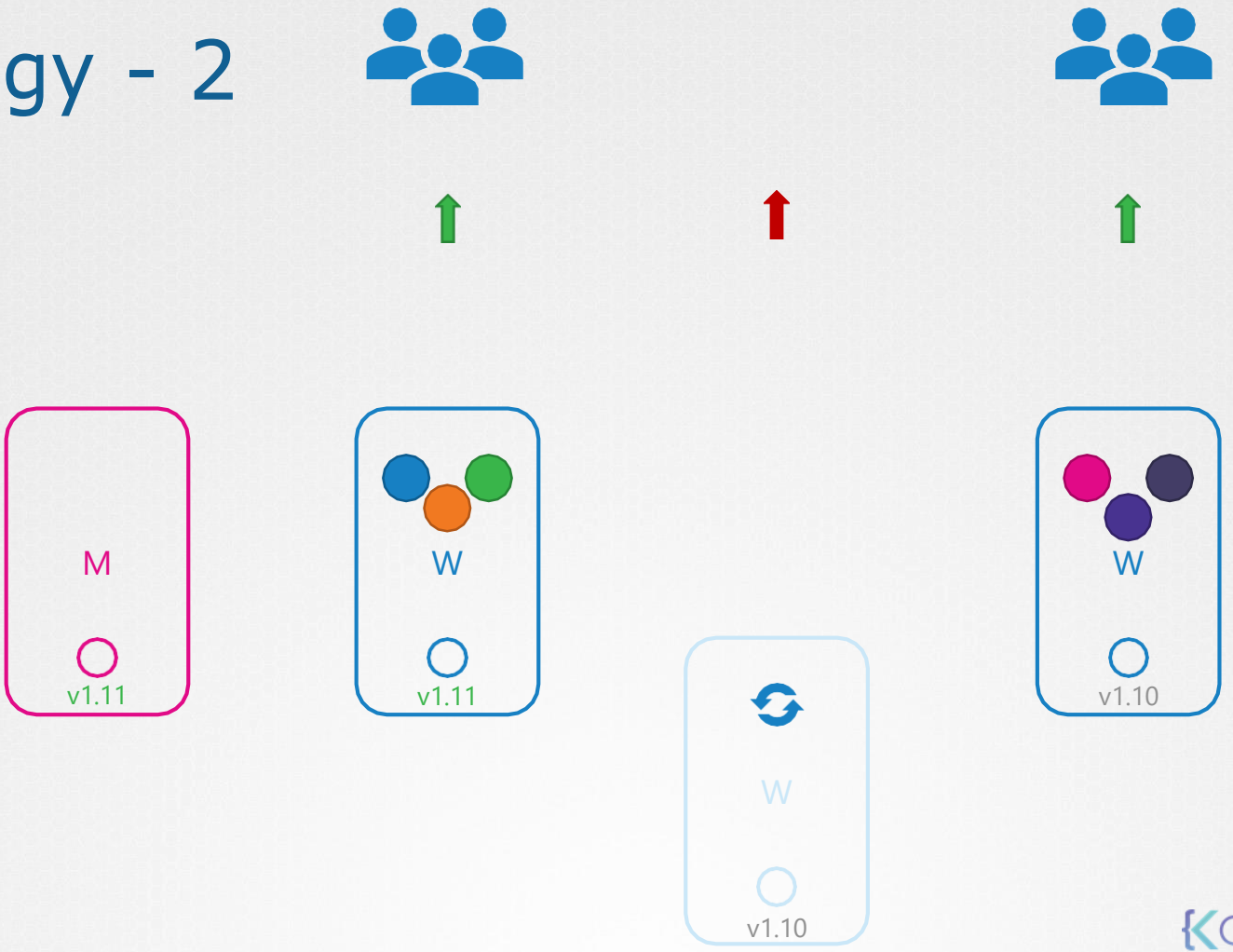


KODEKLOUD

Strategy - 2

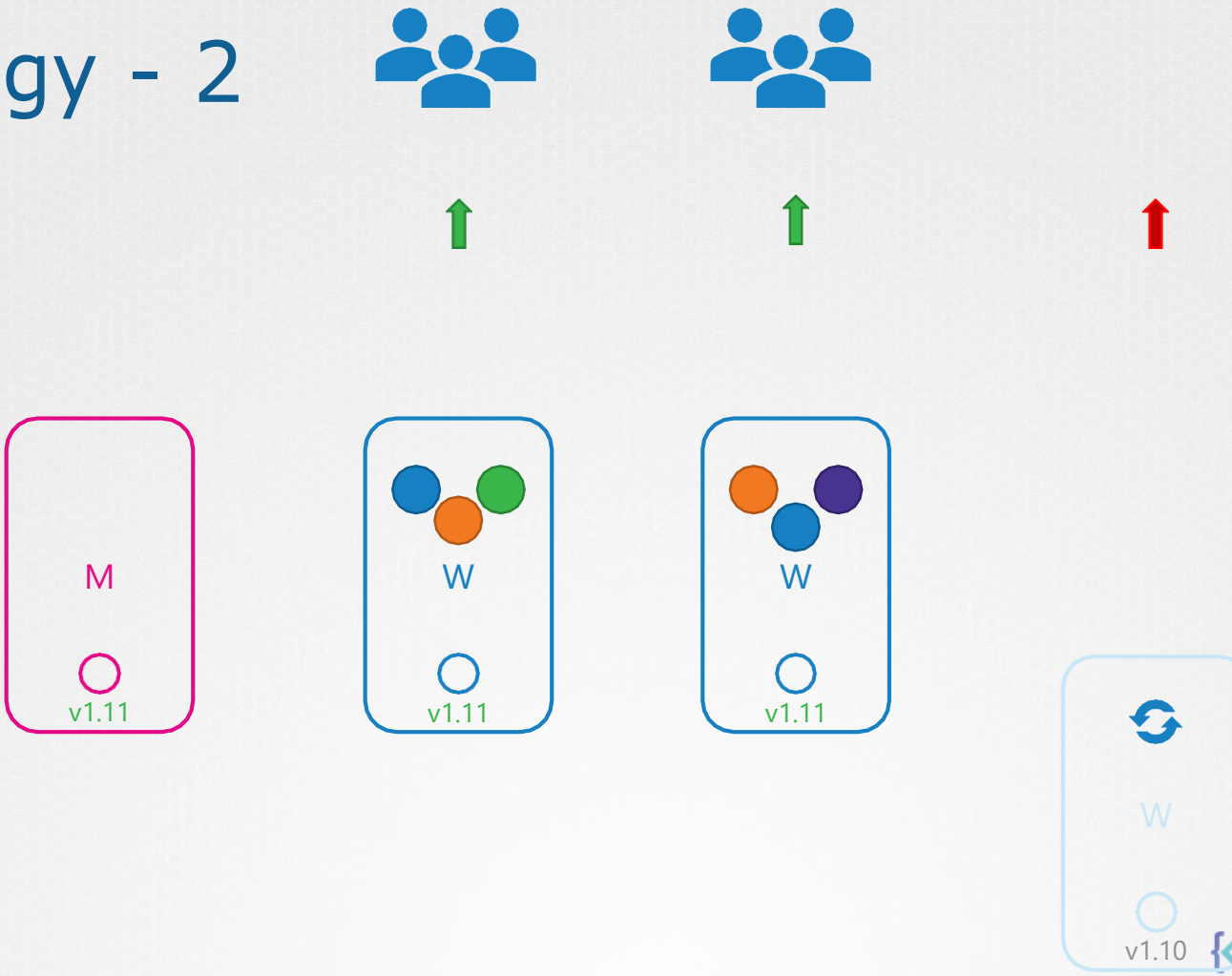
KODEKLOUD

Strategy - 2



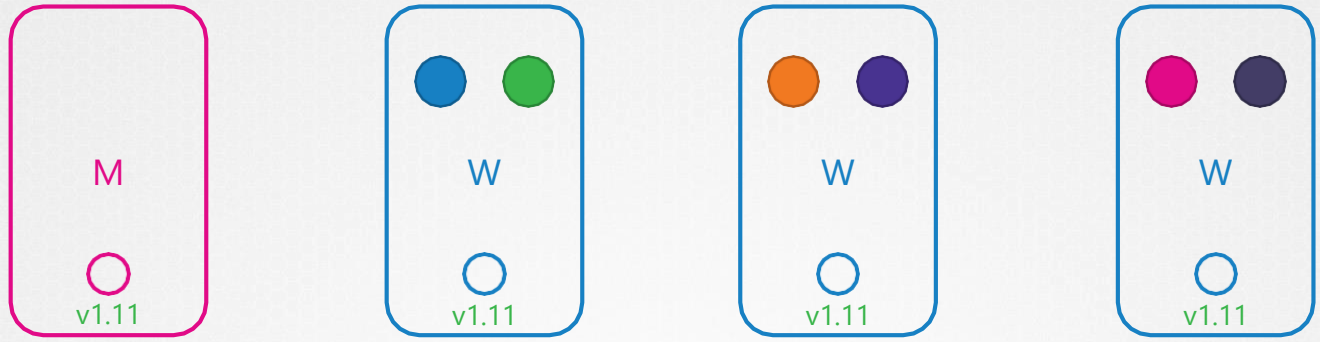
KODEKLOUD

Strategy - 2



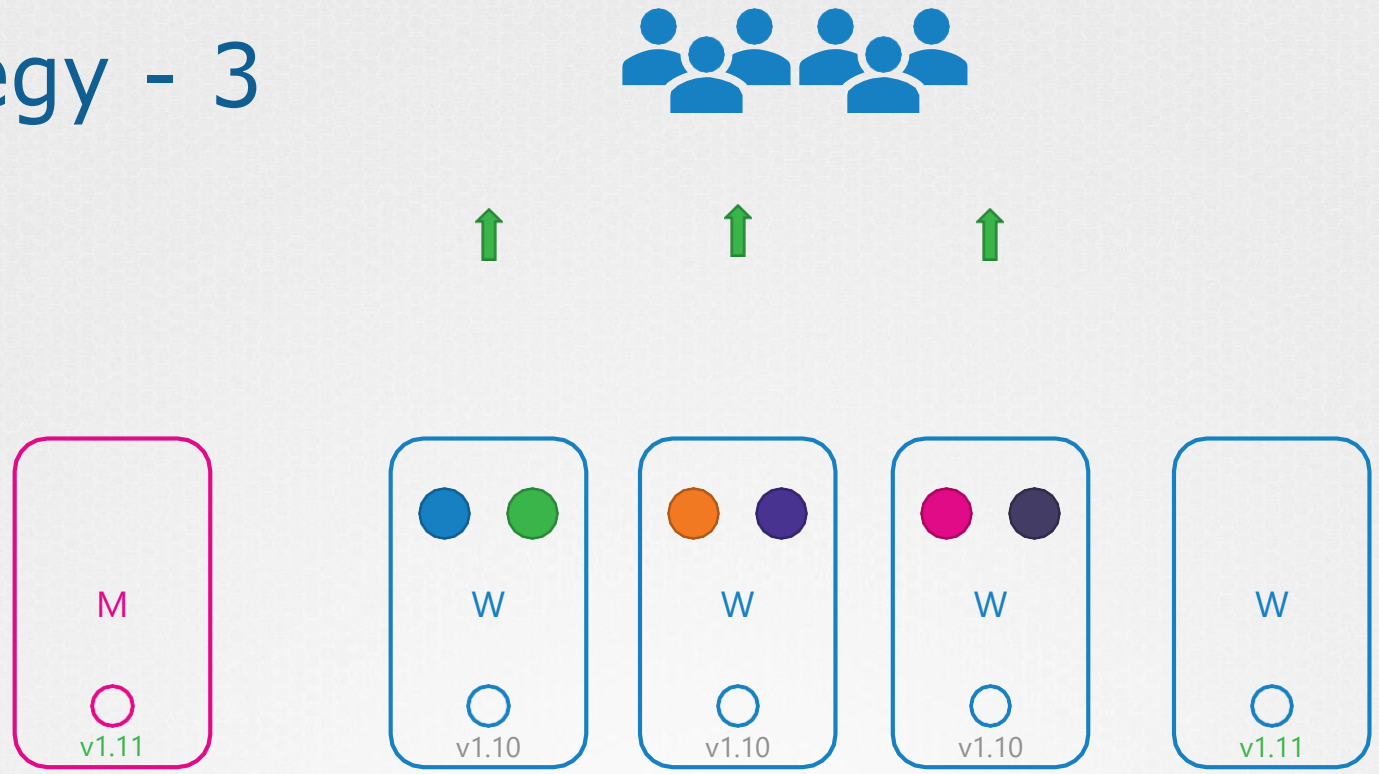
KODEKLOUD

Strategy - 2



KODEKLOUD

Strategy - 3



KODEKLOUD

Strategy - 3

KODEKLOUD

Strategy - 3

KODEKLOUD

kubeadm - upgrade

kubeadm upgrade plan

```
[preflight] Running pre-flight checks.
[upgrade] Making sure the cluster is healthy:
[upgrade/config] Making sure the configuration is correct:
[upgrade] Fetching available versions to upgrade to
[upgrade/versions] Cluster version: v1.11.8
[upgrade/versions] kubeadm version: v1.11.3
[upgrade/versions] Latest stable version: v1.13.4
[upgrade/versions] Latest version in the v1.11 series: v1.11.8

Components that must be upgraded manually after you have
upgraded the control plane with 'kubeadm upgrade apply':
COMPONENT CURRENT  AVAILABLE
Kubelet     3 x v1.11.3   v1.13.4

Upgrade to the latest stable version:

COMPONENT            CURRENT   AVAILABLE
API Server           v1.11.8   v1.13.4
Controller  Manager   v1.11.8   v1.13.4
Scheduler            v1.11.8   v1.13.4
Kube Proxy           v1.11.8   v1.13.4
CoreDNS              1.1.3     1.1.3
Etcd                 3.2.18    N/A

You can now apply the upgrade by executing the following command:
```

kubeadm - u

```
kubeadm upgrade plan
[preflight] Running pre-flight checks.
[upgrade] Making sure the cluster is healthy:
[upgrade/config] Making sure the configuration is correct:
[upgrade] Fetching available versions to upgrade to
[upgrade/versions] Cluster version: v1.11.8
[upgrade/versions] kubeadm version: v1.11.3
[upgrade/versions] Latest stable version: v1.13.4
[upgrade/versions] Latest version in the v1.11 series: v1.11.8

Components that must be upgraded manually after you have
upgraded the control plane with 'kubeadm upgrade apply':
COMPONENT CURRENT  AVAILABLE
Kubelet     3 x v1.11.3   v1.13.4

Upgrade to the latest stable version:

COMPONENT            CURRENT   AVAILABLE
API Server           v1.11.8   v1.13.4
Controller  Manager   v1.11.8   v1.13.4
Scheduler            v1.11.8   v1.13.4
Kube Proxy           v1.11.8   v1.13.4
CoreDNS              1.1.3     1.1.3
Etcd                 3.2.18    N/A

You can now apply the upgrade by executing the following command:

        kubeadm upgrade apply v1.13.4

Note: Before you can perform this upgrade, you have to update kubeadm to v1.13.4.
```

kubeadm - upgrade

```
apt-get upgrade -y kubeadm=1.12.0-00
```

```
kubeadm upgrade apply v1.12.0
```

```
…

[upgrade/successful] SUCCESS! Your cluster was upgraded to "v1.12.0". Enjoy!

[upgrade/kubelet] Now that your control plane is upgraded, please proceed with
upgrading your kubelets if you haven't already done so.
```

```
kubectl get nodes
```

```
NAME      STATUS    ROLES     AGE       VERSION
master   Ready   masternode-    1d        v1.11.3
1    Ready    <none> node-2    1d        v1.11.3
Ready   <none>                  1d        v1.11.3
```

```
apt-get upgrade -y kubelet=1.12.0-00
```

```
systemctl restart kubelet
```

KODEKLOUD

kubeadm - upgrade

```
kubectl get nodes
NAME      STATUS    ROLES     AGE       VERSION
master  Ready  master node-    1d        v1.11.3
1    Ready    <none>  node-2    1d        v1.11.3
Ready  <none>                   1d        v1.11.3
```

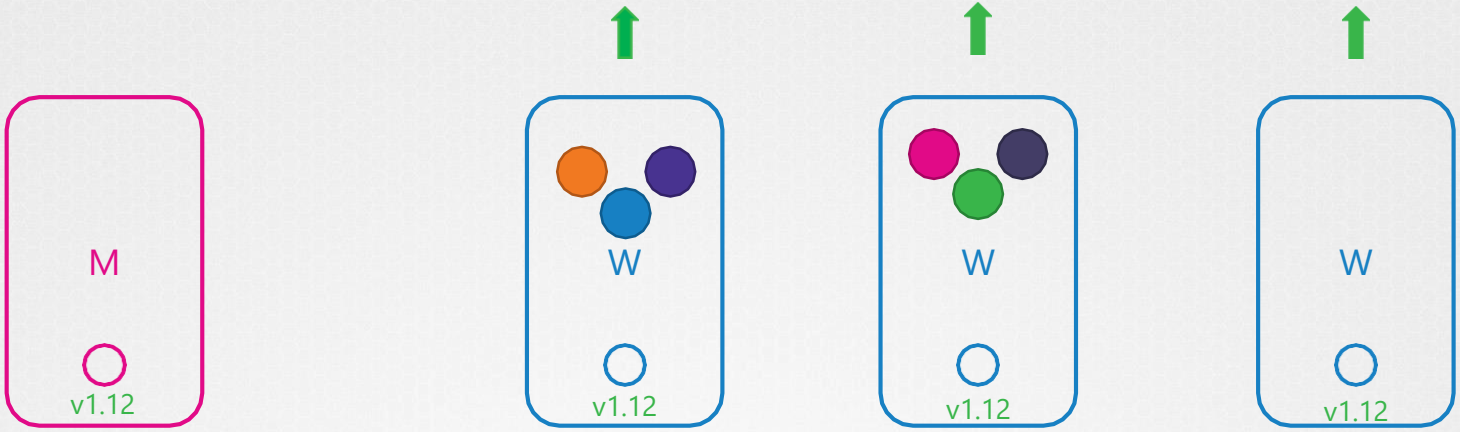
```
apt-get upgrade -y kubelet=1.12.0-00
```

```
systemctl restart kubelet
```

```
kubectl get nodes
NAME      STATUS    ROLES     AGE       VERSION
master  Ready  masternode-    1d        v1.12.0
1    Ready    <none> node-2    1d        v1.11.3
Ready  <none>                   1d        v1.11.3
```

KODEKLOUD

kubeadm - upgrade




```
kubectl drain node-1
kubectl uncordon node-1
kubectl drain node-2
kubectl uncordon node-2
kubectl drain node-3
kubectl uncordon node-3
```

KODEKLOUD

KODEKLOUD

Course Objectives

- Core Concepts
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
  - Kubernetes Release
  - Cluster Upgrade Process
  - Operating System Upgrades
  - Backup and Restore Methodologies
- Security
- Storage
- Networking
- Installation, Configuration & Validation
- Troubleshooting

KODEKLOUD

Backup and Restore

Backup Candidates

Resource Configuration

ETCD Cluster

Persistent Volumes

KODEKLOUD

Imperative

Resource Configuration

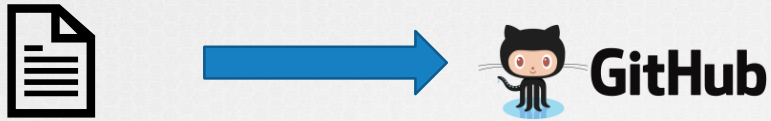
```
kubectl create namespace new-namespace
```

```
kubectl create secret
```

```
kubectl create configmap
```

KODEKLOUD

Declarative



Resource Configuration

pod-definition.yml

```yaml
apiVersion: v1
kind: Pod

metadata:
  name: myapp-pod
  labels:
    app: myapp
    type: front-end
spec:
  containers:
  - name: nginx-container
    image: nginx
```

```
kubectl apply -f pod-definition.yml
```

KODEKLOUD

Backup – Resource Configs

kube-apiserver

Resource Configuration

```
kubectl get all --all-namespaces -o yaml > all-deploy-services.yaml
```

Formerly called ARK by HeptIO

KODEKLOUD

Backup - ETCD

ETCD Cluster

KODEKLOUD

Backup - ETCD

ETCD Cluster

etcd.service

```
ExecStart=/usr/local/bin/etcd \\
  --name ${ETCD_NAME} \\
  --cert-file=/etc/etcd/kubernetes.pem \\
  --key-file=/etc/etcd/kubernetes-key.pem \\
  --peer-cert-file=/etc/etcd/kubernetes.pem \\
  --peer-key-file=/etc/etcd/kubernetes-key.pem \\
  --trusted-ca-file=/etc/etcd/ca.pem \\
  --peer-trusted-ca-file=/etc/etcd/ca.pem \\
  --peer-client-cert-auth \\
  --client-cert-auth \\
  --initial-advertise-peer-urls https://${INTERNAL_IP}:
  --listen-peer-urls https://${INTERNAL_IP}:2380 \\
  --listen-client-urls https://${INTERNAL_IP}:2379,http
  --advertise-client-urls https://${INTERNAL_IP}:2379 \
  --initial-cluster-token etcd-cluster-0 \\
  --initial-cluster controller-0=https://${CONTROLLER0_
  --initial-cluster-state new \\
  --data-dir=/var/lib/etcd
```

KODEKLOUD

Backup - ETCD

ETCD Cluster

```
ETCDCTL_API=3 etcdctl \
      snapshot save snapshot.db
```

```
ls
snapshot.db
```

```
ETCDCTL_API=3 etcdctl \
      snapshot status snapshot.db
+----------+----------+------------+------------+
|   HASH   | REVISION | TOTAL KEYS | TOTAL SIZE |
+----------+----------+------------+------------+
| e63b3fc5 |   473353 |        875 |     4.1 MB |
+----------+----------+------------+------------+
```

KODEKLOUD

Restore - ETCD

ETCD Cluster

```
ETCDCTL_API=3 etcdctl \
     snapshot save snapshot.db
```

```
ls
snapshot.db
```

```
service kube-apiserver stop
Service kube-apiserver stopped
```

```
ETCDCTL_API=3 etcdctl \
  snapshot restore snapshot.db \
  --data-dir /var/lib/etcd-from-backup \
  --initial-cluster master-1=https://192.168.5.11:2380,master-2=https://192.168.5.12:2380 \
  --initial-cluster-token etcd-cluster-1 \
  --initial-advertise-peer-urls https://${INTERNAL_IP}:2380
```

```
I | mvcc: restore compact to 475629
I | etcdserver/membership: added member 5e89ccdfe3 [https://192.168.5.12:2380] to cluster 894c7131f5165a78
I | etcdserver/membership: added member c8246cee7c [https://192.168.5.11:2380] to cluster 894c7131f5165a78
```

KODEKLOUD

Restore - ETCD

ETCD Cluster

```
ETCDCTL_API=3 etcdctl \
    snapshot save snapshot.db
```

```
ls
snapshot.db
```

```
service kube-apiserver stop
Service kube-apiserver stopped
```

```
ETCDCTL_API=3 etcdctl \
   snapshot restore snapshot.db \
   --data-dir /var/lib/etcd-from-backup \
   --initial-cluster master-
1=https://192.168.5.11:2380,master-
2=https://192.168.5.12:2380 \
   --initial-cluster-token etcd-cluster-1 \
   --initial-advertise-peer-urls
https://${INTERNAL_IP}:2380
```

```
I | mvcc: restore compact to 475629
I | etcdserver/membership: added member 5e89ccdfe3
[https://192.168.5.12:2380] to cluster 894c7131f5165a78
I | etcdserver/membership: added member c8246cee7c
[https://192.168.5.11:2380] to cluster 894c7131f5165a78
```

```
systemctl daemon-reload
```

```
service etcd restart
Service etcd restarted
```

etcd.service

```
ExecStart=/usr/local/bin/etcd \\
  --name ${ETCD_NAME} \\
  --cert-file=/etc/etcd/kubernetes.pem \\
  --key-file=/etc/etcd/kubernetes-key.pem \\
  --peer-cert-file=/etc/etcd/kubernetes.pem \\
  --peer-key-file=/etc/etcd/kubernetes-key.pem \\
  --trusted-ca-file=/etc/etcd/ca.pem \\
  --peer-trusted-ca-file=/etc/etcd/ca.pem \\
  --peer-client-cert-auth \\
  --client-cert-auth \\
  --initial-advertise-peer-urls https://${INTERNAL_
  --listen-peer-urls https://${INTERNAL_IP}:2380 \\
  --listen-client-urls https://${INTERNAL_IP}:2379,
  --advertise-client-urls https://${INTERNAL_IP}:23
  --initial-cluster-token etcd-cluster-1    \
  --initial-cluster controller-0=https://${CONTROLL
  --initial-cluster-state new \\
  --data-dir=/var/lib/etcd-from-backup
```

Restore - ETCD

ETCD Cluster

```
ETCDCTL_API=3 etcdctl \
    snapshot save snapshot.db
```

```
ls
snapshot.db
```

```
service kube-apiserver stop
Service kube-apiserver stopped
```

```
ETCDCTL_API=3 etcdctl \
  snapshot restore snapshot.db \
  --data-dir /var/lib/etcd-from-backup \
  --initial-cluster master-1=https://192.168.5.11:2380,master-2=https://192.168.5.12:2380 \
  --initial-cluster-token etcd-cluster-1 \
  --initial-advertise-peer-urls https://${INTERNAL_IP}:2380
I | mvcc: restore compact to 475629
I | etcdserver/membership: added member 5e89ccdfe3 [https://192.168.5.12:2380] to cluster 894c7131f5165a78
I | etcdserver/membership: added member c8246cee7c [https://192.168.5.11:2380] to cluster 894c7131f5165a78
```

```
service kube-apiserver start
Service kube-apiserver started
```

KODEKLOUD

```
systemctl daemon-reload
```

```
service etcd restart
```

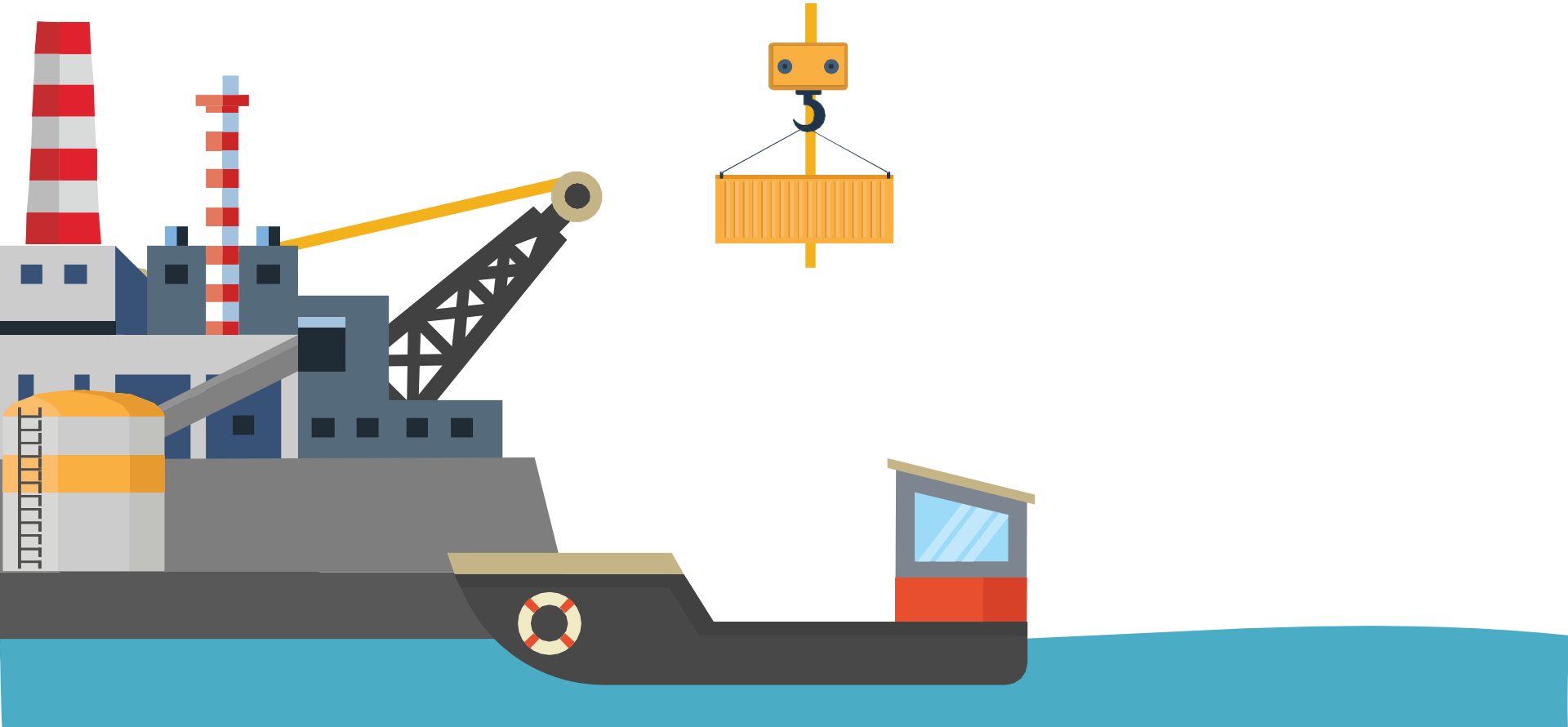
```
Service etcd restarted
```

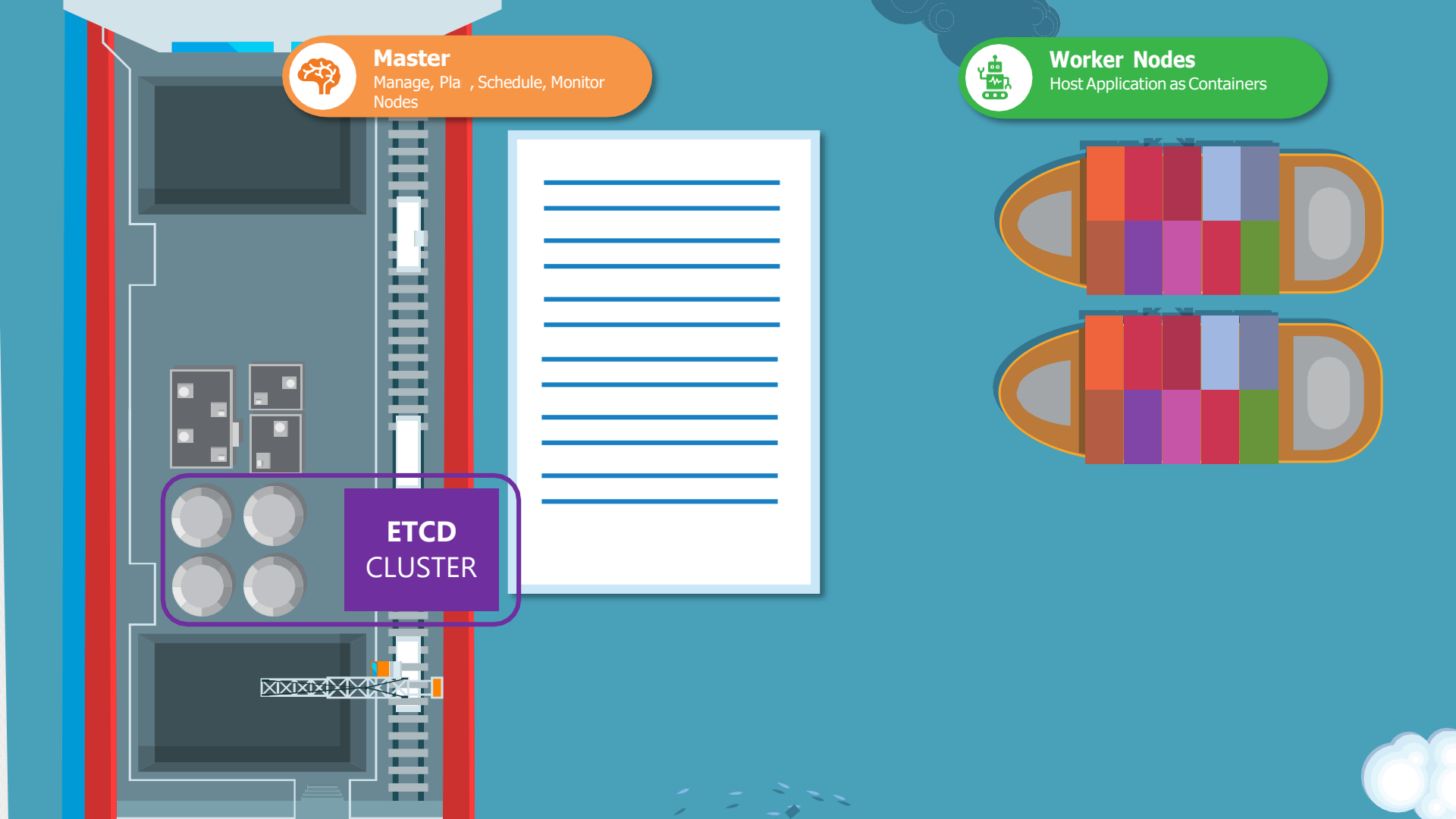
```
ETCDCTL_API=3 etcdctl \
     snapshot save snapshot.db \

     --endpoints=https://127.0.0.1:2379 \
     --cacert=/etc/etcd/ca.crt \
     --cert=/etc/etcd/etcd-server.crt \
     --key=/etc/etcd/etcd-server.key
```

KODEKLOUD

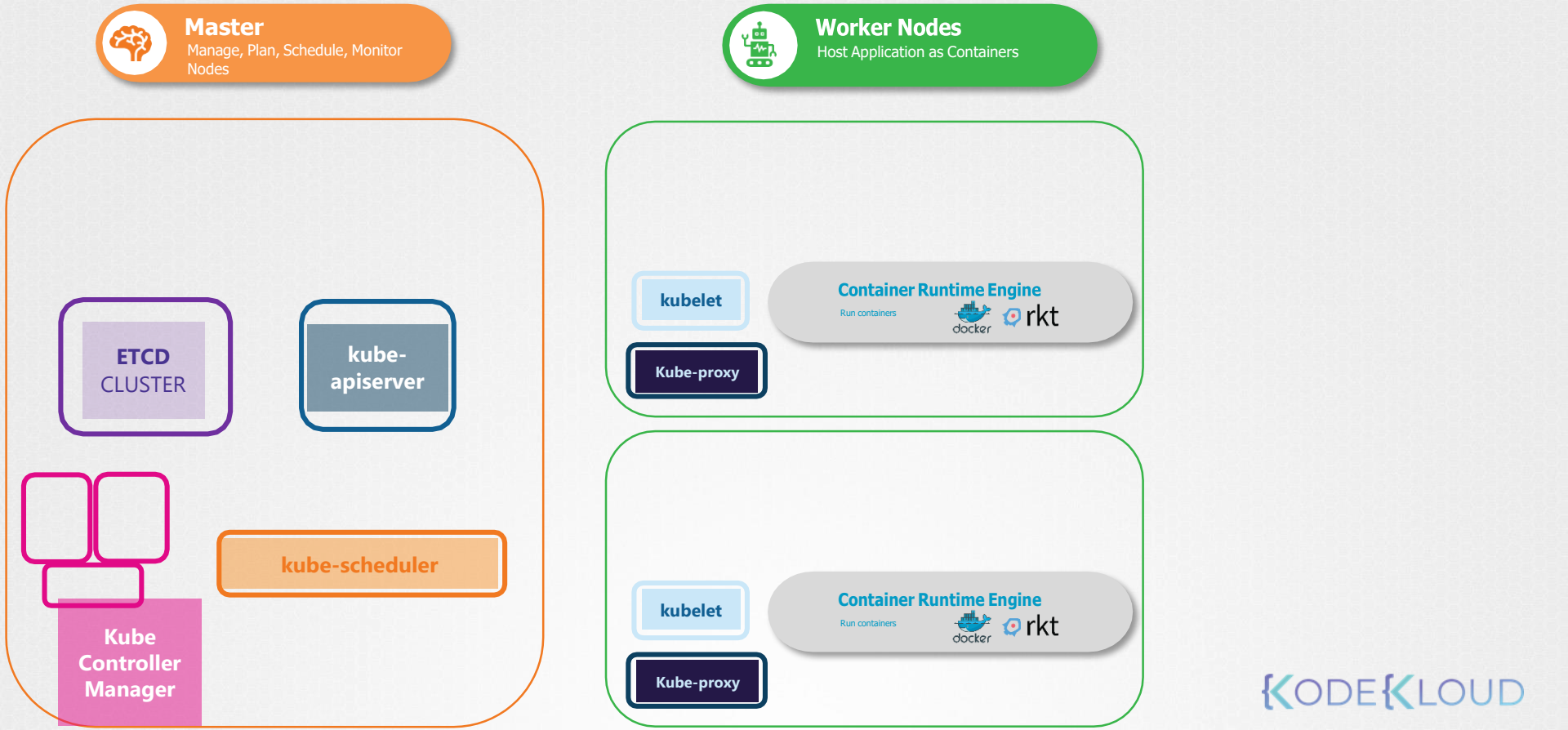
KODEKLOUD

KUBERNETES ARCHITECTURE




Master
Manage, Pla , Schedule, Monitor Nodes
Worker Nodes
Host Application as Containers
ETCD
CLUSTER

Kubernetes Architecture



KODEKLOUD

KODEKLOUD

Course Objectives

- Core Concepts
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
  - Kubernetes Security Primitives
  - Secure Persistent Key Value Store
  - Authentication
  - Authorization
  - Security Contexts
  - TLS Certificates for Cluster Components
  - Images Securely
  - Network Policies
- Storage
- Networking
- Installation, Configuration & Validation
- Troubleshooting

KODEKLOUD

I

SECURITY PRIMITIVES

Secure Hosts

- ❑ Password based authentication disabled
- ❑ SSH Key based authentication

KODEKLOUD

Secure Kubernetes

kube-apiserver

Who can access?

What can they do?

KODEKLOUD

Authentication

Who can access?

- ❑ Files – Username and Passwords
- ❑ Files – Username and Tokens
- ❑ Certificates
- ❑ External Authentication providers - LDAP
- ❑ Service Accounts

KODEKLOUD

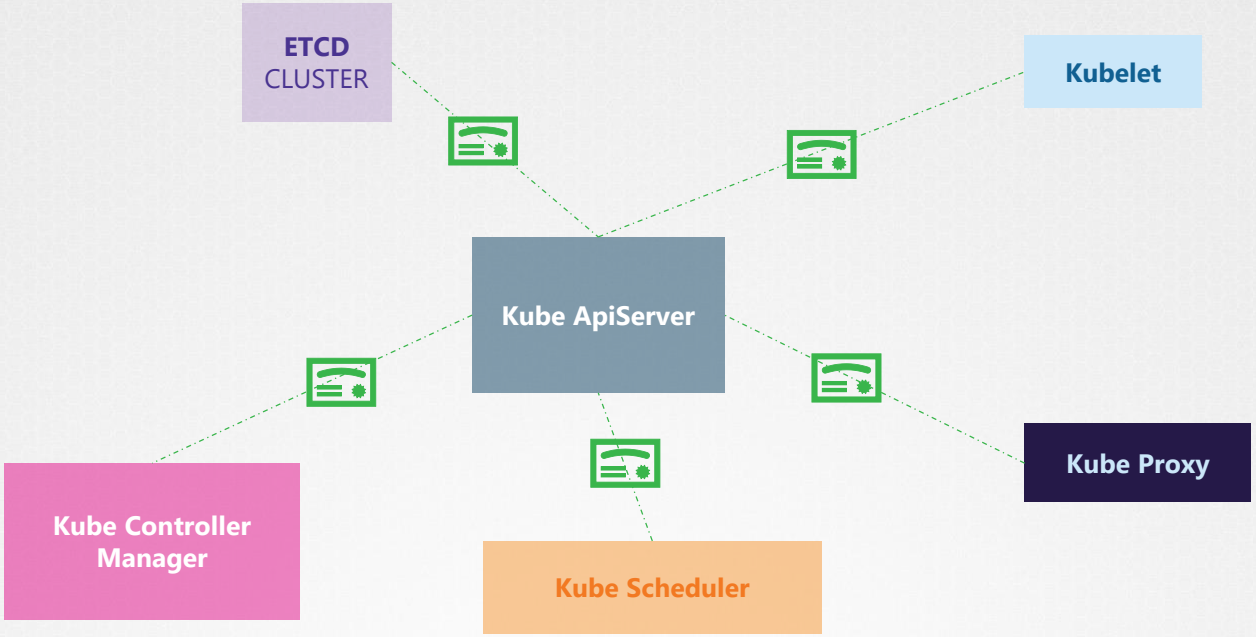
Authorization

What can they do?

- ❑ RBAC Authorization
- ❑ ABAC Authorization
- ❑ Node Authorization
- ❑ Webhook Mode

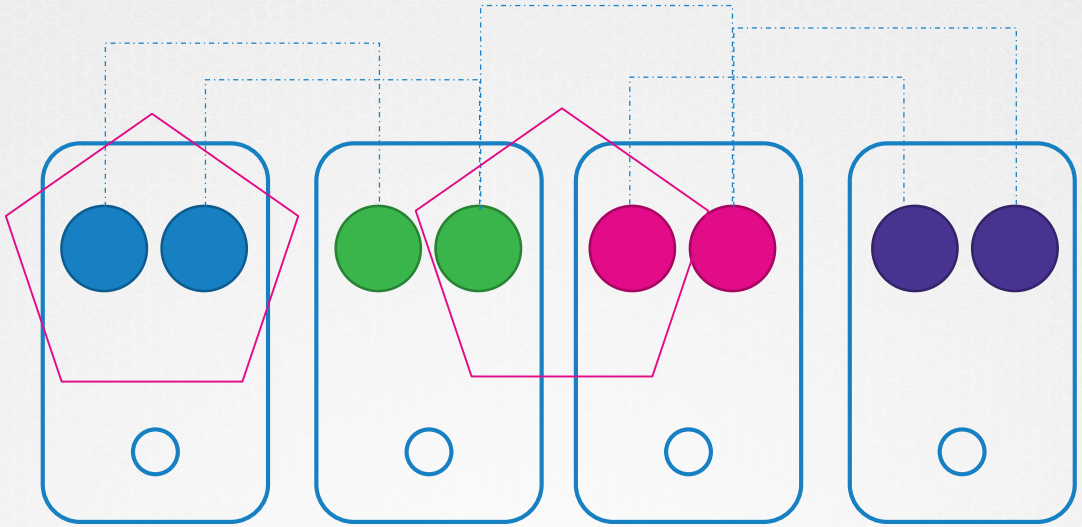
KODEKLOUD

TLS Certificates



KODEKLOUD

Network Policies



KODEKLOUD

KODEKLOUD

Course Objectives

- Core Concepts
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
  - Kubernetes Security Primitives
  - Secure Persistent Key Value Store
  - Authentication
  - Authorization
  - Security Contexts
  - TLS Certificates for Cluster Components
  - Images Securely
  - Network Policies
- Storage
- Networking
- Installation, Configuration & Validation
- Troubleshooting

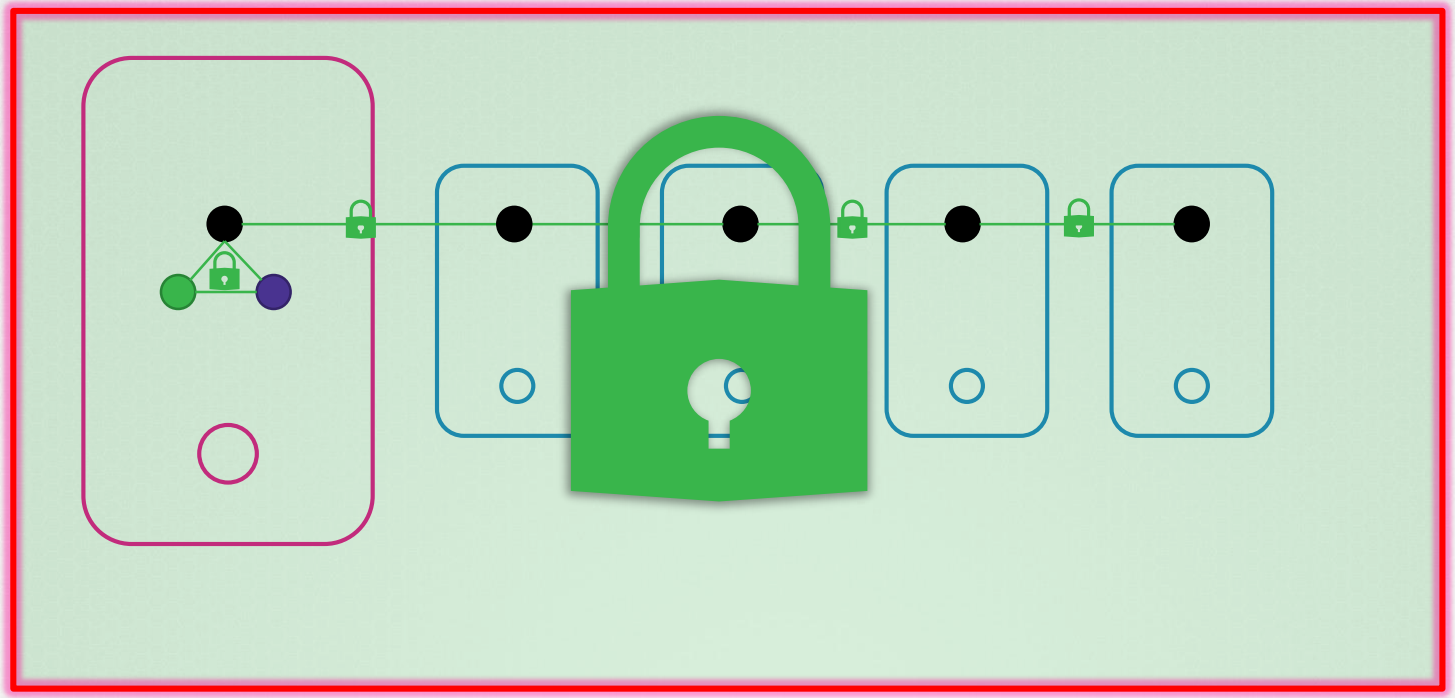
KODEKLOUD

AUTHENTICATION

End Users

Admins

Developers



Bots

KODEKLOUD

Accounts

Admins

Developers

User

Bots

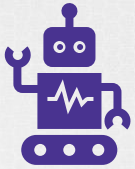
Service Accounts

KODEKLOUD

Accounts

Admins

Developers



Bots

User

Service Accounts

```
kubectl create user user1
user1 Created
```

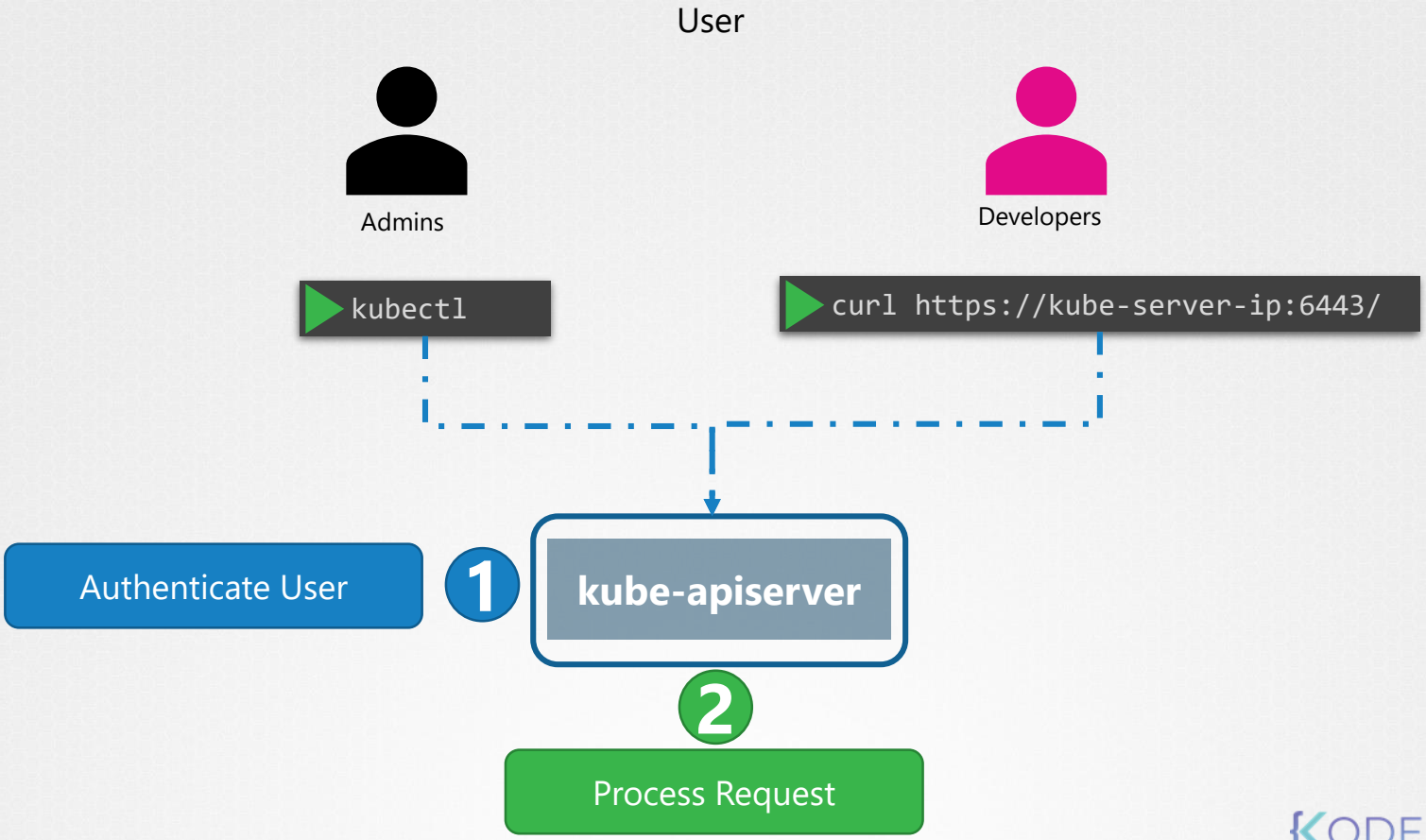
```
kubectl create serviceaccount sa1
Service Account sa1 Created
```

```
kubectl list users
Username
user1
user2
```

```
kubectl list serviceaccount
ServiceAccount
sa1
```

KODEKLOUD

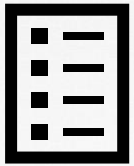
Accounts



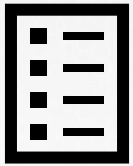
KODEKLOUD

Auth Mechanisms

Static Password File



Static Token File



KODEKLOUD

Auth Mechanisms - Basic

kube-apiserver

```
--basic-auth-file=user-details.csv
```

user-details.csv

```
password123,user1,u0001
password123,user2,u0002
password123,user3,u0003
password123,user4,u0004
password123,user5,u0005
```

kube-apiserver.service

```
ExecStart=/usr/local/bin/kube-apiserver \\
  --advertise-address=${INTERNAL_IP} \\
  --allow-privileged=true \\
  --apiserver-count=3 \\
  --authorization-mode=Node,RBAC \\
  --bind-address=0.0.0.0 \\
  --enable-swagger-ui=true \\
  --etcd-servers=https://127.0.0.1:2379 \\
  --event-ttl=1h \\
  --runtime-config=api/all \\
  --service-cluster-ip-range=10.32.0.0/24 \\
  --service-node-port-range=30000-32767 \\
  --v=2
```

Note: Showing fewer options for simplicity

KODEKLOUD

Kube-api Server Configuration

kube-apiserver.service

```
ExecStart=/usr/local/bin/kube-apiserver \\
  --advertise-address=${INTERNAL_IP} \\
  --allow-privileged=true \\
  --apiserver-count=3 \\
  --authorization-mode=Node,RBAC \\
  --bind-address=0.0.0.0 \\
  --enable-swagger-ui=true \\
  --etcd-servers=https://127.0.0.1:2379 \\
  --event-ttl=1h \\
  --runtime-config=api/all \\
  --service-cluster-ip-range=10.32.0.0/24 \\
  --service-node-port-range=30000-32767 \\
  --v=2
  --basic-auth-file=user-details.csv
```

/etc/kubernetes/manifests/kube-apiserver.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  name: kube-apiserver
  namespace: kube-system
spec:
  containers:
  - command:
    - kube-apiserver
    - --authorization-mode=Node,RBAC
    - --advertise-address=172.17.0.107
    - --allow-privileged=true
    - --enable-admission-plugins=NodeRestriction
    - --enable-bootstrap-token-auth=true

    image: k8s.gcr.io/kube-apiserver-amd64:v1.11.3
    name: kube-apiserver
```

Note: Showing fewer options for simplicity

KODEKLOUD

Authenticate User

```
curl -v -k https://master-node-ip:6443/api/v1/pods -u "user1:password123"
```


```
{
  "kind": "PodList",
  "apiVersion": "v1",
  "metadata": {
    "selfLink": "/api/v1/pods",
    "resourceVersion": "3594"
  },
 "items": [
    {
      "metadata": {
        "name": "nginx-64f497f8fd-krkg6",
        "generateName": "nginx-64f497f8fd-",
        "namespace": "default",
        "selfLink": "/api/v1/namespaces/default/pods/nginx-64f497f8fd-krkg6",
        "uid": "77dd7dfb-2914-11e9-b468-0242ac11006b",
        "resourceVersion": "3569",
        "creationTimestamp": "2019-02-05T07:05:49Z",
        "labels": {
          "pod-template-hash": "2090539498",
          "run": "nginx"
```


KLOUD

Auth Mechanisms - Basic

Static Password File

user-details.csv

```
password123,user1,u0001,group1
password123,user2,u0002,group1
password123,user3,u0003,group2
password123,user4,u0004,group2
password123,user5,u0005,group2
```

Static Token File

user-token-details.csv

```
KpjCVbI7rCFAHYPkByTIzRb7gu1cUc4B,user10,u0010,group1
rJjncHmvtXHc6MlWQddhtvNyyhgTdxSC,user11,u0011,group1
mjpOFIEiFOkL9toikaRNtt59ePtczZSq,user12,u0012,group2
PG41IXhs7QjqwWkmBkvgGT9glOyUqZij,user13,u0013,group2
```

```
--token-auth-file=user-details.csv
```

```
curl -v -k https://master-node-ip:6443/api/v1/pods --header "Authorization: Bearer KpjCVbI7rCFAHYPkBzRb7gu1cUc4B"
```

Note: Showing fewer options for simplicity

KODEKLOUD

Note

- This is not a recommended authentication mechanism
- Consider volume mount while providing the auth file in a kubeadm setup
- Setup Role Based Authorization for the new users

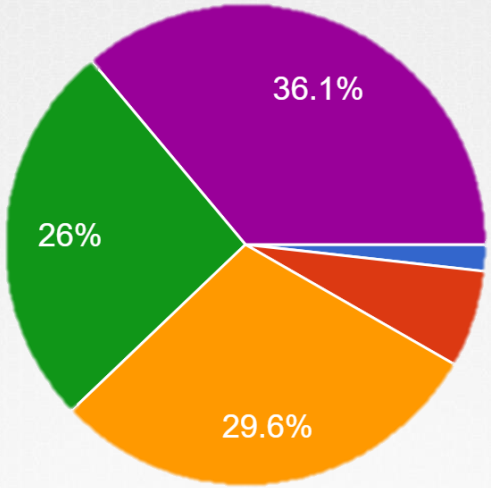
KODEKLOUD

KODEKLOUD

Course Objectives

- Core Concepts
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
  - Kubernetes Security Primitives
  - Secure Persistent Key Value Store
  - Authentication
  - Authorization
  - Security Contexts
  - TLS Certificates for Cluster Components
  - Images Securely
  - Network Policies
- Storage
- Networking
- Installation, Configuration & Validation
- Troubleshooting

KODEKLOUD



No Clue

Not very Comfortable

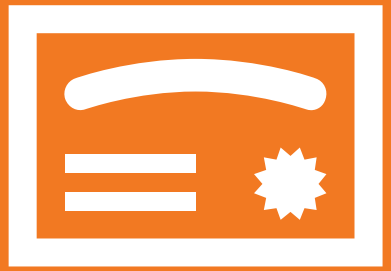
KODEKLOUD

Goals!

- ❑ What are TLS Certificates?
- ❑ How does Kubernetes use Certificates?
- ❑ How to generate them?
- ❑ How to configure them?
- ❑ How to view them?
- ❑ How to troubleshoot issues related to Certificates

KODEKLOUD

TLS CERTIFICATES (PRE-REQ)



User: John
Password: Pass123
LKJSDFK: XZKJSDLF

XKSDJ39K34KJSDF09
34JHSDFSDF3DKSDG

http://my-bank.com

SYMMETRIC
ENCRYPTION

KODEKLOUD

SYMMETRIC ENCRYPTION

KODEKLOUD

KODEKLOUD

ASYMMETRIC ENCRYPTION

Private Key

Public Lock

XCVB: DKSJD
LKJSDFK: XZKJSDLF

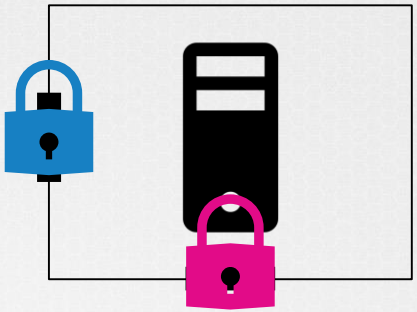
KODEKLOUD

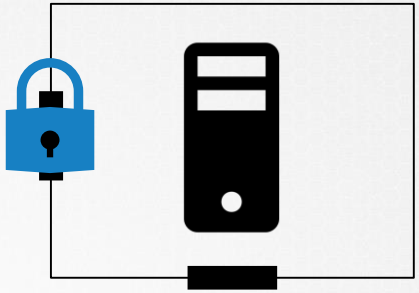
ASYMMETRIC ENCRYPTION - SSH

Private Key

Private Key

Public Lock





```
cat ~/.ssh/authorized_keys
ssh-rsa AAAAB3NzaC1yc…KhtUBfoTzlBqRV1NThvOo4opzEwRQo1mWx user1
ssh-rsa AAAXCVJSDFDF…SLKJSDLKFw23423xckjSDFDFLKJLSDFKJLx user2
```

XCVB: DKSJD
LKJSDFK: XZKJSDLF

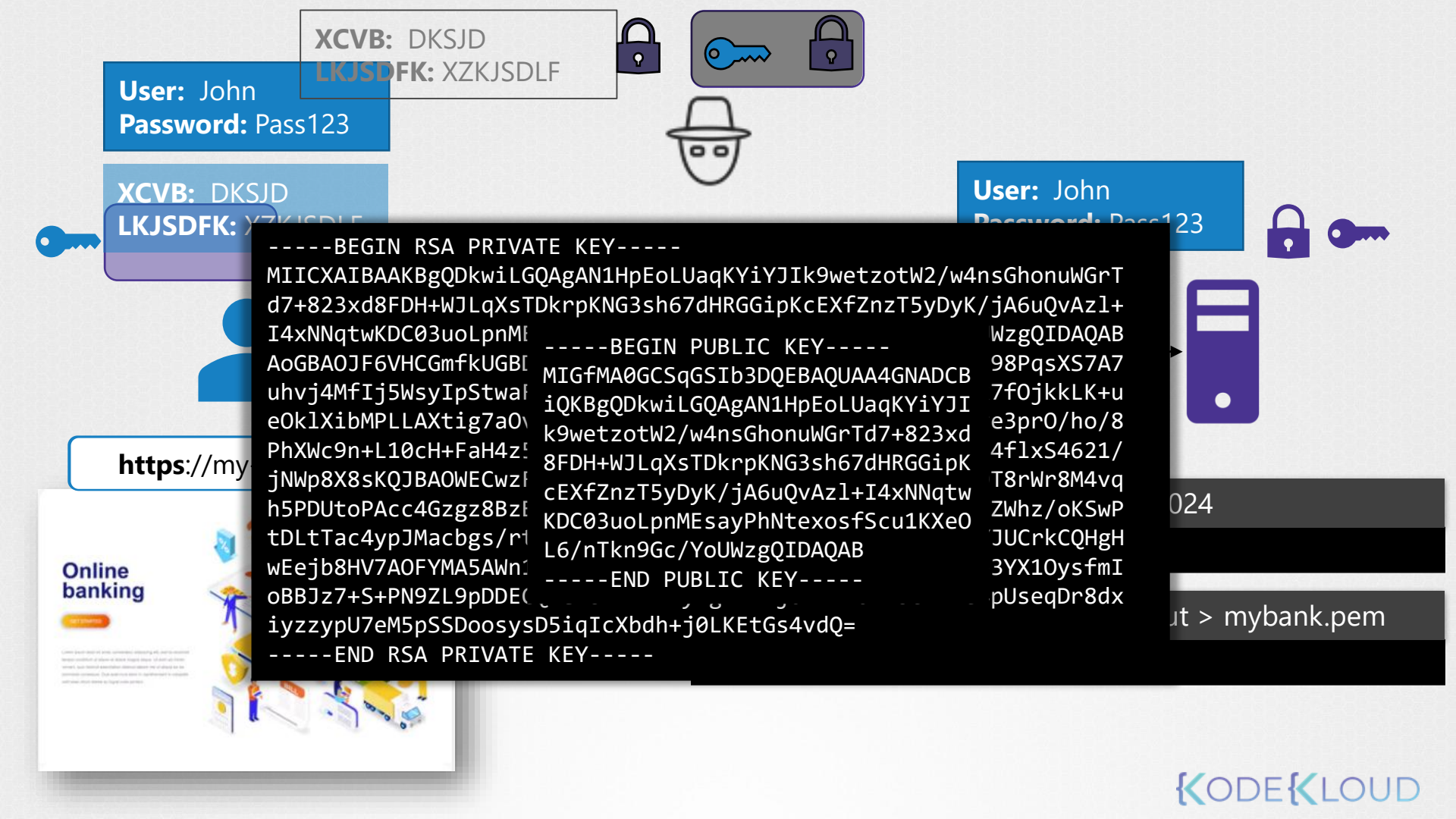
User: John
Password: Pass123
LKJSDFK: XZKJSDLF

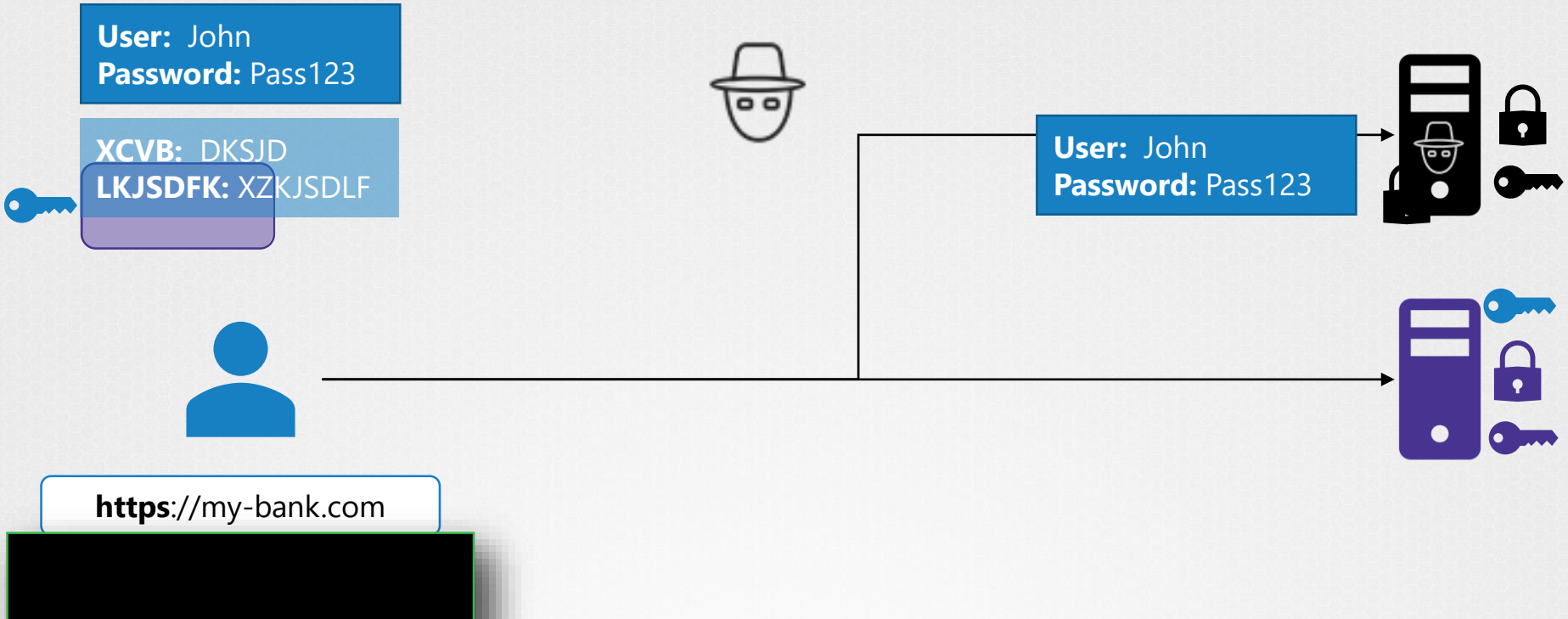
XKSDJ39K34KJSDF09
34JHSDFSDF3DKSDG

http://my-bank.com

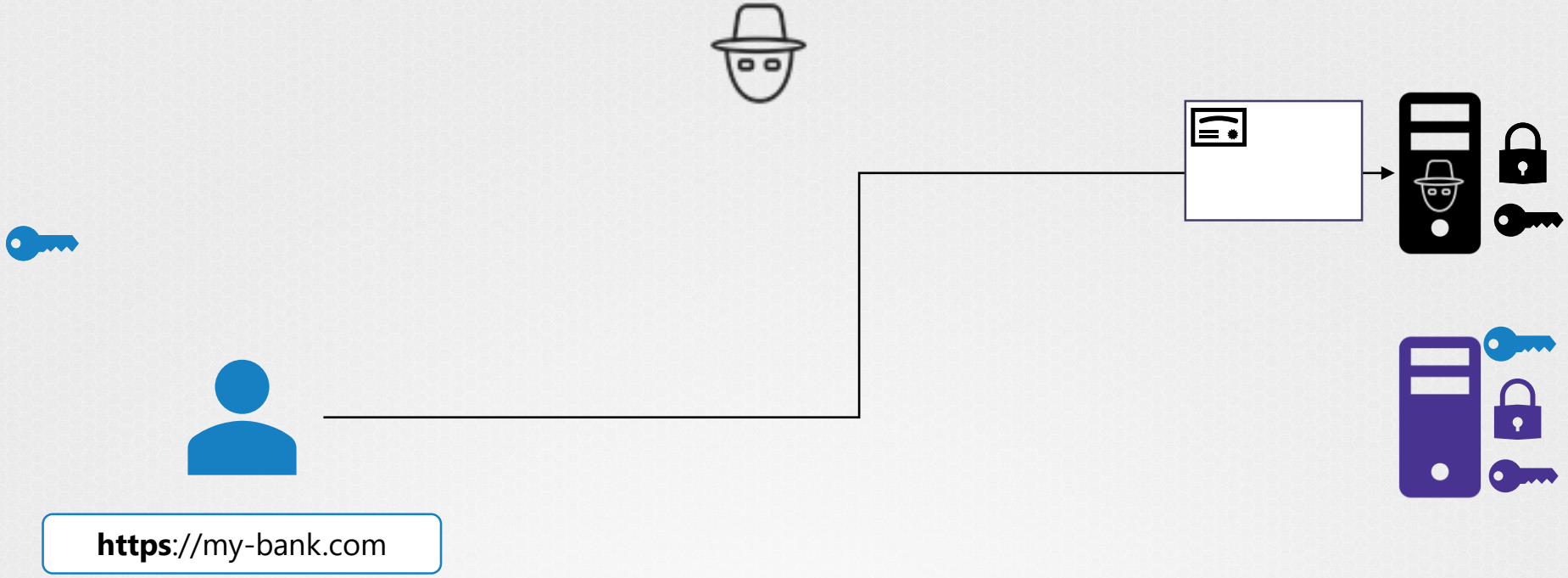
SYMMETRIC
ENCRYPTION

KODEKLOUD


XCVB: DKSJD
LKJSDFK: XZKJSDLF
User: John
Password: Pass123
XCVB: DKSJD
LKJSDFK:
User: John
-----BEGIN RSA PRIVATE KEY-----
MIICXAIBAAKBgQDkwiLGQAgAN1HpEoLUaqKYiYJIk9wetzotW2/w4nsGhonuWGrT
d7+823xd8FDH+WJLqXsTDkrpKNG3sh67dHRGGipKcEXfZnzT5yDyK/jA6uQvAzl+
I4xNNqtwKDC03uoLpnME
AoGBAOJF6VHCGmfkUGBD
uhvj4MfIj5WsyIpStwaF
eOklXibMPLLAXtig7aOv
PhXWc9n+L10cH+FaH4z5
jNWp8X8sKQJBAOWECwzF
h5PDUtoPAcc4Gzgz8BzE
tDLtTac4ypJMacbgs/rt
wEejb8HV7AOFYMA5AWn1
oBBJz7+S+PN9ZL9pDDE
iyzzypU7eM5pSSDoosysD5iqIcXbdh+j0LKEtGs4vdQ=
-----END RSA PRIVATE KEY-----
-----BEGIN PUBLIC KEY-----
MIGfMA0GCSqGSIb3DQEBAQUAA4GNADCB
iQKBgQDkwiLGQAgAN1HpEoLUaqKYiYJI
k9wetzotW2/w4nsGhonuWGrTd7+823xd
8FDH+WJLqXsTDkrpKNG3sh67dHRGGipK
cEXfZnzT5yDyK/jA6uQvAzl+I4xNNqtw
KDC03uoLpnMEsayPhNtexosfScu1KXeO
L6/nTkn9Gc/YoUWzgQIDAQAB
-----END PUBLIC KEY-----
WzgQIDAQAB
98PqsXS7A7
7fOjkkLK+u
e3prO/ho/8
4flxS4621/
T8rWr8M4vq
ZWhz/oKSwP
JUCrkCQHgH
3YX1OysfmI
pUseqDr8dx
https://my
Online banking
024
ut > mybank.pem
KODEKLOUD

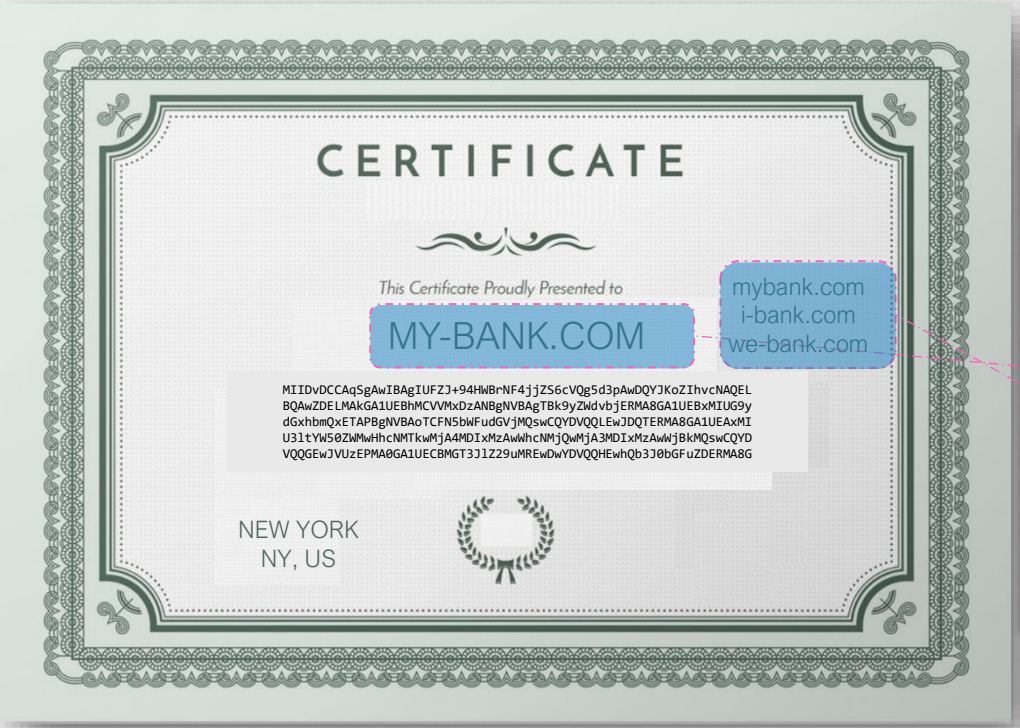

User: John
Password: Pass123
XCVB: DKSJD
LKJSDFK: XZKJSDLF
User: John
Password: Pass123
https://my-bank.com
you have been hacked!

KODEKLOUD



KODEKLOUD

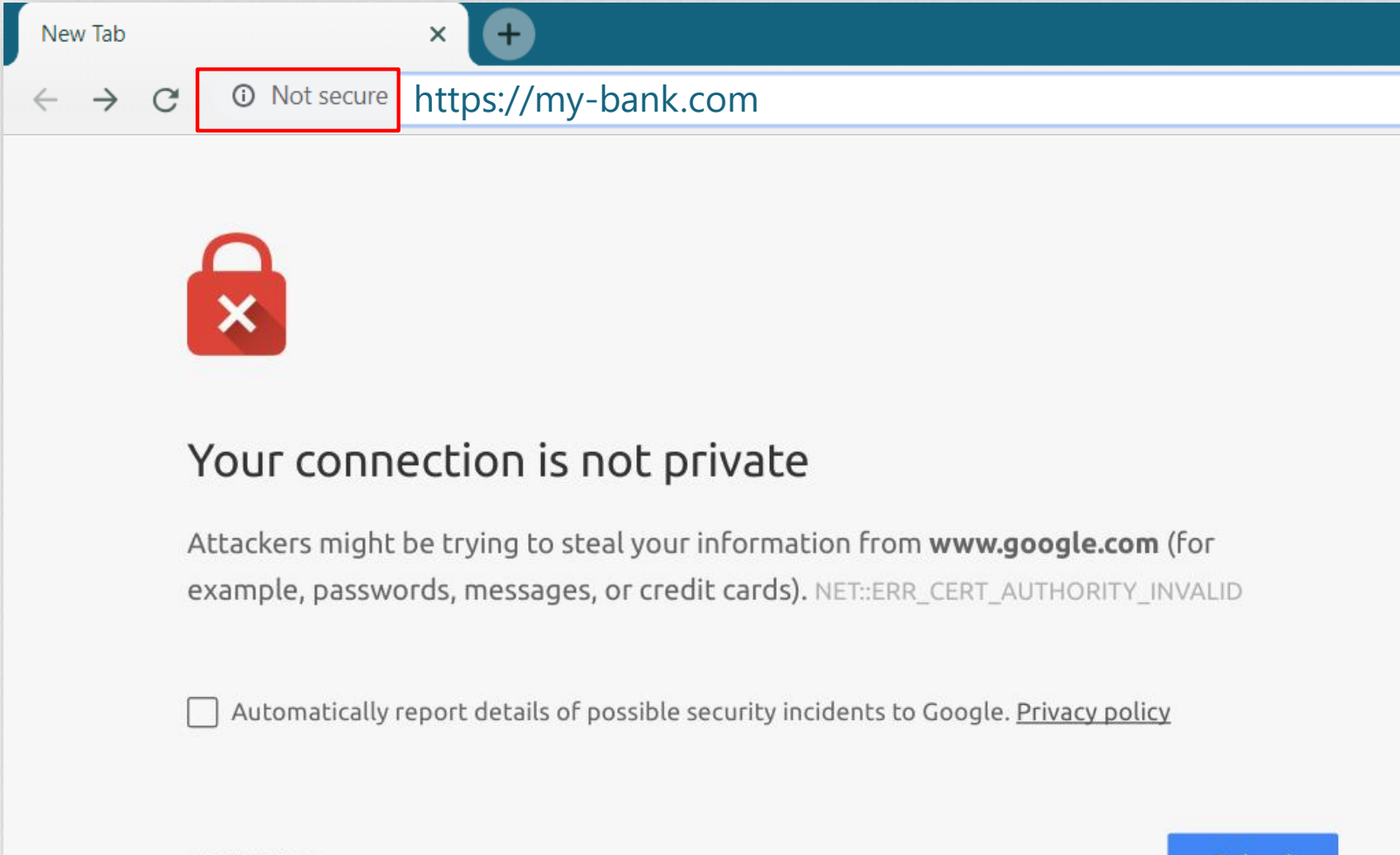
CERTIFICATE
This Certificate Proudly Presented to
MY-BANK.COM
MIIDvDCCAqSgAwIBAgIUFZJ+94HWBrNF4jjZS6cVQg5d3pAwDQYJKoZIhvcNAQEL
BQAwZDELMAkGA1UEBhMCVVMxDzANBgNVBAgTBk9yZWdvbjERMA8GA1UEBxMIUG9y
dGxhbmQxETAPBgNVBAoTCFN5bWFudGVjMQswCQYDVQQLEwJDQTERMA8GA1UEAxMI
U3ltYW50ZWMwHhcNMTkwMjA4MDIxMzAwWhcNMjQwMjA3MDIxMzAwWjBkMQswCQYD
VQQGEwJVUzEPMA0GA1UECBMGT3JlZ29uMREwDwYDVQQHEwhQb3J0bGFuZDERMA8G
NEW YORK
NY, US




```
Certificate:
    Data:
        Serial Number: 420327018966204255
    Signature Algorithm: sha256WithRSAEncryption
        Issuer: CN=kubernetes
        Validity
            Not After : Feb  9 13:41:28 2020 GMT
        Subject: CN=my-bank.com
X509v3  Subject Alternative Name:
                DNS:mybank.com, DNS:i-bank.com,
                DNS:we-bank.com,
        Subject Public Key Info:
                    00:b9:b0:55:24:fb:a4:ef:77:73:
                    7c:9b
```

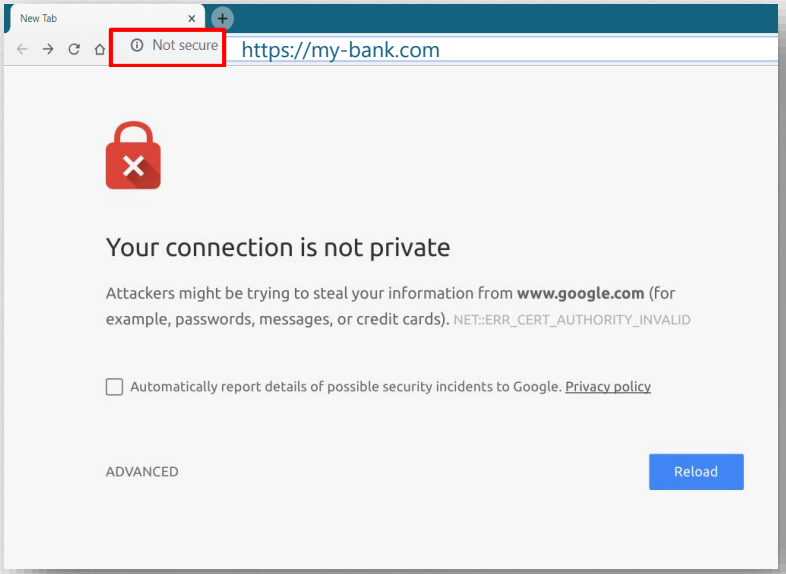
KODEKLOUD

CERTIFICATE
This Certificate Proudly Presented to
MY-BANK.COM
MIIDvDCCAqSgAwIBAgIUFZJ+94HWBrNF4jjZS6cVQg5d3pAwDQYJKoZIhvcNAQEL
BQAwZDELMAkGA1UEBhMCVVMxDzANBgNVBAgTBk9yZWdvbjERMA8GA1UEBxMIUG9y
dGxhbmQxETAPBgNVBAoTCFN5bWFudGVjMQswCQYDVQQLEwJDQTERMA8GA1UEAxMI
U3ltYW50ZWMwHhcNMTkwMjA4MDIxMzAwWhcNMjQwMjA3MDIxMzAwWjBkMQswCQYD
VQQGEwJVUzEPMA0GA1UECBMGT3JlZ29uMREwDwYDVQQHEwhQb3J0bGFuZDERMA8G
NEW YORK
NY, US
Issued by:
MySignature
Self


New Tab
Not secure
https://my-bank.com
Your connection is not private
Attackers might be trying to steal your information from www.google.com (for example, passwords, messages, or credit cards). NET::ERR_CERT_AUTHORITY_INVALID
Automatically report details of possible security incidents to Google. Privacy policy

CERTIFICATE AUTHORITY (CA)

Symantec · GlobalSign · digicert



KODEKLOUD

Symantec

Certificate Signing R

Validate Inforr

Sign and Send C

```
-----BEGIN CERTIFICATE REQUEST-----
MIICjDCCAXQCAQAwRzELMAkGA1UEBhMCVVMxCzAJBgNVBAgMAkNBMRQwEgYDVQQK
DAtNeU9yZywgSW5jLjEVMBMGA1UEAwwMbXlkb21haW4uY29tMIIBIjANBgkqhkiG
9w0BAQEFAAOCAQ8AMIIBCgKCAQEAp8XohAKsHxvjs+/pRKCC2Sqx7O21nuD49Kp4
WDOnDBvxEeXNviY+SuQjpTxmuVr/orIpUC7MHk/fkbIICLT4jrXrBq4MwFfcwla1
n8T0S9A7aLfWKL4rxJGFlU9DAdz4rqGLHXFIC8obLpUWJkTerHpWg++k2UDkuPJE
VQmQJ6Fe/3jWGaMNlnkY/eNyYn+a27NfMd1wQUzs9t5uFPpZbwG81mNjDvVIobA8
yHNfRDNt6gKqvZtv+vGTaMOLfgjedGne2Uq7/Bbq22rSsXgfLM9wHmSpNT57Tjs9
OQSobL4FFzoOnphhSqle1V/cGAjFlCzFIx988fH7xzduw+tRTQIDAQABoAAwDQYJ
KoZIhvcNAQELBQADggEBABtY/tTvjFp4UlUTcI2fl3TFbtYzyIwAYoB7U2sWrjzn
uEe4k2+fosUljXCJxk7EUT4sgGjVtoqJqrFihwQ1SLCViRgTwktLBDtvagViWNnQ
mDJep5YY92JxtAKZZt52wsj8MeUwTUjn6eDuz5NhpoKuiWMf9LoxGFYrgAGi2x1o
Fkse6Zr6zaB/cNdm6daW8m6qVs9hKpudTiqgD3g4MEuLLPK7VNxfFTMoSIfkLUui
OlF8dq2CW/ByrYMhUmONCAkKaag1FwY2WVm55lHY6srcwnCPhszBCri7M5BZf7OE
rgKJPfO6cAhFI7WpeuUz/Oe4Ul2r6YF+Hhk7IDKnLeI=
-----END CERTIFICATE REQUEST-----
```

.com"

KODEKLOUD

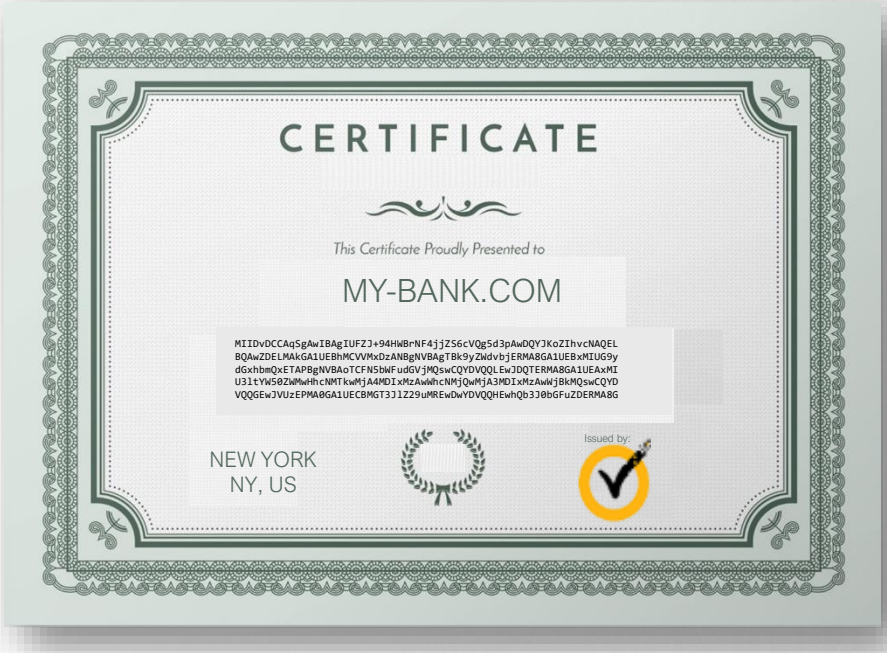
CERTIFICATE AUTHORITY (CA)

Symantec · GlobalSign · digicert

Certificate Signing Request (CSR)

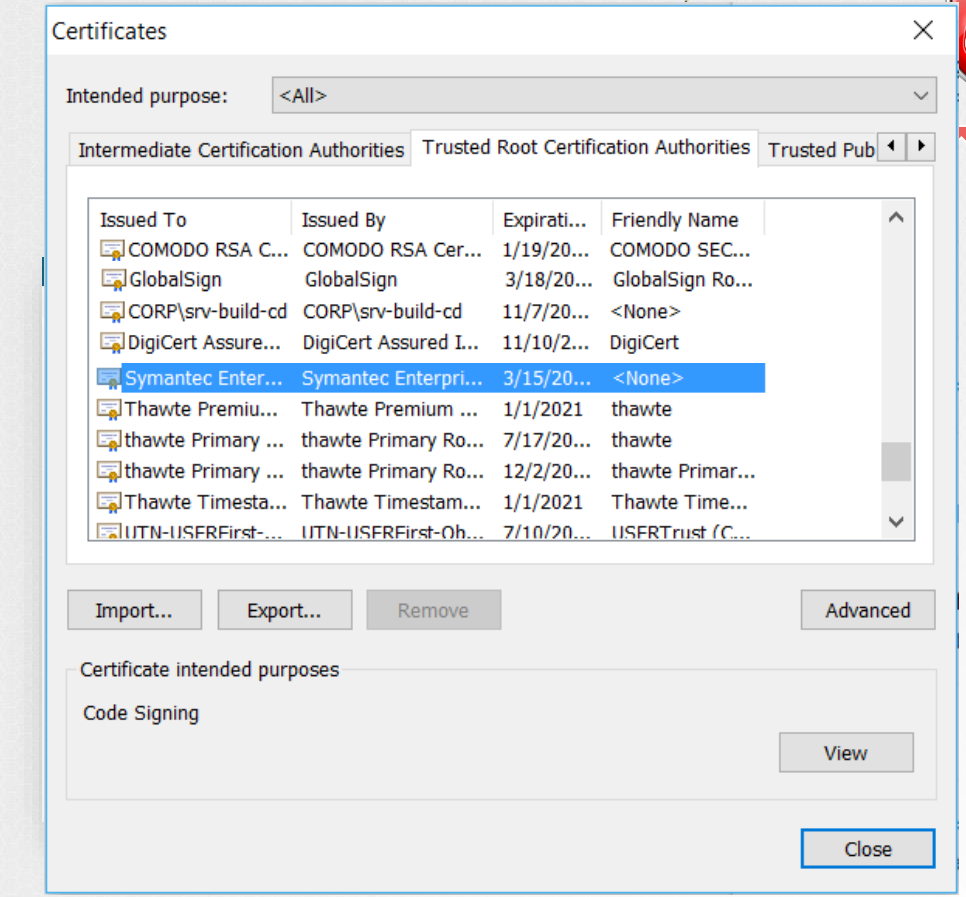
Validate Information

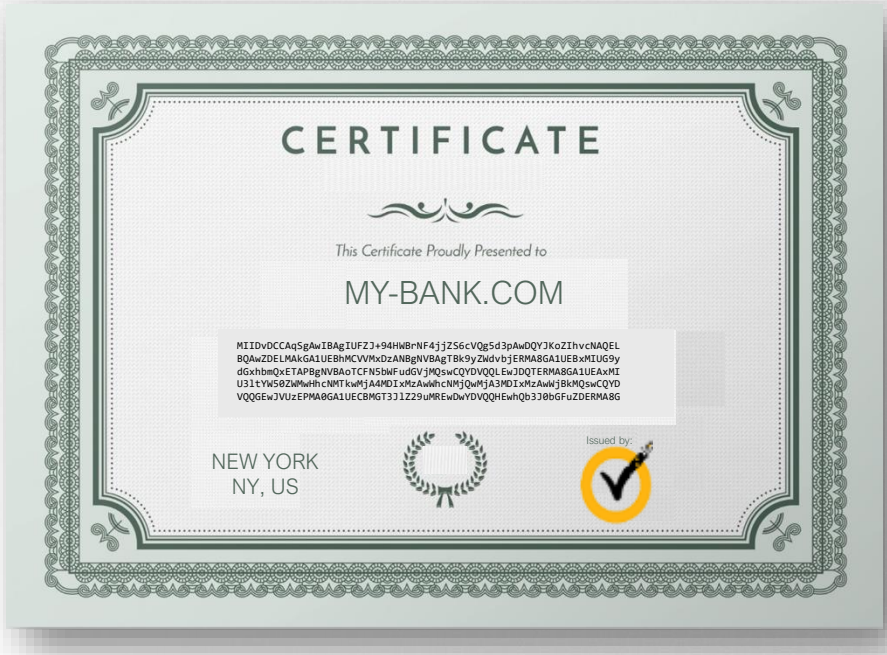
Sign and Send Certificate



KODEKLOUD

CERTIFICATE AUTHORITY (CA)





KODEKLOUD

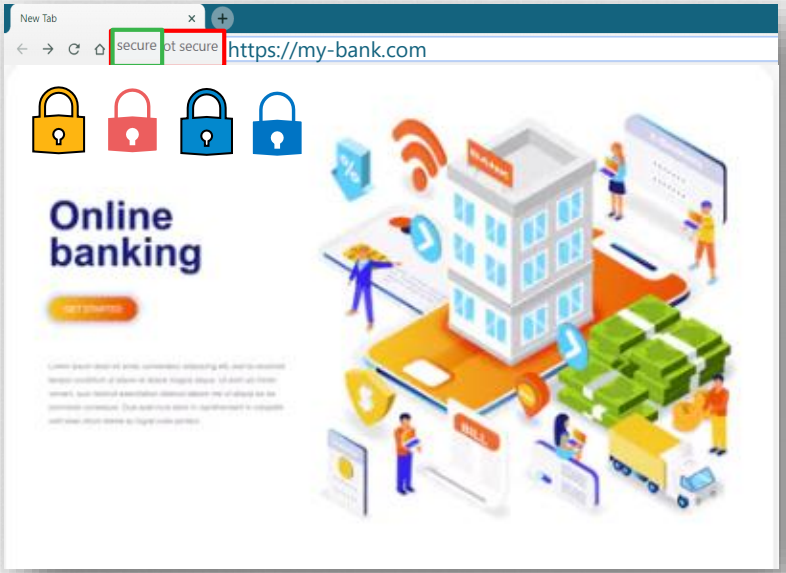
CERTIFICATE AUTHORITY (CA)

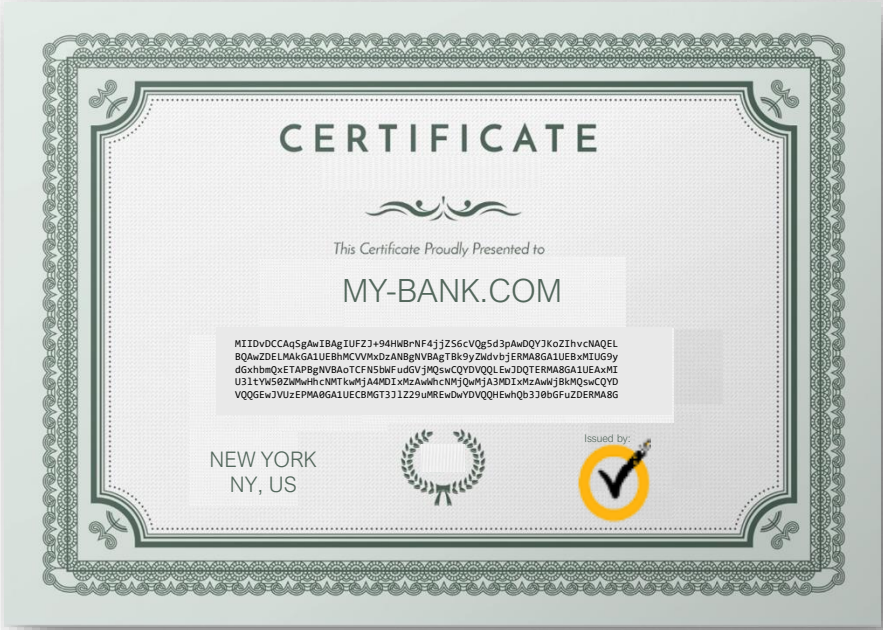
Symantec
Private CA

Symantec

GlobalSign

digicert





KODEKLOUD

CERTIFICATE AUTHORITY (CA)
CERT
Symantec
CERT
PKI
(Public Key Infrastructure)
Symm Key
CSR
CSR
Serving Certificates
Client Certificates
KODEKLOUD

Certificate (Public Key)

*.crt *.pem

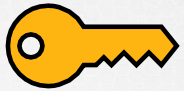
server.crt
server.pem
client.crt
client.pem

Private Key

*.key *-key.pem

server.key
server-key.pem
client.key
client-key.pem

Public Key (Lock)



Private Key

User: John
Password: Pass123

XCVB: DKSJD
LKJSDFK: XZKJSDLF

User: John
Password: Pass123

XCVB: DKSJD
LKJSDFK: XZKJSDLF

KODEKLOUD

KODEKLOUD

Course Objectives

- Core Concepts
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
  - Kubernetes Security Primitives
  - Secure Persistent Key Value Store
  - Authentication
  - Authorization
  - Security Contexts
  - TLS Certificates for Cluster Components
  - Images Securely
  - Network Policies
- Storage
- Networking
- Installation, Configuration & Validation
- Troubleshooting

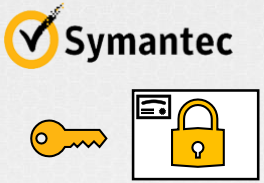
KODEKLOUD

TLS CERTIFICATES

What Certificates?

CERTIFICATE AUTHORITY (CA)

Root Certificates



Client Certificates

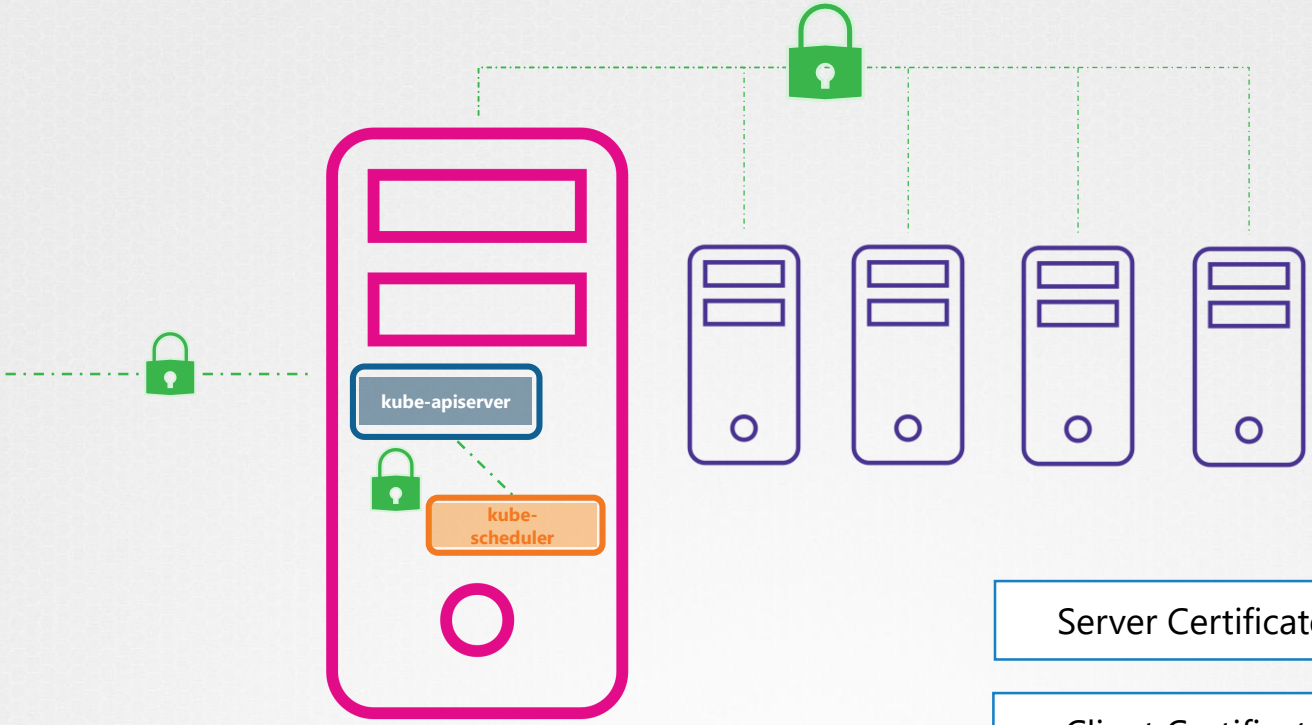
Certificate (Public Key)	Private Key
*.crt *.pem	*.key *-key.pem
server.crt server.pem client.crt client.pem	server.key server-key.pem client.key client-key.pem

Certificate (Public Key)

Private Key

Server Certificates

KODEKLOUD

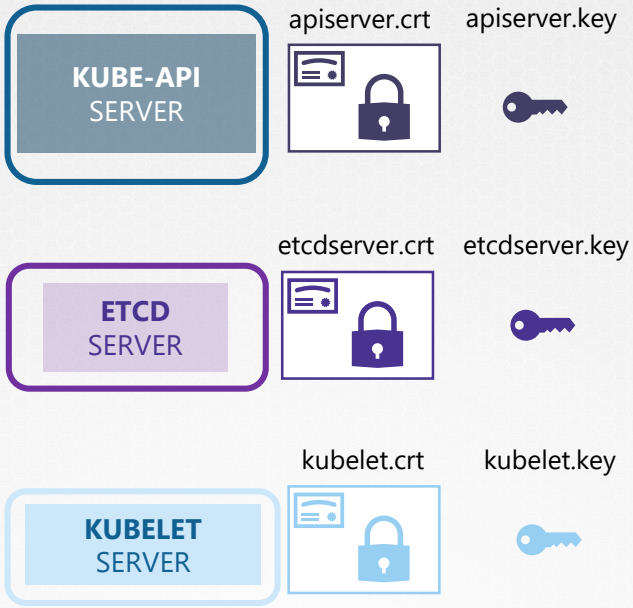


Server Certificates for Servers

Client Certificates for Clients

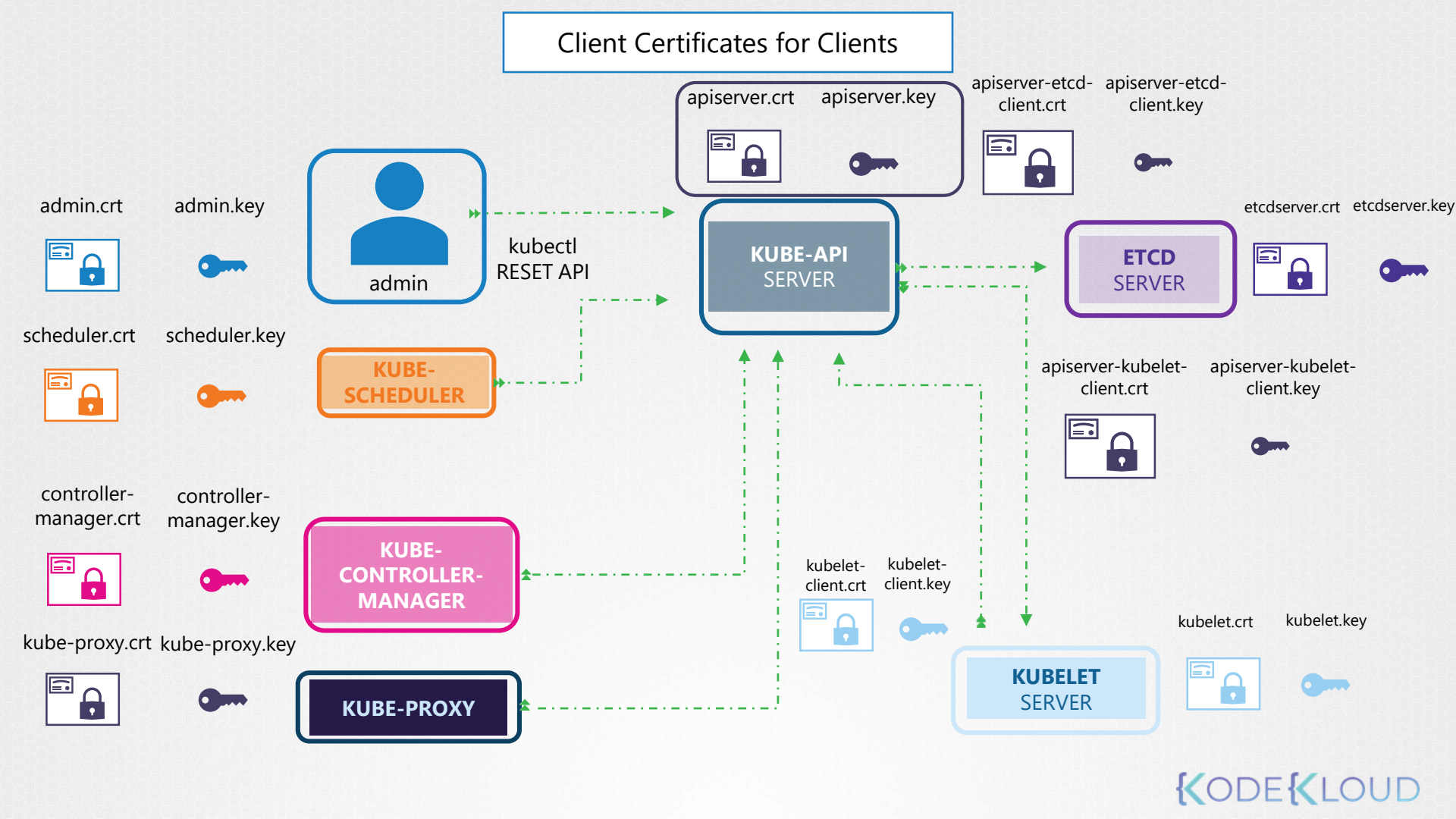
KODEKLOUD

Server Certificates for Servers



KODEKLOUD

Client Certificates for Clients



KODEKLOUD

CERTIFICATE AUTHORITY (CA)

Client Certificates for Clients

admin.crt admin.key

admin

scheduler.crt scheduler.key

KUBE-SCHEDULER

controller-manager.crt controller-manager.key

KUBE-CONTROLLER-MANAGER

kube-proxy.crt kube-proxy.key

KUBE-PROXY

apiserver-kubelet-client.crt apiserver-kubelet-client.key

KUBE-API SERVER

apiserver-etcd-client.crt apiserver-etcd-client.key

KUBE-API SERVER

kubelet-client.crt kubelet-client.key

KUBELET SERVER

Server Certificates for Servers

etcdserver.crt etcdserver.key

ETCD SERVER

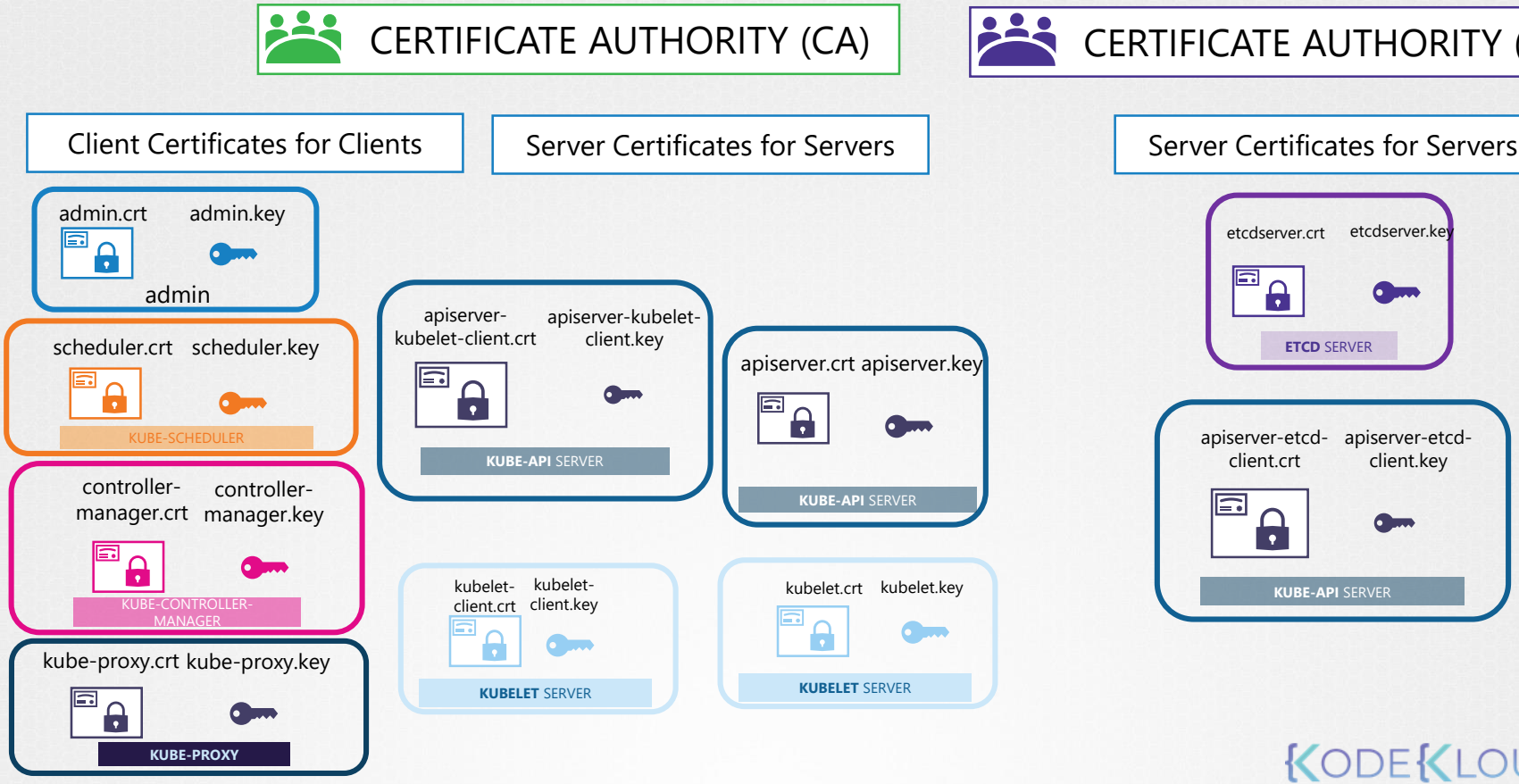
apiserver.crt apiserver.key

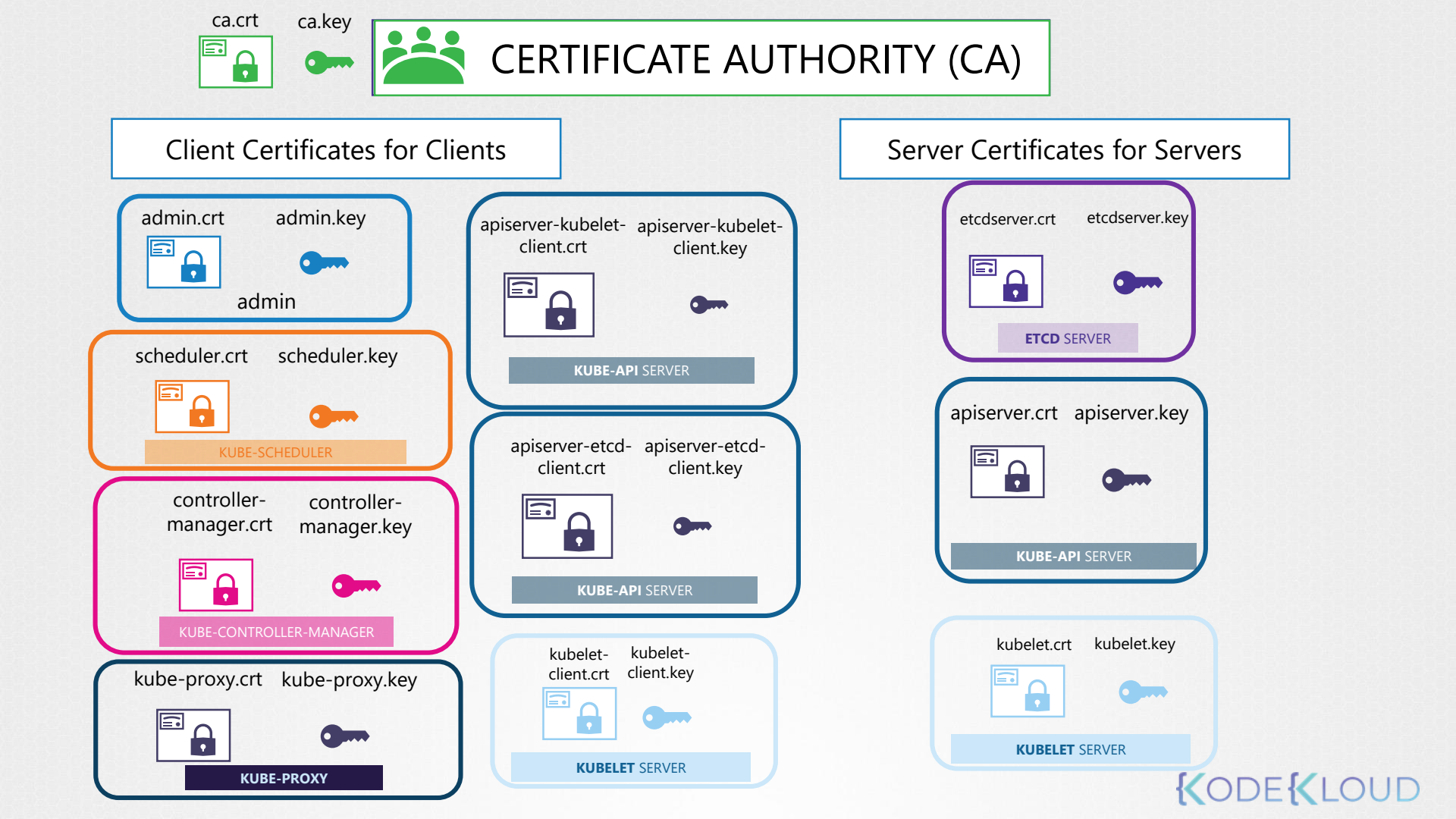
KUBE-API SERVER

kubelet.crt kubelet.key

KUBELET SERVER

KODEKLOUD


CERTIFICATE AUTHORITY (CA)
CERTIFICATE AUTHORITY (CA)
Client Certificates for Clients
Server Certificates for Servers
Server Certificates for Servers
admin.crt
admin.key
admin
scheduler.crt
scheduler.key
KUBE-SCHEDULER
controller-manager.crt
controller-manager.key
KUBE-CONTROLLER-MANAGER
kube-proxy.crt
kube-proxy.key
KUBE-PROXY
apiserver-kubelet-client.crt
apiserver-kubelet-client.key
KUBE-API SERVER
apiserver.crt
apiserver.key
KUBE-API SERVER
kubelet-client.crt
kubelet-client.key
KUBELET SERVER
kubelet.crt
kubelet.key
KUBELET SERVER
etcdserver.crt
etcdserver.key
ETCD SERVER
apiserver-etcd-client.crt
apiserver-etcd-client.key
KUBE-API SERVER


ca.crt
ca.key
CERTIFICATE AUTHORITY (CA)
Client Certificates for Clients
Server Certificates for Servers
admin.crt
admin.key
admin
scheduler.crt
scheduler.key
KUBE-SCHEDULER
controller-manager.crt
controller-manager.key
KUBE-CONTROLLER-MANAGER
kube-proxy.crt
kube-proxy.key
KUBE-PROXY
apiserver-kubelet-client.crt
apiserver-kubelet-client.key
KUBE-API SERVER
apiserver-etcd-client.crt
apiserver-etcd-client.key
KUBE-API SERVER
kubelet-client.crt
kubelet-client.key
KUBELET SERVER
etcdserver.crt
etcdserver.key
ETCD SERVER
apiserver.crt
apiserver.key
KUBE-API SERVER
kubelet.crt
kubelet.key
KUBELET SERVER

KODEKLOUD

Course Objectives

- Core Concepts
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
  - Kubernetes Security Primitives
  - Secure Persistent Key Value Store
  - Authentication
  - Authorization
  - Security Contexts
  - TLS Certificates for Cluster Components
  - Images Securely
  - Network Policies
- Storage
- Networking
- Installation, Configuration & Validation
- Troubleshooting

KODEKLOUD

TLS CERTIFICATES

Generate Certificates

KODEKLOUD

EASYRSA

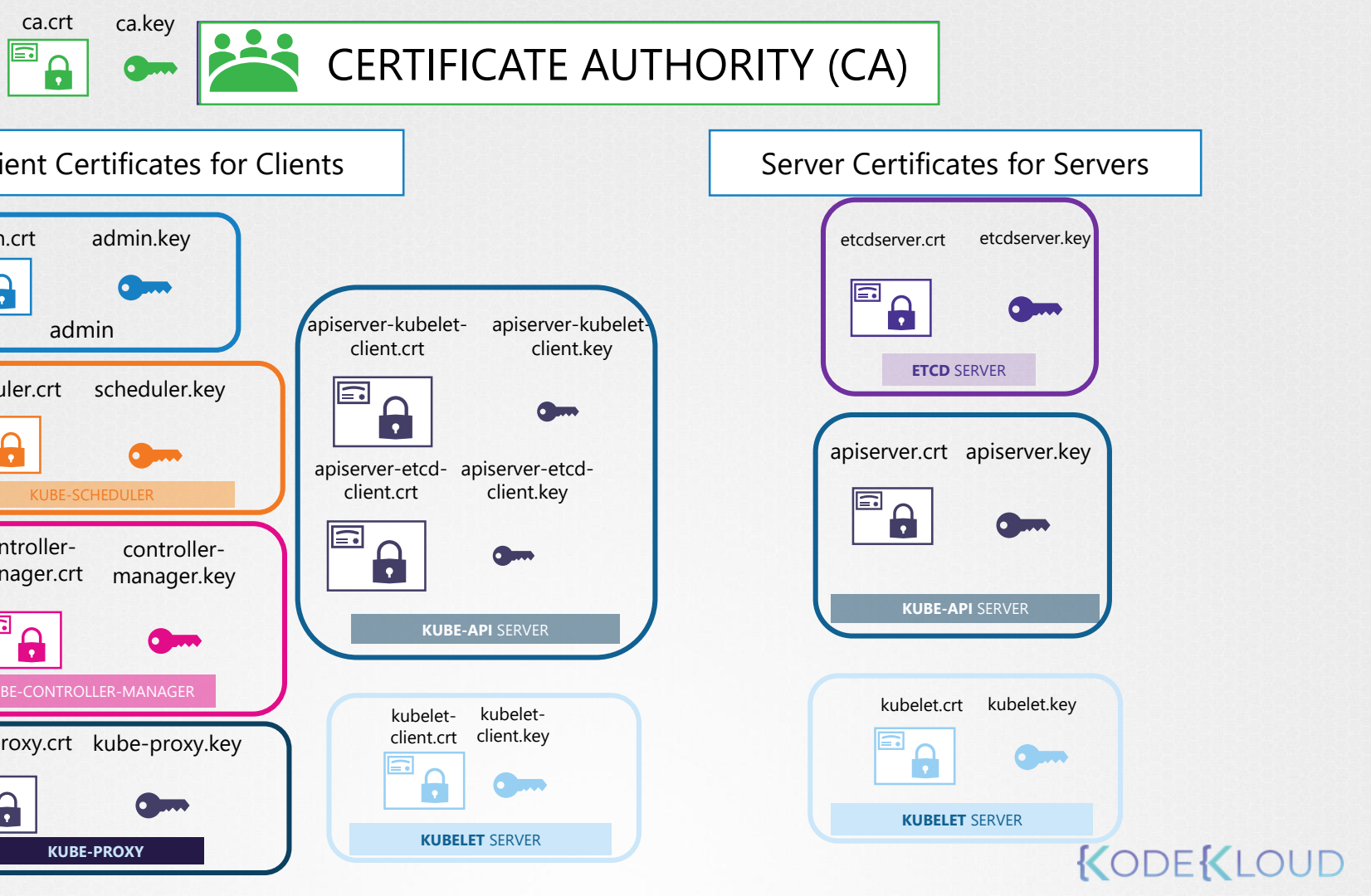
OPENSSL

CFSSL

KODEKLOUD

OPENSSL

KODEKLOUD


ca.crt
ca.key
CERTIFICATE AUTHORITY (CA)
Client Certificates for Clients
admin.crt
admin.key
admin
scheduler.crt
scheduler.key
KUBE-SCHEDULER
controller-manager.crt
controller-manager.key
KUBE-CONTROLLER-MANAGER
kube-proxy.crt
kube-proxy.key
KUBE-PROXY
apiserver-kubelet-client.crt
apiserver-kubelet-client.key
apiserver-etcd-client.crt
apiserver-etcd-client.key
KUBE-API SERVER
kubelet-client.crt
kubelet-client.key
KUBELET SERVER
Server Certificates for Servers
etcdserver.crt
etcdserver.key
ETCD SERVER
apiserver.crt
apiserver.key
KUBE-API SERVER
kubelet.crt
kubelet.key
KUBELET SERVER

CERTIFICATE AUTHORITY (CA)

Generate Keys

ca.key

```
openssl genrsa -out ca.key 2048
ca.key
```

```
new -key ca.key -subj  "/CN=KUBERNETES-CA" -out ca.csr
```

```
-req -in ca.csr -signkey ca.key -out ca.crt
```

KODEKLOUD

ca.key

ca.crt

ADMIN USER

admin.key

Generate Keys

```
openssl genrsa -out admin.key 2048
admin.key
```

admin.csr

Certificate Signing Request

```
openssl req -new -key admin.key -subj  \
        "/CN=kube-admin/OU=system:masters" -out admin.csr
```

```
-in admin.csr –CA ca.crt -CAkey ca.key -out admin.crt
```

KODEKLOUD

KUBE SCHEDULER

ca.key

ca.crt

Generate Keys

Certificate Signing Request

Sign Certificates

scheduler.key

scheduler.csr

scheduler.crt

KODEKLOUD

KUBE CONTROLLER MANGER

ca.key

ca.crt

Generate Keys

Certificate Signing Request

Sign Certificates

controller-manager.key

controller-manager.csr

controller-manager.crt

KODEKLOUD

KUBE PROXY

ca.key

ca.crt

Generate Keys

Certificate Signing Request

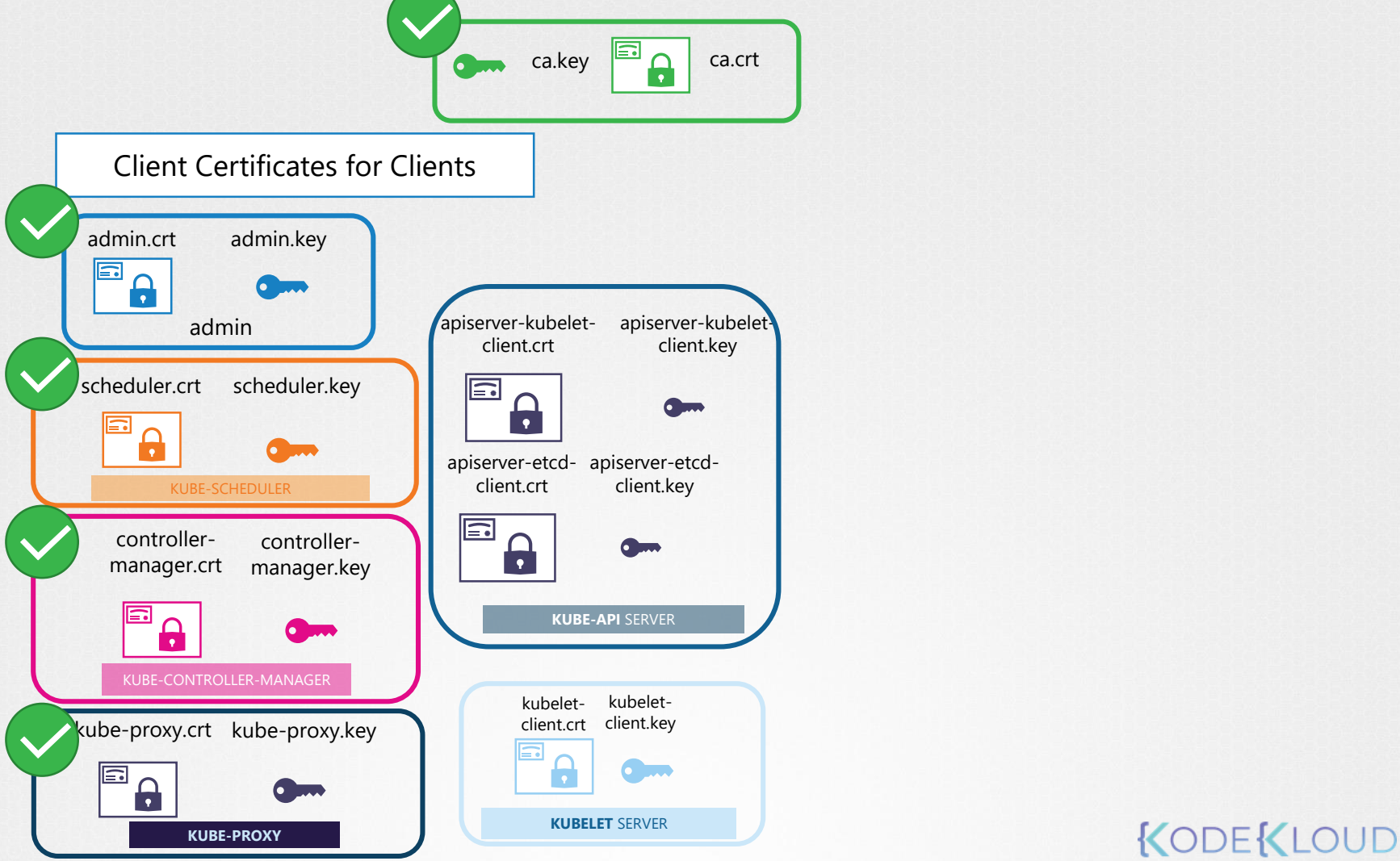
Sign Certificates

kube-proxy.key

kube-proxy.csr

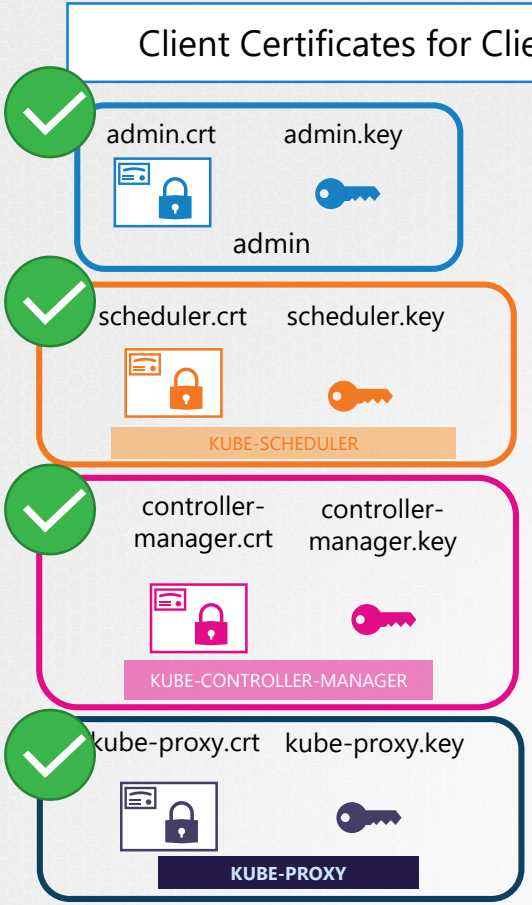
kube-proxy.crt

KODEKLOUD


ca.key
ca.crt
Client Certificates for Clients
admin.crt
admin.key
admin
scheduler.crt
scheduler.key
KUBE-SCHEDULER
controller-manager.crt
controller-manager.key
KUBE-CONTROLLER-MANAGER
kube-proxy.crt
kube-proxy.key
KUBE-PROXY
apiserver-kubelet-client.crt
apiserver-kubelet-client.key
apiserver-etcd-client.crt
apiserver-etcd-client.key
KUBE-API SERVER
kubelet-client.crt
kubelet-client.key
KUBELET SERVER

KODEKLOUD

Client Certificates for Clients



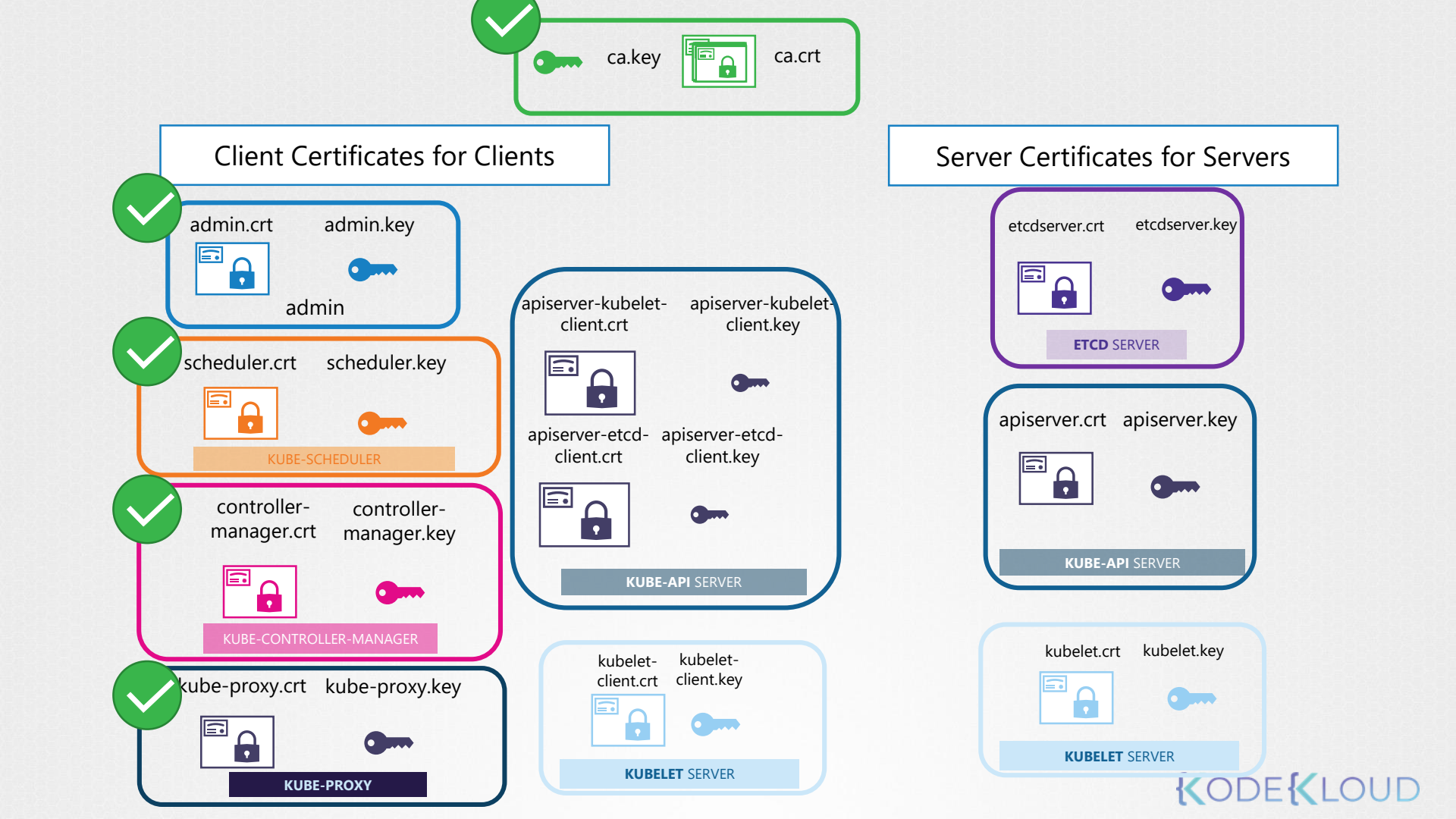

```
curl https://kube-apiserver:6443/api/v1/pods \
  --key admin.key --cert admin.crt
  --cacert ca.crt
```

```
{
  "kind": "PodList",
  "apiVersion": "v1",
  "metadata": {
    "selfLink": "/api/v1/pods",
  },
  "items": []
}
```

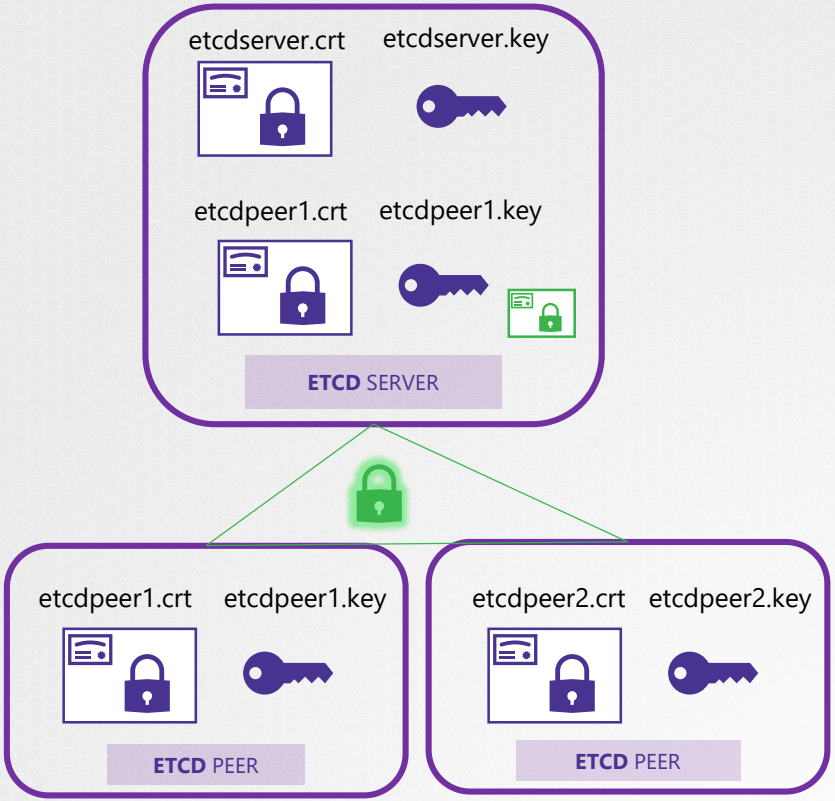
kube-config.yaml

```yaml
apiVersion: v1
clusters:
- cluster:
    certificate-authority: ca.crt
    server: https://kube-apiserver:6443
  name: kubernetes
kind: Config
users:
- name: kubernetes-admin
  user:
    client-certificate: admin.crt
    client-key: admin.key
```

KUBERNETES

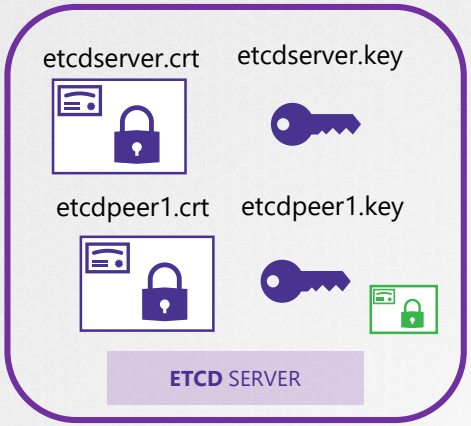

ca.key
ca.crt
Client Certificates for Clients
Server Certificates for Servers
admin.crt
admin.key
admin
scheduler.crt
scheduler.key
KUBE-SCHEDULER
controller-manager.crt
controller-manager.key
KUBE-CONTROLLER-MANAGER
kube-proxy.crt
kube-proxy.key
KUBE-PROXY
apiserver-kubelet-client.crt
apiserver-kubelet-client.key
apiserver-etcd-client.crt
apiserver-etcd-client.key
KUBE-API SERVER
kubelet-client.crt
kubelet-client.key
KUBELET SERVER
etcdserver.crt
etcdserver.key
ETCD SERVER
apiserver.crt
apiserver.key
KUBE-API SERVER
kubelet.crt
kubelet.key
KUBELET SERVER

ETCD SERVERS



KODEKLOUD

ETCD SERVERS

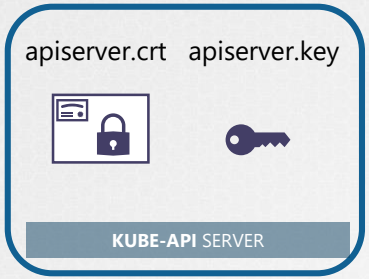



```
cat etcd.yaml
```

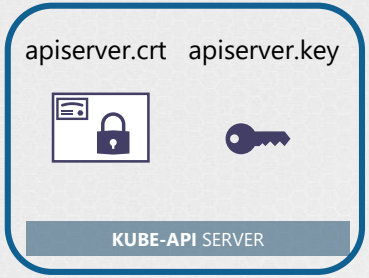
```
- etcd
    - --advertise-client-urls=https://127.0.0.1:2379
    - --key-file=/path-to-certs/etcdserver.key
    - --cert-file=/path-to-certs/etcdserver.crt
    - --client-cert-auth=true
    - --data-dir=/var/lib/etcd
    - --initial-advertise-peer-urls=https://127.0.0.1:2380
    - --initial-cluster=master=https://127.0.0.1:2380
    - --listen-client-urls=https://127.0.0.1:2379
    - --listen-peer-urls=https://127.0.0.1:2380
    - --name=master
    - --peer-cert-file=/path-to-certs/etcdpeer1.crt
    - --peer-client-cert-auth=true
    - --peer-key-file=/etc/kubernetes/pki/etcd/peer.key
    - --peer-trusted-ca-file=/etc/kubernetes/pki/etcd/ca.crt
    - --snapshot-count=10000
    - --trusted-ca-file=/etc/kubernetes/pki/etcd/ca.crt
```

KODEKLOUD

KUBE API SERVER



KODEKLOUD



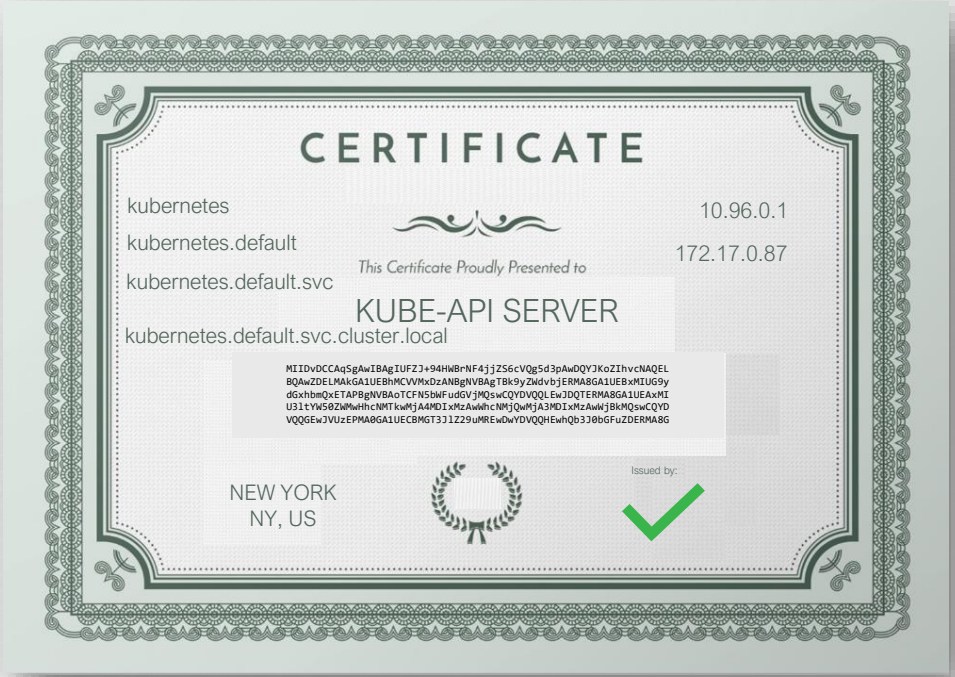
KUBE API SERVER

```
openssl genrsa -out apiserver.key 2048
```

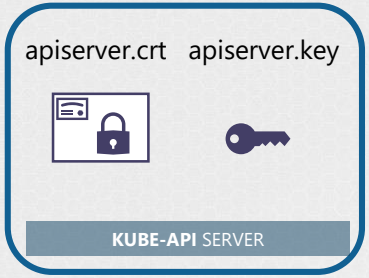
```
apiserver.key
```

```
openssl req -new -key apiserrver.key -subj \
        "/CN=kube-apiserver" -out apiserver.csr
```

```
apiserver.csr
```



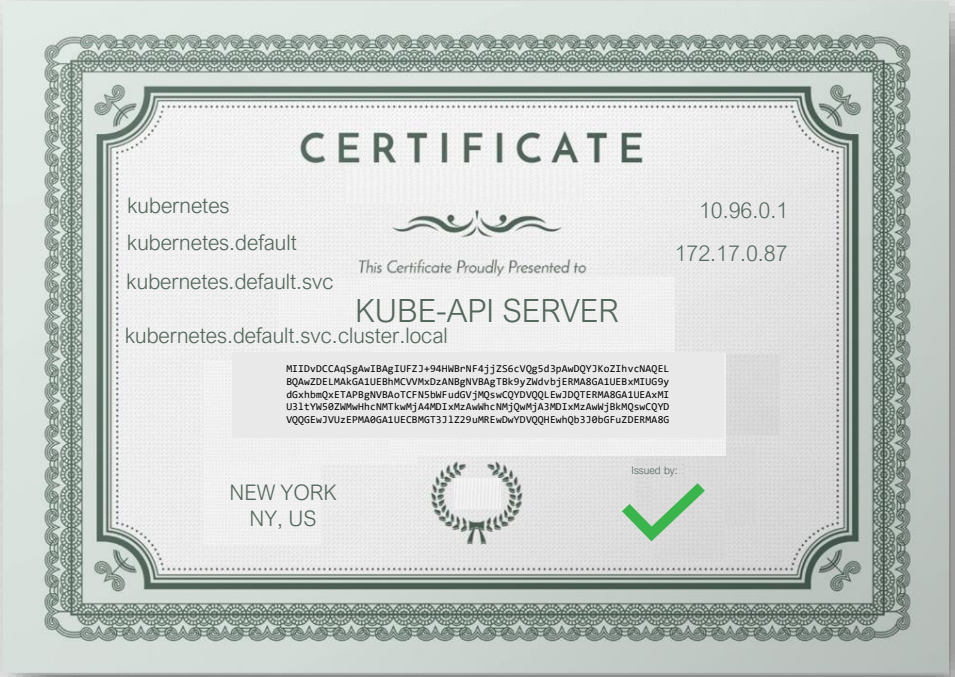
KODEKLOUD



KUBE API SERVER

```
openssl req -new -key apiserver.key -subj \
    "/CN=kube-apiserver" -out apiserver.csr –config openssl.cnf
```

apiserver.csr



openssl.cnf

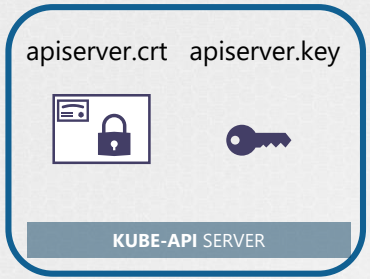
```
[req]
req_extensions = v3_req
[ v3_req ]
basicConstraints = CA:FALSE
keyUsage = nonRepudiation,
subjectAltName = @alt_names
[alt_names]
DNS.1 = kubernetes
DNS.2 = kubernetes.default
DNS.3 = kubernetes.default.svc
DNS.4 = kubernetes.default.svc.cluster.local
IP.1 = 10.96.0.1
IP.2 = 172.17.0.87
```

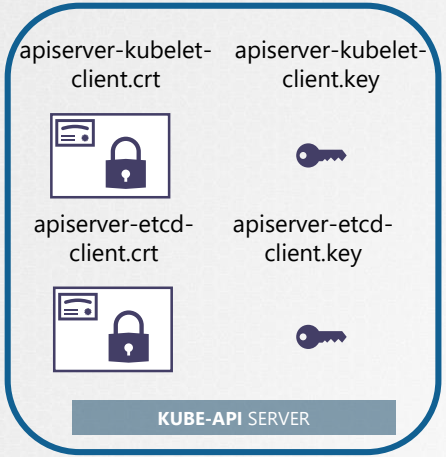
```
openssl x509 -req -in apiserver.csr \
    -CA ca.crt -CAkey ca.key -out apiserver.crt
```

apiserver.crt

KODEKLOUD

KUBE API SERVER






```
ExecStart=/usr/local/bin/kube-apiserver \\
  --advertise-address=${INTERNAL_IP} \\
  --allow-privileged=true \\
  --apiserver-count=3 \\
  --authorization-mode=Node,RBAC \\
  --bind-address=0.0.0.0 \\
  --enable-swagger-ui=true \\
  --etcd-cafile=/var/lib/kubernetes/ca.pem \\
  --etcd-certfile=/var/lib/kubernetes/apiserver-etcd-client.crt \\
  --etcd-keyfile=/var/lib/kubernetes/apiserver-etcd-client.key \\
  --etcd-servers=https://127.0.0.1:2379 \\
  --event-ttl=1h \\
  --kubelet-certificate-authority=/var/lib/kubernetes/ca.pem \\
  --kubelet-client-certificate=/var/lib/kubernetes/apiserver-etcd-client.crt \\
  --kubelet-client-key=/var/lib/kubernetes/apiserver-etcd-client.key \\
  --kubelet-https=true \\
  --runtime-config=api/all \\
  --service-account-key-file=/var/lib/kubernetes/service-account.pem \\
  --service-cluster-ip-range=10.32.0.0/24 \\
  --service-node-port-range=30000-32767 \\
  --client-ca-file=/var/lib/kubernetes/ca.pem \\
  --tls-cert-file=/var/lib/kubernetes/apiserver.crt \\
  --tls-private-key-file=/var/lib/kubernetes/apiserver.key \\
  --v=2
```

KODEKLOUD

KUBECTL NODES (SERVER CERT)

kubelet-config.yaml (node01)

```yaml
kind: KubeletConfiguration
apiVersion: kubelet.config.k8s.io/v1beta1
authentication:
  x509:
    clientCAFile: "/var/lib/kubernetes/ca.pem"
authorization:
  mode: Webhook
clusterDomain: "cluster.local"
clusterDNS:
  - "10.32.0.10"
podCIDR: "${POD_CIDR}"
resolvConf: "/run/systemd/resolve/resolv.conf"
runtimeRequestTimeout: "15m"
tlsCertFile: "/var/lib/kubelet/kubelet-node01.crt"
tlsPrivateKeyFile: "/var/lib/kubelet/kubelet-node01.key"
```

KODEKLOUD

KUBECTL NODES (CLIENT CERT)

KODEKLOUD

KODEKLOUD

Course Objectives

- Core Concepts
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
  - Kubernetes Security Primitives
  - Secure Persistent Key Value Store
  - Authentication
  - Authorization
  - Security Contexts
  - TLS Certificates for Cluster Components
  - Images Securely
  - Network Policies
- Storage
- Networking
- Installation, Configuration & Validation
- Troubleshooting

KODEKLOUD

TLS CERTIFICATES

View Certificate Details

KODEKLOUD

“The Hard Way”

kubeadm

KODEKLOUD

“The Hard Way”

```
cat /etc/systemd/system/kube-apiserver.service
```

```
[Service]
ExecStart=/usr/local/bin/kube-apiserver \\
  --advertise-address=172.17.0.32 \\
  --allow-privileged=true \\
  --apiserver-count=3 \\
  --authorization-mode=Node,RBAC \\
  --bind-address=0.0.0.0 \\
  --client-ca-file=/var/lib/kubernetes/ca.pem \\
  --enable-swagger-ui=true \\
  --etcd-cafile=/var/lib/kubernetes/ca.pem \\
  --etcd-certfile=/var/lib/kubernetes/kubernetes.pem \\
  --etcd-keyfile=/var/lib/kubernetes/kubernetes-key.pem \\
  --event-ttl=1h \\
  --kubelet-certificate-authority=/var/lib/kubernetes/ca.pem \\
  --kubelet-client-key=/var/lib/kubernetes/kubernetes-key.pem \\
  --kubelet-https=true \\
  --service-node-port-range=30000-32767 \\
  --tls-cert-file=/var/lib/kubernetes/kubernetes.pem \\
  --tls-private-key-file=/var/lib/kubernetes/kubernetes-key.pem
  --v=2
```

kubeadm

```
cat /etc/kubernetes/manifests/kube-apiserver.yaml
```

```
spec:
  containers:
  - command:
    - kube-apiserver
    - --authorization-mode=Node,RBAC
    - --advertise-address=172.17.0.32
    - --allow-privileged=true
    - --client-ca-file=/etc/kubernetes/pki/ca.crt
    - --disable-admission-plugins=PersistentVolumeLabel
    - --enable-admission-plugins=NodeRestriction
    - --enable-bootstrap-token-auth=true
    - --etcd-cafile=/etc/kubernetes/pki/etcd/ca.crt
    - --etcd-certfile=/etc/kubernetes/pki/apiserver-etcd-client.cr
    - --etcd-keyfile=/etc/kubernetes/pki/apiserver-etcd-client.key
    - --etcd-servers=https://127.0.0.1:2379
    - --insecure-port=0
    - --kubelet-client-certificate=/etc/kubernetes/pki/apiserver-k
    - --kubelet-client-key=/etc/kubernetes/pki/apiserver-kubelet-c
    - --kubelet-preferred-address-types=InternalIP,ExternalIP,Host
    - --proxy-client-cert-file=/etc/kubernetes/pki/front-proxy-cli
    - --proxy-client-key-file=/etc/kubernetes/pki/front-proxy-clie
    - --requestheader-allowed-names=front-proxy-client
```

kubeadm

Component	Type	Certificate Path	CN Name	ALT Names	Organization	Issuer	Expiration
kube-apiserver	Server						
kube-apiserver	Server						
kube-apiserver	Server						
kube-apiserver	Client (Kubelet)						
kube-apiserver	Client (Kubelet)						
kube-apiserver	Client (Etcd)						
kube-apiserver	Client (Etcd)						
kube-apiserver	Client (Etcd)						

KODEKLOUD

```
cat /etc/kubernetes/manifests/kube-apiserver.yaml
```

```
spec:
  containers:
  - command:
    - kube-apiserver
    - --authorization-mode=Node,RBAC
    - --advertise-address=172.17.0.32
    - --allow-privileged=true
    - --client-ca-file=/etc/kubernetes/pki/ca.crt
    - --disable-admission-plugins=PersistentVolumeLabel
    - --enable-admission-plugins=NodeRestriction
    - --enable-bootstrap-token-auth=true
    - --etcd-cafile=/etc/kubernetes/pki/etcd/ca.crt
    - --etcd-certfile=/etc/kubernetes/pki/apiserver-etcd-client.crt
    - --etcd-keyfile=/etc/kubernetes/pki/apiserver-etcd-client.key
    - --etcd-servers=https://127.0.0.1:2379
    - --insecure-port=0
    - --kubelet-client-certificate=/etc/kubernetes/pki/apiserver-kubelet-client.crt
    - --kubelet-client-key=/etc/kubernetes/pki/apiserver-kubelet-client.key
    - --kubelet-preferred-address-types=InternalIP,ExternalIP,Hostname
    - --proxy-client-cert-file=/etc/kubernetes/pki/front-proxy-client.crt
    - --proxy-client-key-file=/etc/kubernetes/pki/front-proxy-client.key
    - --secure-port=6443
    - --service-account-key-file=/etc/kubernetes/pki/sa.pub
    - --service-cluster-ip-range=10.96.0.0/12
    - --tls-cert-file=/etc/kubernetes/pki/apiserver.crt
    - --tls-private-key-file=/etc/kubernetes/pki/apiserver.key
```

KODEKLOUD

/etc/kubernetes/pki/apiserver.crt

```
openssl x509 -in /etc/kubernetes/pki/apiserver.crt -text -noout
```

```
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number: 3147495682089747350 (0x2bae26a58f090396)
    Signature Algorithm: sha256WithRSAEncryption
        Issuer: CN=kubernetes
        Validity
            Not Before: Feb 11 05:39:19 2019 GMT
            Not After : Feb 11 05:39:20 2020 GMT
        Subject: CN=kube-apiserver
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                Public-Key: (2048 bit)
                Modulus:
                    00:d9:69:38:80:68:3b:b7:2e:9e:25:00:e8:fd:01:

                Exponent: 65537 (0x10001)
        X509v3 extensions:
            X509v3 Key Usage: critical
                Digital Signature, Key Encipherment
            X509v3 Extended Key Usage:
                TLS Web Server Authentication
            X509v3 Subject Alternative Name:
                DNS:master, DNS:kubernetes, DNS:kubernetes.default,
DNS:kubernetes.default.svc, DNS:kubernetes.default.svc.cluster.local, IP
Address:10.96.0.1, IP Address:172.17.0.27
```

KODEKLOUD

kubeadm

Certificate Path	CN Name	ALT Names	Organization	Issuer	Expiration
/etc/kubernetes/pki/apiserver.crt	kube-apiserver	DNS:master DNS:kubernetes DNS:kubernetes.default DNS:kubernetes.default.svc IP Address:10.96.0.1 IP Address:172.17.0.27		kubernetes	Feb 11 05:39:20 2020
/etc/kubernetes/pki/apiserver.key					
/etc/kubernetes/pki/ca.crt	kubernetes			kubernetes	Feb 8 05:39:19 2029
/etc/kubernetes/pki/apiserver-kubelet-client.crt	kube-apiserver-kubelet-client		system:masters	kubernetes	Feb 11 05:39:20 2020
/etc/kubernetes/pki/apiserver-kubelet-client.key					
/etc/kubernetes/pki/apiserver-etcd-client.crt	kube-apiserver-etcd-client		system:masters	self	Feb 11 05:39:22 2020
/etc/kubernetes/pki/apiserver-etcd-client.key					
/etc/kubernetes/pki/etcd/ca.crt	kubernetes			kubernetes	Feb 8 05:39:21 2017

KODEKLOUD

Default CN	Parent CA	O (in Subject)	kind	hosts (SAN)
kube-etcd	etcd-ca		server, client [1][etcdbug]	`localhost`, `127.0.0.1`
kube-etcd-peer	etcd-ca		server, client	`<hostname>`, `<Host_IP>`, `localhost`, `127.0.0.1`
kube-etcd-healthcheck-client	etcd-ca		client	
kube-apiserver-etcd-client	etcd-ca	system:masters	client	
kube-apiserver	kubernetes-ca		server	`<hostname>`, `<Host_IP>`, `<advertise_IP>`, `[1]`
kube-apiserver-kubelet-client	kubernetes-ca	system:masters	client	
front-proxy-client	kubernetes-front-proxy-ca		client	

Default CN	recommend key path	recommended cert path	command	key argument	cert argument
etcd-ca		etcd/ca.crt	kube-apiserver		--etcd-cafile
etcd-client	apiserver-etcd-client.key	apiserver-etcd-client.crt	kube-apiserver	--etcd-keyfile	--etcd-certfile
kubernetes-ca		ca.crt	kube-apiserver		--client-ca-file
kube-apiserver	apiserver.key	apiserver.crt	kube-apiserver	--tls-private-key-file	--tls-cert-file
apiserver-kubelet-client		apiserver-kubelet-client.crt	kube-apiserver		--kubelet-client-certificate
front-proxy-ca		front-proxy-ca.crt	kube-apiserver		--requestheader-client-ca-file
front-proxy-client	front-proxy-client.key	front-proxy-client.crt	kube-apiserver	--proxy-client-key-file	--proxy-client-cert-file
etcd-ca		etcd/ca.crt	etcd		--trusted-ca-file, --peer-trusted-ca-file
kube-etcd	etcd/server.key	etcd/server.crt	etcd	--key-file	--cert-file
kube-etcd-peer	etcd/peer.key	etcd/peer.crt	etcd	--peer-key-file	--peer-cert-file
etcd-ca		etcd/ca.crt	etcdctl[2]		--cacert
kube-etcd-healthcheck-client	etcd/healthcheck-client.key	etcd/healthcheck-client.crt	etcdctl[2]	--key	--cert

Inspect Service Logs

```
journalctl -u etcd.service -l
```

```
2019-02-13 02:53:28.144631 I | etcdmain: etcd Version: 3.2.18
2019-02-13 02:53:28.144680 I | etcdmain: Git SHA: eddf599c6
2019-02-13 02:53:28.144684 I | etcdmain: Go Version: go1.8.7
2019-02-13 02:53:28.144688 I | etcdmain: Go OS/Arch: linux/amd64
2019-02-13 02:53:28.144692 I | etcdmain: setting maximum number of CPUs to 4, total number of available CPUs is 4
2019-02-13 02:53:28.144734 N | etcdmain: the server is already initialized as member before, starting as etcd
member...
2019-02-13 02:53:28.146625 I | etcdserver: name = master
2019-02-13 02:53:28.146637 I | etcdserver: data dir = /var/lib/etcd
2019-02-13 02:53:28.146642 I | etcdserver: member dir = /var/lib/etcd/member
2019-02-13 02:53:28.146645 I | etcdserver: heartbeat = 100ms
2019-02-13 02:53:28.146648 I | etcdserver: election = 1000ms
2019-02-13 02:53:28.146651 I | etcdserver: snapshot count = 10000
2019-02-13 02:53:28.146677 I | etcdserver: advertise client URLs = 2019-02-13 02:53:28.185353 I | etcdserver/api:
enabled capabilities for version 3.2
2019-02-13 02:53:28.185588 I | embed: ClientTLS: cert = /etc/kubernetes/pki/etcd/server.crt, key =
/etc/kubernetes/pki/etcd/server.key, ca = , trusted-ca = /etc/kubernetes/pki/etcd/old-ca.crt, client-cert-auth =
true
2019-02-13 02:53:30.080017 I | embed: ready to serve client requests
2019-02-13 02:53:30.080130 I | etcdserver: published {Name:master ClientURLs:[https://127.0.0.1:2379]} to cluster
c9be114fc2da2776
2019-02-13 02:53:30.080281 I | embed: serving client requests on 127.0.0.1:2379
WARNING: 2019/02/13 02:53:30 Failed to dial 127.0.0.1:2379: connection error: desc = "transport: authentication
handshake failed: remote error: tls: bad certificate"; please retry.
```

KODEKLOUD

View Logs

```
kubectl logs etcd-master
```

```
2019-02-13 02:53:28.144631 I | etcdmain: etcd Version: 3.2.18
2019-02-13 02:53:28.144680 I | etcdmain: Git SHA: eddf599c6
2019-02-13 02:53:28.144684 I | etcdmain: Go Version: go1.8.7
2019-02-13 02:53:28.144688 I | etcdmain: Go OS/Arch: linux/amd64
2019-02-13 02:53:28.144692 I | etcdmain: setting maximum number of CPUs to 4, total number of available CPUs is 4
2019-02-13 02:53:28.144734 N | etcdmain: the server is already initialized as member before, starting as etcd
member...
2019-02-13 02:53:28.146625 I | etcdserver: name = master
2019-02-13 02:53:28.146637 I | etcdserver: data dir = /var/lib/etcd
2019-02-13 02:53:28.146642 I | etcdserver: member dir = /var/lib/etcd/member
2019-02-13 02:53:28.146645 I | etcdserver: heartbeat = 100ms
2019-02-13 02:53:28.146648 I | etcdserver: election = 1000ms
2019-02-13 02:53:28.146651 I | etcdserver: snapshot count = 10000
2019-02-13 02:53:28.146677 I | etcdserver: advertise client URLs = 2019-02-13 02:53:28.185353 I | etcdserver/api:
enabled capabilities for version 3.2
2019-02-13 02:53:28.185588 I | embed: ClientTLS: cert = /etc/kubernetes/pki/etcd/server.crt, key =
/etc/kubernetes/pki/etcd/server.key, ca = , trusted-ca = /etc/kubernetes/pki/etcd/old-ca.crt, client-cert-auth =
true
2019-02-13 02:53:30.080017 I | embed: ready to serve client requests
2019-02-13 02:53:30.080130 I | etcdserver: published {Name:master ClientURLs:[https://127.0.0.1:2379]} to cluster
c9be114fc2da2776
2019-02-13 02:53:30.080281 I | embed: serving client requests on 127.0.0.1:2379
WARNING: 2019/02/13 02:53:30 Failed to dial 127.0.0.1:2379: connection error: desc = "transport: authentication
handshake failed: remote error: tls: bad certificate"; please retry.
```

KODEKLOUD

View Logs

```
docker ps -a
```

```
CONTAINER ID                                                
```

View Logs

```
docker logs 87fc
```

```
2019-02-13 02:53:28.144631 I | etcdmain: etcd Version: 3.2.18
2019-02-13 02:53:28.144680 I | etcdmain: Git SHA: eddf599c6
2019-02-13 02:53:28.144684 I | etcdmain: Go Version: go1.8.7
2019-02-13 02:53:28.144688 I | etcdmain: Go OS/Arch: linux/amd64
2019-02-13 02:53:28.144692 I | etcdmain: setting maximum number of CPUs to 4, total number of available CPUs is 4
2019-02-13 02:53:28.144734 N | etcdmain: the server is already initialized as member before, starting as etcd
member...
2019-02-13 02:53:28.146625 I | etcdserver: name = master
2019-02-13 02:53:28.146637 I | etcdserver: data dir = /var/lib/etcd
2019-02-13 02:53:28.146642 I | etcdserver: member dir = /var/lib/etcd/member
2019-02-13 02:53:28.146645 I | etcdserver: heartbeat = 100ms
2019-02-13 02:53:28.146648 I | etcdserver: election = 1000ms
2019-02-13 02:53:28.146651 I | etcdserver: snapshot count = 10000
2019-02-13 02:53:28.146677 I | etcdserver: advertise client URLs = 2019-02-13 02:53:28.185353 I | etcdserver/api:
enabled capabilities for version 3.2
2019-02-13 02:53:28.185588 I | embed: ClientTLS: cert = /etc/kubernetes/pki/etcd/server.crt, key =
/etc/kubernetes/pki/etcd/server.key, ca = , trusted-ca = /etc/kubernetes/pki/etcd/old-ca.crt, client-cert-auth =
true
2019-02-13 02:53:30.080017 I | embed: ready to serve client requests
2019-02-13 02:53:30.080130 I | etcdserver: published {Name:master ClientURLs:[https://127.0.0.1:2379]} to cluster
c9be114fc2da2776
2019-02-13 02:53:30.080281 I | embed: serving client requests on 127.0.0.1:2379
WARNING: 2019/02/13 02:53:30 Failed to dial 127.0.0.1:2379: connection error: desc = "transport: authentication
handshake failed: remote error: tls: bad certificate"; please retry.
```

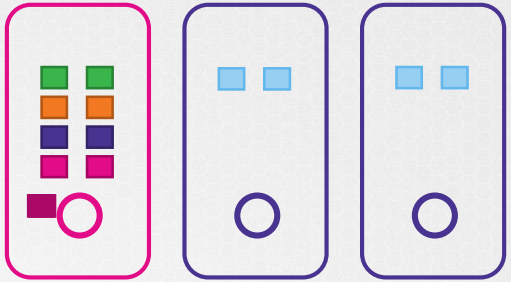
KODEKLOUD

KODEKLOUD

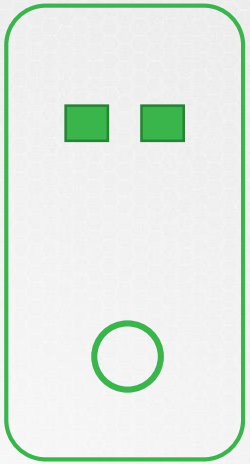
TLS CERTIFICATES

Certificate Workflow & API

KODEKLOUD



KODEKLOUD



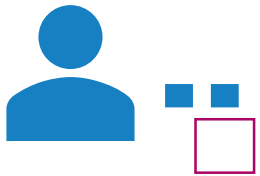
KODEKLOUD

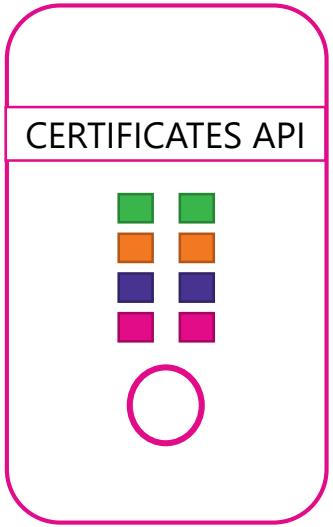
1. Create CertificateSigningRequest Object

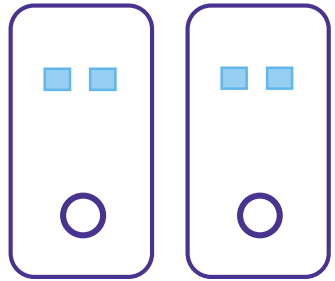
2. Review Requests

3. Approve Requests

4. Share Certs to Users







KODEKLOUD

```
openssl genrsa -out jane.key 2048
```

```
jane.key
```

```
openssl req -new -key jane.key -subj  "/CN=jane" -out jane.csr
```

```
jane.csr


-----BEGIN CERTIFICATE REQUEST-----
MIICWDCCAUACAQAwEzERMA8GA1UEAwwIbmV3LXVzZXIwggEiMA0GCSqGSIb3DQEB
AQUAA4IBDwAwggEKAoIBAQDO0WJW+DXsAJSIrjpNo5vRIBplnzg+6xc9+UVwkKi0
LfC27t+1eEnON5Muq99NevmMEOnrDUO/thyVqP2w2XNIDRXjYyF40FbmD+5zWyCK
9w0BAQsFAAOCAQEAS9iS6C1uxTuf5BBYSU7QFQHUzalNxAdYsaORRQNwHZwHqGi4
hOK4a2zyNyi44OOijyaD6tUW8DSxkr8BLK8Kg3srREtJql5rLZy9LRVrsJghD4gY
P9NL+aDRSxROVSqBaB2nWeYpM5cJ5TF53lesNSNMLQ2++RMnjDQJ7juPEic8/dhk
Wr2EUM6UawzykrdHImwTv2mlMY0R+DNtV1Yie+0H9/YElt+FSGjh5L5YUvI1Dqiy
4l3E/y3qL71WfAcuH3OsVpUUnQISMdQs0qWCsbE56CC5DhPGZIpUbnKUpAwka+8E
vwQ07jG+hpknxmuFAeXxgUwodALaJ7ju/TDIcw==
-----END CERTIFICATE REQUEST-----
```

KODEKLOUD

jane.csr

```
-----BEGIN CERTIFICATE REQUEST-----
MIICWDCCAUACAQAwEzERMA8GA1UEAwwIbmV3LXVzZXIwggEiMA0GCSqGSIb3DQEB
AQUAA4IBDwAwggEKAoIBAQDO0WJW+DXsAJSIrjpNo5vRIBplnzg+6xc9+UVwkKi0
LfC27t+1eEnON5Muq99NevmMEOnrDUO/thyVqP2w2XNIDRXjYyF40FbmD+5zWyCK
9w0BAQsFAAOCAQEAS9iS6C1uxTuf5BBYSU7QFQHUzalNxAdYsaORRQNwHZwHqGi4
hOK4a2zyNyi44OOijyaD6tUW8DSxkr8BLK8Kg3srREtJql5rLZy9LRVrsJghD4gY
P9NL+aDRSxROVSqBaB2nWeYpM5cJ5TF53lesNSNMLQ2++RMnjDQJ7juPEic8/dhk
Wr2EUM6UawzykrdHImwTv2mlMY0R+DNtV1Yie+0H9/YE1t+FSGjh5L5YUvI1Dqiy
4l3E/y3qL71WfAcuH3OsVpUUnQISMdQs0qWCsbE56CC5DhPGZIpUbnKUpAwka+8E
vwQ07jG+hpknxmuFAeXxgUwodALaJ7ju/TDIcw==
-----END CERTIFICATE REQUEST-----
```

cat jane.csr | base64

```
LS0tLS1CRUdJTiBDRVJUSUZJQ0FURSBSRVFVRVNULS0
tLS0KTUIJQ1dEQ0NBVUFDQVFBd0V6RVJNQThHQTFVRU
F3d0libVYzTFnweIpYSXdnZ0VpTUEwR0NTcUdTSWIzR
FFQgpBUVVBQTRJQkR3QXdnZ0VLQW9JQkFRRE8wV0pX
k0RYcDrKU01yanBObzVZUk1CcGxuemcrNnhjOStVVnd
rS2kwCkxmQzI3dCsxZUVuT041TXVxOTloZXZtTUVPbn
JnhjOStVVndrS2kwCkxmQzI3dCsxZUVuT0
41TXVxOTloZXZtTUVPbnJ
```

jane-csr.yaml

```yaml
apiVersion: certificates.k8s.io/v1beta1
kind: CertificateSigningRequest
metadata:
  name: jane
spec:
  groups:
  - system:authenticated
  usages:
  - digital signature
  - key encipherment
  - server auth
  request:
```

KODEKLOUD

```
kubectl get csr
```

```
NAME                    AGE       REQUESTOR                CONDITION
jane                    10m       admin@example.com        Pending
```

```
kubectl certificate approve jane
```

```
jane approved!
```

KODEKLOUD

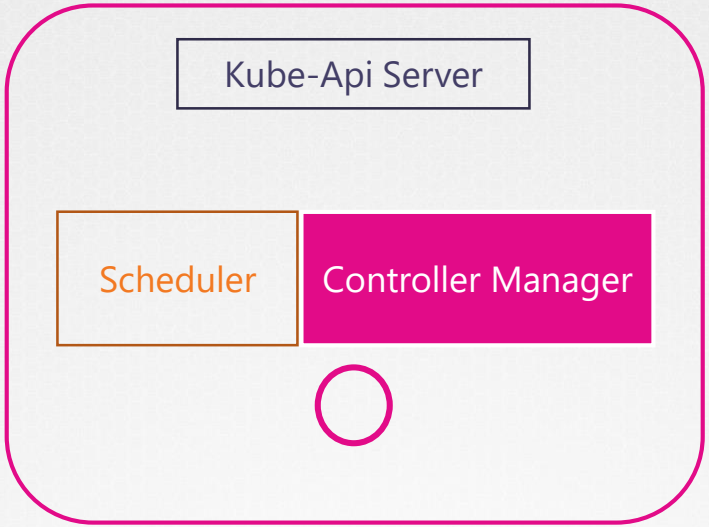
kubectl get csr jane -o yaml

```
apiVersion: certificates.k8s.io/v1beta1
kind: CertificateSigningRequest
metadata:
  creationTimestamp: 2019-02-13T16:36:43Z
  name: new-user
spec:
  groups:
  - system:masters
  - system:authenticated
usages:
  - digital signature
  - key encipherment
  - server auth
  username: kubernetes-admin
status:
  certificate:
LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0tLS0tCk1JSURDakNDQWZLZ0F3SUJBZ0lVRmwy
Q2wxYXoxaWl5M3JNVisreFRYQUowU3dnd0RRWUpLb1pJaHZjTkFRUwKQlFBd0ZURVRN
QkVHQTFVRUF4TUthM1ZpWlhKdVpYUmxjekFlRncweE9UQXlNVE14TmpNeU1EQmFGd1dn
Y0ZFeDl2ajNuSXY3eFdDS1NIRm5sU041c0t5Z0VxUkwzTFM5V29GelhHZDdWCmlEZ2FO
MVVRMFBXTVhjN09FVnVjSWc1Yk4weEVHTkVwRU5tdUlBNlZWeHVjS1h6aG9ldDY0MEd1
MGU0YXFKWVIKWmVMbjBvRTFCY3dod2xic0I1ND0KLS0tLS1FTkQgQ0VSVElGSUNBVEUt
LS0tLQo=
  conditions:
  - lastUpdateTime: 2019-02-13T16:37:21Z
    message: This CSR was approved by kubectl certificate approve.
    reason: KubectlApprove
    type: Approved
```

echo “LS0…Qo=” | base64 --decode

```
-----BEGIN CERTIFICATE -----
MIICWDCCAUACAQAwEzERMA8GA1UEAwwIbmV3LXVzZXIwgg
AQUAA4IBDwAwggEKAoIBAQDO0WJW+DXsAJSIrjpNo5vRIB
LfC27t+1eEnON5Muq99NevmMEOnrDUO/thyVqP2w2XNIDR
y3BihhB93MJ7Oql3UTvZ8TELqyaDknRl/jv/SxgXkok0AB
IF5nxAttMVkDPQ7NbeZRG43b+QWlVGR/z6DWOfJnbfezOt
EcCXAwqChjBLkz2BHPR4J89D6Xb8k39pu6jpyngV6uP0tI
j2qEL+hZEWkkFz80lNNtyT5LxMqENDCnIgwC4GZiRGbrAg
9w0BAQsFAAOCAQEAS9iS6C1uxTuf5BBYSU7QFQHUzalNxA
hOK4a2zyNyi44OOijyaD6tUW8DSxkr8BLK8Kg3srREtJql
P9NL+aDRSxROVSqBaB2nWeYpM5cJ5TF53lesNSNMLQ2++R
Wr2EUM6UawzykrdHImwTv2mlMY0R+DNtV1Yie+0H9/YElt
4l3E/y3qL71WfAcuH3OsVpUUnQISMdQs0qWCsbE56CC5Dh
vwQ07jG+hpknxmuFAeXxgUwodALaJ7ju/TDIcw==
-----END CERTIFICATE -----
```

KODEKLOUD



KODEKLOUD

KODEKLOUD

```
cat /etc/kubernetes/manifests/kube-controller-manager.yaml
```

```
spec:
  containers:
  - command:
    - kube-controller-manager
    - --address=127.0.0.1
    - --cluster-signing-cert-file=/etc/kubernetes/pki/ca.crt
    - --cluster-signing-key-file=/etc/kubernetes/pki/ca.key
    - --controllers=*,bootstrapsigner,tokencleaner
    - --kubeconfig=/etc/kubernetes/controller-manager.conf
    - --leader-elect=true
    - --root-ca-file=/etc/kubernetes/pki/ca.crt
    - --service-account-private-key-file=/etc/kubernetes/pki/sa.key
    - --use-service-account-credentials=true
```

KODEKLOUD

Client Certificates for Clients

kube-scheduler

kube-controller-manager

- CA CERT for Cluster Signing
- CA KEY for Cluster Signing
- CA CERT
- KEY for SERVICE ACCOUNT

kubelet

kube-proxy

Server Certificates for Servers

kube-apiserver

Kube-api.service
- CA CERT for ETCD
- CERT for ETCD
- KEY for ETCD
- CA CERT for KUBELET
- CERT for KUBELET CLIE
- KEY for KUBELET CLIEN
- CERT for Service Accou
- CERT for TLS
- KEY for TLS

kubelet

Kubelet-config.yaml
- tlsCertFile
- tlsPrivateKeyFile

ETCD CLUSTER

KODEKLOUD

KODEKLOUD

Provision Certificate Authority

ca-key.pem

ca.pem

kube-apiserver

kubernetes-key.pem

kubernetes.pem

admin user

admin-key.pem

admin.pem

kube-controller-manager

kcm-key.pem

kcm.pem

kube-scheduler

Kube-scheduler-key.pem

Kube-scheduler.pem

Kube-proxy

kube-proxy-key.pem

kube-proxy.pem

kubelet

worker01-key.pem

worker01.pem

worker02-key.pem

worker02.pem

Default CN	Parent CA	O (in Subject)	kind	hosts (SAN)
kube-etcd	etcd-ca		server, client [1][etcdbug]	`localhost`, `127.0.0.1`
kube-etcd-peer	etcd-ca		server, client	`<hostname>`, `<Host_IP>`, `localhost`, `127.0.0.1`
kube-etcd-healthcheck-client	etcd-ca		client	
kube-apiserver-etcd-client	etcd-ca	system:masters	client	
kube-apiserver	kubernetes-ca		server	`<hostname>`, `<Host_IP>`, `<advertise_IP>`, `[1]`
kube-apiserver-kubelet-client	kubernetes-ca	system:masters	client	
front-proxy-client	kubernetes-front-proxy-ca		client	

Default CN	recommend key path	recommended cert path	command	key argument	cert argument
etcd-ca		etcd/ca.crt	kube-apiserver		--etcd-cafile
etcd-client	apiserver-etcd-client.key	apiserver-etcd-client.crt	kube-apiserver	--etcd-keyfile	--etcd-certfile
kubernetes-ca		ca.crt	kube-apiserver		--client-ca-file
kube-apiserver	apiserver.key	apiserver.crt	kube-apiserver	--tls-private-key-file	--tls-cert-file
apiserver-kubelet-client		apiserver-kubelet-client.crt	kube-apiserver		--kubelet-client-certificate
front-proxy-ca		front-proxy-ca.crt	kube-apiserver		--requestheader-client-ca-file
front-proxy-client	front-proxy-client.key	front-proxy-client.crt	kube-apiserver	--proxy-client-key-file	--proxy-client-cert-file
etcd-ca		etcd/ca.crt	etcd		--trusted-ca-file, --peer-trusted-ca-file
kube-etcd	etcd/server.key	etcd/server.crt	etcd	--key-file	--cert-file
kube-etcd-peer	etcd/peer.key	etcd/peer.crt	etcd	--peer-key-file	--peer-cert-file
etcd-ca		etcd/ca.crt	etcdctl[2]		--cacert
kube-etcd-healthcheck-client	etcd/healthcheck-client.key	etcd/healthcheck-client.crt	etcdctl[2]	--key	--cert

Configure certificates for user accounts

You must manually configure these administrator account and service accounts:

filename	credential name	Default CN	O (in Subject)
admin.conf	default-admin	kubernetes-admin	system:masters
kubelet.conf	default-auth	system:node: **<nodeName>** (see note)	system:nodes
controller-manager.conf	default-controller-manager	system:kube-controller-manager	
scheduler.conf	default-manager	system:kube-scheduler	

KODEKLOUD

Component	Certificate	Purpose
API server	Cluster CA	Authenticate clients, TLS
API server	Etcd CA	Etcd server authentication
API server	Etcd client cert	Etcd client authentication
API server	Serving certificate	Serving API over HTTPS
API server	Kubelet client cert	Authenticating against Kubelet
Controller Manager	Client certificate	Authenticating against API server
Controller Manager	Cluster CA	Embedding in service account secrets
Scheduler	Client certificate	Authenticating against API server
Kubelet	Serving certificate	Serving API over HTTPS
Kubelet	Client certificate	Authenticating against API server
Kubelet	Cluster CA	Authenticating clients
Kube Proxy	Client certificate	Authenticating against API server

- kube-apiserver
  - --authorization-mode=Node,RBAC
  - --advertise-address=172.17.0.18
  - --allow-privileged=true
  - --client-ca-file=/etc/kubernetes/pki/ca.crt
  - --disable-admission-plugins=PersistentVolumeLabel
  - --enable-admission-plugins=NodeRestriction
  - --enable-bootstrap-token-auth=true
  - --etcd-cafile=/etc/kubernetes/pki/etcd/ca.crt
  - --etcd-certfile=/etc/kubernetes/pki/apiserver-etcd-client.crt
  - --etcd-keyfile=/etc/kubernetes/pki/apiserver-etcd-client.key
  - --etcd-servers=https://127.0.0.1:2379
  - --insecure-port=0
  - --kubelet-client-certificate=/etc/kubernetes/pki/apiserver-kubelet-client.crt
  - --kubelet-client-key=/etc/kubernetes/pki/apiserver-kubelet-client.key
  - --kubelet-preferred-address-types=InternalIP,ExternalIP,Hostname
  - --proxy-client-cert-file=/etc/kubernetes/pki/front-proxy-client.crt
  - --proxy-client-key-file=/etc/kubernetes/pki/front-proxy-client.key
  - --requestheader-allowed-names=front-proxy-client
  - --requestheader-client-ca-file=/etc/kubernetes/pki/front-proxy-ca.crt
  - --requestheader-extra-headers-prefix=X-Remote-Extra-
  - --requestheader-group-headers=X-Remote-Group
  - --requestheader-username-headers=X-Remote-User
  - --secure-port=6443
  - --service-account-key-file=/etc/kubernetes/pki/sa.pub
  - --service-cluster-ip-range=10.96.0.0/12
  - --tls-cert-file=/etc/kubernetes/pki/apiserver.crt
  - --tls-private-key-file=/etc/kubernetes/pki/apiserver.key

KODEKLOUD

```
openssl genrsa -out old-ca.key 2048
openssl req -new -key old-ca.key -subj "/CN=old-ca" -out old-ca.csr
openssl x509 -req -in old-ca.csr -signkey old-ca.key -out old-ca.crt -days 365

openssl x509 -req -in ca.csr -signkey ca.key -out server.crt -days 365

openssl req -new -key apiserver-kubelet-client.key -out apiserver-kubelet-client.csr -subj "/CN=kube-apiserver-kubelet-client/O=system:masters"

openssl req -new -key apiserver-kubelet-client.key -out apiserver-kubelet-client.csr -subj "/CN=kube-apiserver-kubelet-client/O=system:masters"
openssl x509 -req -in apiserver-kubelet-client.csr -CA /root/new-ca/old-ca.crt -CAkey /root/new-ca/old-ca.key -CAcreateserial -out apiserver-kubelet-client-new.crt -days 365


openssl req -new -key apiserver-etcd-client.key -out apiserver-etcd-client.csr -subj "/CN=kube-apiserver-etcd-client/O=system:masters"
openssl x509 -req -in apiserver-etcd-client.csr -CA /root/new-ca/old-ca.crt -CAkey /root/new-ca/old-ca.key -CAcreateserial -out apiserver-etcd-client-new.crt -days 365


openssl req -new -key apiserver-etcd-client.key -out apiserver-etcd-client.csr -subj "/CN=kube-apiserver-etcd-client/O=system:masters"
openssl x509 -req -in apiserver-etcd-client.csr -CA /root/new-ca/old-ca.crt -CAkey /root/new-ca/old-ca.key -CAcreateserial -out apiserver-etcd-client-new.crt -days 365


openssl req -new -key /etc/kubernetes/pki/apiserver-etcd-client.key -out apiserver-etcd-client.csr -subj "/CN=kube-apiserver-etcd-client/O=system:masters"

openssl x509 -req -in apiserver-etcd-client.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out apiserver-etcd-client.crt -days -10

openssl x509 -req -in apiserver-etcd-client.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out apiserver-etcd-client.crt -startdate 190101010101Z

20170101000000Z
200801010000Z


"openssl", "req", "-new", "-key" ,"/etc/kubernetes/pki/apiserver-etcd-client.key", "-out", "/etc/kubernetes/pki/apiserver-etcd-client.csr", "-subj", "/CN=kube-apiserver-etcd-
client/O=system:masters"

"openssl", "x509", "-req", "-in", "/etc/kubernetes/pki/apiserver-etcd-client.csr", "-CA", "/etc/kubernetes/pki/etcd/ca.crt", "-CAkey", "/etc/kubernetes/pki/etcd/ca.key", "-CAcreateserial", "-out",
"/etc/kubernetes/pki/apiserver-etcd-client.crt"

openssl x509 -req -in /etc/kubernetes/pki/apiserver-etcd-client.csr -CA /etc/kubernetes/pki/etcd/ca.crt -CAkey /etc/kubernetes/pki/etcd/ca.key -CAcreateserial -out
/etc/kubernetes/pki/apiserver-etcd-client.crt -days 100


openssl x509 -req -in apiserver.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out apiserver.crt
```

KODEKLOUD

KODEKLOUD

Security
KUBECONFIG

```
curl https://my-kube-playground:6443/api/v1/pods \
  --key admin.key
  --cert admin.crt
  --cacert ca.crt
```


```
{
  "kind": "PodList",
  "apiVersion": "v1",
  "metadata": {
    "selfLink": "/api/v1/pods",
  },
  "items": []
}
```


```
kubectl get pods
    --server my-kube-playground:6443
    --client-key admin.key
    --client-certificate admin.crt
    --certificate-authority ca.crt
```

```
No resources found.
```

KODEKLOUD

config $HOME/.kube/config

KubeConfig File

```
--server my-kube-playground:6443
--client-key admin.key
--client-certificate admin.crt
--certificate-authority ca.crt
```

```
kubectl get pods
    --kubeconfig config
```

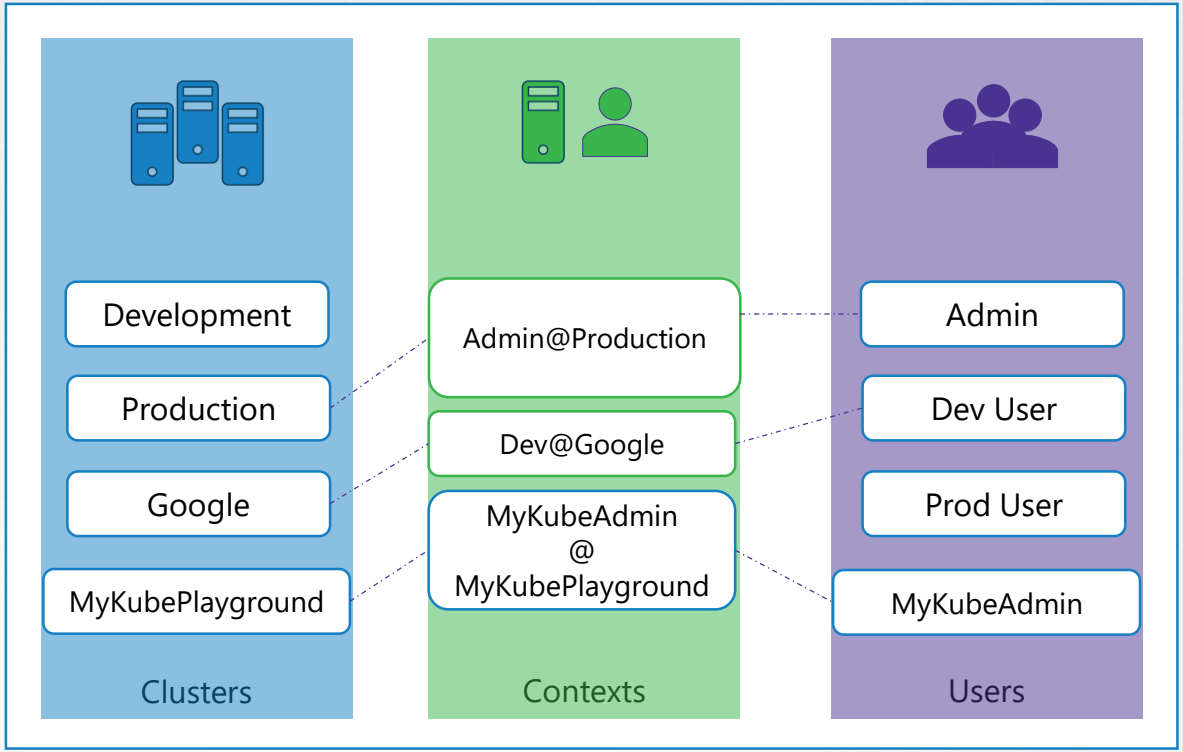
```
No resources found.
```

KODEKLOUD

KubeConfig File

$HOME/.kube/config

```
--server my-kube-playground:6443
--client-key admin.key
--client-certificate admin.crt
--certificate-authority ca.crt
```

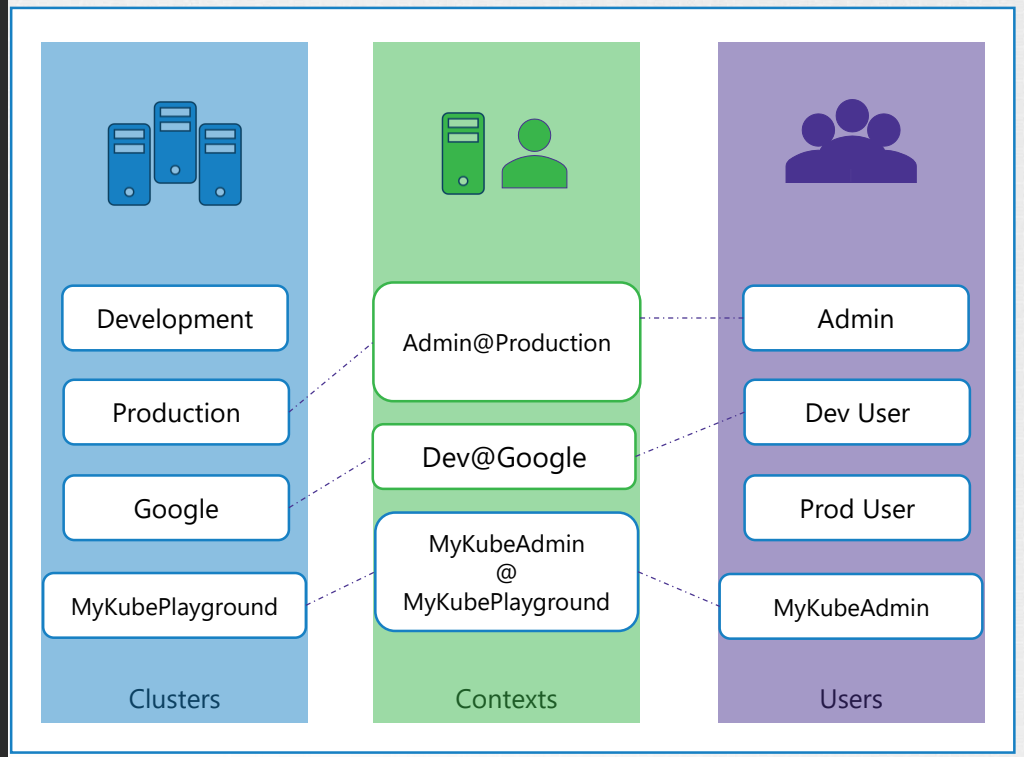


KODEKLOUD

KubeConfig File

```yaml
apiVersion: v1
kind: Config

clusters:

- name: my-kube-playground
  cluster:
    certificate-authority: ca.crt
    server: https://my-kube-playground:6443


contexts:

- name: my-kube-admin@my-kube-playground
  context:
    cluster:
    user:



users:

- name: my-kube-admin
  user:
    client-certificate: admin.crt
    client-key: admin.key
```

$HOME/.kube/config

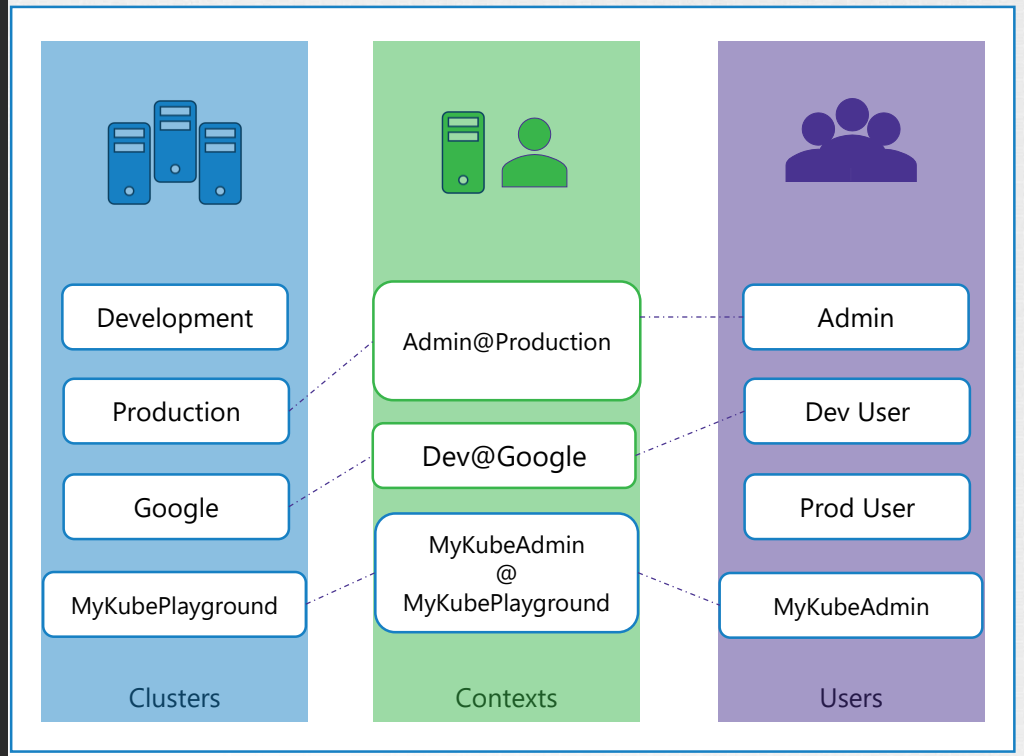


KODEKLOUD

KubeConfig File

```
apiVersion: v1
kind: Config
current-context: dev-user@google

clusters:
- name: my-kube-playground   (values hidden...)
- name: development
- name: production
- name: google

contexts:
- name: my-kube-admin@my-kube-playground
- name: dev-user@google
- name: prod-user@production

users:
- name: my-kube-admin
- name: admin
- name: dev-user
- name: prod-user
```

$HOME/.kube/config



KODEKLOUD

Kubectl config

kubectl config view

```
apiVersion: v1

kind: Config
current-context: kubernetes-admin@kubernetes

clusters:
- cluster:
    certificate-authority-data: REDACTED
    server: https://172.17.0.5:6443
  name: kubernetes

contexts:
- context:
    cluster: kubernetes
    user: kubernetes-admin
  name: kubernetes-admin@kubernetes


users:
- name: kubernetes-admin
  user:
    client-certificate-data: REDACTED
    client-key-data: REDACTED
```

kubectl config view –kubeconfig=my-custom-config

```
apiVersion: v1

kind: Config
current-context: my-kube-admin@my-kube-playground


clusters:
- name: my-kube-playground
- name: development
- name: production


contexts:
-  name: my-kube-admin@my-kube-playground
-  Name: prod-user@production


users:
- name: my-kube-admin
- name: prod-user
```

KODEKLOUD

Kubectl config

kubectl config view

```
apiVersion: v1

kind: Config
current-context: my-kube-admin@my-kube-playground


clusters:
- name: my-kube-playground
- name: development
- name: production


contexts:
-  name: my-kube-admin@my-kube-playground
-  Name: prod-user@production


users:
- name: my-kube-admin
- name: prod-user
```

kubectl config use-context prod-user@production

```
apiVersion: v1

kind: Config
current-context: prod-user@production


clusters:
- name: my-kube-playground
- name: development
- name: production


contexts:
-  name: my-kube-admin@my-kube-playground
-  Name: prod-user@production


users:
- name: my-kube-admin
- name: prod-user
```

KODEKLOUD

Kubectl config

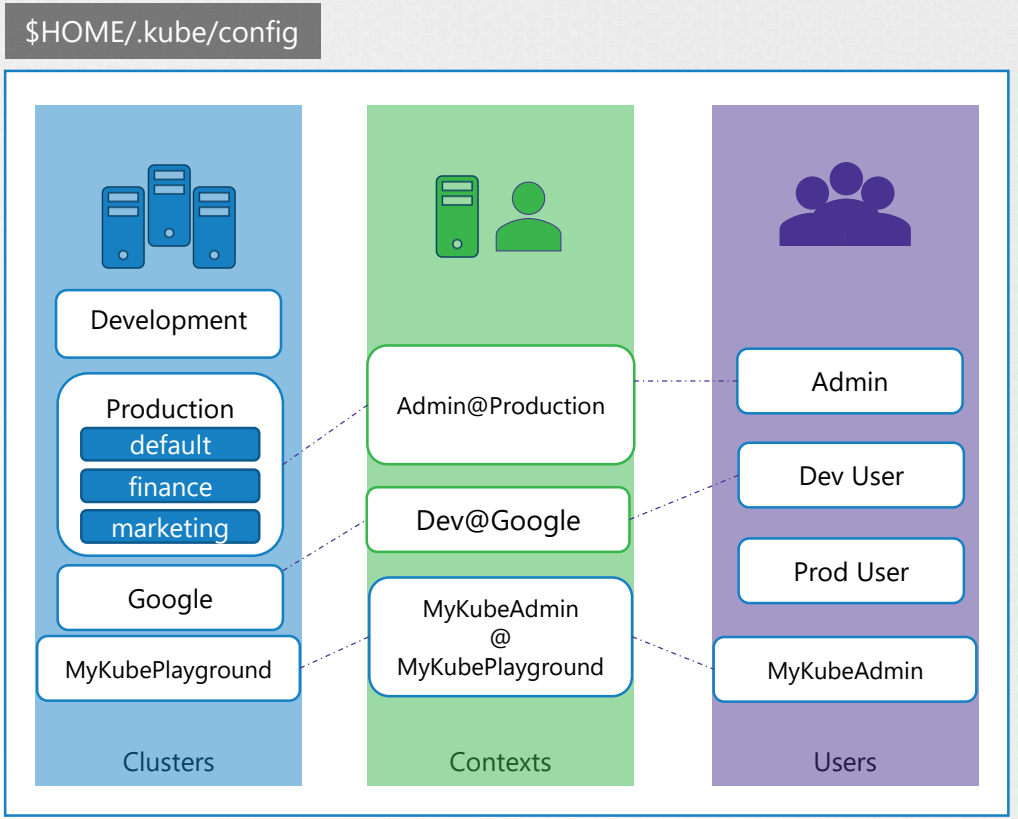
```
kubectl config -h
```

```
Available Commands:
  current-context Displays the current-context
  delete-cluster  Delete the specified cluster from the kubeconfig
  delete-context  Delete the specified context from the kubeconfig
  get-clusters    Display clusters defined in the kubeconfig
  get-contexts    Describe one or many contexts
  rename-context  Renames a context from the kubeconfig file.
  set             Sets an individual value in a kubeconfig file
  set-cluster     Sets a cluster entry in kubeconfig
  set-context     Sets a context entry in kubeconfig
  set-credentials Sets a user entry in kubeconfig
  unset           Unsets an individual value in a kubeconfig file
  use-context     Sets the current-context in a kubeconfig file
  view            Display merged kubeconfig settings or a specified kubeconfig file
```

KODEKLOUD

Namespaces

```
apiVersion: v1
kind: Config

clusters:
- name: production
  cluster:
    certificate-authority: ca.crt
    server: https://172.17.0.51:6443

contexts:
- name: admin@production
  context:
    cluster: production
    user: admin
    namespace: finance

users:
- name: admin
  user:
    client-certificate: admin.crt
    client-key: admin.key
```



KODEKLOUD

Certificates in KubeConfig

```yaml
apiVersion: v1
kind: Config

clusters:
- name: production
  cluster:
    certificate-authority: /etc/kubernetes/pki/ca.crt
    server: https://172.17.0.51:6443

contexts:
- name: admin@production
  context:
    cluster: production
    user: admin
    namespace: finance

users:
- name: admin
  user:
    client-certificate: /etc/kubernetes/pki/users/admin.crt
    client-key: /etc/kubernetes/pki/users/admin.key
```

KODEKLOUD

Certificates in KubeConfig

```yaml
apiVersion: v1
kind: Config

clusters:
- name: production
  cluster:
    certificate-authority: /etc/kubernetes/pki/ca.crt

    certificate-authority-data:
```

```
-----BEGIN CERTIFICATE -----
MIICWDCCAUACAQAwEzERMA8GA1UEAwwIbmV3LXVzZXIwggEiMA0GC
AQUAA4IBDwAwggEKAoIBAQDO0WJW+DXsAJSIrjpNo5vRIBplnzg+6
LfC27t+1eEnON5Muq99NevmMEOnrDUO/thyVqP2w2XNIDRXjYyF40
y3BihhB93MJ7Oql3UTvZ8TELqyaDknRl/jv/SxgXkok0ABUTpWMx4
IF5nxAttMVkDPQ7NbeZRG43b+QWlVGR/z6DWOfJnbfezOtaAydGLT
EcCXAwqChjBLkz2BHPR4J89D6Xb8k39pu6jpyngV6uP0tIbOzpqNv
j2qEL+hZEWkkFz80lNNtyT5LxMqENDCnIgwC4GZiRGbrAgMBAAGgA
9w0BAQsFAAOCAQEAS9iS6C1uxTuf5BBYSU7QFQHUzalNxAdYsaORR
hOK4a2zyNyi44OOijyaD6tUW8DSxkr8BLK8Kg3srREtJql5rLZy9L
P9NL+aDRSxROVSqBaB2nWeYpM5cJ5TF53lesNSNMLQ2++RMnjDQJ7
Wr2EUM6UawzykrdHImwTv2mlMY0R+DNtV1Yie+0H9/YElt+FSGjh5
4l3E/y3qL71WfAcuH3OsVpUUnQISMdQs0qWCsbE56CC5DhPGZIpUb
vwQ07jG+hpknxmuFAeXxgUwodALaJ7ju/TDIcw==
-----END CERTIFICATE -----
```

cat ca.crt | base64

```
LS0tLS1CRUdJTiBDRVJUSUZJQ0FURSBSRVFVRVN
tLS0KTUlJQ1dEQ0NBVUFDQVFBd0V6RVJNQThHQT
F3d0libVYzTFhWelpYSXdnZ0VpTUEwR0NTcUdIS
FFFQgpBUVVBQTRJQkR3QXdnZ0VLQW9JQkFRRE8w
K0RYc0FKU0lyanBObzV2UklCcGxuemcrNnhjOSt
rS2kwCkxmQzI3dCsxZUVuT041TXVxOTlOZXZtTU
J
U0lyanBObzV2UklCcGxuemcrNnhjOStV
VndrS2kwCkxmQzI3dCsxZUVuT041TXVxOTlOZXZtTU
```

KODEKLOUD

Course Objectives

- Core Concepts
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
  - Kubernetes Security Primitives
  - Secure Persistent Key Value Store
  - Authentication
  - Authorization
  - Security Contexts
  - TLS Certificates for Cluster Components
  - Images Securely
  - Network Policies
- Storage
- Networking
- Installation, Configuration & Validation
- Troubleshooting

KODEKLOUD

API Groups

Pre-Requisite

KODEKLOUD

```
curl https://kube-master:6443/version
```

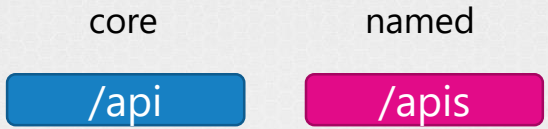

```
{
  "major": "1",
  "minor": "13",
  "gitVersion": "v1.13.0",
  "gitCommit": "ddf47ac13c1a9483ea035a79cd7c10005ff21a6d",
  "gitTreeState": "clean",
  "buildDate": "2018-12-03T20:56:12Z",
  "goVersion": "go1.11.2",
  "compiler": "gc",
  "platform": "linux/amd64"
}
```


```
curl https://kube-master:6443/api/v1/pods
```

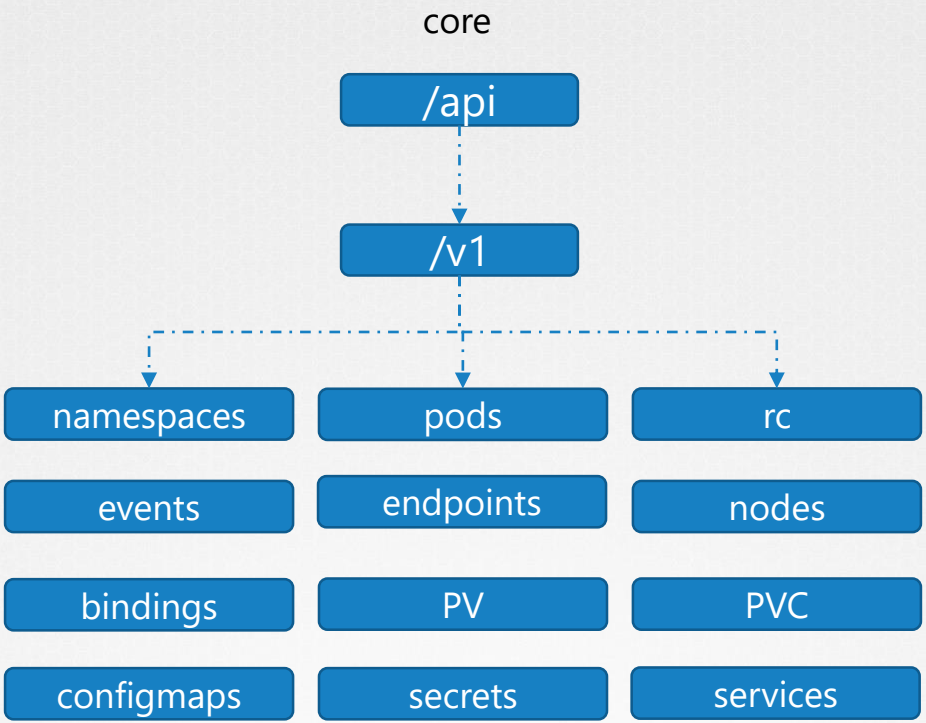

```
{
  "kind": "PodList",
  "apiVersion": "v1",
  "metadata": {
    "selfLink": "/api/v1/pods",
    "resourceVersion": "153068"
  },
  "items": [
    {
      "metadata": {
        "name": "nginx-5c7588df-ghsbd",
        "generateName": "nginx-5c7588df-",
        "namespace": "default",
        "creationTimestamp": "2019-03-20T10:57:48Z",
        "labels": {
          "app": "nginx",
          "pod-template-hash": "5c7588df"
        },
        "ownerReferences": [
          {
            "apiVersion": "apps/v1",
            "kind": "ReplicaSet",
            "name": "nginx-5c7588df",
            "uid": "398ce179-4af9-11e9-beb6-020d3114c7a7",
            "controller": true,
            "blockOwnerDeletion": true
          }
        ]
      },
```


KODEKLOUD

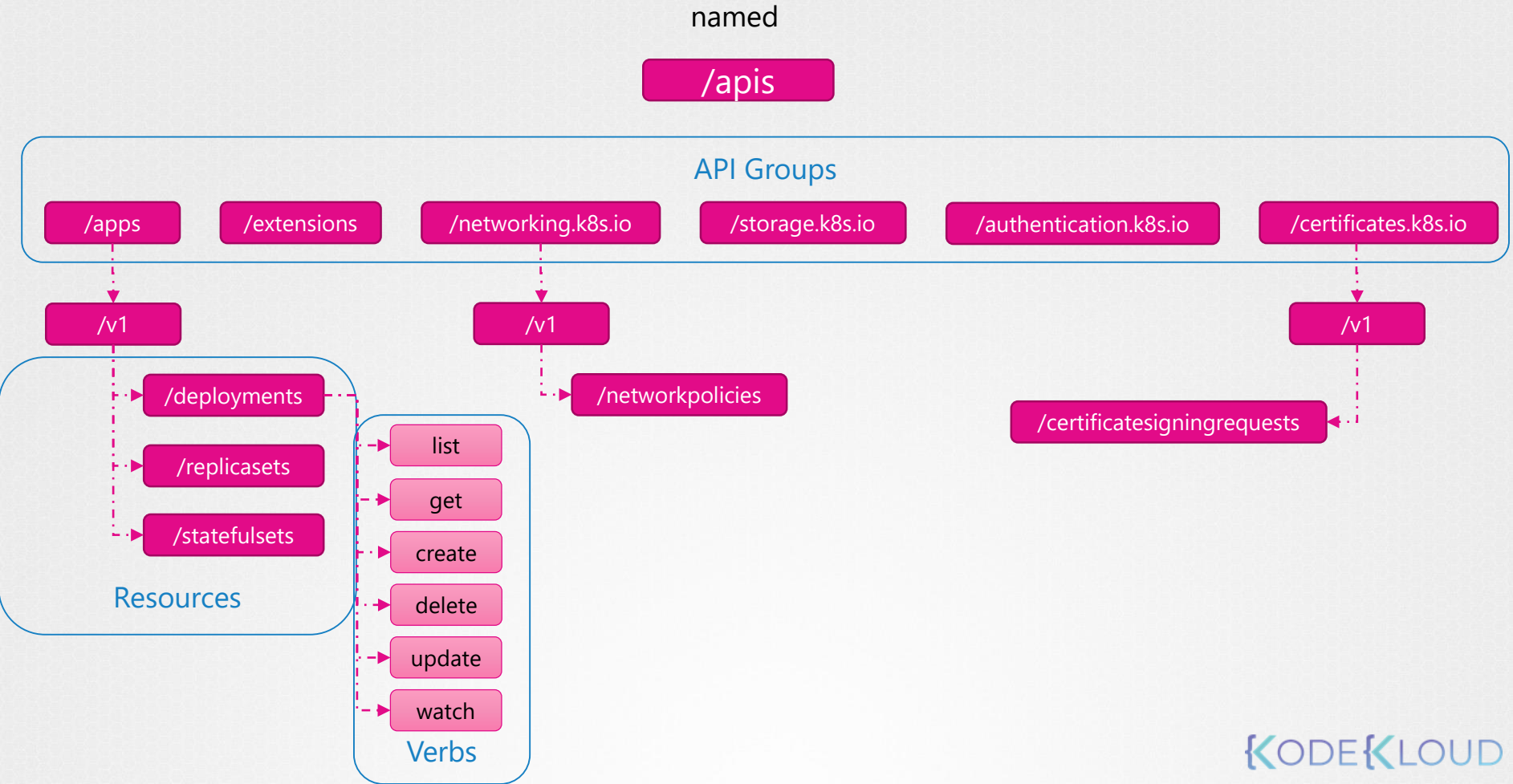
KODEKLOUD



KODEKLOUD


core
/api
/v1
namespaces
pods
rc
events
endpoints
nodes
bindings
PV
PVC
configmaps
secrets
services

KODEKLOUD


named
/apis
API Groups
/apps
/extensions
/networking.k8s.io
/storage.k8s.io
/authentication.k8s.io
/certificates.k8s.io
/v1
/v1
/v1
/deployments
/replicasets
/statefulsets
Resources
/networkpolicies
/certificatesigningrequests
list
get
create
delete
update
watch
Verbs

KODEKLOUD

https://kubernetes.io/docs/reference/generated/kubernetes-api/v1.13/#pod-v1-core

Bookmarks Kubernetes Cloud modules —...

- Overview
- WORKLOADS APIS
- Container v1 core
- CronJob v1beta1 batch
- DaemonSet v1 apps
- Deployment v1 apps
- Job v1 batch
- Pod v1 core
  - Write Operations
  - Read Operations
  - Status Operations
  - Proxy Operations
  - Misc Operations
- ReplicaSet v1 apps
- ReplicationController v1 core
- StatefulSet v1 apps

Pod v1 core

kubectl example | curl example

Group	Version
core	v1

Warning:

It is recommended that users create Pods only through a Controller, and not directly. See Controllers: Deploy

Appears In:

- PodList [core/v1]

Field	Description
apiVersion *string*	APIVersion defines the versioned schema of this representation of an object. Servers should c https://git.k8s.io/community/contributors/devel/api-conventions.md#resources

https://kubernetes.io/docs/reference/generated/kubernetes-api/v1.13

KODEKLOUD

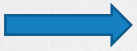
```
curl http://localhost:6443 -k
```

```
{
  "paths": [
    "/api",
    "/api/v1",
    "/apis",
    "/apis/",
    "/healthz",
    "/logs",
    "/metrics",
    "/openapi/v2",
    "/swagger-2.0.0.json",
```

```
curl http://localhost:6443/apis -k | grep “name”
```

```
      "name": "extensions",
      "name": "apps",
      "name": "events.k8s.io",
      "name": "authentication.k8s.io",
      "name": "authorization.k8s.io",
      "name": "autoscaling",
      "name": "batch",
      "name": "certificates.k8s.io",
      "name": "networking.k8s.io",
      "name": "policy",
      "name": "rbac.authorization.k8s.io",
      "name": "storage.k8s.io",
      "name": "admissionregistration.k8s.io",
      "name": "apiextensions.k8s.io",
      "name": "scheduling.k8s.io",
```

KODEKLOUD



Kube ApiServer

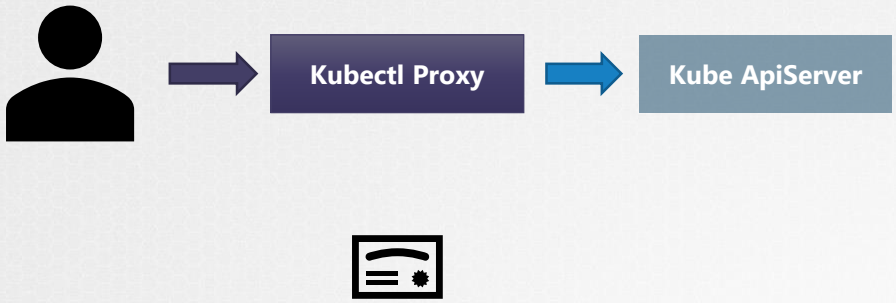
```
curl http://localhost:6443 -k
```


```
{
  "kind": "Status",
  "apiVersion": "v1",
  "metadata": {

  },
  "status": "Failure",
  "message": "forbidden: User \"system:anonymous\" cannot get path \"/\"",
  "reason": "Forbidden",
  "details": {

  },
  "code": 403
}
```


```
curl http://localhost:6443 –k
     --key admin.key
     --cert admin.crt
     --cacert ca.crt
```


```
{
  "paths": [
    "/api",
    "/api/v1",
    "/apis",
    "/apis/",
    "/healthz",
    "/logs",
    "/metrics"
```

kubectl proxy




```
kubectl proxy
Starting to serve on 127.0.0.1:8001
```

```
curl http://localhost:8001 -k
{
  "paths": [
    "/api",
    "/api/v1",
    "/apis",
    "/apis/",
    "/healthz",
    "/logs",
    "/metrics",
    "/openapi/v2",
    "/swagger-2.0.0.json",
```

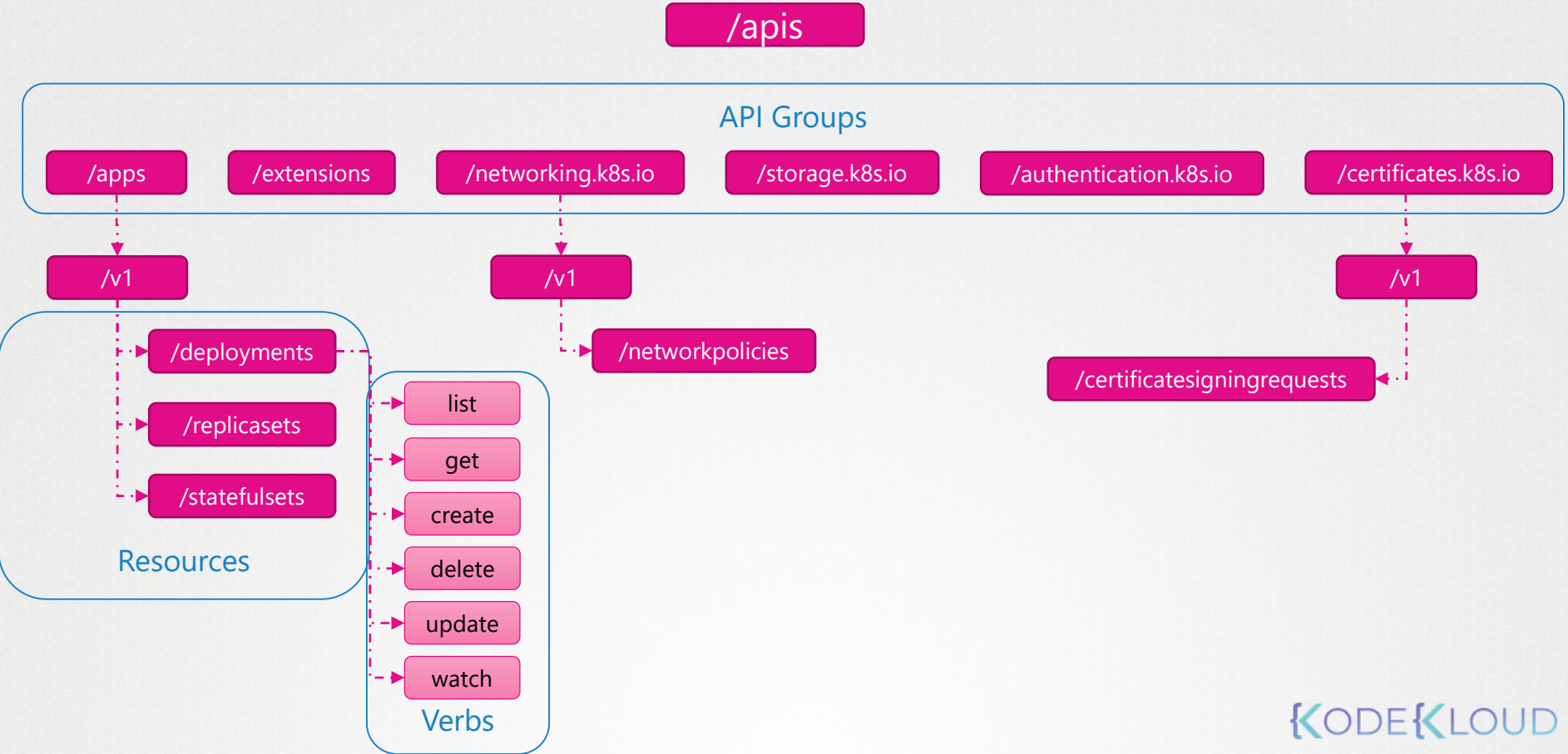
KODEKLOUD

Kube proxy ≠ Kubectl proxy

KODEKLOUD

Key Takeaways

named



KODEKLOUD

KODEKLOUD

Course Objectives

- Core Concepts
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
  - Kubernetes Security Primitives
  - Secure Persistent Key Value Store
  - Authentication
  - Authorization
  - Security Contexts
  - TLS Certificates for Cluster Components
  - Images Securely
  - Network Policies
- Storage
- Networking
- Installation, Configuration & Validation
- Troubleshooting

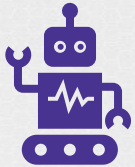
KODEKLOUD

AUTHORIZATION

Why Authorization?

Admins

Developers



Bots

```
kubectl get pods
NAME       STATUS   ROLES    AGE     VERSION
worker-1   Ready    <none>   5d21h   v1.13.0
worker-2   Ready    <none>   5d21h   v1.13.0
```

```
kubectl get pods
NAME       STATUS   ROLES    AGE     VERSION
worker-1   Ready    <none>   5d21h   v1.13.0
worker-2   Ready    <none>   5d21h   v1.13.0
```

```
kubectl get pods
Error from server (Forbidden): nodes
"worker-1" is forbidden: User “Bot-1"
delete resource "nodes"
```

```
kubectl get nodes
NAME       STATUS   ROLES    AGE     VERSION
worker-1   Ready    <none>   5d21h   v1.13.0
worker-2   Ready    <none>   5d21h   v1.13.0
```

```
kubectl get nodes
NAME       STATUS   ROLES    AGE     VERSION
worker-1   Ready    <none>   5d21h   v1.13.0
worker-2   Ready    <none>   5d21h   v1.13.0
```

```
kubectl get nodes
Error from server (Forbidden): nodes
"worker-1" is forbidden: User “Bot-1"
delete resource "nodes"
```

```
kubectl delete node worker-2
Node worker-2 Deleted!
```

```
kubectl delete node worker-2
Error from server (Forbidden): nodes
"worker-1" is forbidden: User "developer"
cannot delete resource "nodes"
```

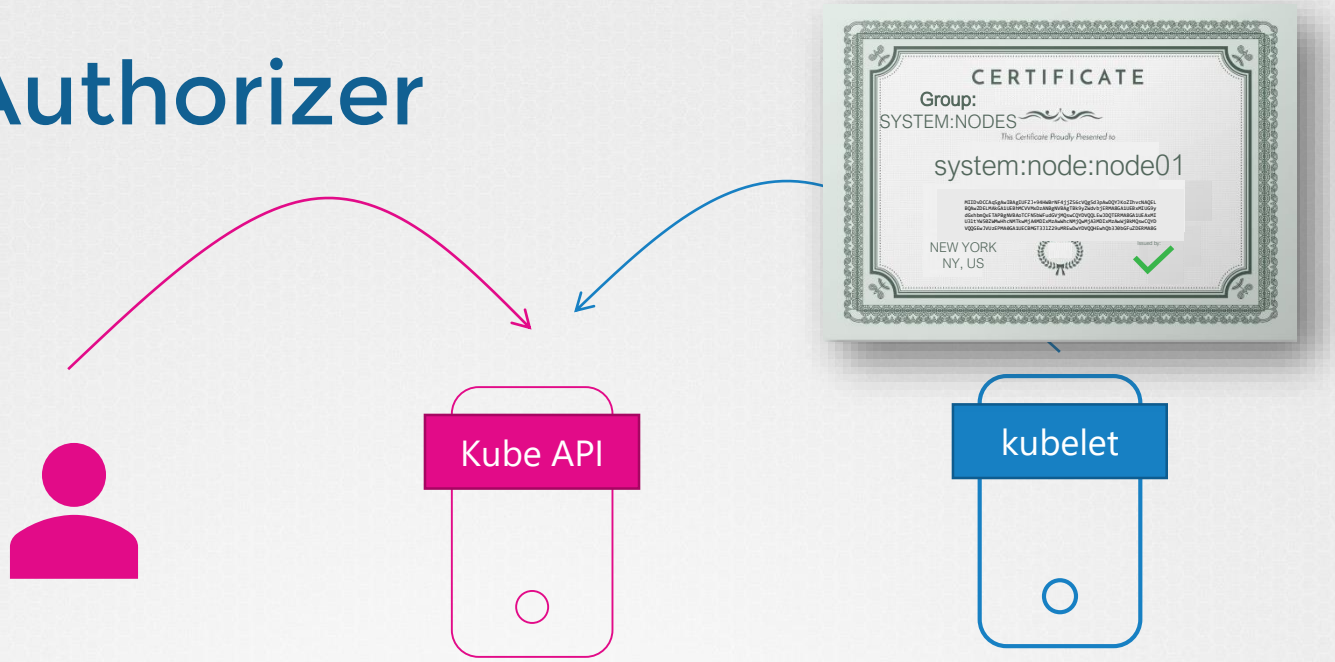
```
kubectl delete node worker
Error from server (Forbidden): nodes
"worker-1" is forbidden: User “Bot-1"
delete resource "nodes"
```

Authorization Mechanisms

Node | ABAC | RBAC | Webhook

KODEKLOUD

Node Authorizer



- Read
  - Services
  - Endpoints
  - Nodes
  - Pods
- Write
  - Node status
  - Pod status
  - events

KODEKLOUD

ABAC

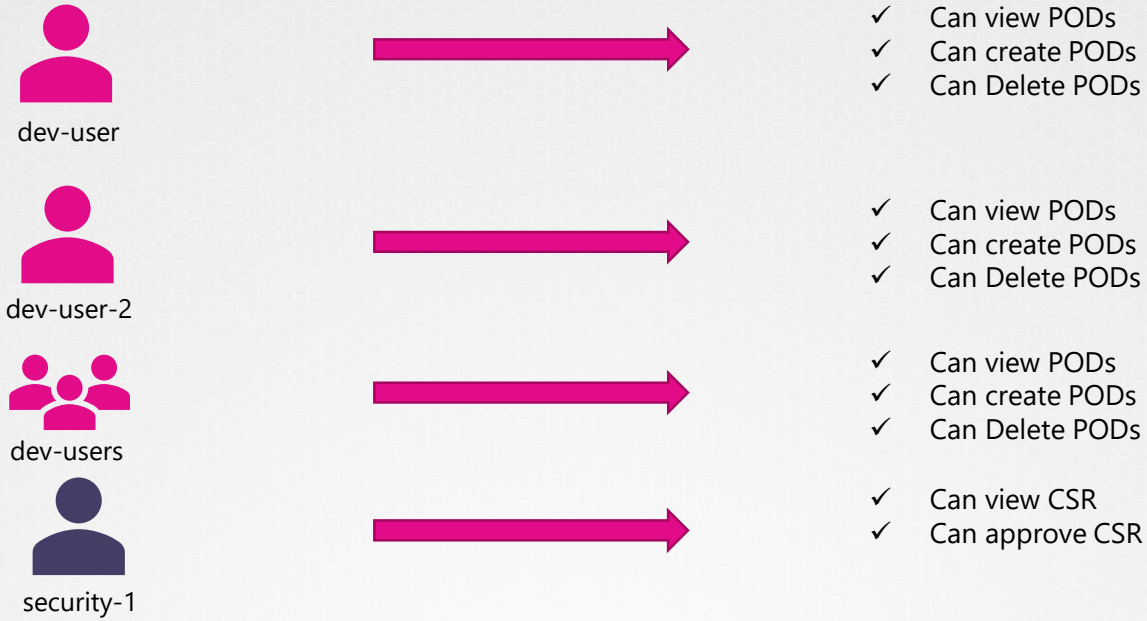
dev-user

- ✓ Can view PODs
- ✓ Can create PODs
- ✓ Can Delete PODs

```
{"kind": "Policy", "spec": {"user": "dev-user", "namespace": "*", "resource": "pods", "apiGroup": "*"}}
```

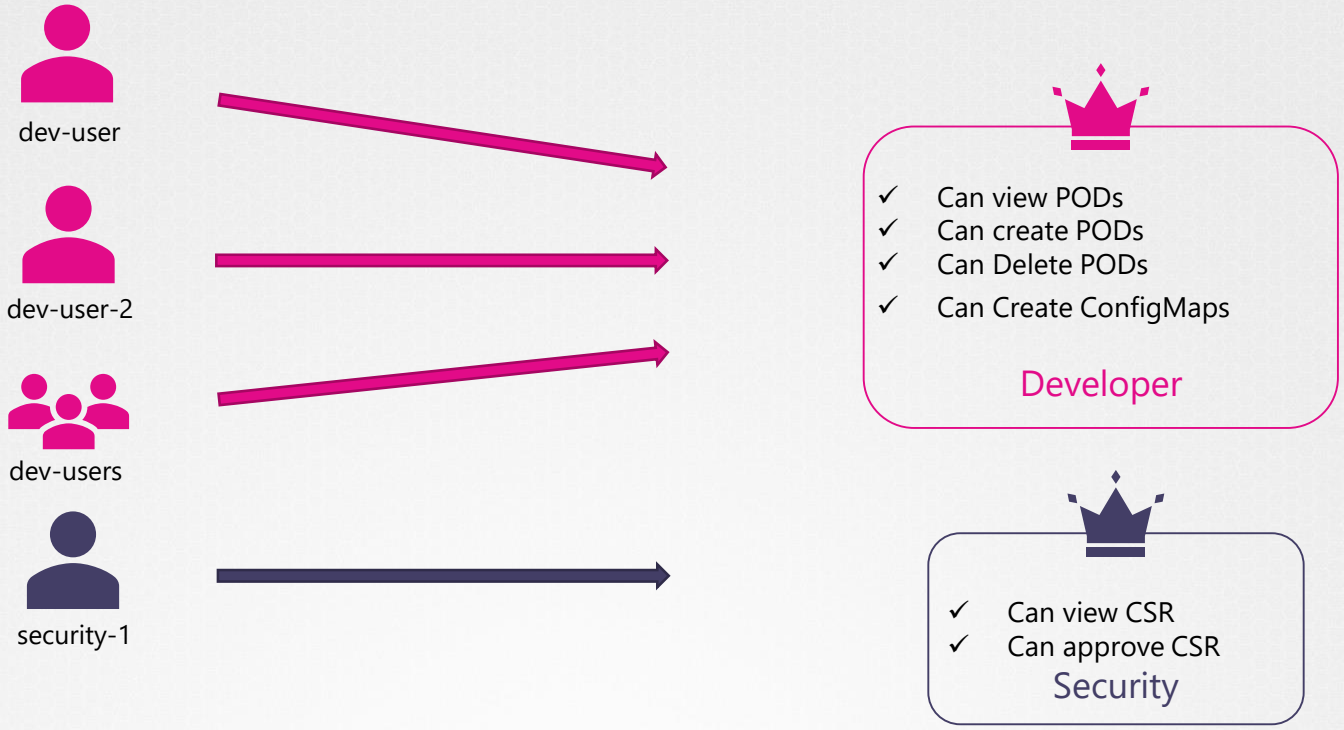
KODEKLOUD

ABAC



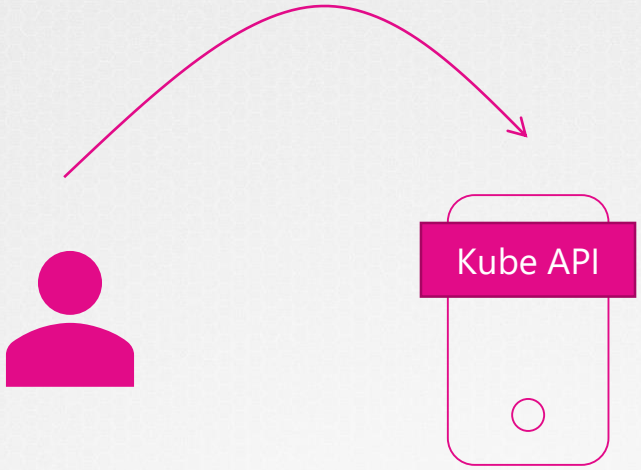

```
{"kind": "Policy", "spec": {"user": "dev-user", "namespace": "*", "resource": "pods", "apiGroup": "*"}}
{"kind": "Policy", "spec": {"user": "dev-user-2", "namespace": "*", "resource": "pods", "apiGroup": "*"}}
{"kind": "Policy", "spec": {"group": "dev-users", "namespace": "*", "resource": "pods", "apiGroup": "*"}}
{"kind": "Policy", "spec": {"user": "security-1", "namespace": "*", "resource": "csr", "apiGroup": "*"}}
```

RBAC



KODEKLOUD

Webhook



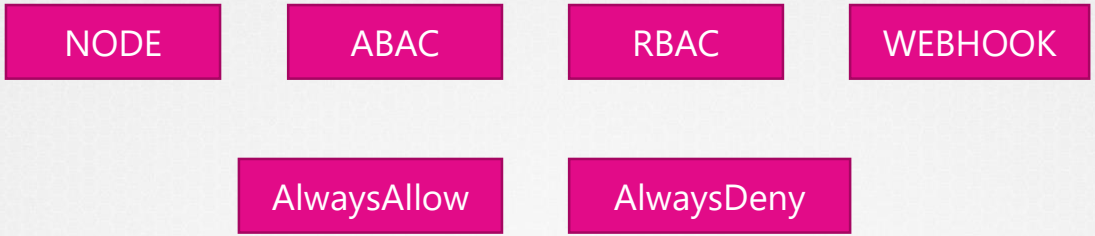
Open Policy Agent

User **dev-user** requested read access to **Pods**. Should I allow?

I checked. Yes!

KODEKLOUD

Authorization Mode



KODEKLOUD

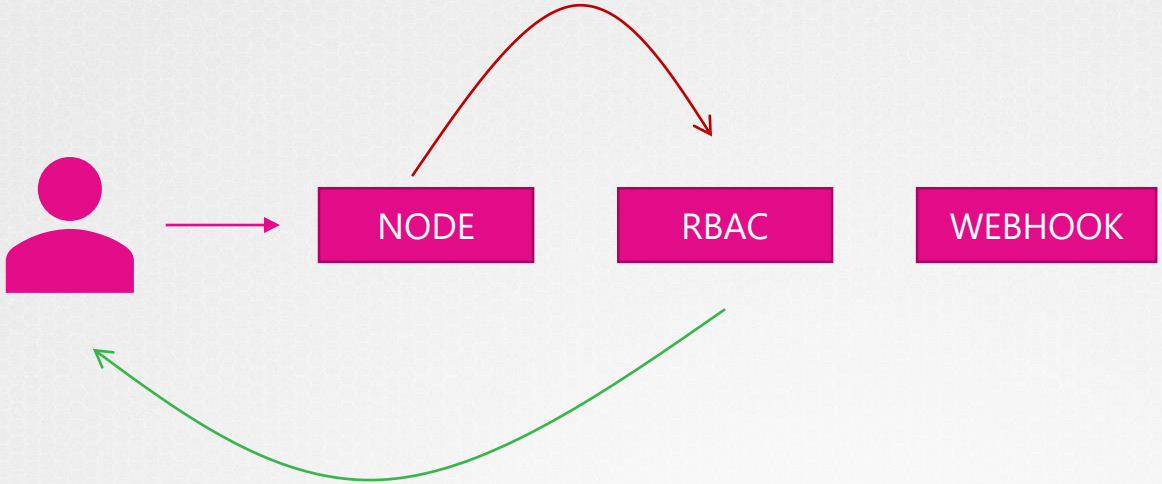
Authorization Mode

AlwaysAllow | NODE | ABAC | RBAC | WEBHOOK | AlwaysDeny

```
ExecStart=/usr/local/bin/kube-apiserver \\
  --advertise-address=${INTERNAL_IP} \\
  --allow-privileged=true \\
  --apiserver-count=3 \\
  --authorization-mode=Node,RBAC,Webhook \\
  --bind-address=0.0.0.0 \\
  --enable-swagger-ui=true \\
  --etcd-cafile=/var/lib/kubernetes/ca.pem \\
  --etcd-certfile=/var/lib/kubernetes/apiserver-etcd-client.crt \\
  --etcd-keyfile=/var/lib/kubernetes/apiserver-etcd-client.key \\
  --etcd-servers=https://127.0.0.1:2379 \\
  --event-ttl=1h \\
  --kubelet-certificate-authority=/var/lib/kubernetes/ca.pem \\
  --kubelet-client-certificate=/var/lib/kubernetes/apiserver-etcd-client.crt \\
  --kubelet-client-key=/var/lib/kubernetes/apiserver-etcd-client.key \\
  --service-node-port-range=30000-32767 \\
  --client-ca-file=/var/lib/kubernetes/ca.pem \\
  --tls-cert-file=/var/lib/kubernetes/apiserver.crt \\
  --tls-private-key-file=/var/lib/kubernetes/apiserver.key \\
  --v=2
```

KODEKLOUD

Authorization Mode




```
ExecStart=/usr/local/bin/kube-apiserver \\
  --advertise-address=${INTERNAL_IP} \\
  --allow-privileged=true \\
  --apiserver-count=3 \\
  --authorization-mode=Node,RBAC,Webhook \\
  --bind-address=0.0.0.0 \\
```

DE{KLOUD

KODEKLOUD

Course Objectives

- Core Concepts
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
  - Kubernetes Security Primitives
  - Secure Persistent Key Value Store
  - Authentication
  - Authorization
  - Security Contexts
  - TLS Certificates for Cluster Components
  - Images Securely
  - Network Policies
- Storage
- Networking
- Installation, Configuration & Validation
- Troubleshooting

KODEKLOUD

RBAC

KODEKLOUD

RBAC

- ✓ Can view PODs
- ✓ Can create PODs
- ✓ Can Delete PODs
- ✓ Can Create ConfigMaps

Developer

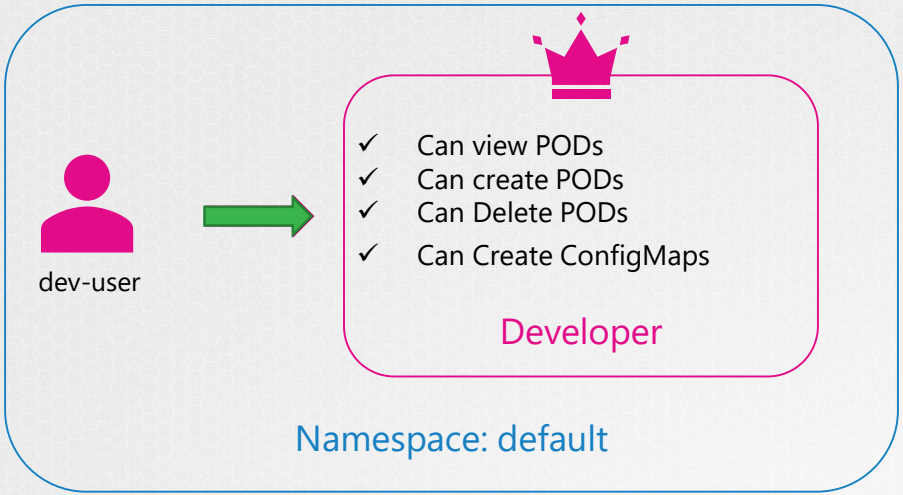
developer-role.yaml

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: developer
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["list", "get", "create", "update", "delete"]
- apiGroups: [""]
  resources: ["ConfigMap"]
  verbs: ["create"]
```

```
kubectl create -f developer-role.yaml
```

KODEKLOUD

RBAC




```
kubectl create -f devuser-developer-binding.yaml
```

developer-role.yaml

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: developer
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["list", "get", "create", "update", "de
- apiGroups: [""]
  resources: ["ConfigMap"]
  verbs: ["create"]
```

devuser-developer-binding.yaml

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: devuser-developer-binding
subjects:
- kind: User
  name: dev-user
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: developer
  apiGroup: rbac.authorization.k8s.io
```

View RBAC

```
kubectl get roles
```

```
NAME        AGE
developer   4s
```

```
kubectl get rolebindings
```

```
NAME                        AGE
devuser-developer-binding   24s
```

```
kubectl describe role developer
```

```
Name:         developer
Labels:       <none>
Annotations:  <none>
PolicyRule:
  Resources  Non-Resource URLs  Resource Names  Verbs
  ---------  -----------------  --------------  -----
  ConfigMap  []                 []              [create]
  pods       []                 []              [get watch list create delete]
```

KODEKLOUD

View RBAC

```
kubectl describe rolebinding devuser-developer-binding
Name:         devuser-developer-binding
Labels:       <none>
Annotations:  <none>
Role:
  Kind:  Role
  Name:  developer
Subjects:
  Kind  Name      Namespace
  ----  ----      ---------
  User  dev-user
```

KODEKLOUD

Check Access

```
kubectl auth can-i create deployments
yes
```

```
kubectl auth can-i delete nodes
no
```

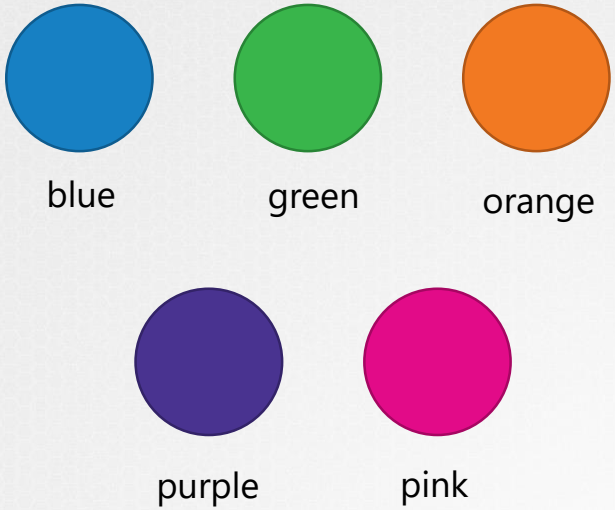
```
kubectl auth can-i create deployments --as dev-user
no
```

```
kubectl auth can-i create pods --as dev-user
yes
```

```
kubectl auth can-i create pods --as dev-user --namespace test
no
```

KODEKLOUD

Resource Names



developer-role.yaml

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: developer
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "create", "update"]
  resourceNames: ["blue", "orange"]
```

KODEKLOUD

KODEKLOUD

Course Objectives

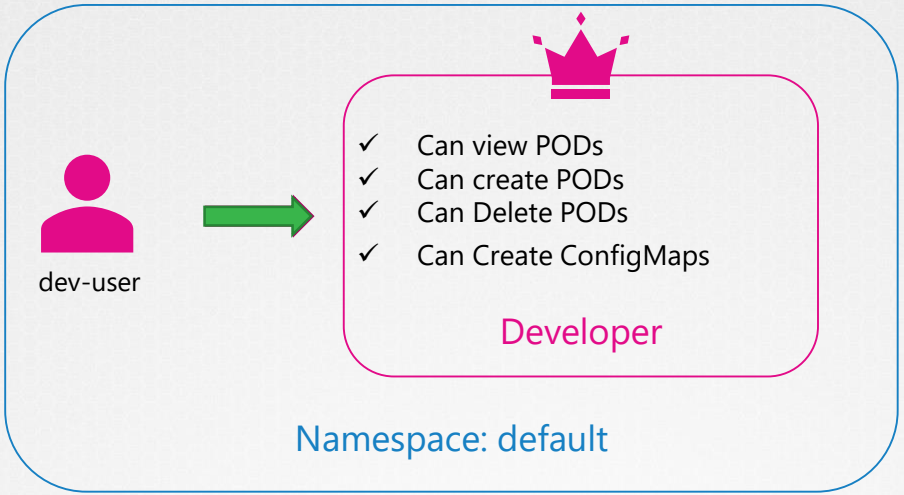
- Core Concepts
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
  - Kubernetes Security Primitives
  - Secure Persistent Key Value Store
  - Authentication
  - Authorization
  - Security Contexts
  - TLS Certificates for Cluster Components
  - Images Securely
  - Network Policies
- Storage
- Networking
- Installation, Configuration & Validation
- Troubleshooting

KODEKLOUD

Cluster Roles

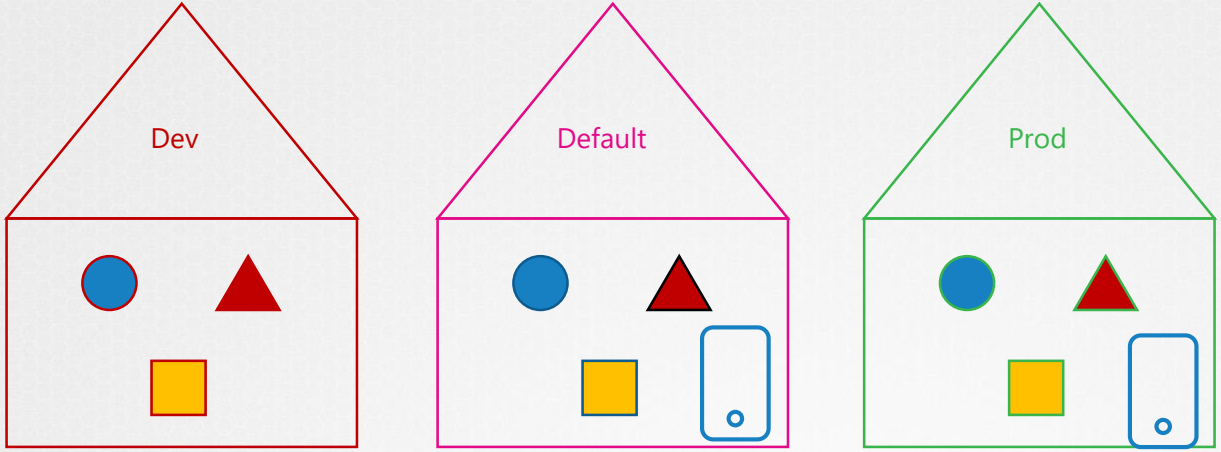
KODEKLOUD

Roles



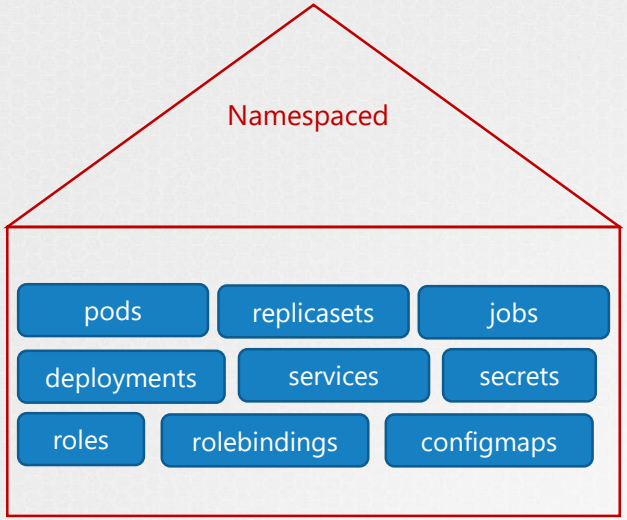
KODEKLOUD

Namespace



KODEKLOUD

Namespace



Cluster Scoped

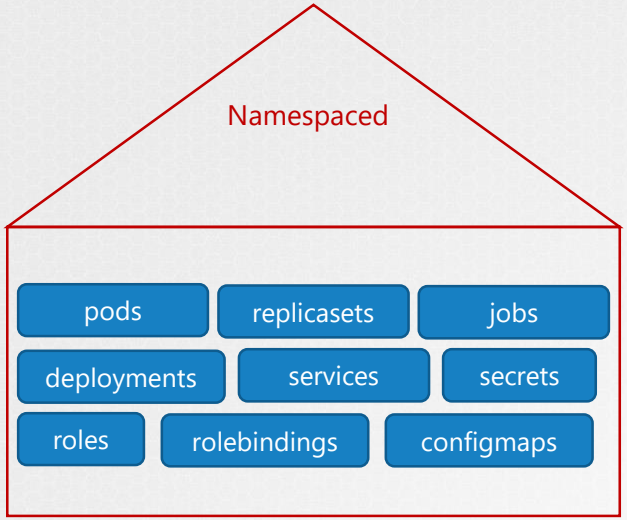
nodes
PV
PVC
clusterroles
clusterrolebindings
certificatesigningrequests
namespaces

```
kubectl api-resources --namespaced=true
```

```
kubectl api-resources --namespaced=false
```

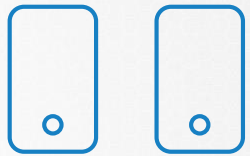
KODEKLOUD

Namespace



Cluster Scoped

nodes
PV
PVC
clusterroles
clusterrolebindings
certificatesigningrequests
namespaces



KODEKLOUD

clusterroles

- ✓ Can view Nodes
- ✓ Can create Nodes
- ✓ Can delete Nodes

Cluster Admin

- ✓ Can view PVs
- ✓ Can create PVs
- ✓ Can delete PVCs

Storage Admin

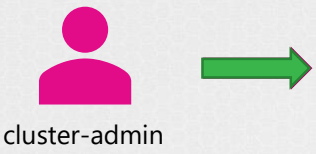
cluster-admin-role.yaml

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: cluster-administrator
rules:
- apiGroups: [""]
  resources: ["nodes"]
  verbs: ["list", "get", "create", "delete"]
```

```
kubectl create -f cluster-admin-role.yaml
```

KODEKLOUD

clusterrolebinding



- ✓ Can view Nodes
- ✓ Can create Nodes
- ✓ Can delete Nodes

Cluster Admin

cluster-admin-role.yaml

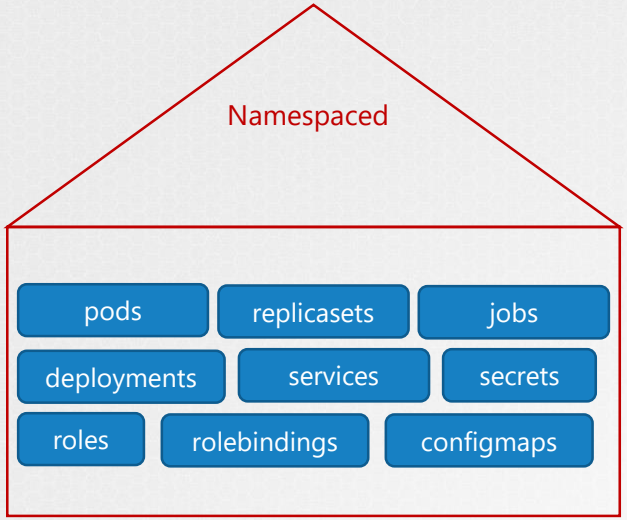
```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: cluster-administrator
rules:
- apiGroups: [""]
  resources: ["nodes"]
  verbs: ["list", "get", "create", "delete"]
```

cluster-admin-role-binding.yaml

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: cluster-admin-role-binding
subjects:
- kind: User
  name: cluster-admin
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: cluster-administrator
  apiGroup: rbac.authorization.k8s.io
```

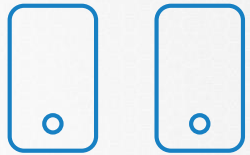
```
kubectl create -f cluster-admin-role-binding.yaml
```

Cluster Roles



Cluster Scoped

nodes
PV
PVC
clusterroles
clusterrolebindings
certificatesigningrequests
namespaces



KODEKLOUD

KODEKLOUD

Course Objectives

- Core Concepts
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
  - Kubernetes Security Primitives
  - Secure Persistent Key Value Store
  - Authentication
  - Authorization
  - Security Contexts
  - TLS Certificates for Cluster Components
  - mages Securely
  - Network Policies
- Storage
- Networking
- Installation, Configuration & Validation
- Troubleshooting

KODEKLOUD

Image Security

Image

nginx-pod.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  containers:
  - name: nginx
    image: nginx
```

KODEKLOUD

Image

image: `docker.io/nginx/nginx`

Registry

User/
Account

Image/
Repository

`gcr.io/ kubernetes-e2e-test-images/dnsutils`

KODEKLOUD

Private Repository

```
docker login private-registry.io
```

```
Login with your Docker ID to push and pull images from Docker Hub. If you don't have a Docker ID, head over to
https://hub.docker.com to create one.
Username: registry-user
Password:
WARNING! Your password will be stored unencrypted in /home/vagrant/.docker/config.json.

Login Succeeded
```

```
docker run private-registry.io/apps/internal-app
```

KODEKLOUD

Private Repository

```
docker login private-registry.io
```

```
docker run private-registry.io/apps/internal-app
```

nginx-pod.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  containers:
  - name: nginx
    image:
  imagePullSecrets:
  - name: regcred
```

```
kubectl create secret docker-registry regcred \
    --docker-server= private-registry.io     \
    --docker-username= registry-user  \
    --docker-password= registry-password  \
    --docker-email= registry-user@org.com
```

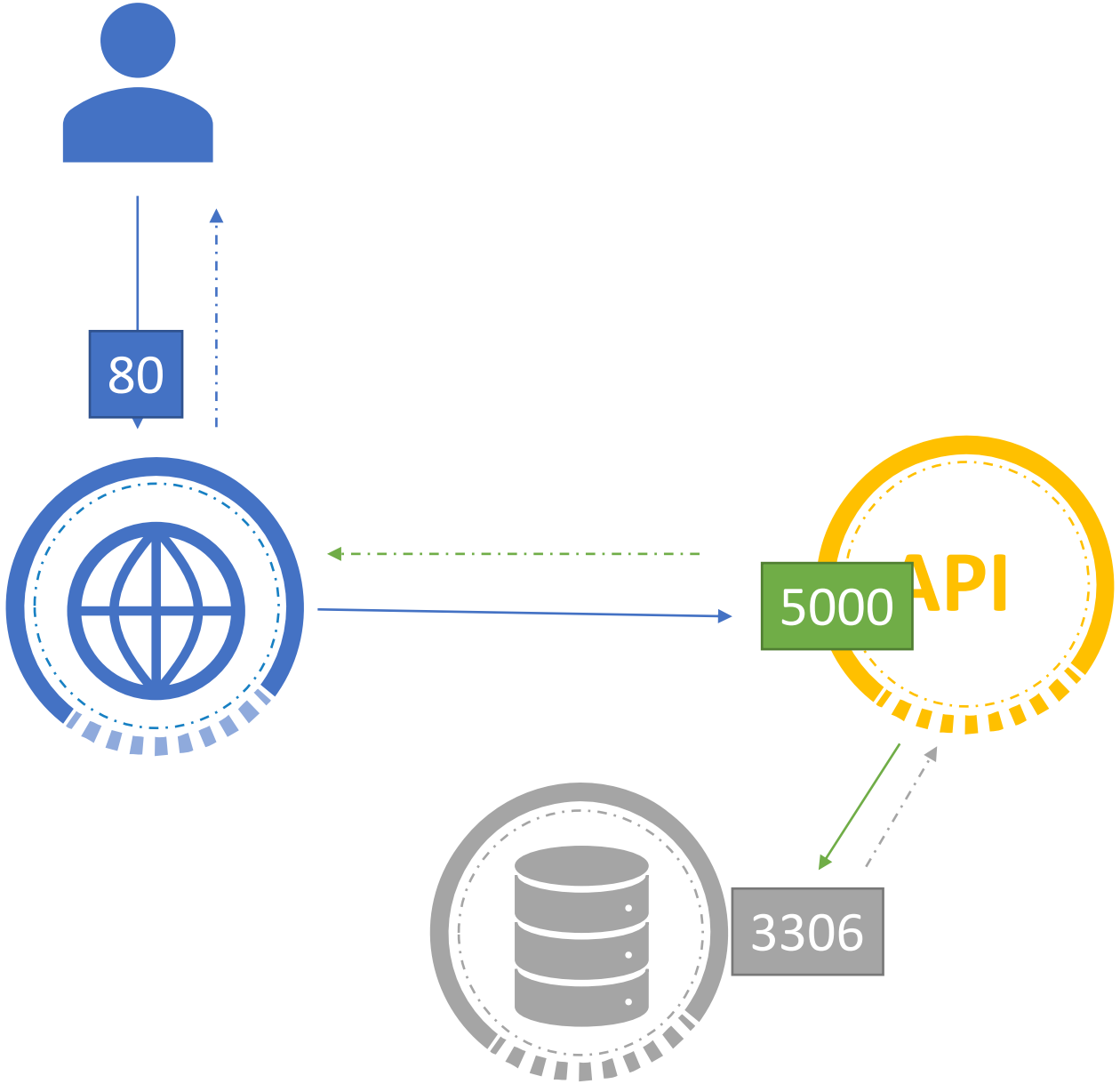
KODEKLOUD

KODEKLOUD

Network Policies

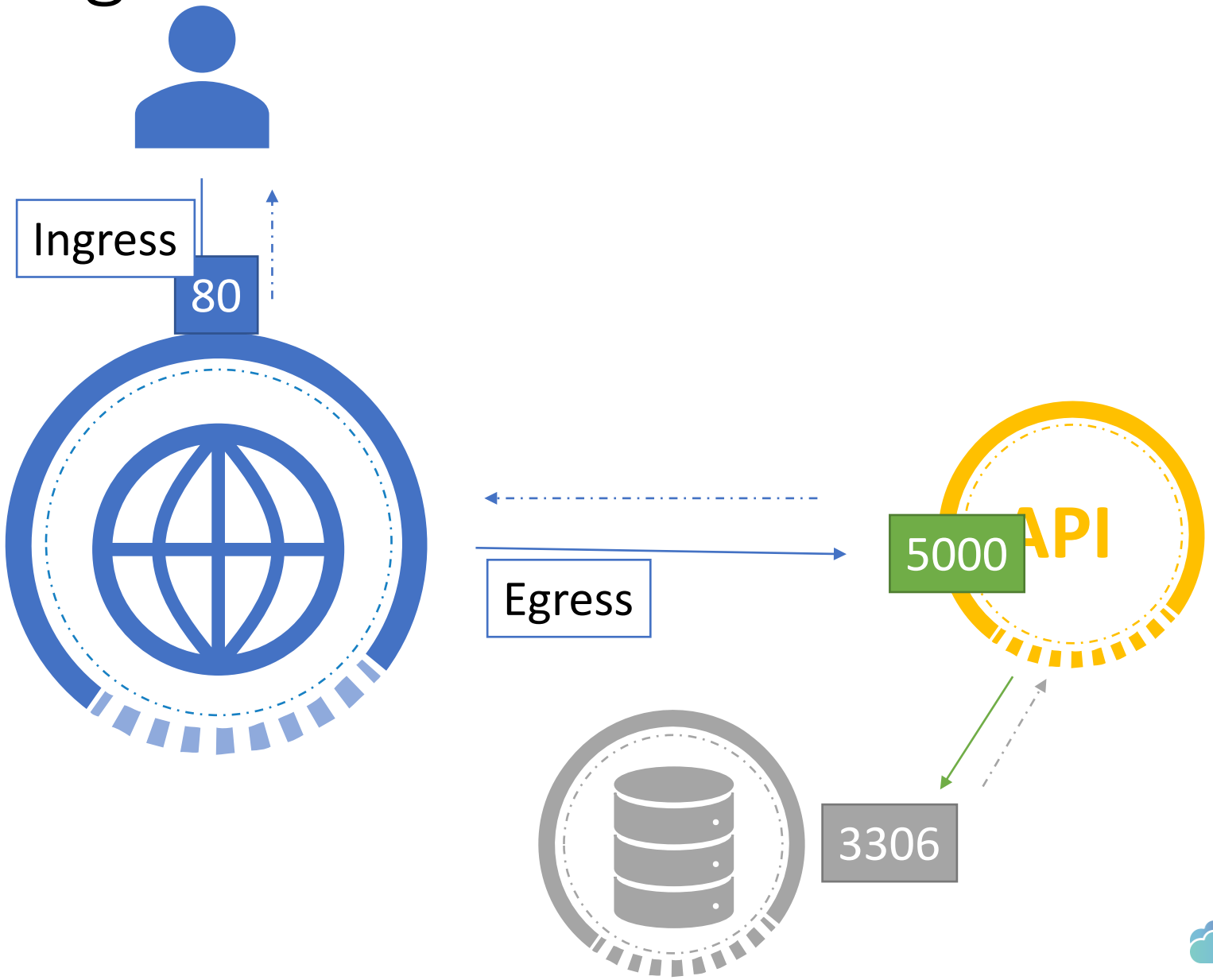
KODEKLOUD

Traffic



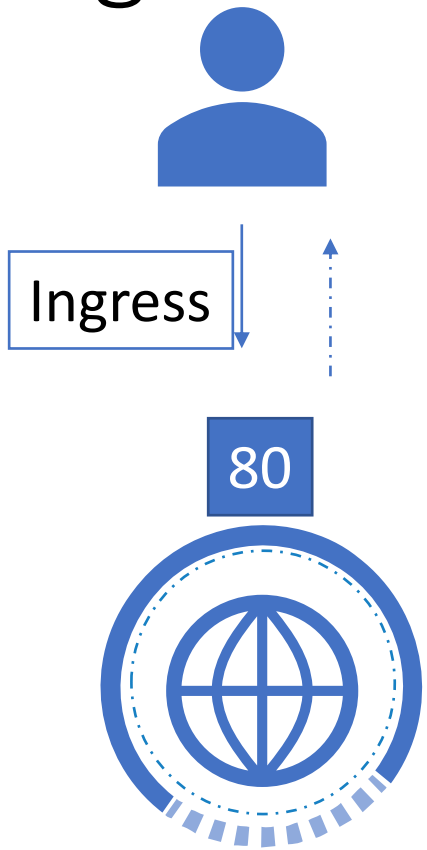
KODEKLOUD

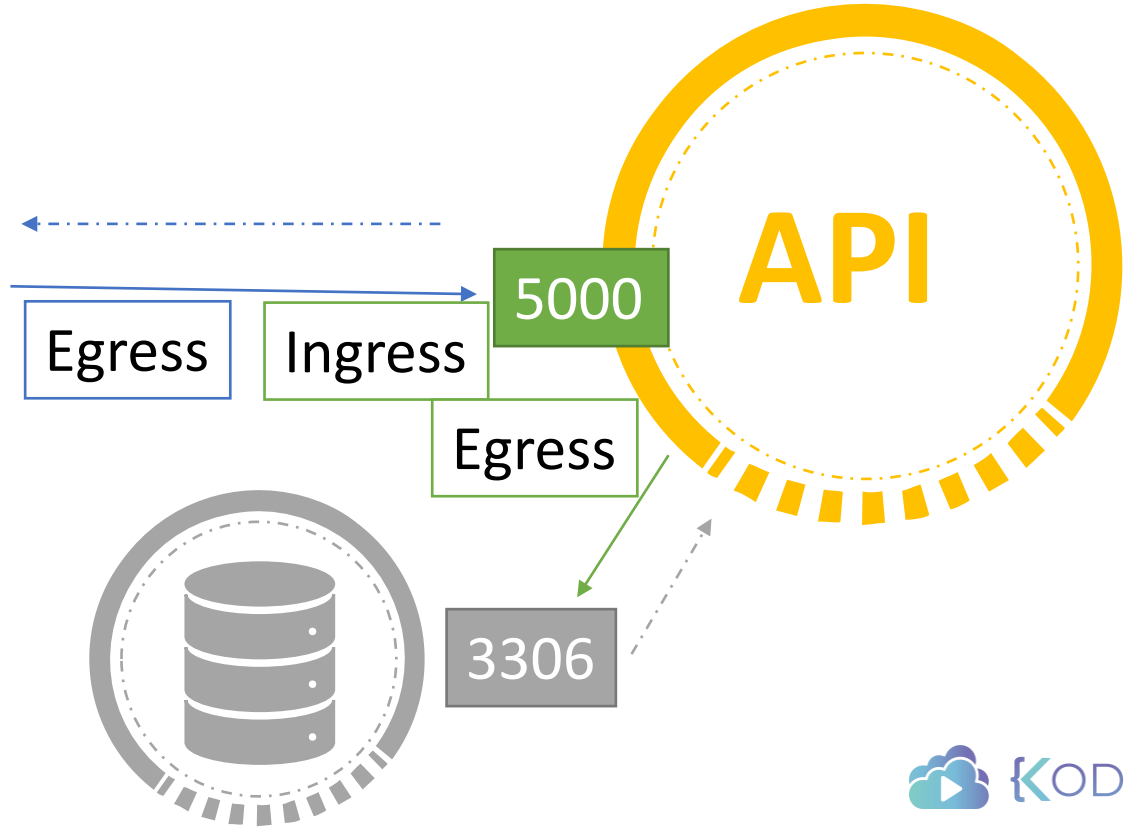
Ingress & Egress



KODEKLOUD

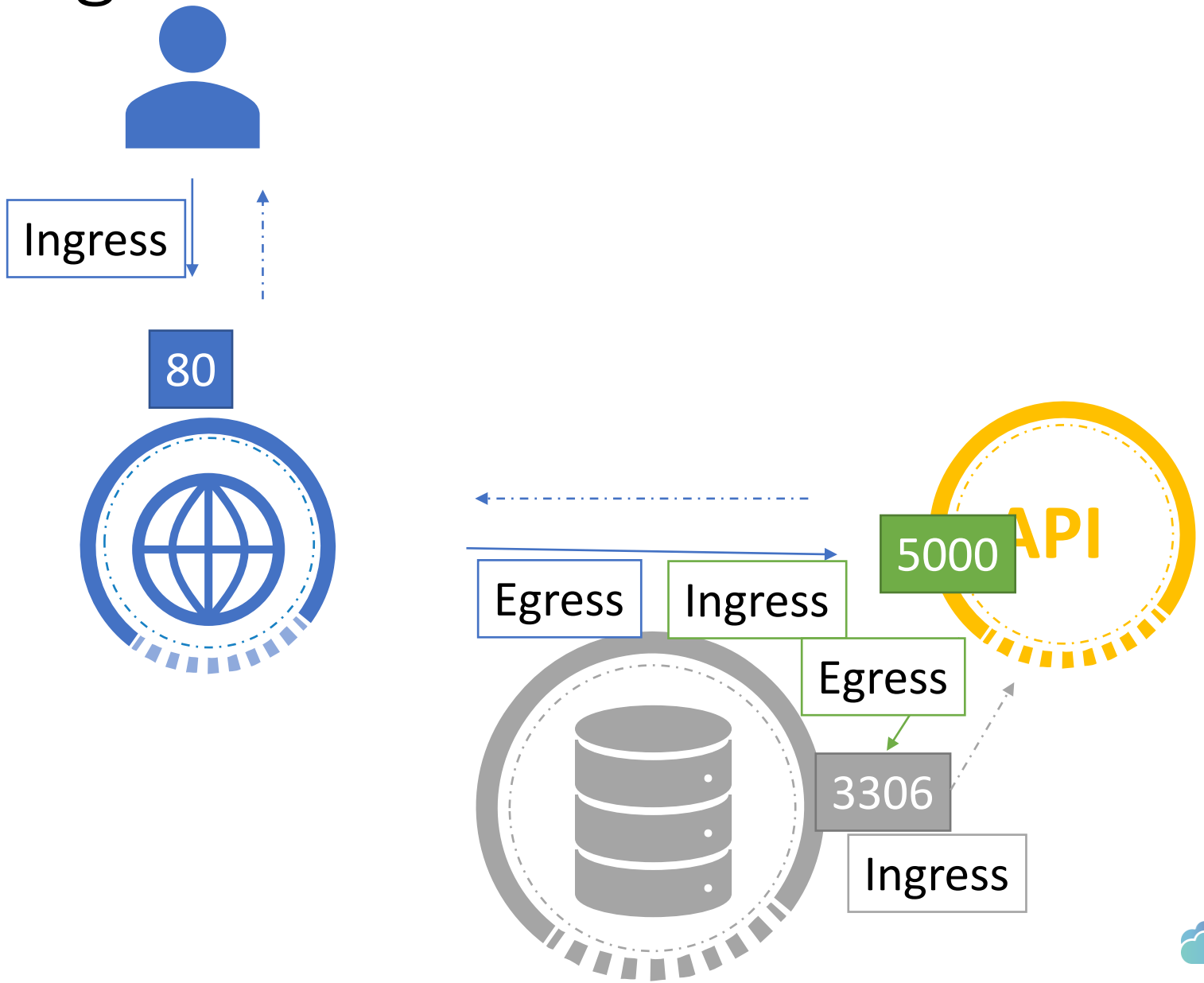
Ingress & Egress





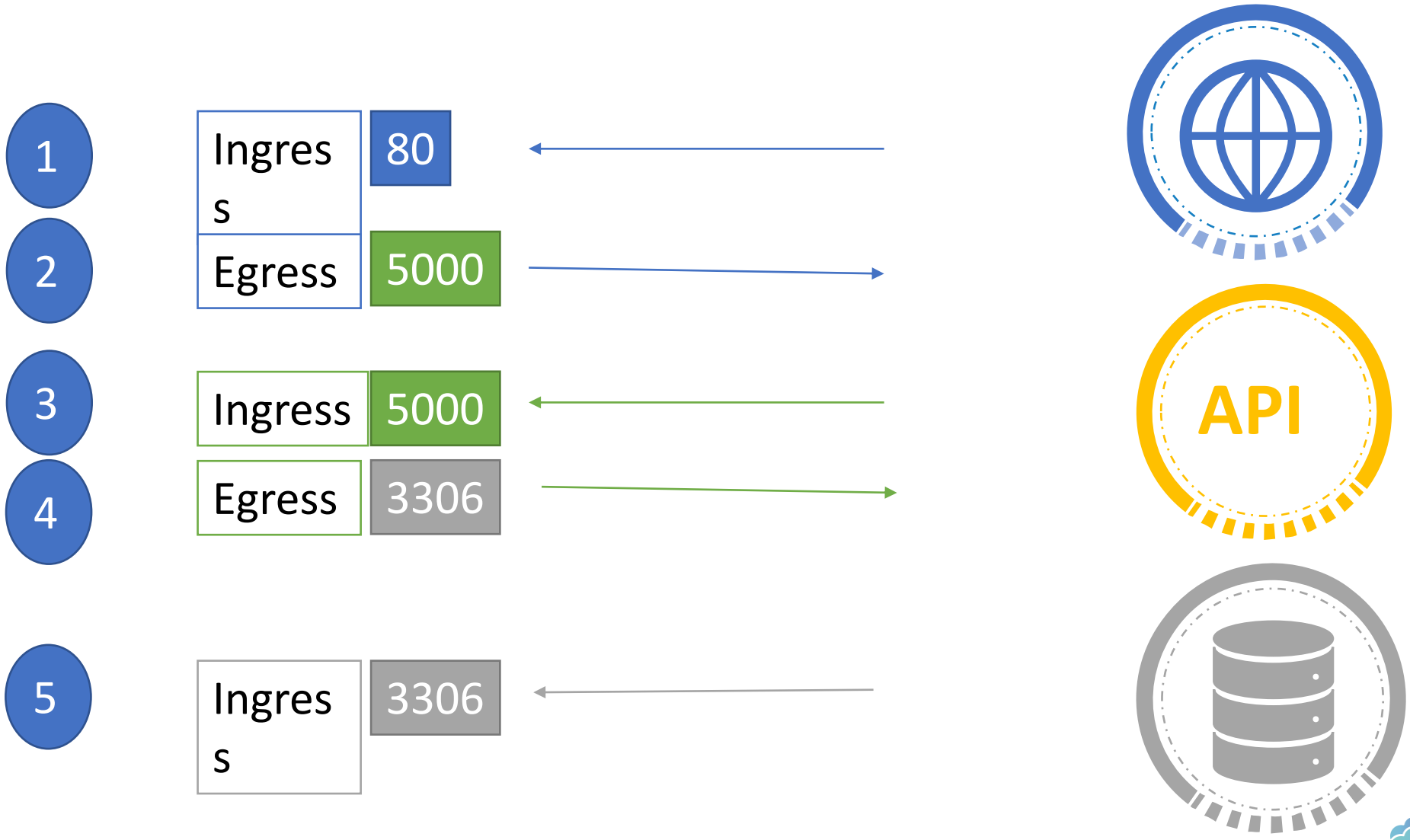
KODEKLOUD

Ingress & Egress



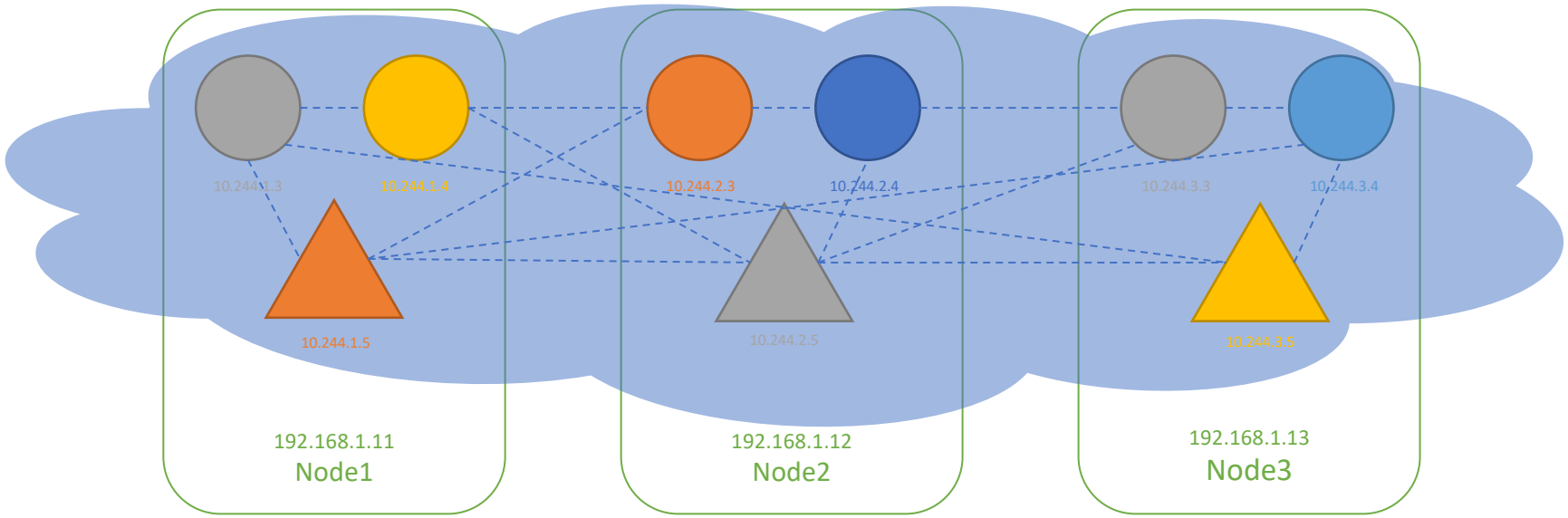
KODEKLOUD

Traffic



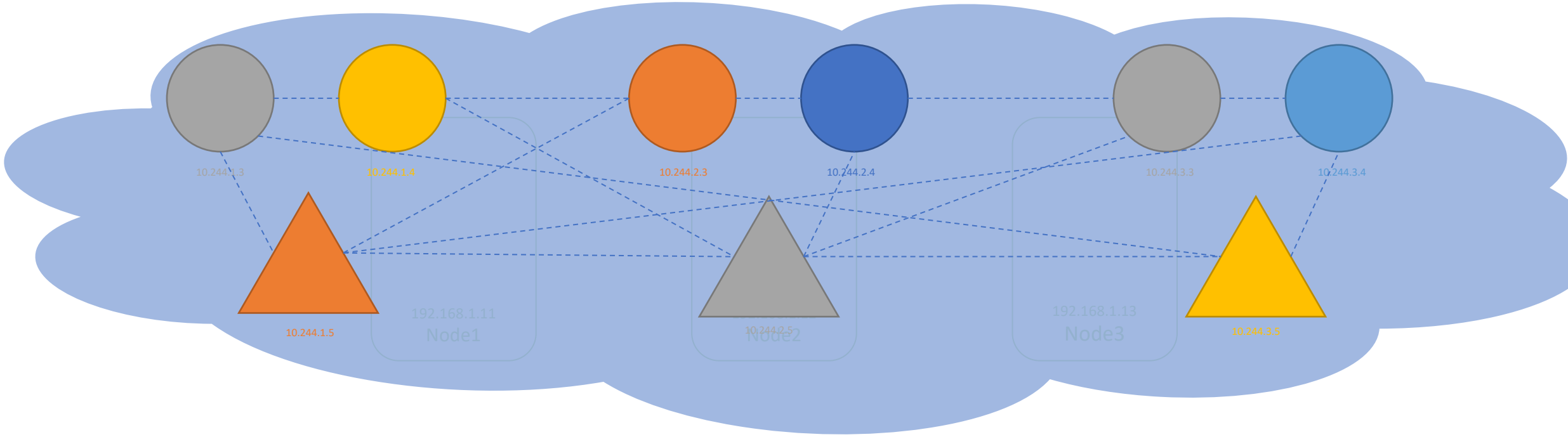
KODEKLOUD

Network Security



KODEKLOUD

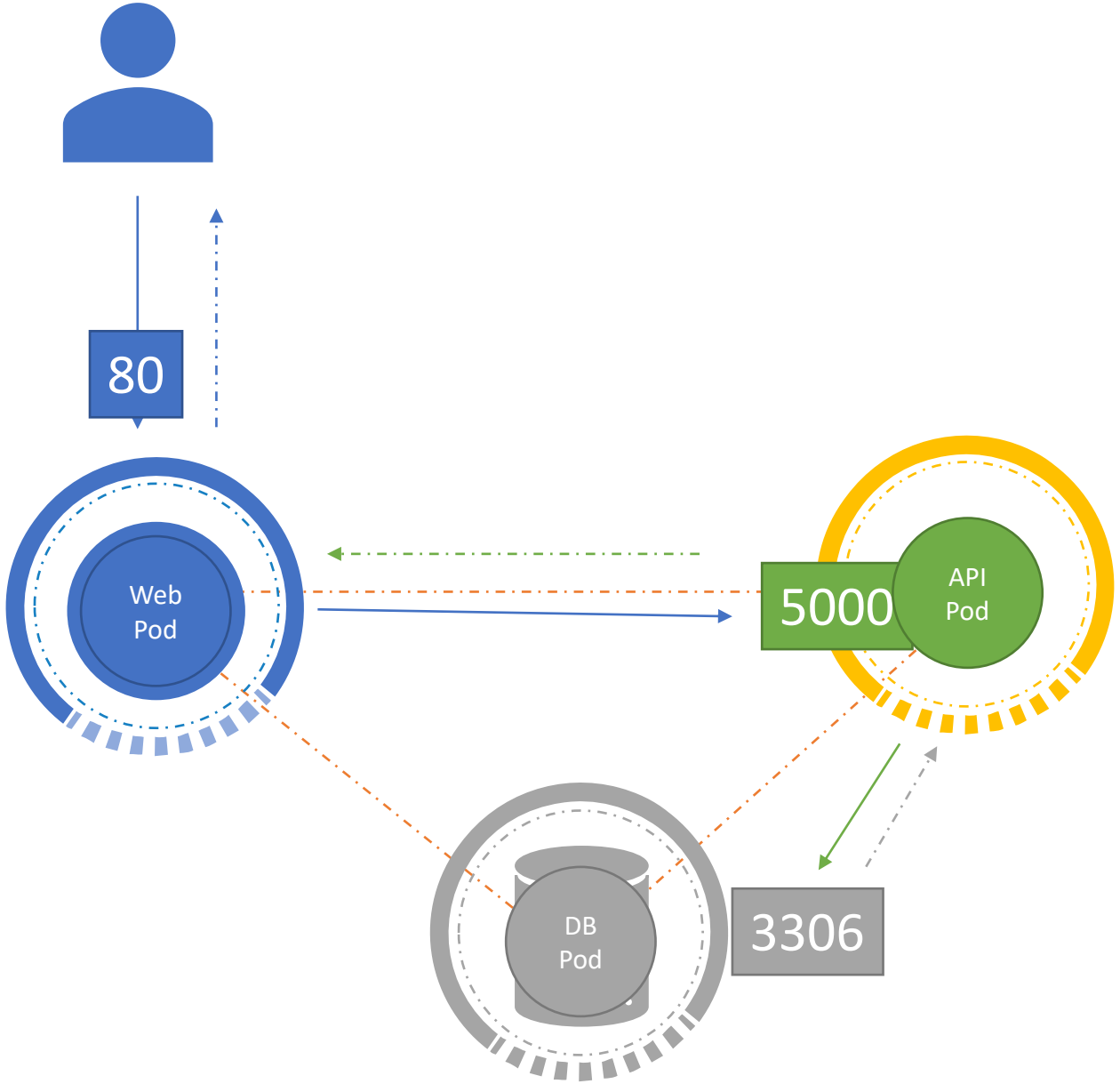
Network Security



"All Allow"

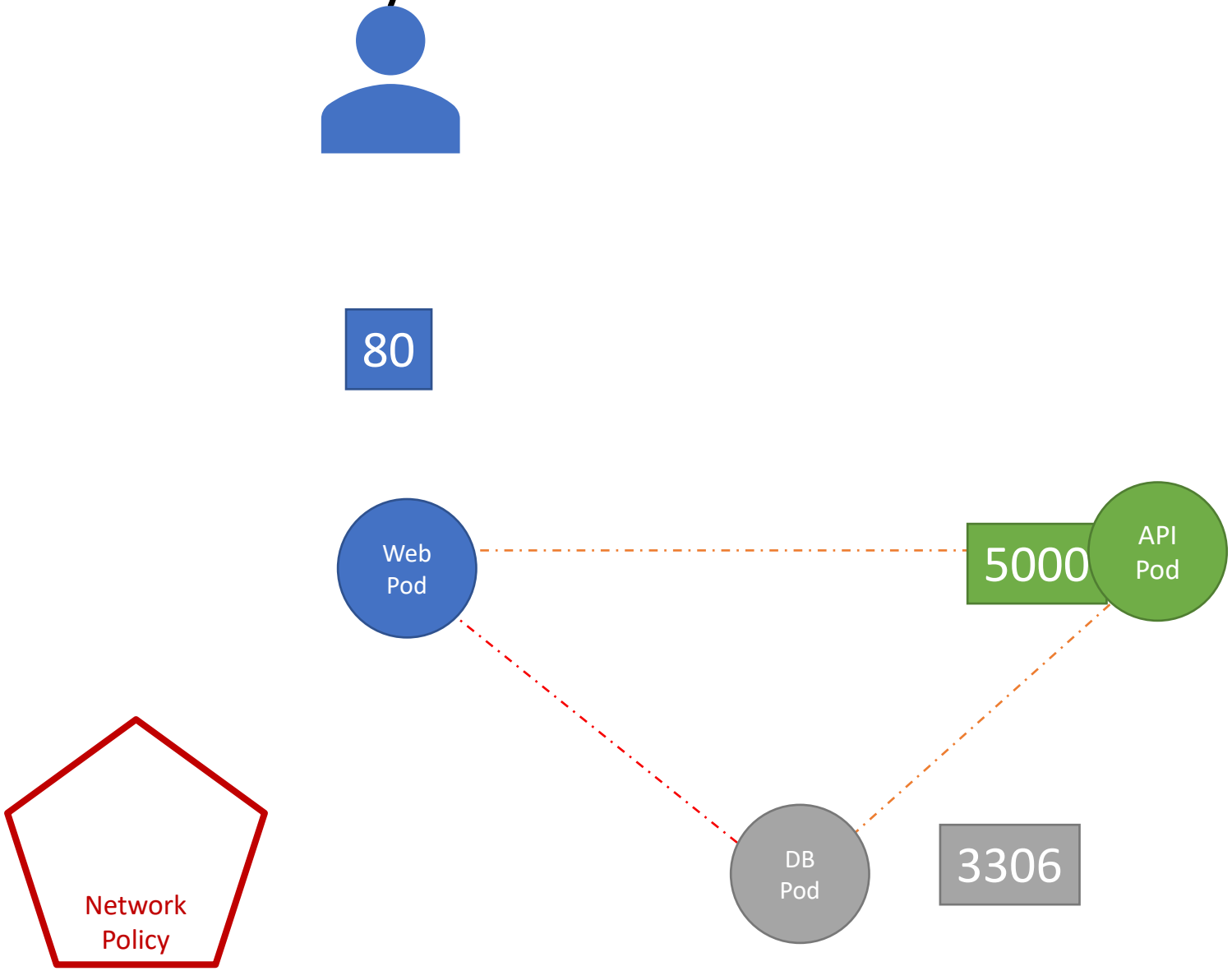
KODEKLOUD

Traffic



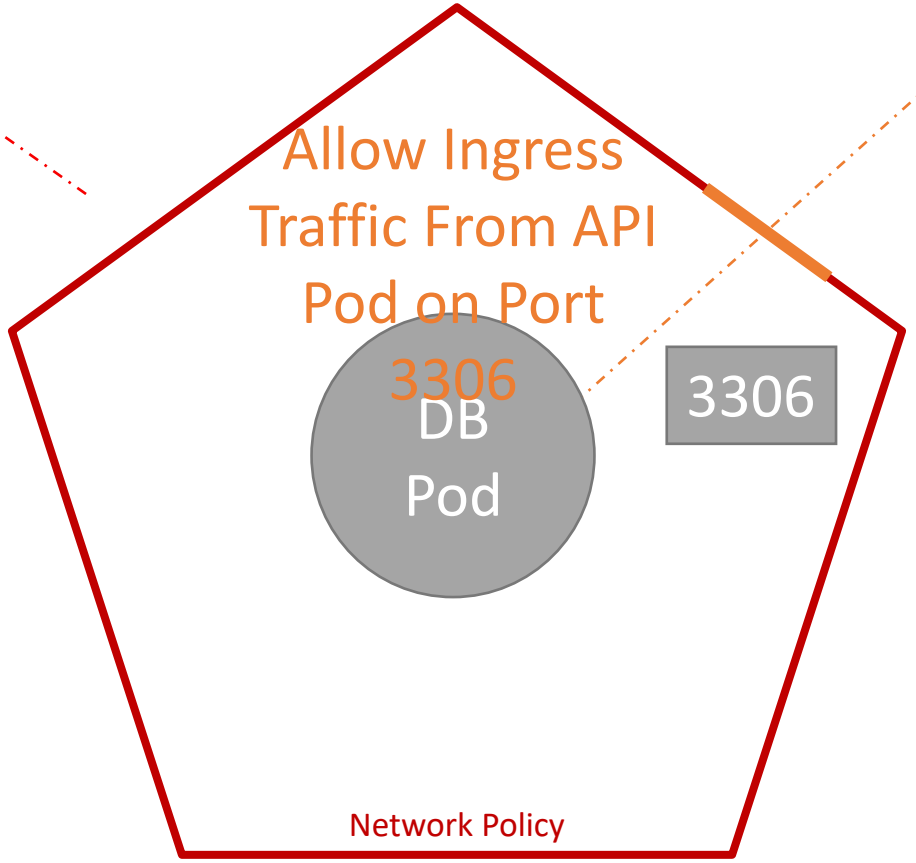
KODEKLOUD

Network Policy



KODEKLOUD

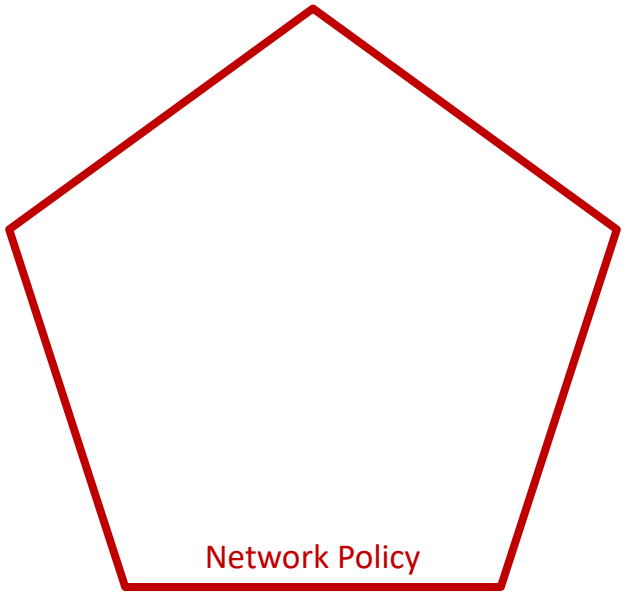
Network Policy



KODEKLOUD

Network Policy - Selectors

Allow Ingress
Traffic From API
Pod on Port
3306

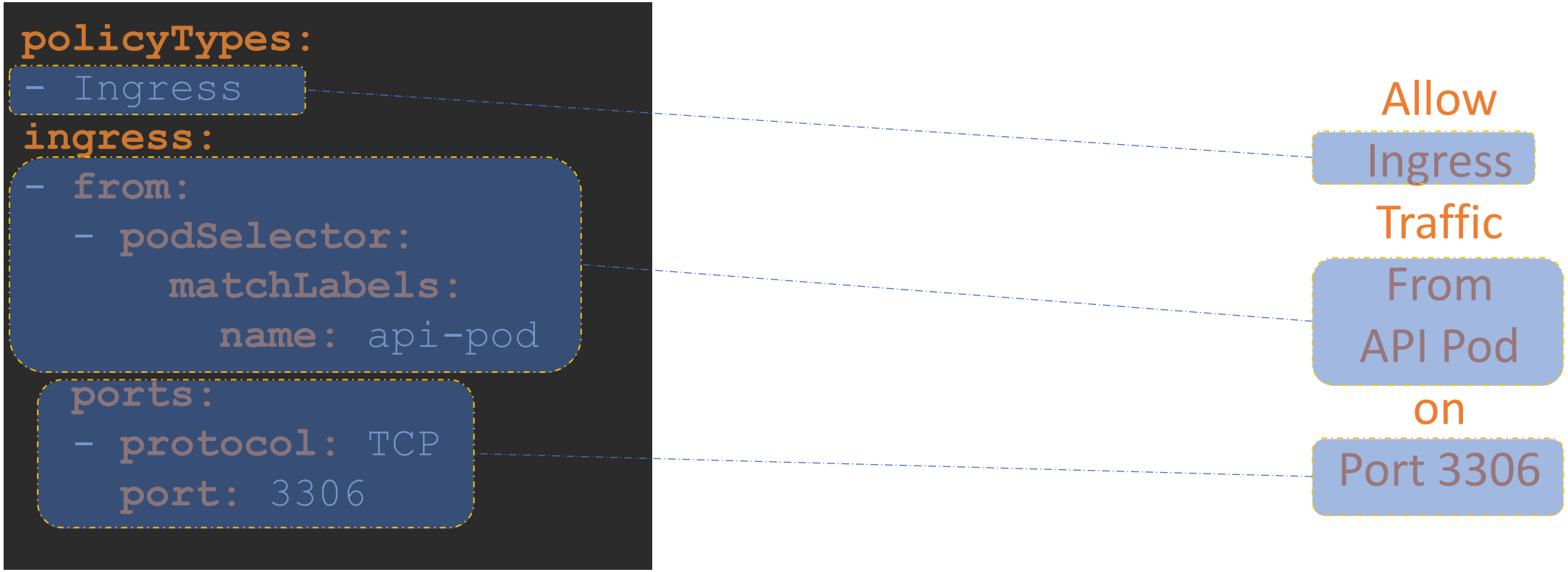


```
podSelector:
  matchLabels:
    role: db
```

```
labels:
  role: db
```

KODEKLOUD

Network Policy - Rules



KODEKLOUD

Network Policy

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-policy
spec:
```

```
kubectl create -f policy-definition.yaml
```

```
    matchLabels:
      role: db
```

```
policyTypes:
- Ingress
ingress:
- from:
  - podSelector:
      matchLabels:
        name: api-pod
  ports:
  - protocol: TCP
    port: 3306
```

KODEKLOUD

Note

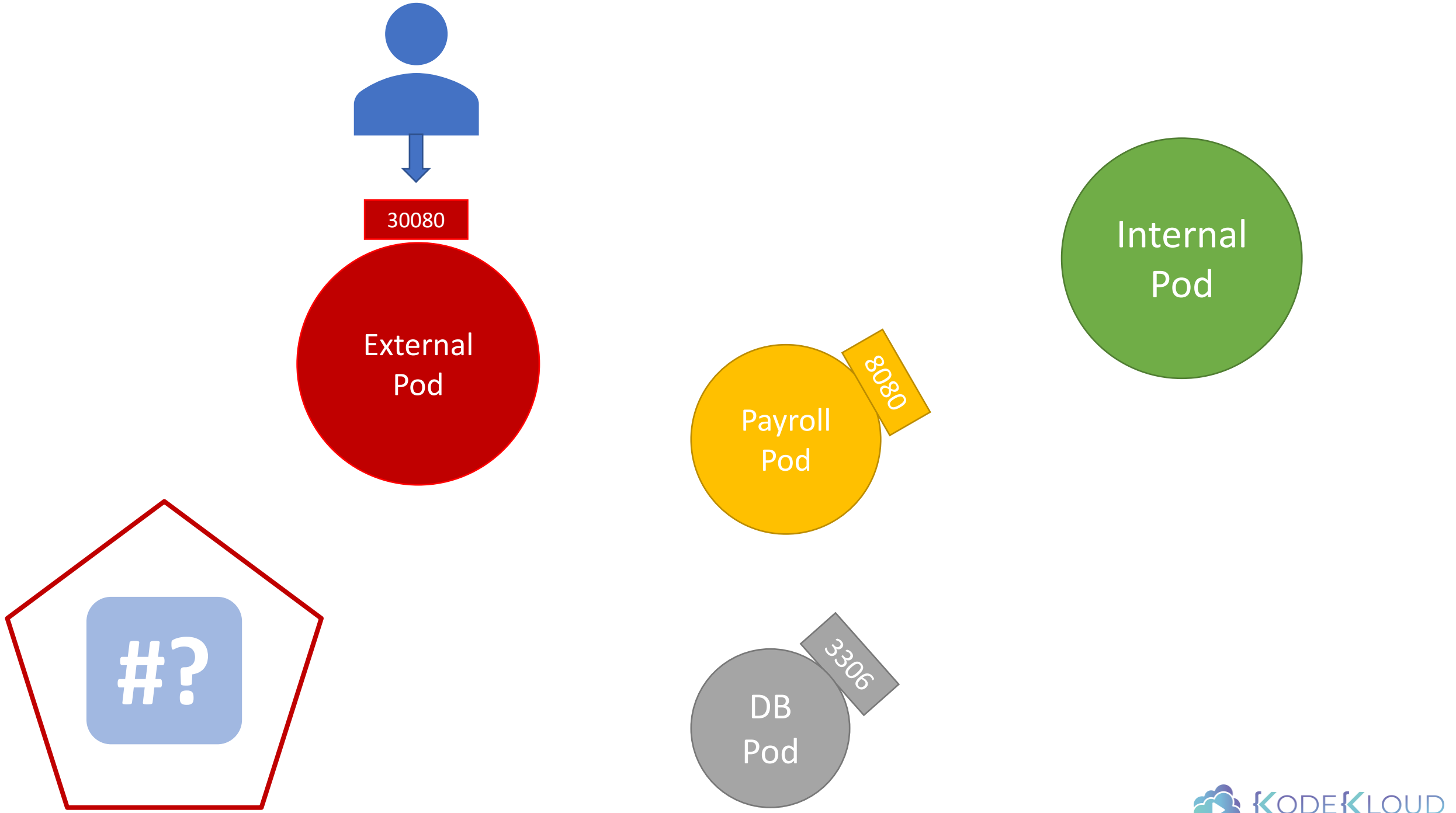
Solutions that Support Network Policies:

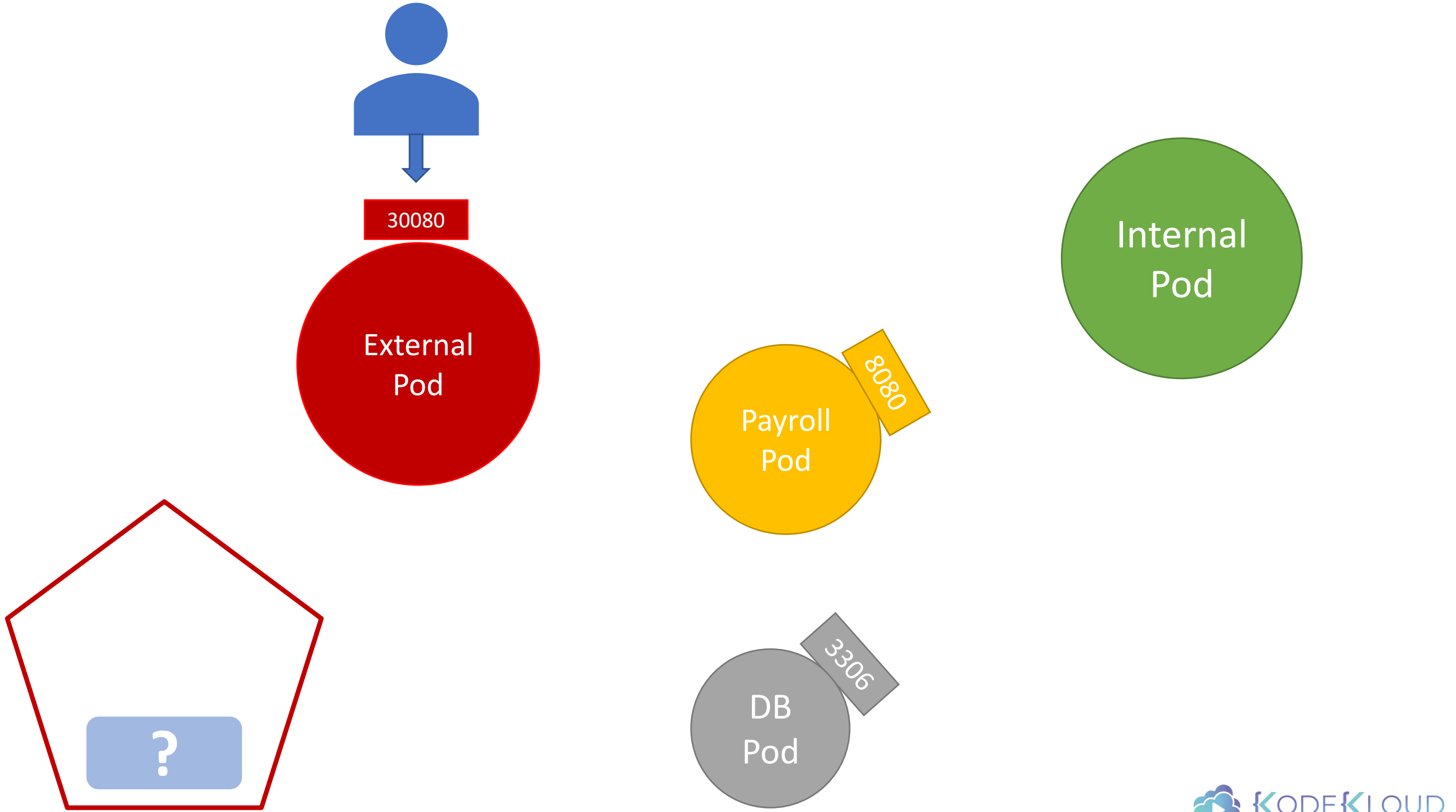
- Kube-router
- Calico
- Romana
- Weave-net

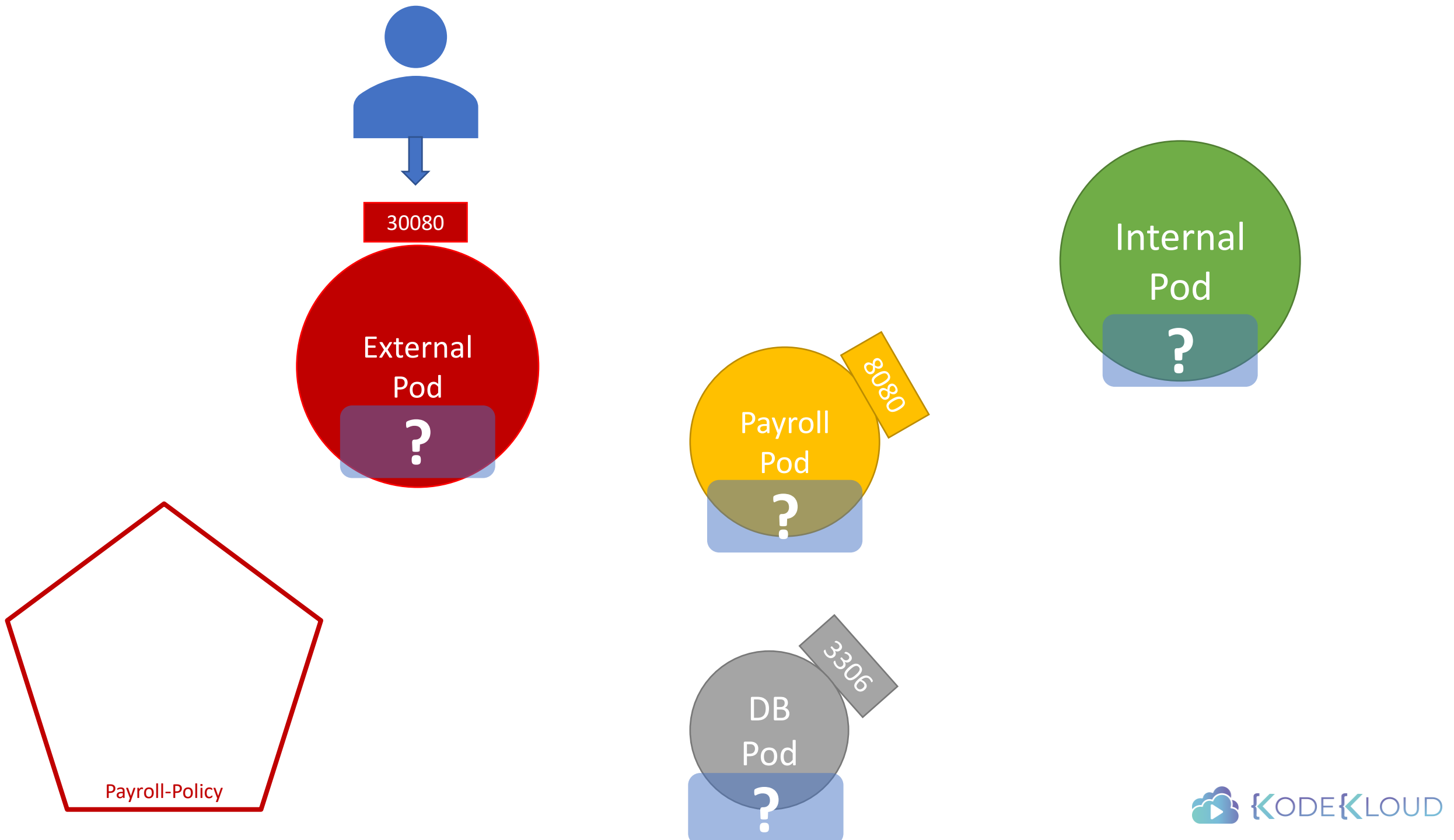
Solutions that DO NOT Support Network Poli

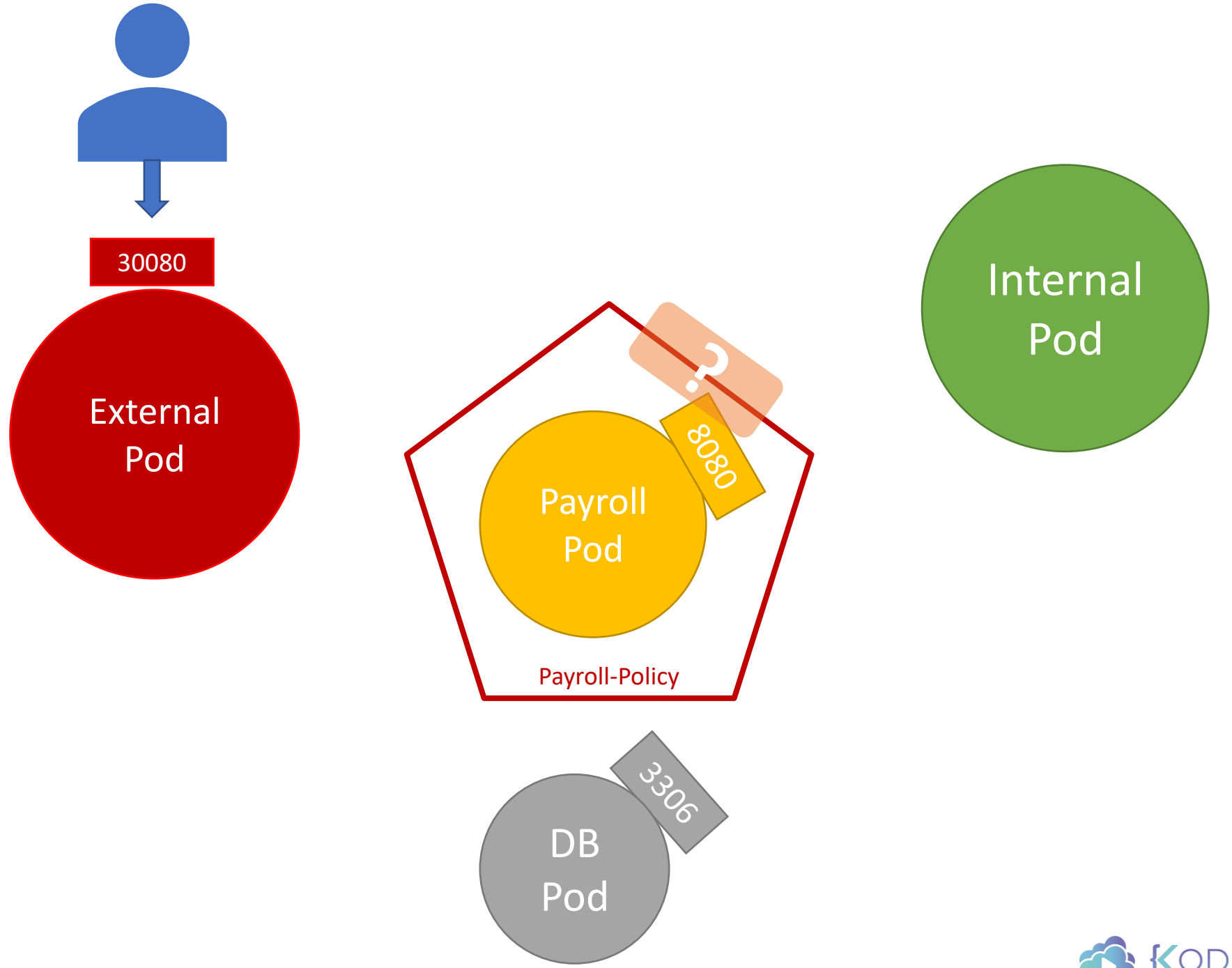
- Flannel

KODEKLOUD

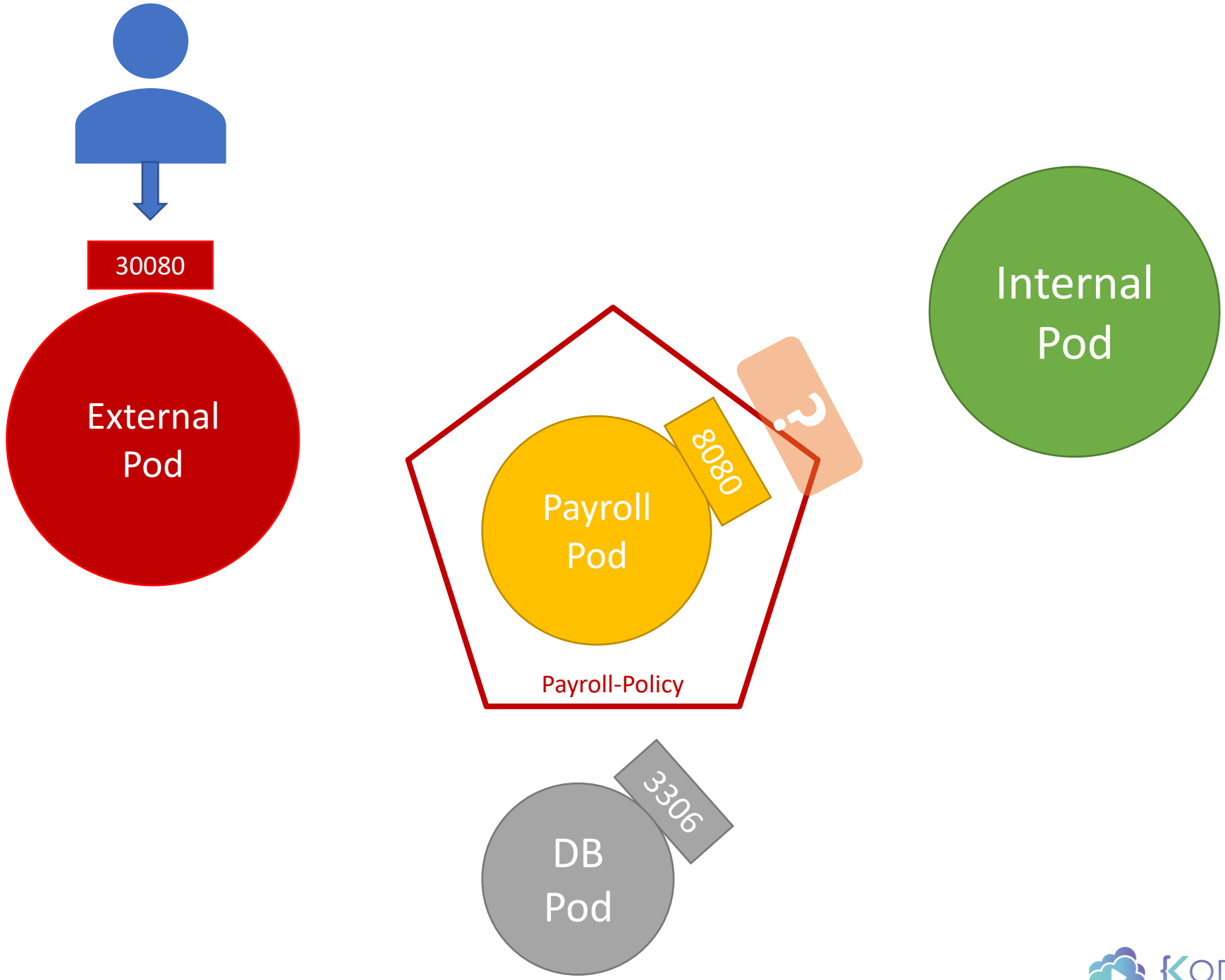

30080
External Pod
Internal Pod
Payroll Pod
8080
#?
DB Pod
3306
KODEKLOUD


30080
External Pod
Internal Pod
Payroll Pod
8080
?
DB Pod
3306
KODEKLOUD

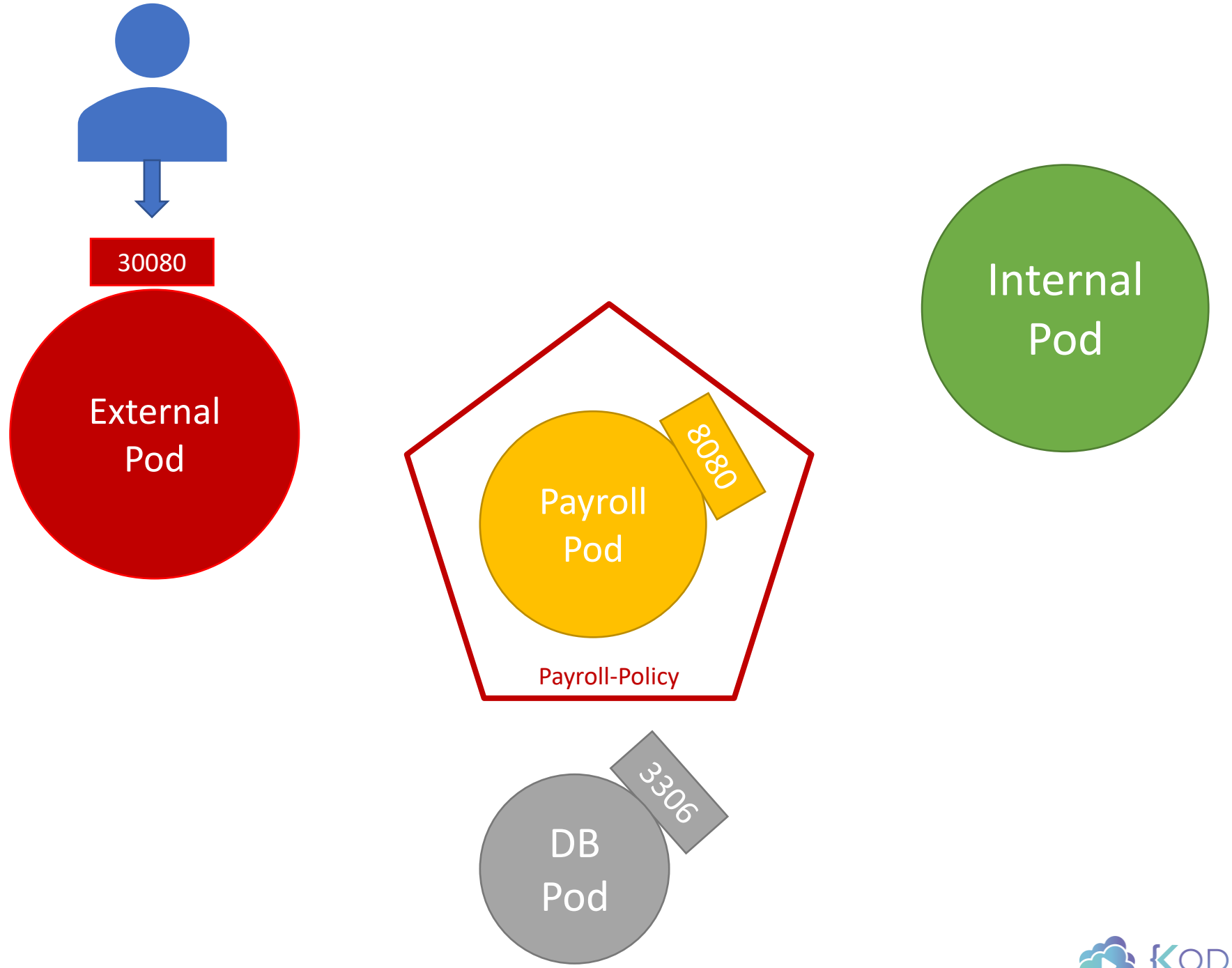

30080
External Pod
?
Internal Pod
?
8080
Payroll Pod
?
3306
DB Pod
?
Payroll-Policy
KODEKLOUD

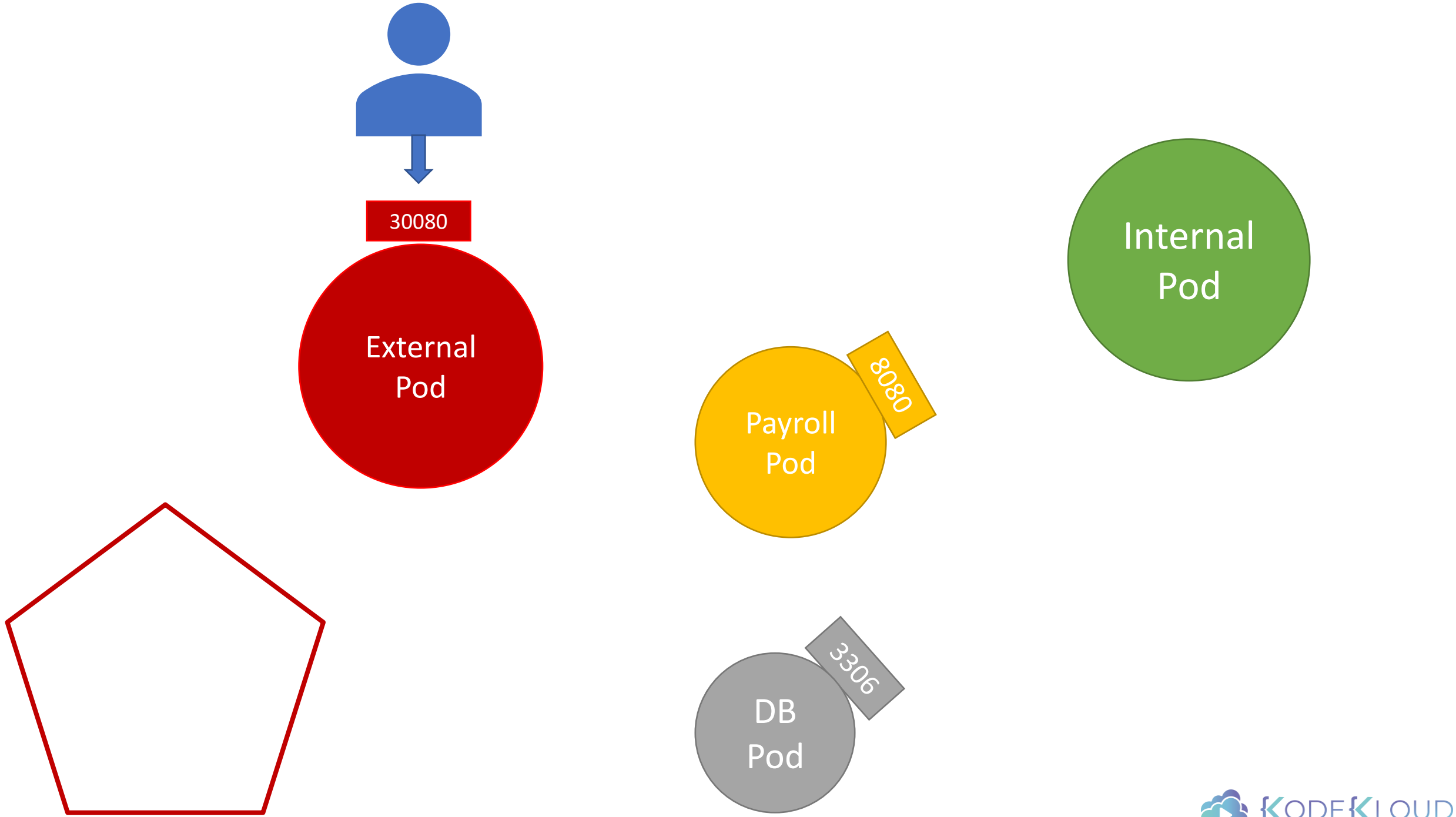

30080
External Pod
?
8080
Payroll Pod
Payroll-Policy
Internal Pod
3306
DB Pod

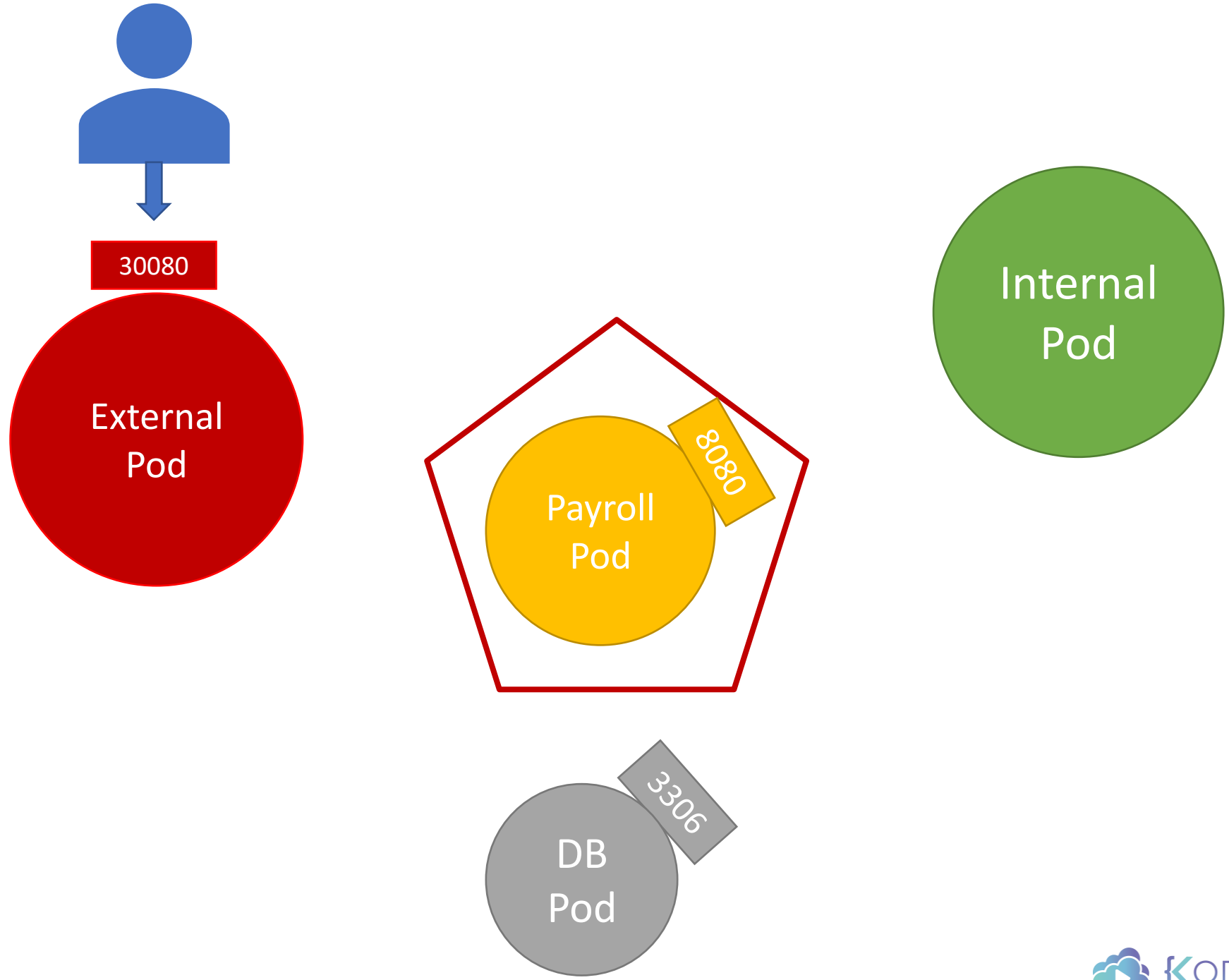
KODEKLOUD


30080
External Pod
Payroll Pod
8080
?
Payroll-Policy
Internal Pod
DB Pod
3306

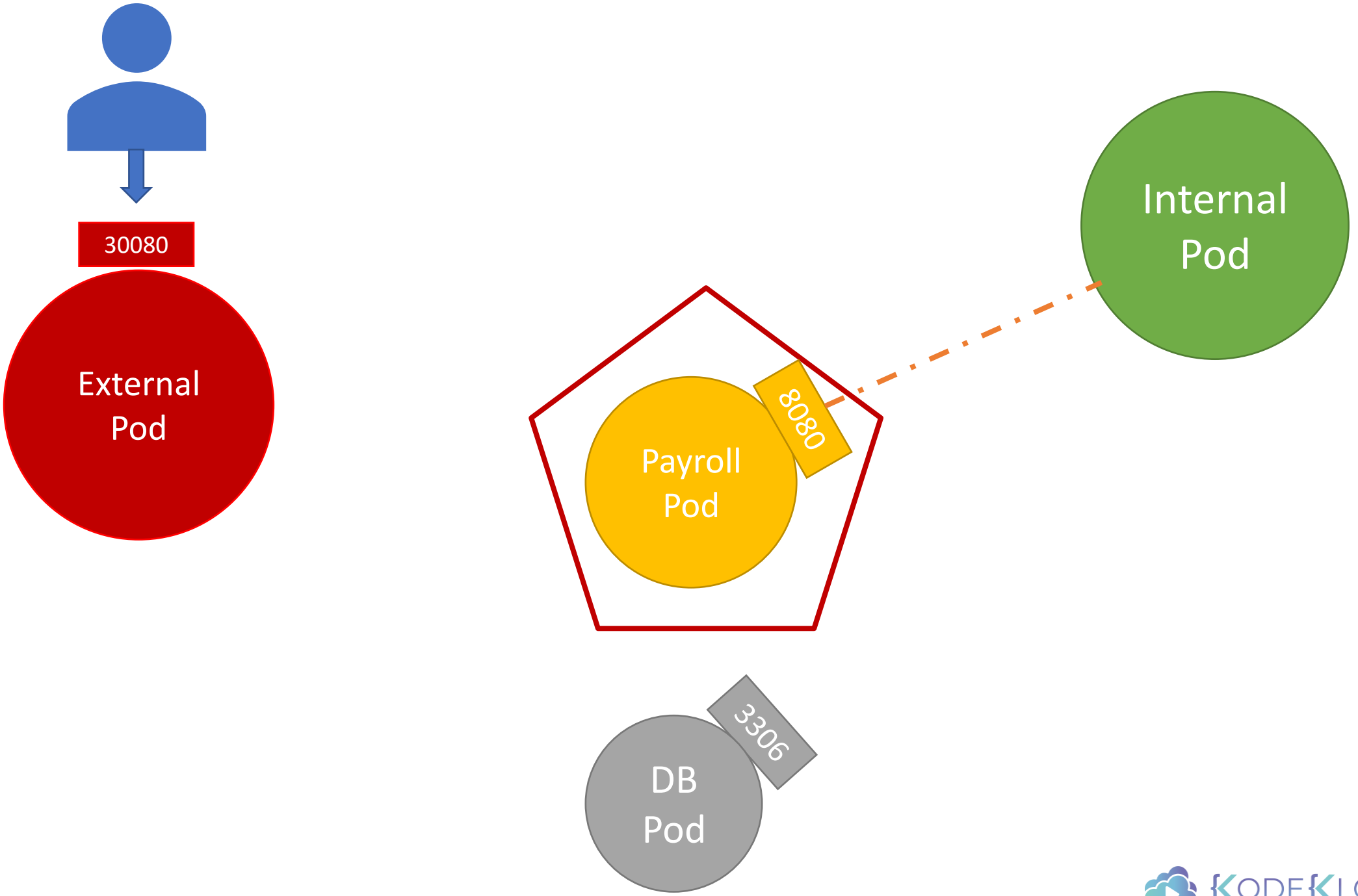
KODEKLOUD

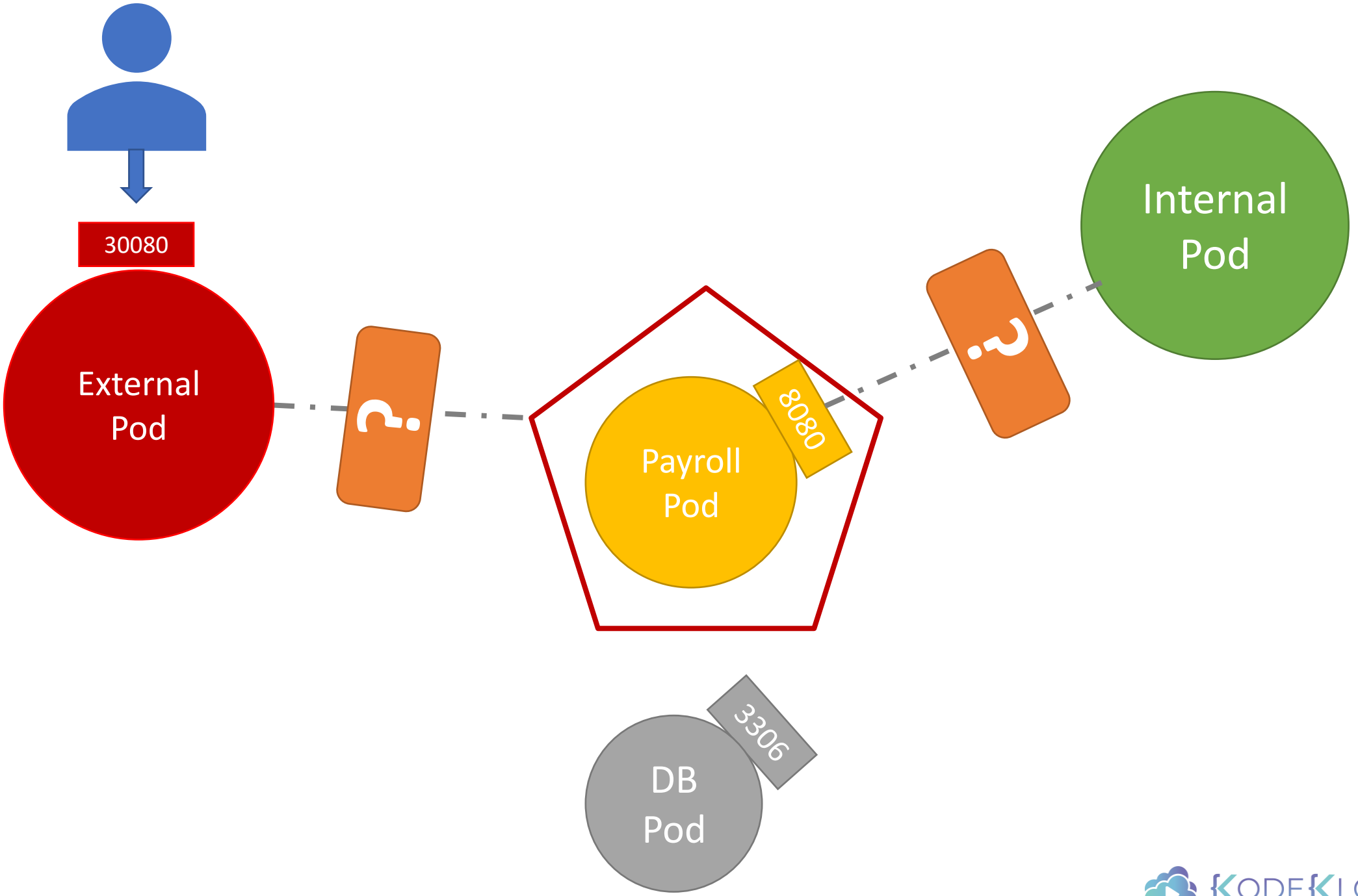

30080
External Pod
Internal Pod
Payroll Pod
8080
Payroll-Policy
DB Pod
3306

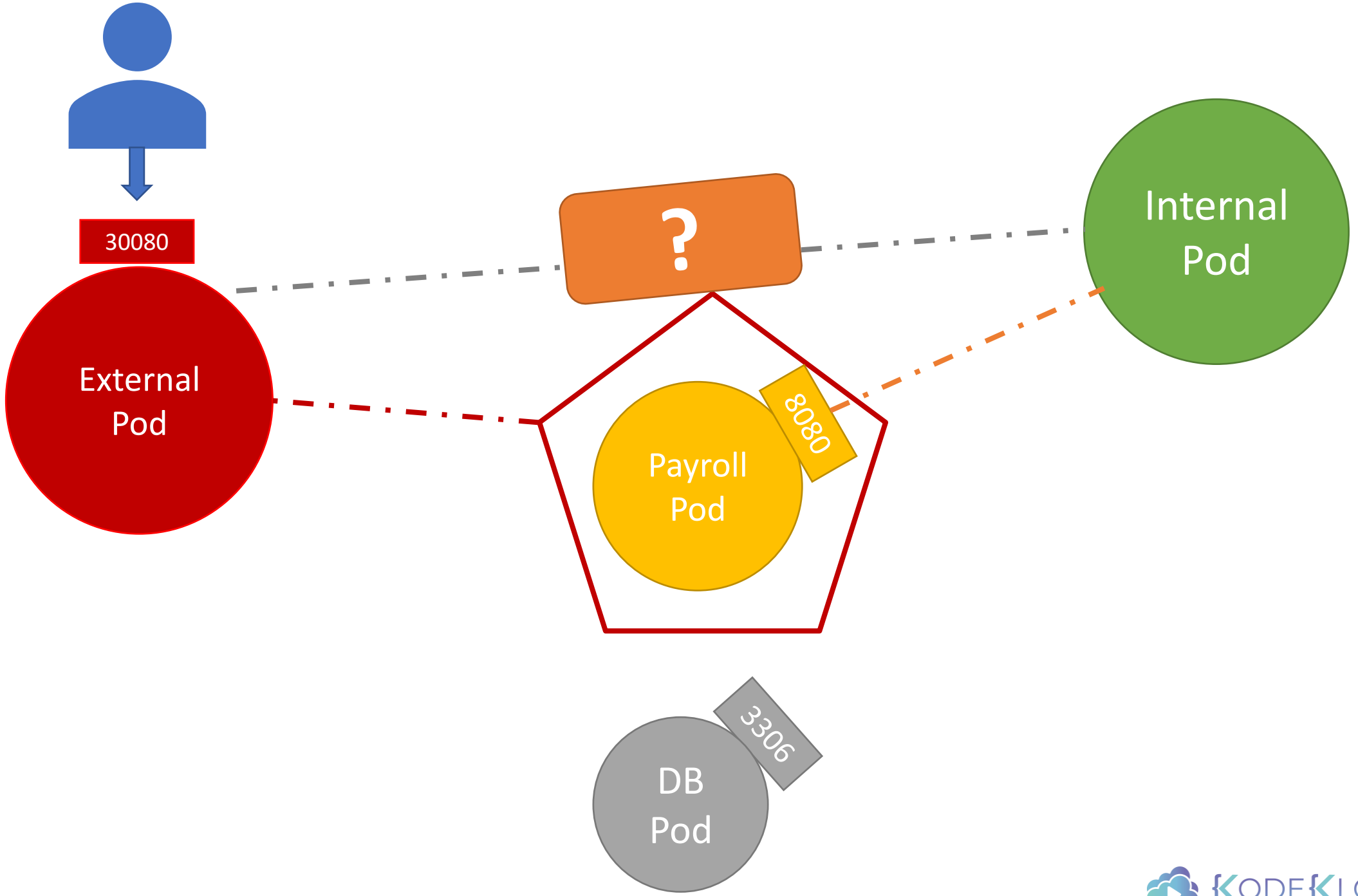

30080
External Pod
Internal Pod
Payroll Pod
8080
DB Pod
3306
KODEKLOUD


30080
External Pod
Payroll Pod
8080
Internal Pod
DB Pod
3306

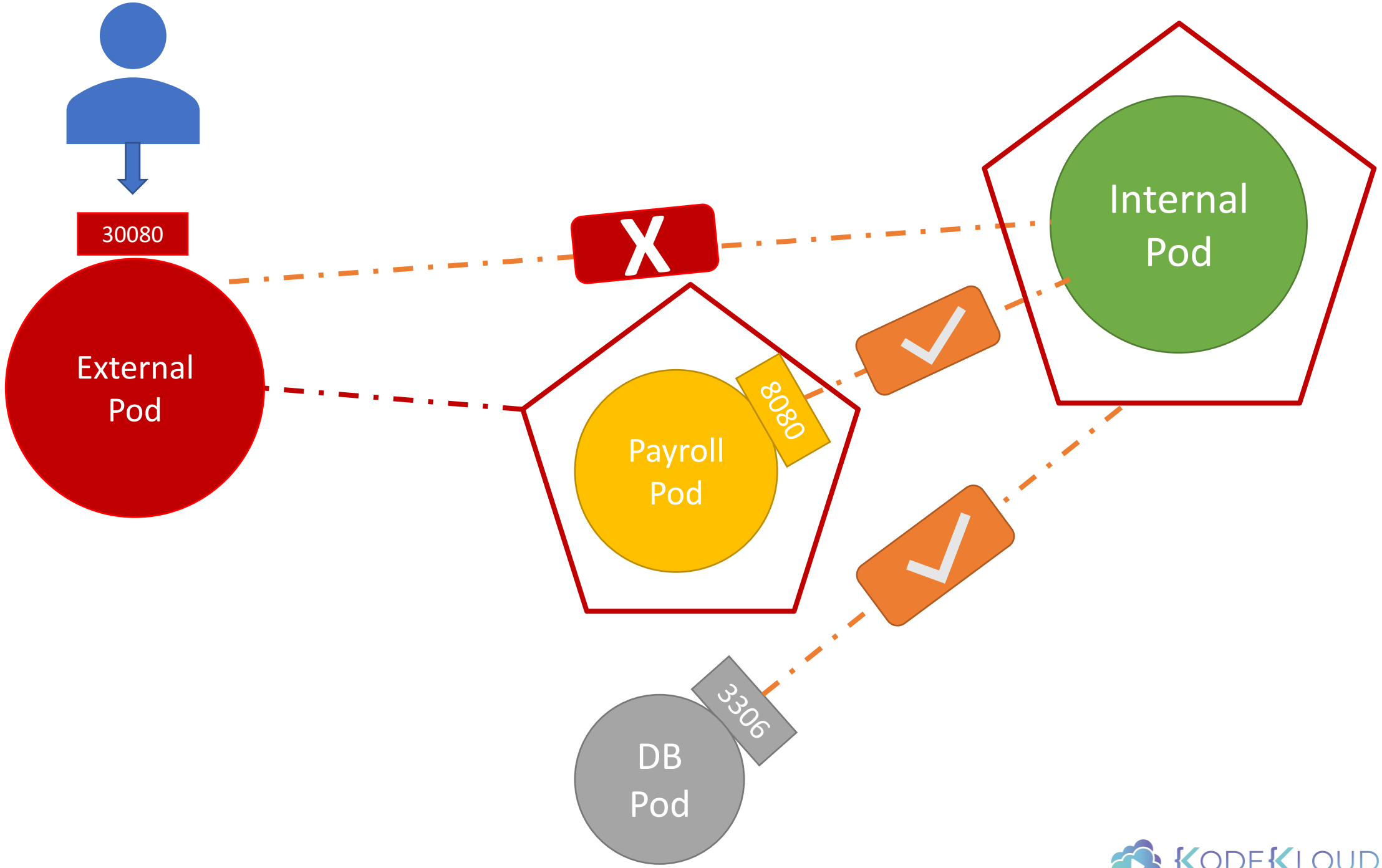
KODEKLOUD

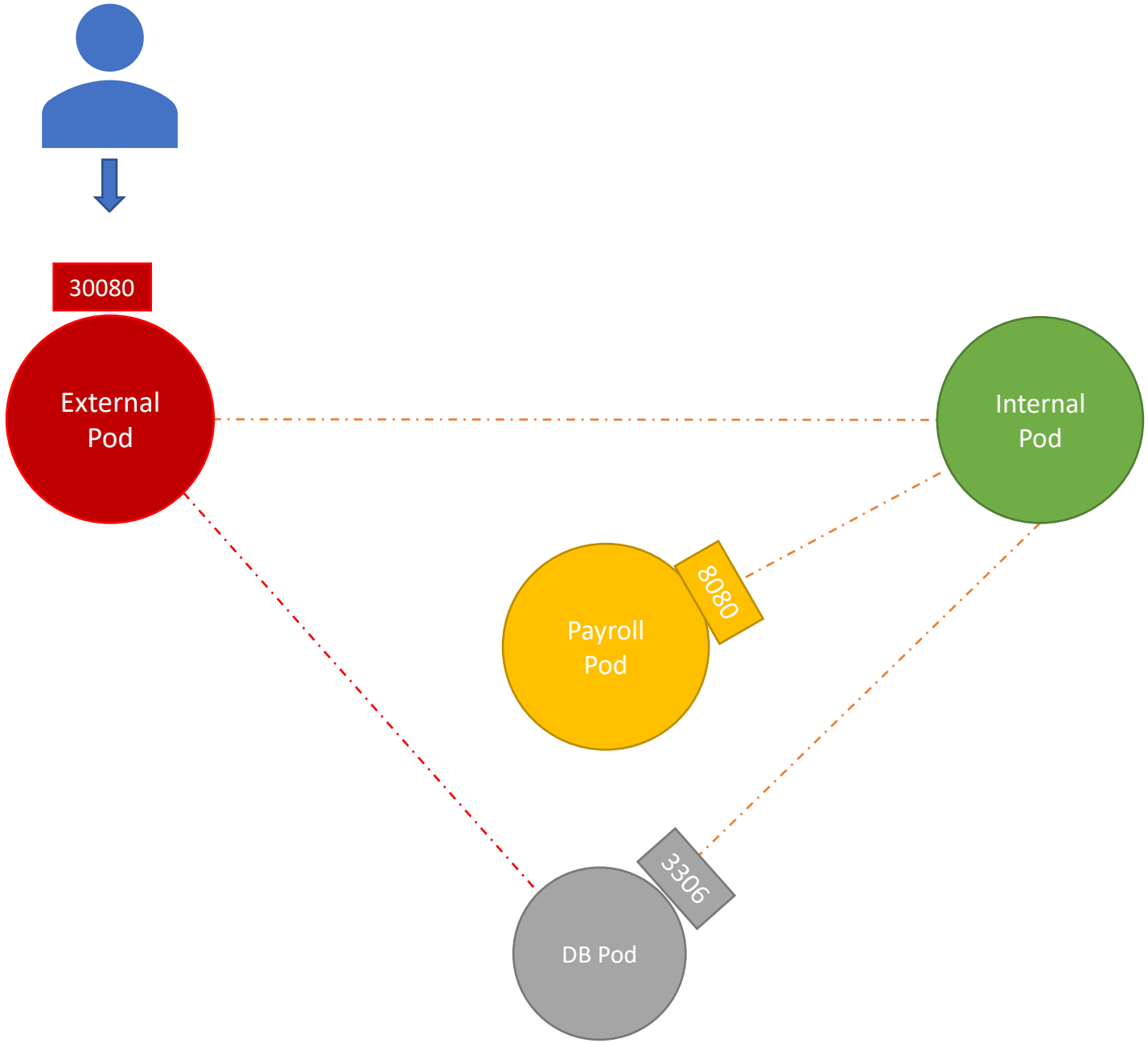

30080
External Pod
Internal Pod
Payroll Pod
8080
DB Pod
3306


30080
External Pod
?
Payroll Pod
8080
?
Internal Pod
3306
DB Pod


30080
External Pod
?
Internal Pod
Payroll Pod
8080
DB Pod
3306

KODEKLOUD


30080
External Pod
X
Payroll Pod
8080
Internal Pod
3306
DB Pod


30080
External Pod
Internal Pod
Payroll Pod
8080
3306
DB Pod

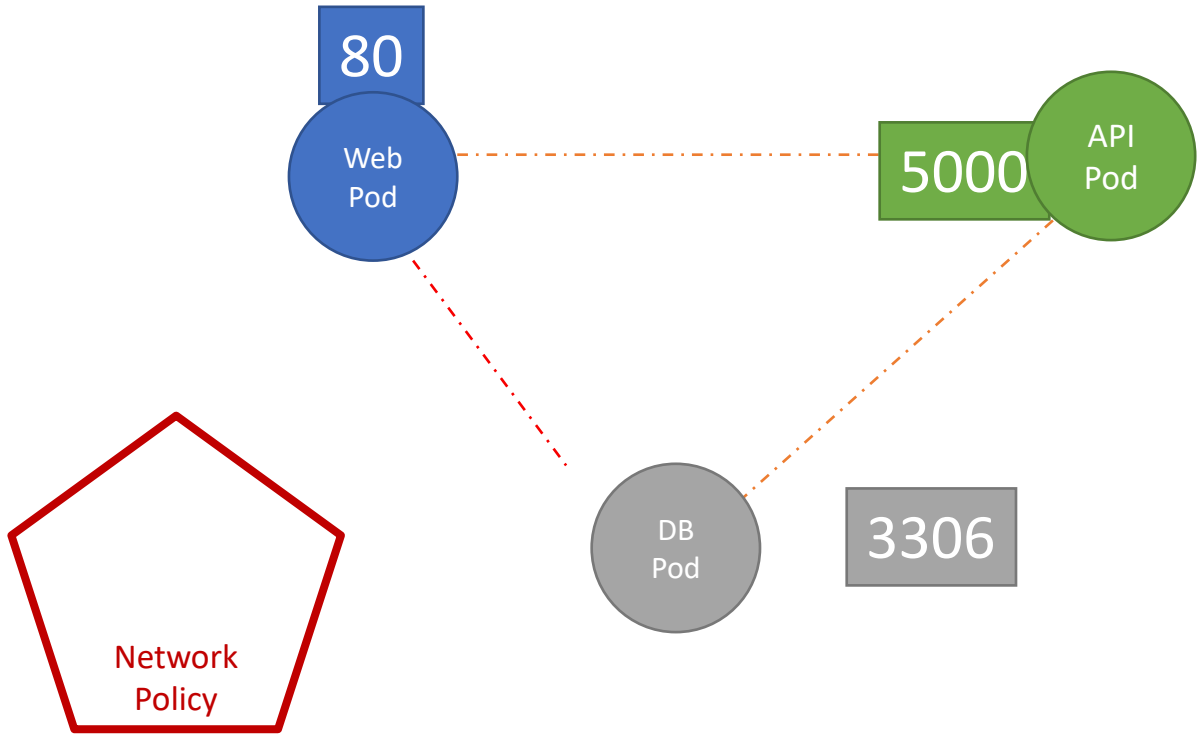
KODEKLOUD

KODEKLOUD

Network Policies

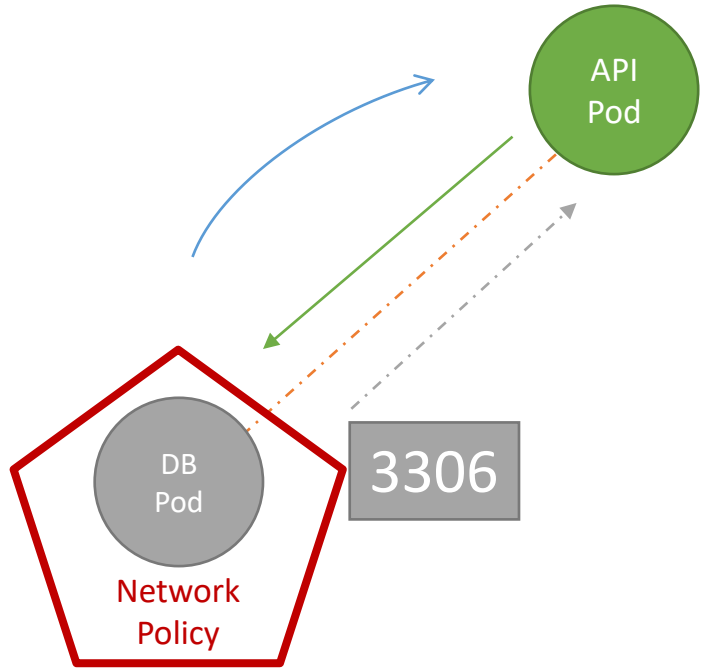
KODEKLOUD

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-policy
spec:
  podSelector:
    matchLabels:
      role: db
```



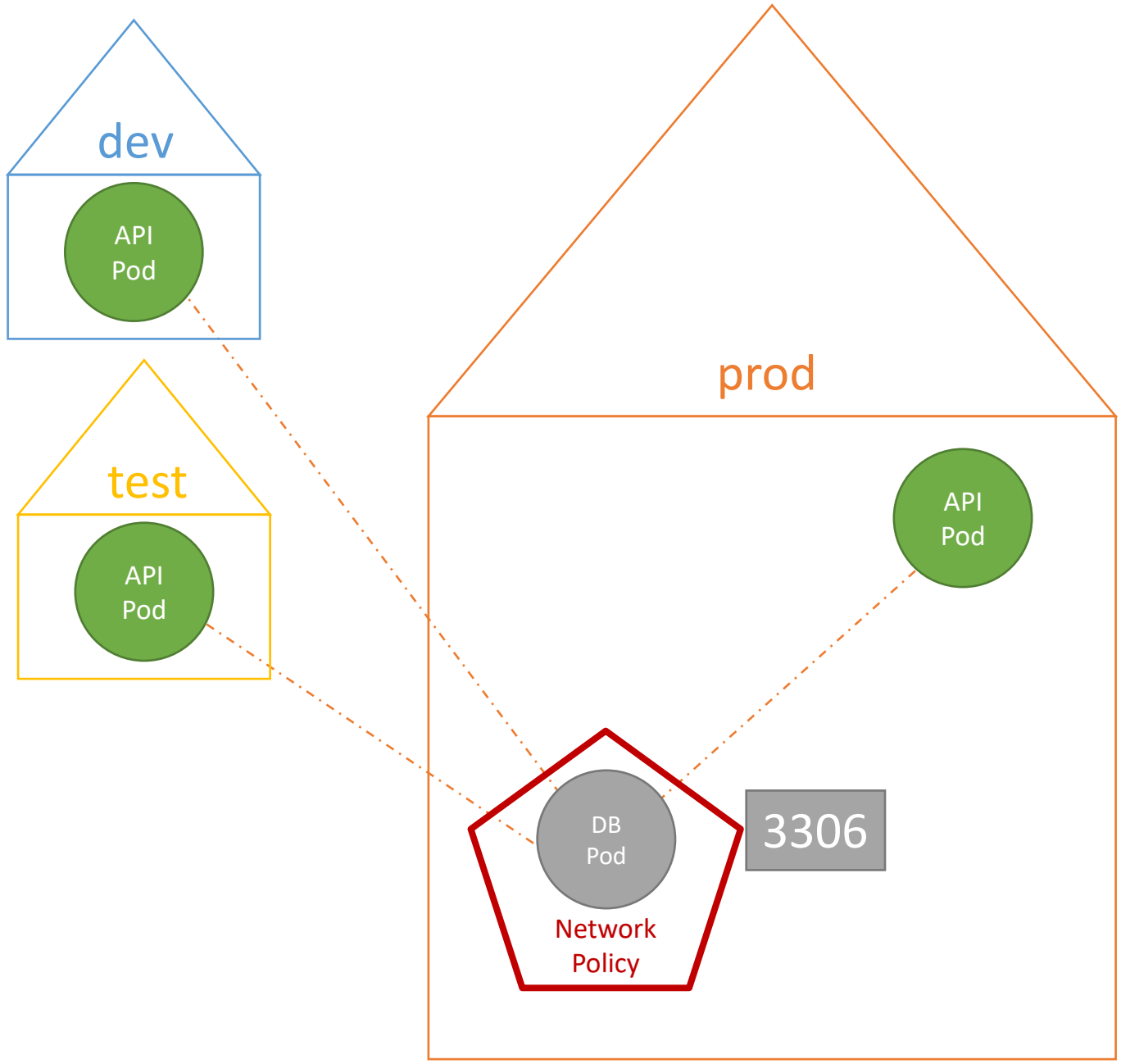
KODEKLOUD

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-policy
spec:
  podSelector:
    matchLabels:
      role: db

  policyTypes:
  - Ingress
```



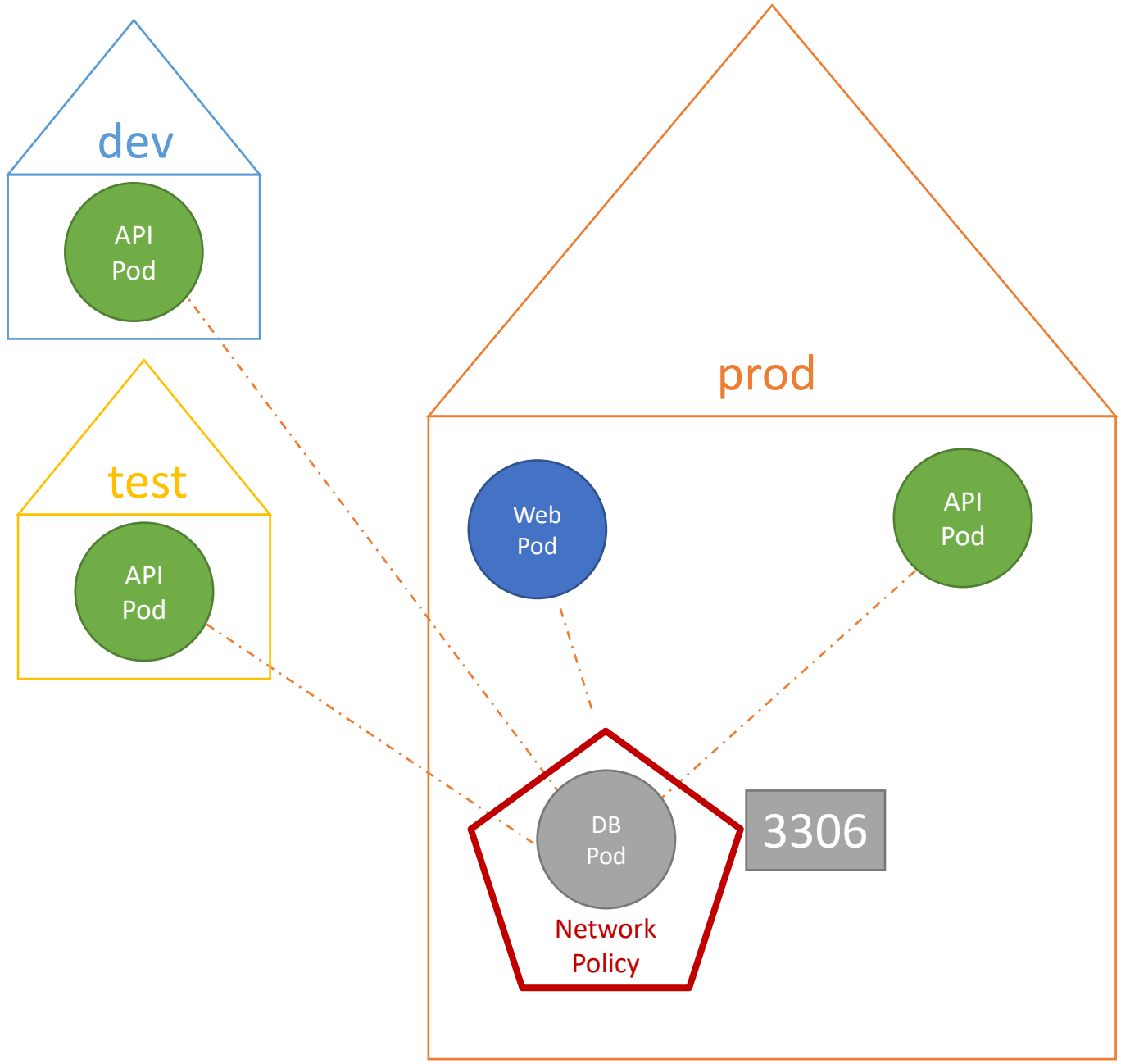
KODEKLOUD

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-policy
spec:
  podSelector:
    matchLabels:
      role: db

  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          name: api-pod
    ports:
    - protocol: TCP
      port: 3306
```



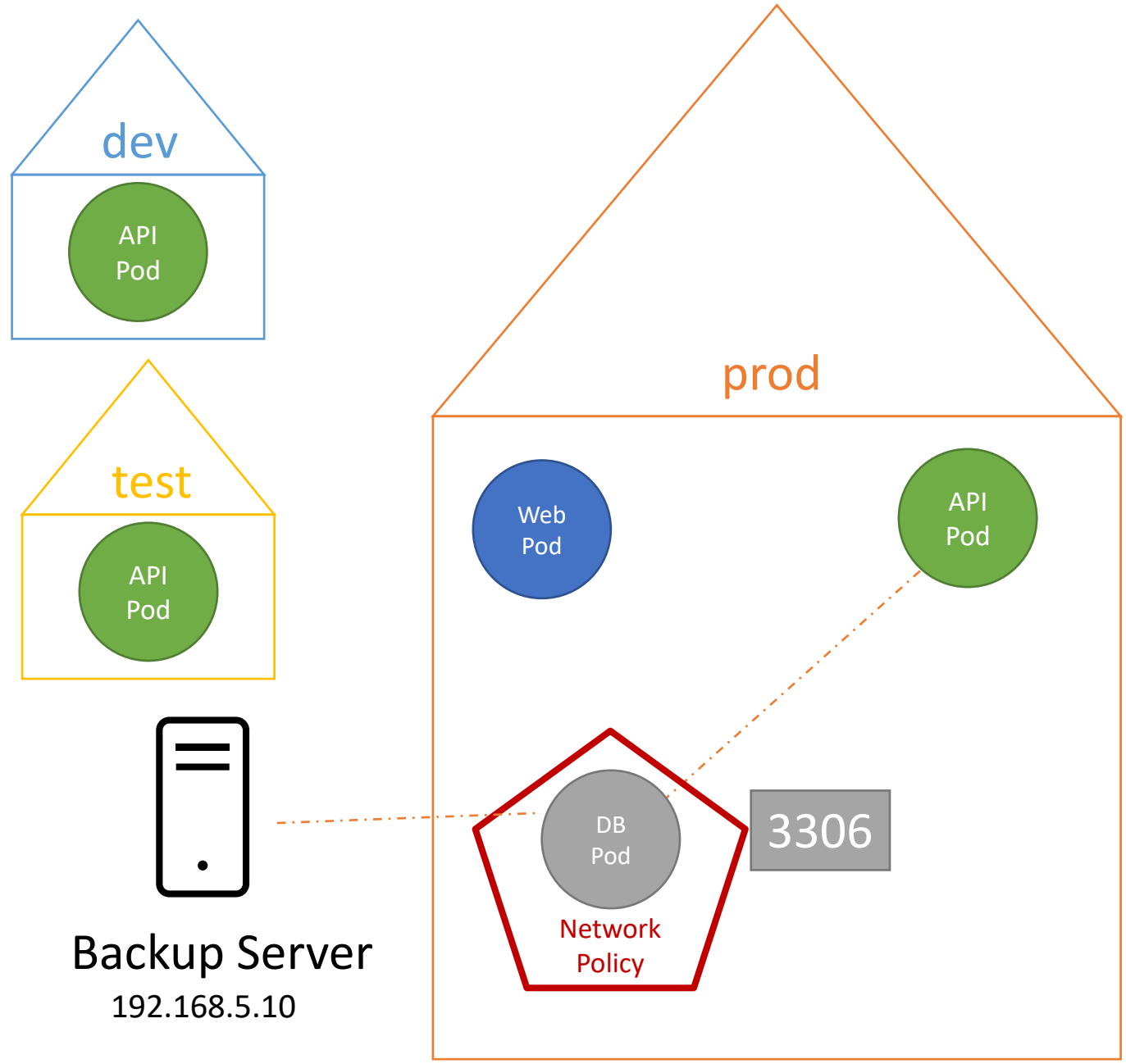
KODEKLOUD

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-policy
spec:
  podSelector:
    matchLabels:
      role: db
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          name: api-pod
      namespaceSelector:
        matchLabels:
          name: prod
    ports:
    - protocol: TCP
      port: 3306
```



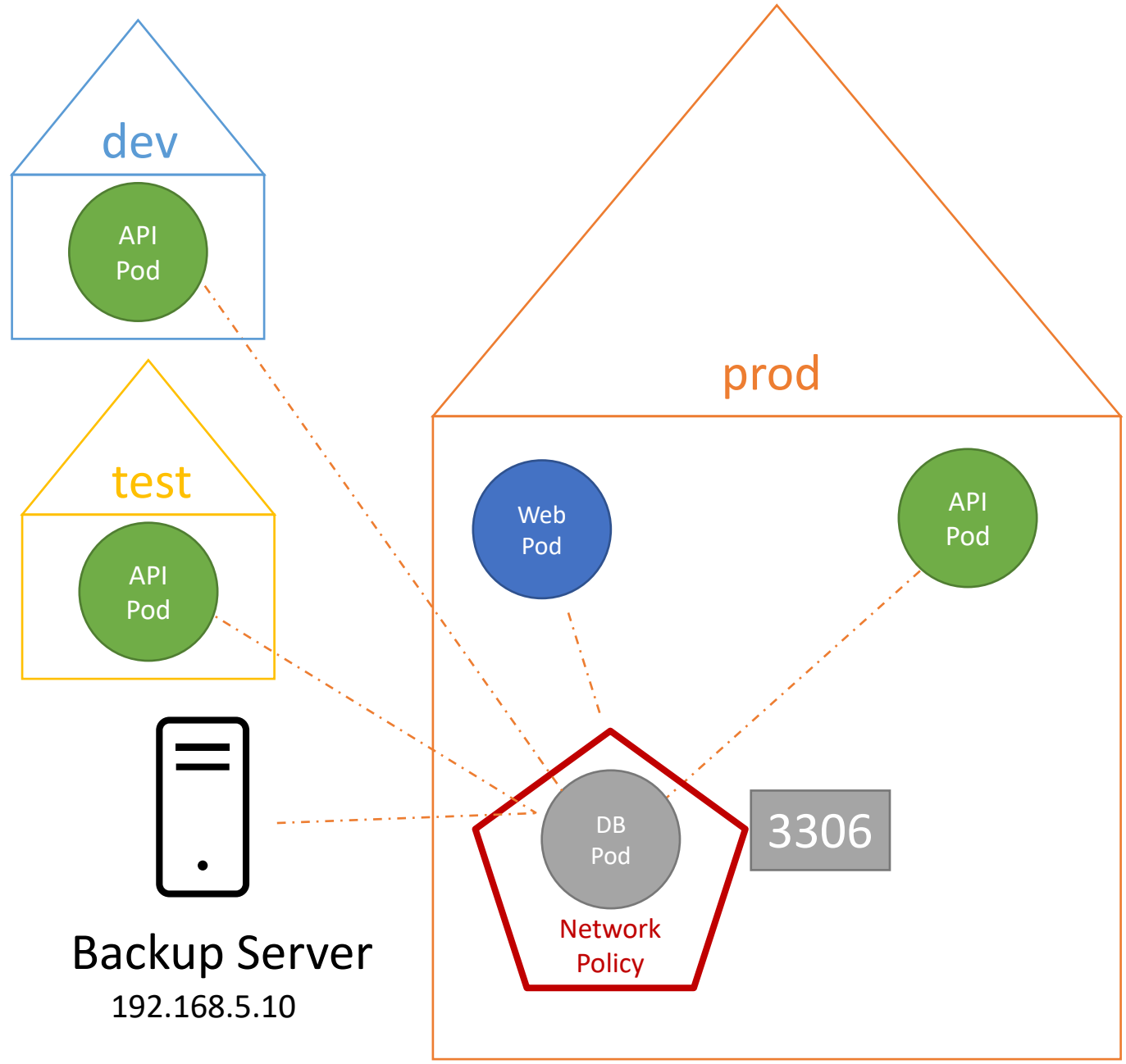
KODEKLOUD

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-policy
spec:
  podSelector:
    matchLabels:
      role: db
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          name: api-pod
      namespaceSelector:
        matchLabels:
          name: prod
    ports:
    - protocol: TCP
      port: 3306
```



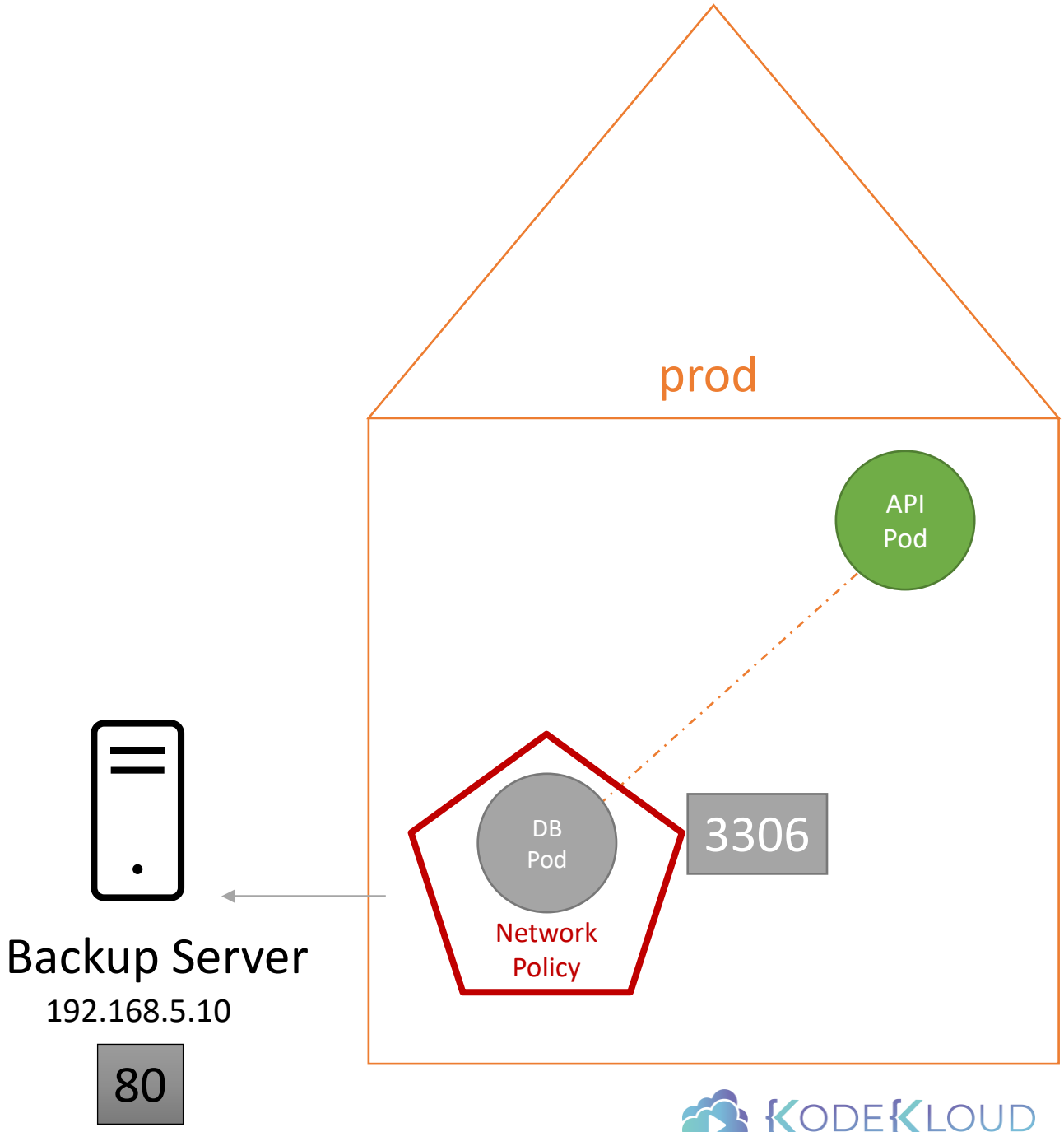
KODEKLOUD

```yaml
spec:
  podSelector:
    matchLabels:
      role: db
  policyTypes:
  - Ingress

  ingress:
  - from:
    - podSelector:
        matchLabels:
          name: api-pod
    - namespaceSelector:
        matchLabels:
          name: prod
    - ipBlock:
        cidr: 192.168.5.10/32


    ports:
    - protocol: TCP
      port: 3306
```



KODEKLOUD

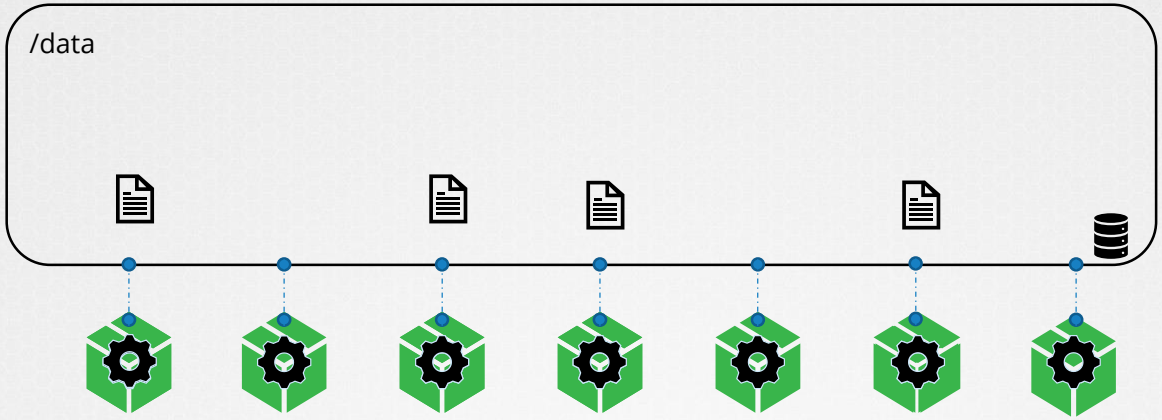
```yaml
spec:
  podSelector:
    matchLabels:
      role: db
  policyTypes:
  - Ingress
  - Egress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          name: api-pod
    ports:
    - protocol: TCP
      port: 3306
  egress:
  - to:
    - ipBlock:
        cidr: 192.168.5.10/32
    ports:
    - protocol: TCP
      port: 80
```



KODEKLOUD

Volumes

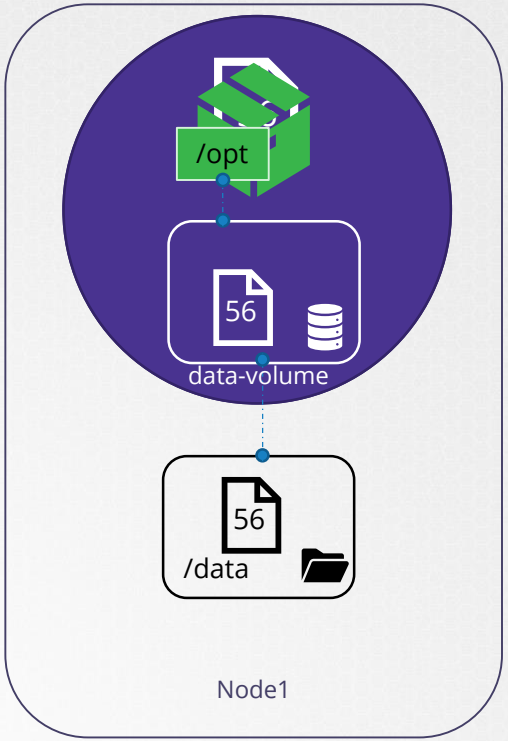
Volume



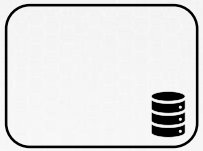
Volumes

Volumes & Mounts

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: random-number-generator
spec:
  containers:
  - image: alpine
    name: alpine
    command: ["/bin/sh","-c"]
    args: ["shuf -i 0-100 -n 1 >> /opt/number.out;"]
    volumeMounts:
    - mountPath: /opt
      name: data-volume

  volumes:
  - name: data-volume
    hostPath:
      path: /data
      type: Directory
```



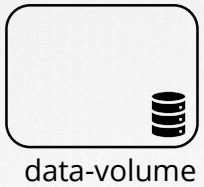
```yaml
volumes:
- name: data-volume
  hostPath:
    path: /data
    type: Directory
```

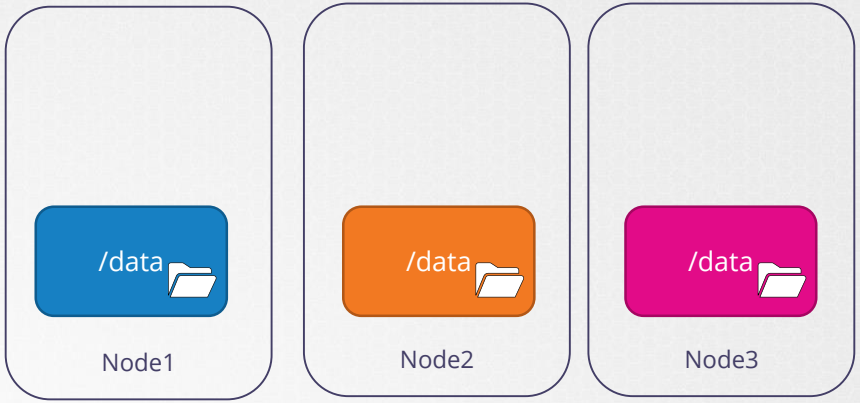


data-volume

Node1

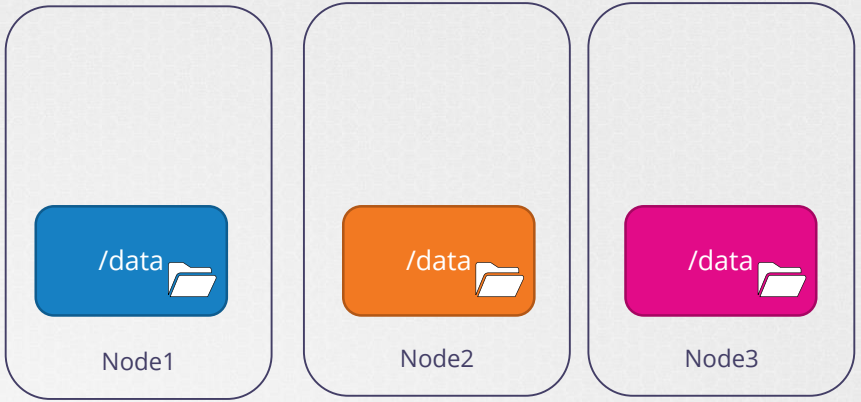
```
volumes:
- name: data-volume
  hostPath:
    path: /data
    type: Directory
```



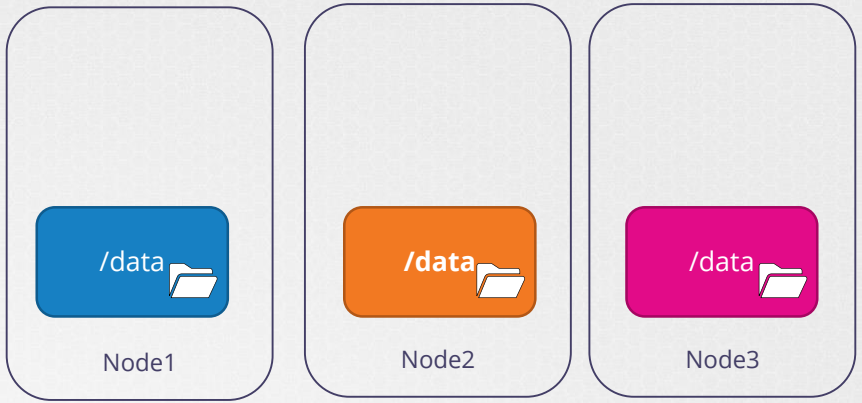


Volume Types

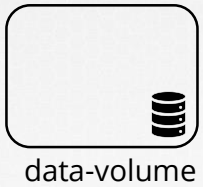
```
volumes:
- name: data-volume
  hostPath:
    path: /data
    type: Directory
```



Volume Types

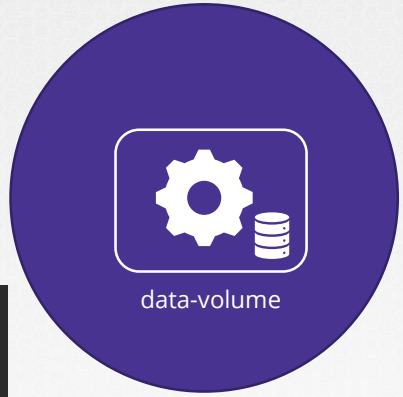



```
volumes:
- name: data-volume
  awsElasticBlockStore:
    volumeID: <volume-id>
    fsType: ext4
```



Persistent Volumes

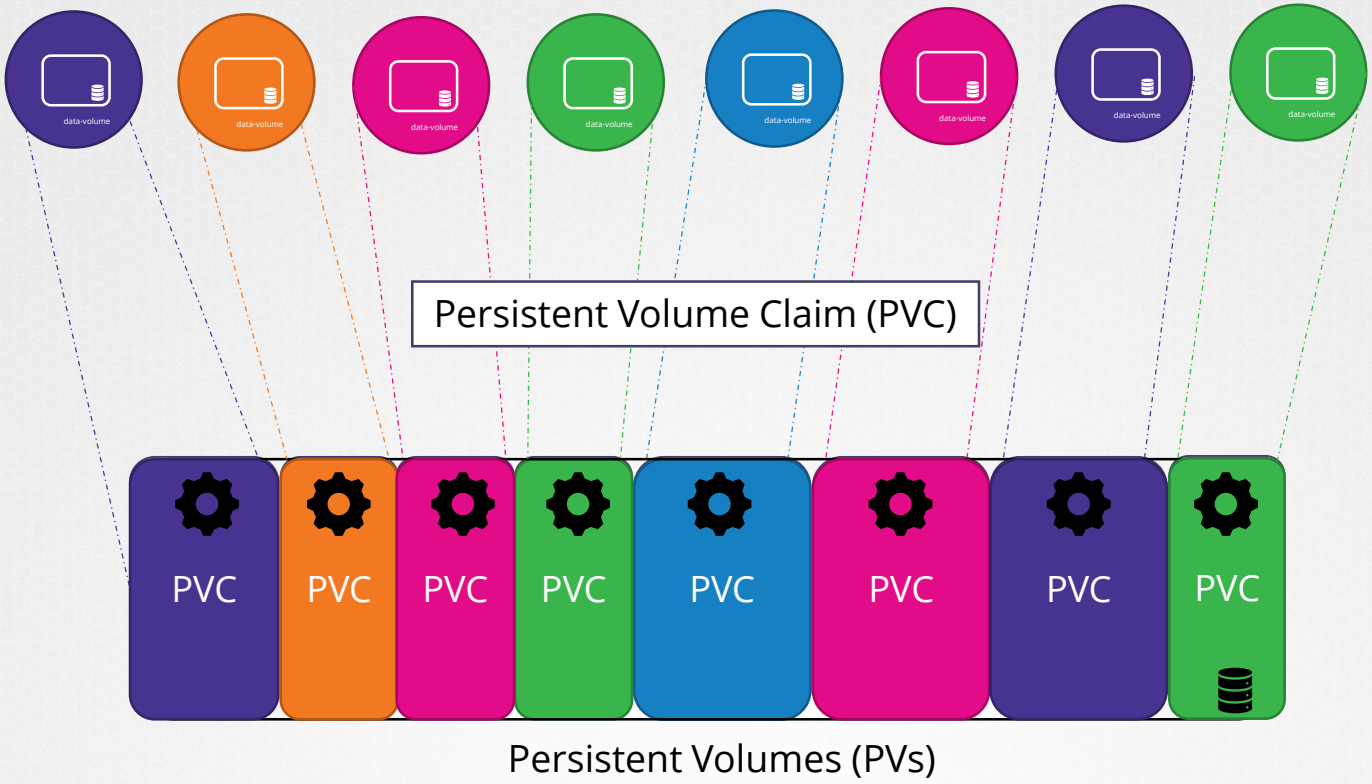
```
volumes:
- name: data-volume
  awsElasticBlockStore:
    volumeID: <volume-id>
    fsType: ext4
```



data-volume
data-volume
data-volume
data-volume
data-volume
data-volume
data-volume
data-volume

data-volume
data-volume
data-volume
data-volume
data-volume
data-volume
data-volume
data-volume

Persistent Volume

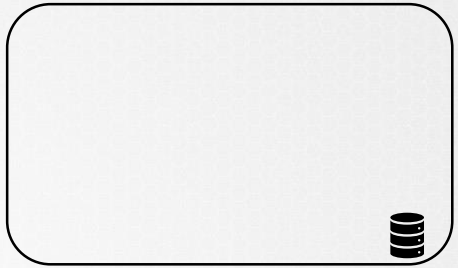


Persistent Volume

pv-definition.yaml

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
   name: pv-vol1
spec:
  accessModes:
      - ReadWriteOnce
  capacity:
      storage: 1Gi
  awsElasticBlockStore:
    volumeID: <volume-id>
    fsType: ext4
```

- ReadOnlyMany
- ReadWriteOnce
- ReadWriteMany



Persistent Volume (PV)

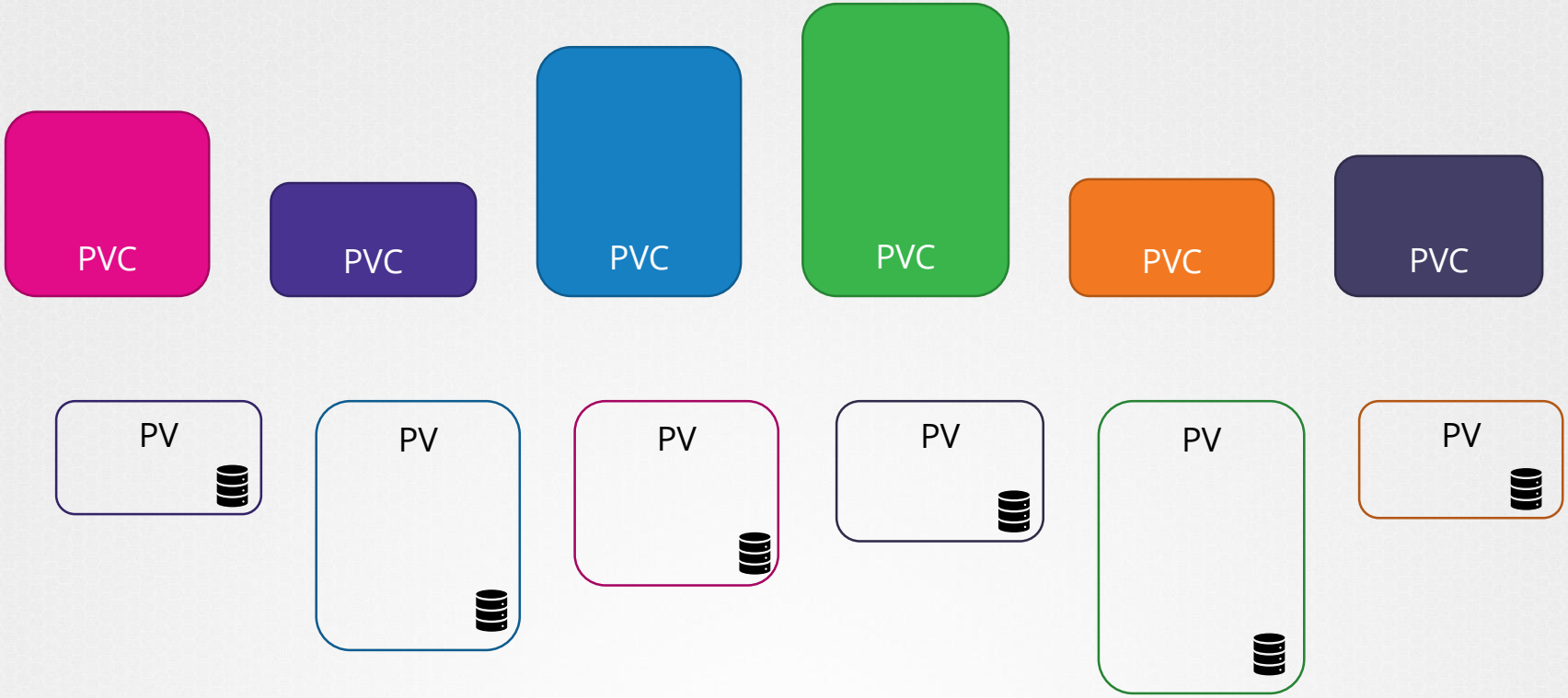
```
kubectl create -f pv-definition.yaml
```

```
kubectl get persistentvolume
```

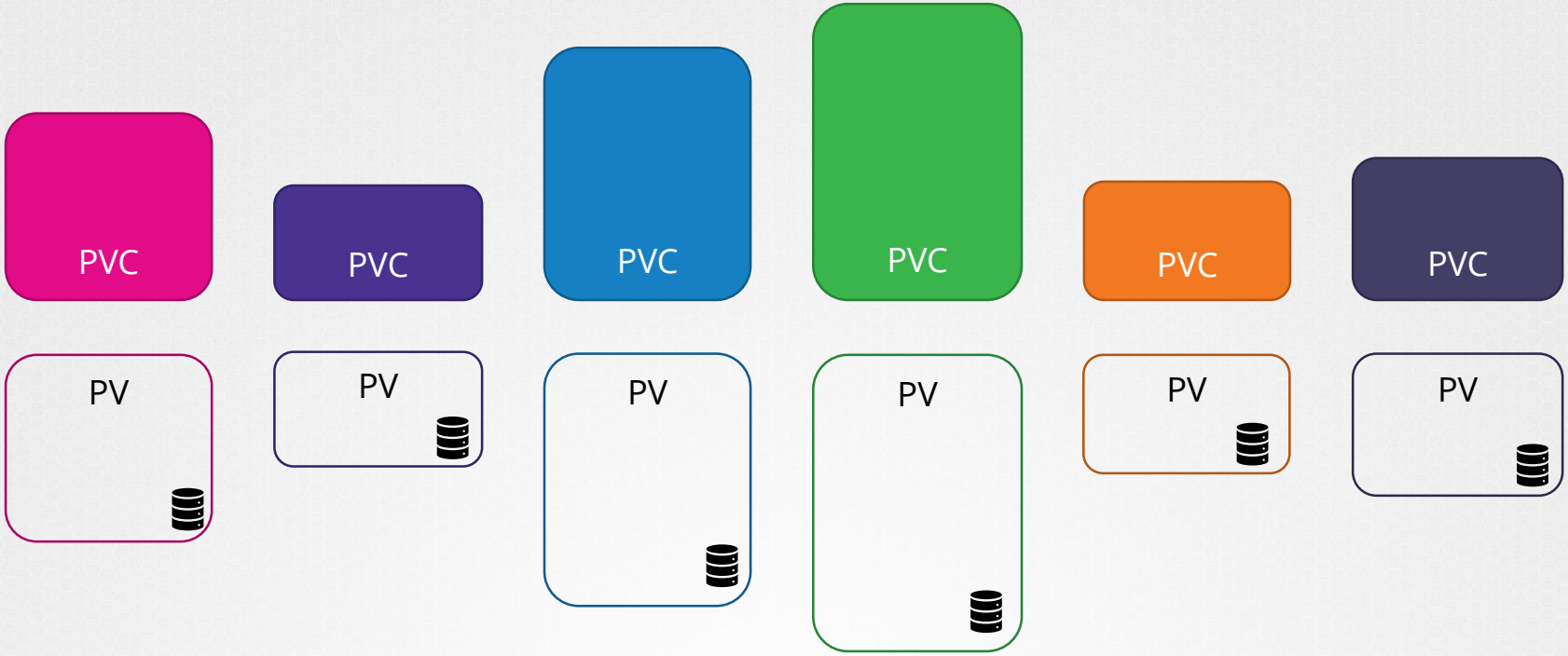
NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM	STORAGECLASS	REASON	AGE
pv-vol1	1Gi	RWO	Retain	Available				3m

Persistent Volume Claims

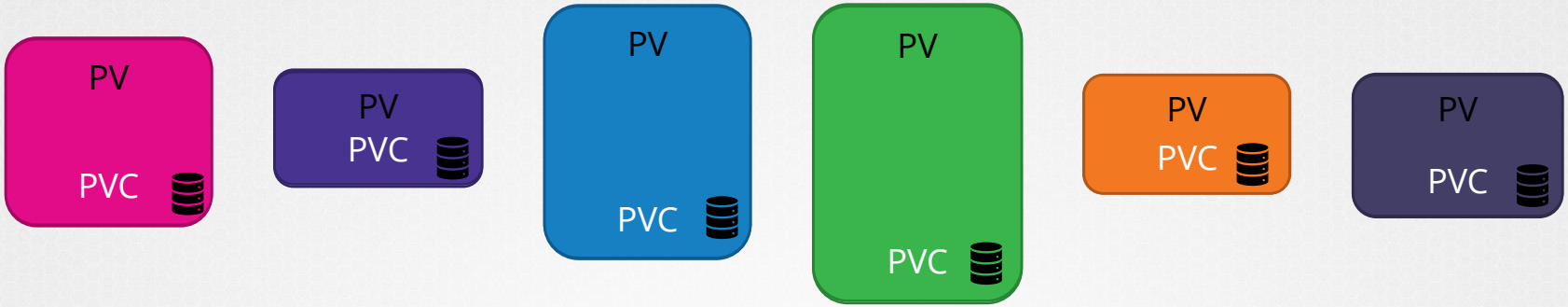
Persistent Volume Claim



Binding



Binding



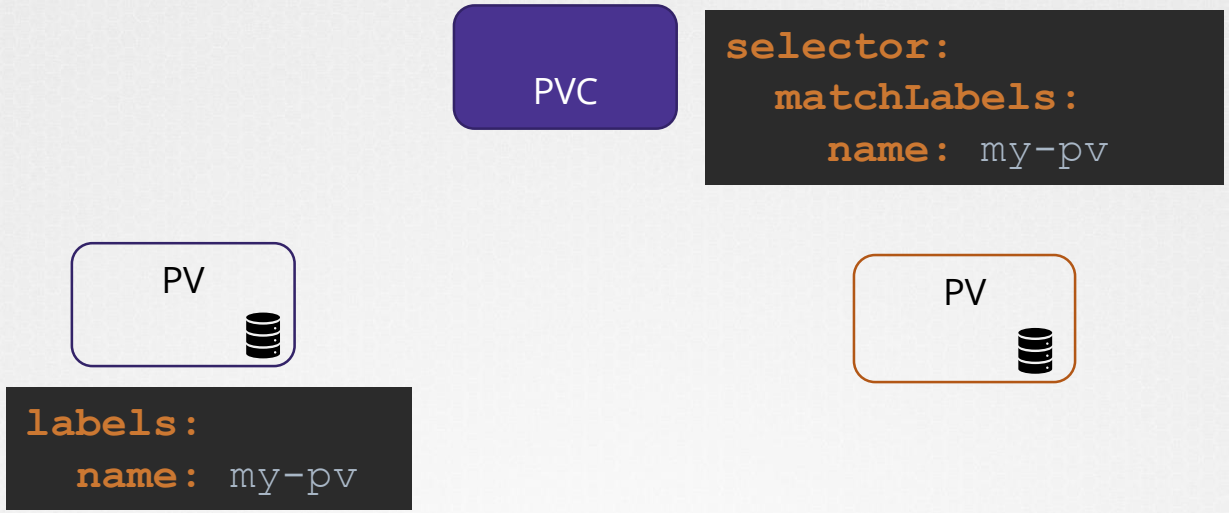
Sufficient Capacity

Access Modes

Volume Modes

Storage Class

Binding



Sufficient Capacity

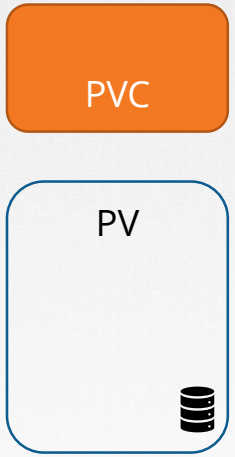
Access Modes

Volume Modes

Storage Class

Selector

Binding



Pending

Sufficient Capacity

Access Modes

Volume Modes

Storage Class

Selector

Persistent Volume Claim

pvc-definition.yaml

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: myclaim
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 500Mi
```

```
kubectl create -f pvc-definition.yaml
```

```
kubectl get persistentvolumeclaim
```

```
NAME        STATUS    VOLUME    CAPACITY   ACCESS MODES
myclaim     Pending
```

Persistent Volume Claim

pvc-definition.yaml

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: myclaim
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 500Mi
```

```
kubectl create -f pvc-definition.yaml
```

pv-definition.yaml

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-vol1
spec:
  accessModes:
      - ReadWriteOnce
  capacity:
      storage: 1Gi
  awsElasticBlockStore:
    volumeID: <volume-id>
    fsType: ext4
```

View PVCs

```
kubectl get persistentvolumeclaim
```

```
NAME        STATUS    VOLUME    CAPACITY   ACCESS MODES   STORAGECLASS   AGE
myclaim     Bound     pv-vol1   1Gi        RWO                           43m
```

Delete PVCs

```
kubectl delete persistentvolumeclaim myclaim
persistentvolumeclaim "myclaim" deleted
```

```
persistentVolumeReclaimPolicy:   Recycle
```

KODEKLOUD

Course Objectives

- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Troubleshooting
- Core Concepts
- Networking
  - Pre-Requisites – Network, Switching, Routing, Tools
  - Pre-Requisites – Network Namespaces
  - Pre-Requisites – DNS, IPAM, Firewalls, LBs
  - Pre-Requisites – Networking in Docker
  - Networking Configuration on Cluster Nodes
  - Service Networking
  - POD Networking Concepts
  - Network Loadbalancer
  - Ingress
  - Cluster DNS
  - CNI

KODEKLOUD

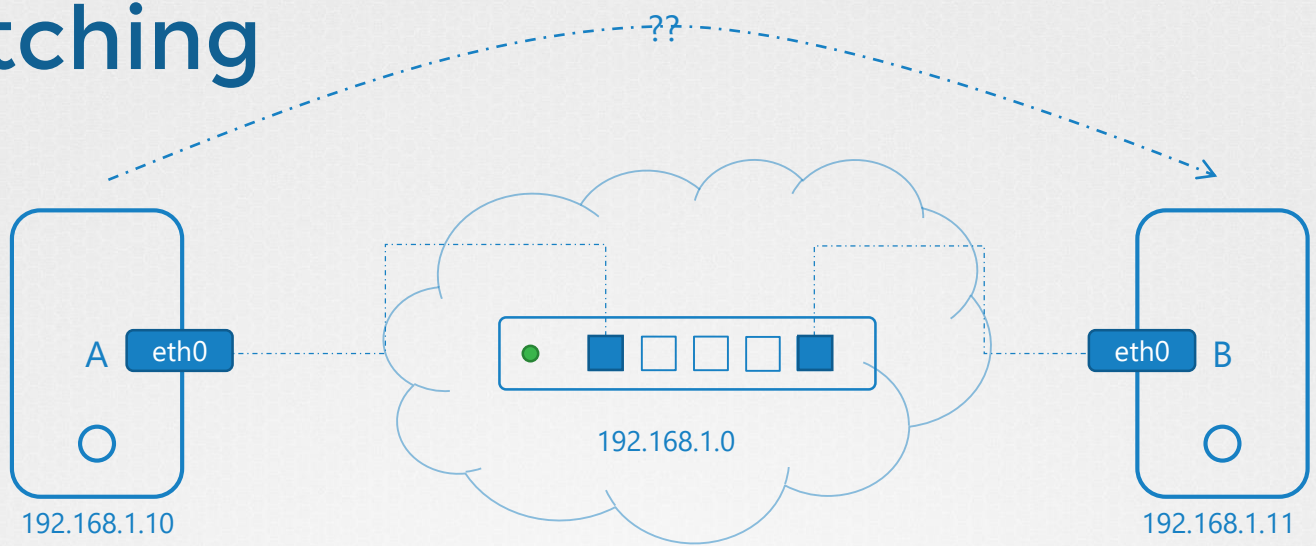
LINUX NETWORKING BASICS

Networking Pre-Requisites

- Switching and Routing
  - Switching
  - Routing
  - Default Gateway
  - NAT
- Linux Interfaces for Virtual Networking
  - Bridge Network
  - VLAN
  - VXLAN
- IP Address Management & Name Resolution
  - DNS
  - IPAM
  - DHCP
- Firewalls
- Load-Balancers
- Tools:
  - Ping
  - NC - NetCat
  - TCPDUMP
  - IPTABLES

KODEKLOUD

Switching




```
ip link
```

```
eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc
fq_codel state UP mode DEFAULT group default qlen 1000
```

```
ip addr add 192.168.1.10/24 dev eth0
```

```
ip link
```

```
eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdis
fq_codel state UP mode DEFAULT group default qlen 100
```

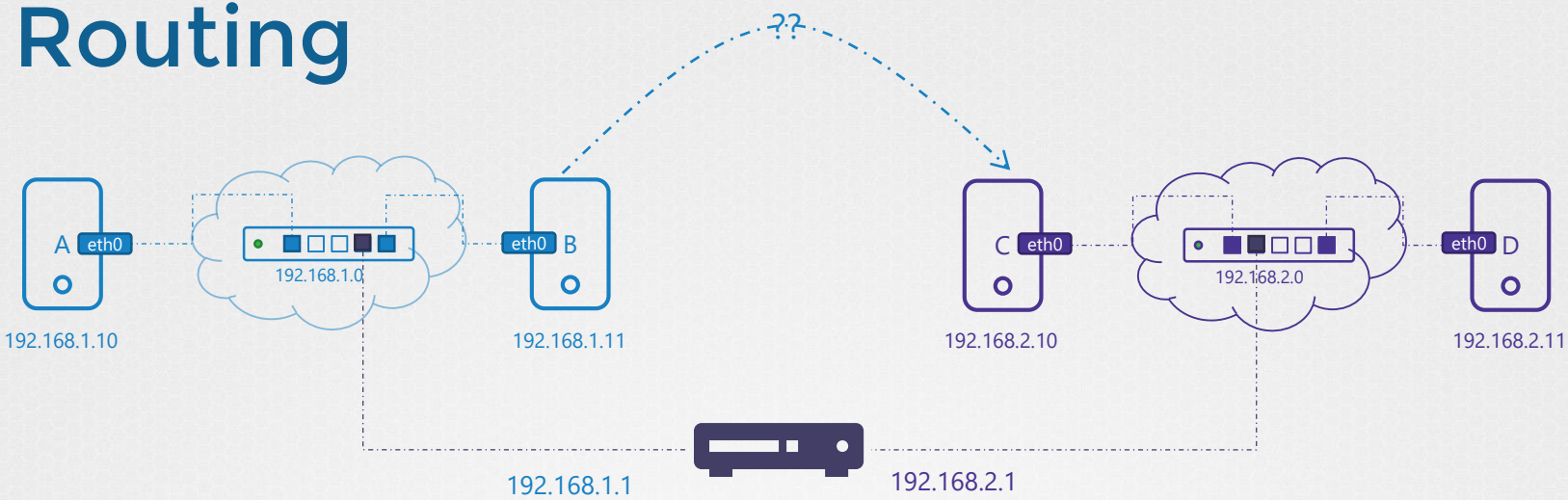
```
ip addr add 192.168.1.11/24 dev eth0
```

```
ping 192.168.1.11
```

```
Reply from 192.168.1.11: bytes=32 time=4ms TTL=117
Reply from 192.168.1.11: bytes=32 time=4ms TTL=117
```

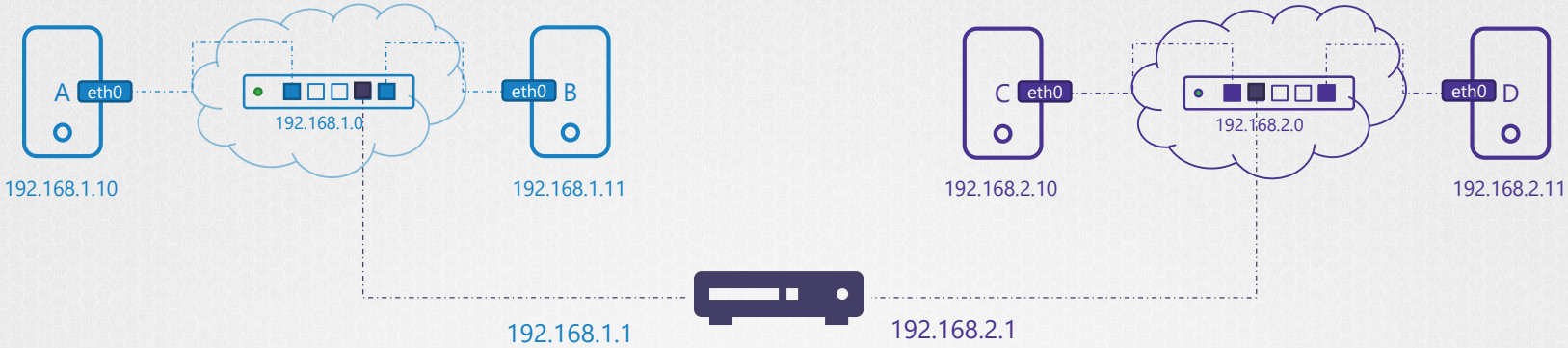
KODEKLOUD

Routing



KODEKLOUD

Gateway




```
route
```

```
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
```

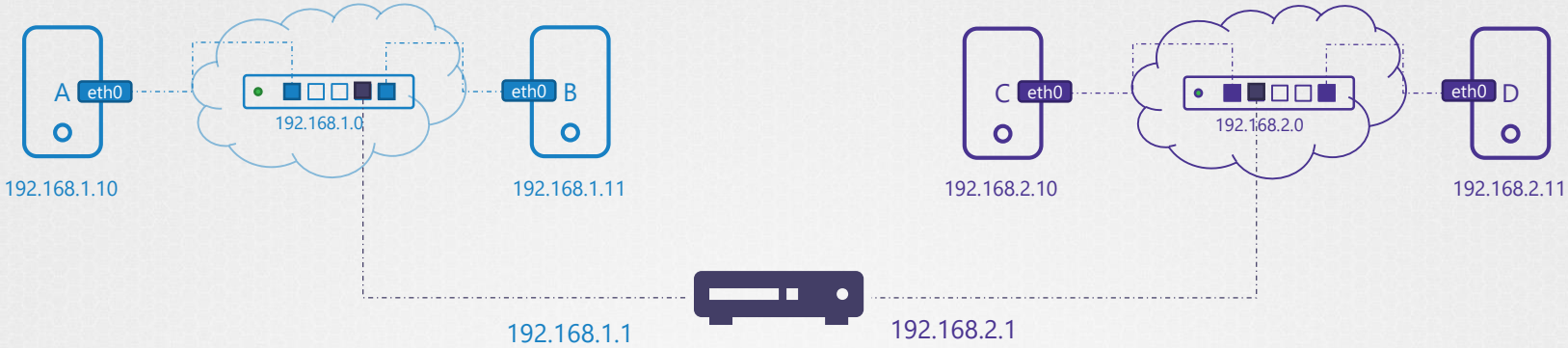
```
ip route add 192.168.2.0/24 via 192.168.1.1
```

```
route
```

```
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
192.168.2.0     192.168.1.1     255.255.255.0   UG    0      0        0 eth0
```

KODEKLOUD

Gateway



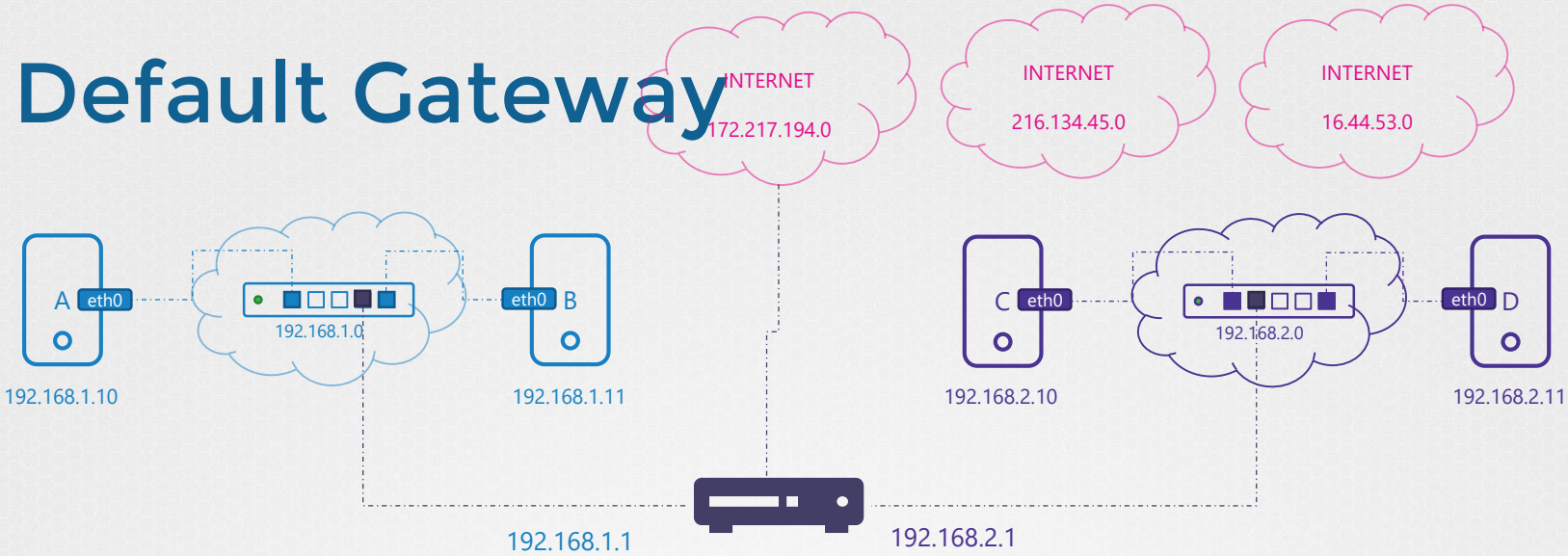

```
ip route add 192.168.1.0/24 via 192.168.2.1
```

```
route
```

```
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
192.168.1.0     192.168.2.1     255.255.255.0   UG    0      0        0 eth0
```

KODEKLOUD

Default Gateway



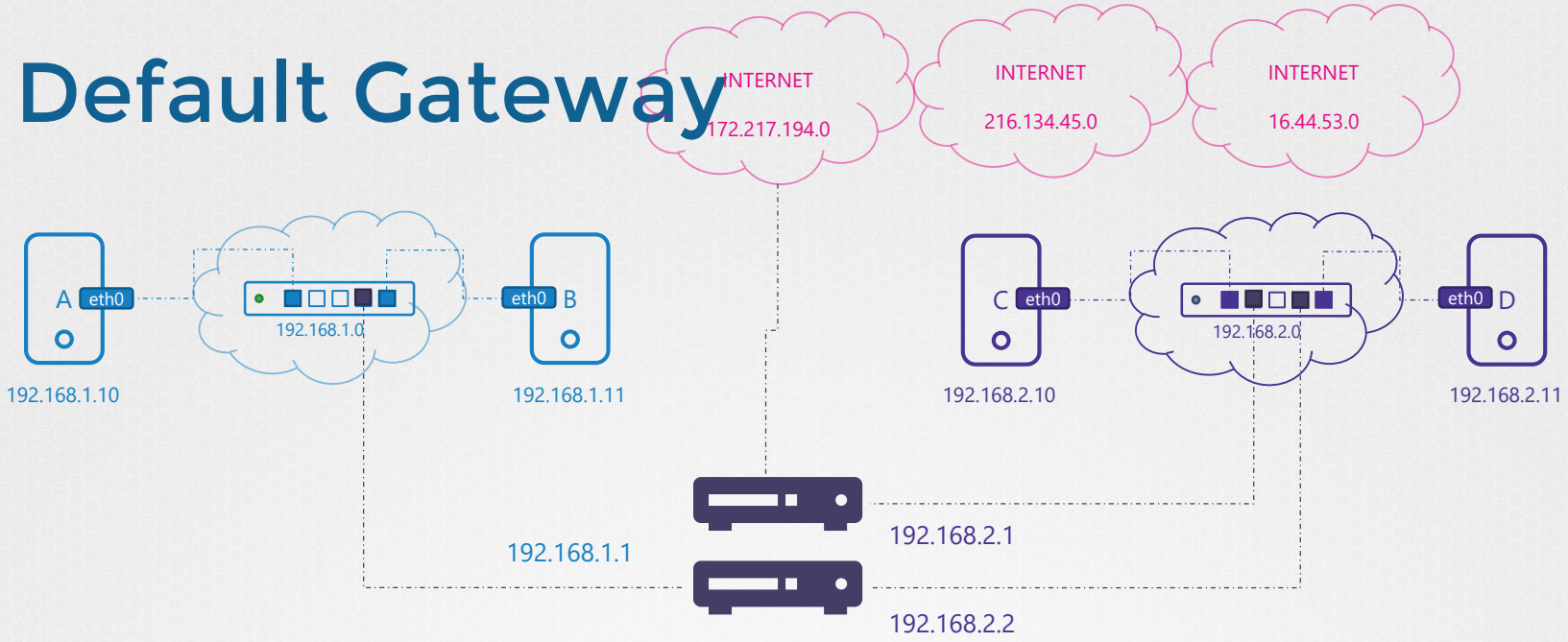

```
ip route add 192.168.1.0/24 via 192.168.2.1
```

```
ip route add default via 192.168.2.1
```

```
route
```

```
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
192.168.1.0     192.168.2.1     255.255.255.0   UG    0      0        0 eth0
0.0.0.0         192.168.2.1     255.255.255.0   UG    0      0        0 eth0
192.168.2.0     0.0.0.0         255.255.255.0   UG    0      0        0 eth0
```

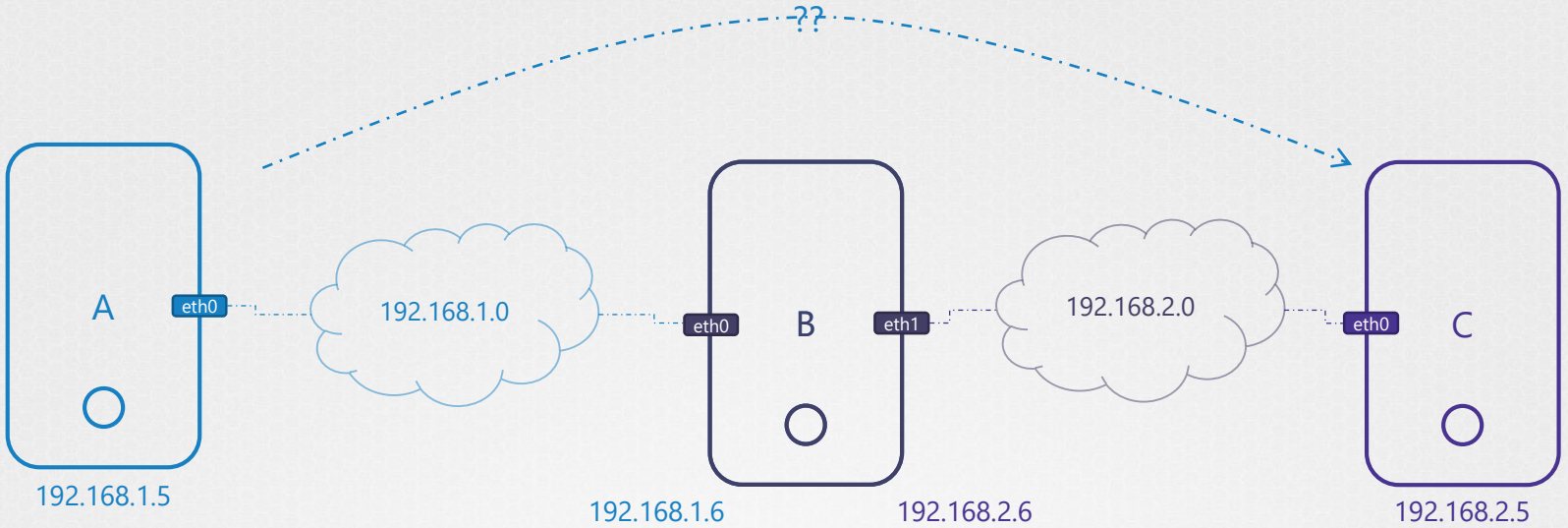
Default Gateway




```
ip route add 192.168.1.0/24 via 192.168.2.2
```

```
route
```

```
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
default         192.168.2.1     255.255.255.0   UG    0      0        0 eth0
192.168.1.0     192.168.2.2     255.255.255.0   UG    0      0        0 eth0
```



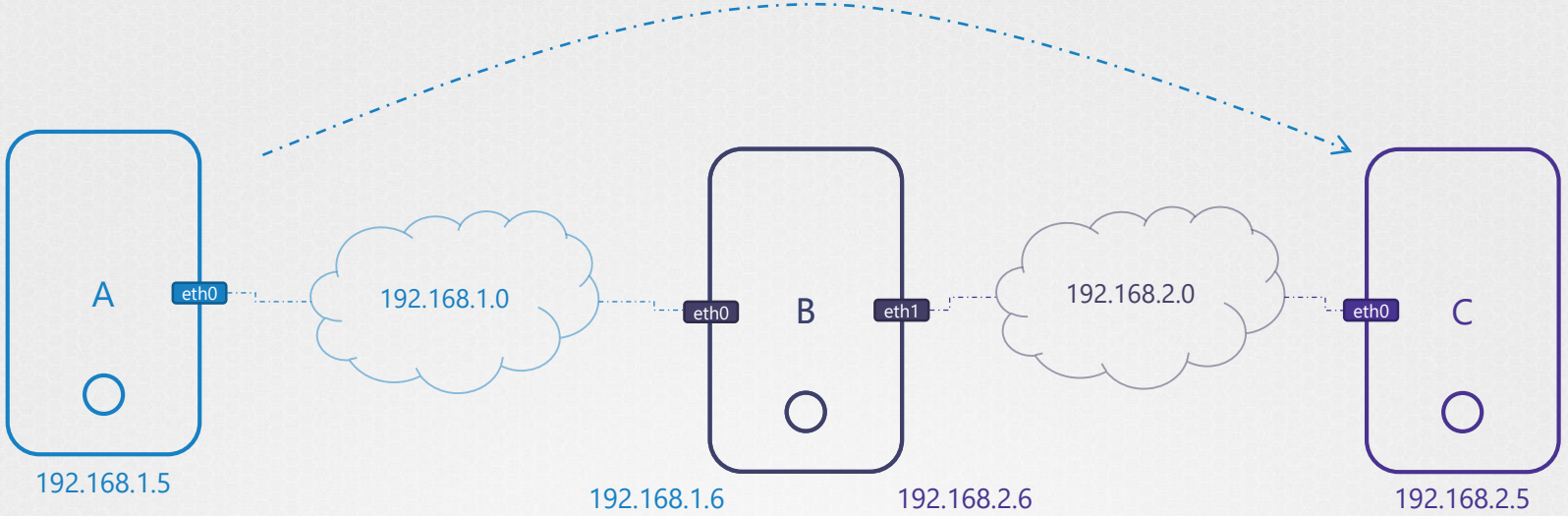

```
ping 192.168.2.5
Connect: Network is unreachable
```

```
ip route add 192.168.2.0/24 via 192.168.1.6
```

```
ip route add 192.168.1.0/24 via 192.168.2.6
```

```
ping 192.168.2.5
```

KODEKLOUD




```
cat /proc/sys/net/ipv4/ip_forward
0
```

```
1.6
```

```
echo 1 > /proc/sys/net/ipv4/ip_forward
1
```

/etc/sysctl.conf

```
…
net.ipv4.ip_forward = 1
…
```

```
ping 192.168.2.5
Reply from 192.168.2.5: bytes=32 time=4ms TTL=117
Reply from 192.168.2.5: bytes=32 time=4ms TTL=117
Reply from 192.168.2.5: bytes=32 time=4ms TTL=117
Reply from 192.168.2.5: bytes=32 time=4ms TTL=117
```

KODEKLOUD

Take Aways

```
ip link
```

```
ip addr
```

```
ip addr add 192.168.1.10/24 dev eth0
```

```
ip route
```

```
route
```

```
ip route add 192.168.1.0/24 via 192.168.2.1
```

```
cat /proc/sys/net/ipv4/ip_forward
1
```

KODEKLOUD

KODEKLOUD

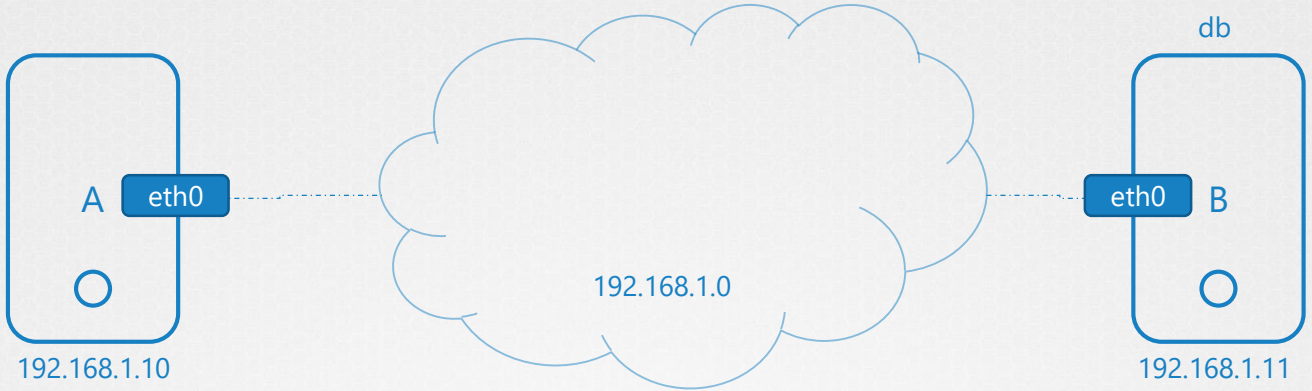
Course Objectives

- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Troubleshooting
- Core Concepts
- Networking
  - Pre-Requisites – Network, Switching, Routing, Tools
  - Pre-Requisites – Network Namespaces
  - Pre-Requisites – DNS, IPAM, Firewalls, LBs
  - Pre-Requisites – Networking in Docker
  - Networking Configuration on Cluster Nodes
  - Service Networking
  - POD Networking Concepts
  - Network Loadbalancer
  - Ingress
  - Cluster DNS
  - CNI

KODEKLOUD

DNS

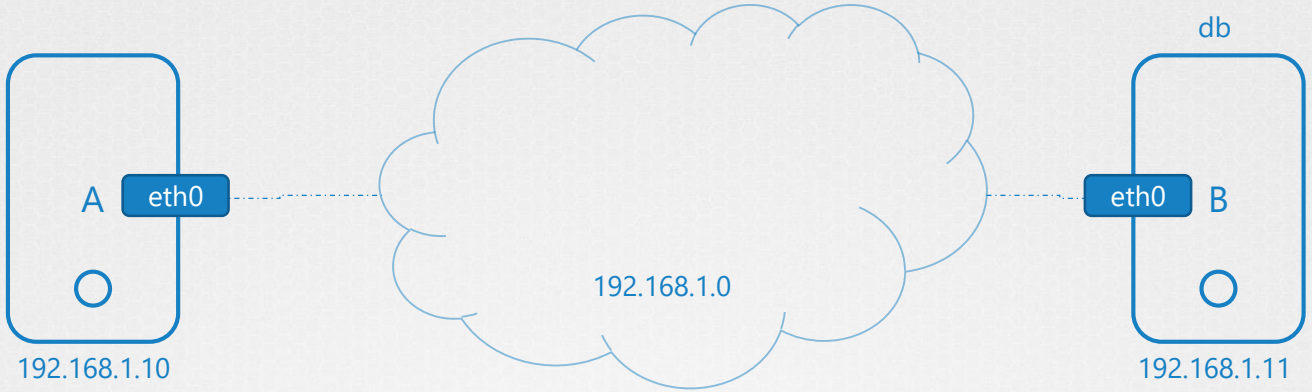
For the Absolute Beginners




```
ping 192.168.1.11
Reply from 192.168.1.11: bytes=32 time=4ms TTL=117
Reply from 192.168.1.11: bytes=32 time=4ms TTL=117
```

```
ping db
ping: unknown host db
```

KODEKLOUD



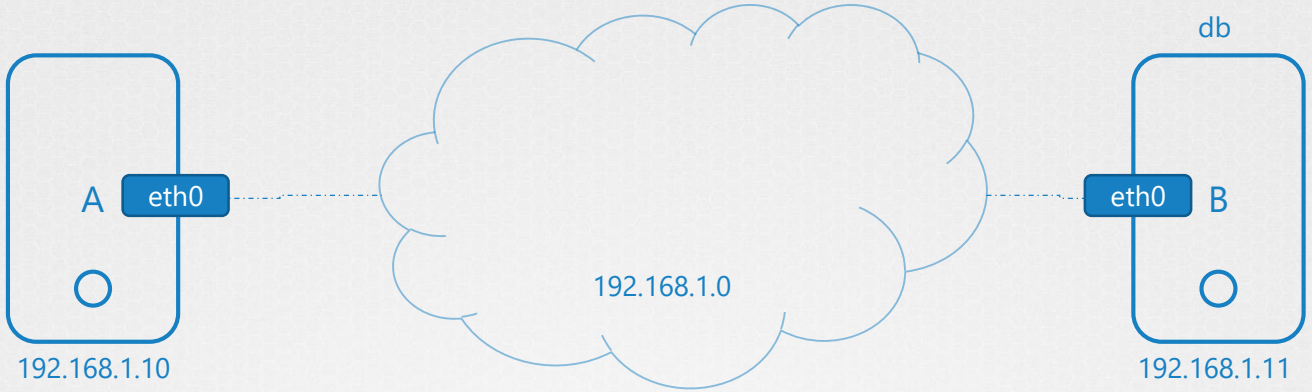

```
ping db
ping: unknown host db
```

```
hostname
host-2
```

```
cat >> /etc/hosts
192.168.1.11        db
```

```
ping db
PING db (192.168.1.11) 56(84) bytes of data.
64 bytes from db (192.168.1.11): icmp_seq=1 ttl=64 time=0.052 ms
64 bytes from db (192.168.1.11): icmp_seq=2 ttl=64 time=0.079 ms
```

KODEKLOUD




```
cat >> /etc/hosts
```

```
192.168.1.11        db
192.168.1.11        www.google.com
```

```
hostname
```

```
host-2
```

```
ping db
```

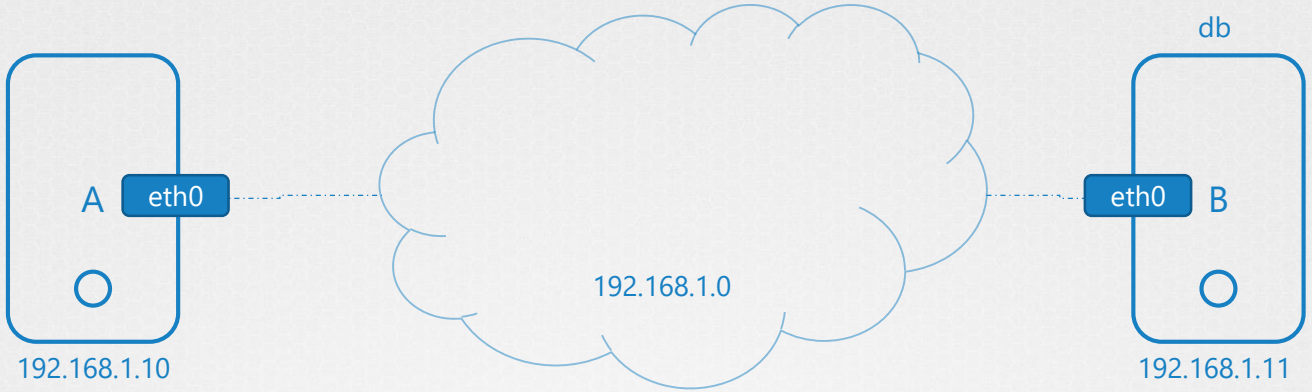
```
PING db (192.168.1.11) 56(84) bytes of data.
64 bytes from db (192.168.1.11): icmp_seq=1 ttl=64 time=0.052 ms
64 bytes from db (192.168.1.11): icmp_seq=2 ttl=64 time=0.079 ms
```

```
ping www.google.com
```

```
PING www.google.com (192.168.1.11) 56(84) bytes of data.
64 bytes from www.google.com (192.168.1.11): icmp_seq=1 ttl=64 time=0.052 ms
64 bytes from www.google.com (192.168.1.11): icmp_seq=2 ttl=64 time=0.079 ms
```

KODEKLOUD

Name Resolution




```
cat >> /etc/hosts
192.168.1.11        db
192.168.1.11        www.google.com
```

```
ping db
```

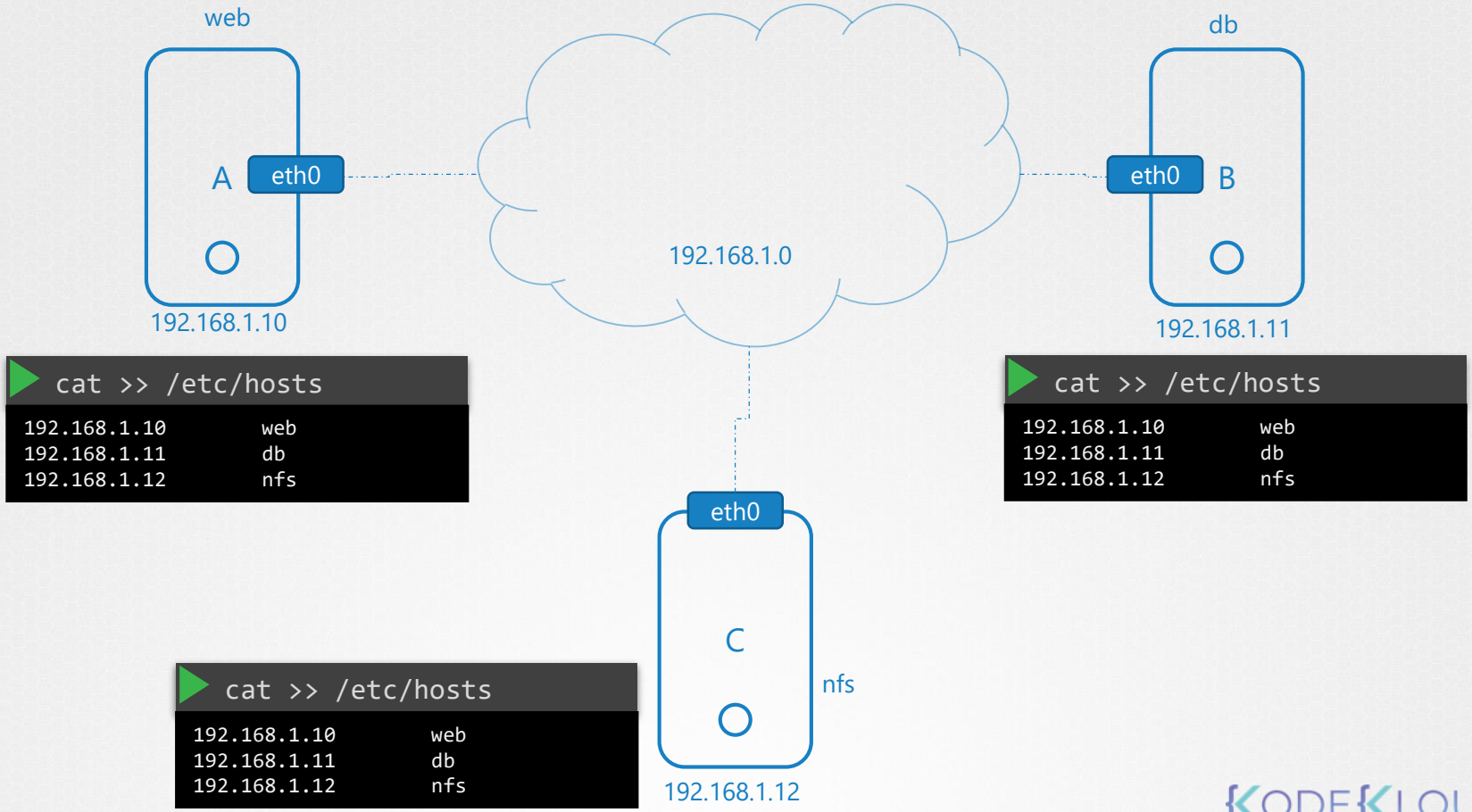
```
ssh db
```

```
curl http://www.google.com
```

```
hostname
host-2
```

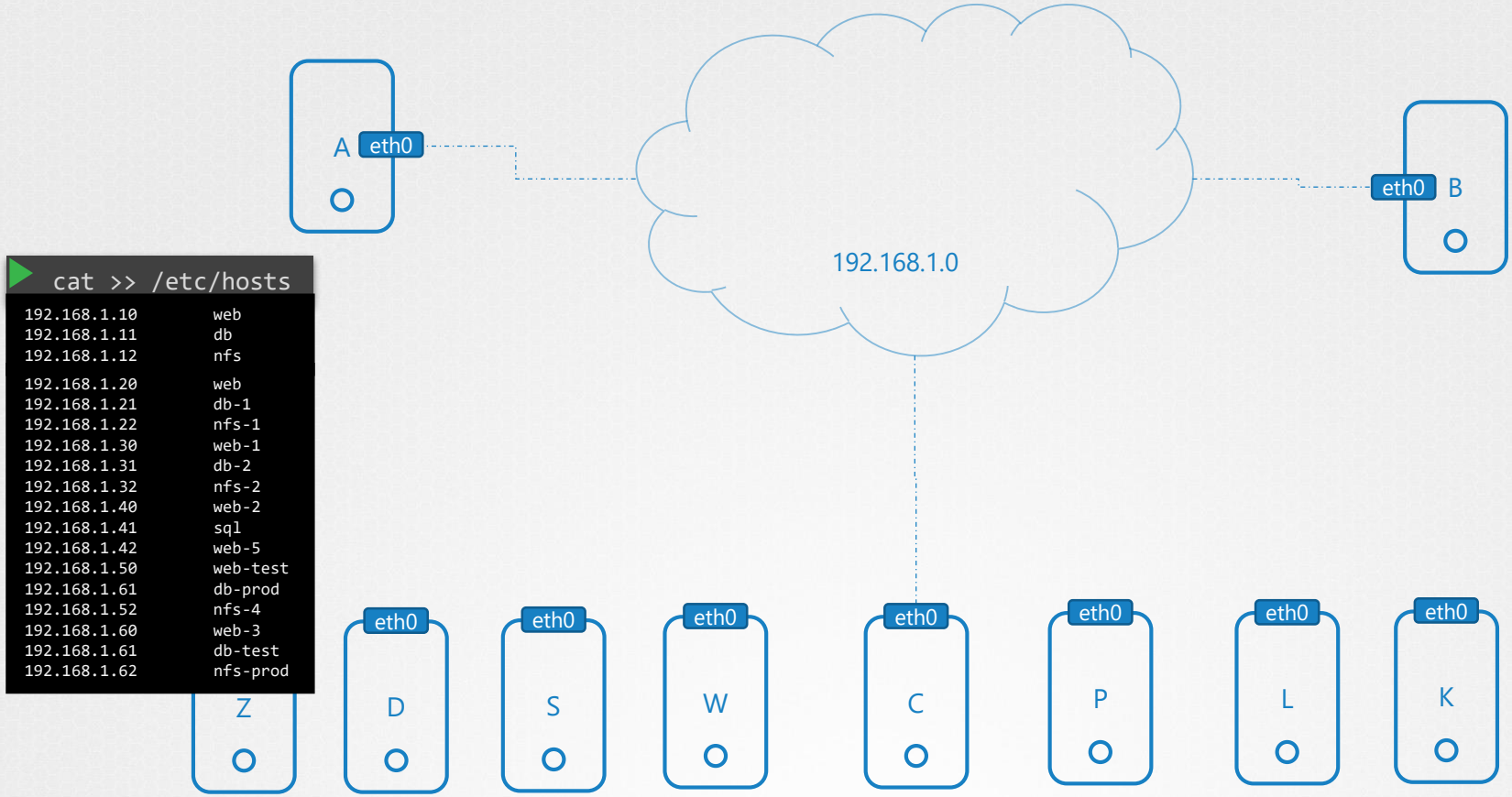
KODEKLOUD

Name Resolution



KODEKLOUD

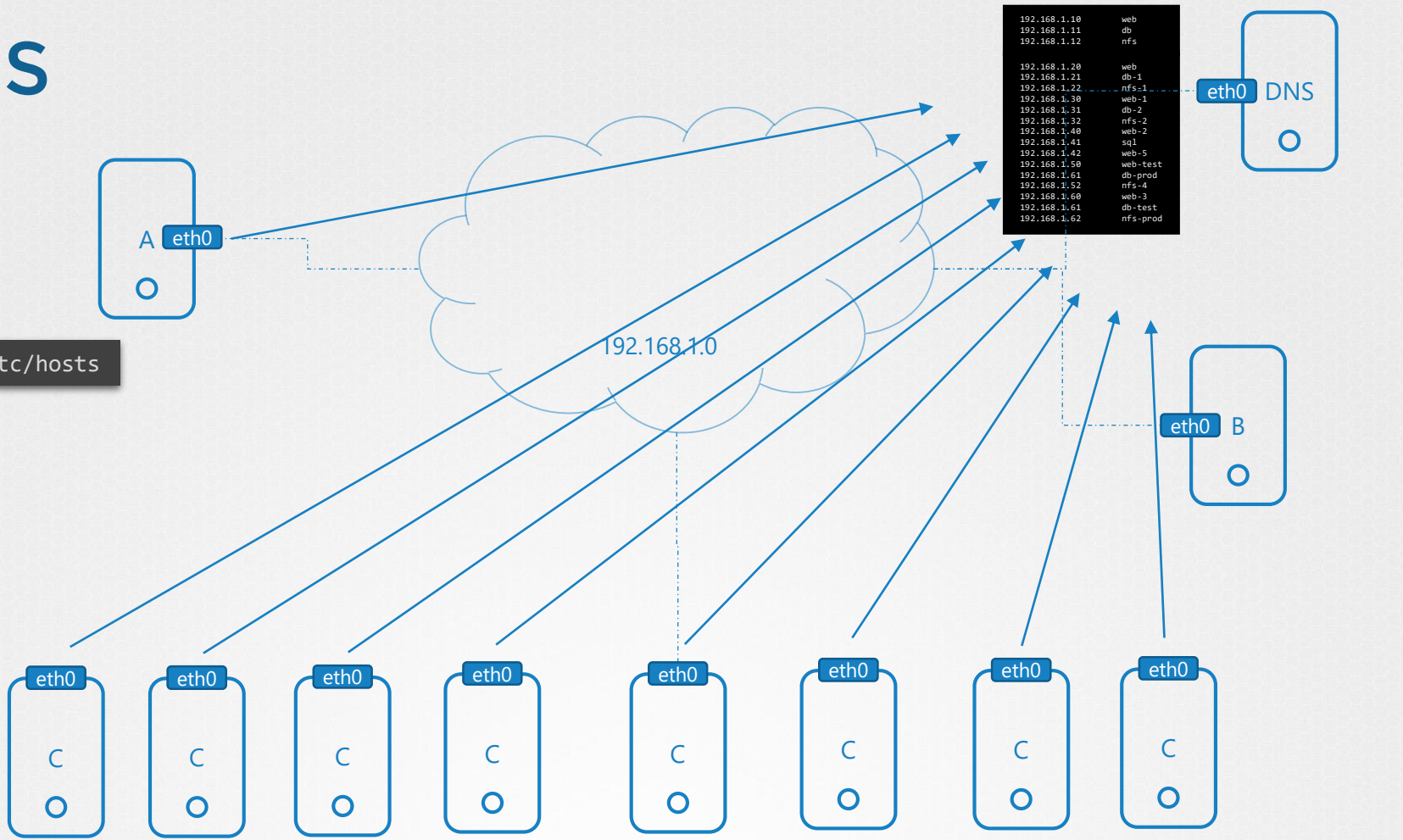
Name Resolution



KODEKLOUD

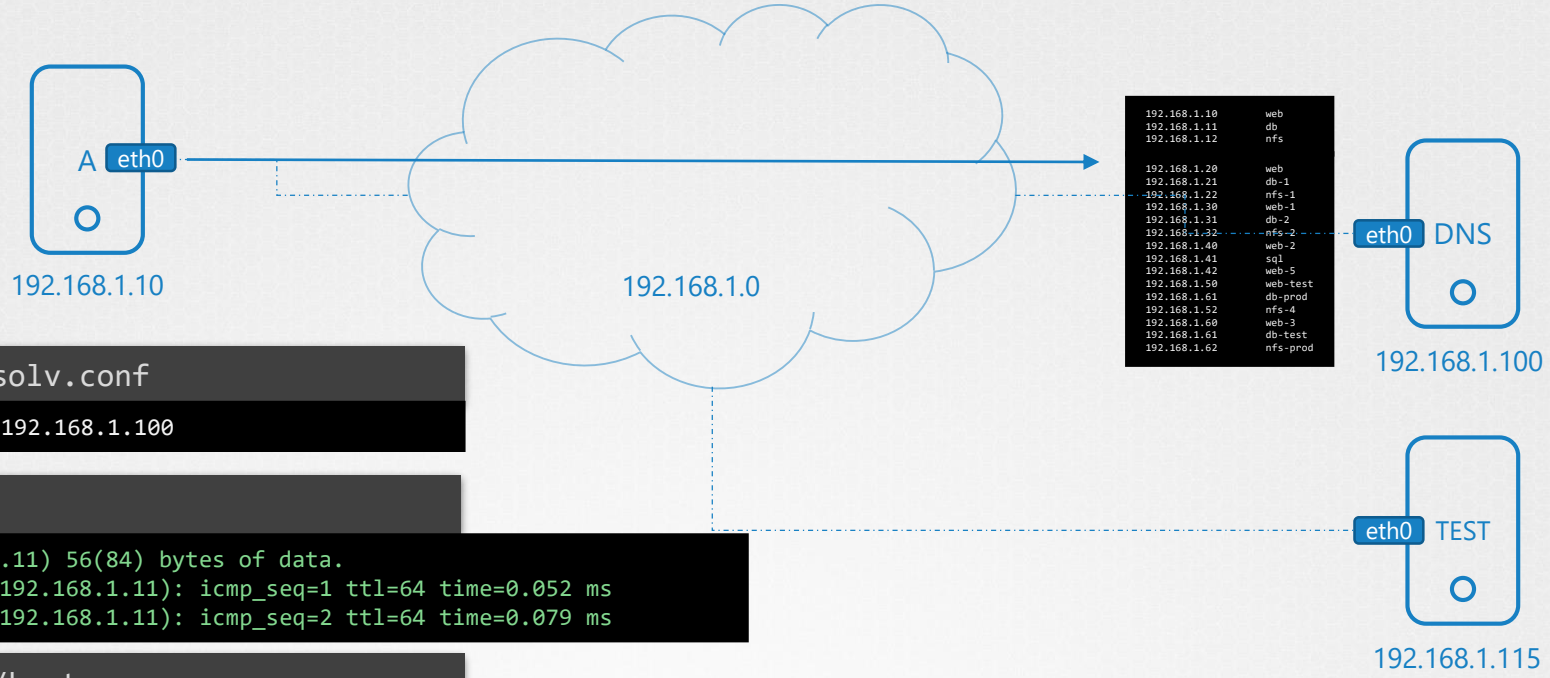
DNS

```
cat >> /etc/hosts
```



KODEKLOUD

DNS



cat /etc/resolv.conf

```
nameserver        192.168.1.100
```

ping db

```
PING db (192.168.1.11) 56(84) bytes of data.
64 bytes from db (192.168.1.11): icmp_seq=1 ttl=64 time=0.052 ms
64 bytes from db (192.168.1.11): icmp_seq=2 ttl=64 time=0.079 ms
```

cat >> /etc/hosts

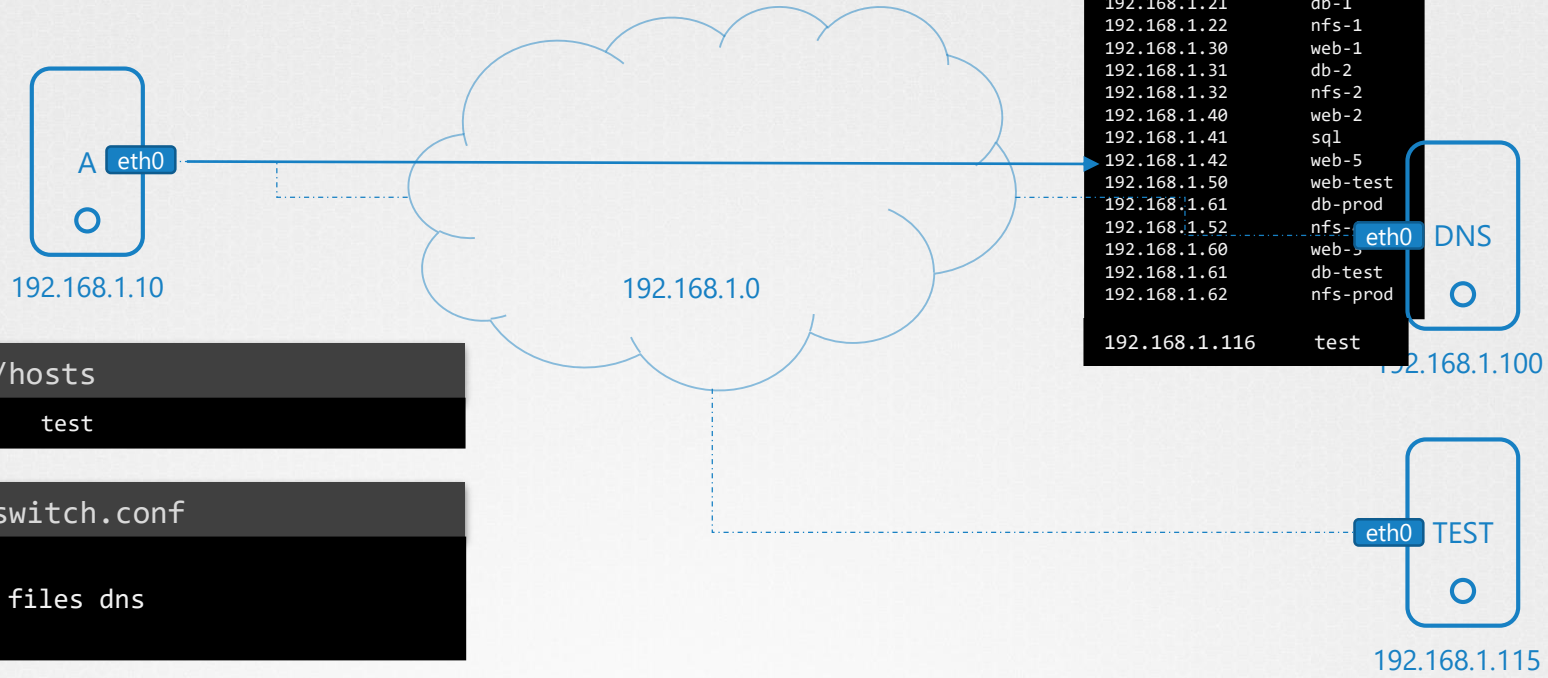
```
192.168.1.115        test
```

ping test

```
PING test (192.168.1.115) 56(84) bytes of data.
64 bytes from test (192.168.1.115): icmp_seq=1 ttl=64 time=0.052 ms
64 bytes from test (192.168.1.115): icmp_seq=2 ttl=64 time=0.079 ms
```

KODEKLOUD

DNS

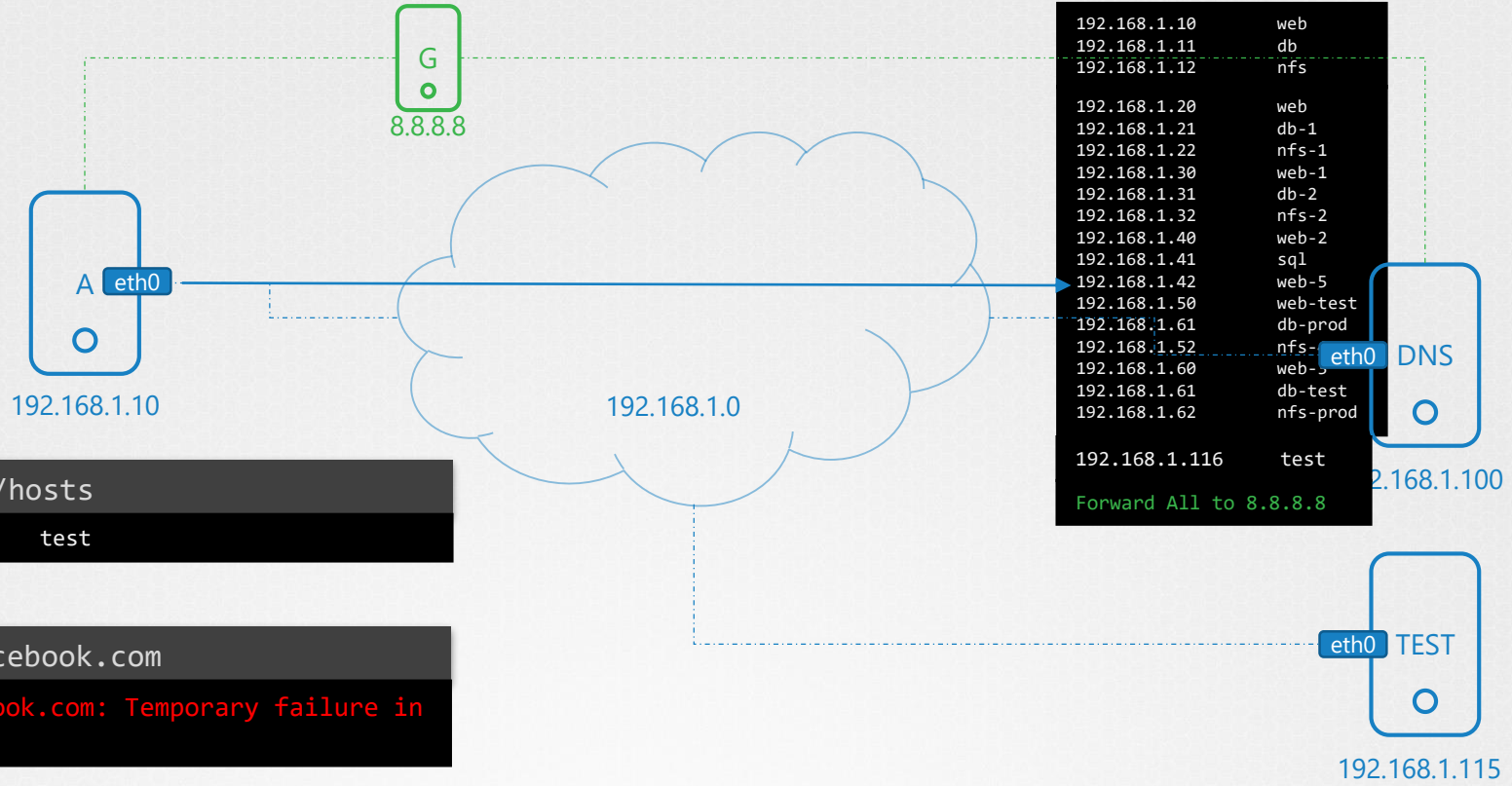



```
cat >> /etc/hosts
192.168.1.115       test
```

```
cat /etc/nsswitch.conf
…
hosts:          files dns
…
```

KODEKLOUD

DNS




```
cat >> /etc/hosts
192.168.1.115        test
```

```
ping www.facebook.com
ping: www.facebook.com: Temporary failure in
name resolution
```

```
cat >> /etc/resolv.conf
nameserver        192.168.1.100
```

```
ping www.facebook.com
PING star-mini.c10r.facebook.com (157.240.13.35) 56(84) bytes of data.
64 bytes from edge-star-mini-shv-02-sin6.facebook.com (157.240.13.35): icmp_seq=1 ttl=50 time=5.70 ms
```

LOUD

Domain Names

wwwkubernetes.io

www.codepen.io

www.facebook.com

www.un.org

www.mit.edu

www.google.com

www.behance.net

www.speedtest.net

www.stanford.edu

www.care.org

KODEKLOUD

Domain Names

.com	.net	.edu	.org	.io
www.google	www.behance	www.stanford	www.care	www.kubernetes
www.facebook	www.speedtest	www.mit	www.un	www.codepen

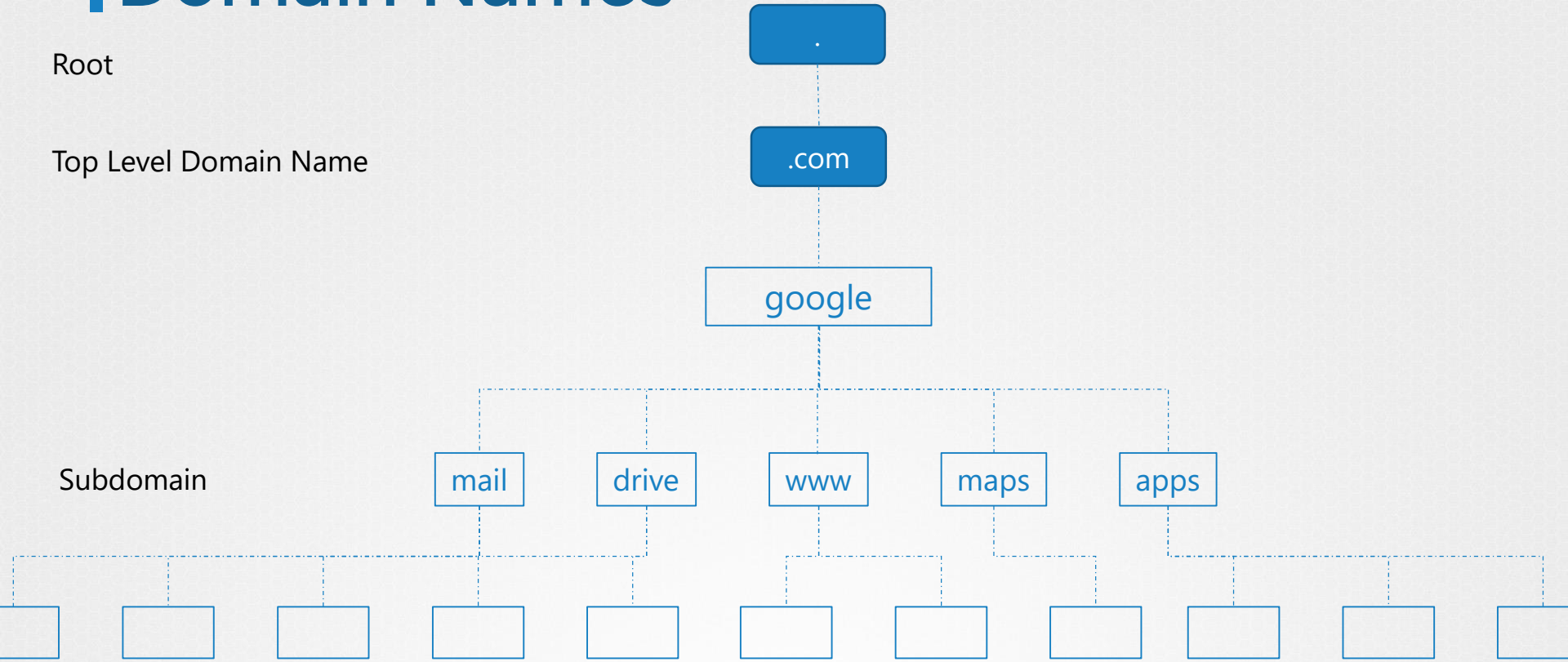
KODEKLOUD

Domain Names

Root

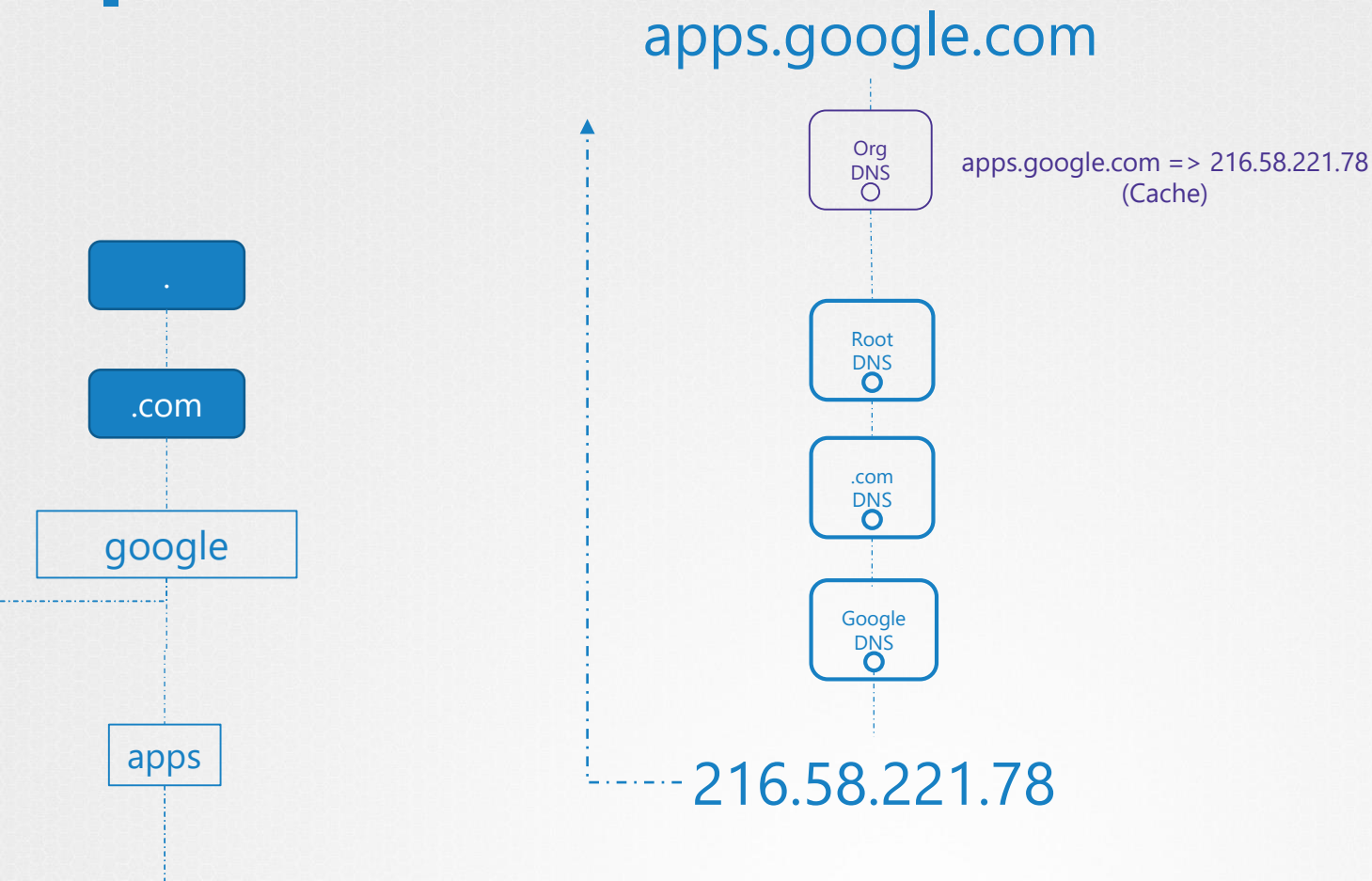
Top Level Domain Name

Subdomain



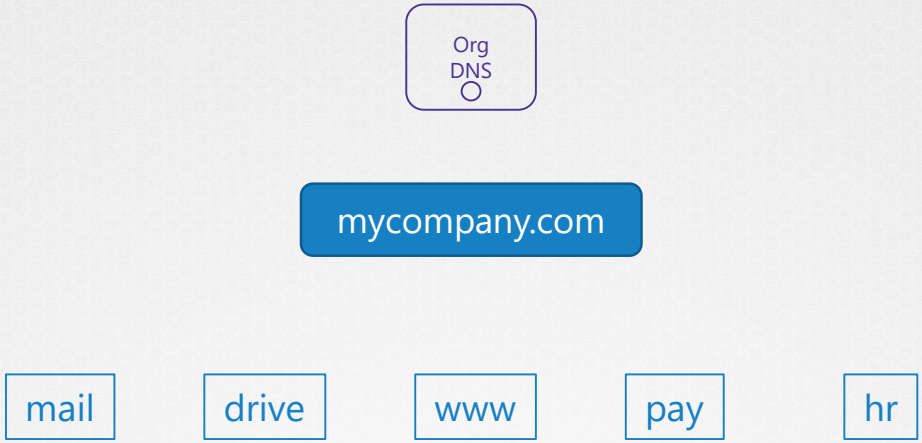
KODEKLOUD

Domain Names



KODEKLOUD

Domain Names



KODEKLOUD

Search Domain

```
192.168.1.10        web.mycompany.com
192.168.1.11        db.mycompany.com
192.168.1.12        nfs.mycompany.com
192.168.1.13        web-1.mycompany.com
192.168.1.14        sql.mycompany.com
```

mycompany.com

nfs web mail drive www pay hr sql

```
cat >> /etc/resolv.conf
nameserver          192.168.1.100
search              mycompany.com  prod.mycompany.com
```

```
ping web
PING web (192.168.1.10) 56(84) bytes of data.
64 bytes from web (192.168.1.10): icmp_seq=1 ttl=64 time=0.052 ms
64 bytes from web (192.168.1.10): icmp_seq=2 ttl=64 time=0.079 ms
```

```
ping web
PING web.mycompany.com (192.168.1.10) 56(84) bytes of data.
64 bytes from web.mycompany.com (192.168.1.10): … time=0.052 ms
64 bytes from web.mycompany.com (192.168.1.10): … time=0.079 ms
```

```
ping web
…: web: Temporary failure in name resolution
```

```
ping web.mycompany.com
PING web.mycompany.com (192.168.1.10) 56(84) bytes of data.
64 bytes from web.mycompany.com (192.168.1.10): ttl=64 time=0.052 ms
```

```
ping web.mycompany.com
PING web.mycompany.com (192.168.1.10) 56(84) bytes of data.
…om web.mycompany.com (192.168.1.10): ttl=64 time=0.052 ms
```

KODEKLOUD

Record Types

A	web-server	192.168.1.1
AAAA	web-server	2001:0db8:85a3:0000:0000:8a2e:0370:7334
CNAME	food.web-server	eat.web-server, hungry.web-server

KODEKLOUD

nslookup

```
nslookup www.google.com
Server:         8.8.8.8
Address:        8.8.8.8#53

Non-authoritative answer:
Name:   www.google.com
Address: 172.217.0.132
```

KODEKLOUD

dig

```
dig www.google.com
```

```
; <<>> DiG 9.10.3-P4-Ubuntu <<>> www.google.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 28065
;; flags: qr rd ra; QUERY: 1, ANSWER: 6, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 512
;; QUESTION SECTION:
;www.google.com.                        IN      A

;; ANSWER SECTION:
www.google.com.         245     IN      A       64.233.177.103
www.google.com.         245     IN      A       64.233.177.105
www.google.com.         245     IN      A       64.233.177.147
www.google.com.         245     IN      A       64.233.177.106
www.google.com.         245     IN      A       64.233.177.104
www.google.com.         245     IN      A       64.233.177.99

;; Query time: 5 msec
;; SERVER: 8.8.8.8#53(8.8.8.8)
;; WHEN: Sun Mar 24 04:34:33 UTC 2019
;; MSG SIZE  rcvd: 139
```

KODEKLOUD

KODEKLOUD

Course Objectives

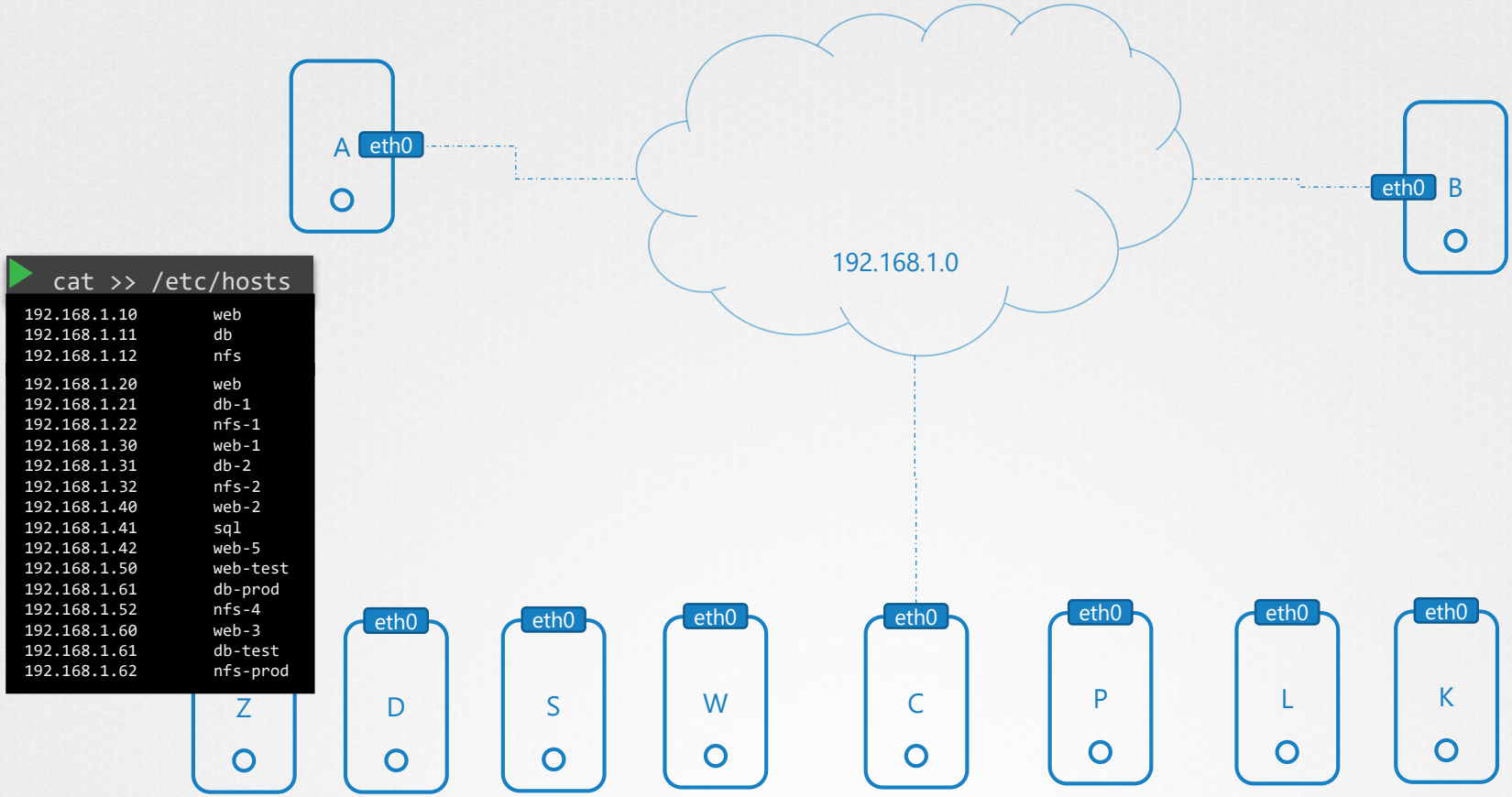
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Troubleshooting
- Core Concepts
- Networking
  - Pre-Requisites – Network, Switching, Routing, Tools
  - Pre-Requisites – Network Namespaces
  - Pre-Requisites – DNS, IPAM, Firewalls, LBs
  - Pre-Requisites – Networking in Docker
  - Networking Configuration on Cluster Nodes
  - Service Networking
  - POD Networking Concepts
  - Network Loadbalancer
  - Ingress
  - Cluster DNS
  - CNI

KODEKLOUD

CoreDNS

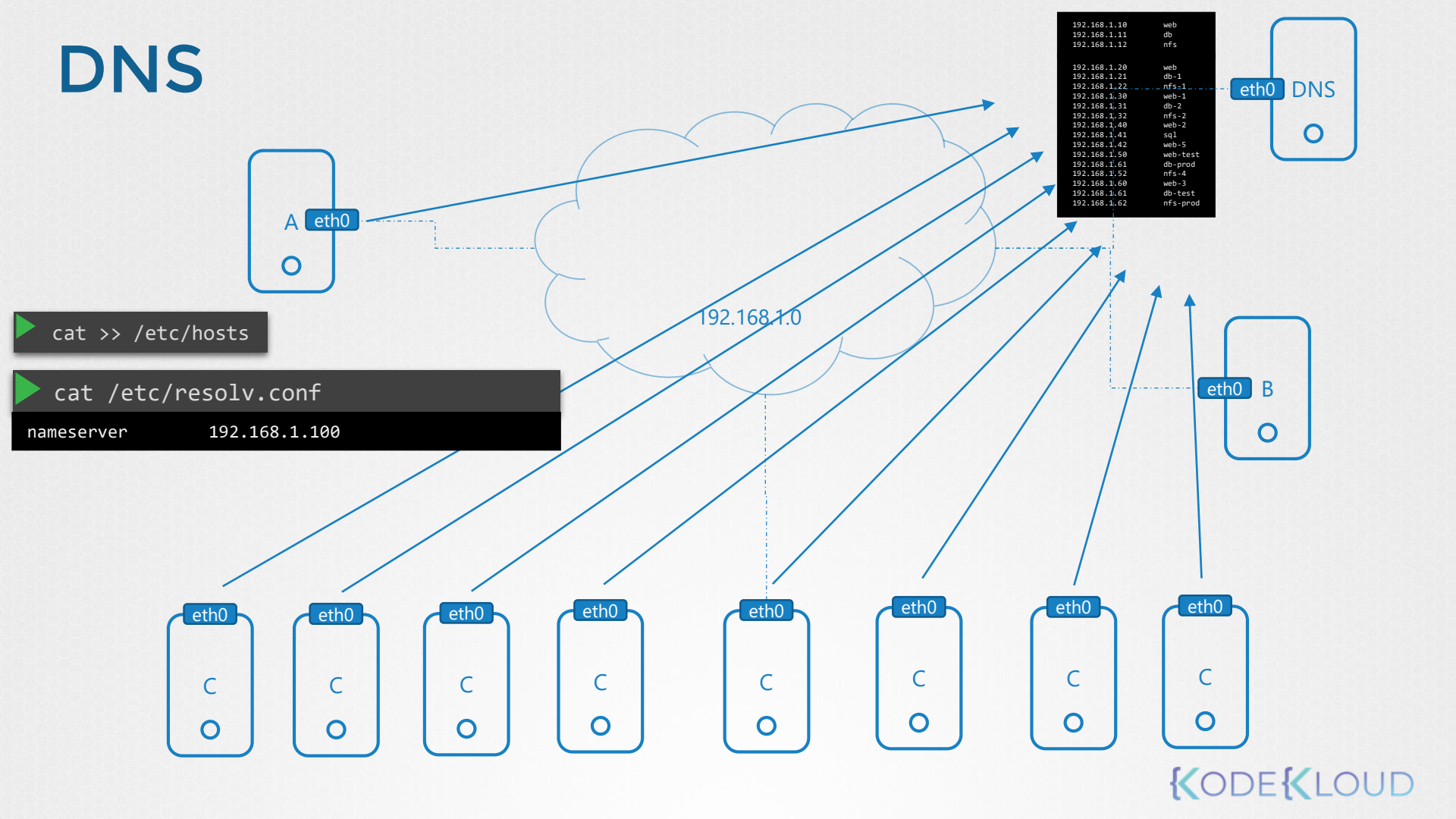
KODEKLOUD

Name Resolution



KODEKLOUD

DNS



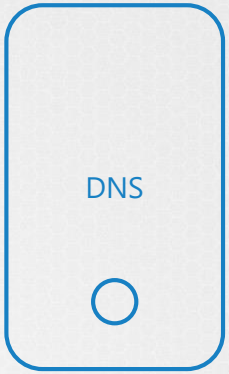

```
cat >> /etc/hosts
```

```
cat /etc/resolv.conf
nameserver        192.168.1.100
```

KODEKLOUD

CoreDNS

```
192.168.1.10        web
192.168.1.11        db

192.168.1.20        web
192.168.1.21        db-1
192.168.1.22        nfs-1
192.168.1.30        web-1
192.168.1.31        db-2
192.168.1.32        nfs-2
192.168.1.40        web-2
192.168.1.41        sql
192.168.1.42        web-5
192.168.1.50        web-test
192.168.1.61        db-prod
192.168.1.52        nfs-4
192.168.1.60        web-3
192.168.1.61        db-test
192.168.1.62        nfs-prod
```



KODEKLOUD

CoreDNS

```
wget https://github.com/coredns/coredns/releases/download/v1.4.0/coredns_1.4.0_linux_amd64.tgz
```

```
coredns_1.4.0_linux_amd64.tgz
```

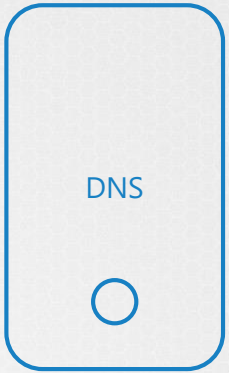
```
tar -xzvf coredns_1.4.0_linux_amd64.tgz
```

```
coredns
```

```
./coredns
```

```
.:53
2019-03-04T10:46:13.756Z [INFO] CoreDNS-1.4.0
2019-03-04T10:46:13.756Z [INFO] linux/amd64, go1.12,
8dcc7fc
CoreDNS-1.4.0
linux/amd64, go1.12, 8dcc7fc
```

```
192.168.1.10    web
192.168.1.11    db

192.168.1.20    web
192.168.1.21    db-1
192.168.1.22    nfs-1
192.168.1.30    web-1
192.168.1.31    db-2
192.168.1.32    nfs-2
192.168.1.40    web-2
192.168.1.41    sql
192.168.1.42    web-5
192.168.1.50    web-test
192.168.1.61    db-prod
192.168.1.52    nfs-4
192.168.1.60    web-3
192.168.1.61    db-test
192.168.1.62    nfs-prod
```

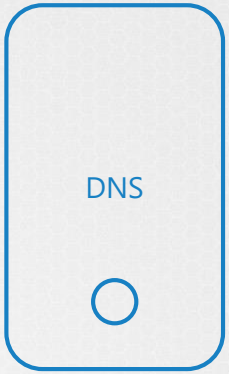


KODEKLOUD

CoreDNS

```
cat /etc/hosts
```

```
192.168.1.10        web
192.168.1.11        db

192.168.1.20        web
192.168.1.21        db-1
192.168.1.22        nfs-1
192.168.1.30        web-1
192.168.1.31        db-2
192.168.1.32        nfs-2
192.168.1.40        web-2
192.168.1.41        sql
192.168.1.42        web-5
192.168.1.50        web-test
192.168.1.61        db-prod
192.168.1.52        nfs-4
192.168.1.60        web-3
192.168.1.61        db-test
192.168.1.62        nfs-prod
```



KODEKLOUD

CoreDNS

```
cat > /etc/hosts
```

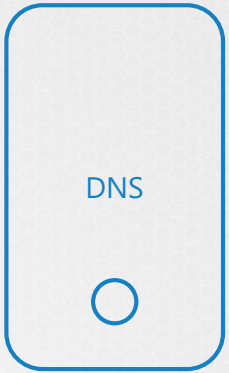
```
192.168.1.10    web
192.168.1.11    db

192.168.1.20    web
192.168.1.21    db-1
192.168.1.22    nfs-1
192.168.1.30    web-1
192.168.1.31    db-2
192.168.1.32    nfs-2
192.168.1.40    web-2
192.168.1.41    sql
192.168.1.42    web-5
192.168.1.50    web-test
192.168.1.61    db-prod
192.168.1.52    nfs-4
192.168.1.60    web-3
192.168.1.61    db-test
192.168.1.62    nfs-prod
```

```
cat > Corefile
```

```
. {
    hosts /etc/hosts
}
```

```
./coredns
```

```
.:53
2019-03-04T10:46:13.756Z [INFO] CoreDNS-1.4.0
2019-03-04T10:46:13.756Z [INFO] linux/amd64, go1.12,
8dcc7fc
CoreDNS-1.4.0
linux/amd64, go1.12, 8dcc7fc
```



KODEKLOUD

KODEKLOUD

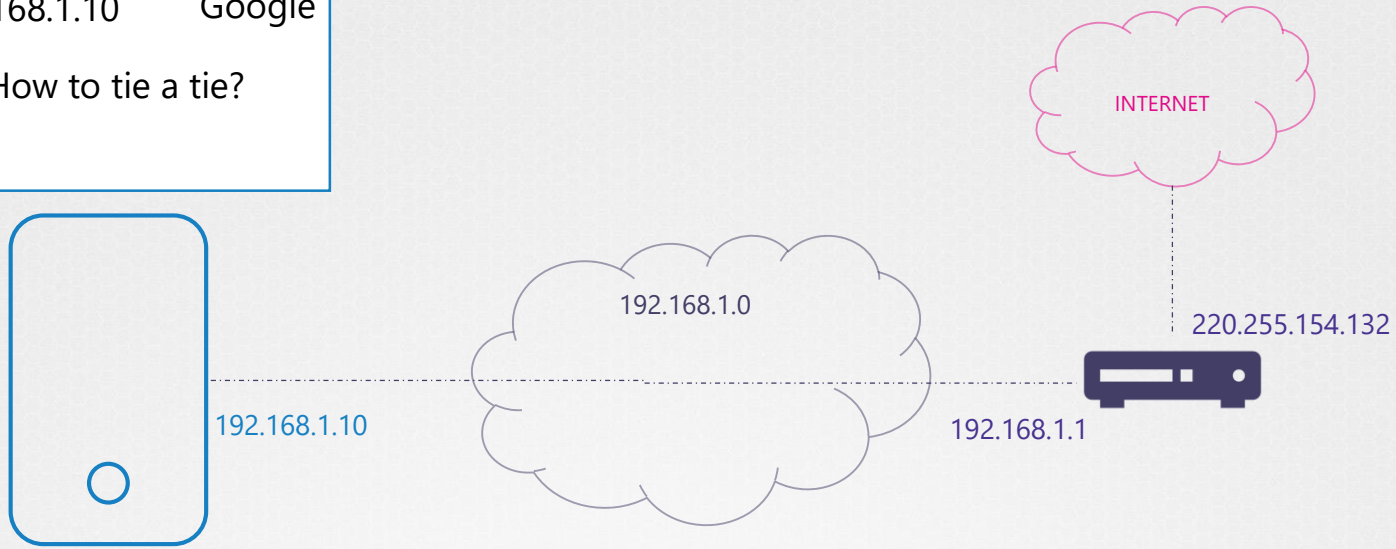
PRE-REQUISITE

NETWORK ADDRESS TRANSLATION (NAT)

From: 192.168.1.10

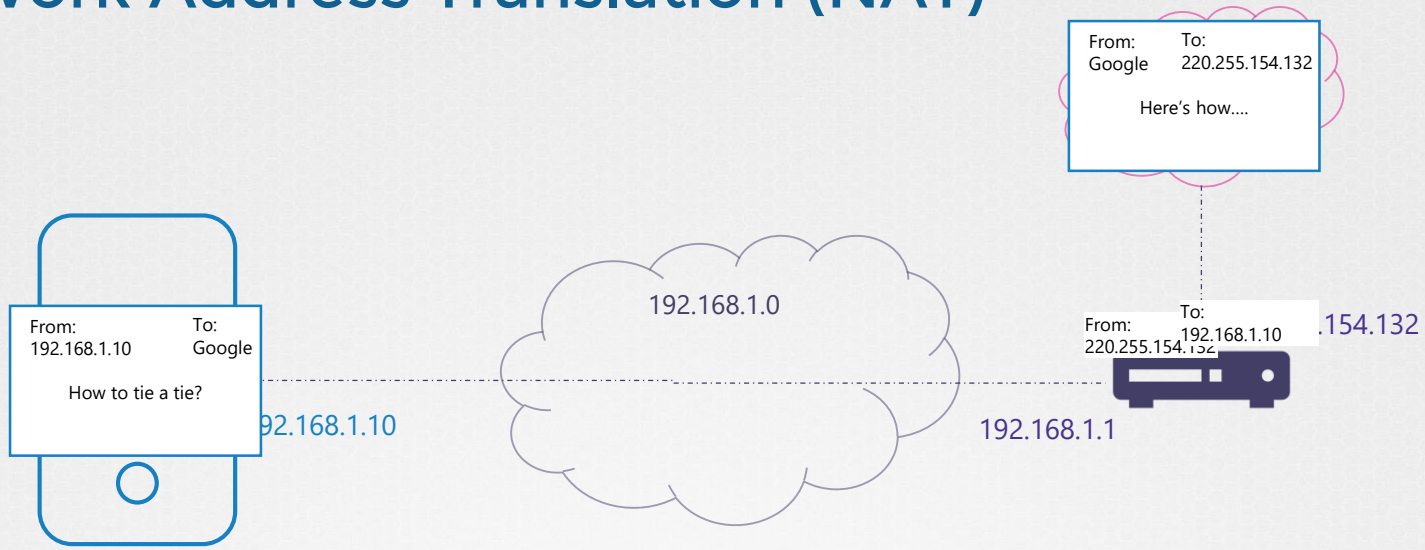
To: Google

How to tie a tie?



KODEKLOUD

Network Address Translation (NAT)



NAT TABLE

Private IP	Public IP
192.168.1.10	220.255.154.132

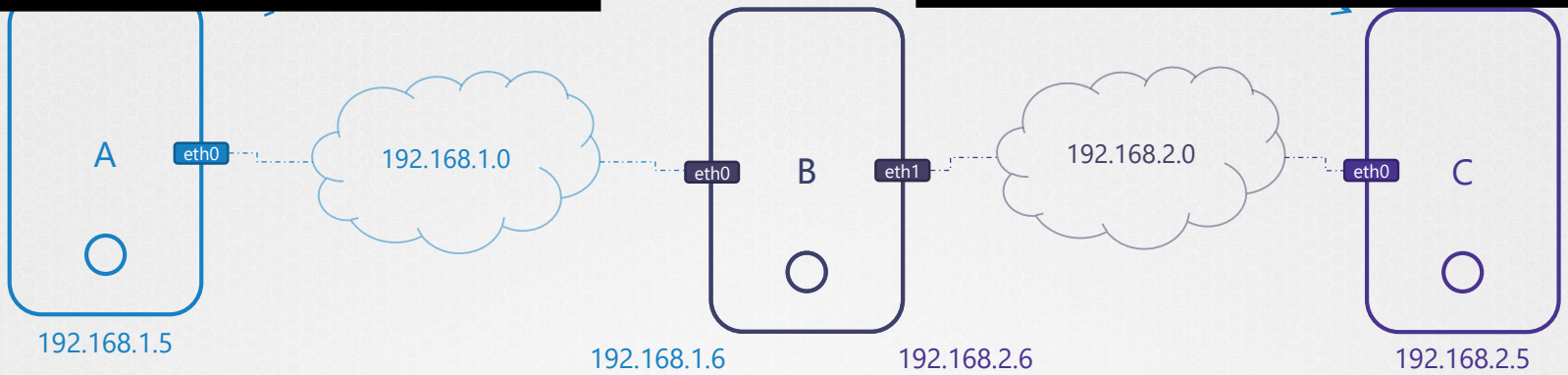
KODEKLOUD

```
ping 192.168.2.5
```

```
Reply from 192.168.2.5: bytes=32 time=4ms TTL=117
Reply from 192.168.2.5: bytes=32 time=4ms TTL=117
Reply from 192.168.2.5: bytes=32 time=4ms TTL=117
Reply from 192.168.2.5: bytes=32 time=4ms TTL=117
```

```
tcpdump -l
```

```
05:52:50.901754 IP 192.168.1.5 > 192.168.2.5: ICMP echo requh 64
05:52:50.901754 IP 192.168.1.5 > 192.168.2.5: ICMP echo requh 64
05:52:50.901754 IP 192.168.1.5 > 192.168.2.5: ICMP echo requh 64
05:52:50.901754 IP 192.168.1.5 > 192.168.2.5: ICMP echo requh 64
```




```
echo 1 > /proc/sys/net/ipv4/ip_forward
```

```
1
```

```
ip route add 192.168.2.0/24 via 192.168.1.6
```

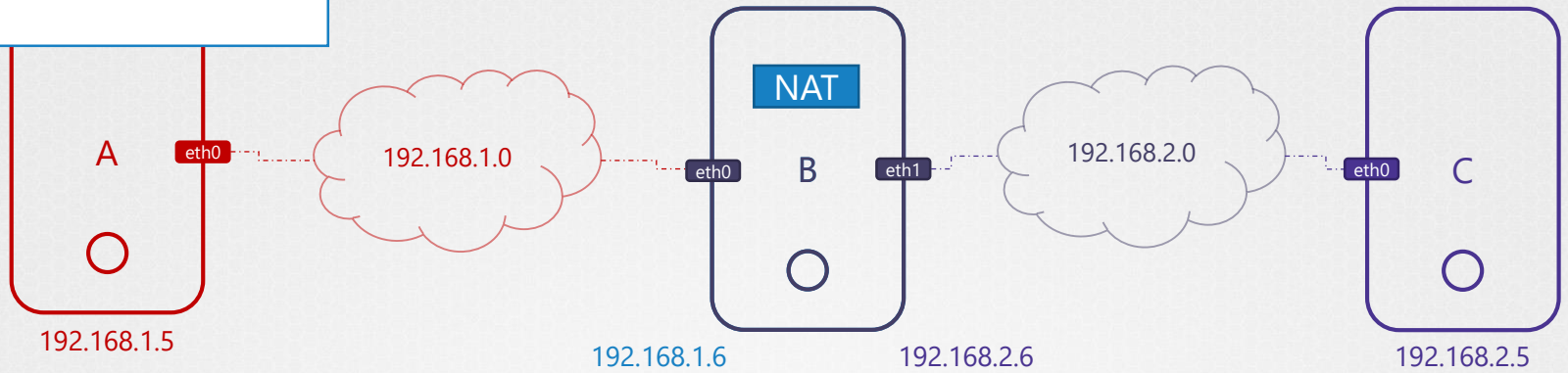
```
ip route add 192.168.1.0/24 via 192.168.2.6
```

KODEKLOUD

From: 192.168.1.5
To: 192.168.2.5

How to tie a tie?

From: 192.168.2.6




```
iptables -t nat -A POSTROUTING -s 192.168.5.0/24 -j MASQUERADE
```

```
echo 1 > /proc/sys/net/ipv4/ip_forward
```

```
1
```

```
ip route add 192.168.2.0/24 via 192.168.1.6
```

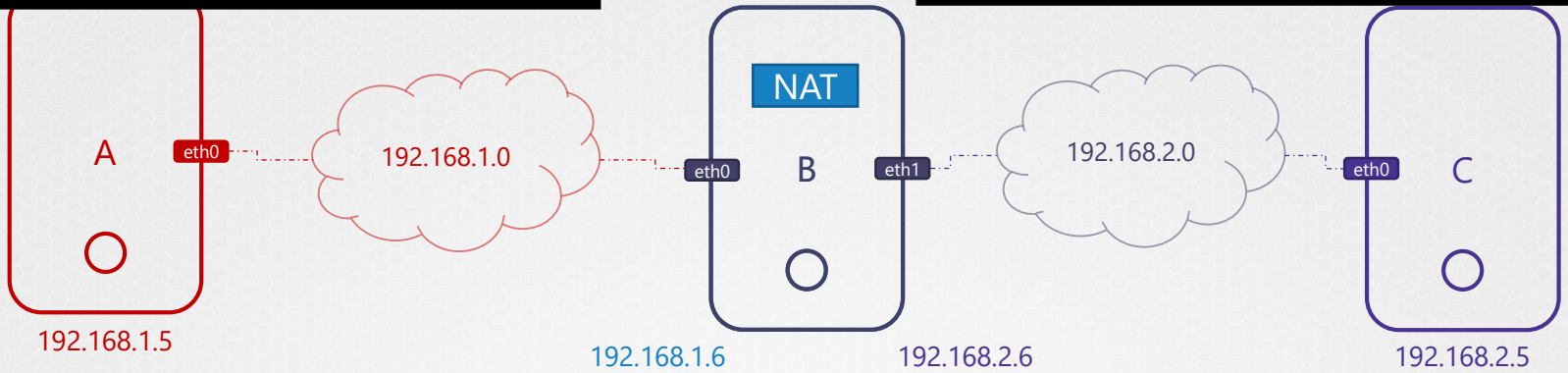
```
ip route add 192.168.1.0/24 via 192.168.2.6
```

```
ping 192.168.2.5
```

```
Reply from 192.168.2.5: bytes=32 time=4ms TTL=117
Reply from 192.168.2.5: bytes=32 time=4ms TTL=117
Reply from 192.168.2.5: bytes=32 time=4ms TTL=117
Reply from 192.168.2.5: bytes=32 time=4ms TTL=117
```

```
tcpdump -l
```

```
05:52:50.901754 IP 192.168.2.6 > 192.168.2.5: ICMP echo requh 64
05:52:50.901754 IP 192.168.2.6 > 192.168.2.5: ICMP echo requh 64
05:52:50.901754 IP 192.168.2.6 > 192.168.2.5: ICMP echo requh 64
05:52:50.901754 IP 192.168.2.6 > 192.168.2.5: ICMP echo requh 64
```



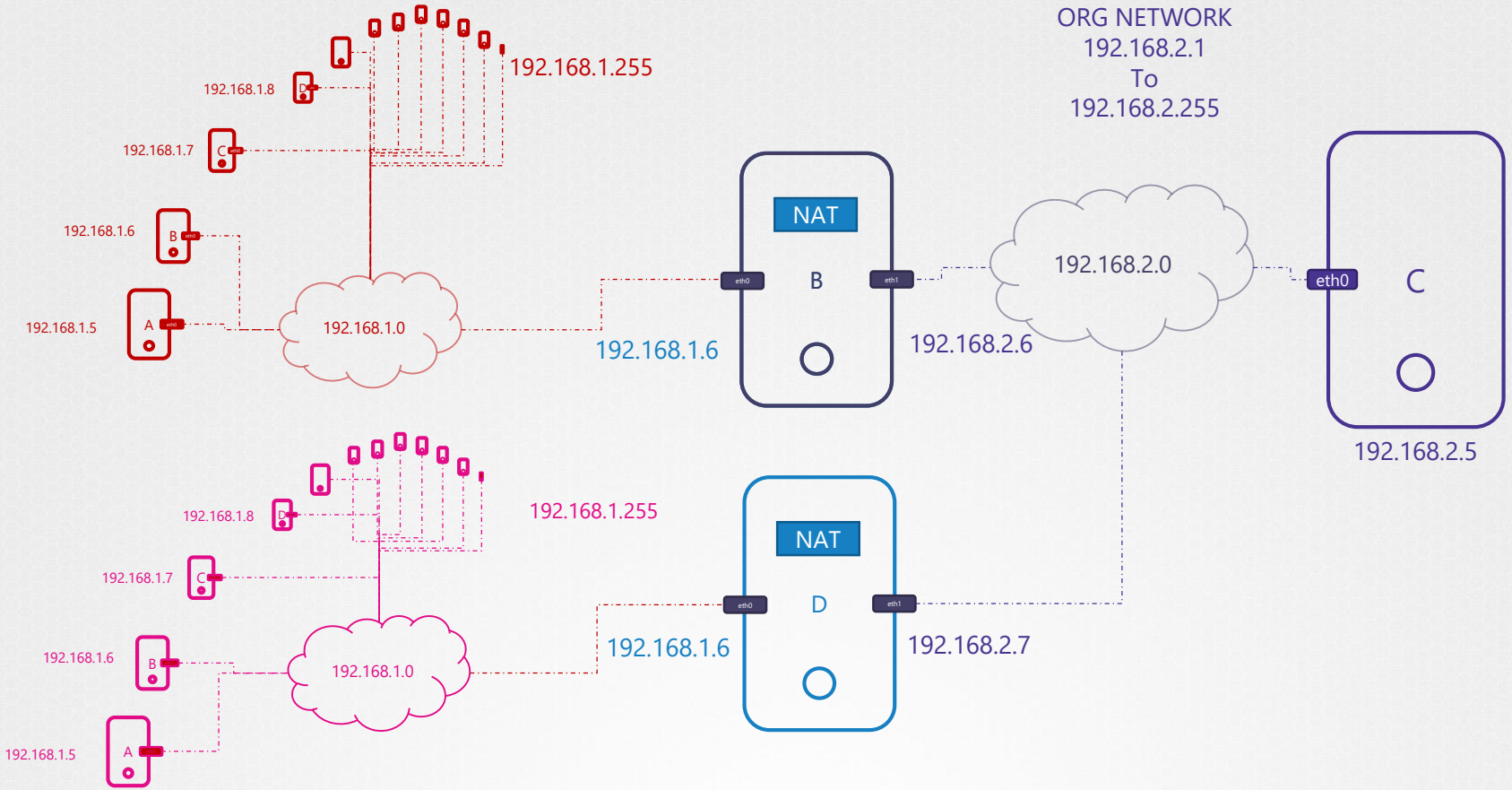

```
iptables -t nat -A POSTROUTING -s 192.168.5.0/24 -j MASQUERADE
```

```
echo 1 > /proc/sys/net/ipv4/ip_forward
```

```
1
```

```
ip route add 192.168.2.0/24 via 192.168.1.6
```

KODEKLOUD


ORG NETWORK
192.168.2.1
To
192.168.2.255
192.168.1.255
192.168.1.8
192.168.1.7
192.168.1.6
192.168.1.5
192.168.1.0
NAT
B
eth0
eth1
192.168.1.6
192.168.2.6
192.168.2.0
eth0
C
192.168.2.5
192.168.1.255
192.168.1.8
192.168.1.7
192.168.1.6
192.168.1.5
192.168.1.0
NAT
D
eth0
eth1
192.168.1.6
192.168.2.7

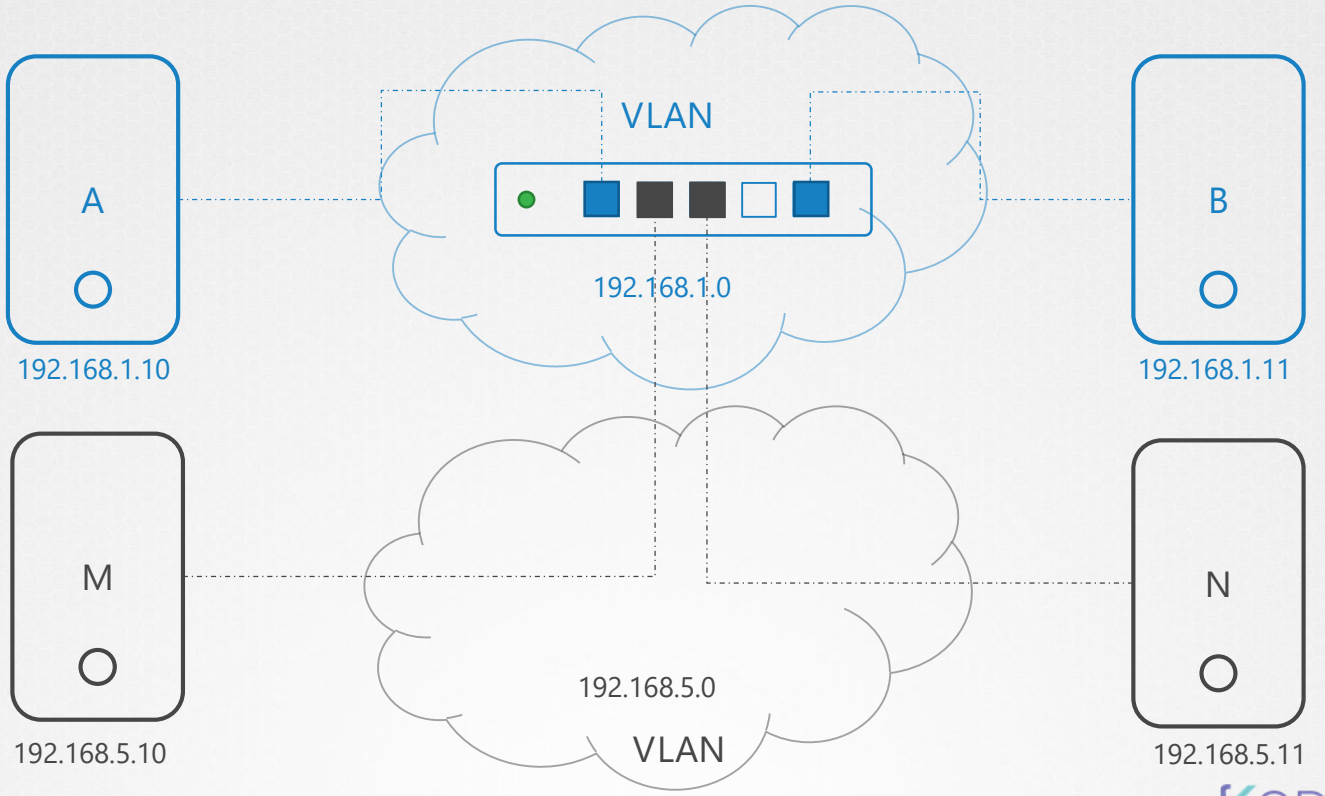
KODEKLOUD

KODEKLOUD

PRE-REQUISITE

NETWORKING VLAN & VXLAN

VLAN

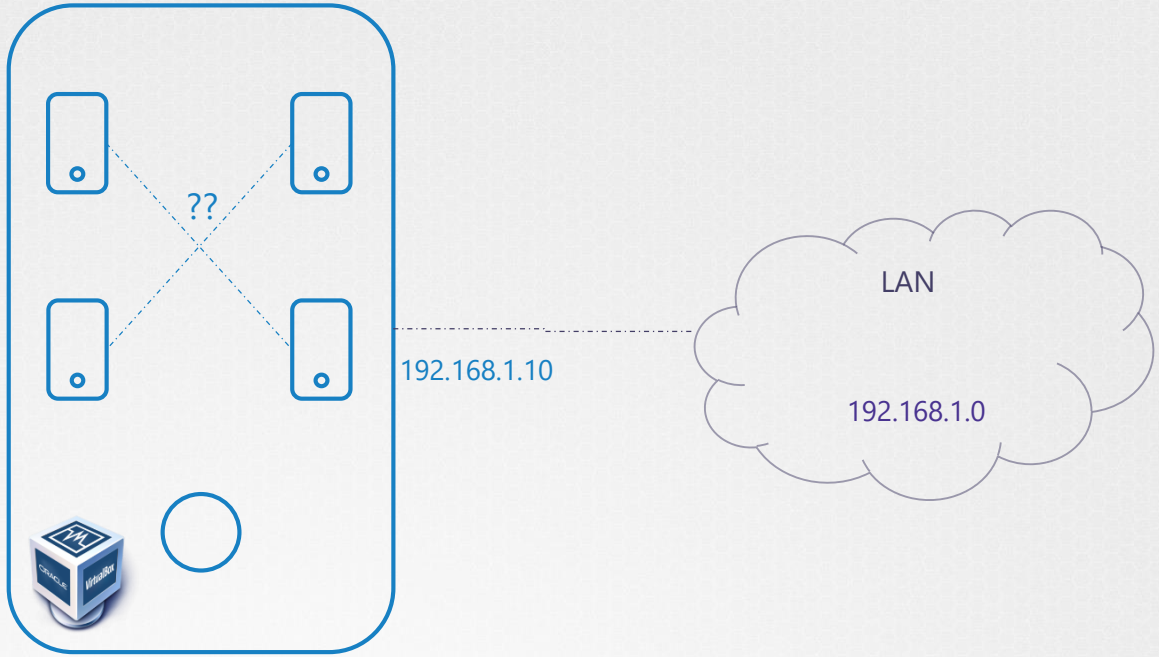


KODEKLOUD

KODEKLOUD

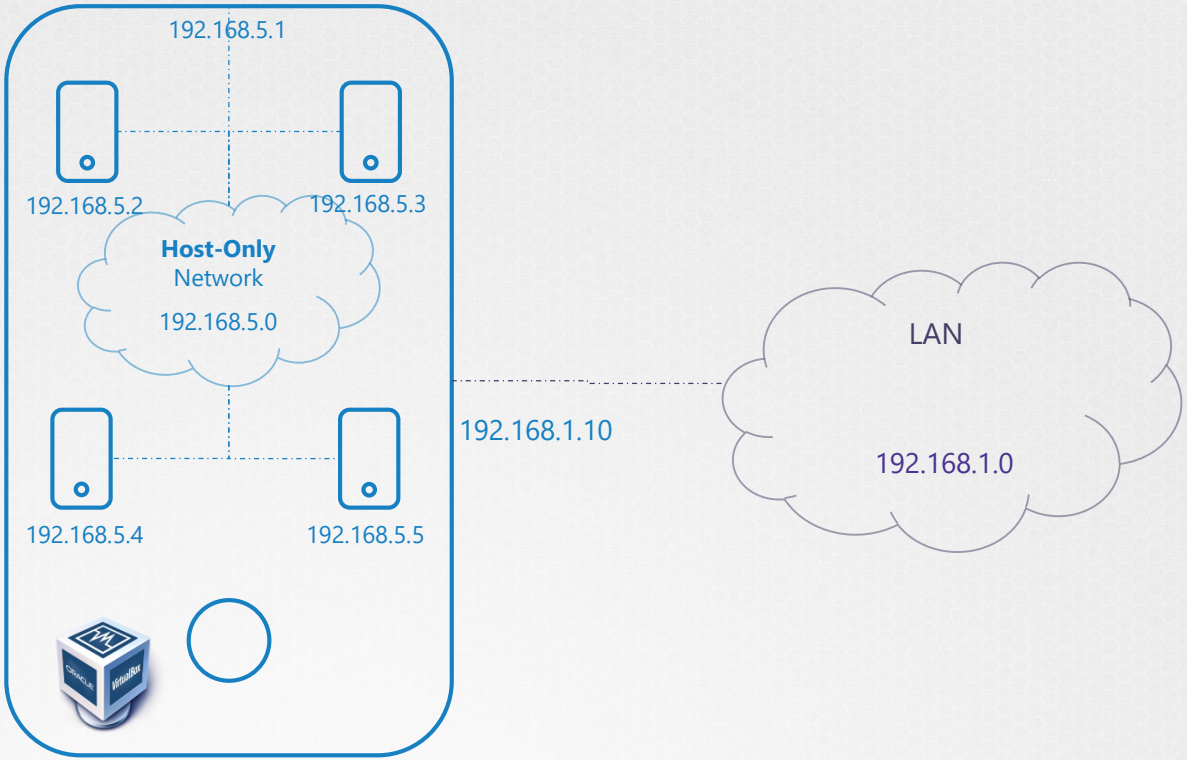
PRE-REQUISITE

NETWORKING VIRTUAL MACHINES



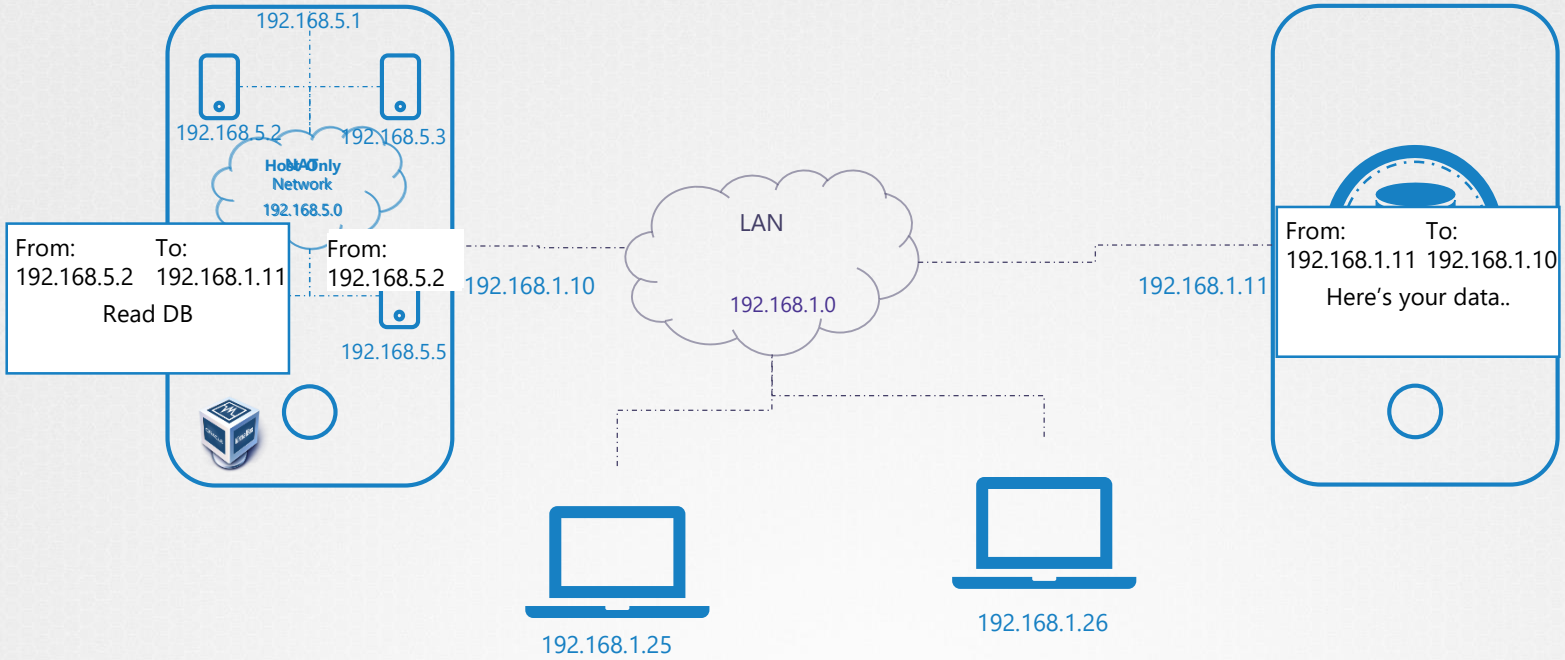
KODEKLOUD

Host Only



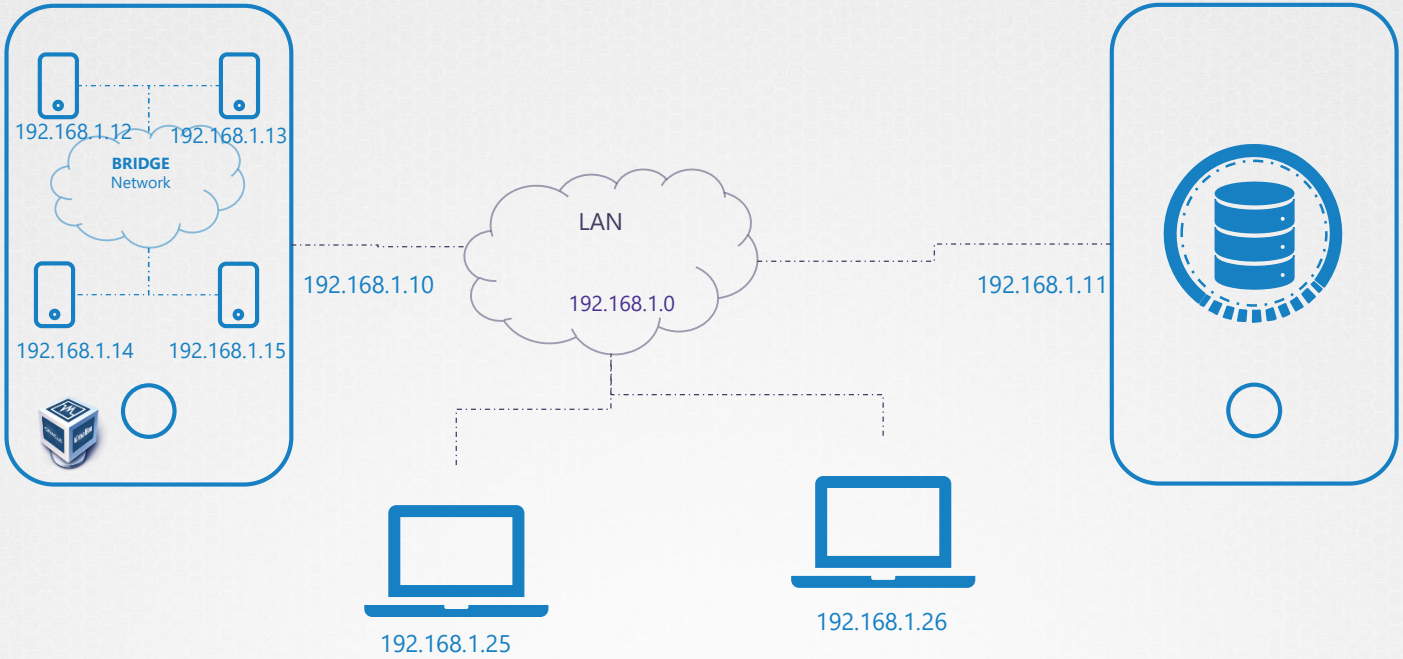
KODEKLOUD

Network Address Translation (NAT)



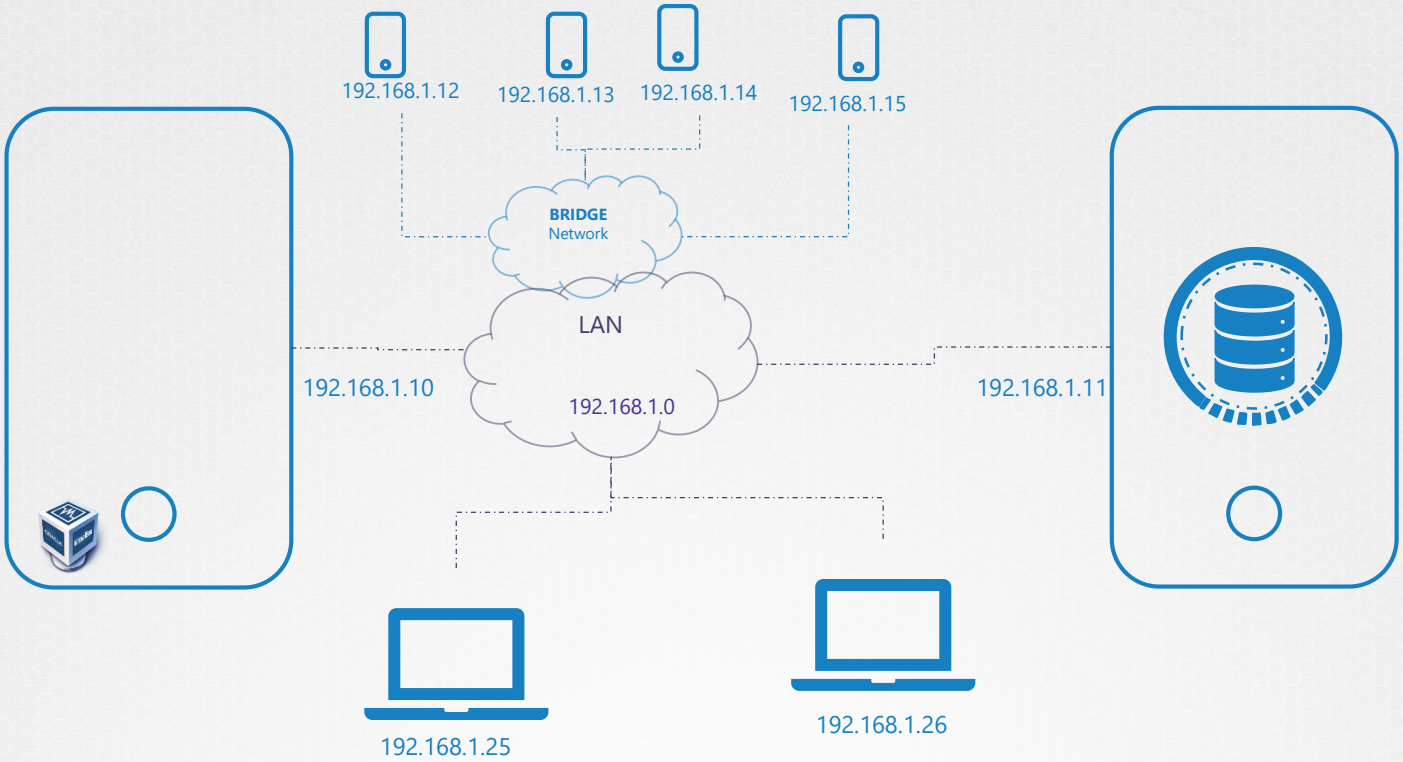
KODEKLOUD

Bridge Network



KODEKLOUD

Bridge Network



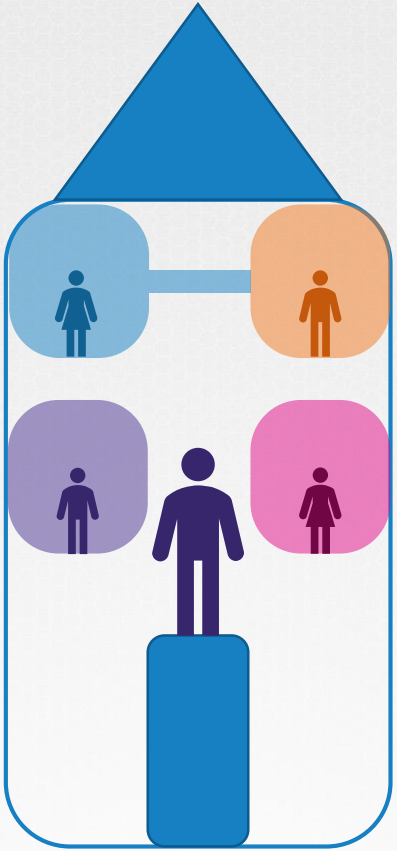
KODEKLOUD

KODEKLOUD

PRE-REQUISITE

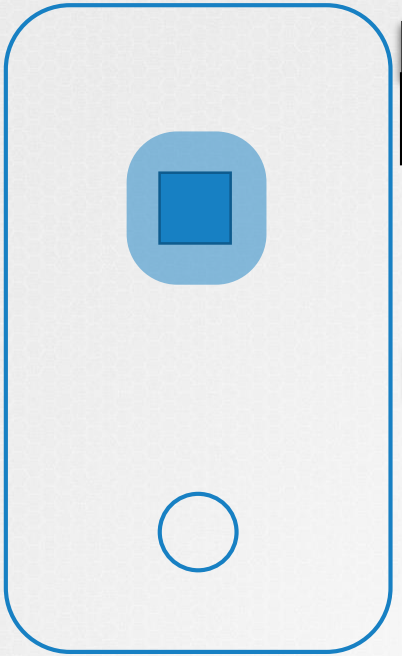
NETWORK NAMESPACES

NAMESPACE



KODEKLOUD

PROCESS NAMESPACE



```
ps aux
```

(On the container)

```
USER        PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root          1  0.0  0.0   4528   828 ?        Ss   03:06   0:00 nginx
```

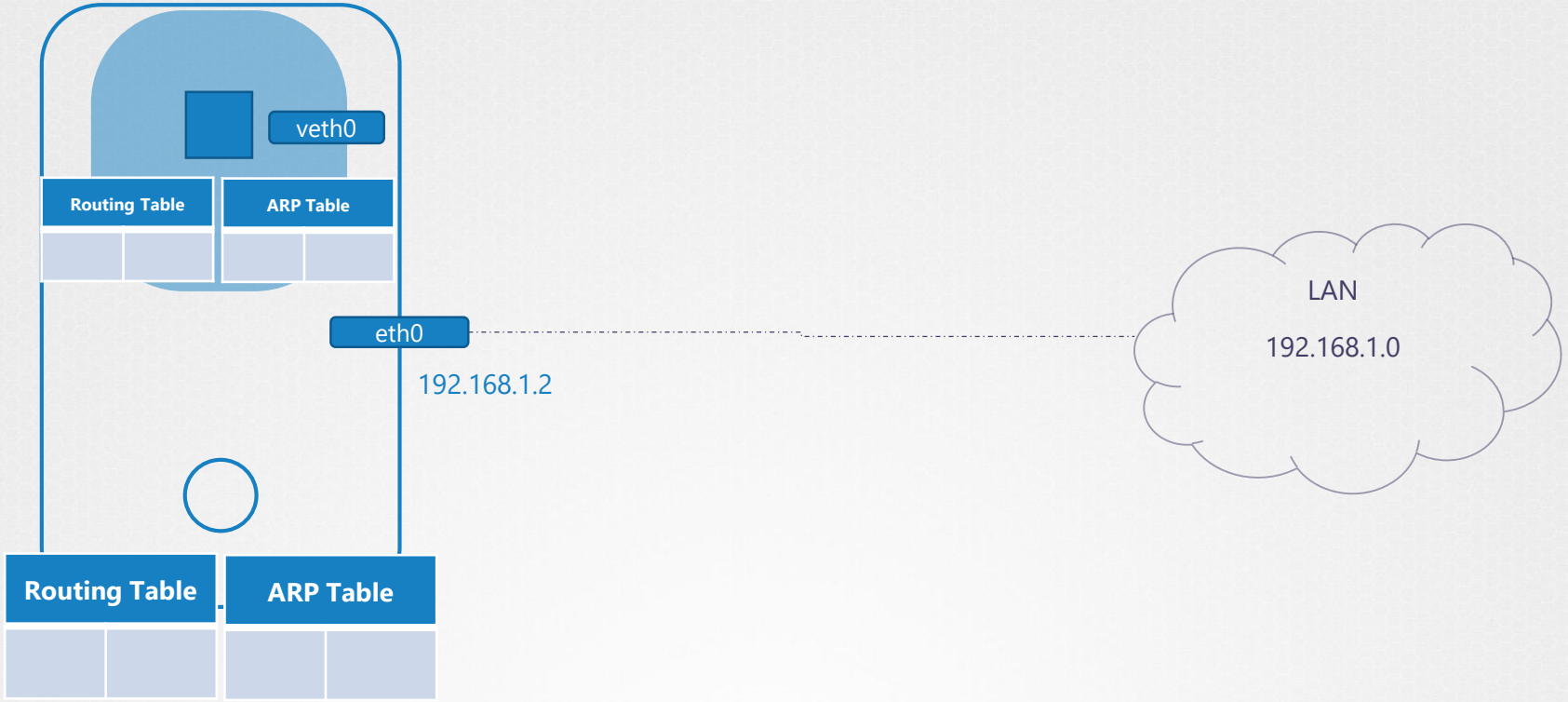
```
ps aux
```

(On the host)

```
USER        PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
project    3720  0.1  0.1  95500  4916 ?        R    06:06   0:00 sshd: project@pts/0
project    3725  0.0  0.1  95196  4132 ?        S    06:06   0:00 sshd: project@notty
project    3727  0.2  0.1  21352  5340 pts/0    Ss   06:06   0:00 -bash
root       3802  0.0  0.0   8924  3616 ?        Sl   06:06   0:00 docker-containerd-
shim -namespace m
root       3816  1.0  0.0   4528   828 ?        Ss   06:06   0:00 nginx
```

KODEKLOUD

NETWORK NAMESPACE



KODEKLOUD

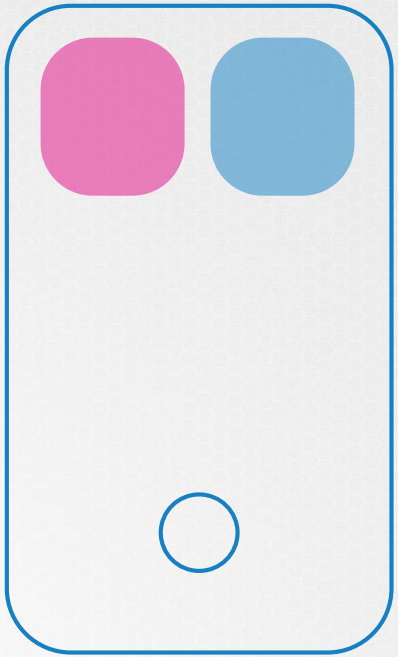
CREATE NETWORK NS

```
ip netns add red
```

```
ip netns add blue
```

```
ip netns
```

```
red
blue
```



KODEKLOUD

EXEC IN NETWORK NS

```
ip link
```

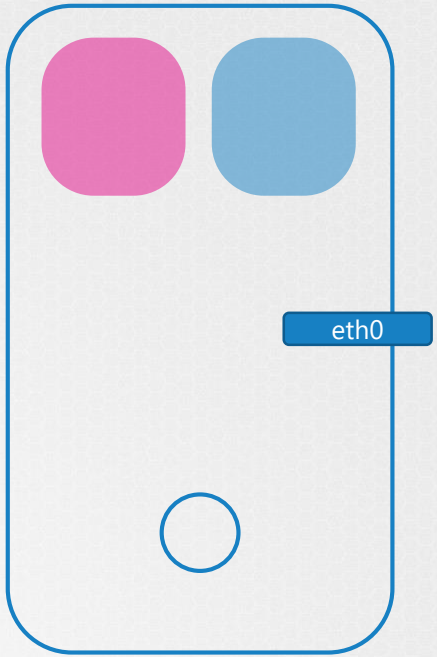
```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc state UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc state UP mode DEFAULT qlen 1000
    link/ether 02:42:ac:11:00:08 brd ff:ff:ff:ff:ff:ff
```

```
ip netns exec red
```

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc state UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
```

```
ip -n red
```

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc state UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
```




```
ip netns exec red ip link
          =
ip    -n red         link
```

KODEKLOUD

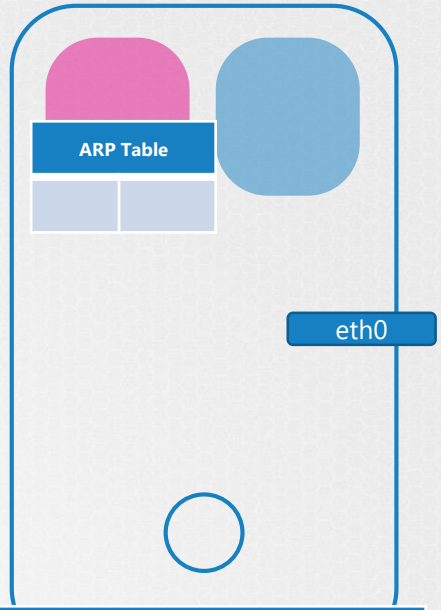
EXEC IN NETWORK NS

```
arp
```

```
Address                  HWtype  HWaddress           Flags Mask            Iface
172.17.0.21              ether   02:42:ac:11:00:15   C                     eth0
172.16.0.8               ether   06:fe:d3:b5:59:65   C                     eth0
_gateway                 ether   02:42:d5:7a:84:8e   C                     eth0
host01                   ether   02:42:ac:11:00:1c   C                     eth0
```

```
ip netns exec red arp
```

```
Address                  HWtype  HWaddress           Flags Mask            Iface
```



ARP Table	
172.17.0.21	02:42:ac:11:00:15
172.16.0.8	06:fe:d3:b5:59:65

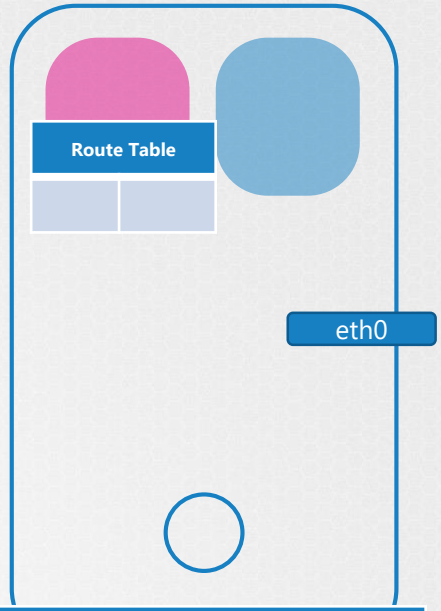
EXEC IN NETWORK NS

```
route
```

```
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
default         _gateway        0.0.0.0         UG    202    0        0 eth0
172.17.0.0      0.0.0.0         255.255.0.0     U     202    0        0 eth0
172.17.0.0      0.0.0.0         255.255.255.0   U     0      0        0 docker0
```

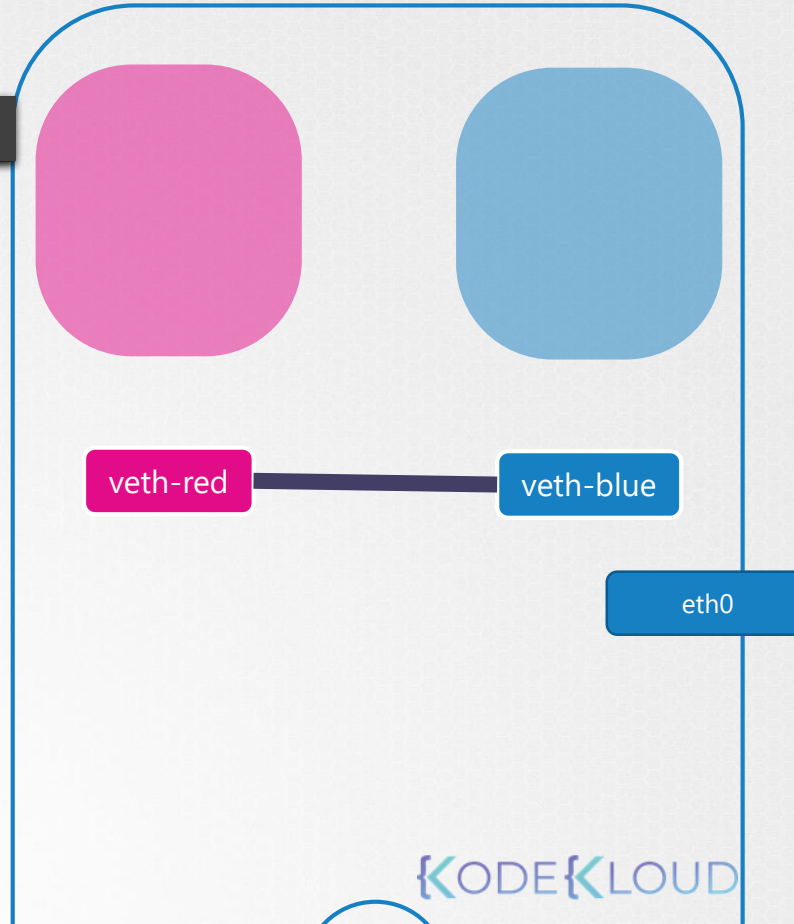
```
ip netns exec red route
```

```
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
```



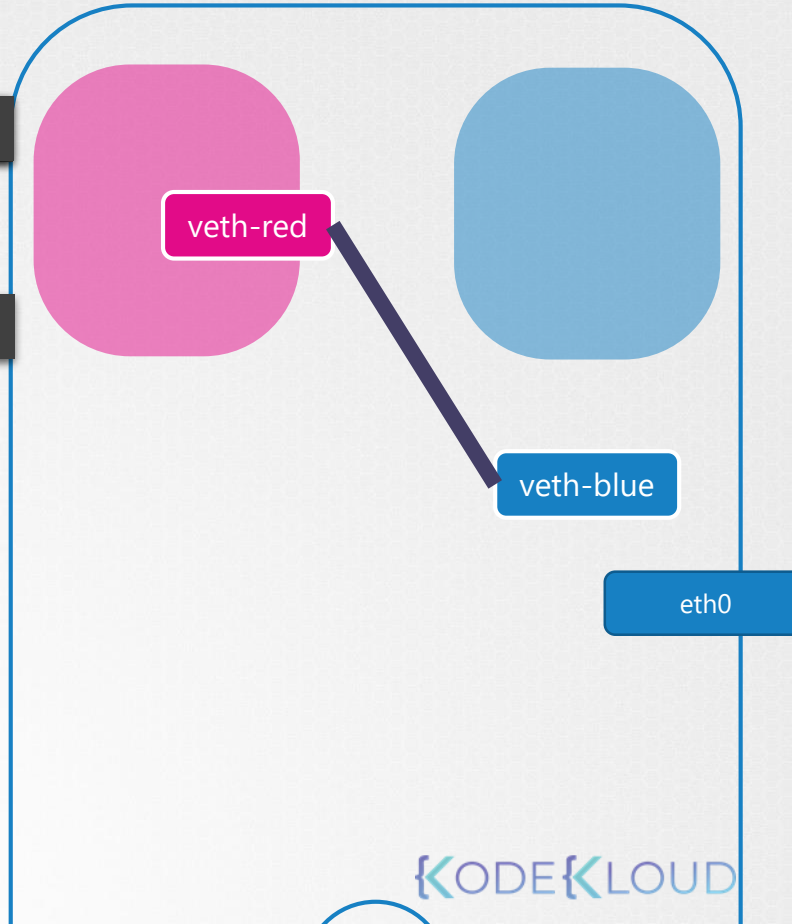
Route Table	
172.17.0.0	0.0.0.0
17.18.0.0	0.0.0.0

```
ip link add veth-red type veth peer name veth-blue
```



```
ip link add veth-red type veth peer name veth-blue
```

```
ip link set veth-red netns red
```

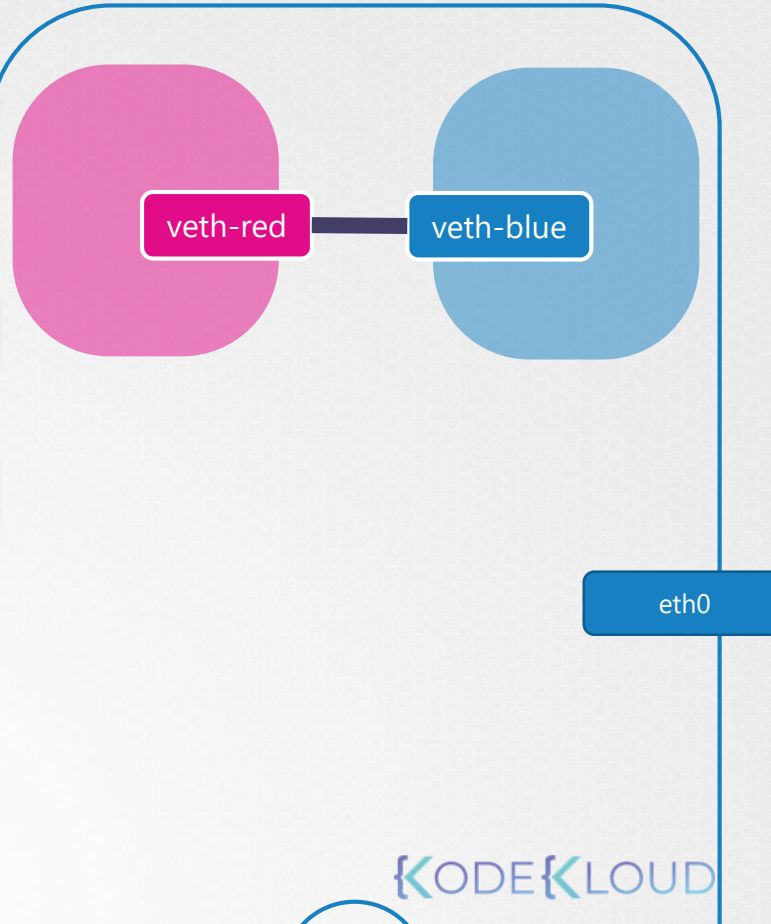


KODEKLOUD

```
ip link add veth-red type veth peer name veth-blue
```

```
ip link set veth-red netns red
```

```
ip link set veth-blue netns blue
```



KODEKLOUD

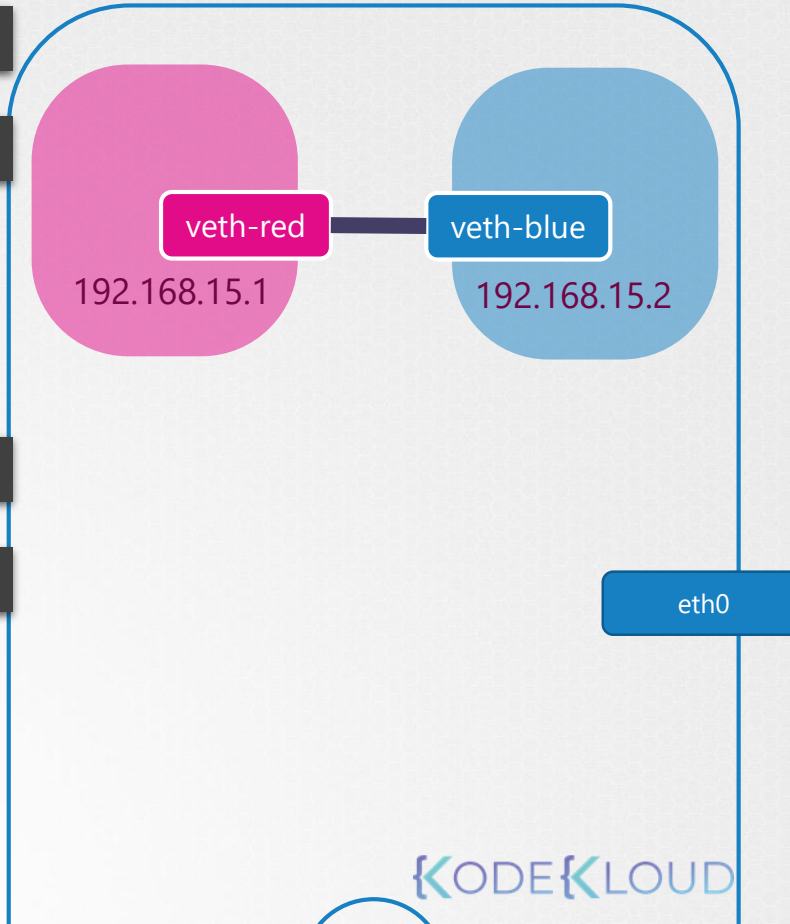
```
ip link add veth-red type veth peer name veth-blue
```

```
ip link set veth-red netns red
```

```
ip link set veth-blue netns blue
```

```
ip -n red  addr add 192.168.15.1 dev veth-red
```

```
ip -n blue addr add 192.168.15.2 dev veth-blue
```



KODEKLOUD

```
ip link add veth-red type veth peer name veth-blue
```

```
ip link set veth-red netns red
```

```
ip link set veth-blue netns blue
```

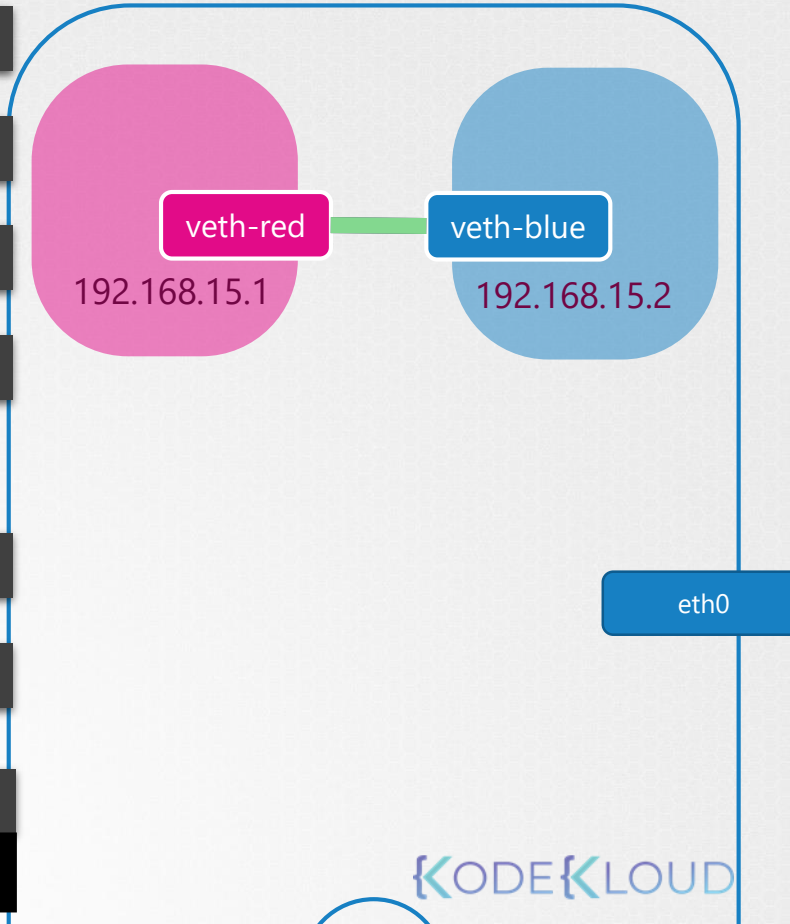
```
ip -n red  addr add 192.168.15.1 dev veth-red
```

```
ip -n blue addr add 192.168.15.2 dev veth-blue
```

```
ip -n red  link set veth-red up
```

```
ip -n blue link set veth-blue up
```

```
ip netns exec red ping 192.168.15.2
PING 192.168.15.2 (192.168.15.2) 56(84) bytes of data.
64 bytes from 192.168.15.2: icmp_seq=1 ttl=64 time=0.026 ms
```



KODEKLOUD

```
ip -n red  link set veth-red up
```

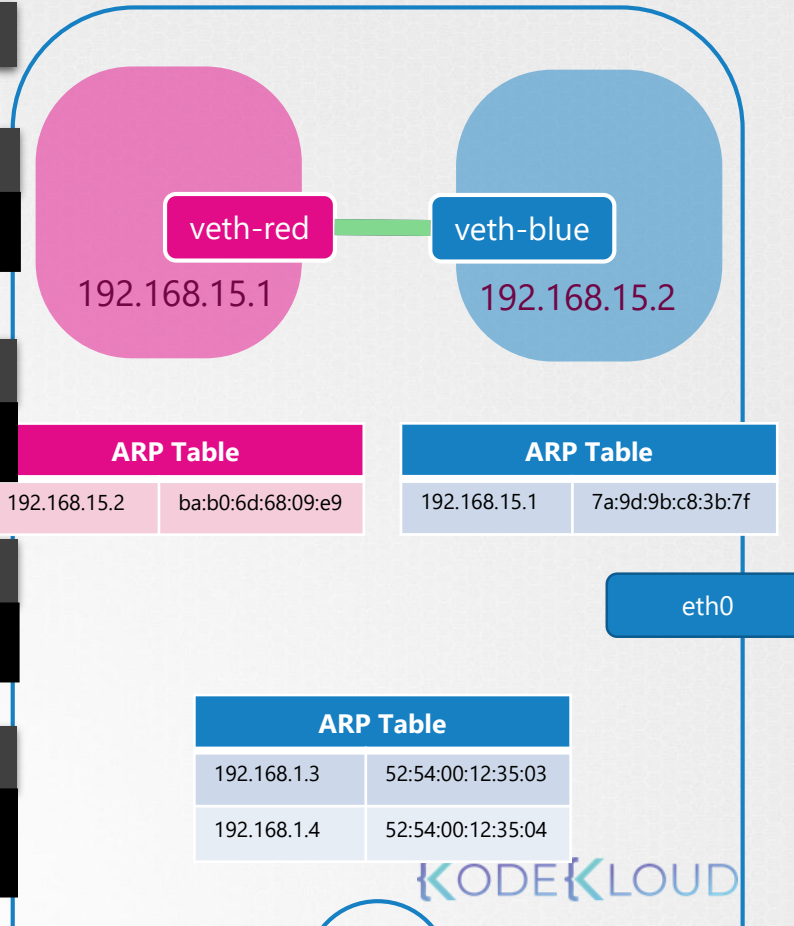
```
ip -n blue link set veth-blue up
```

```
ip netns exec red ping 192.168.15.2
PING 192.168.15.2 (192.168.15.2) 56(84) bytes of data.
64 bytes from 192.168.15.2: icmp_seq=1 ttl=64 time=0.026 ms
```

```
ip netns exec red arp
Address          HWtype  HWaddress           Flags Mask            Iface
192.168.15.2     ether   ba:b0:6d:68:09:e9   C                     veth-red
```

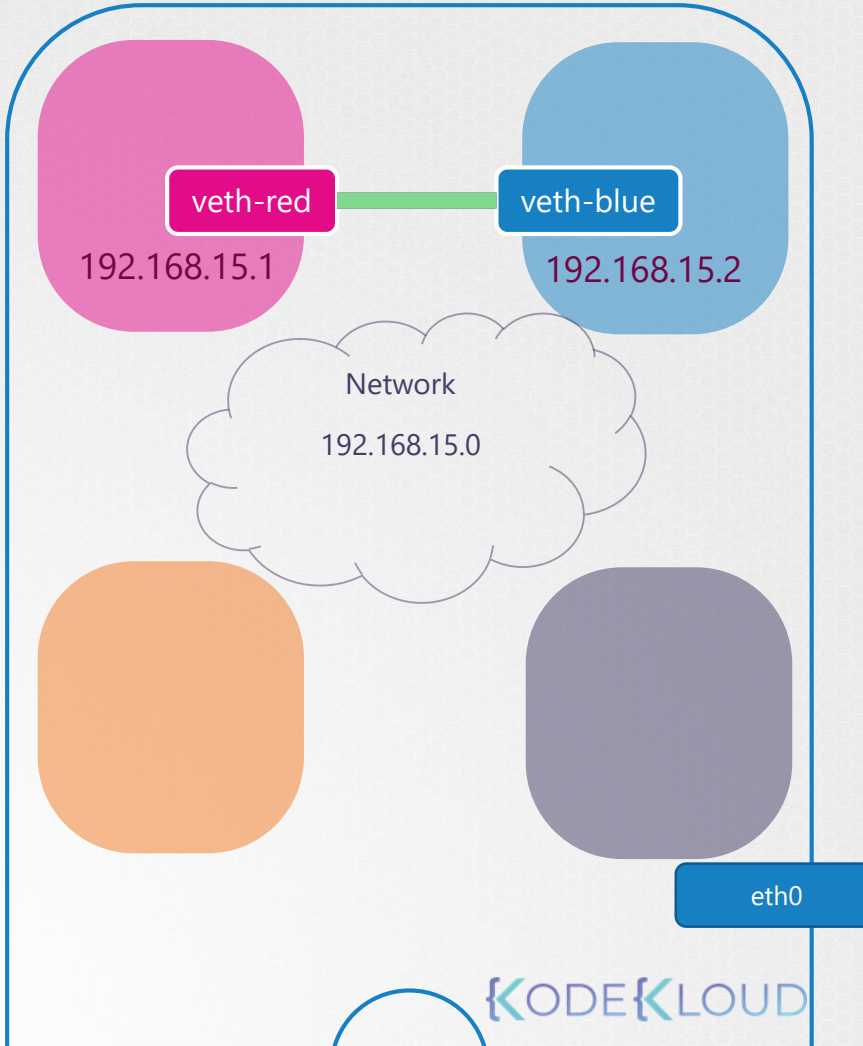
```
ip netns exec blue arp
Address          HWtype  HWaddress           Flags Mask            Iface
192.168.15.1     ether   7a:9d:9b:c8:3b:7f   C                     veth-blue
```

```
arp
Address                  HWtype  HWaddress           Flags Mask            Iface
192.168.1.3              ether   52:54:00:12:35:03   C                     eth0
192.168.1.4              ether   52:54:00:12:35:04   C                     eth0
```



LINUX BRIDGE

OvS
Open vSwitch



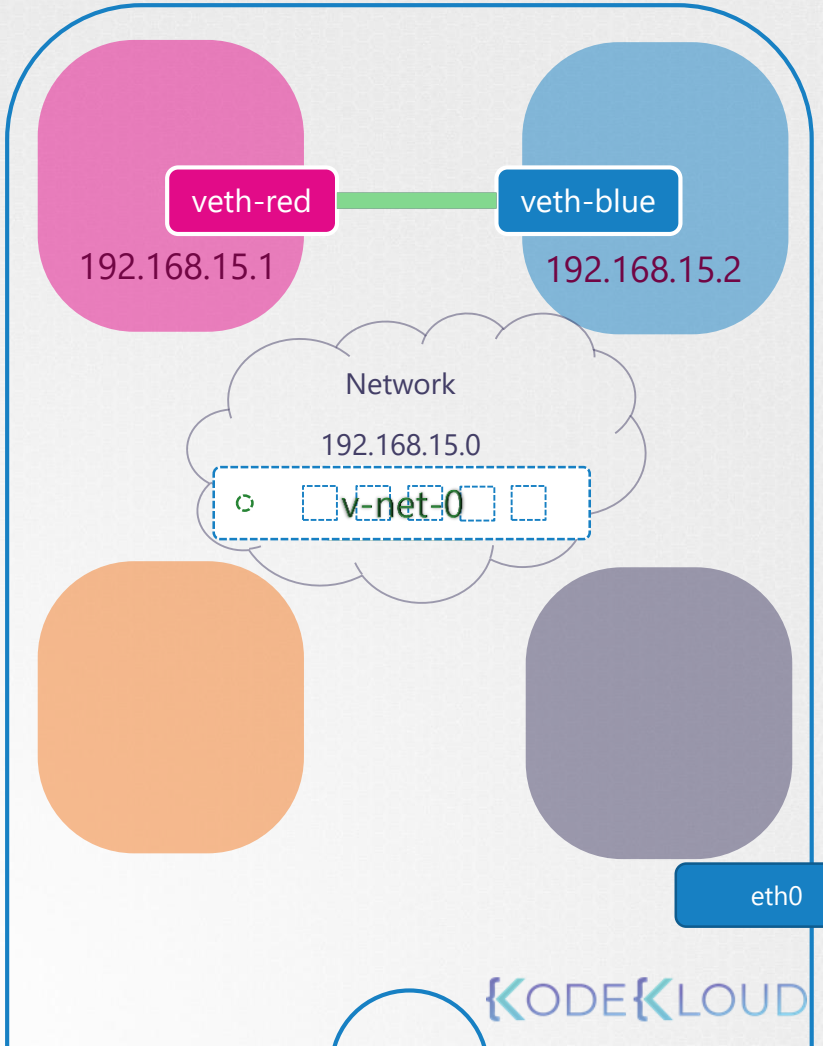
LINUX BRIDGE

```
ip link add v-net-0 type bridge
```

```
ip link
```

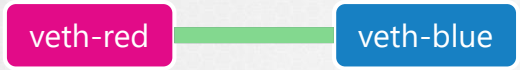
```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state
UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc
fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether 02:0d:31:14:c7:a7 brd ff:ff:ff:ff:ff:ff
6: v-net-0: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN
mode DEFAULT group default qlen 1000
    link/ether 06:9d:69:52:6f:61 brd ff:ff:ff:ff:ff:ff
```

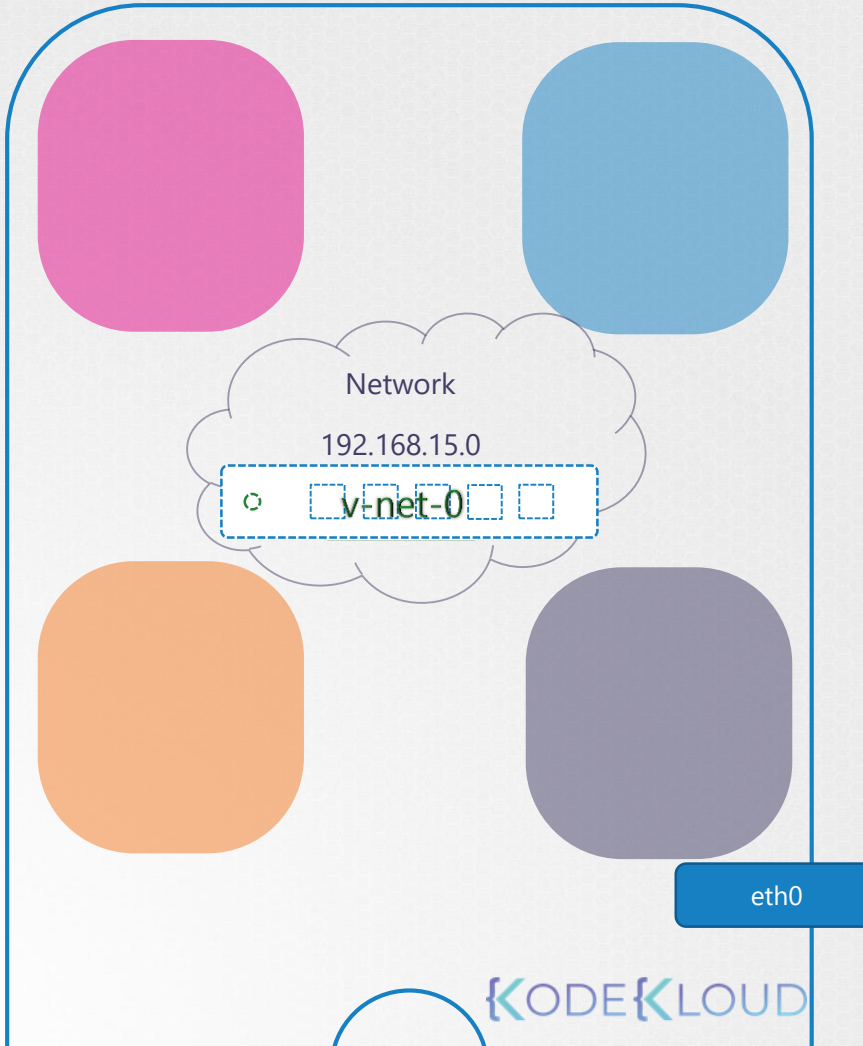
```
ip link set dev v-net-0 up
```



LINUX BRIDGE

```
ip -n red link del veth-red
```

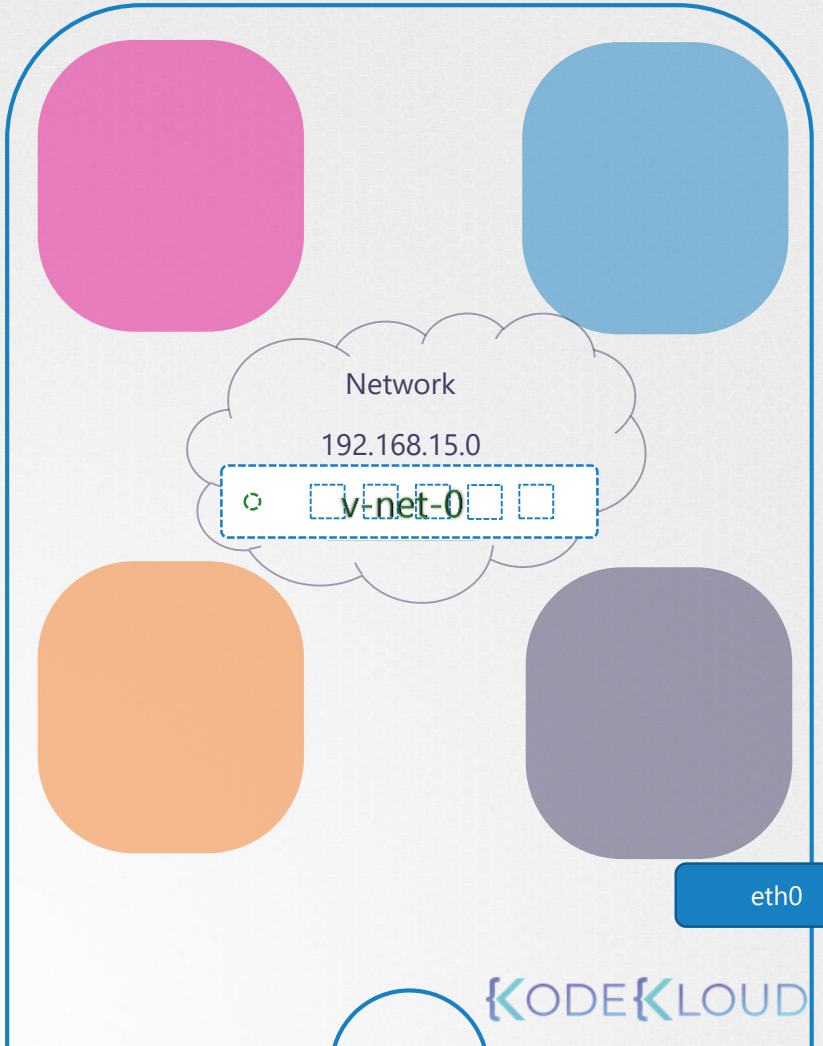




LINUX BRIDGE

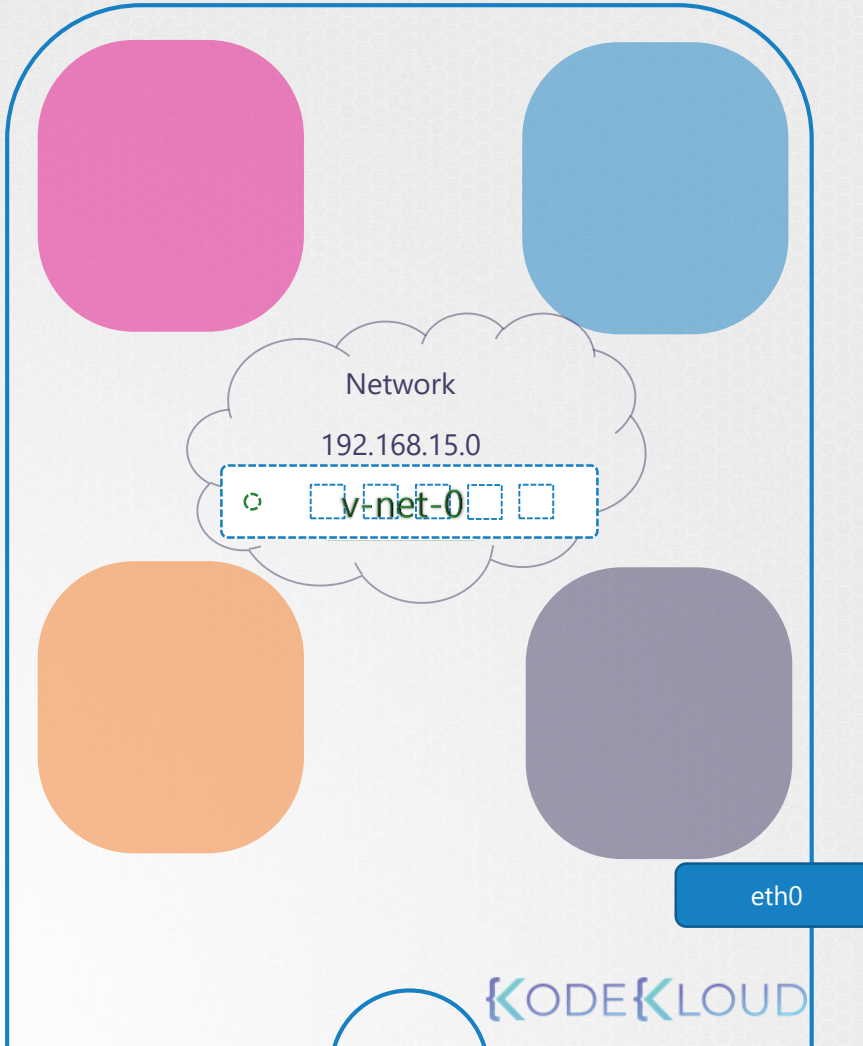
```
ip link add veth-red type veth peer name veth-red-br
```

```
ip link add veth-blue type veth peer name veth-blue-br
```



LINUX BRIDGE

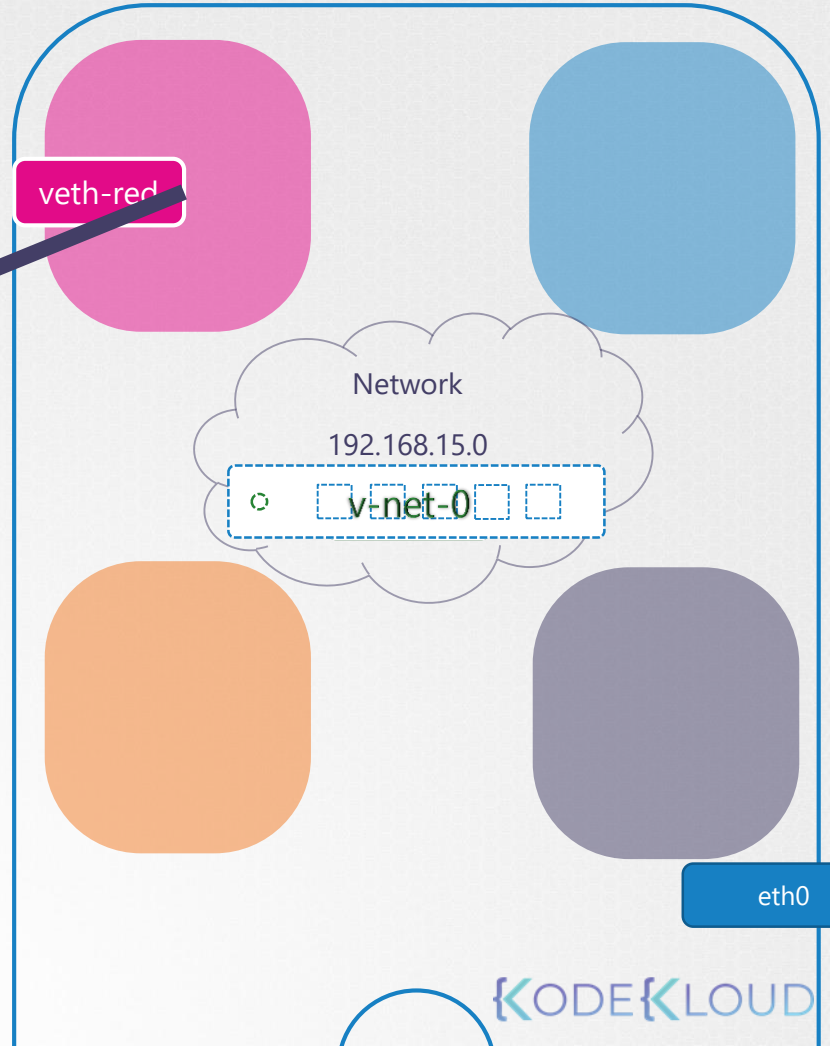
```
ip link set veth-red netns red
```



KODEKLOUD

LINUX BRIDGE

```
ip link set veth-red netns red
ip link set veth-red-br master v-net-0
```



veth-red-br

veth-blue — veth-blue-br

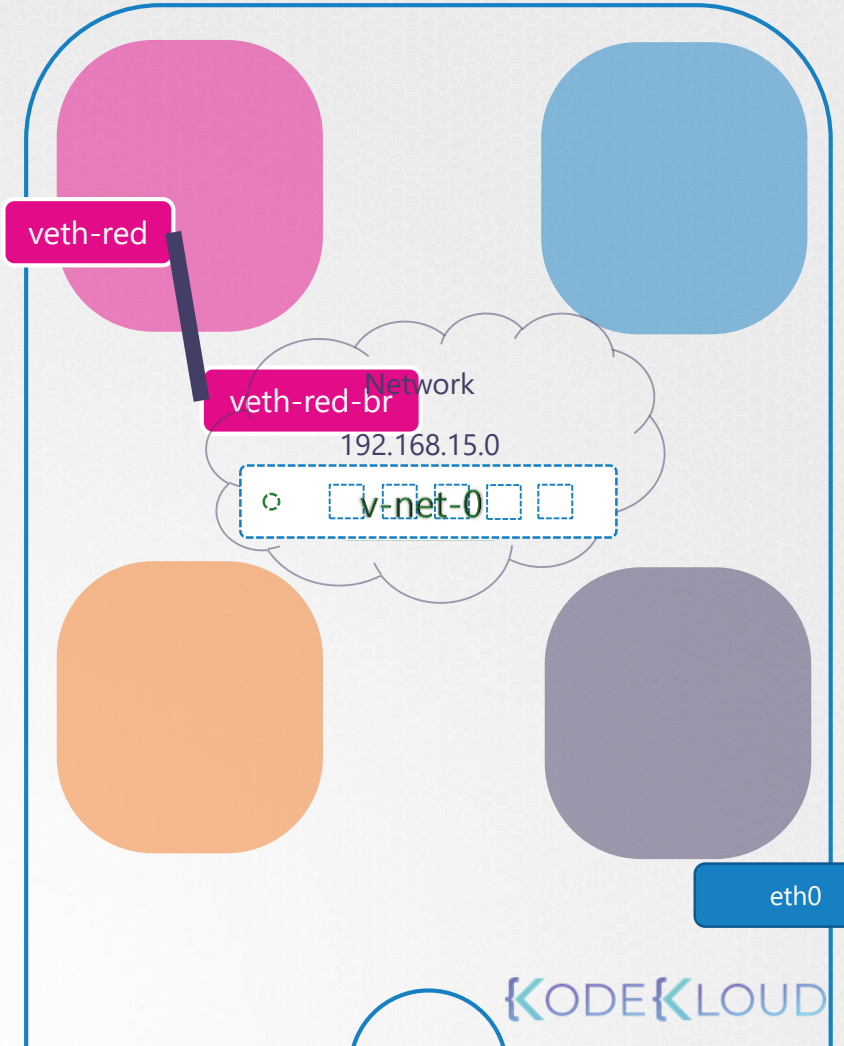
KODEKLOUD

LINUX BRIDGE

```
ip link set veth-red netns red
```

```
ip link set veth-red-br master v-net-0
```

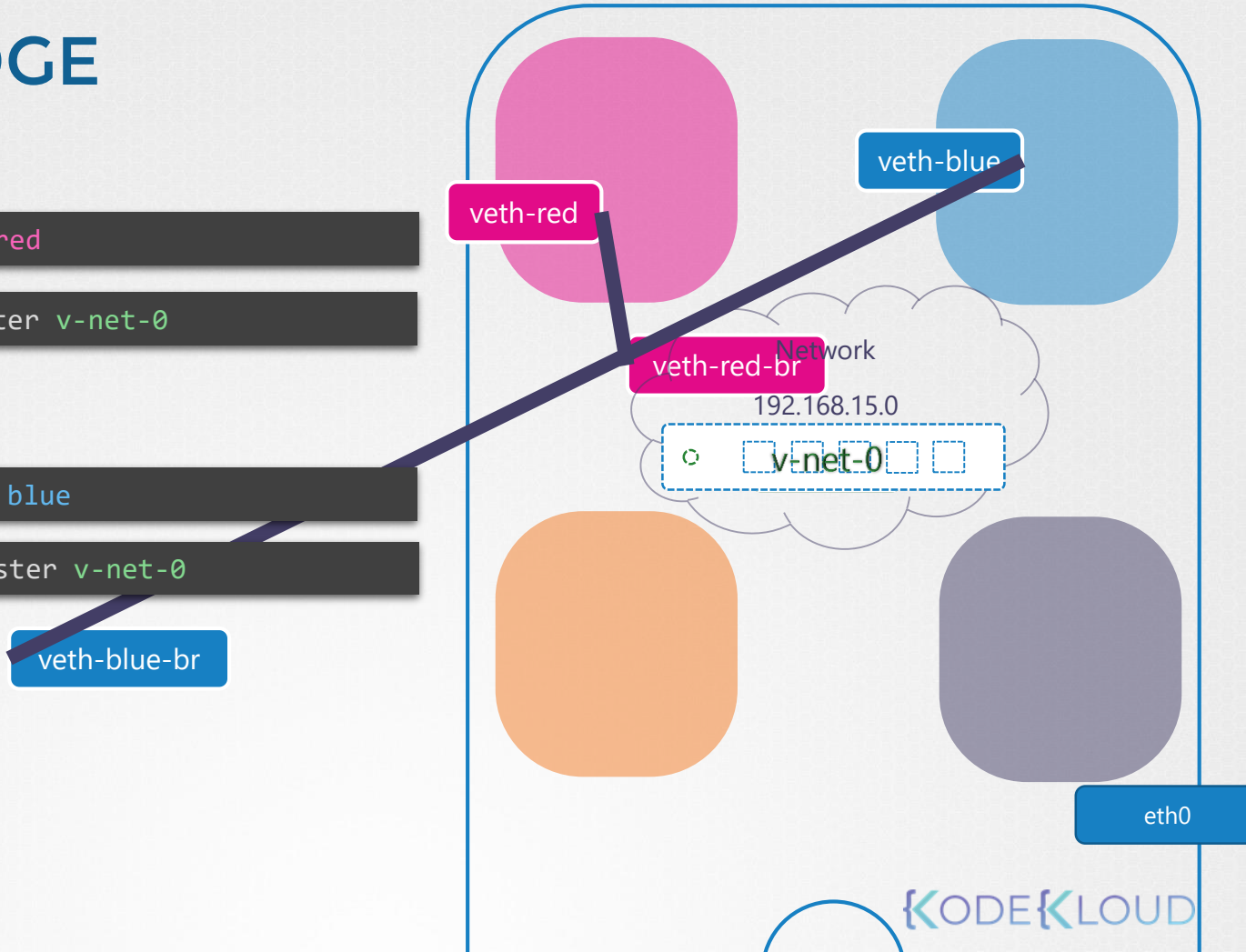
```
ip link set veth-blue netns blue
```



KODEKLOUD

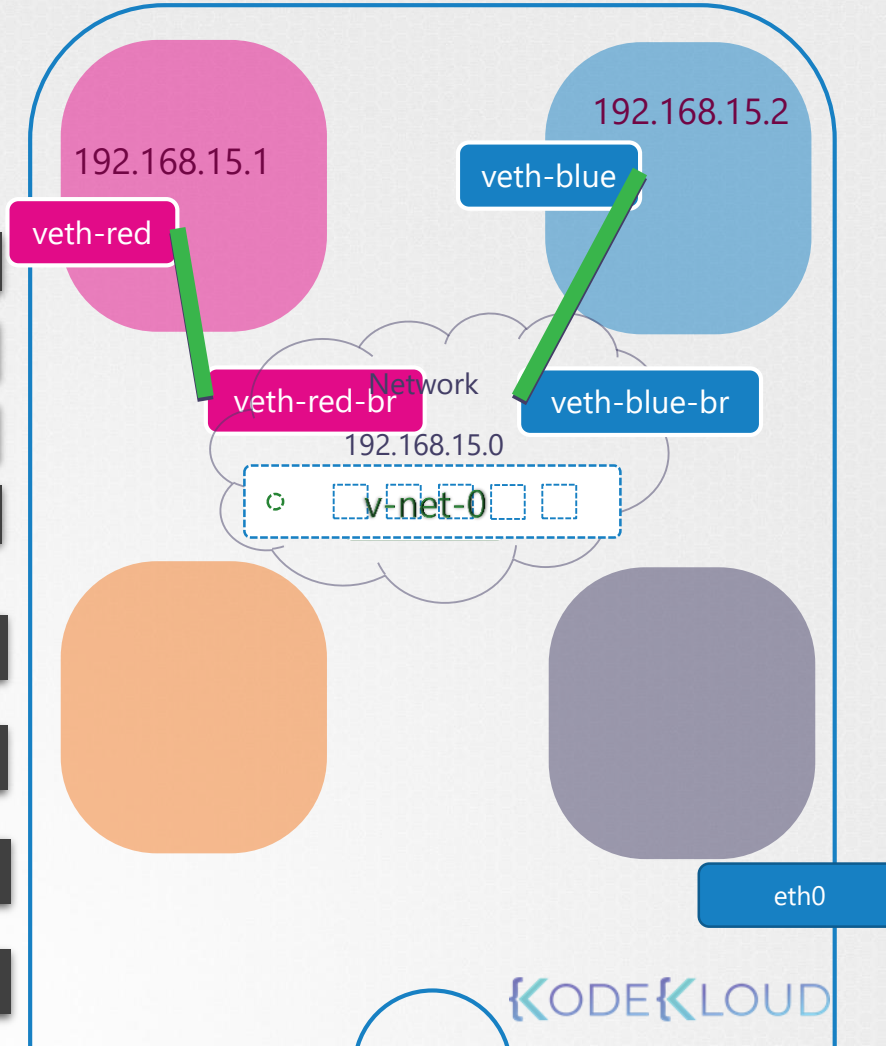
LINUX BRIDGE

```
ip link set veth-red netns red
ip link set veth-red-br master v-net-0
ip link set veth-blue netns blue
ip link set veth-blue-br master v-net-0
```



LINUX BRIDGE

```
ip link set veth-red netns red
ip link set veth-red-br master v-net-0
ip link set veth-blue netns blue
ip link set veth-blue-br master v-net-0

ip -n red addr add 192.168.15.1 dev veth-red
ip -n blue addr add 192.168.15.2 dev veth-blue
ip -n red link set veth-red up
ip -n blue link set veth-blue up
```



LINUX BRIDGE

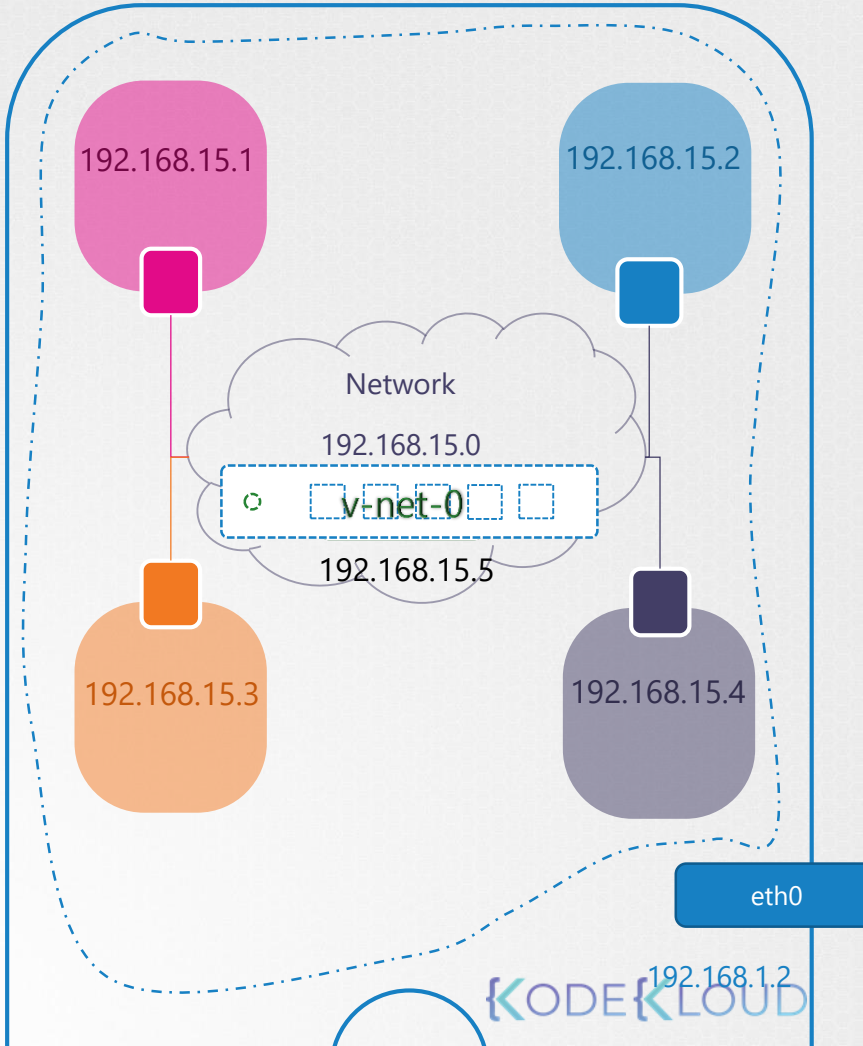
```
ping 192.168.15.1
```

```
Not Reachable!
```

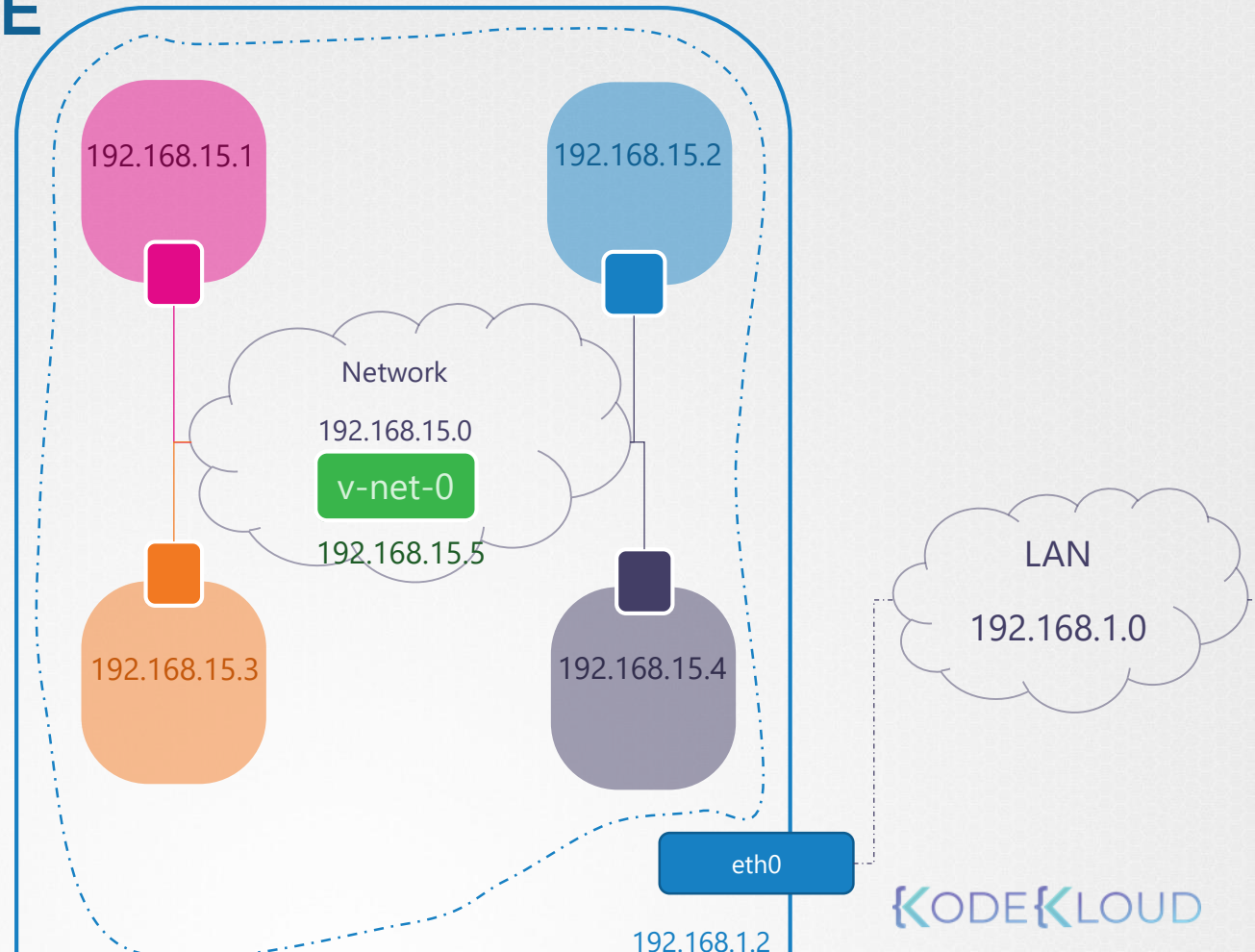
```
ip addr add 192.168.15.5/24 dev v-net-0
```

```
ping 192.168.15.1
```

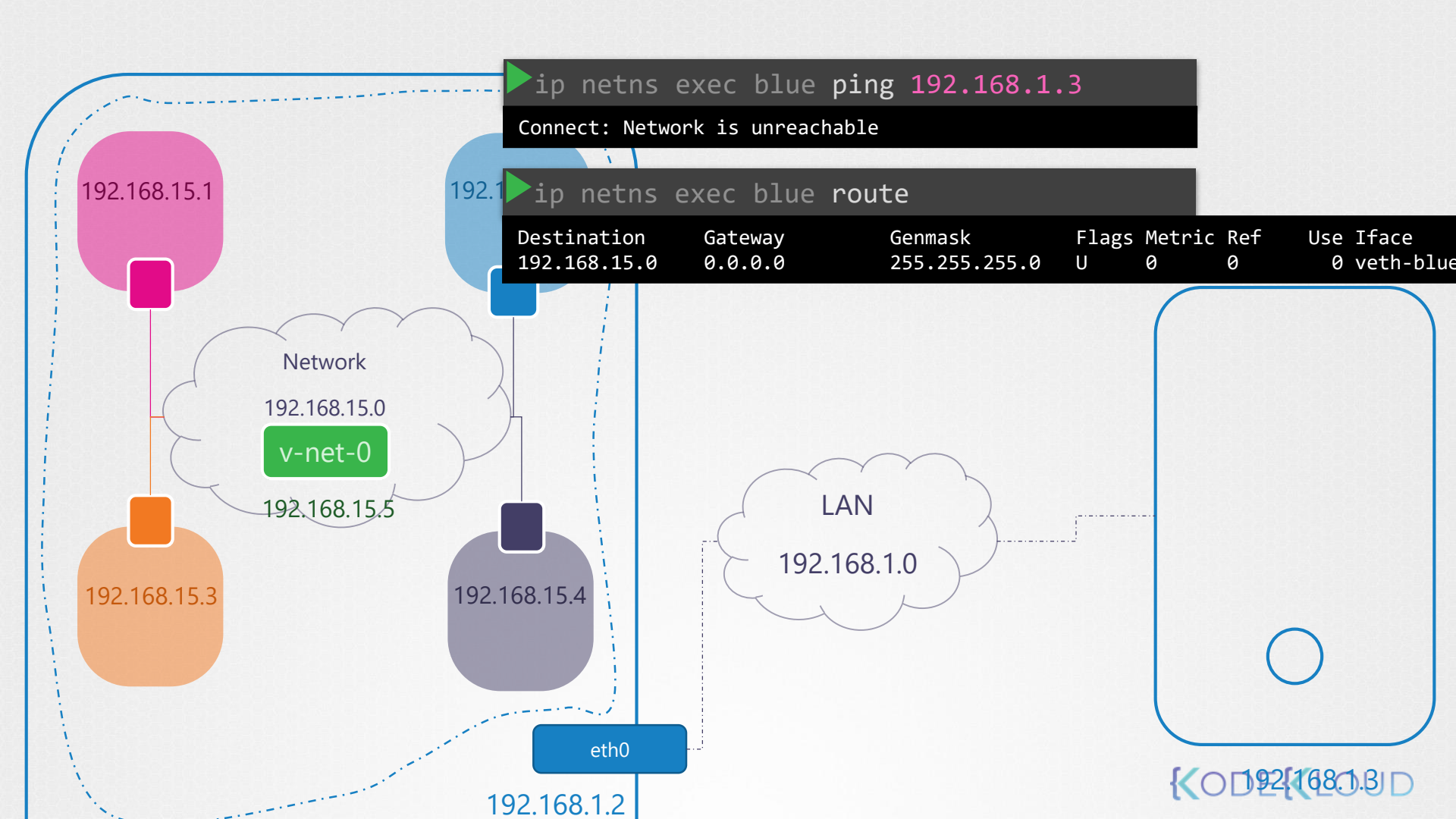
```
PING 192.168.15.1 (192.168.15.1) 56(84) bytes of data.
64 bytes from 192.168.15.1: icmp_seq=1 ttl=64 time=0.026 ms
```



LINUX BRIDGE



KODEKLOUD

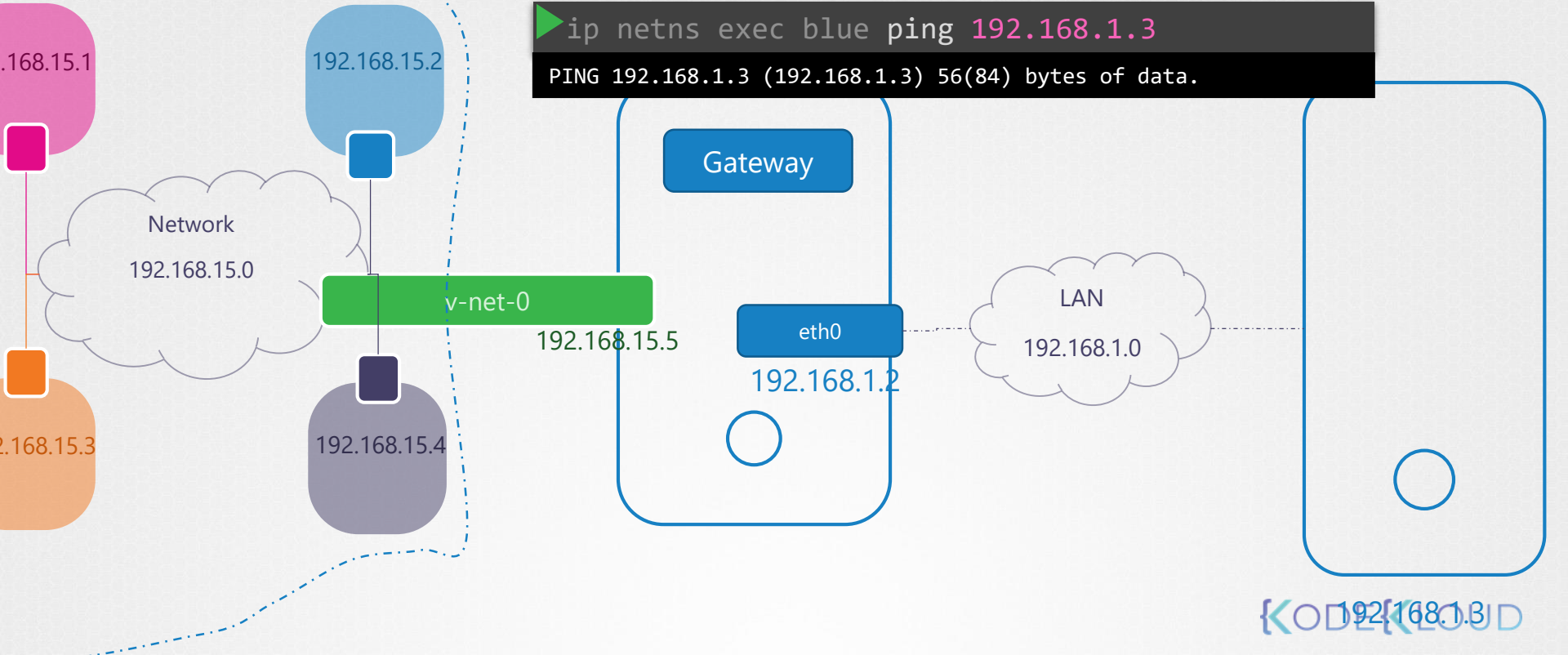

ip netns exec blue ping 192.168.1.3
Connect: Network is unreachable
ip netns exec blue route
Destination Gateway Genmask Flags Metric Ref Use Iface
192.168.15.0 0.0.0.0 255.255.255.0 U 0 0 0 veth-blue
192.168.15.1
192.1
Network
192.168.15.0
v-net-0
192.168.15.5
192.168.15.3
192.168.15.4
LAN
192.168.1.0
eth0
192.168.1.2
192.168.1.3
KODEKLOUD

```
ip netns exec blue ip route add 192.168.1.0/24 via 192.168.15.5
```

```
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
192.168.15.0    0.0.0.0         255.255.255.0   U     0      0        0 veth-blue
192.168.1.0     192.168.15.5    255.255.255.0   UG    0      0        0 veth-blue
```

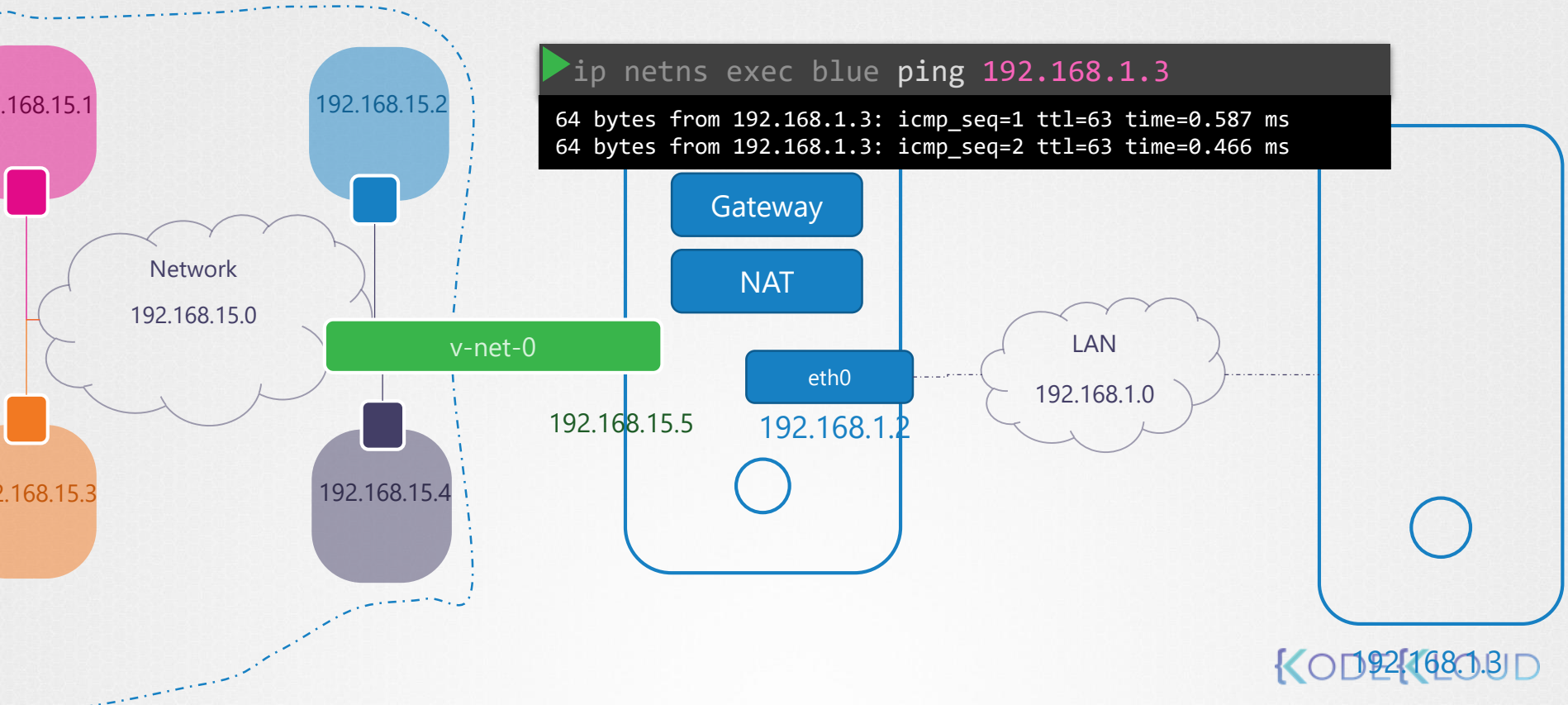
```
ip netns exec blue ping 192.168.1.3
```

```
PING 192.168.1.3 (192.168.1.3) 56(84) bytes of data.
```



KODEKLOUD

```
iptables -t nat -A POSTROUTING -s 192.168.15.0/24 -j MASQUERADE
```

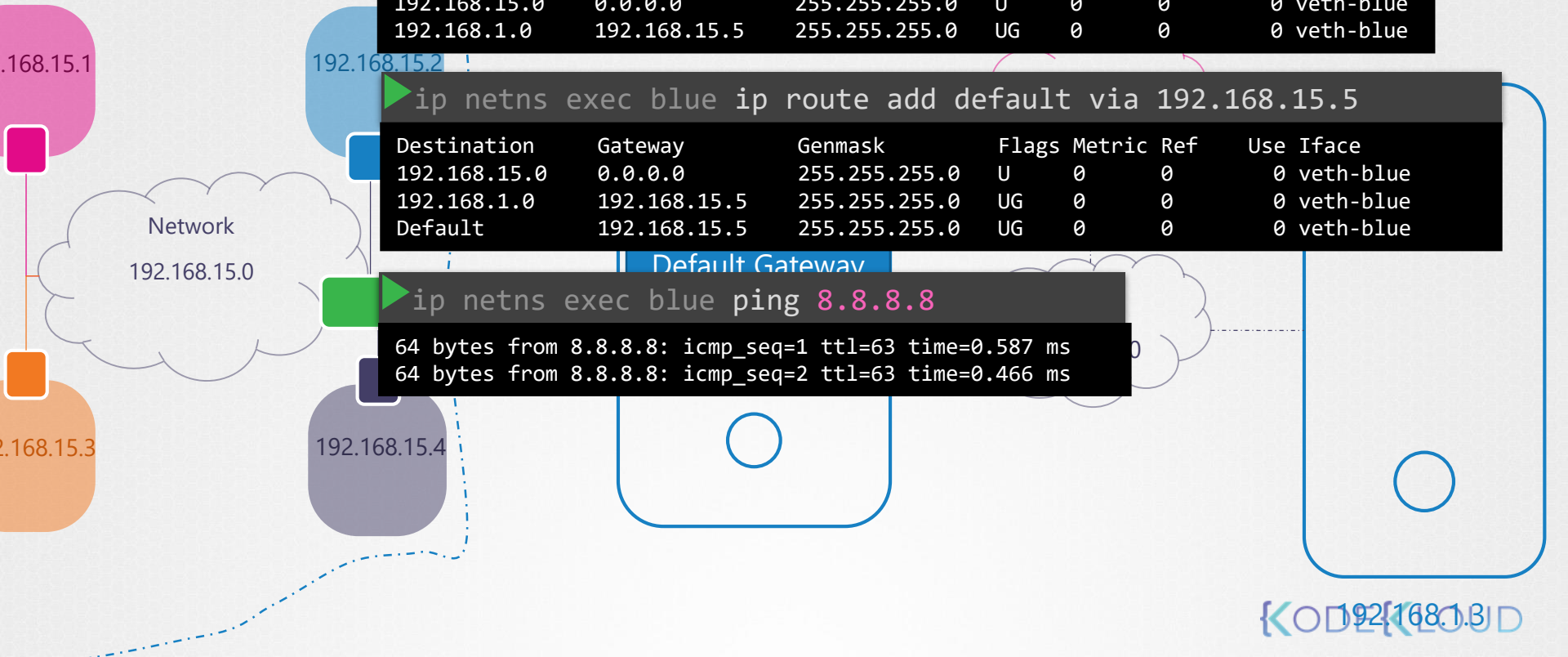


```
ip netns exec blue ping 8.8.8.8
Connect: Network is unreachable
```

```
ip netns exec blue route
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
192.168.15.0    0.0.0.0         255.255.255.0   U     0      0        0 veth-blue
192.168.1.0     192.168.15.5    255.255.255.0   UG    0      0        0 veth-blue
```

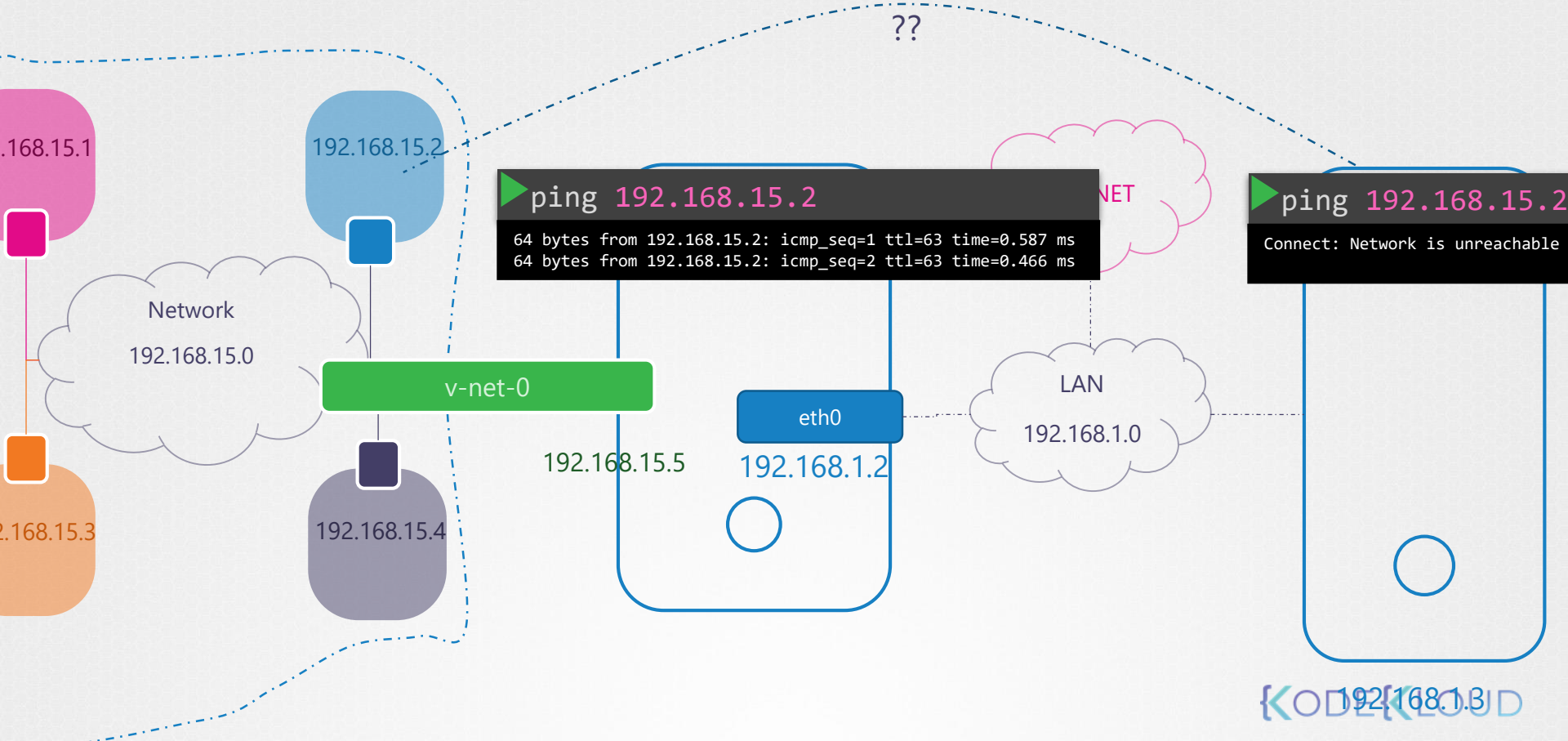
```
ip netns exec blue ip route add default via 192.168.15.5
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
192.168.15.0    0.0.0.0         255.255.255.0   U     0      0        0 veth-blue
192.168.1.0     192.168.15.5    255.255.255.0   UG    0      0        0 veth-blue
Default         192.168.15.5    255.255.255.0   UG    0      0        0 veth-blue
```

```
ip netns exec blue ping 8.8.8.8
64 bytes from 8.8.8.8: icmp_seq=1 ttl=63 time=0.587 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=63 time=0.466 ms
```



KODEKLOUD

```
iptables -t nat -A PREROUTING --dport 80 --to-destination 192.168.15.2:80 -j DNAT
```



KODEKLOUD

```
# Create network namespaces
ip netns add red
ip netns add blue

# Create veth pairs
ip link add veth-red type veth peer name veth-blue

# Create Add veth to respective namespaces
ip link set veth-red netns red
ip link set veth-blue netns blue

# Set IP Addresses
ip -n red addr add 192.168.1.1 dev veth-red
ip -n blue addr add 192.168.1.2 dev veth-blue

# Check IP Addresses
ip -n red addr
ip -n blue addr

# Bring up interfaces
ip -n red link set veth-red up
ip -n blue link set veth-blue up

# Bring Loopback devices up
ip -n red link set lo up
ip -n blue link set lo up

# Add default gateway
ip netns exec red ip route add default via 192.168.1.1 dev
veth-red
ip netns exec blue ip route add default via 192.168.1.2 dev
```

KODEKLOUD

```
ip netns del red
ip netns del blue

ip link del v-net-0

iptables -t nat -D POSTROUTING 1

#


ip netns add red
ip netns add blue

ip link add veth-red type veth peer name veth-red-br
ip link add veth-blue type veth peer name veth-blue-br

ip link set veth-red netns red
ip link set veth-blue netns blue

ip -n red addr add 192.168.15.2/24 dev veth-red

ip -n blue addr add 192.168.15.3/24 dev veth-blue


brctl addbr v-net-0

ip link set dev v-net-0 up

ip link set veth-red-br up
```

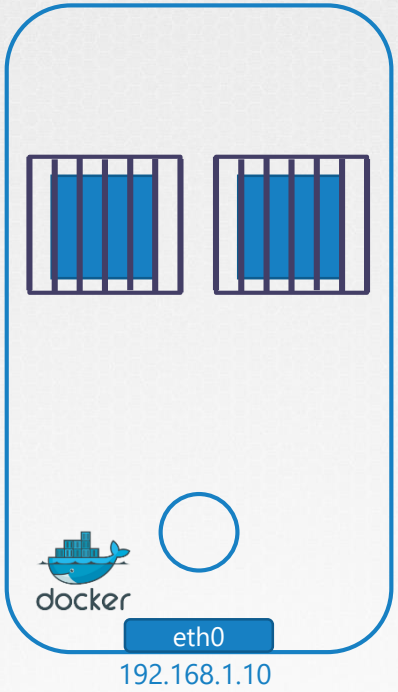
KODEKLOUD

KODEKLOUD

PRE-REQUISITE

DOCKER NETWORKING

NONE



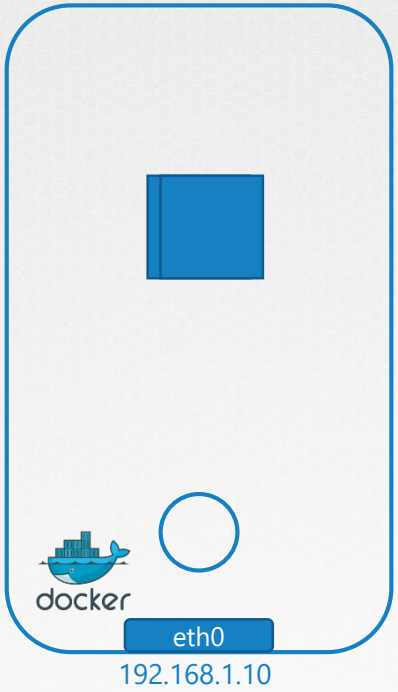

```
docker run --network none nginx
```

```
docker run --network none nginx
```

KODEKLOUD

HOST

```
http://192.168.1.10:80
```

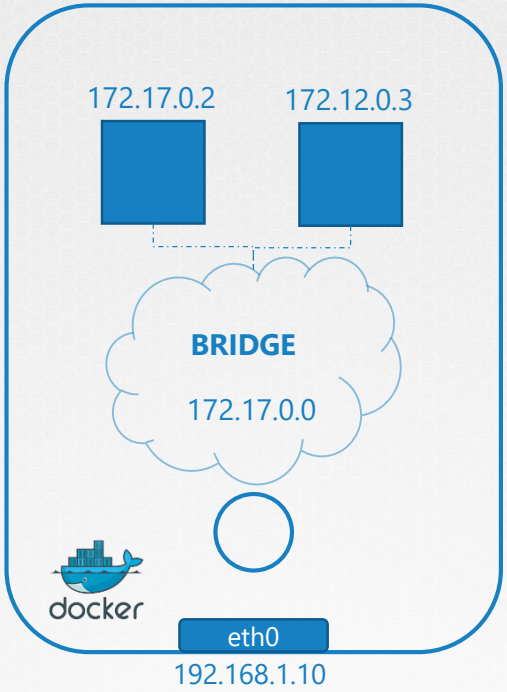



```
docker run --network host nginx
```

```
docker run --network host nginx
```

KODEKLOUD

BRIDGE




```
docker run nginx
```

```
docker run nginx
```

KODEKLOUD

KODEKLOUD

DOCKER NETWORKING Deep Dive

BRIDGE

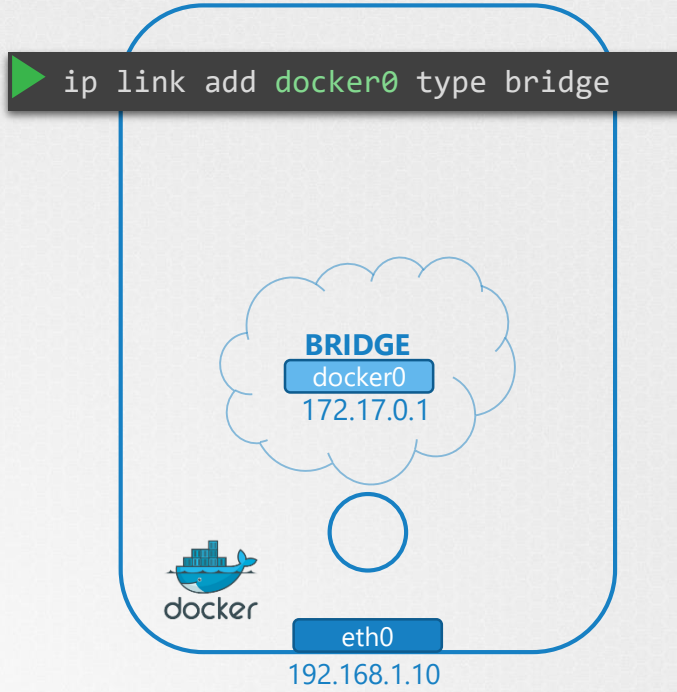
```
docker network ls
```

```
NETWORK ID          NAME                DRIVER              SCOPE
2b60087261b2        bridge              bridge              local
0beb4870b093        host                host                local
99035e02694f        none                null                local
```

```
ip link
```

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc
fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether 02:42:ac:11:00:08 brd ff:ff:ff:ff:ff:ff
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc
noqueue state DOWN mode DEFAULT group default
    link/ether 02:42:88:56:50:83 brd ff:ff:ff:ff:ff:ff
```

```
ip link add docker0 type bridge
```



KODEKLOUD

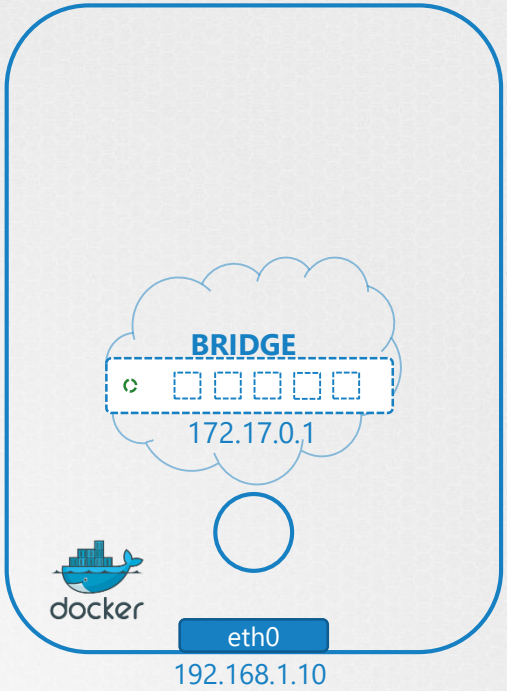
BRIDGE

ip link

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc
fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether 02:42:ac:11:00:08 brd ff:ff:ff:ff:ff:ff
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc
noqueue state DOWN mode DEFAULT group default
    link/ether 02:42:88:56:50:83 brd ff:ff:ff:ff:ff:ff
```

ip addr

```
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc
noqueue state DOWN group default
    link/ether 02:42:88:56:50:83 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/24 brd 172.17.0.255 scope global docker0
       valid_lft forever preferred_lft forever
```



KODEKLOUD

BRIDGE

ip addr

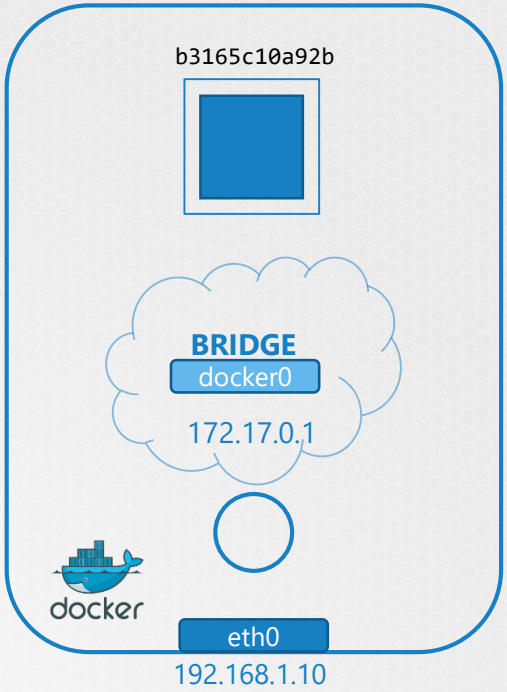
```
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc
noqueue state DOWN group default
    link/ether 02:42:88:56:50:83 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/24 brd 172.17.0.255 scope global docker0
       valid_lft forever preferred_lft forever
```

ip netns

```
b3165c10a92b
```

docker inspect 942d70e585b2

```
 "NetworkSettings": {
            "Bridge": "",
            "SandboxID": "b3165c10a92b50edce4c8aa5f37273e180907ded31",
           "SandboxKey": "/var/run/docker/netns/b3165c10a92b",
```



docker run nginx

```
2e41deb9ef1b8b3d141c7bb55d883541b4
```

KODEKLOUD

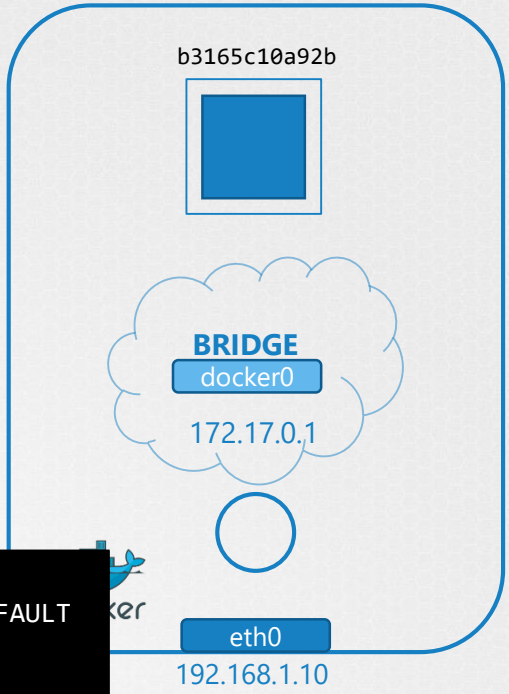
BRIDGE

ip netns

```
b3165c10a92b
```

ip link

```
...
4: docker0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP mode DEFAULT
group default
    link/ether 02:42:9b:5f:d6:21 brd ff:ff:ff:ff:ff:ff
8: vethbb1c343@if7: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker0
state UP mode DEFAULT group default
    link/ether 9e:71:37:83:9f:50 brd ff:ff:ff:ff:ff:ff link-netnsid 1
```



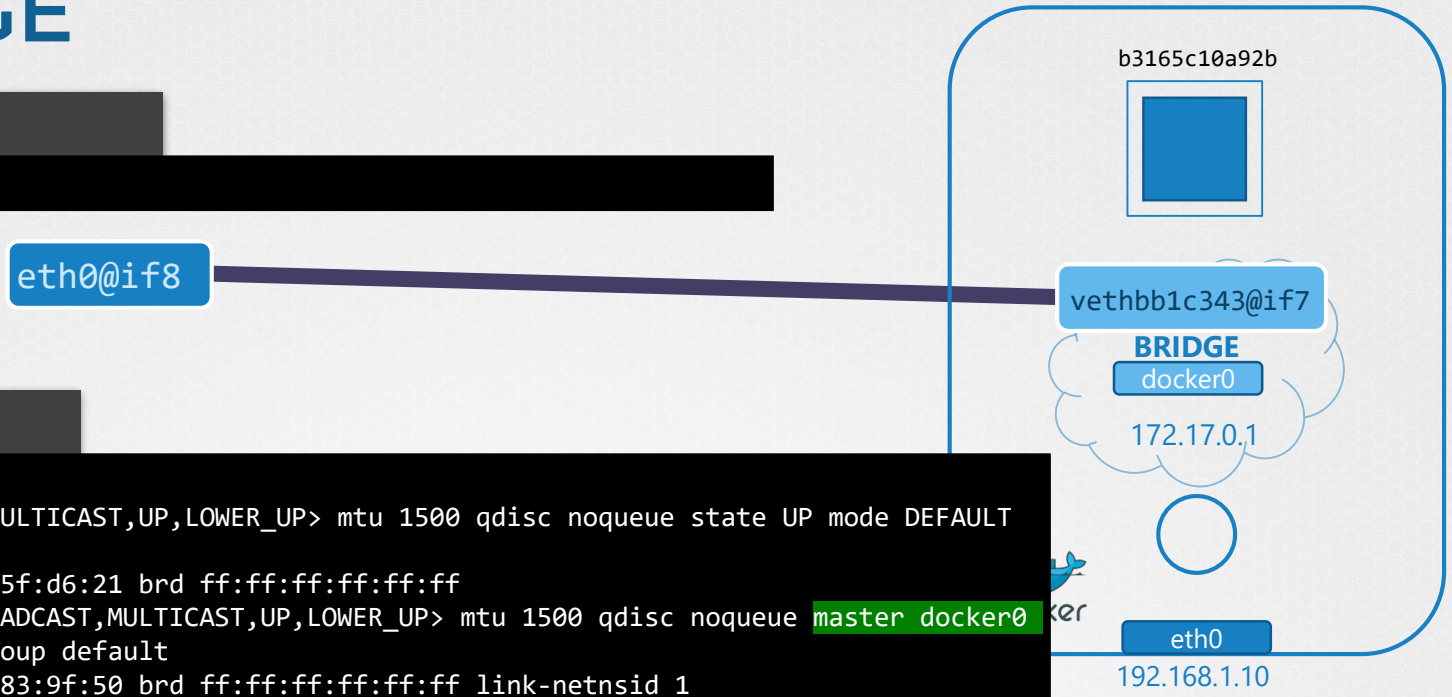
ocker run nginx

```
2e41deb9ef1b8b3d141c7bb55d883541b4
```

KODEKLOUD

BRIDGE

```
ip netns
b3165c10a92b
```




```
ip link
...
4: docker0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP mode DEFAULT
group default
    link/ether 02:42:9b:5f:d6:21 brd ff:ff:ff:ff:ff:ff
8: vethbb1c343@if7: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker0
state UP mode DEFAULT group default
    link/ether 9e:71:37:83:9f:50 brd ff:ff:ff:ff:ff:ff link-netnsid 1
```

```
ip link -n b3165c10a92b
...
7: eth0@if8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP mode DEFAULT
group default
    link/ether 02:42:ac:11:00:03 brd ff:ff:ff:ff:ff:ff link-netnsid 0
```

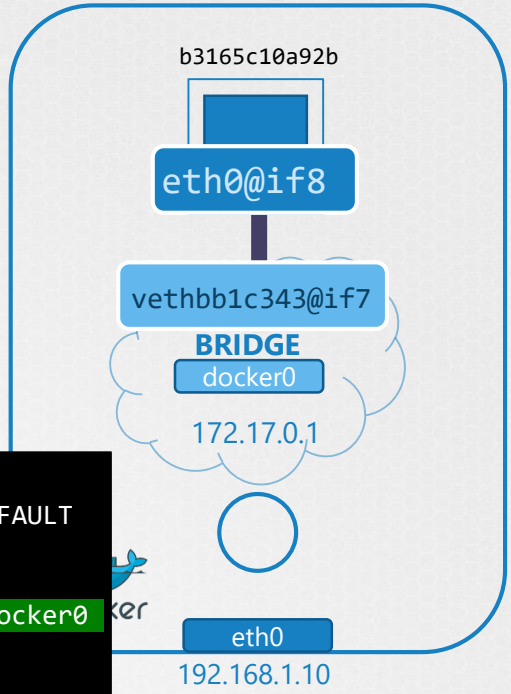
```
docker run nginx
2e41deb9ef1b8b3d141c7bb55
```

KODEKLOUD

BRIDGE

```
ip netns
```

```
b3165c10a92b
```




```
ip link
```

```
...
4: docker0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP mode DEFAULT
group default
    link/ether 02:42:9b:5f:d6:21 brd ff:ff:ff:ff:ff:ff
8: vethbb1c343@if7: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker0
state UP mode DEFAULT group default
    link/ether 9e:71:37:83:9f:50 brd ff:ff:ff:ff:ff:ff link-netnsid 1
```

```
ip -n b3165c10a92b link
```

```
...
7: eth0@if8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP mode DEFAULT
group default
    link/ether 02:42:ac:11:00:03 brd ff:ff:ff:ff:ff:ff link-netnsid 0
```

```
docker run nginx
```

```
2e41deb9ef1b8b3d141c7bb55
```

KODEKLOUD

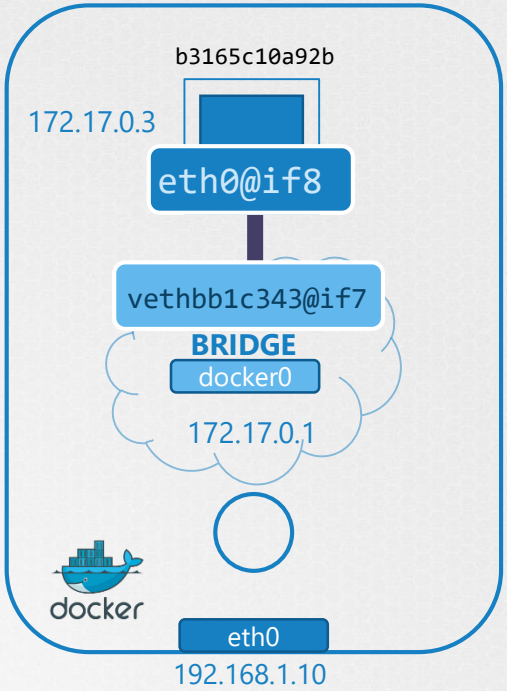
BRIDGE

```
ip netns
```

```
b3165c10a92b
```

```
ip addr b3165c10a92b
```

```
7: eth0@if8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue
state UP group default
    link/ether 02:42:ac:11:00:03 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 172.17.0.3/16 brd 172.17.255.255 scope global eth0
       valid_lft forever preferred_lft forever
```

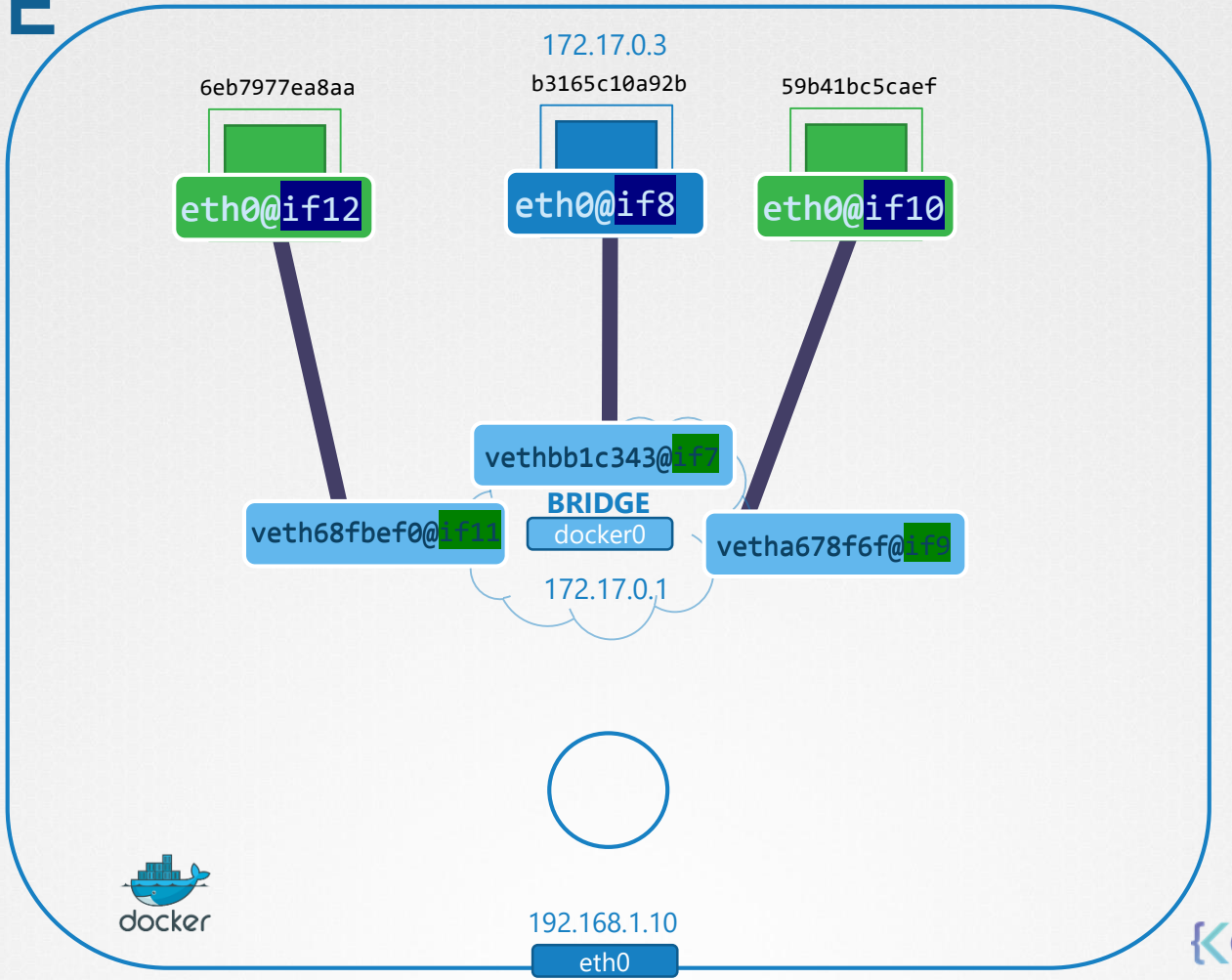



```
docker run nginx
```

```
2e41deb9ef1b8b3d141c7bb55
```

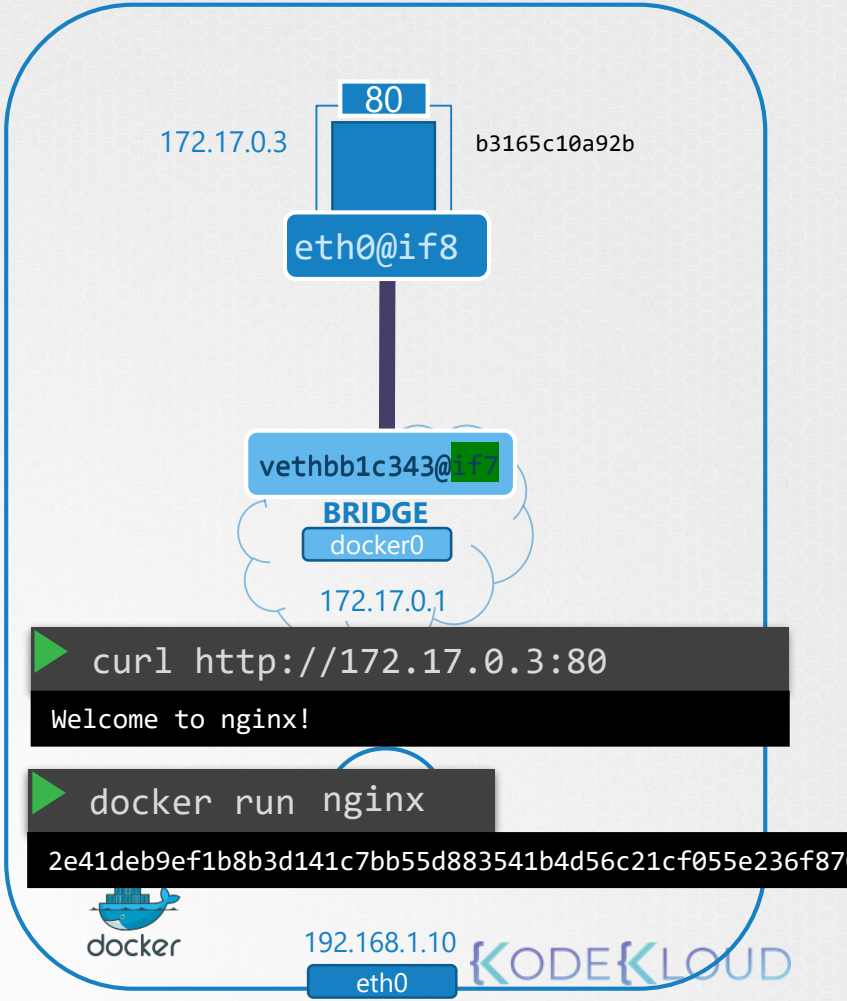
KODEKLOUD

BRIDGE



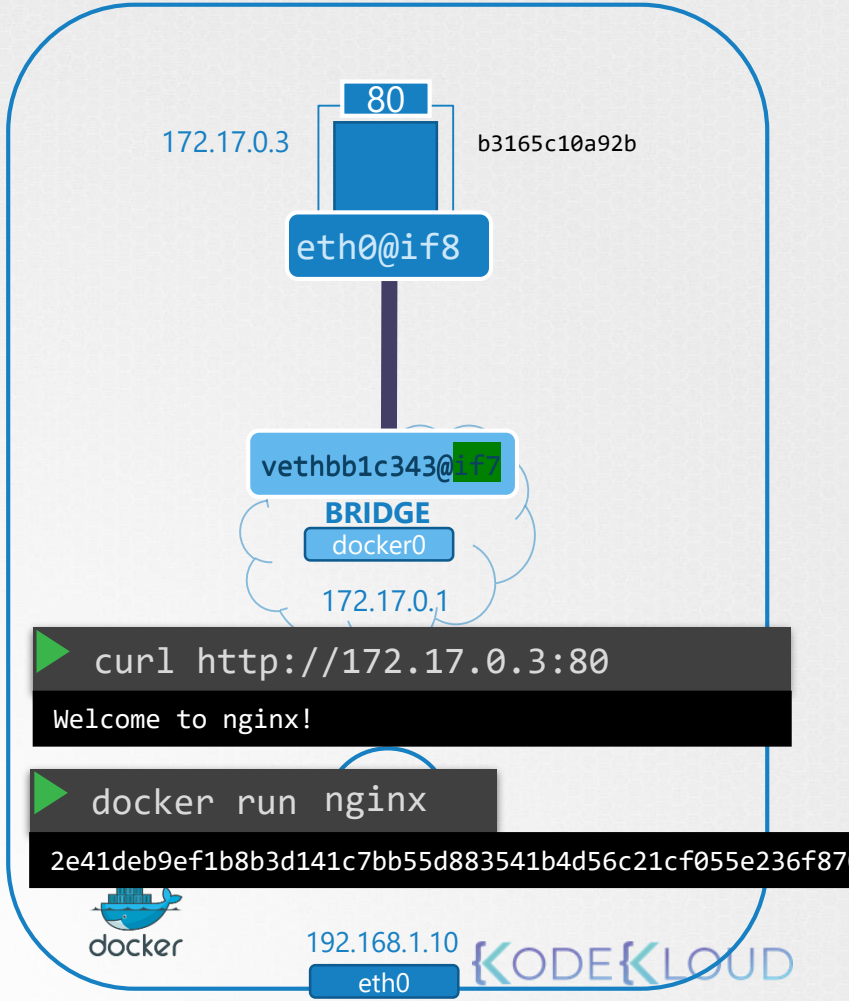
KODEKLOUD

BRIDGE


80
172.17.0.3
b3165c10a92b
eth0@if8
vethbb1c343@if7
BRIDGE
docker0
172.17.0.1
curl http://172.17.0.3:80
Welcome to nginx!
docker run nginx
2e41deb9ef1b8b3d141c7bb55d883541b4d56c21cf055e236f87
docker
192.168.1.10
eth0
KODEKLOUD

BRIDGE

```
curl http://172.17.0.3:80
curl: (7) Failed to connect... No route to host
```

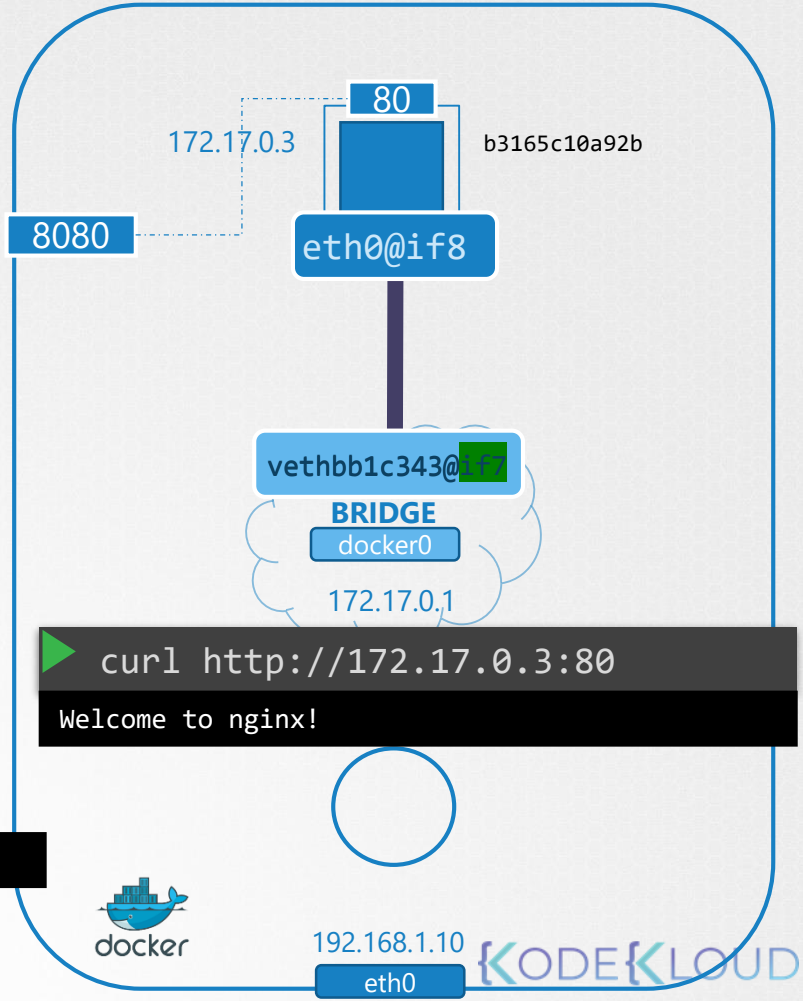


BRIDGE

```
curl http://172.17.0.3:80
curl: (7) Failed to connect... No route to host
```

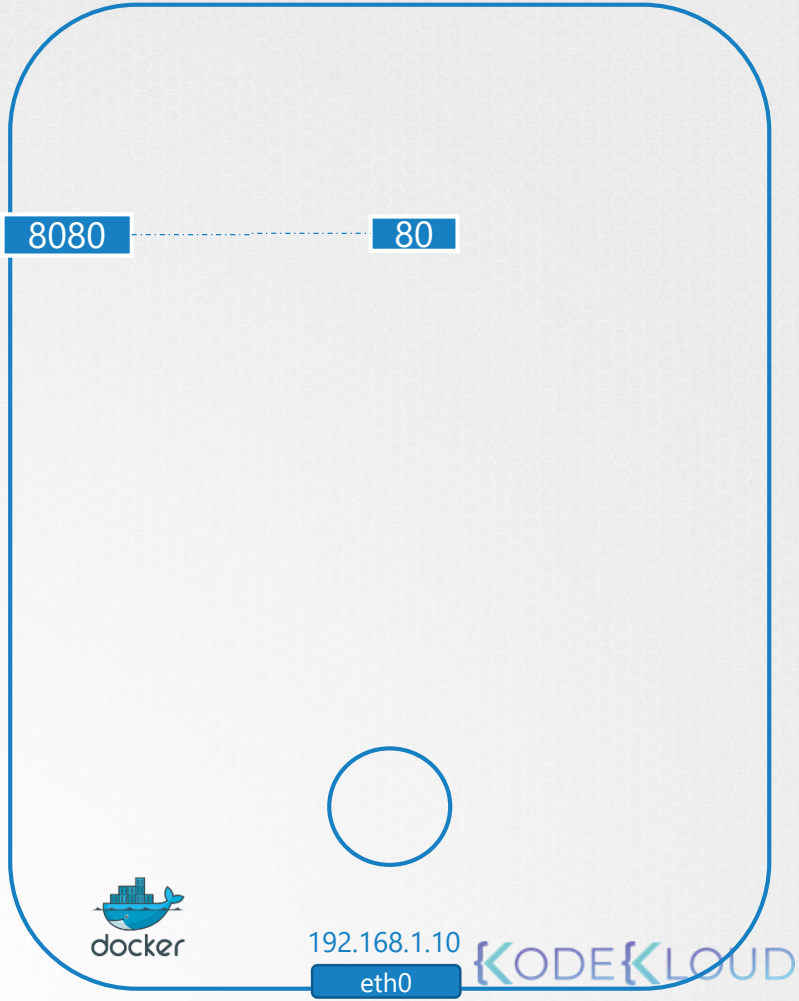
```
curl http://192.168.1.10:8080
Welcome to nginx!
```

```
docker run -p 8080:80 nginx
2e41deb9ef1b8b3d141c7bb55d883541b4d56c21cf055e236f870bd0f274e52b
```



BRIDGE

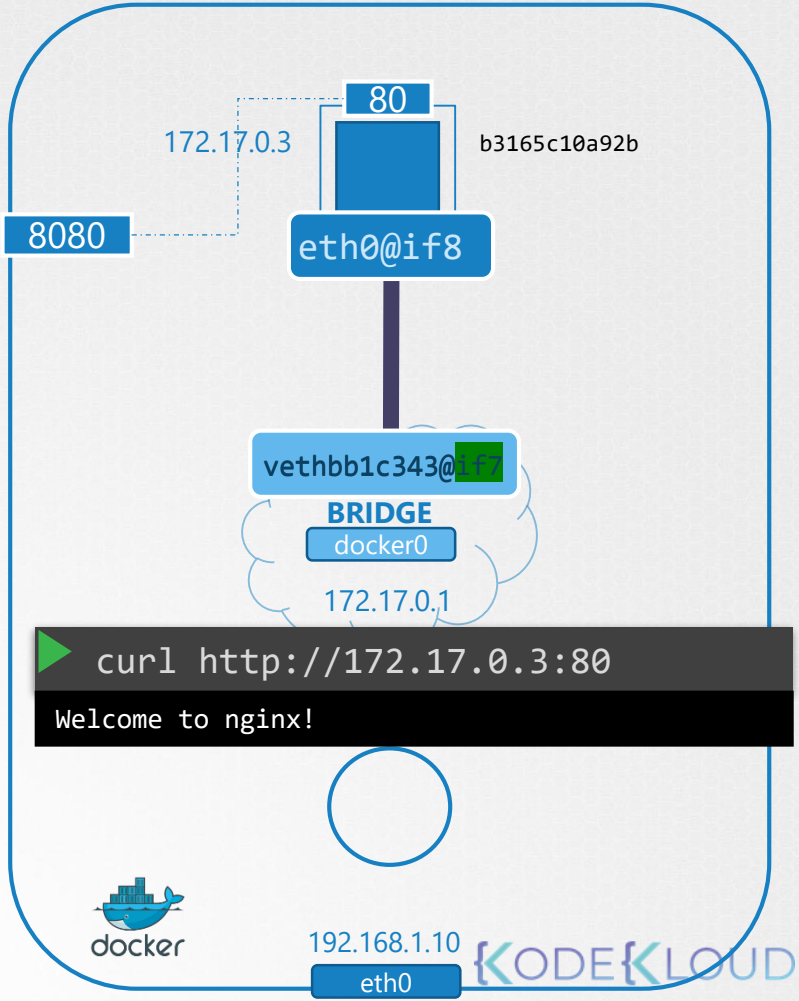
```
iptables \
      -t nat \
      -A PREROUTING \
      -j DNAT \
      --dport 8080 \
      --to-destination 80
```



BRIDGE

```
iptables \
      -t nat \
      -A PREROUTING \
      -j DNAT \
      --dport 8080 \
      --to-destination 80
```

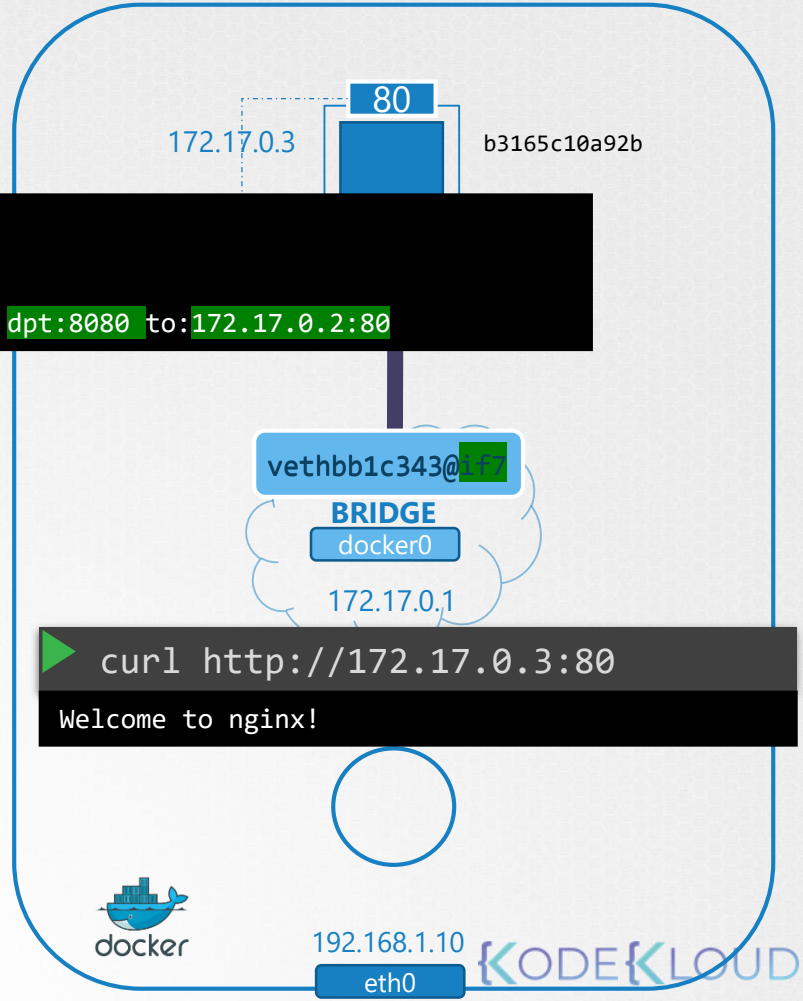
```
iptables \
      -t nat \
      -A DOCKER \
      -j DNAT \
      --dport 8080 \
      --to-destination 172.17.0.3:80
```



BRIDGE

```
iptables -nvL -t nat
```

```
Chain DOCKER (2 references)
target     prot opt source               destination
RETURN     all  --  anywhere             anywhere
DNAT       tcp  --  anywhere             anywhere             tcp dpt:8080 to:172.17.0.2:80
```



https://docs.docker.com/v17.09/engine/userguide/networking/default_network/container-communication/#communicating-to-the-outside-world

ln -s /var/run/docker/netns /var/run

sudo route add 172.17.0.6/32 gateway 192.168.176.14 enp0s8

sudo ip neighbor add 172.17.0.6 lladdr 02:42:ac:11:00:06 dev enp0s8

sudo bridge fdb add 02:42:ac:11:00:06 dev enp0s8 self

sudo iptables -P FORWARD ACCEPT

KODEKLOUD

KODEKLOUD

Container Networking Interface (CNI)

Network Namespaces

1. Create Network Namespace
2. Create Bridge Network/Interface
3. Create VETH Pairs (Pipe, Virtual Cable)
4. Attach vEth to Namespace
5. Attach Other vEth to Bridge
6. Assign IP Addresses
7. Bring the interfaces up
8. Enable NAT – IP Masquerade

1. Create Network Namespace
2. Create Bridge Network/Interface
3. Create VETH Pairs (Pipe, Virtual Cable)
4. Attach vEth to Namespace
5. Attach Other vEth to Bridge
6. Assign IP Addresses
7. Bring the interfaces up
8. Enable NAT – IP Masquerade

1. Create Network Namespace
2. Create Bridge Network/Interface
3. Create VETH Pairs (Pipe, Virtual Cable)
4. Attach vEth to Namespace
5. Attach Other vEth to Bridge
6. Assign IP Addresses
7. Bring the interfaces up
8. Enable NAT – IP Masquerade

1. Create Network Namespace
2. Create Bridge Network/Interface
3. Create VETH Pairs (Pipe, Virtual Cable)
4. Attach vEth to Namespace
5. Attach Other vEth to Bridge
6. Assign IP Addresses
7. Bring the interfaces up
8. Enable NAT – IP Masquerade

1. Create Network Namespace
2. Create Bridge Network/Interface
3. Create VETH Pairs (Pipe, Virtual Cable)
4. Attach vEth to Namespace
5. Attach Other vEth to Bridge
6. Assign IP Addresses
7. Bring the interfaces up
8. Enable NAT – IP Masquerade

KODEKLOUD

Network Namespaces

1. Create Network Namespace

1. Create Network Namespace

1. Create Network Namespace

```
bridge add <cid> <namespace>
```

1. Create Network Namespace

```
bridge add <cid> <namespace>
```

1. Create Network Namespace

```
bridge add 2e34dcf34 /var/run/netns/2e34dcf34
```

BRIDGE

2. Create Bridge Network/Interface

3. Create VETH Pairs (Pipe, Virtual Cable)

4. Attach vEth to Namespace

5. Attach Other vEth to Bridge

6. Assign IP Addresses

7. Bring the interfaces up

8. Enable NAT – IP Masquerade

KODEKLOUD

Network Namespaces

docker

1. Create Network Namespace

rkt

1. Create Network Namespace

MESOS

1. Create Network Namespace

1. Create Network Namespace

CONTAINER NETWORK INTERFACE

BRIDGE

2. Create Bridge Network/Interface

3. Create VETH Pairs (Pipe, Virtual Cable)

4. Attach vEth to Namespace

5. Attach Other vEth to Bridge

6. Assign IP Addresses

7. Bring the interfaces up

8. Enable NAT – IP Masquerade

KODEKLOUD

CONTAINER NETWORK INTERFACE

- ☑ Container Runtime must create network namespace
- ☑ Identify network the container must attach to
- ☑ Container Runtime to invoke Network Plugin (bridge) when container is ADDed.
- ☑ Container Runtime to invoke Network Plugin (bridge) when container is DELeted.
- ☑ JSON format of the Network Configuration

rkt

MESOS

- ☑ Must support command line arguments ADD/DEL/CHECK
- ☑ Must support parameters container id, network ns etc..
- ☑ Must manage IP Address assignment to PODs
- ☑ Must Return results in a specific format

BRIDGE

2. Create Bridge Network/Interface
3. Create VETH Pairs (Pipe, Virtual Cable)
4. Attach vEth to Namespace
5. Attach Other vEth to Bridge
6. Assign IP Addresses
7. Bring the interfaces up
8. Enable NAT – IP Masquerade

CONTAINER NETWORK INTERFACE

```
docker run --network=cni-bridge nginx
```

```
docker run --network=none nginx
```

```
bridge add 2e34dcf34 /var/run/netns/2e34dcf34
```

docker

rkt

MESOS

CONTAINER NETWORK MODEL (CNM)

BRIDGE | VLAN | IPVLAN | MACVLAN | WINDOWS

DHCP | host-local

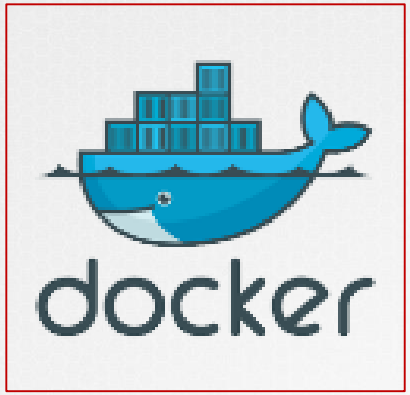
weaveworks

flannel

cilium

vmware NSX

CONTAINER NETWORK INTERFACE



```
docker run --network=none nginx
```

```
bridge add 2e34dcf34 /var/run/netns/2e34dcf34
```

KODEKLOUD

KODEKLOUD

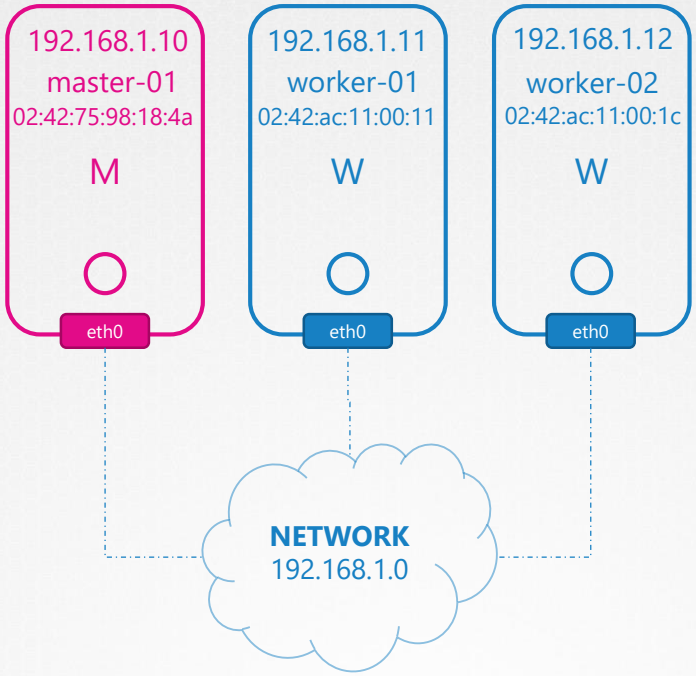
Course Objectives

- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Troubleshooting
- Core Concepts
- Networking
  - Pre-Requisites – Switching, Routing
  - Pre-Requisites – DNS
  - Networking Configuration on Cluster Nodes
  - Cluster DNS
  - Service Networking
  - Pre-Requisites – Tools
  - Pre-Requisites – Networking in Docker
  - POD Networking Concepts
  - Network Loadbalancer
  - Pre-Requisites CNI
  - CNI in Kubernetes
  - Ingress

KODEKLOUD

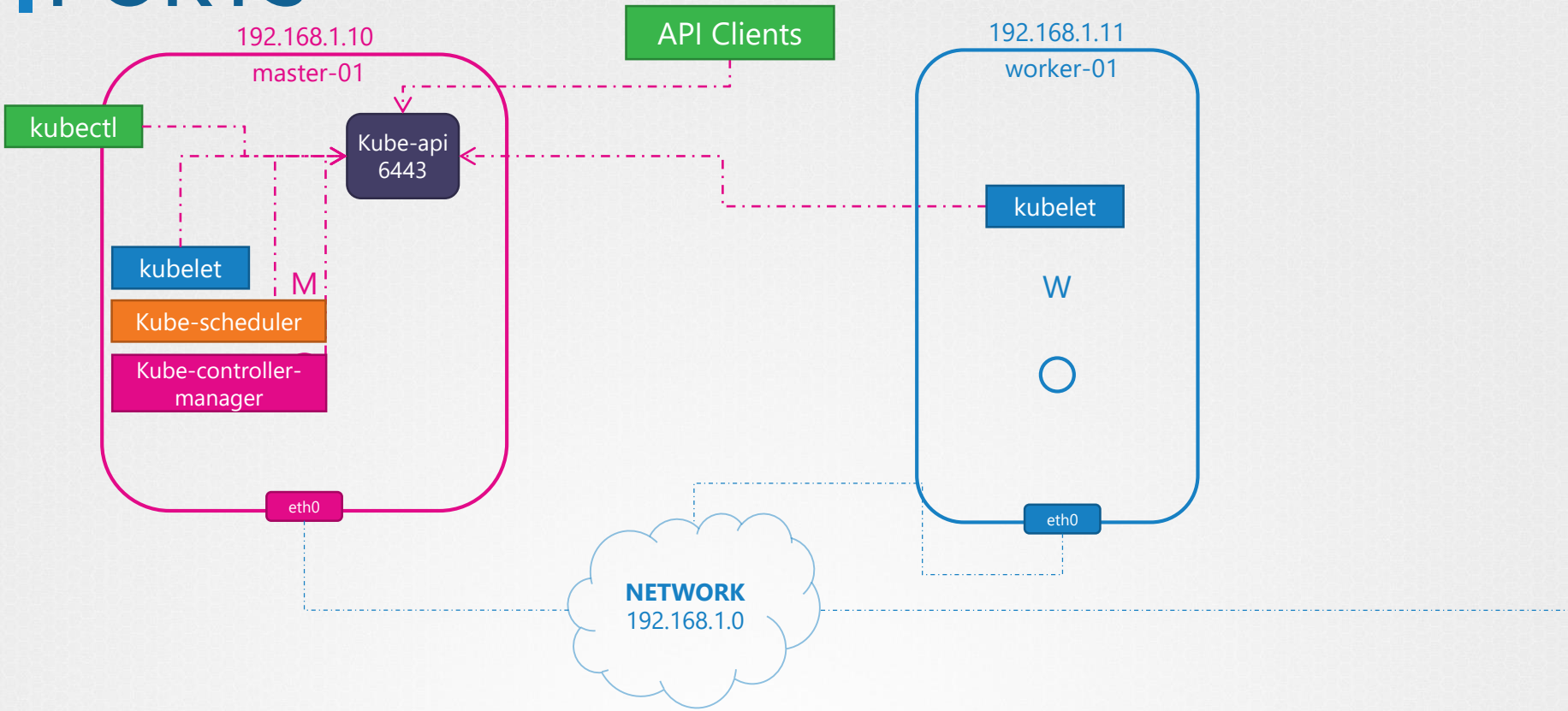
Networking Cluster Nodes

IP & FQDN



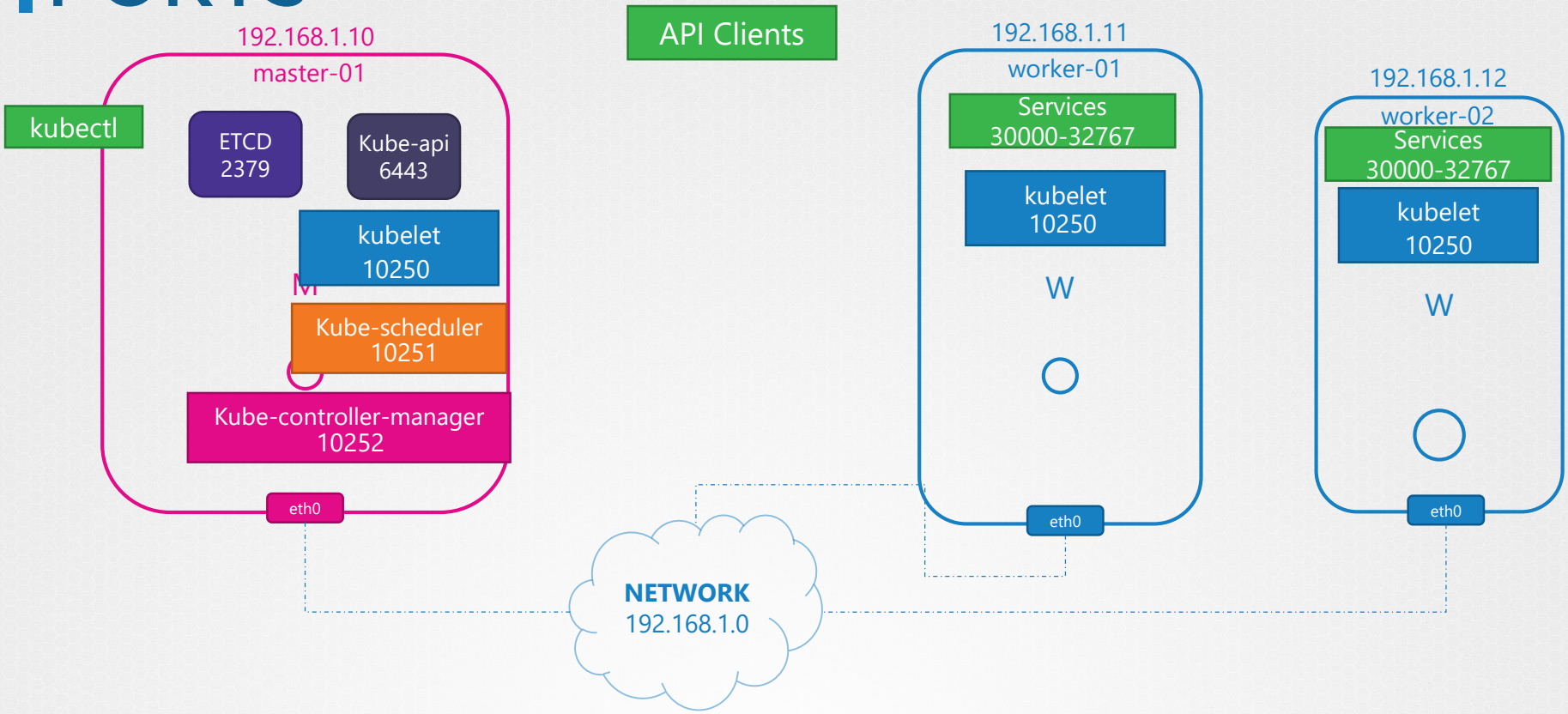
KODEKLOUD

PORTS



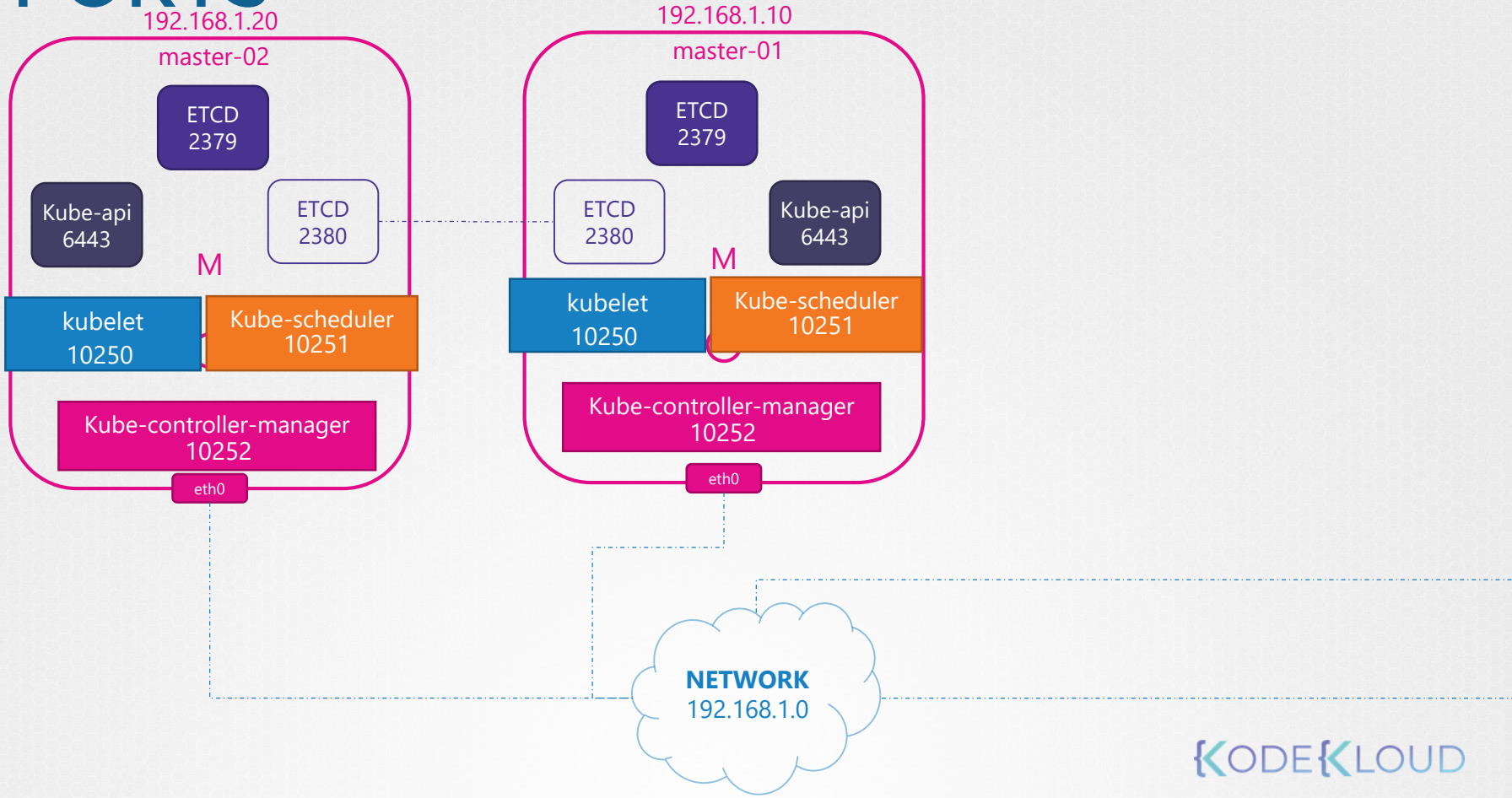
KODEKLOUD

PORTS



KODEKLOUD

PORTS



KODEKLOUD

Documentation

Check required ports

Master node(s)

Protocol	Direction	Port Range	Purpose	Used By
TCP	Inbound	6443*	Kubernetes API server	All
TCP	Inbound	2379-2380	etcd server client API	kube-apiserver, etcd
TCP	Inbound	10250	Kubelet API	Self, Control plane
TCP	Inbound	10251	kube-scheduler	Self
TCP	Inbound	10252	kube-controller-manager	Self

Worker node(s)

Protocol	Direction	Port Range	Purpose	Used By
TCP	Inbound	10250	Kubelet API	Self, Control plane
TCP	Inbound	30000-32767	NodePort Services**	All

https://kubernetes.io/docs/setup/independent/install-kubeadm/#check-required-ports

KODEKLOUD

COMMMANDS

```
ip link
```

```
ip addr
```

```
ip addr add 192.168.1.10/24 dev eth0
```

```
ip route
```

```
ip route add 192.168.1.0/24 via 192.168.2.1
```

```
cat /proc/sys/net/ipv4/ip_forward
1
```

```
arp
```

```
netstat -plnt
```

```
route
```

KODEKLOUD

KODEKLOUD

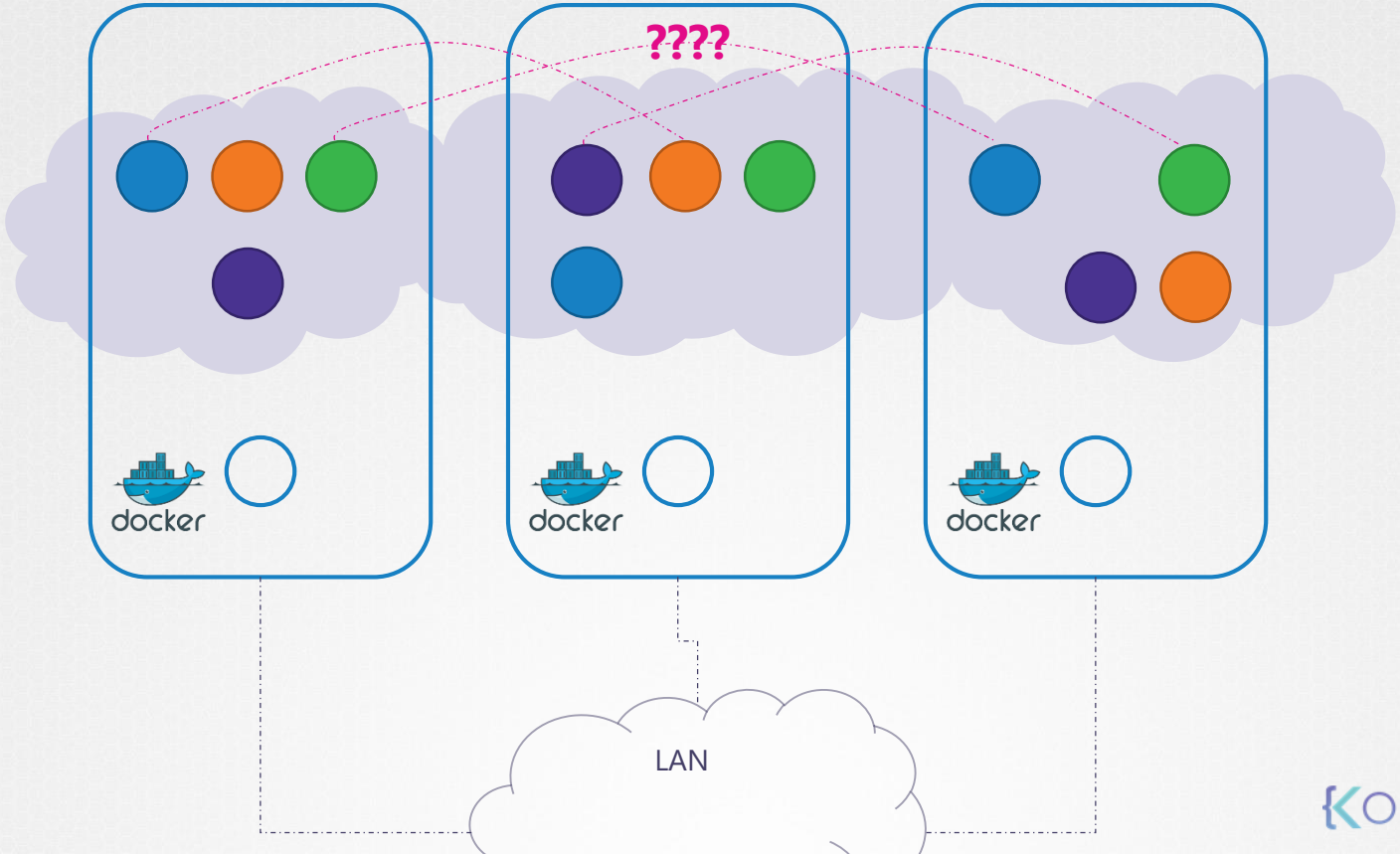
Course Objectives

- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Troubleshooting
- Core Concepts
- Networking
  - Pre-Requisites – Switching, Routing
  - Pre-Requisites – DNS
  - Networking Configuration on Cluster Nodes
  - Cluster DNS
  - Service Networking
  - Pre-Requisites – Tools
  - Pre-Requisites – Networking in Docker
  - POD Networking Concepts
  - Network Loadbalancer
  - Pre-Requisites CNI
  - CNI in Kubernetes
  - Ingress

KODEKLOUD

POD Networking Concepts

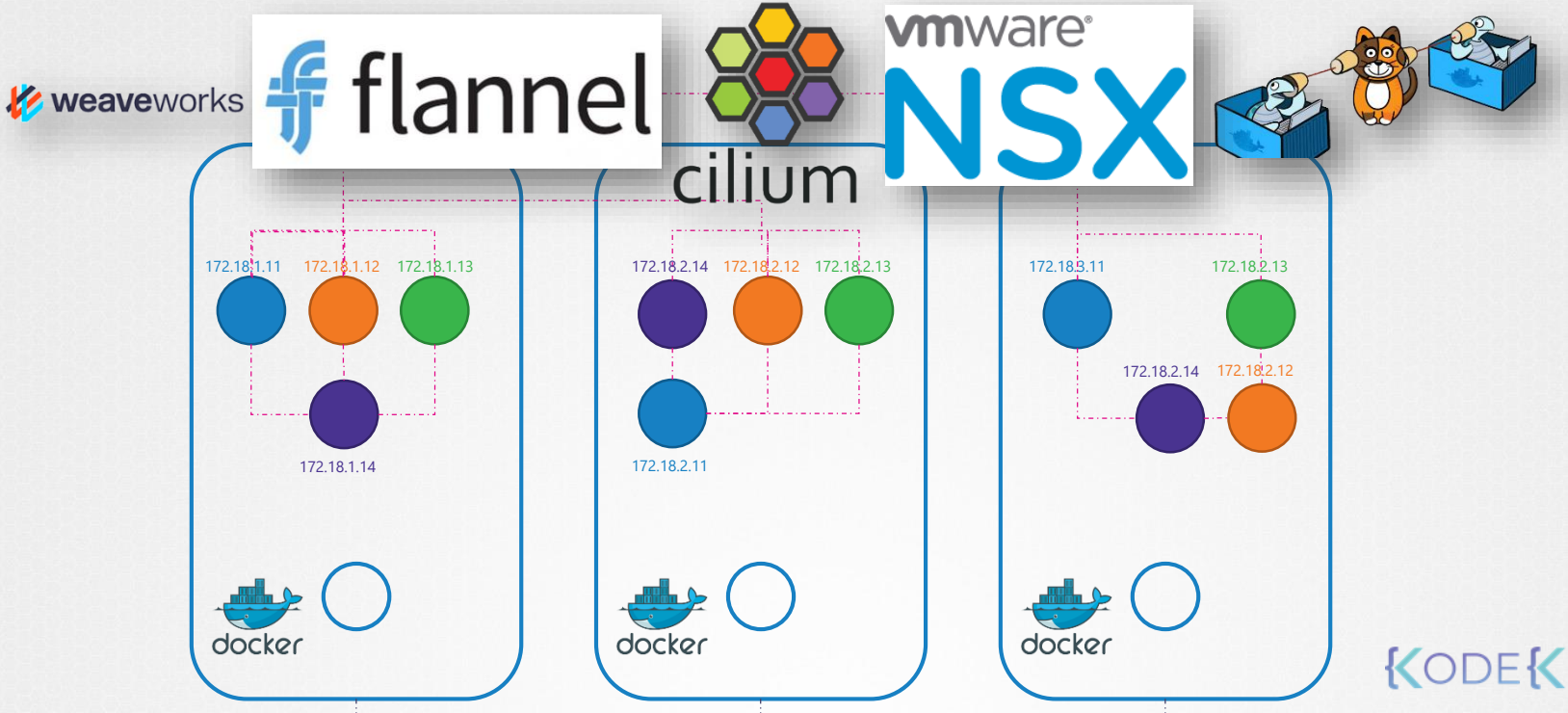
KODEKLOUD



KODEKLOUD

Networking Model

- ❑ Every POD should have an IP Address
- ❑ Every POD should be able to communicate with every other POD in the same node.
- ❑ Every POD should be able to communicate with every other POD on other nodes without NAT.

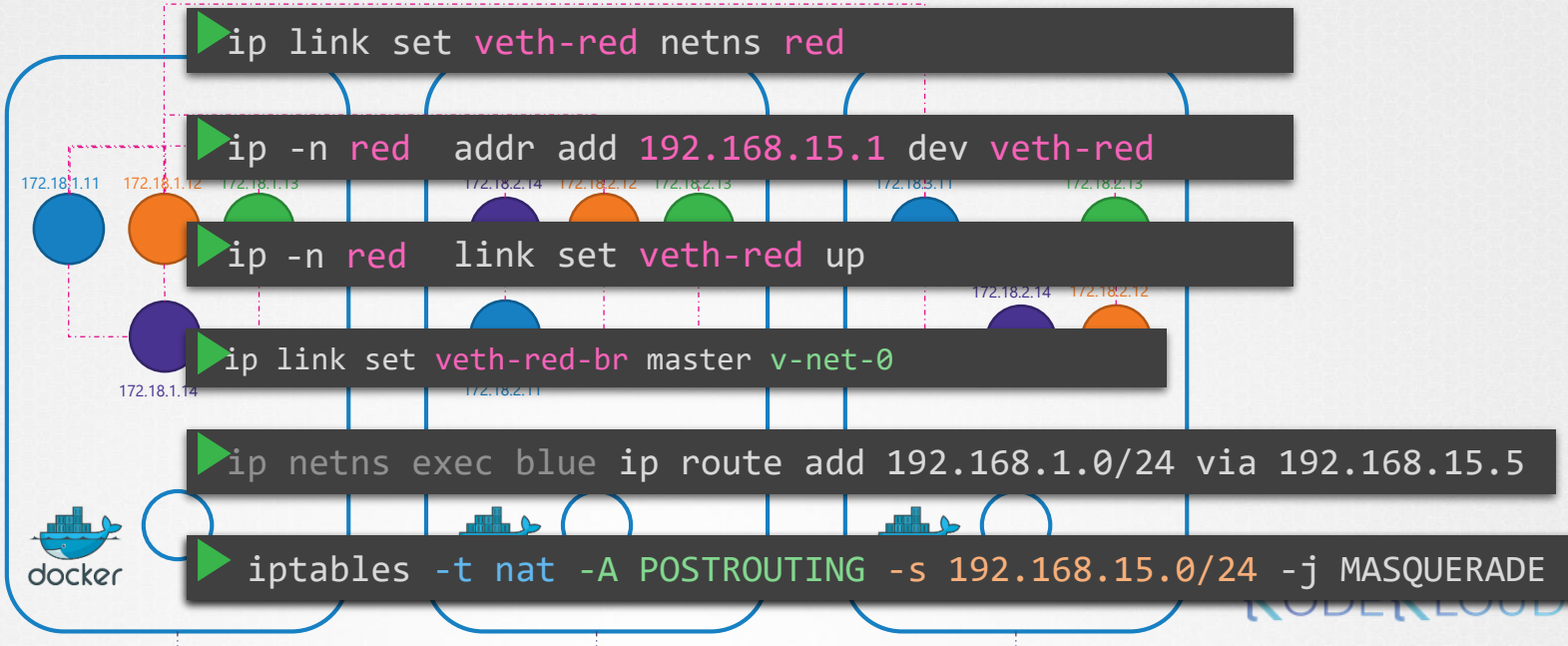


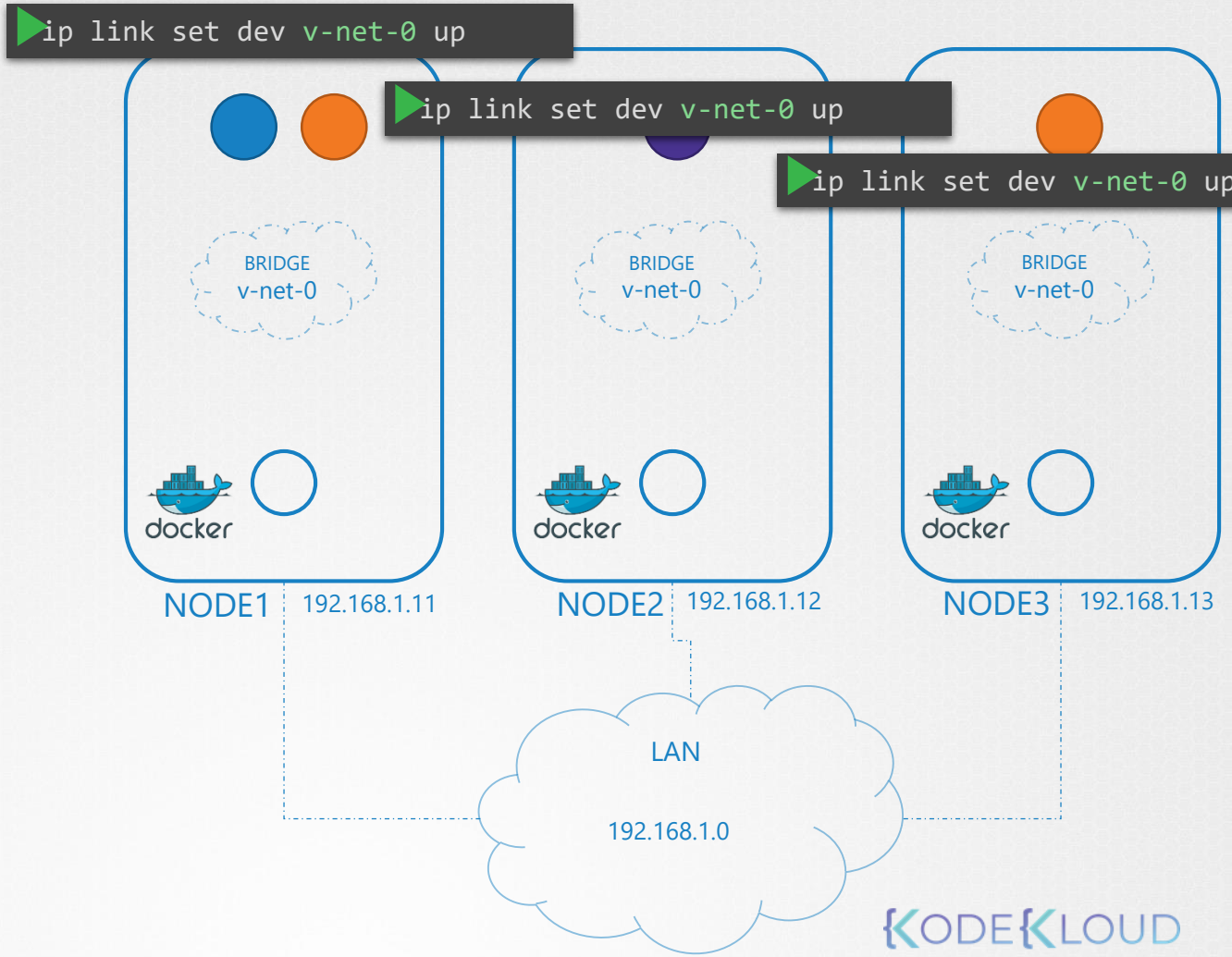
KODEKLOUD

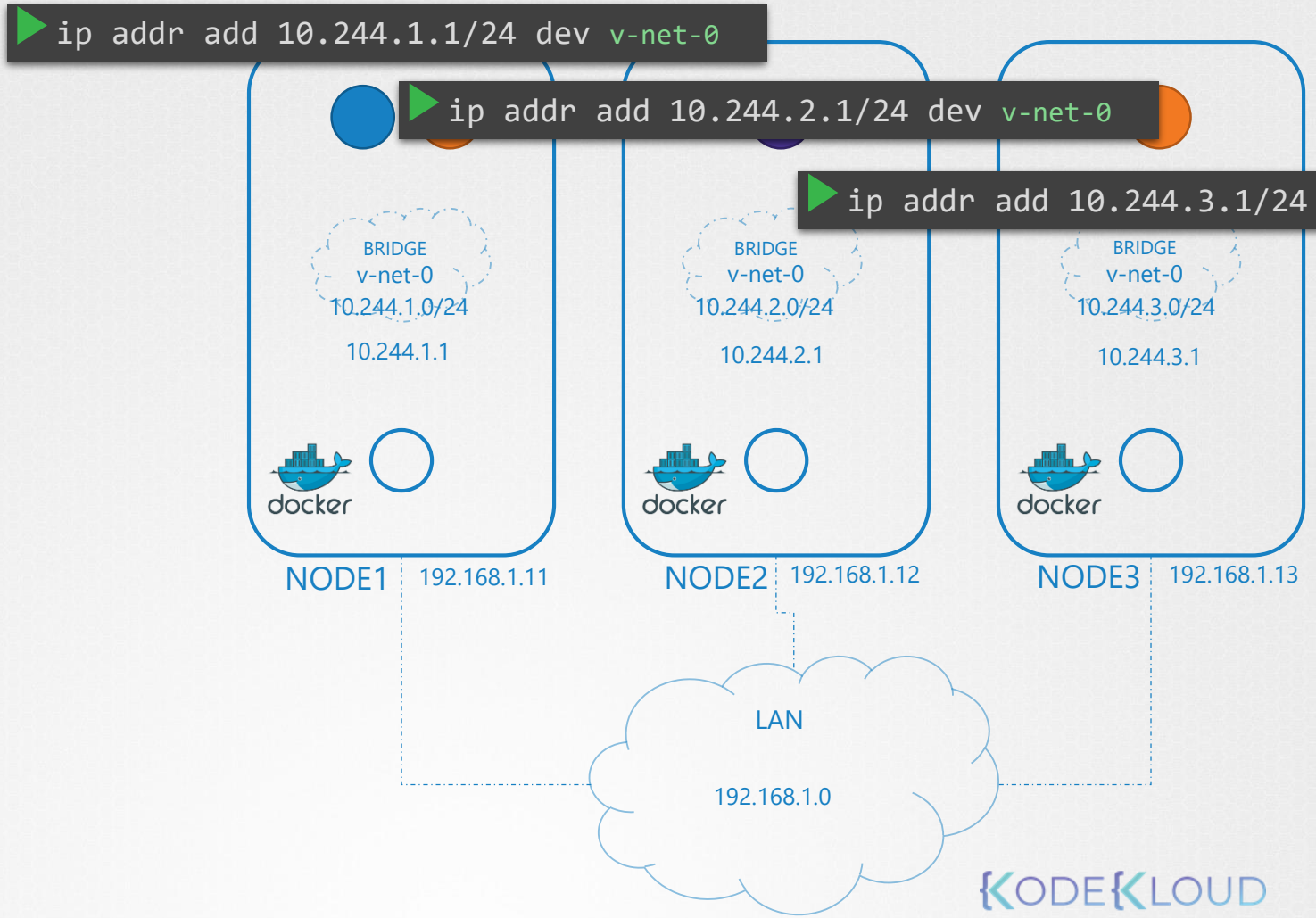
Networking Model

- Every POD should have an IP Address
- Every POD shou ame node.
- Every POD should be able to communicate with every other POD on other nodes without NAT.

```
ip link add v-net-0 type bridge
ip link set dev v-net-0 up
ip addr add 192.168.15.5/24 dev v-net-0
ip link add veth-red type veth peer name veth-red-br
ip link set veth-red netns red
ip -n red  addr add 192.168.15.1 dev veth-red
ip -n red  link set veth-red up
ip link set veth-red-br master v-net-0
ip netns exec blue ip route add 192.168.1.0/24 via 192.168.15.5
iptables -t nat -A POSTROUTING -s 192.168.15.0/24 -j MASQUERADE
```

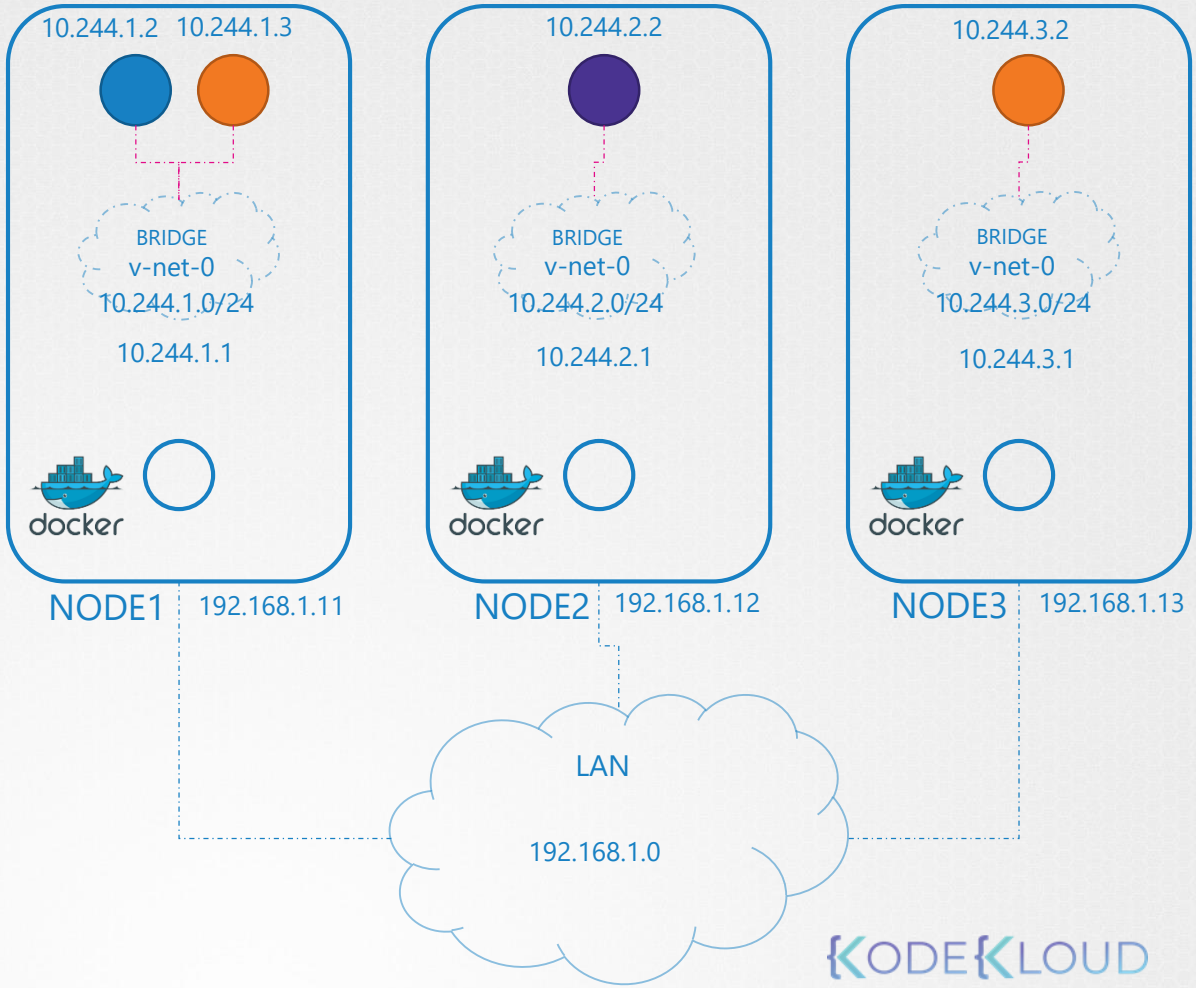


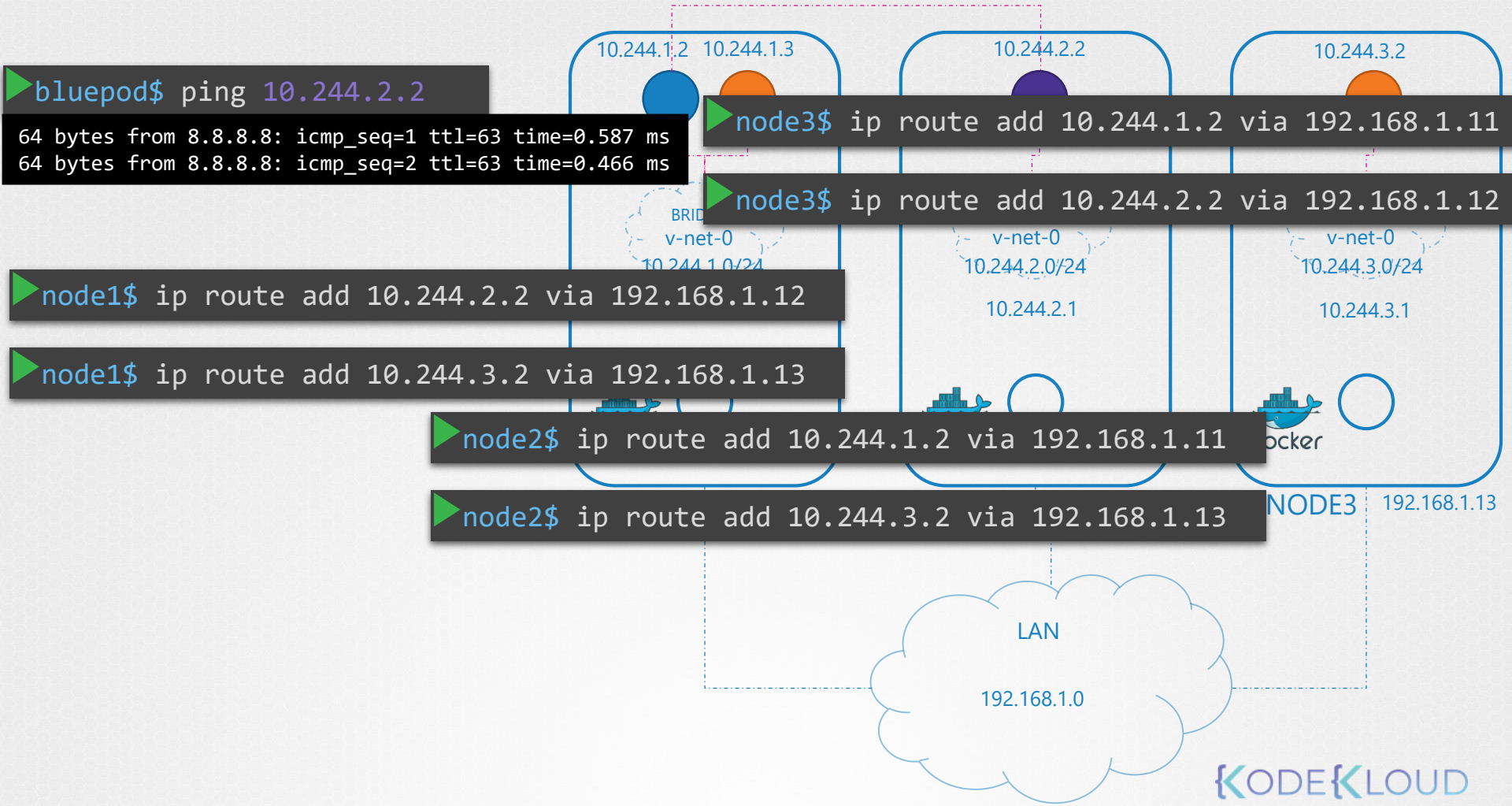

ip link set dev v-net-0 up
ip link set dev v-net-0 up
ip link set dev v-net-0 up
BRIDGE
v-net-0
BRIDGE
v-net-0
BRIDGE
v-net-0
docker
docker
docker
NODE1
192.168.1.11
NODE2
192.168.1.12
NODE3
192.168.1.13
LAN
192.168.1.0
KODEKLOUD

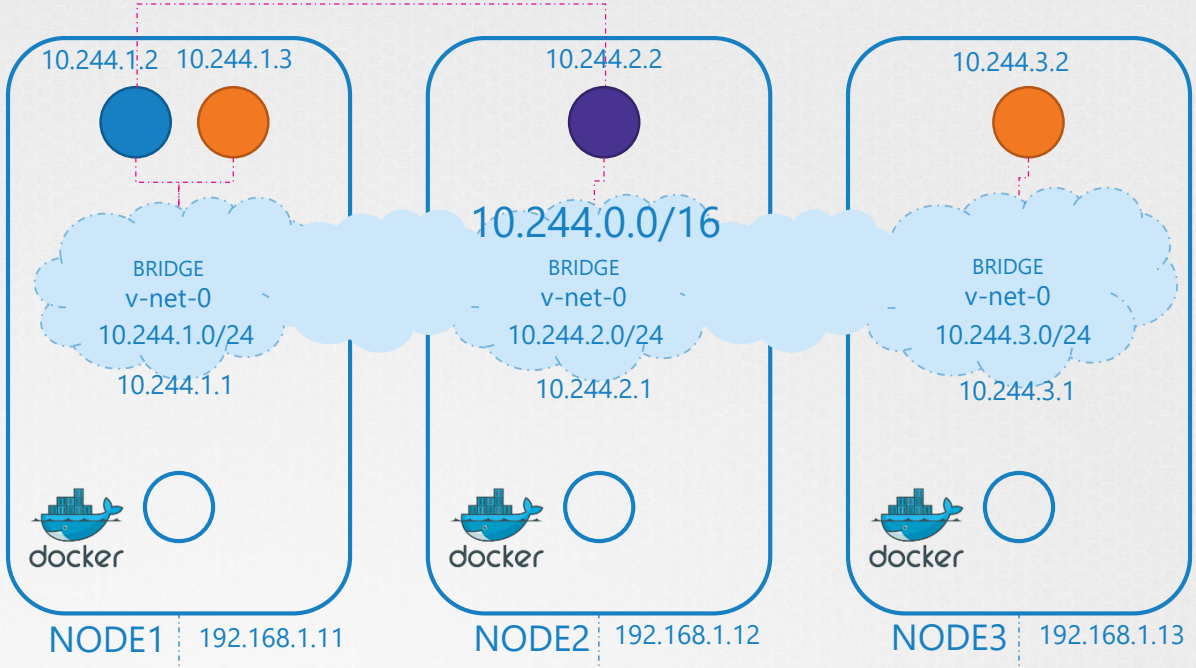

ip addr add 10.244.1.1/24 dev v-net-0
ip addr add 10.244.2.1/24 dev v-net-0
ip addr add 10.244.3.1/24
BRIDGE
v-net-0
10.244.1.0/24
10.244.1.1
BRIDGE
v-net-0
10.244.2.0/24
10.244.2.1
BRIDGE
v-net-0
10.244.3.0/24
10.244.3.1
docker
docker
docker
NODE1
192.168.1.11
NODE2
192.168.1.12
NODE3
192.168.1.13
LAN
192.168.1.0
KODEKLOUD

net-script.sh

```
# Create veth pair
ip link add ……

# Attach veth pair
ip link set ……

ip link set ……

# Assign IP Address
ip -n <namespace> addr add ……

ip -n <namespace> route add ……

# Bring Up Interface
ip -n <namespace> link set ……
```




bluepod$ ping 10.244.2.2
64 bytes from 8.8.8.8: icmp_seq=1 ttl=63 time=0.587 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=63 time=0.466 ms
node3$ ip route add 10.244.1.2 via 192.168.1.11
node3$ ip route add 10.244.2.2 via 192.168.1.12
node1$ ip route add 10.244.2.2 via 192.168.1.12
node1$ ip route add 10.244.3.2 via 192.168.1.13
node2$ ip route add 10.244.1.2 via 192.168.1.11
node2$ ip route add 10.244.3.2 via 192.168.1.13
10.244.1.2
10.244.1.3
10.244.2.2
10.244.3.2
v-net-0
10.244.2.0/24
10.244.2.1
v-net-0
10.244.3.0/24
10.244.3.1
NODE3
192.168.1.13
LAN
192.168.1.0
KODEKLOUD



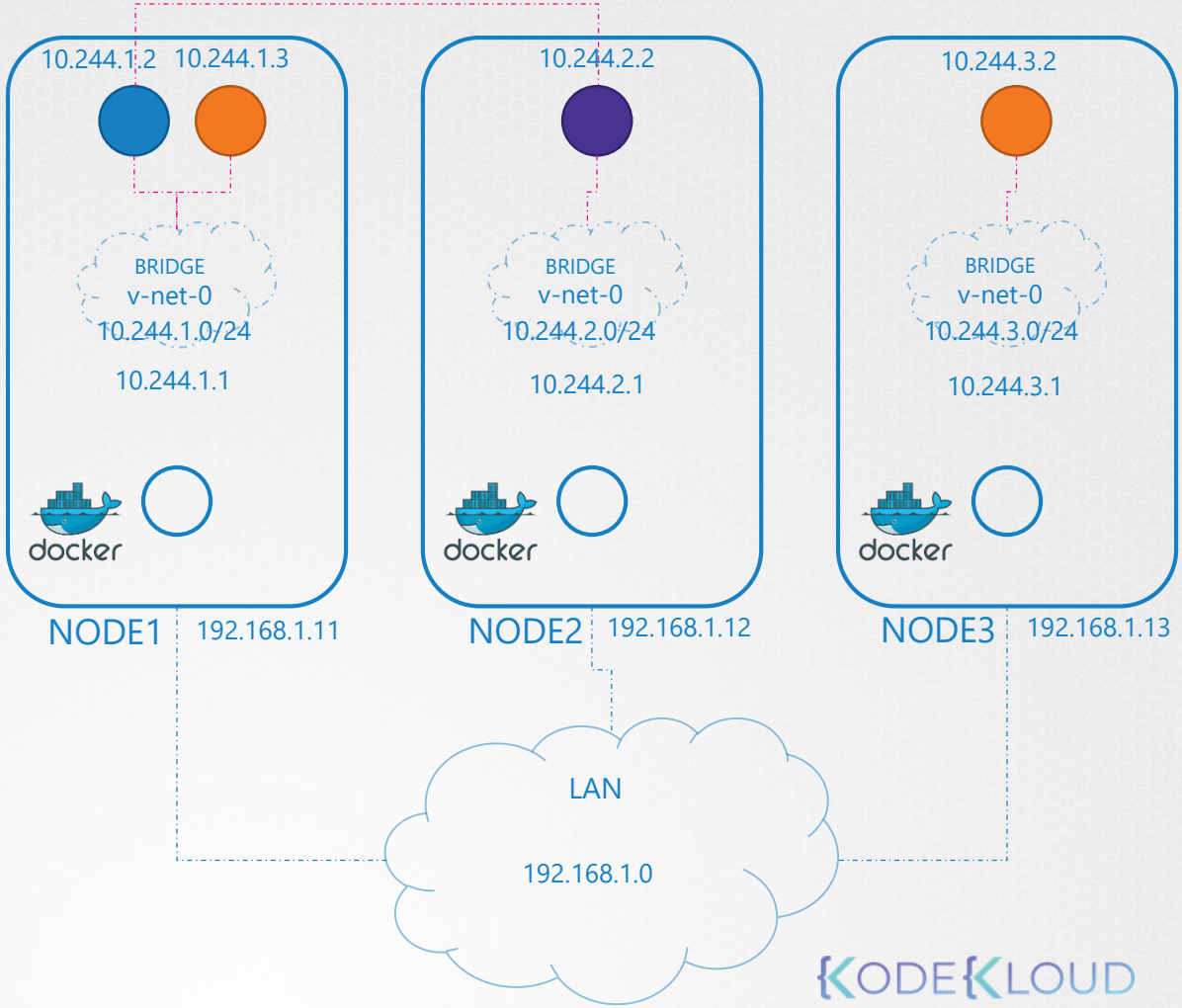
NETWORK	GATEWAY
10.244.1.0/24	192.168.1.11
10.244.2.0/24	192.168.1.12
10.244.3.0/24	192.168.1.13

KLOUD

CONTAINER NETWORK INTERFACE (CNI)

net-script.sh

```
# Create veth pair
ip link add ……

# Attach veth pair
ip link set ……
ip link set ……

# Assign IP Address
ip -n <namespace> addr add ……
ip -n <namespace> route add ……
```



KODEKLOUD

CONTAINER NETWORK INTERFACE (CNI)

kubelet

--cni-conf-dir=/etc/cni/net.d

--cni-bin-dir=/etc/cni/bin

./net-script.sh add <container> <namespace>

10.244.1.2 10.244.1.3

BRIDGE
v-net-0
10.244.1.0/24

10.244.1.1

docker

NODE1 192.168.1.11

10.244.2.2

BRIDGE
v-net-0
10.244.2.0/24

10.244.2.1

docker

NODE2 192.168.1.12

LAN

192.168.1.0

net-script.sh

```
ADD)
  # Create veth pair
  # Attach veth pair
  # Assign IP Address
  # Bring Up Interface
  ip -n <namespace> link set ……
DEL)
  # Delete veth pair
  ip link del ……
```

KODEKLOUD

KODEKLOUD

Course Objectives

- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Troubleshooting
- Core Concepts
- Networking
  - Pre-Requisites – Switching, Routing
  - Pre-Requisites – DNS
  - Networking Configuration on Cluster Nodes
  - Cluster DNS
  - Service Networking
  - Pre-Requisites – Tools
  - Pre-Requisites – Networking in Docker
  - POD Networking Concepts
  - Network Loadbalancer
  - Pre-Requisites CNI
  - CNI in Kubernetes
  - Ingress

KODEKLOUD

Container Networking Interface (CNI)
IN KUBERNETES

Pre-Requisites

- ✓ Network Namespaces in Linux
- ✓ Networking in Docker
- ✓ Why and what is Container Network Interface (CNI)?
- ✓ CNI Plugins

KODEKLOUD

CONTAINER NETWORK INTERFACE

- ✓ Container Runtime must create network namespace
- ✓ Identify network the container must attach to
- ✓ Container Runtime to invoke Network Plugin (bridge) when container is ADDed.
- ✓ Container Runtime to invoke Network Plugin (bridge) when container is DELeted.
- ✓ JSON format of the Network Configuration

KODEKLOUD

Configuring CNI

kubelet.service

```
ExecStart=/usr/local/bin/kubelet \\
  --config=/var/lib/kubelet/kubelet-config.yaml \\
  --container-runtime=remote \\
  --container-runtime-endpoint=unix:///var/run/containerd/containerd.sock \\
  --image-pull-progress-deadline=2m \\
  --kubeconfig=/var/lib/kubelet/kubeconfig \\
  --network-plugin=cni \\
  --cni-bin-dir=/opt/cni/bin \\
  --cni-conf-dir=/etc/cni/net.d \\
  --register-node=true \\
  --v=2
```

KODEKLOUD

View kubelet options

```
ps -aux | grep kubelet
```

```
root      2095  1.8  2.4 960676 98788 ?        Ssl  02:32   0:36 /usr/bin/kubelet --bootstrap-kubeconfig=/etc/kubernetes/bootstrap-kubelet.conf --kubeconfig=/etc/kubernetes/kubelet.conf --config=/var/lib/kubelet/config.yaml --cgroup-driver=cgroupfs --cni-bin-dir=/opt/cni/bin --cni-conf-dir=/etc/cni/net.d --network-plugin=cni
```

```
ls /opt/cni/bin
```

```
bridge  dhcp  flannel  host-local  ipvlan  loopback  macvlan  portmap  ptp  sample  tuning  vlan  weave-ipam  weave-net  weave-plugin-2.2.1
```

```
ls /etc/cni/net.d
```

```
10-bridge.conf
```

KODEKLOUD

View kubelet options

```
ls /etc/cni/net.d
```

```
10-bridge.conf
```

```
cat /etc/cni/net.d/10-bridge.conf
```

```
{
    "cniVersion": "0.2.0",
    "name": "mynet",
    "type": "bridge",
    "bridge": "cni0",
    "isGateway": true,
    "ipMasq": true,
    "ipam": {
        "type": "host-local",
        "subnet": "10.22.0.0/16",
        "routes": [
            { "dst": "0.0.0.0/0" }
        ]
    }
}
```

KODEKLOUD

KODEKLOUD

Course Objectives

- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Troubleshooting
- Core Concepts
- Networking
  - Pre-Requisites – Switching, Routing
  - Pre-Requisites – Tools
  - Pre-Requisites CNI
  - Pre-Requisites – DNS, IPAM, Firewalls, LBs
  - Pre-Requisites – Networking in Docker
  - Networking Configuration on Cluster Nodes
  - POD Networking Concepts
  - CNI in Kubernetes
  - Cluster DNS
  - Service Networking
  - Network Loadbalancer
  - Ingress

KODEKLOUD

WeaveWorks (CNI)

CONTAINER NETWORK INTERFACE (CNI)

kubelet

--cni-conf-dir=/etc/cni/net.d

--cni-bin-dir=/etc/cni/bin

./net-script.sh add <container> <namespace>

10.244.1.2 10.244.1.3

BRIDGE
v-net-0
10.244.1.0/24

10.244.1.1

docker

NODE1 192.168.1.11

10.244.2.2

BRIDGE
v-net-0
10.244.2.0/24

10.244.2.1

docker

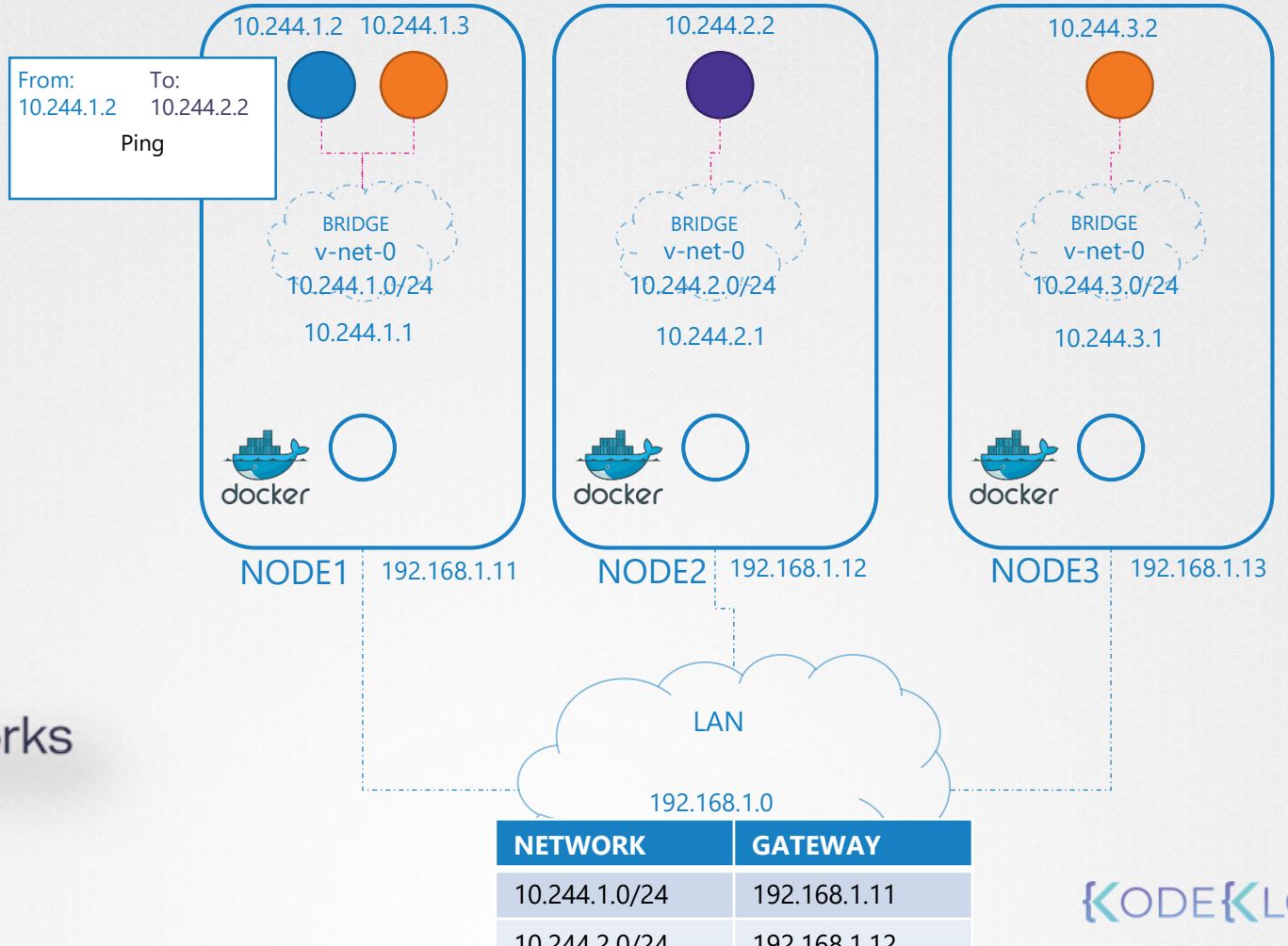
NODE2 192.168.1.12

LAN

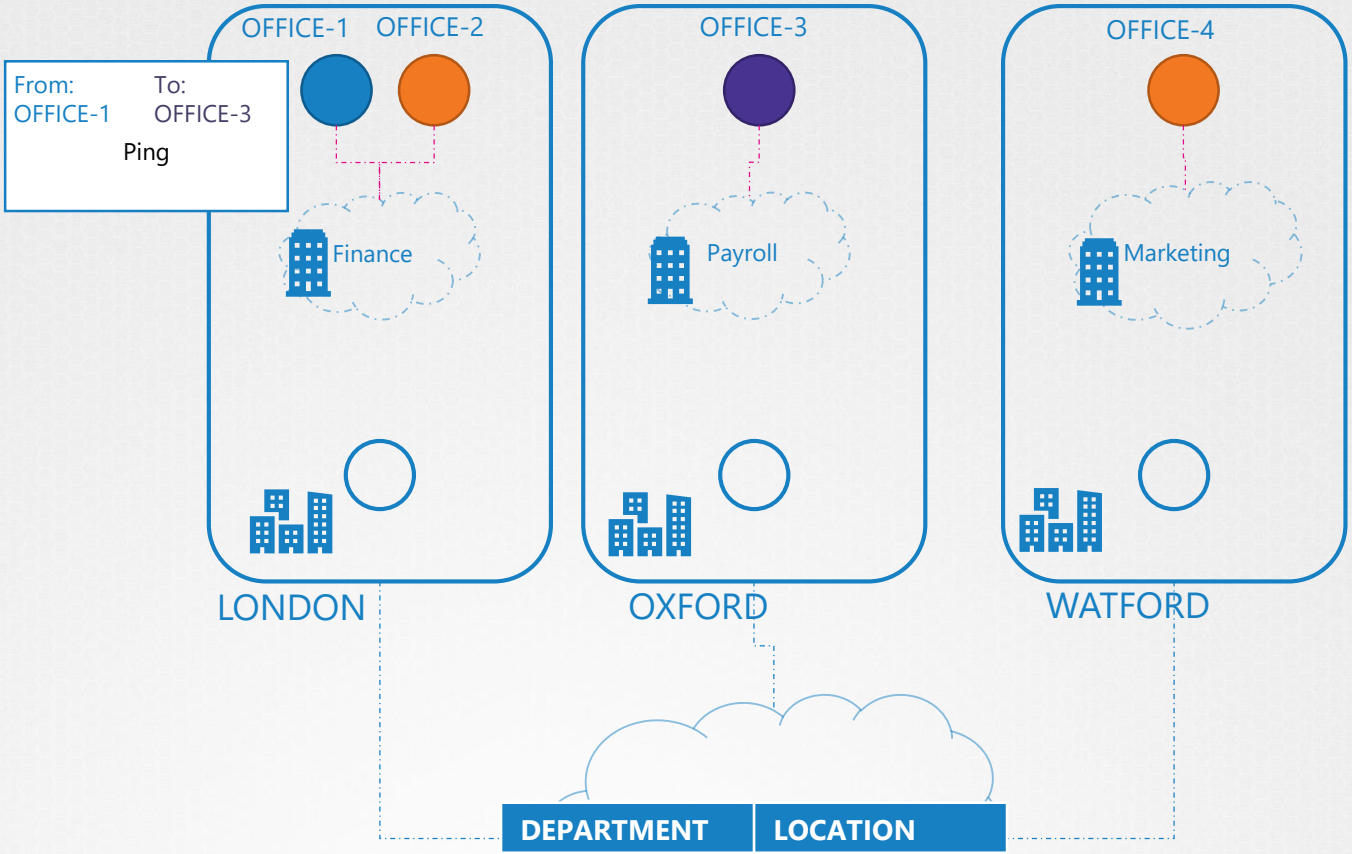
NETWORK	GATEWAY
10.244.1.0/24	192.168.1.11
10.244.2.0/24	192.168.1.12
10.244.3.0/24	192.168.1.13

net-script.sh

```
ADD)
  # Create veth pair
  # Attach veth pair
  # Assign IP Address
  # Bring Up Interface
  ip -n <namespace> link set ……
DEL)
  # Delete veth pair
  ip link del ……
```

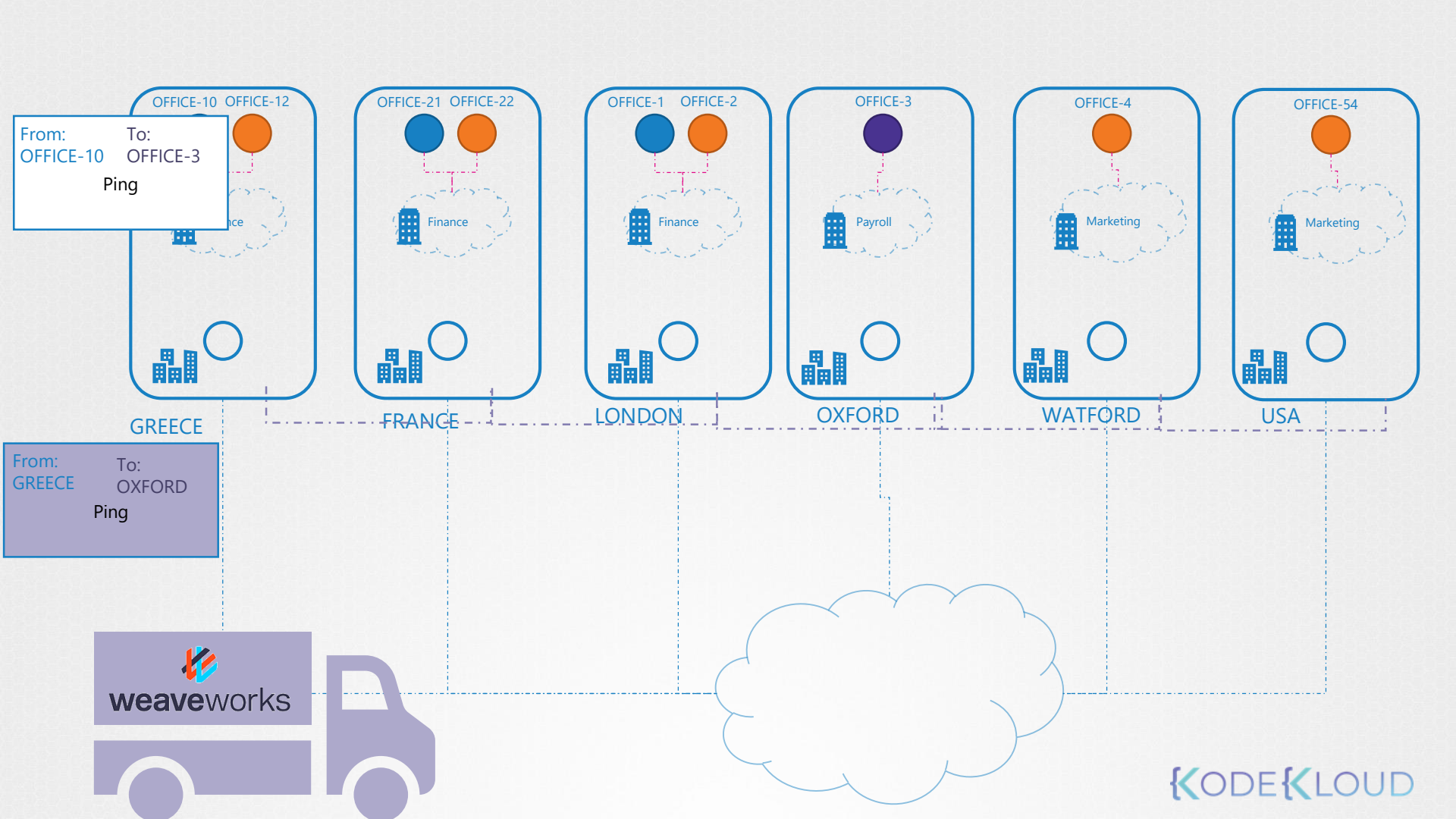


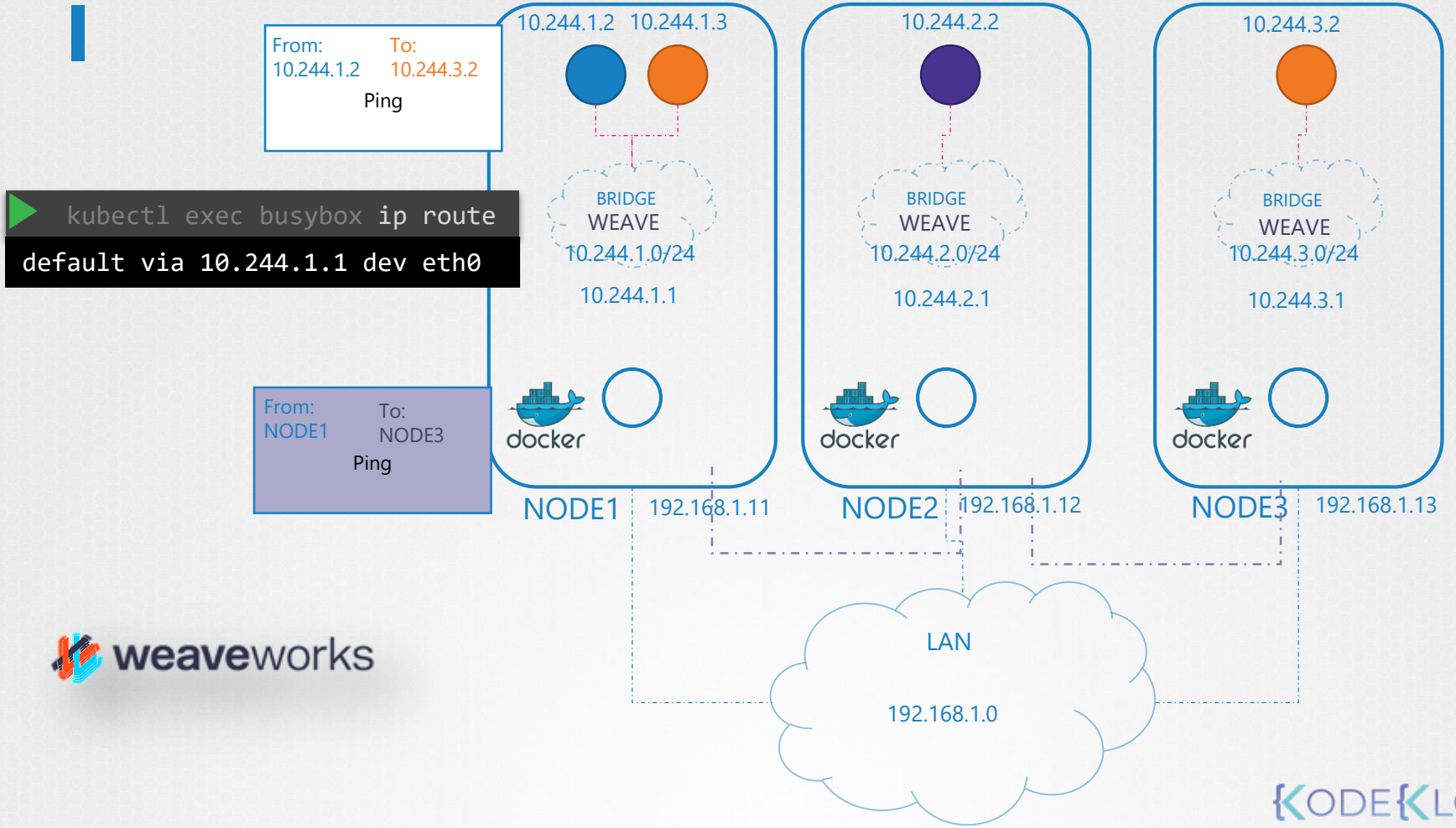
NETWORK	GATEWAY
10.244.1.0/24	192.168.1.11
10.244.2.0/24	192.168.1.12



DEPARTMENT	LOCATION
FINANCE	LONDON
PAYROLL	OXFORD
MARKETING	WATFORD

KODEKLOUD


From:
OFFICE-10
To:
OFFICE-3
Ping
OFFICE-10
OFFICE-12
OFFICE-21
OFFICE-22
OFFICE-1
OFFICE-2
OFFICE-3
OFFICE-4
OFFICE-54
Finance
Finance
Payroll
Marketing
Marketing
GREECE
FRANCE
LONDON
OXFORD
WATFORD
USA
From:
GREECE
To:
OXFORD
Ping
weaveworks
KODEKLOUD


From:
10.244.1.2
To:
10.244.3.2
Ping
kubectl exec busybox ip route
default via 10.244.1.1 dev eth0
From:
NODE1
To:
NODE3
Ping
10.244.1.2
10.244.1.3
BRIDGE
WEAVE
10.244.1.0/24
10.244.1.1
docker
NODE1
192.168.1.11
10.244.2.2
BRIDGE
WEAVE
10.244.2.0/24
10.244.2.1
docker
NODE2
192.168.1.12
10.244.3.2
BRIDGE
WEAVE
10.244.3.0/24
10.244.3.1
docker
NODE3
192.168.1.13
LAN
192.168.1.0
weaveworks
KODEKLOUD

Deploy Weave

```
kubectl apply -f "https://cloud.weave.works/k8s/net?k8s-version=$(kubectl version | base64 | tr -d '\n')"
```

```
serviceaccount/weave-net created
clusterrole.rbac.authorization.k8s.io/weave-net created
clusterrolebinding.rbac.authorization.k8s.io/weave-net created
role.rbac.authorization.k8s.io/weave-net created
rolebinding.rbac.authorization.k8s.io/weave-net created
daemonset.extensions/weave-net created
```

KODEKLOUD

Weave Peers

```
kubectl get pods –n kube-system
```

```
NAME                                READY     STATUS    RESTARTS   AGE       IP              NODE
NOMINATED NODE
coredns-78fcdf6894-99khw            1/1       Running   0          19m       10.44.0.2       master    <none>
coredns-78fcdf6894-p7dpj            1/1       Running   0          19m       10.44.0.1       master    <none>
etcd-master                         1/1       Running   0          18m       172.17.0.11     master    <none>
kube-apiserver-master               1/1       Running   0          18m       172.17.0.11     master    <none>
kube-scheduler-master               1/1       Running   0          17m       172.17.0.11     master    <none>
weave-net-5gcmb                     2/2       Running   1          19m       172.17.0.30     node02    <none>
weave-net-fr9n9                     2/2       Running   1          19m       172.17.0.11     master    <none>
weave-net-mc6s2                     2/2       Running   1          19m       172.17.0.23     node01    <none>
weave-net-tbzvz                     2/2       Running   1          19m       172.17.0.52     node03    <none>
```

```
kubectl logs weave-net-5gcmb weave –n kube-system
```

```
INFO: 2019/03/03 03:41:08.643858 Command line options: map[status-addr:0.0.0.0:6782 http-addr:127.0.0.1:6784 ipalloc-range:10.32.0.0/12 name:9e:96:c8:09:bf:c4 nickname:node02 conn-limit:30
datapath:datapath db-prefix:/weavedb/weave-net host-root:/host port:6783 docker-api: expect-npc:true ipalloc-init:consensus=4 no-dns:true]
INFO: 2019/03/03 03:41:08.643980 weave  2.2.1
INFO: 2019/03/03 03:41:08.751508 Bridge type is bridged_fastdp
INFO: 2019/03/03 03:41:08.751526 Communication between peers is unencrypted.
INFO: 2019/03/03 03:41:08.753583 Our name is 9e:96:c8:09:bf:c4(node02)
INFO: 2019/03/03 03:41:08.753615 Launch detected - using supplied peer list: [172.17.0.11 172.17.0.23 172.17.0.30 172.17.0.52]
INFO: 2019/03/03 03:41:08.753632 Checking for pre-existing addresses on weave bridge
INFO: 2019/03/03 03:41:08.756183 [allocator 9e:96:c8:09:bf:c4] No valid persisted data
INFO: 2019/03/03 03:41:08.761033 [allocator 9e:96:c8:09:bf:c4] Initialising via deferred consensus
INFO: 2019/03/03 03:41:08.761091 Sniffing traffic on datapath (via ODP)
INFO: 2019/03/03 03:41:08.761659 ->[172.17.0.23:6783] attempting connection
INFO: 2019/03/03 03:41:08.817477 overlay_switch ->[8a:31:f6:b1:38:3f(node03)] using fastdp
INFO: 2019/03/03 03:41:08.819493 sleeve ->[172.17.0.52:6783|8a:31:f6:b1:38:3f(node03)]: Effective MTU verified at 1438
INFO: 2019/03/03 03:41:09.107287 Weave version 2.5.1 is available; please update at https://github.com/weaveworks/weave/releases/download/v2.5.1/weave
INFO: 2019/03/03 03:41:09.284907 Discovered remote MAC 8a:dd:b5:14:8f:a3 at 8a:dd:b5:14:8f:a3(node01)
INFO: 2019/03/03 03:41:09.331952 Discovered remote MAC 8a:31:f6:b1:38:3f at 8a:31:f6:b1:38:3f(node03)
INFO: 2019/03/03 03:41:09.355976 Discovered remote MAC 8a:a5:9c:d2:86:1f at 8a:31:f6:b1:38:3f(node03)
```

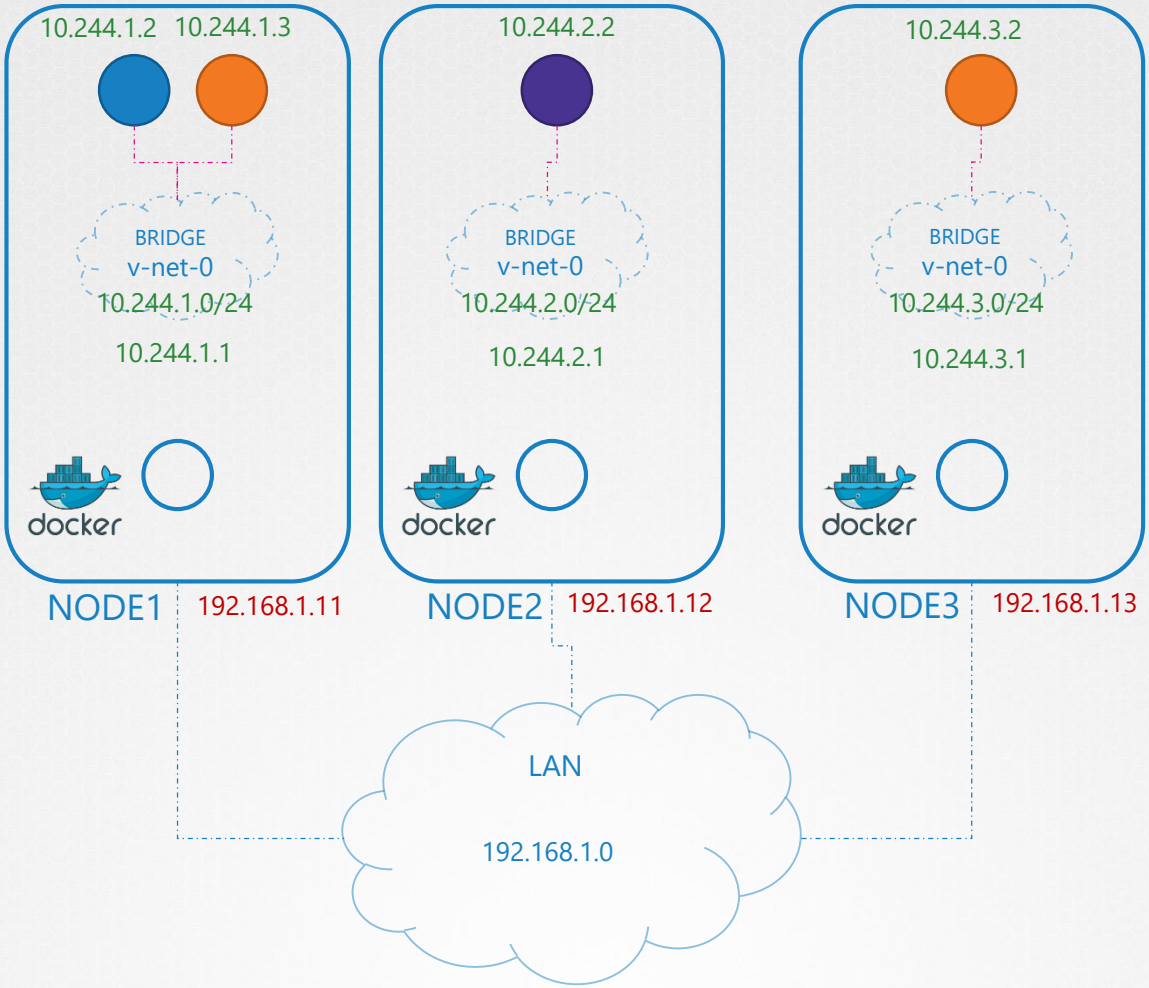
KODEKLOUD

Course Objectives

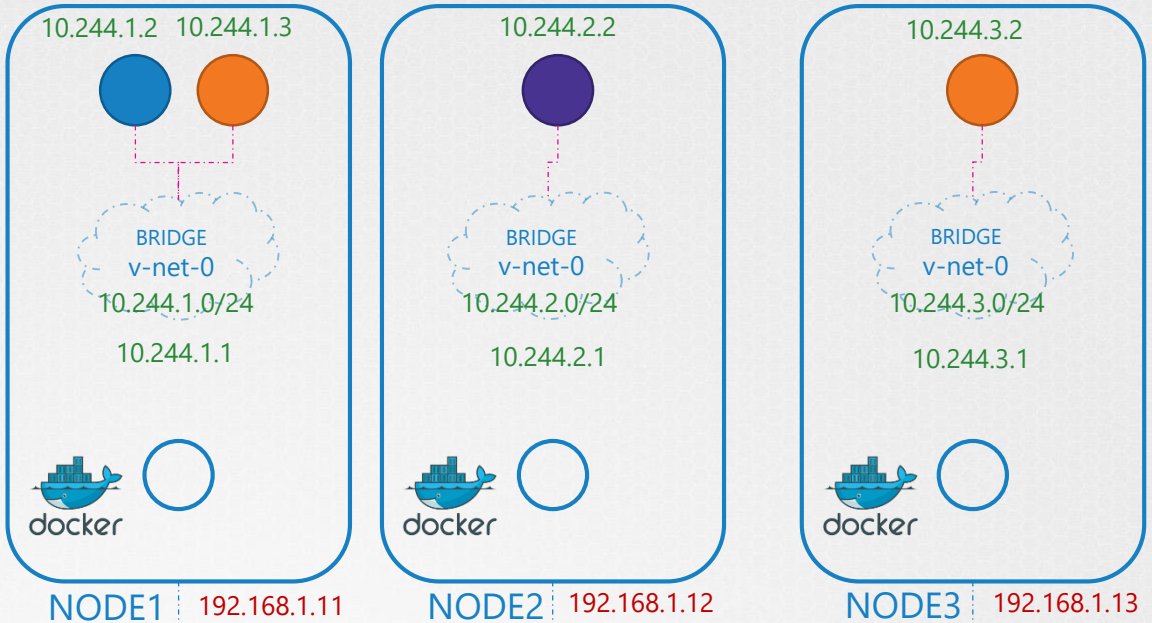
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Troubleshooting
- Core Concepts
- Networking
  - Pre-Requisites – Switching, Routing
  - Pre-Requisites – DNS
  - Networking Configuration on Cluster Nodes
  - Cluster DNS
  - Service Networking
  - Pre-Requisites – Tools
  - Pre-Requisites – Networking in Docker
  - POD Networking Concepts
  - Network Loadbalancer
  - Pre-Requisites CNI
  - CNI in Kubernetes
  - Ingress

KODEKLOUD

IPAM (CNI)


10.244.1.2
10.244.1.3
BRIDGE
v-net-0
10.244.1.0/24
10.244.1.1
docker
NODE1
192.168.1.11
10.244.2.2
BRIDGE
v-net-0
10.244.2.0/24
10.244.2.1
docker
NODE2
192.168.1.12
10.244.3.2
BRIDGE
v-net-0
10.244.3.0/24
10.244.3.1
docker
NODE3
192.168.1.13
LAN
192.168.1.0

CONTAINER NETWORK INTERFACE (CNI)



CNI Plugin Responsibilities:

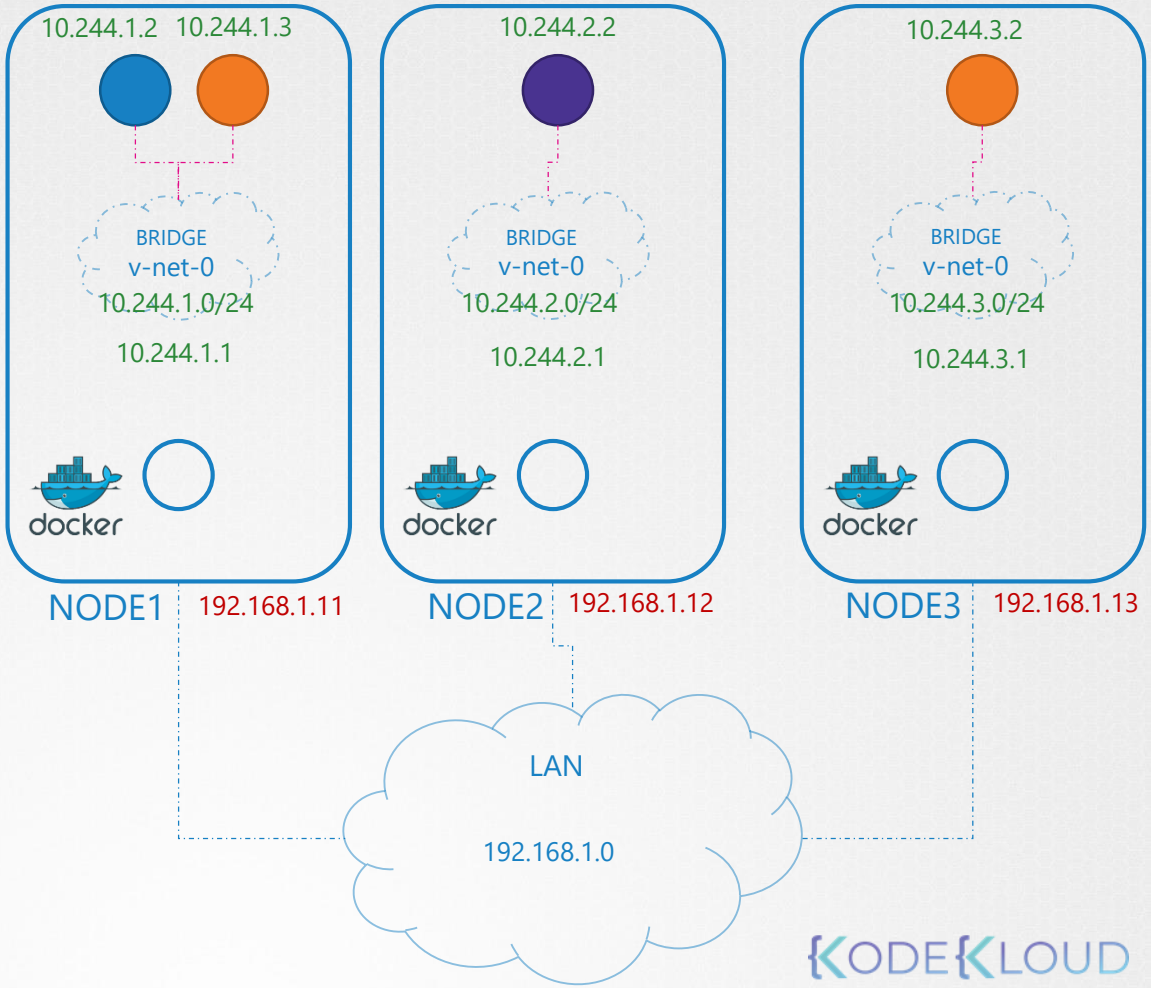
- ✓ Must support arguments ADD/DEL/CHECK
- ✓ Must support parameters container id, network ns etc..
- ✓ Must manage IP Address assignment to PODs
- ✓ Must Return results in a specific format

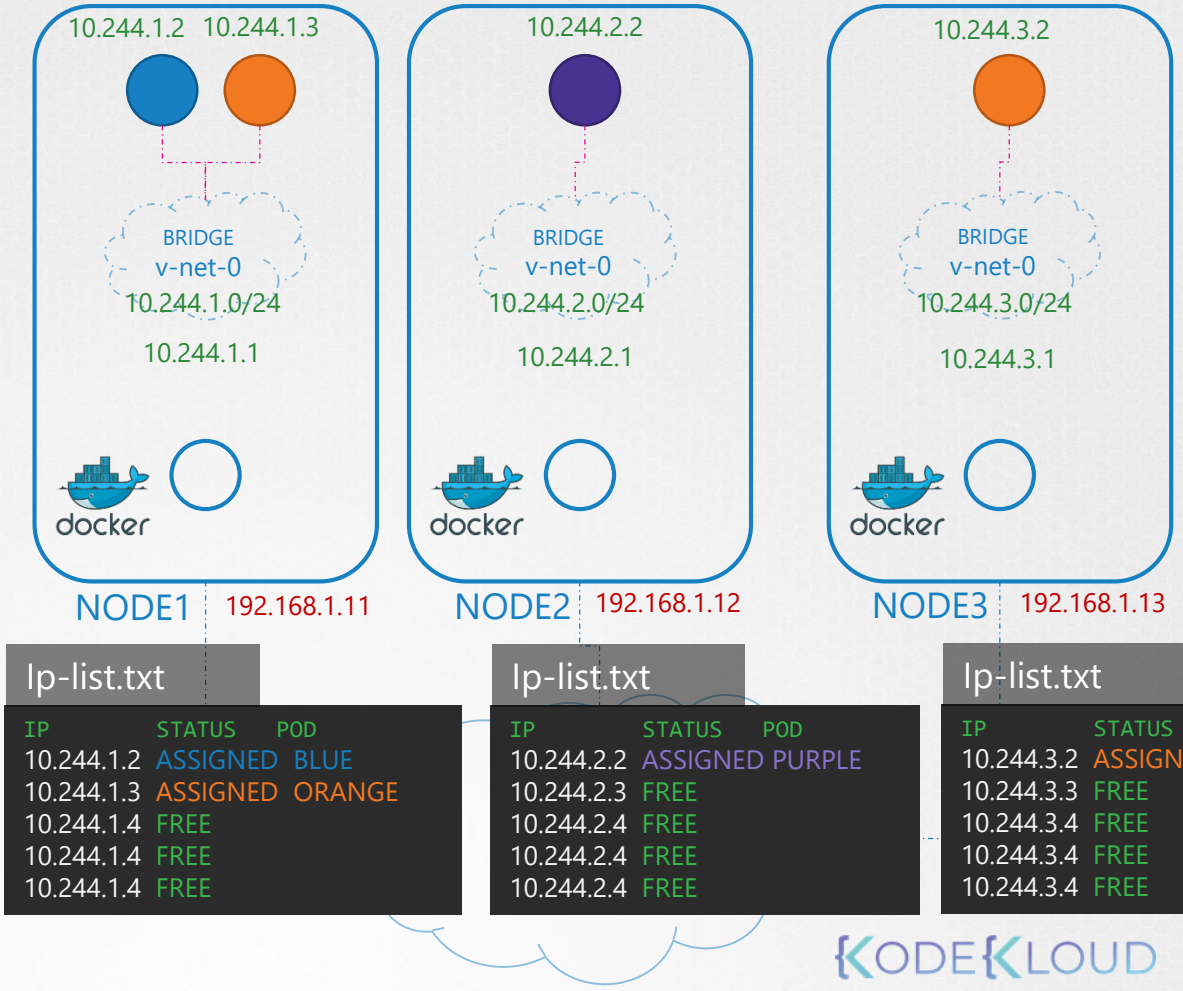
KODEKLOUD

CONTAINER NETWORK INTERFACE (CNI)

net-script.sh

```
ADD)
  # Create veth pair
  # Attach veth pair
  # Assign IP Address
  # Bring Up Interface
  ip -n <namespace> link set ……
DEL)
  # Delete veth pair
  ip link del ……
```





CONTAINER NETWORK INTERFACE (CNI)

DHCP

host-local

```
# Invoke IPAM host-local plugin
ip = get_free_ip_from_host_local()
# Assign IP Address
ip -n <namespace> addr add ……
ip -n <namespace> route add ……
```


KODEKLOUD

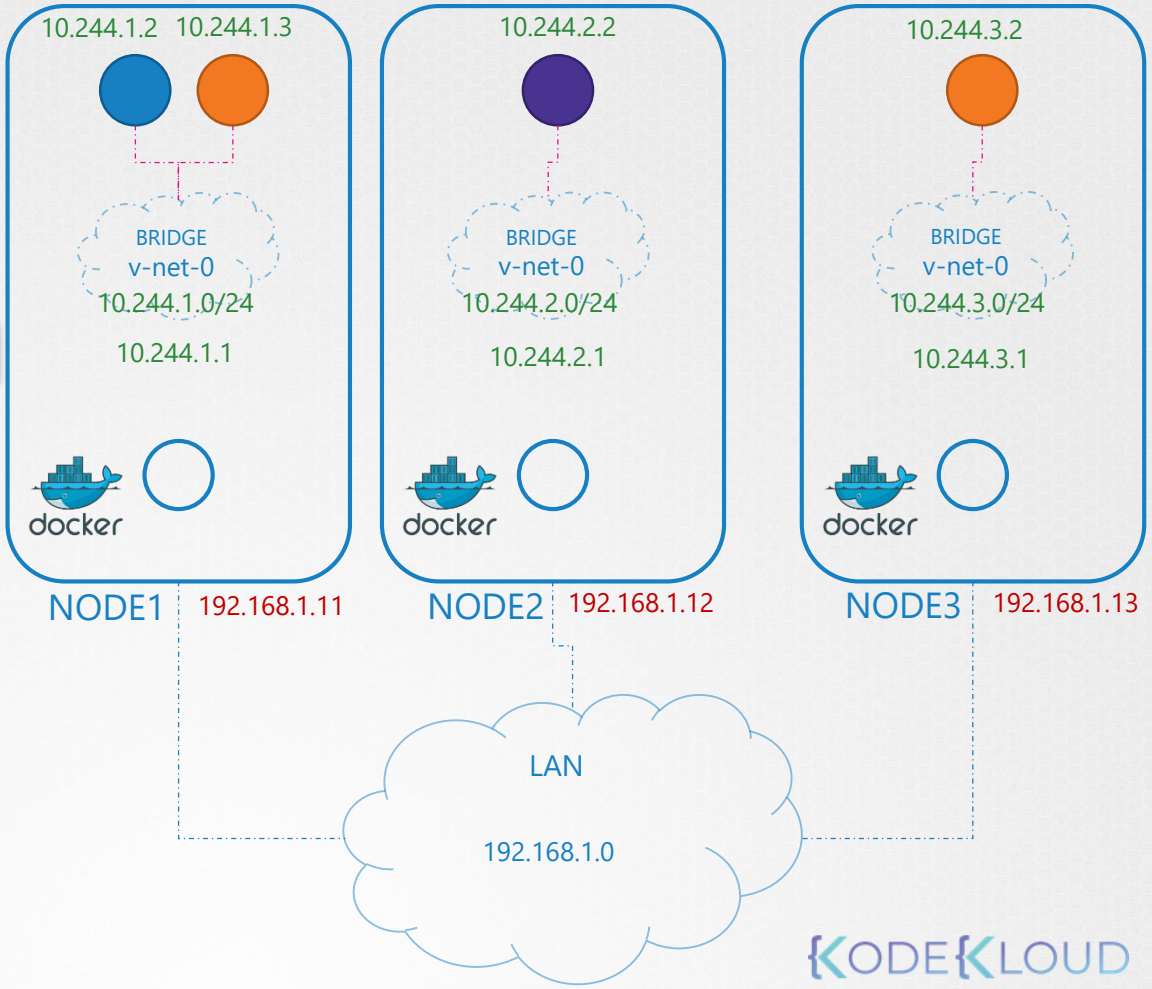
CONTAINER NETWORK INTERFACE (CNI)

DHCP

host-local

```
cat /etc/cni/net.d/net-script.conf
```

```
{
    "cniVersion": "0.2.0",
    "name": "mynet",
    "type": "net-script",
    "bridge": "cni0",
    "isGateway": true,
    "ipMasq": true,
    "ipam": {
        "type": "host-local",
        "subnet": "10.244.0.0/16",
        "routes": [
            { "dst": "0.0.0.0/0" }
        ]
    }
}
```



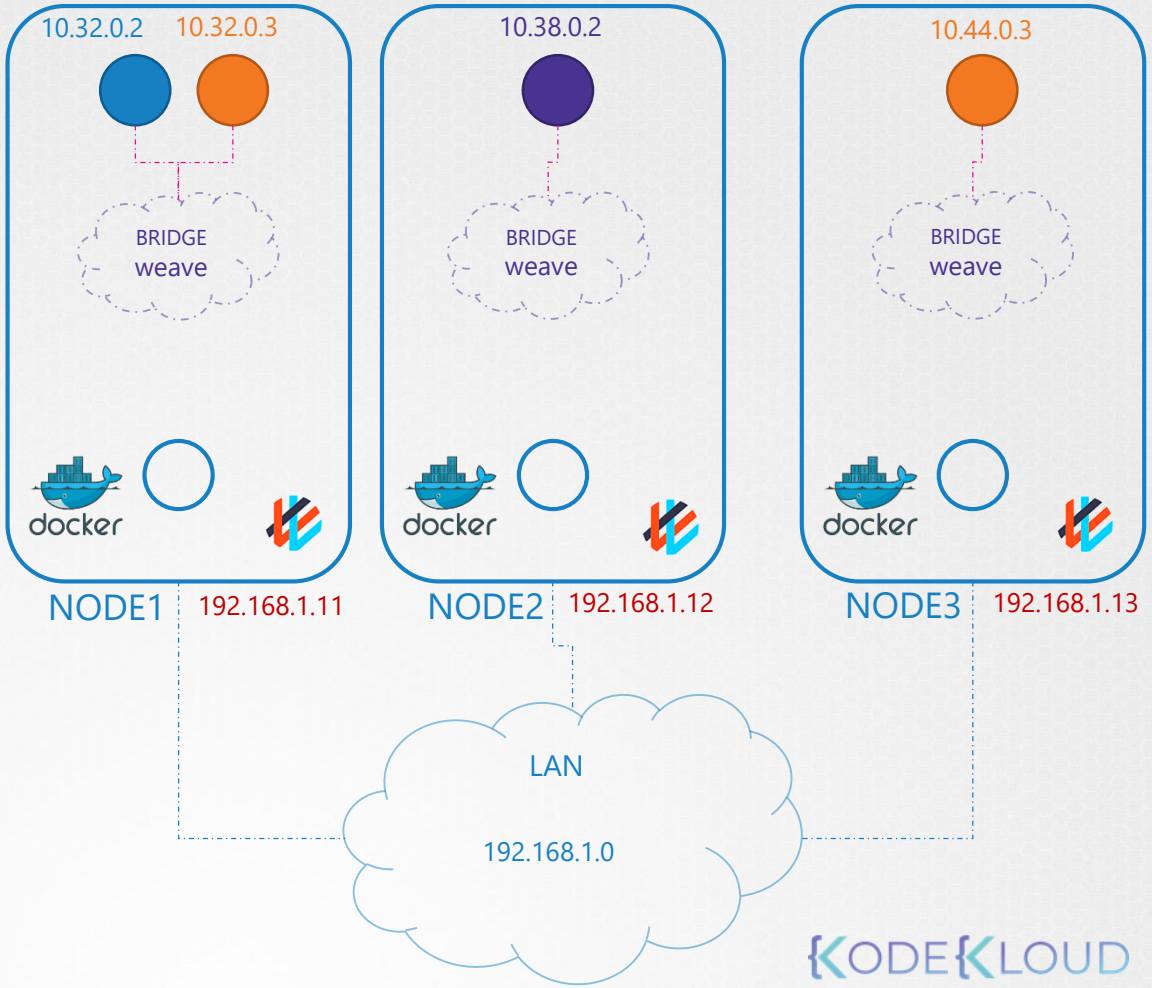
KODEKLOUD

10.32.0.0/12

10.32.0.1 > 10.47.255.254

1,048,574 !!!!!

10.32.0.1 10.38.0.0 10.44.0.0



weaveworks

KODEKLOUD

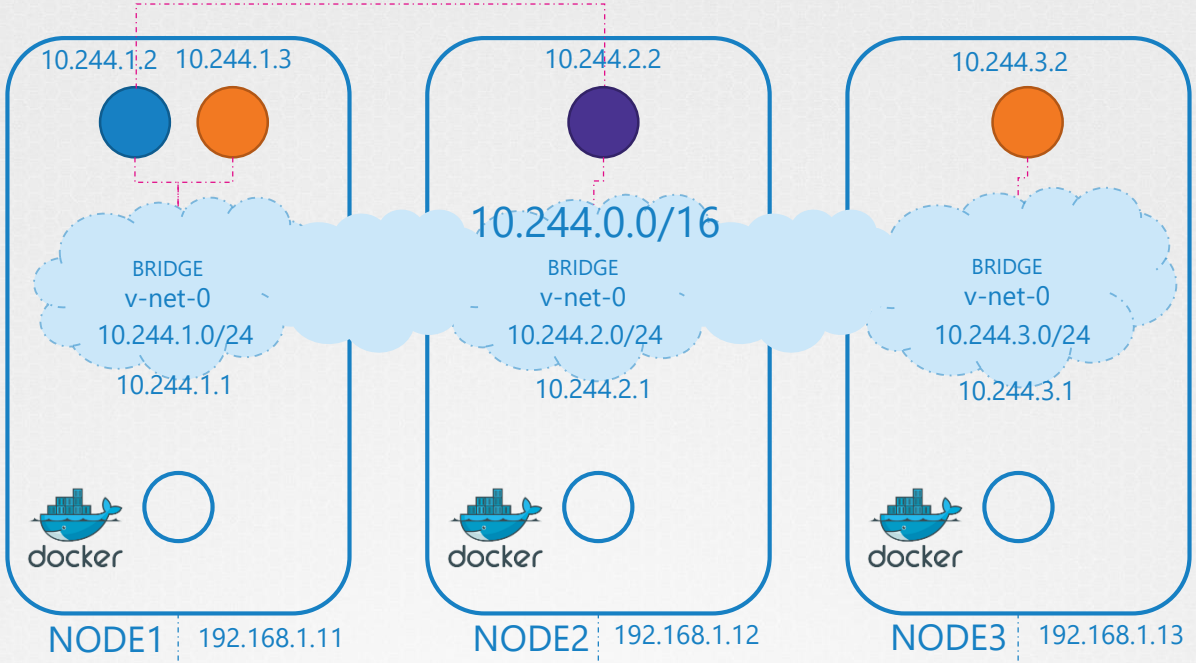
KODEKLOUD

Course Objectives

- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Troubleshooting
- Core Concepts
- Networking
  - Pre-Requisites – Switching, Routing
  - Pre-Requisites – Tools
  - Pre-Requisites CNI
  - Pre-Requisites – DNS
  - Pre-Requisites – Networking in Docker
  - Networking Configuration on Cluster Nodes
  - POD Networking Concepts
  - CNI in Kubernetes
  - Service Networking
  - Cluster DNS
  - Network Loadbalancer
  - Ingress

KODEKLOUD

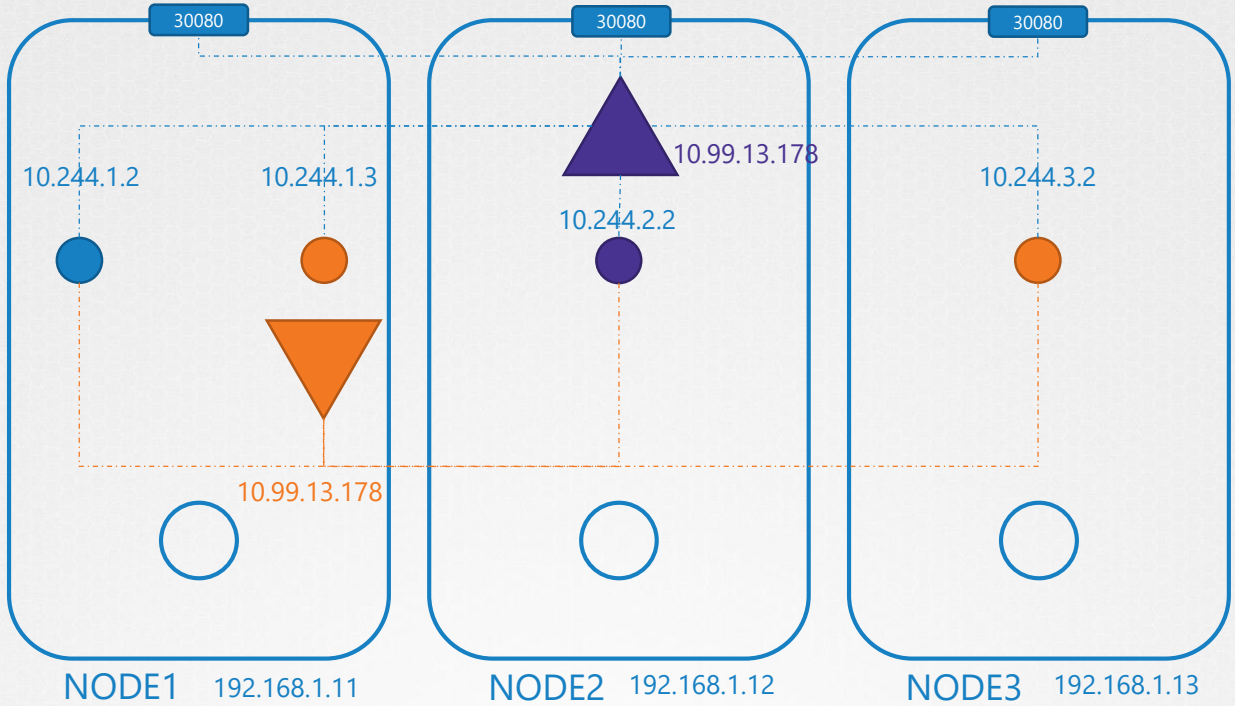
Service Networking



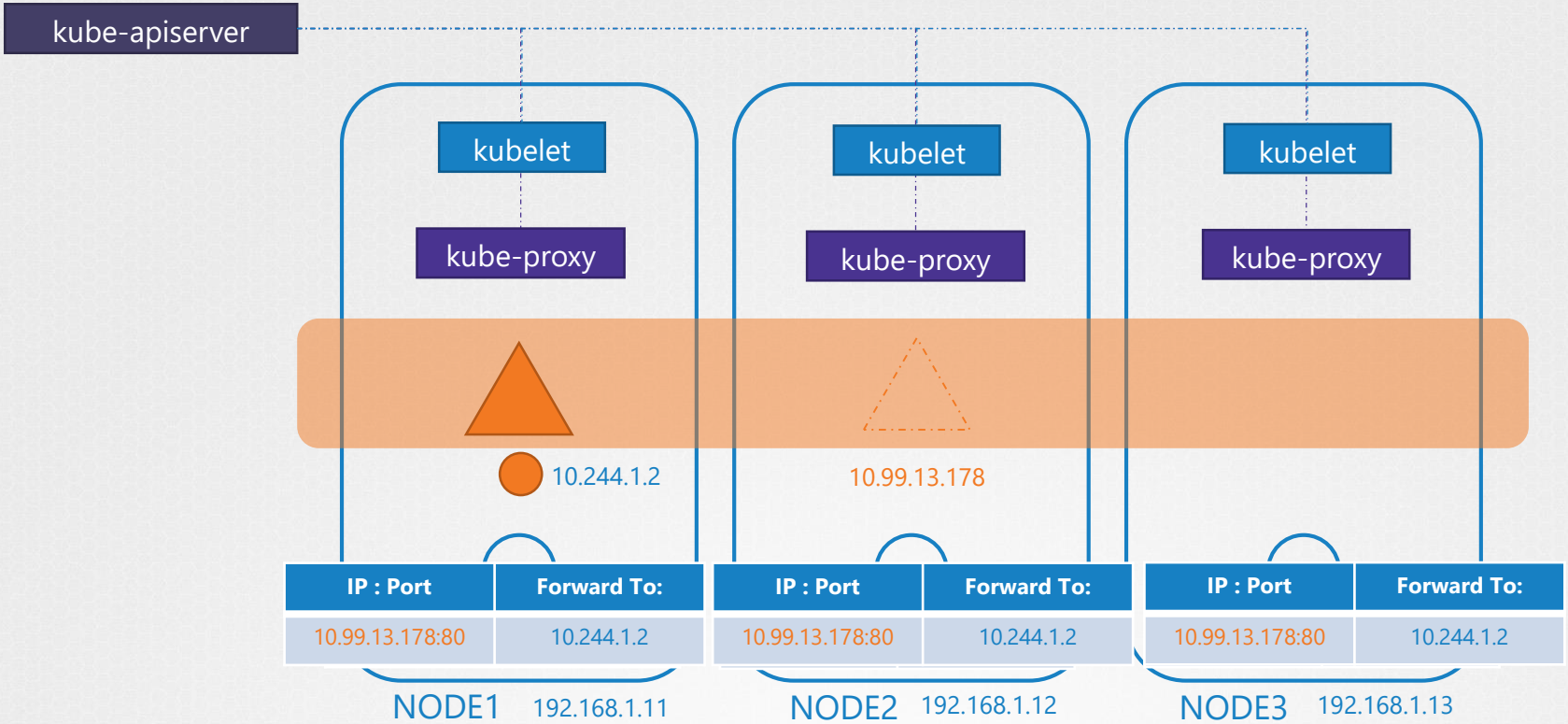
NETWORK	GATEWAY
10.244.1.0/24	192.168.1.11
10.244.2.0/24	192.168.1.12
10.244.3.0/24	192.168.1.13

KODEKLOUD

NodePort



KODEKLOUD


kube-apiserver
kubelet
kube-proxy
kubelet
kube-proxy
kubelet
kube-proxy
10.244.1.2
10.99.13.178
IP : Port
Forward To:
10.99.13.178:80
10.244.1.2
IP : Port
Forward To:
10.99.13.178:80
10.244.1.2
IP : Port
Forward To:
10.99.13.178:80
10.244.1.2
NODE1 192.168.1.11
NODE2 192.168.1.12
NODE3 192.168.1.13

KODEKLOUD

kube-proxy

userspace | iptables | ipvs

```
kube-proxy --proxy-mode [userspace | iptablkes | ipvs ] …
```

IP : Port	Forward To:
10.99.13.178:80	10.244.1.2

KODEKLOUD

kube-proxy

iptables

IP : Port	Forward To:
10.99.13.178:80	10.244.1.2

```
kubelet get pods -o wide
```

```
NAME        READY     STATUS    RESTARTS   AGE       IP           NODE
db          1/1       Running   0          14h       10.244.1.2   node-1
```

```
kubelet get service
```

```
NAME             TYPE        CLUSTER-IP       PORT(S)          AGE
db-service       ClusterIP   10.103.132.104   3306/TCP         12h
```

10.244.1.2

10.244.0.0/16

10.244.0.0 => 10.244.255.255

10.96.0.0 => 10.111.255.255

```
kube-api-server --service-cluster-ip-range ipNet (Default: 10.0.0.0/24)
```

```
ps aux | grep kube-api-server
```

```
kube-apiserver --authorization-mode=Node,RBAC --service-cluster-ip-
range=10.96.0.0/12
```

KODEKLOUD

kube-proxy

iptables

IP : Port	Forward To:
10.99.13.178:80	10.244.1.2

```
kubelet get pods -o wide
```

```
NAME       READY     STATUS    RESTARTS   AGE       IP           NODE
db         1/1       Running   0          14h       10.244.1.2   node-1
```

```
kubelet get service
```

```
NAME             TYPE        CLUSTER-IP       PORT(S)          AGE
db-service       ClusterIP   10.103.132.104   3306/TCP         12h
```

```
iptables -L -t net | grep db-service
```

```
KUBE-SVC-XA5OGUC7YRHOS3PU  tcp  --  anywhere   10.103.132.104   /* default/db-service: cluster IP */ tcp dpt:3306
DNAT                       tcp  --  anywhere   anywhere         /* default/db-service: */ tcp to:10.244.1.2:3306
KUBE-SEP-JBWCWHHQM57V2WN7  all  --  anywhere   anywhere         /* default/db-service: */
```

KODEKLOUD

kube-proxy

iptables

IP : Port	Forward To:
10.99.13.178:80	10.244.1.2

```
iptables -L -t net | grep db-service
```

```
KUBE-SVC-XA5OGUC7YRHOS3PU  tcp  --  anywhere   10.103.132.104   /* default/db-service: cluster IP */ tcp dpt:3306
DNAT                       tcp  --  anywhere   anywhere         /* default/db-service: */ tcp to:10.244.1.2:3306
KUBE-SEP-JBWCWHHQM57V2WN7  all  --  anywhere   anywhere         /* default/db-service: */
```

```
cat /var/log/kube-proxy.log
```

```
I0307 04:29:29.883941       1 server_others.go:140] Using iptables Proxier.
I0307 04:29:29.912037       1 server_others.go:174] Tearing down inactive rules.
I0307 04:29:30.027360       1 server.go:448] Version: v1.11.8
I0307 04:29:30.049773       1 conntrack.go:98] Set sysctl 'net/netfilter/nf_conntrack_max' to 131072
I0307 04:29:30.049945       1 conntrack.go:52] Setting nf_conntrack_max to 131072
I0307 04:29:30.050701       1 conntrack.go:83] Setting conntrack hashsize to 32768
I0307 04:29:30.050701       1 proxier.go:294] Adding new service “default/db-service:3306" at 10.103.132.104:3306/TCP
```

KODEKLOUD

KODEKLOUD

Course Objectives

- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Troubleshooting
- Core Concepts
- Networking
  - Pre-Requisites – Switching, Routing
  - Pre-Requisites – DNS
  - Networking Configuration on Cluster Nodes
  - Service Networking
  - Cluster DNS
  - Pre-Requisites – Tools
  - Pre-Requisites – Networking in Docker
  - POD Networking Concepts
  - Network Loadbalancer
  - Pre-Requisites CNI
  - CNI in Kubernetes
  - Ingress

KODEKLOUD

Cluster DNS

Pre-Requisite

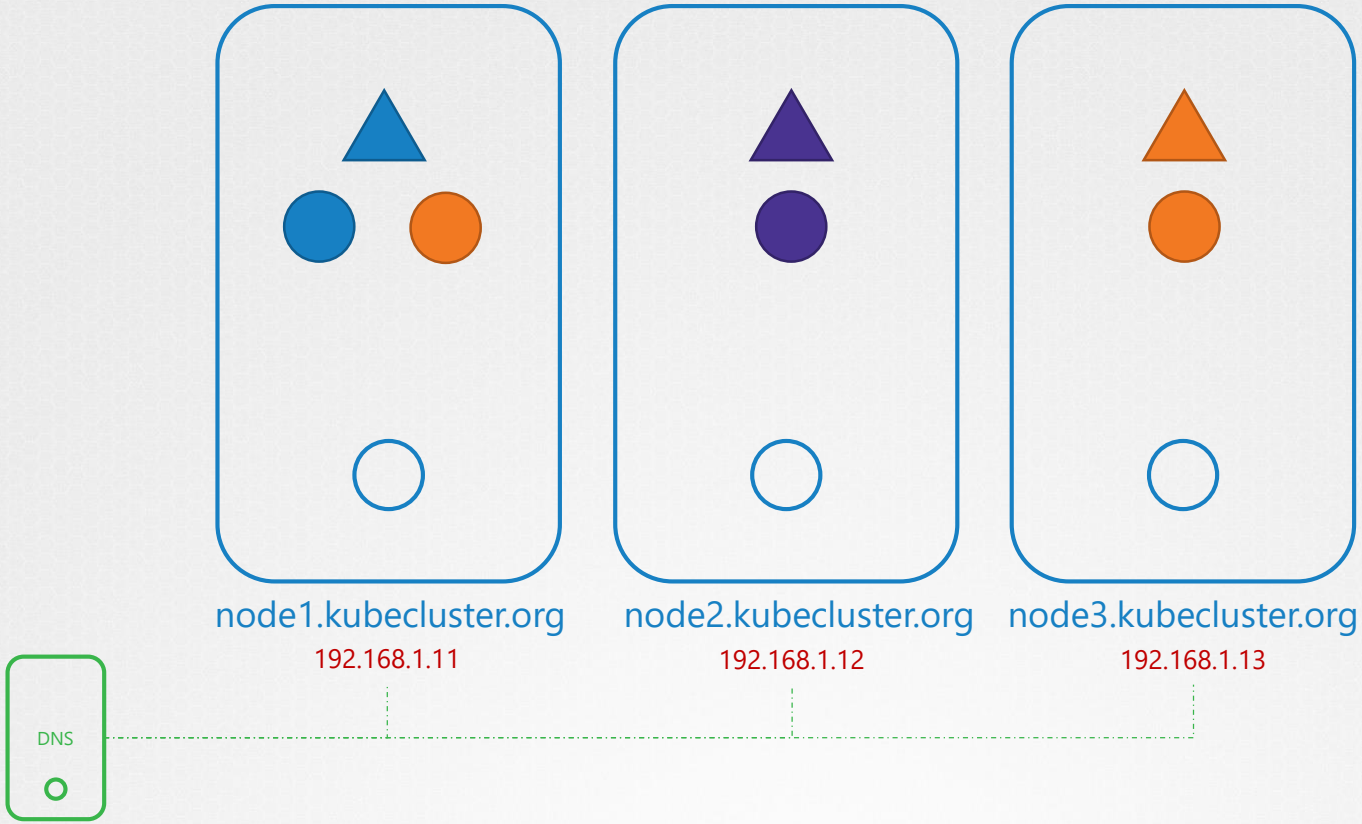
- ✓ What is DNS?
- ✓ Host/NS Lookup, Dig utility
- ✓ Recorded Types – A, CNAME
- ✓ Domain Name Hierarchy

KODEKLOUD

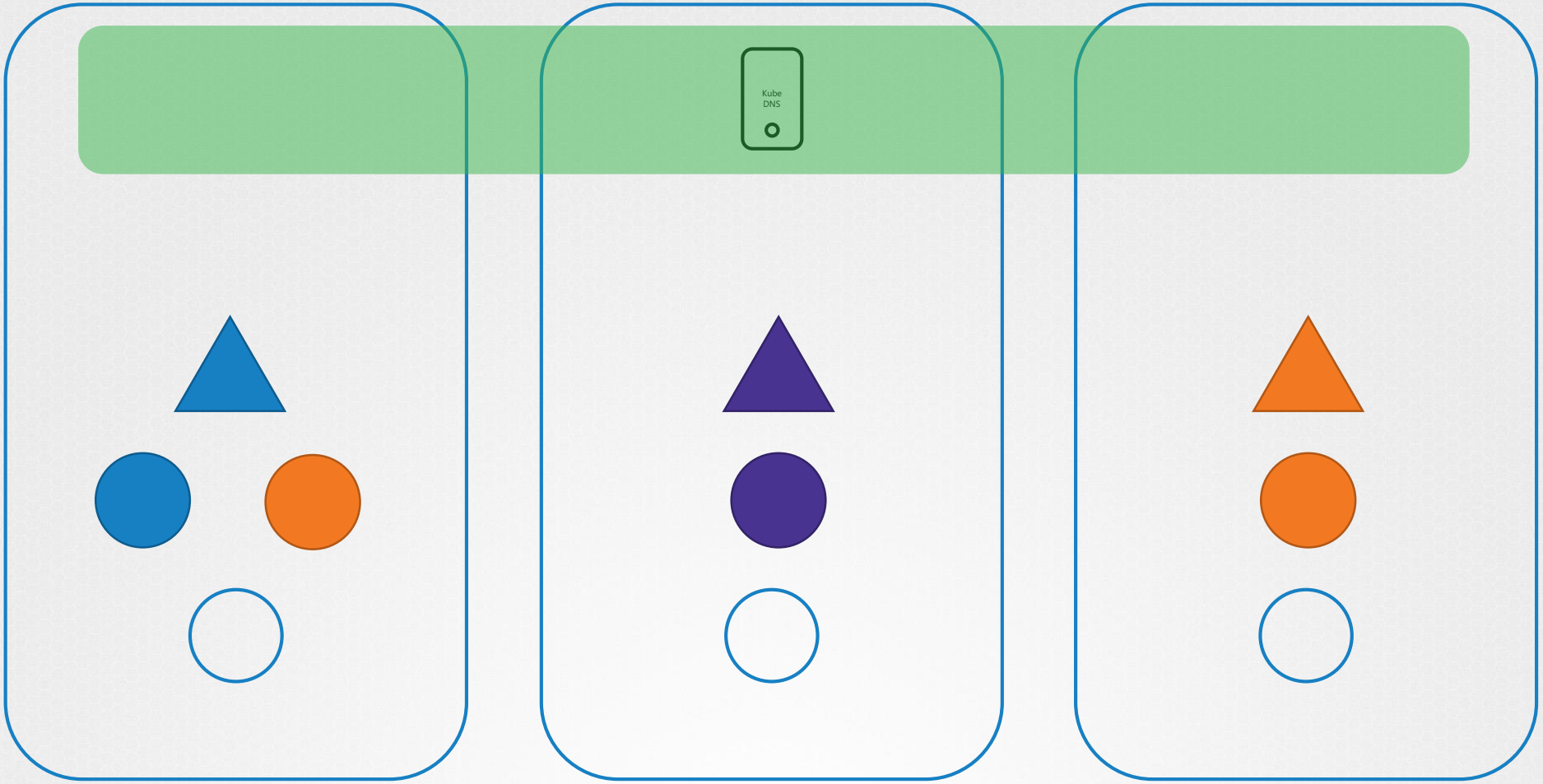
Objectives

- ❑ What names are assigned to what objects?
- ❑ Service DNS records
- ❑ POD DNS Recrods

KODEKLOUD


node1.kubecluster.org
192.168.1.11
node2.kubecluster.org
192.168.1.12
node3.kubecluster.org
192.168.1.13
DNS

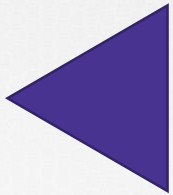
KODEKLOUD


Kube
DNS
node1.kubecluster.org
192.168.1.11
node2.kubecluster.org
192.168.1.12
node3.kubecluster.org
192.168.1.13

Hostname	IP Address
web-service	10.107.37.188

10.244.1.5

test



10.107.37.188

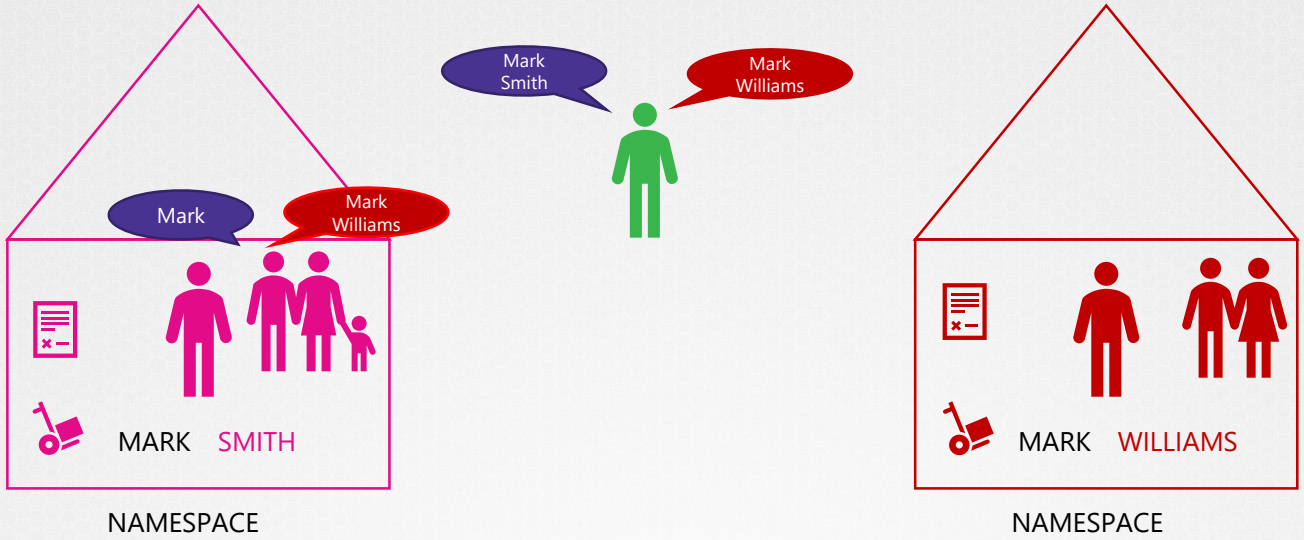
web-service

10.244.2.5

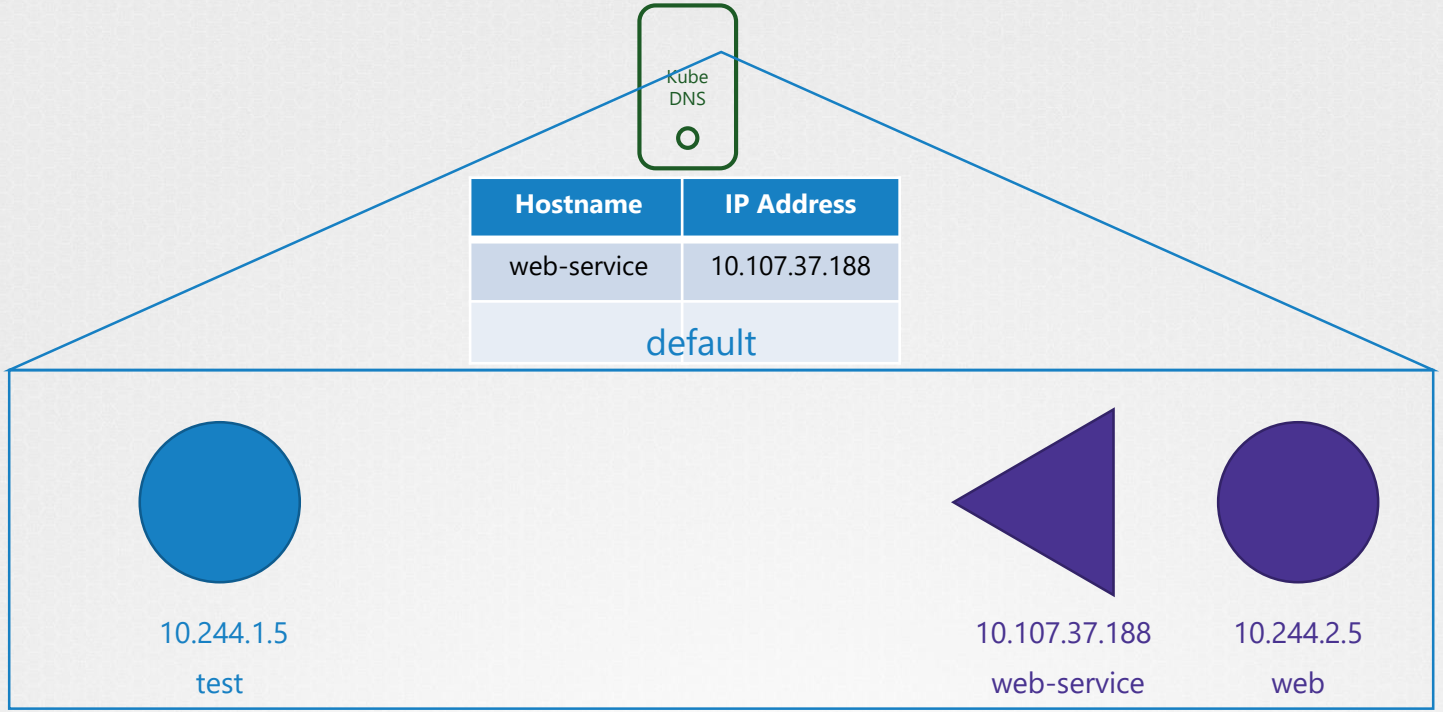
web

```
curl http://web-service
Welcome to NGINX!
```

KODEKLOUD

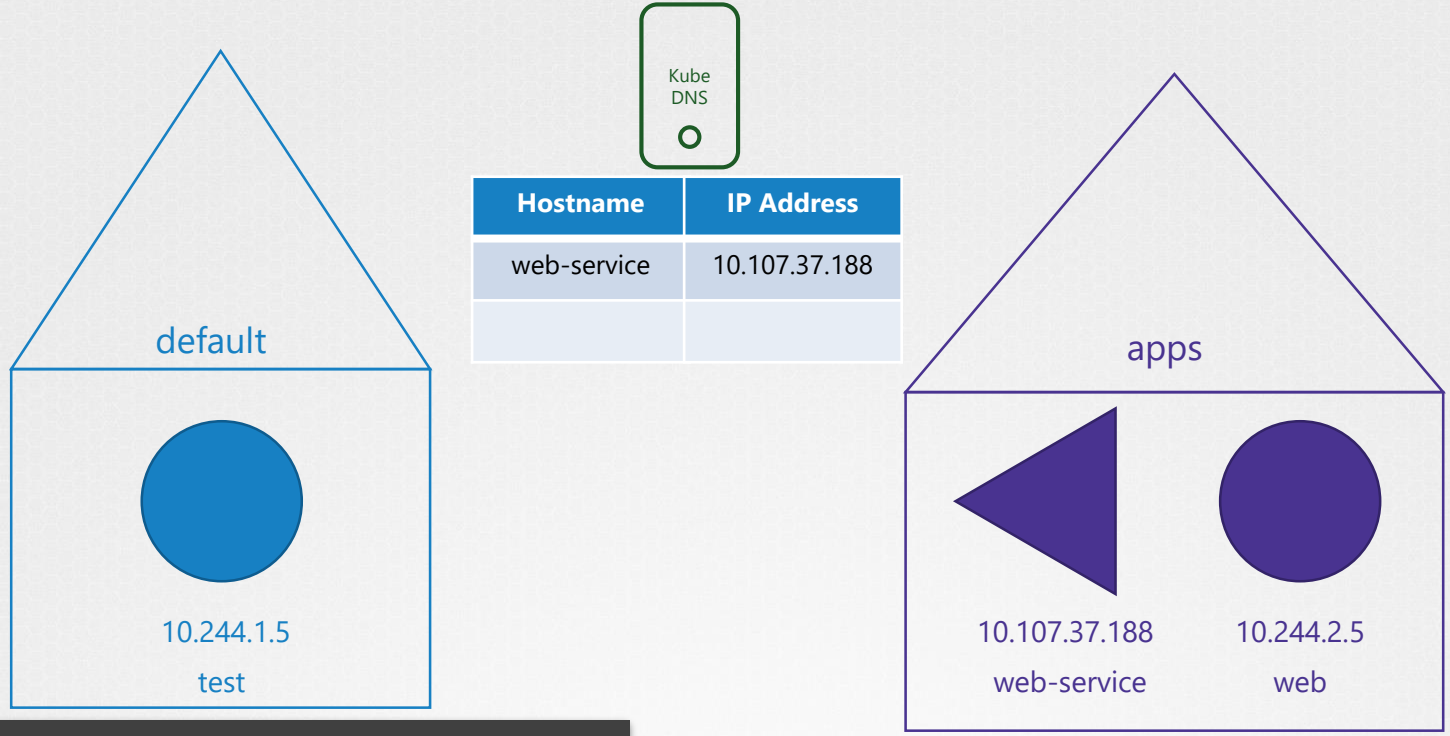

Mark
Smith
Mark
Williams
Mark
Mark
Williams
MARK SMITH
NAMESPACE
MARK WILLIAMS
NAMESPACE

KODEKLOUD




```
curl http://web-service
Welcome to NGINX!
```

KODEKLOUD

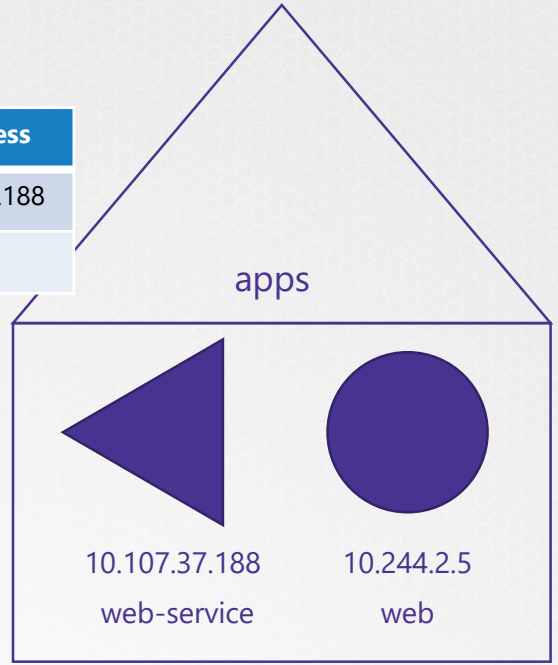


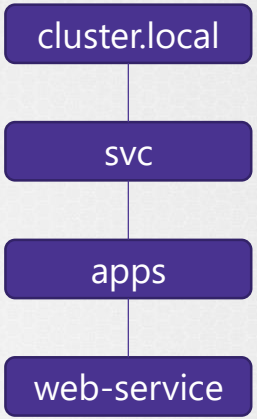

```
curl http://web-service
Welcome to NGINX!
```

```
curl http://web-service.apps
Welcome to NGINX!
```

KODEKLOUD

Hostname	Namespace	Type	Root	IP Address
web-service	apps	svc	cluster.local	10.107.37.188






```
curl http://web-service.apps
Welcome to NGINX!
```

```
curl http://web-service.apps.svc
Welcome to NGINX!
```

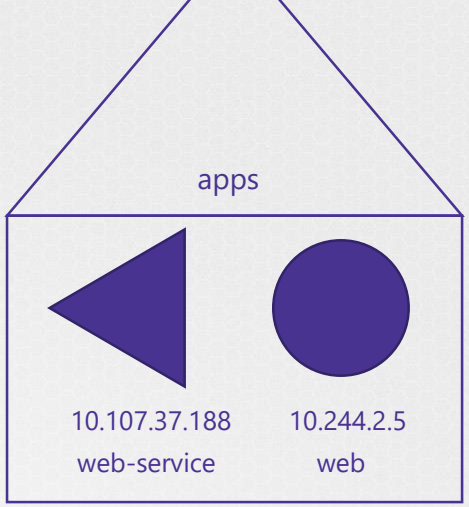
```
curl http://web-service.apps.svc.cluster.local
Welcome to NGINX!
```

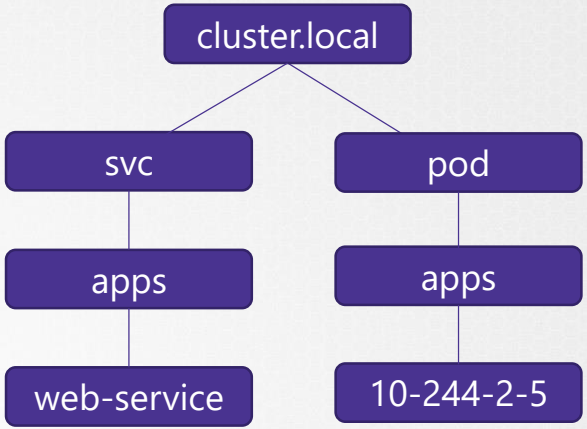
KODEKLOUD

Hostname	Namespace	Type	Root	IP Address
web-service	apps	svc	cluster.local	10.107.37.188
10-244-2-5	apps	pod	cluster.local	10.244.2.5

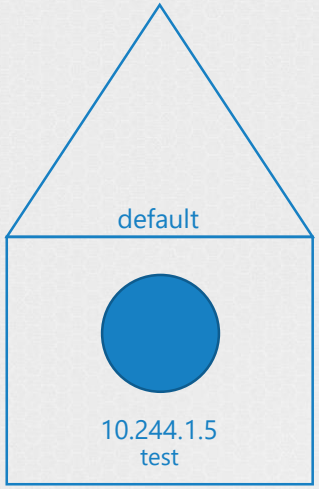
10-244-2-5 ⟵ 10.244.2.5

```
curl http://10-244-2-5.apps.pod.cluster.local
Welcome to NGINX!
```



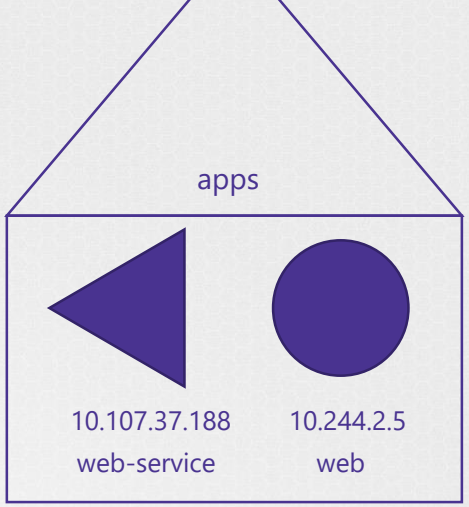


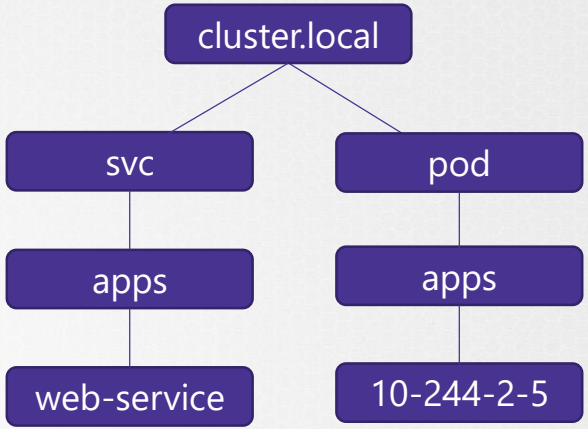
KODEKLOUD



Kube DNS

Hostname	Namespace	Type	Root	IP Address
web-service	apps	svc	cluster.local	10.107.37.188
10-244-2-5	apps	pod	cluster.local	10.244.2.5
10-244-1-5	default	pod	cluster.local	10.244.1.5






```
curl http://10-244-2-5.apps.pod.cluster.local
Welcome to NGINX!
```

KODEKLOUD

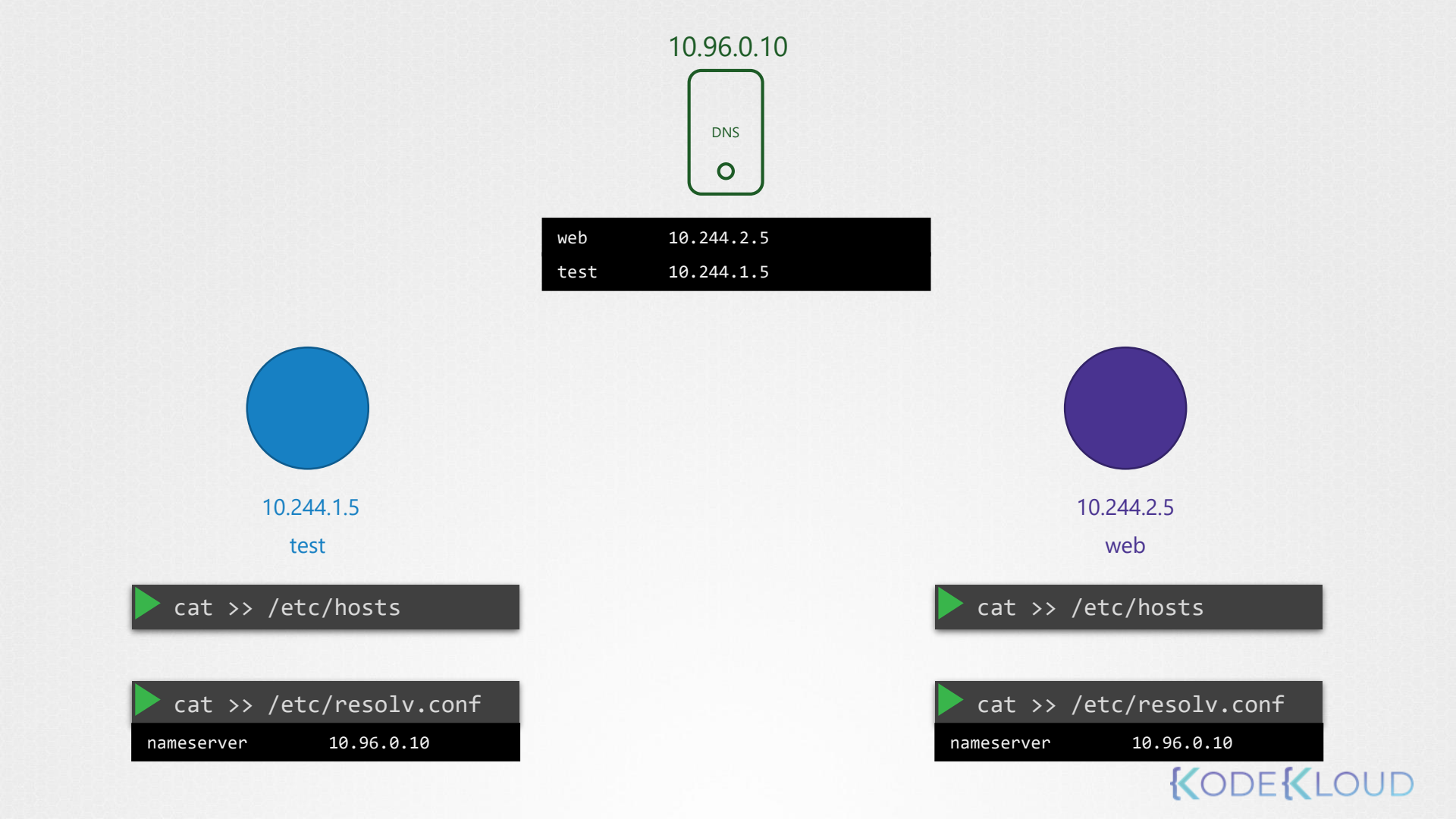
KODEKLOUD

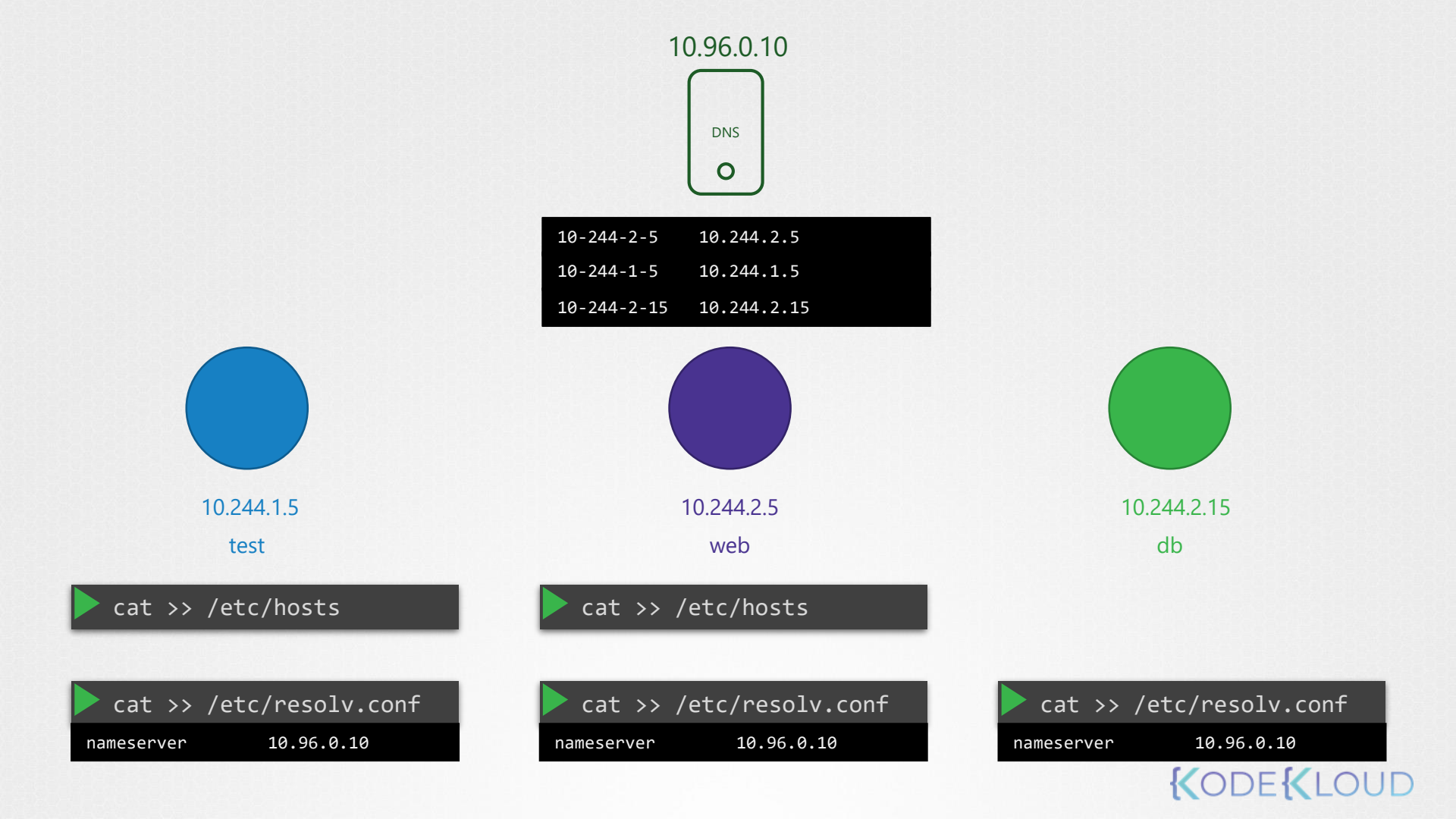
Course Objectives

- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Troubleshooting
- Core Concepts
- Networking
  - Pre-Requisites – Switching, Routing
  - Pre-Requisites – DNS
  - Networking Configuration on Cluster Nodes
  - Service Networking
  - Cluster DNS
  - Pre-Requisites – Tools
  - Pre-Requisites – Networking in Docker
  - POD Networking Concepts
  - Network Loadbalancer
  - Pre-Requisites CNI
  - CNI in Kubernetes
  - Ingress

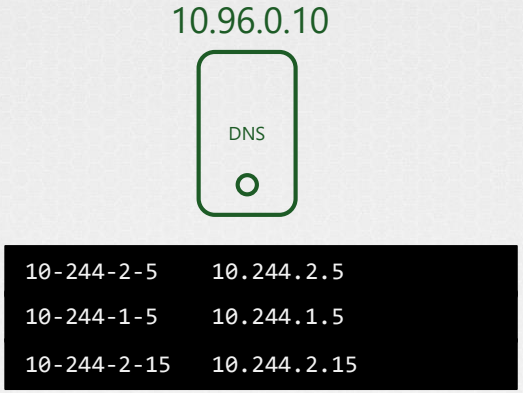
KODEKLOUD

How Kubernetes Implements DNS?


10.96.0.10
DNS
web 10.244.2.5
test 10.244.1.5
10.244.1.5
test
10.244.2.5
web
cat >> /etc/hosts
cat >> /etc/resolv.conf
nameserver 10.96.0.10
cat >> /etc/hosts
cat >> /etc/resolv.conf
nameserver 10.96.0.10


10.96.0.10
DNS
10-244-2-5 10.244.2.5
10-244-1-5 10.244.1.5
10-244-2-15 10.244.2.15
10.244.1.5
test
10.244.2.5
web
10.244.2.15
db
cat >> /etc/hosts
cat >> /etc/hosts
cat >> /etc/resolv.conf
nameserver 10.96.0.10
cat >> /etc/resolv.conf
nameserver 10.96.0.10
cat >> /etc/resolv.conf
nameserver 10.96.0.10
KODEKLOUD

kube-dns

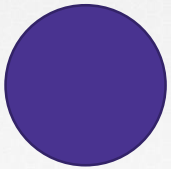


10.244.1.5

test

```
cat >> /etc/hosts
```

```
cat >> /etc/resolv.conf
nameserver 10.96.0.10
```



10.244.2.5

web

```
cat >> /etc/hosts
```

```
cat >> /etc/resolv.conf
nameserver 10.96.0.10
```

10.244.2.15

db

```
cat >> /etc/resolv.conf
nameserver 10.96.0.10
```

KODEKLOUD

CoreDNS

10.96.0.10

DNS

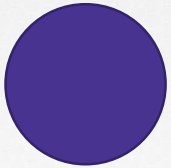
```
10-244-2-5     10.244.2.5
10-244-1-5     10.244.1.5
10-244-2-15    10.244.2.15
```

10.244.1.5

test

```
cat >> /etc/hosts
```

```
cat >> /etc/resolv.conf
nameserver        10.96.0.10
```



10.244.2.5

web

```
cat >> /etc/hosts
```

```
cat >> /etc/resolv.conf
nameserver        10.96.0.10
```

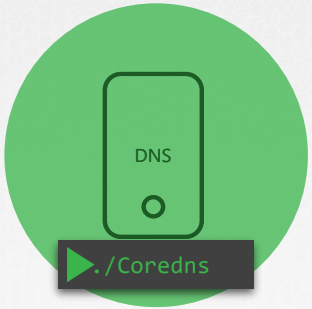
10.244.2.15

db

```
cat >> /etc/resolv.conf
nameserver        10.96.0.10
```

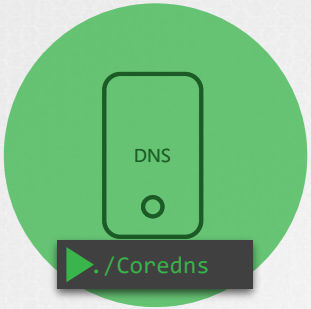
KODEKLOUD

CoreDNS



KODEKLOUD

CoreDNS

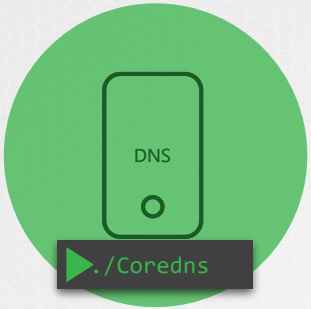



```
cat /etc/coredns/Corefile
.:53 {
    errors
    health
    kubernetes cluster.local in-addr.arpa ip6.arpa {
       pods insecure
       upstream
       fallthrough in-addr.arpa ip6.arpa
    }
    prometheus :9153
    proxy . /etc/resolv.conf
    cache 30
    reload
}
```

10-244-1-5	default	pod	cluster.local	10.244.1.5

KODEKLOUD

CoreDNS




```
cat /etc/coredns/Corefile
```

```
.:53 {
    errors
    health
    kubernetes cluster.local in-addr.arpa ip6.arpa {
       pods insecure
       upstream
       fallthrough in-addr.arpa ip6.arpa
    }
    prometheus :9153
    proxy . /etc/resolv.conf
    cache 30
    reload
}
```

```
kubectl get configmap -n kube-system
```

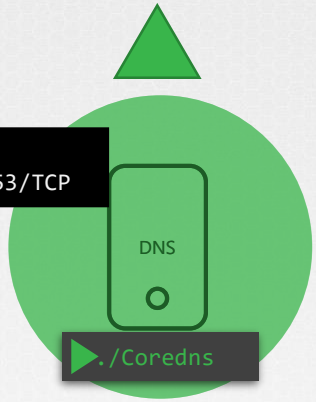
```
NAME                                 DATA      AGE
coredns                              1         168d
```

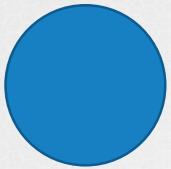
KODEKLOUD

CoreDNS

```
kubectl get service -n kube-sytem
```

```
NAME        TYPE        CLUSTER-IP    EXTERNAL-IP   PORT(S)
kube-dns    ClusterIP   10.96.0.10    <none>        53/UDP,53/TCP
```





10.244.1.5

test

```
10-244-1-5      10.244.1.5
10-244-2-5      10.244.2.5
web-service     10.107.37.188
```

10.107.37.188

web-service

10.244.2.5

web

```
cat /etc/resolv.conf
```

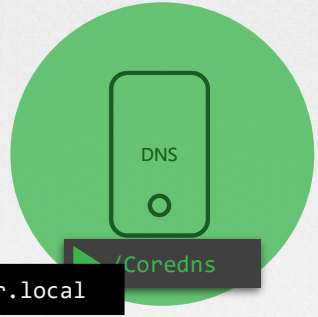
```
nameserver              10.96.0.10
```

```
cat /var/lib/kubelet/config.yaml
```

```
...
clusterDNS:
- 10.96.0.10
clusterDomain: cluster.local
```

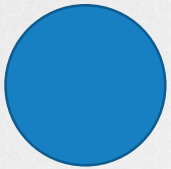
KODEKLOUD

CoreDNS




```
cat /etc/resolv.conf
nameserver              10.96.0.10
search default.svc.cluster.local svc.cluster.local cluster.local
```

```
10-244-1-5      10.244.1.5
10-244-2-5      10.244.2.5
web-service     10.107.37.188
```



10.244.1.5

test

10.107.37.188

web-service

10.244.2.5

web

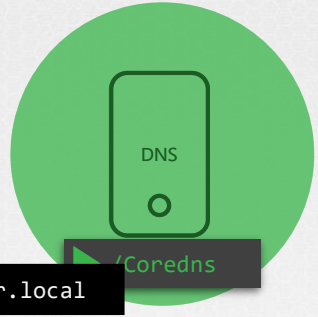
```
host web-service
web-service.default.svc.cluster.local has address 10.97.206.196
```

```
host web-service.default
web-service.default.svc.cluster.local has address 10.97.206.196
```

```
host web-service.default.svc
web-service.default.svc.cluster.local has address 10.97.206.196
```

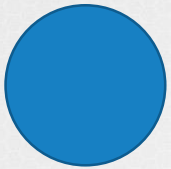
KODEKLOUD

CoreDNS




```
cat /etc/resolv.conf
nameserver              10.96.0.10
search default.svc.cluster.local svc.cluster.local cluster.local
```

```
10-244-1-5      10.244.1.5
10-244-2-5      10.244.2.5
web-service     10.107.37.188
```



10.244.1.5

test

10.107.37.188

web-service

10.244.2.5

web

```
host web-service
web-service.default.svc.cluster.local has address 10.97.206.196
```

```
host 10-244-2-5
Host 10-244-2-5 not found: 3(NXDOMAIN)
```

```
host 10-244-2-5.default.pod.cluster.local
10-244-2-5.default.pod.cluster.local has address 10.244.2.5
```

KODEKLOUD

KODEKLOUD

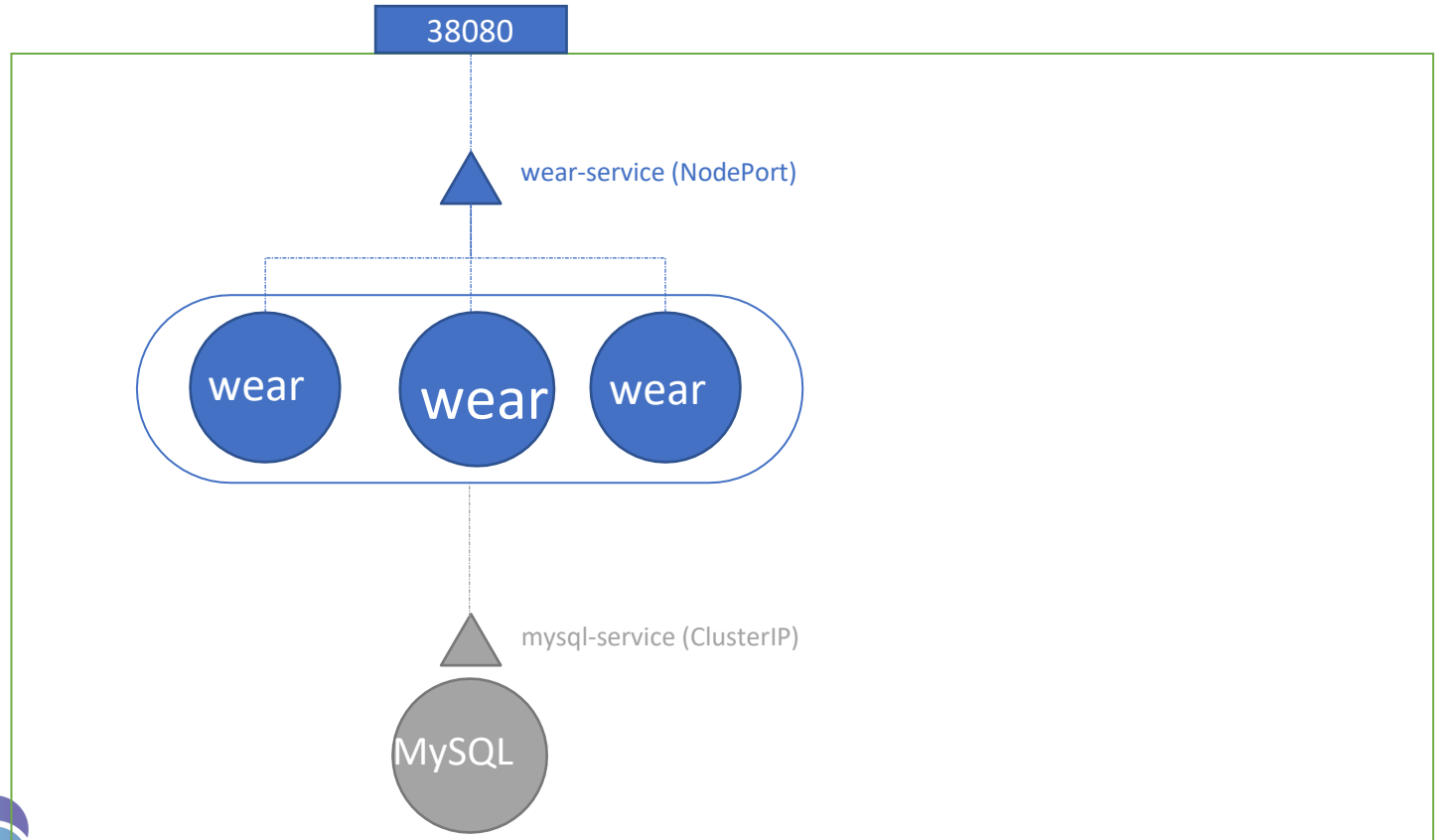
INGRESS

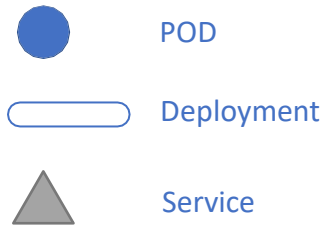
www.my-online-store.com

KODEKLOUD

www.my-online-store.com

http://<node-ip>:38080

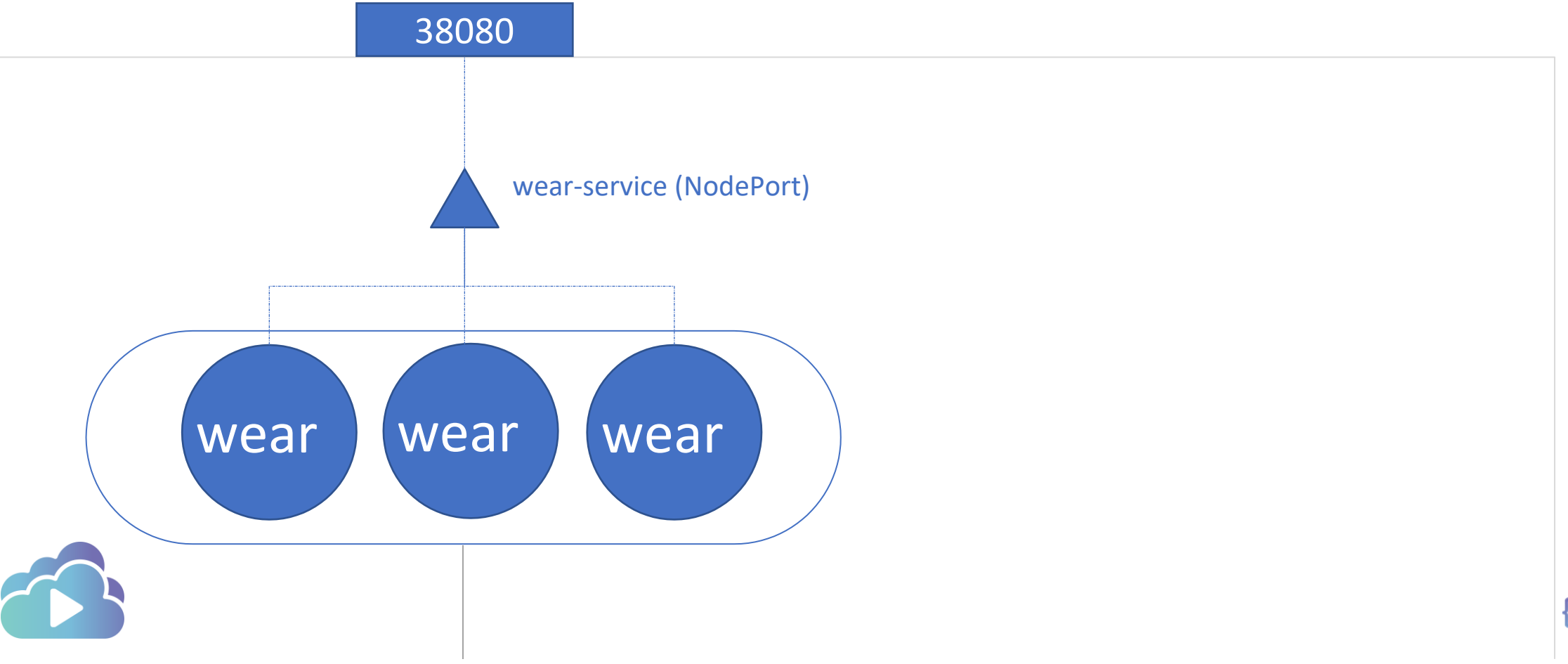




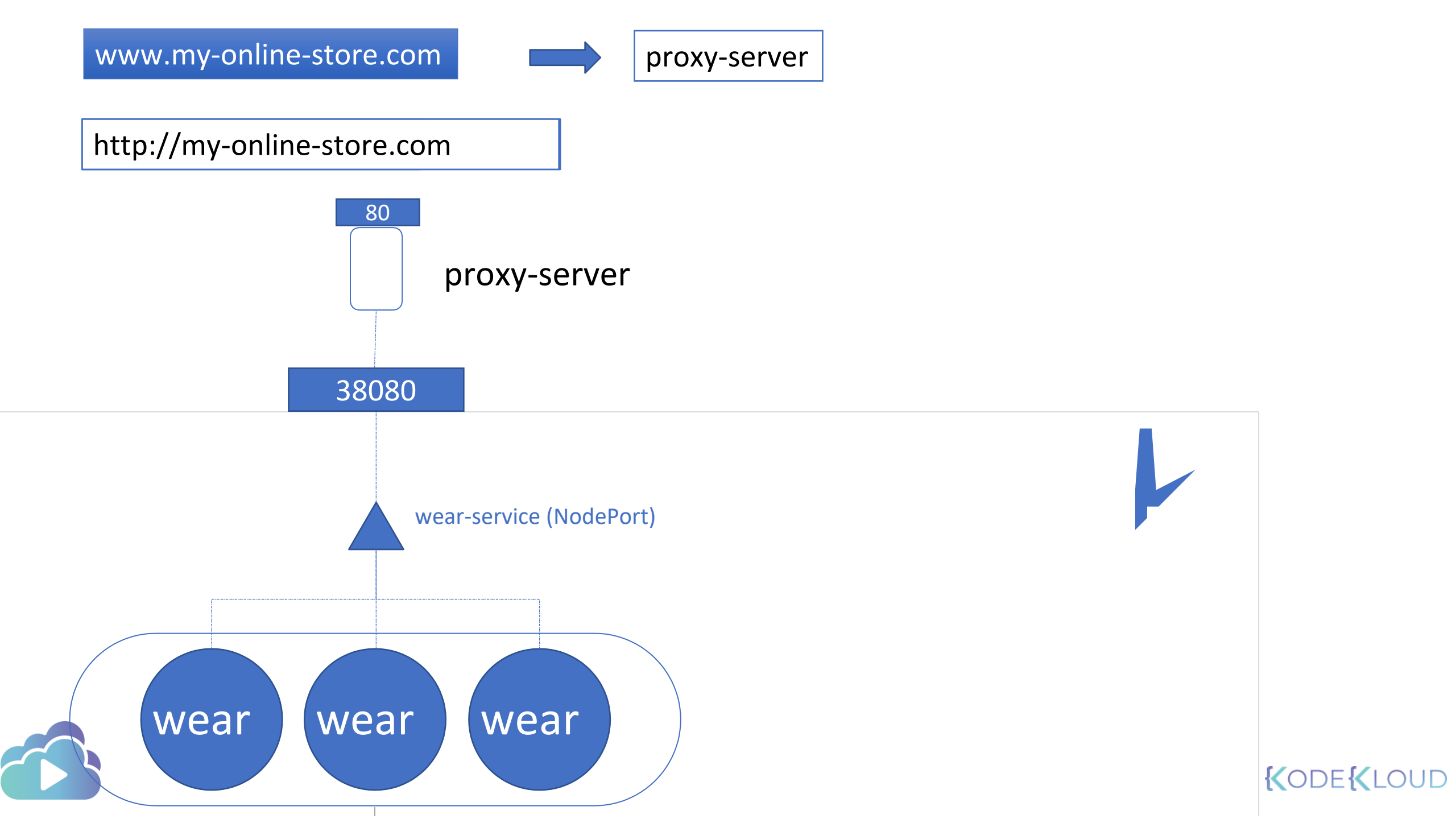
KODEKLOUD

www.my-online-store.com → <node-ip>

http://my-online-store.com:38080

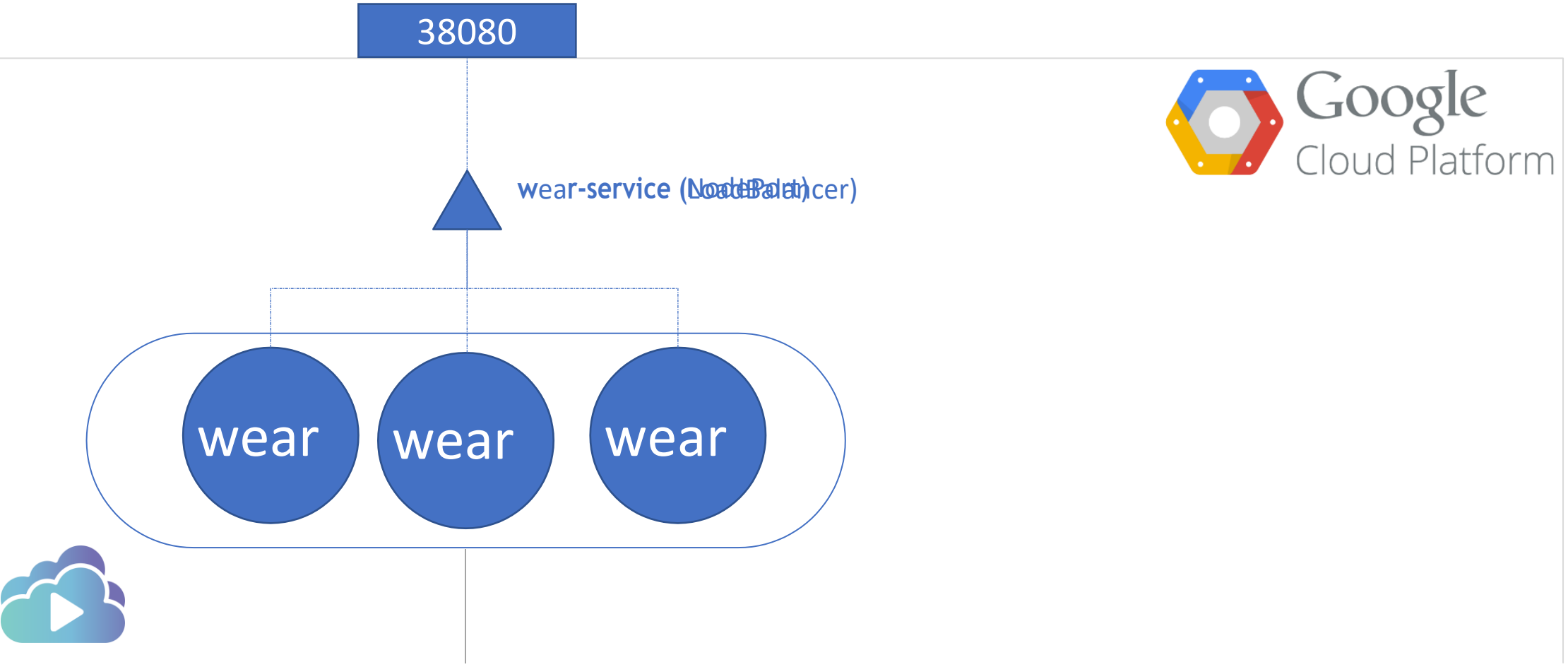


KODEKLOUD


www.my-online-store.com
proxy-server
http://my-online-store.com
80
proxy-server
38080
wear-service (NodePort)
wear
wear
wear

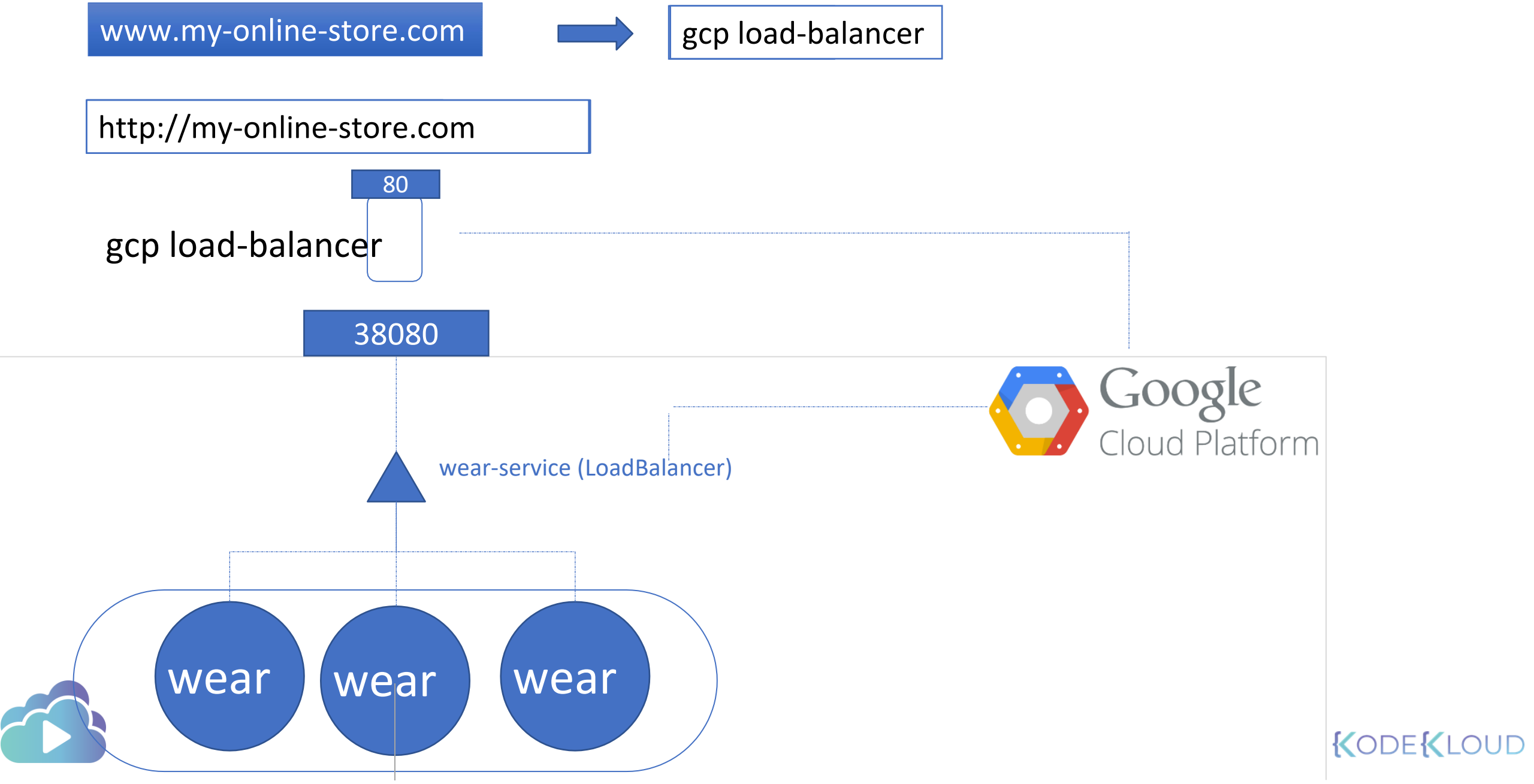
www.my-online-store.com → <node-ip>

http://my-online-store.com:38080

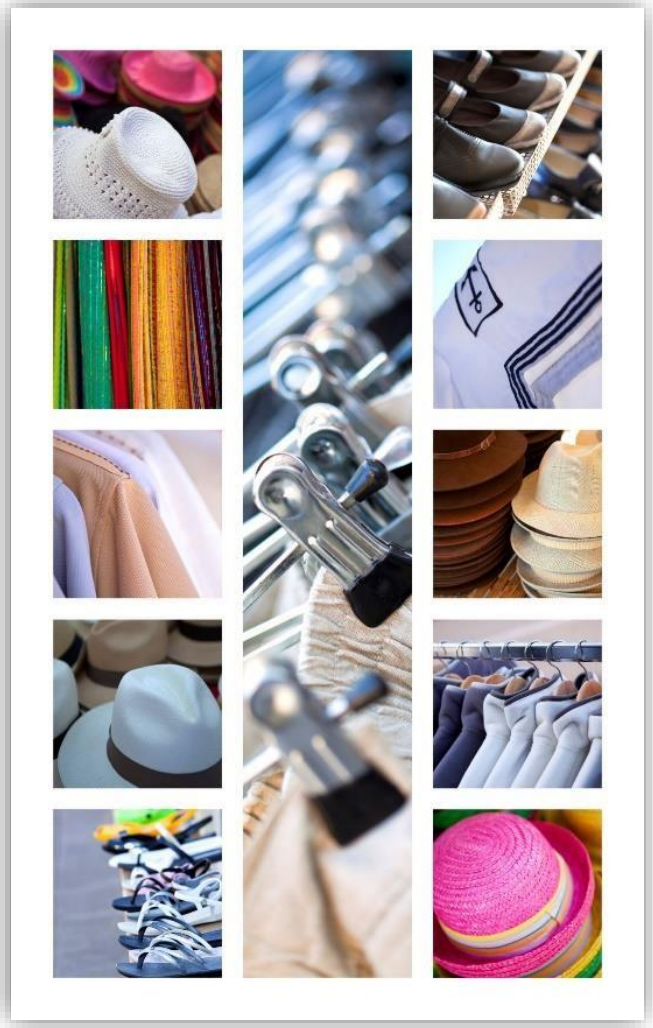


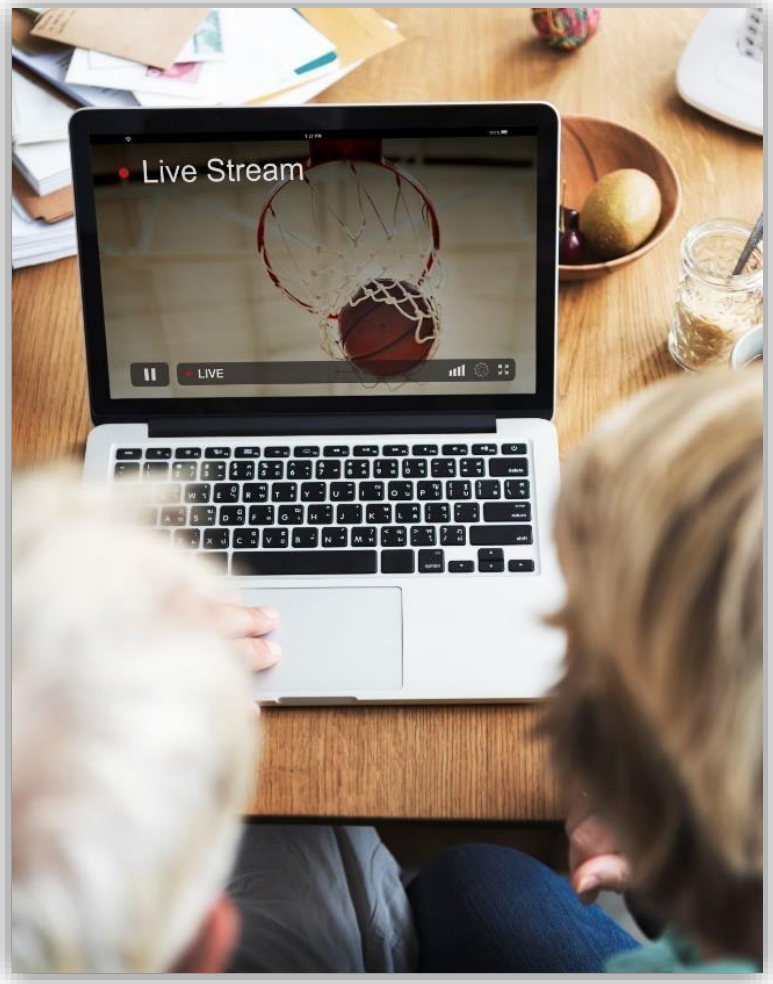
Google Cloud Platform

KODEKLOUD


www.my-online-store.com
gcp load-balancer
http://my-online-store.com
80
gcp load-balancer
38080
Google
Cloud Platform
wear-service (LoadBalancer)
wear
wear
wear
KODEKLOUD

www.my-online-store.com/wear www.my-online-store.com/watch



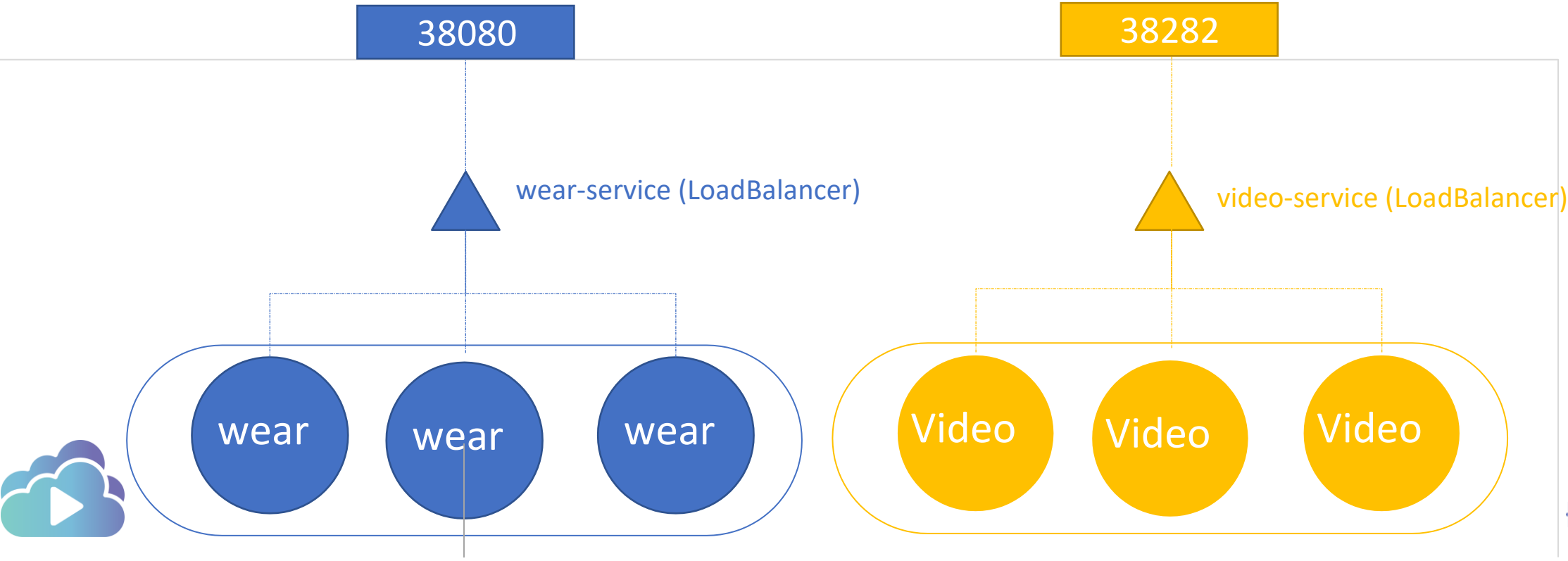


www.my-online-store.com → gcp load-balancer

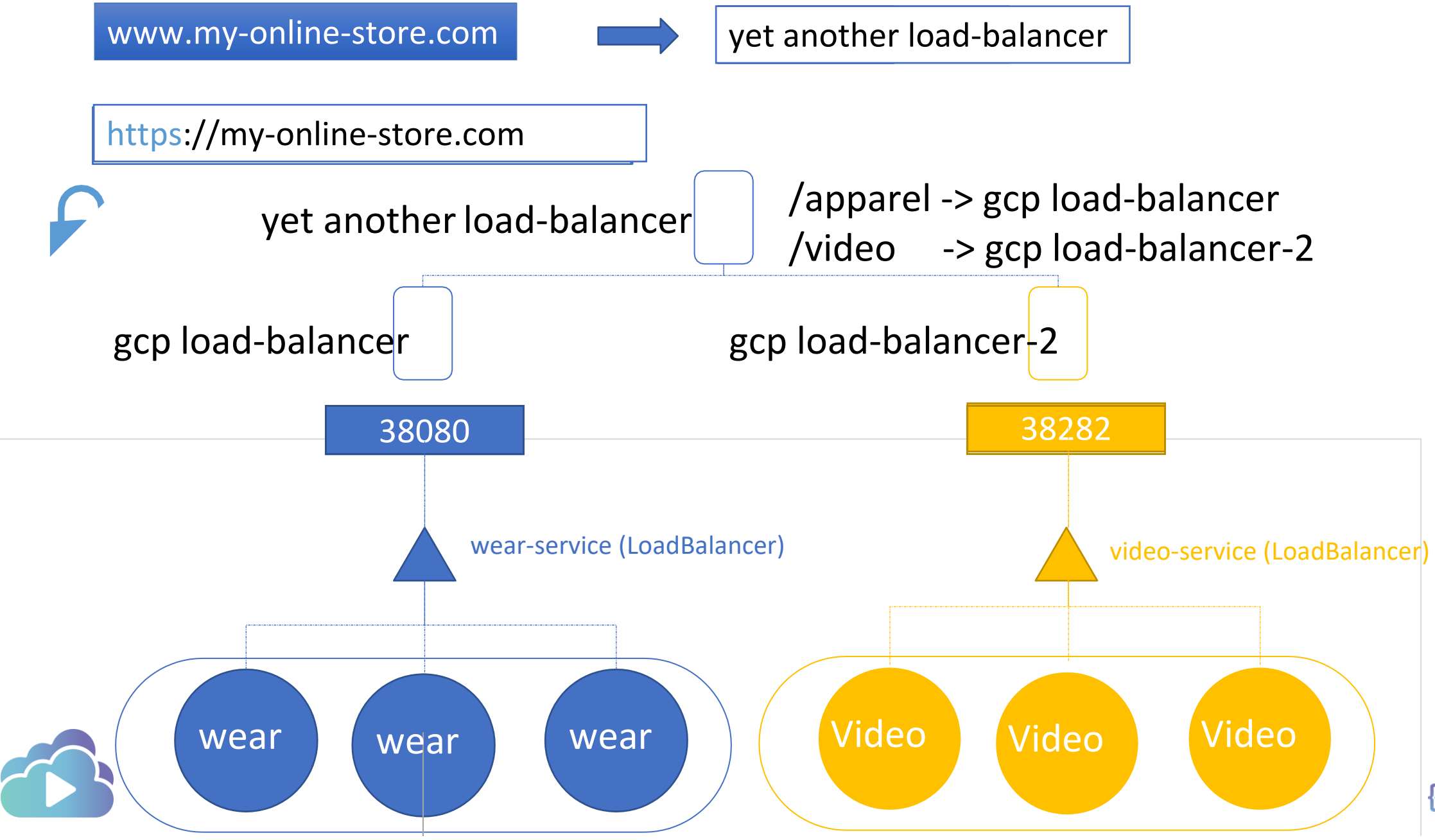
http://my-online-store.com

gcp load-balancer

gcp load-balancer-2



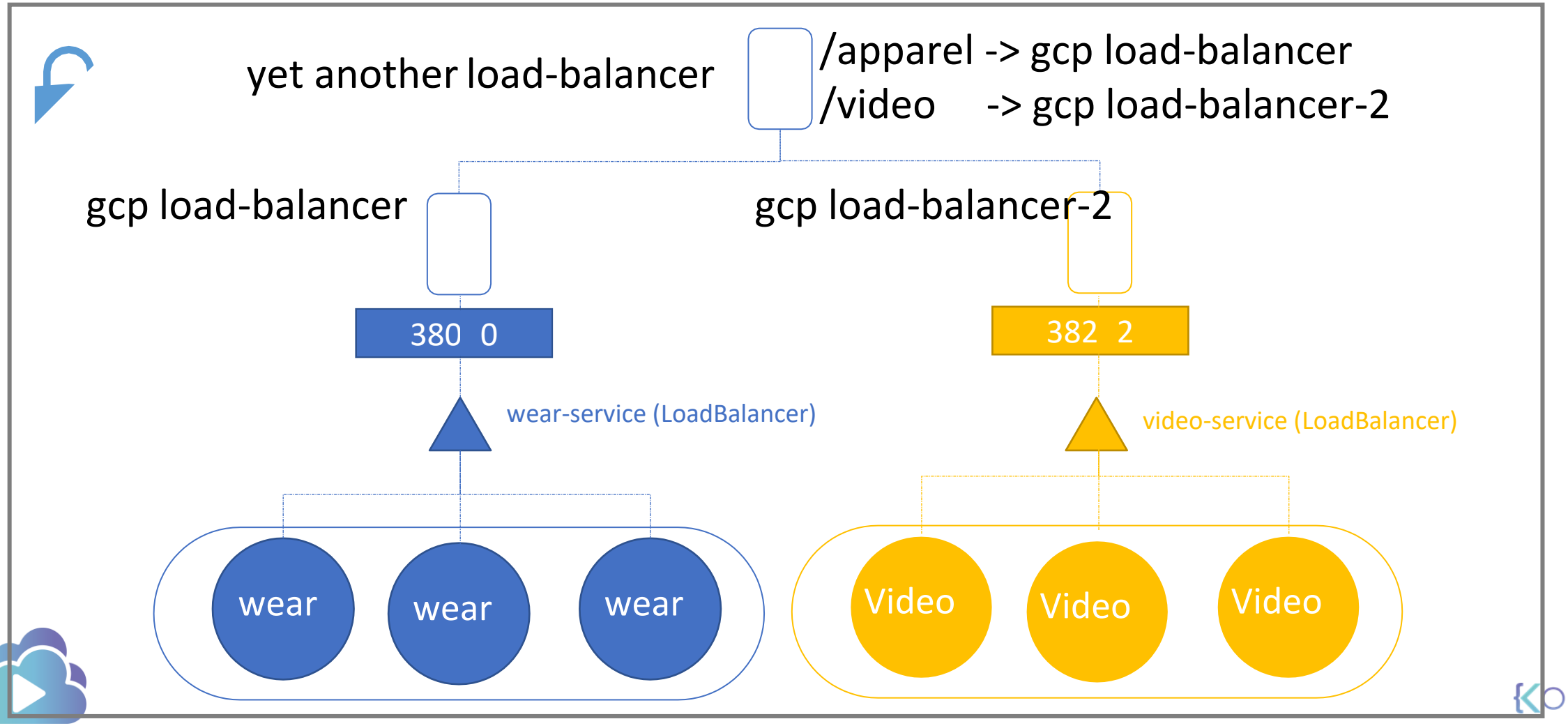
KODEKLOUD


www.my-online-store.com
yet another load-balancer
https://my-online-store.com
yet another load-balancer
/apparel -> gcp load-balancer
/video -> gcp load-balancer-2
gcp load-balancer
gcp load-balancer-2
38080
38282
wear-service (LoadBalancer)
video-service (LoadBalancer)
wear
wear
wear
Video
Video
Video

www.my-online-store.com

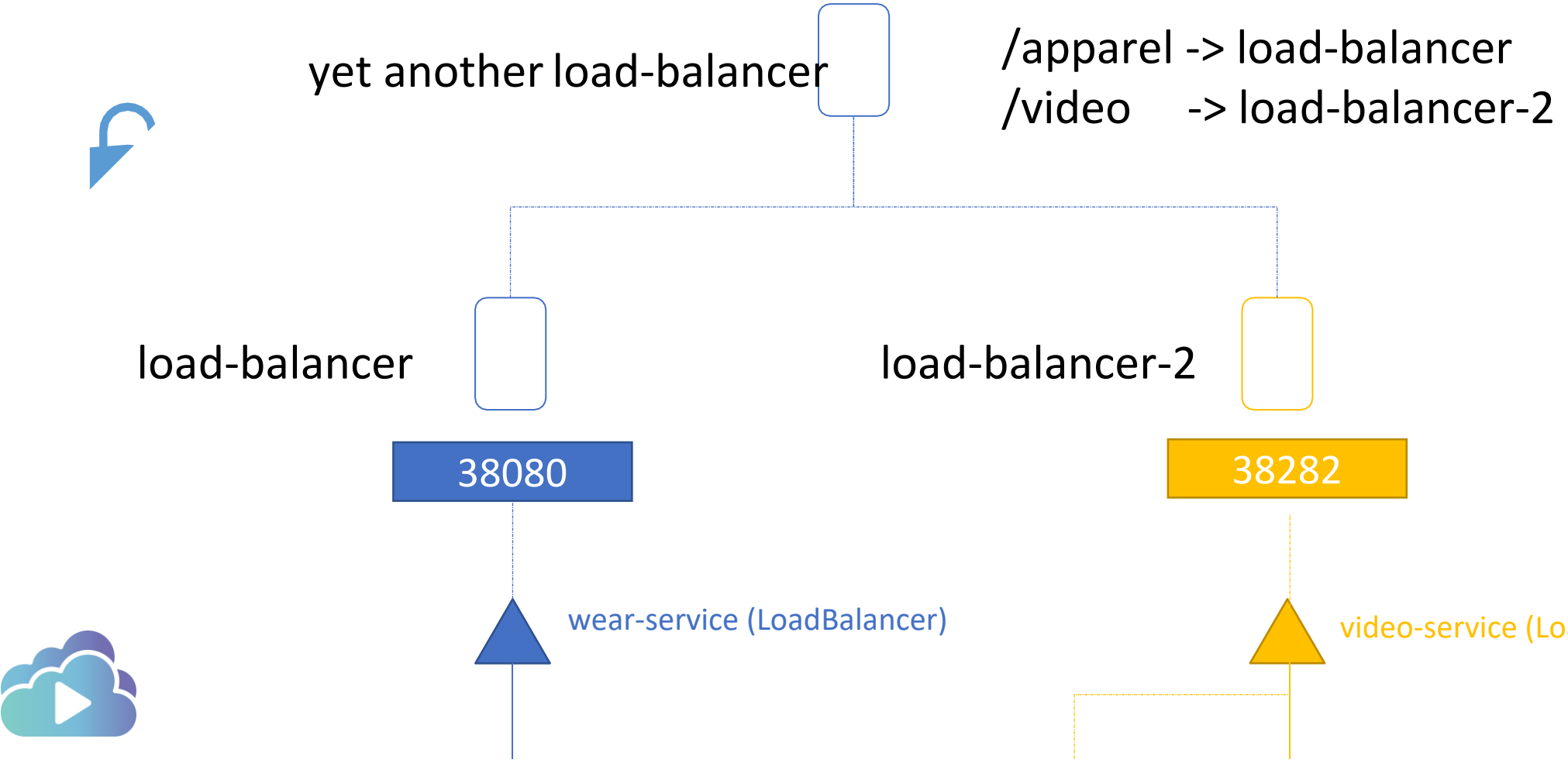
yet another load-balancer

https://my-online-store.com

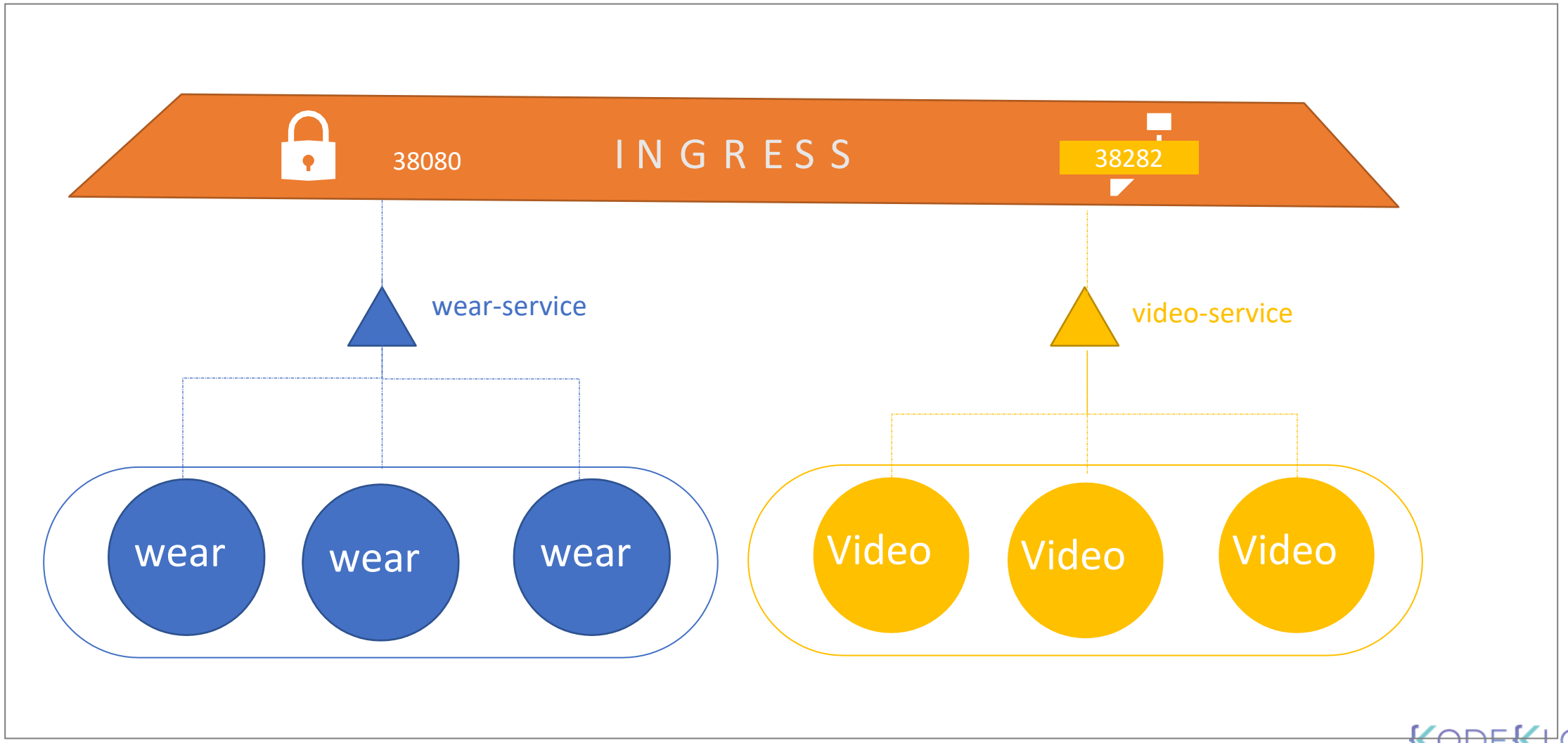


KODEKLOUD

Ingress

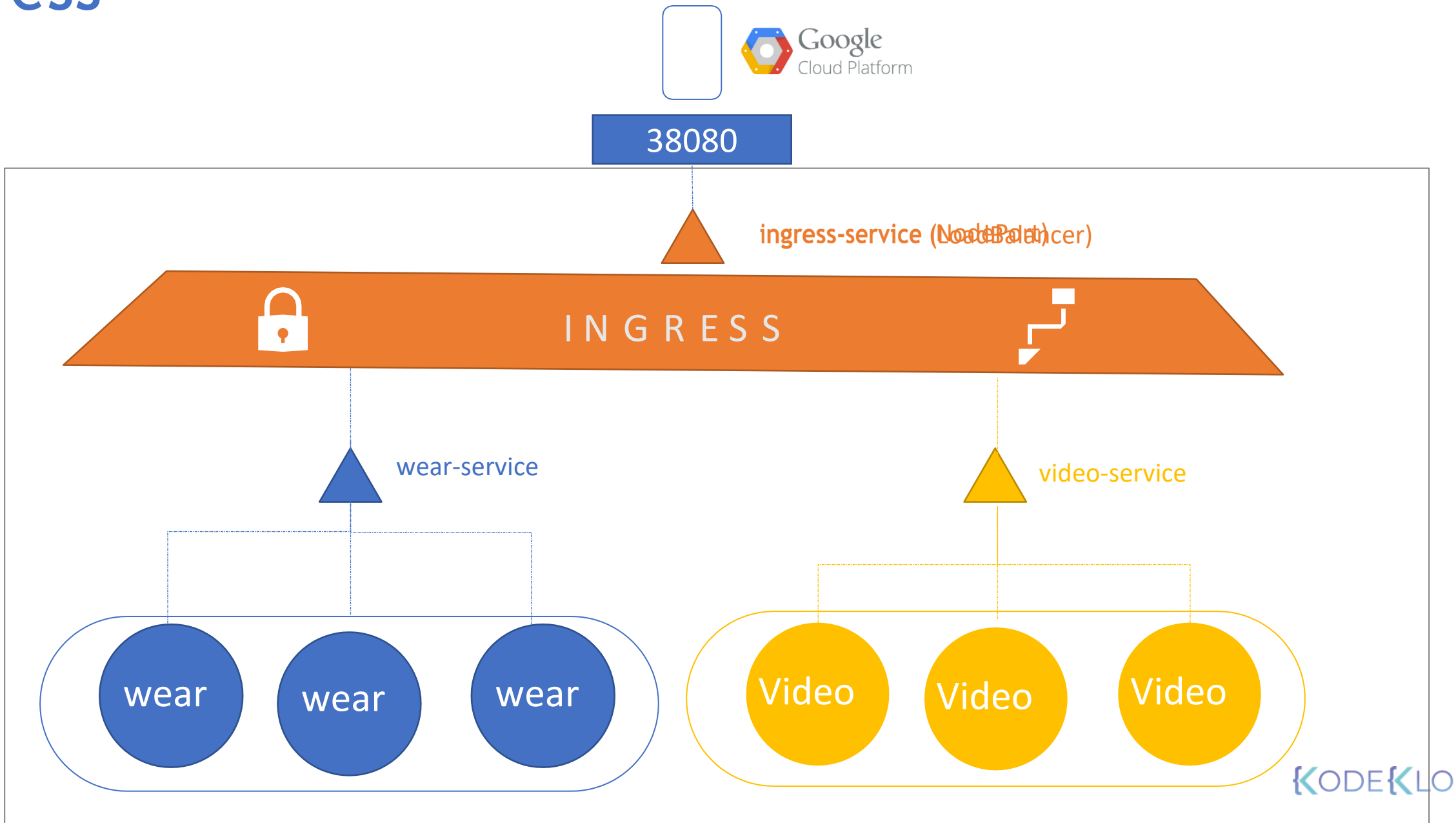

yet another load-balancer
/apparel -> load-balancer
/video -> load-balancer-2
load-balancer
load-balancer-2
38080
38282
wear-service (LoadBalancer)
video-service (LoadBalancer)

Ingress



KODEKLOUD

Ingress


Google
Cloud Platform
38080
ingress-service (LoadBalancer)
INGRESS
wear-service
video-service
wear
wear
wear
Video
Video
Video

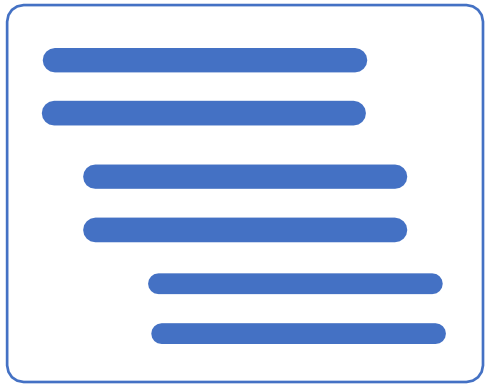
Ingress

1. Deploy

INGRESS CONTROLLER

+

2. Configure



INGRESS RESOURCES

KODEKLOUD

INGRESS CONTROLLER

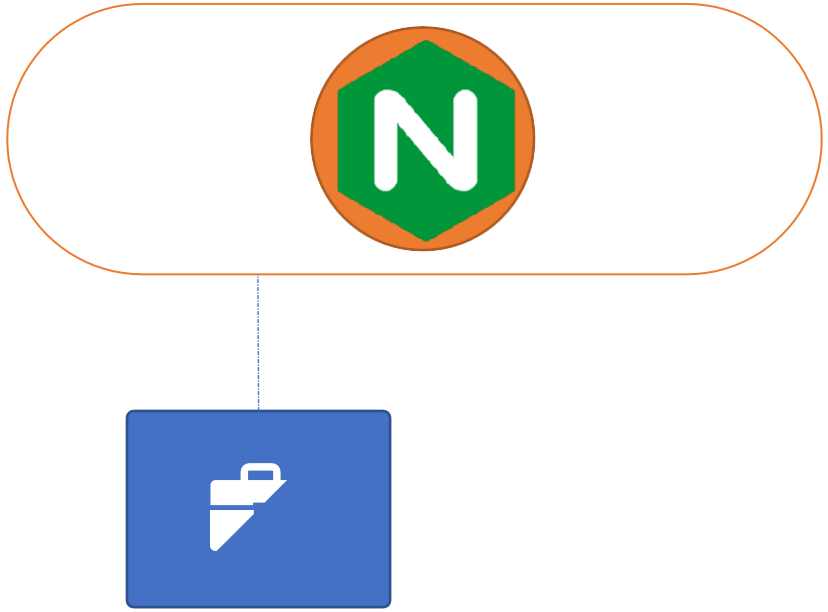
GCP HTTP(S)
Load Balancer (GCE)

Contour

Istio

KODEKLOUD

INGRESS CONTROLLER



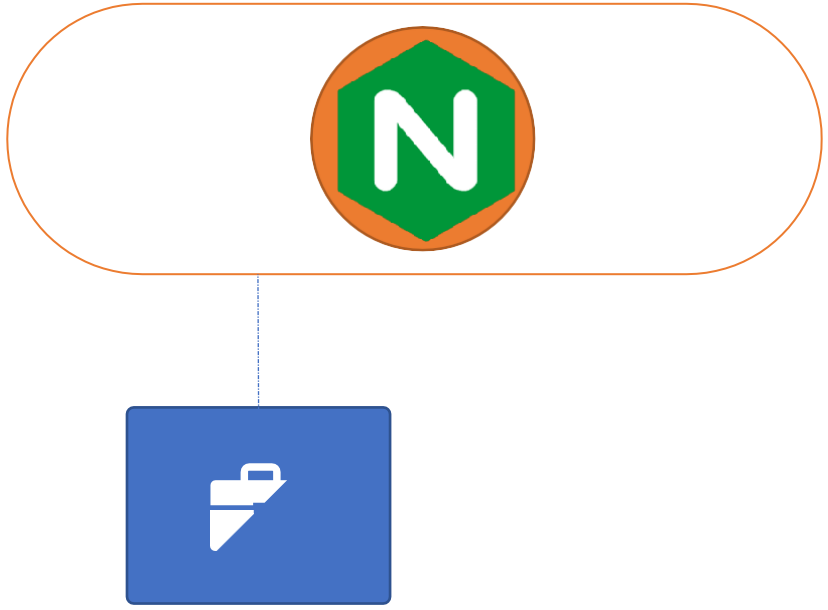
ConfigMap

nginx-configuration

```
kind: ConfigMap
apiVersion: v1
metadata:
  name: nginx-configuration
```

```
apiVersion: extensions/v1beta1
kind: Deployment
metadata:
  name: nginx-ingress-controller
spec:
  replicas: 1
  selector:
    matchLabels:
      name: nginx-ingress
  template:
    metadata:
      labels:
        name: nginx-ingress
    spec:
      containers:
        - name: nginx-ingress-controller
          image:quay.io/kubernetes-ingress-
                controller/nginx-ingress-controller:0.21.0

       args:
         - /nginx-ingress-controller
         - --configmap=$(POD_NAMESPACE)/nginx-configuration
```

INGRESS CONTROL

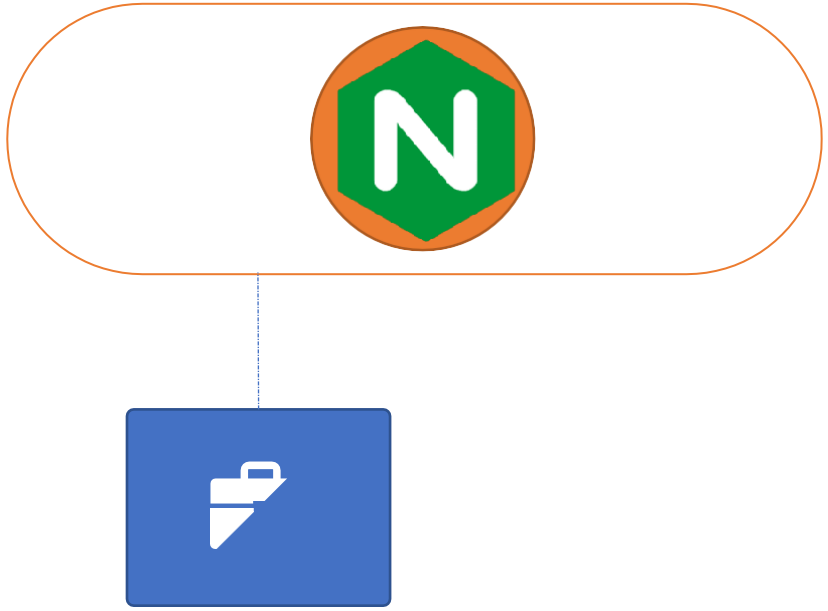


ConfigMap
nginx-configuration

```yaml
kind: ConfigMap
apiVersion: v1
metadata:
  name: nginx-configuration
```

```yaml
  name: nginx-ingress-controller
spec:
  replicas: 1
  selector:
    matchLabels:
      name: nginx-ingress
  template:
    metadata:
      labels:
        name: nginx-ingress
    spec:
      containers:
        - name: nginx-ingress-controller
          image:quay.io/kubernetes-ingress-
                controller/nginx-ingress-controller:0.21.0

        args:
          - /nginx-ingress-controller
          - --configmap=$(POD_NAMESPACE)/nginx-configuration

        env:
          - name: POD_NAME
            valueFrom:
              fieldRef:
                fieldPath: metadata.name
          - name: POD_NAMESPACE
            valueFrom:
              fieldRef:
                fieldPath: metadata.namespace
```

INGRESS CONTROL



ConfigMap

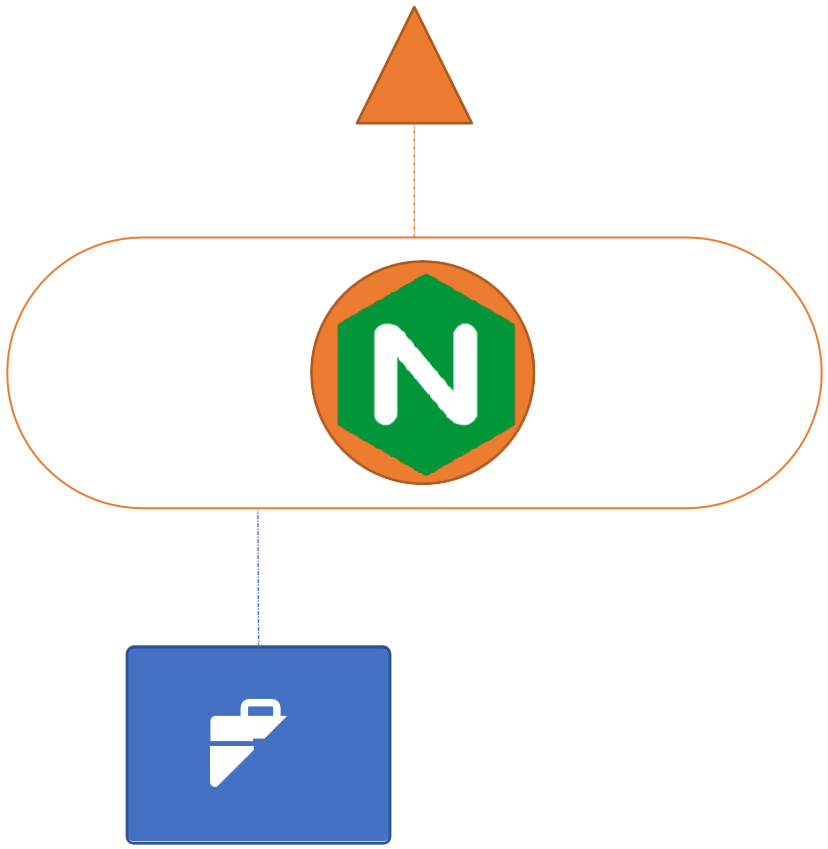
nginx-configuration

```
kind: ConfigMap
apiVersion: v1
metadata:
  name: nginx-configuration
```

```
    - name: nginx-ingress-controller
      image: quay.io/kubernetes-ingress-
             controller/nginx-ingress-controller:0.21.0

  args:
    - /nginx-ingress-controller
    - --configmap=$(POD_NAMESPACE)/nginx-configuration

  env:
    - name: POD_NAME
      valueFrom:
        fieldRef:
          fieldPath: metadata.name
    - name: POD_NAMESPACE
      valueFrom:
        fieldRef:
          fieldPath: metadata.namespace

  ports:
    - name: http
      containerPort: 80
    - name: https
      containerPort: 443
```

KODEKLOUD

INGRESS CONTROL



ConfigMap

nginx-configuration

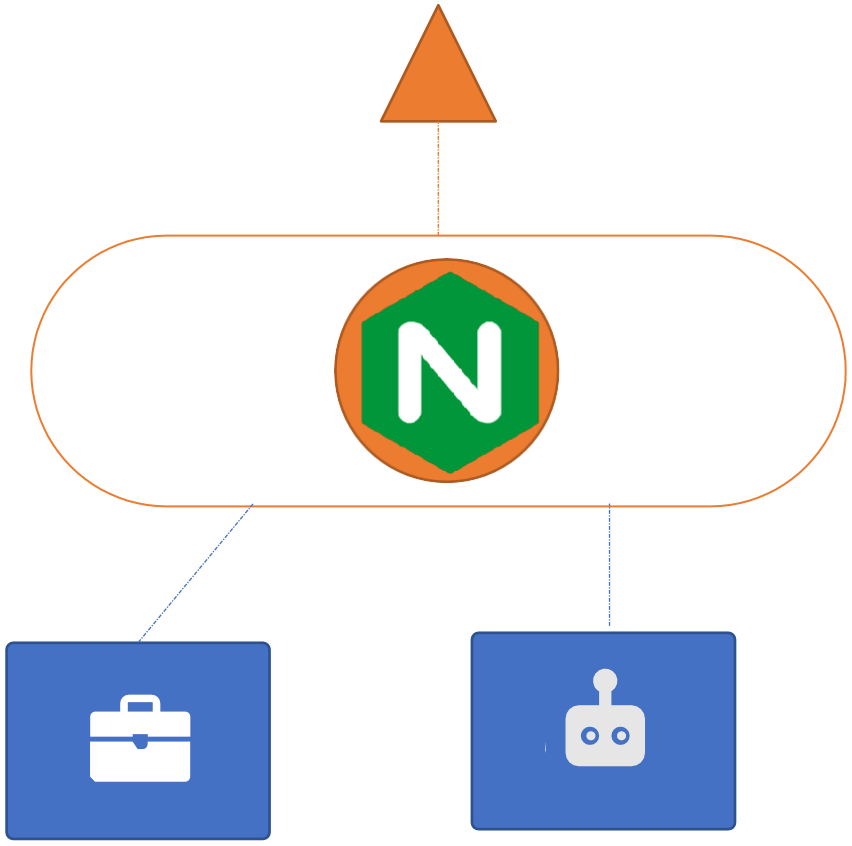
```
kind: ConfigMap
apiVersion: v1
metadata:
  name: nginx-configuration
```

```
            fieldPath: metadata.namespace
      ports:
        - name: http
          containerPort: 80
        - name: https
          containerPort: 443
```

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-ingress
spec:
  type: NodePort
  ports:
  - port: 80
    targetPort: 80
    protocol: TCP
    name: http
  - port: 443
    targetPort: 443
    protocol: TCP
    name: https
  selector:
    name: nginx-ingress
```

KODEKLOUD

INGRESS CONTRO



ConfigMap

nginx-configuration

ServiceAccount

nginx-ingress-serviceaccount

```
kind: ConfigMap
apiVersion: v1
metadata:
  name: nginx-configuration
```

```
    protocol: TCP
    name: http
  - port: 443
    targetPort: 443
    protocol: TCP
    name: https
  selector:
    name: nginx-ingress
```

```
apiVersion:  v1
kind: ServiceAccount
metadata:
  name: nginx-ingress-serviceaccount
```

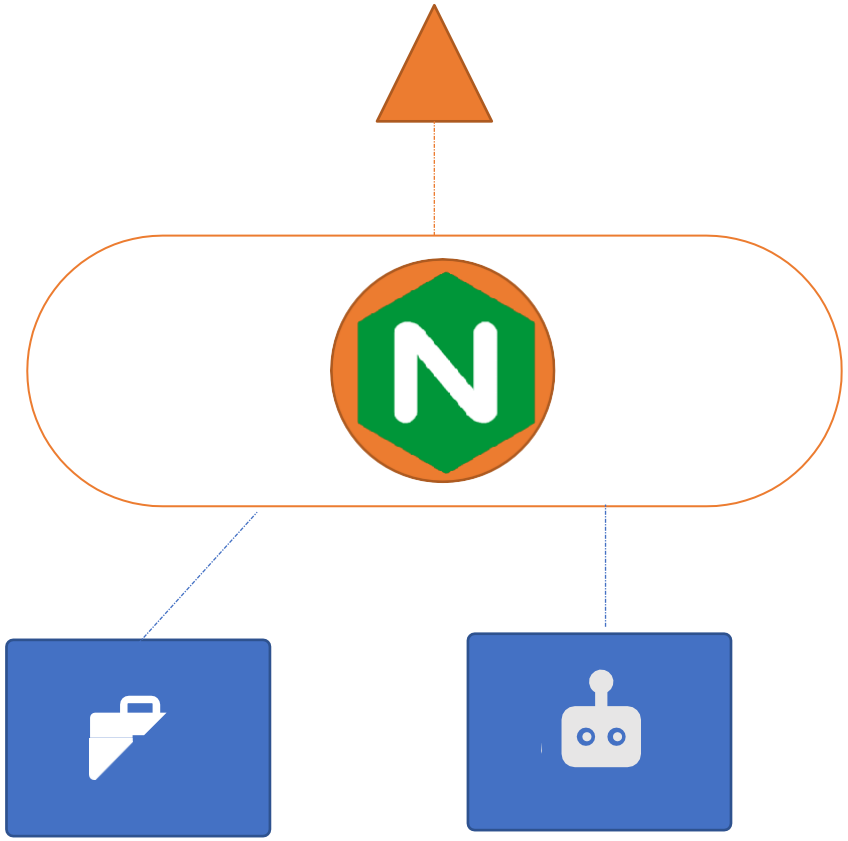
Roles

~~ClusterRol~~

~~RoleBinding~~

KODEKLOUD

INGRESS CONTROLLER



ConfigMap
nginx-configuration

ServiceAccount
nginx-ingress-serviceaccount

Deployment

```yaml
apiVersion: extensions/v1beta1
kind: Deployment
metadata:
  name: nginx-ingress-controller
spec:
  replicas: 1
  selector:
    matchLabels:
      name: nginx-ingress
  template:
    metadata:
      labels:
        name: nginx-ingress
    spec:
      containers:
        - name: nginx-ingress-controller
          image: quay.io/kubernetes-ingress-
                 controller/nginx-ingress-controller:0.21.0
       args:
         - /nginx-ingress-controller
         - --configmap=$(POD_NAMESPACE)/nginx-configuration

       env:
         - name: POD_NAME
           valueFrom:
             fieldRef:
               fieldPath: metadata.name
         - name: POD_NAMESPACE
           valueFrom:
             fieldRef:
               fieldPath: metadata.namespace

       ports:
         - name: http
           containerPort: 80
         - name: https
           containerPort: 443
```

Service

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-ingress
spec:
  type: NodePort
  ports:
  - port: 80
    targetPort: 80
    protocol: TCP
    name: http
  - port: 443
    targetPort: 443
    protocol: TCP
    name: https
  selector:
    name: nginx-ingress
```

ConfigMap

```yaml
kind: ConfigMap
apiVersion: v1
metadata:
  name: nginx-configuration
```

Auth

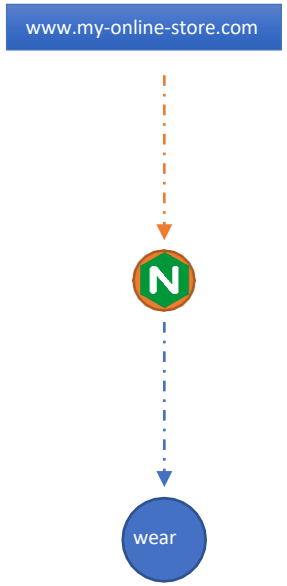
```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: nginx-ingress-serviceaccount
```

Roles | ClusterRoles | RoleBindings

KODEKLOUD

INGRESS RESOURCE

INGRESS RESOURCE

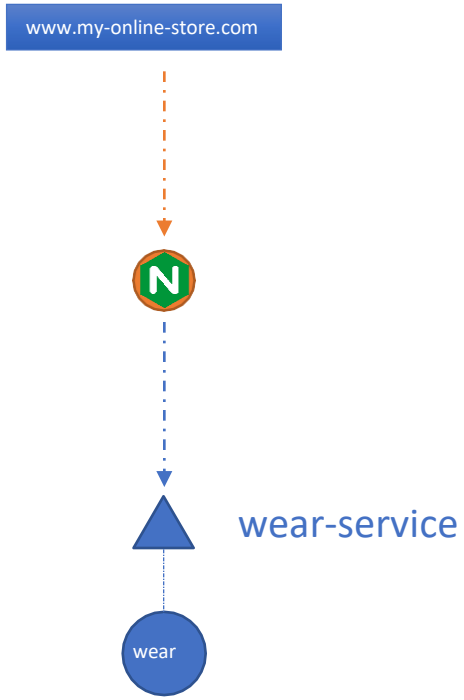


Ingress-wear.yaml

```
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
  name: ingress-wear
spec:
```

KODEKLOUD

INGRESS RESOURCE



Ingress-wear.yaml

```
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
  name: ingress-wear
spec:
 backend:
      serviceName: wear-service
      servicePort: 80
```

```
kubectl create -f Ingress-wear.yaml
ingress.extensions/ingress-wear created

kubectl get ingress

ingress-wear        *                                80        2s
```

KODEKLOUD

INGRESS RESOURCE - RULES

www.my-online-store.com	www.wear.my-online-store.com	www.watch.my-online-store.com	Everything Else
Rule 1	Rule 2	Rule 3	Rule 4

KODEKLOUD

INGRESS RESOURCE - RULES

DNS Name	Forward IP
www.my-online-store.com	10.123.23.12 (INGRESS SERVICE)
www.wear.my-online-store.com	10.123.23.12
www.watch.my-online.store.com	10.123.23.12
www.my-wear-store.com	10.123.23.12
www.my-watch-store.com	10.123.23.12

www.my-online-sto

ing Else

Rule 1

Rule 4

KODEKLOUD

INGRESS RESOURCE - RULES

www.my-online-store.com

www.wear.my-online-store.com

www.watch.my-online-store.com

Everything Else

http://www.my-online-store.com/wear

http://www.my-online-store.com/watch

http://www.my-online-store.com/listen

Rule 1

Path /wear

Path /watch

Path /

Rule 2

Rule 3

Rule 4

KODEKLOUD

INGRESS RESOURCE - RULES

www.my-online-store.com | www.wear.my-online-store.com | www.watch.my-online-store.com | Everything Else

tp://www.my-online-store.com/wear → http://www.wear.my-online-store.com/

tp://www.my-online-store.com/watch → http://www.wear.my-online-store.com/returns

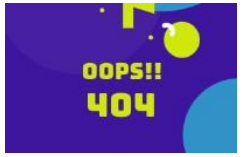
tp://www.my-online-store.com/listen → http://www.wear.my-online-store.com/support

Rule 1

Path /wear

Path /watch

Path /



Rule 2

Path /

Path /returns

Path /support

Rule 3

Rule 4

KODEKLOUD

INGRESS RESOURCE - RULES

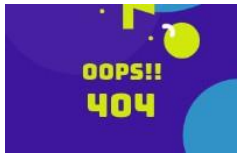
www.my-online-store.com	www.wear.my-online-store.com	www.watch.my-online-store.com	Everything Else
http://www.my-online-store.com/wear	http://www.wear.my-online-store.com/	http://www.watch.my-online-store.com/	
http://www.my-online-store.com/watch	http://www.wear.my-online-store.com/returns	http://www.watch.my-online-store.com/movies	
http://www.my-online-store.com/listen	http://www.wear.my-online-store.com/support	http://www.watch.my-online-store.com/tv	

Rule 1

Path /wear

Path /watch

Path /



Rule 2

Path /

Path /returns

Path /support

Rule 3

Path /

Path /movies

Path /tv

Rule 4

KODEKLOUD

INGRESS RESOURCE - RULES

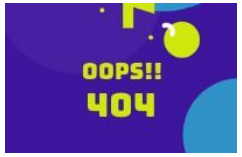
www.my-online-store.com	www.wear.my-online-store.com	www.watch.my-online-store.com	Everything Else
http://www.my-online-store.com/wear	http://www.wear.my-online-store.com/	http://www.watch.my-online-store.com/	http://www.listen.my-online-store.com/
http://www.my-online-store.com/watch	http://www.wear.my-online-store.com/returns	http://www.watch.my-online-store.com/movies	http://www.eat.my-online-store.com/
http://www.my-online-store.com/listen	http://www.wear.my-online-store.com/support	http://www.watch.my-online-store.com/tv	http://www.drink.my-online-store.com/tv

Rule 1

Path /wear

Path /watch

Path /



Rule 2

Path /

Path /returns

Path /support

Rule 3

Path /

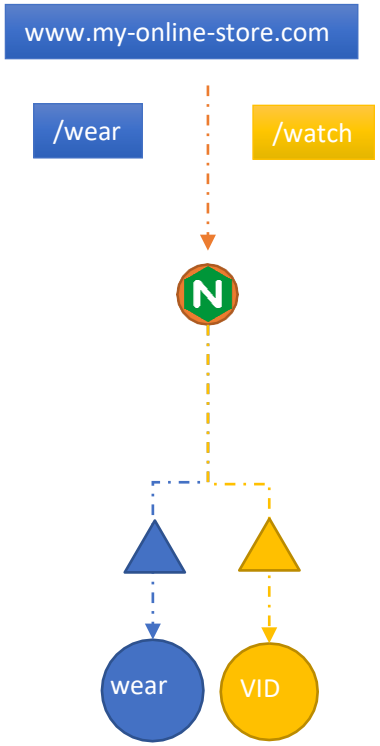
Path /movies

Path /tv

Rule 4

Path /

INGRESS RESOURCE



Ingress-wear.yaml

```yaml
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
  name: ingress-wear
spec:
  backend:
    serviceName: wear-service
    servicePort: 80
```

Ingress-wear-watch.yaml

```yaml
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
  name: ingress-wear-watch
spec:
  rules:
  - http:
      paths:
      - path: /wear



      - path: /watch
        backend:
          serviceName: watch-service
          servicePort: 80
```

KODEKLOUD

INGRESS RESOURCE

```
kubectl describe ingress ingress-wear-watch
```

```
Name:             ingress-wear-watch
Namespace:        default
Address:
Default backend:  default-http-backend:80 (<none>)
Rules:
  Host  Path  Backends
  ----  ----  --------
  *
        /wear   wear-service:80 (<none>)
        /watch  watch-service:80 (<none>)
Annotations:
Events:
  Type    Reason  Age   From                      Message
  ----    ------  ----  ----                      -------
  Normal  CREATE  14s   nginx-ingress-controller  Ingress default/ingress-wear-watch
```

KODEKLOUD

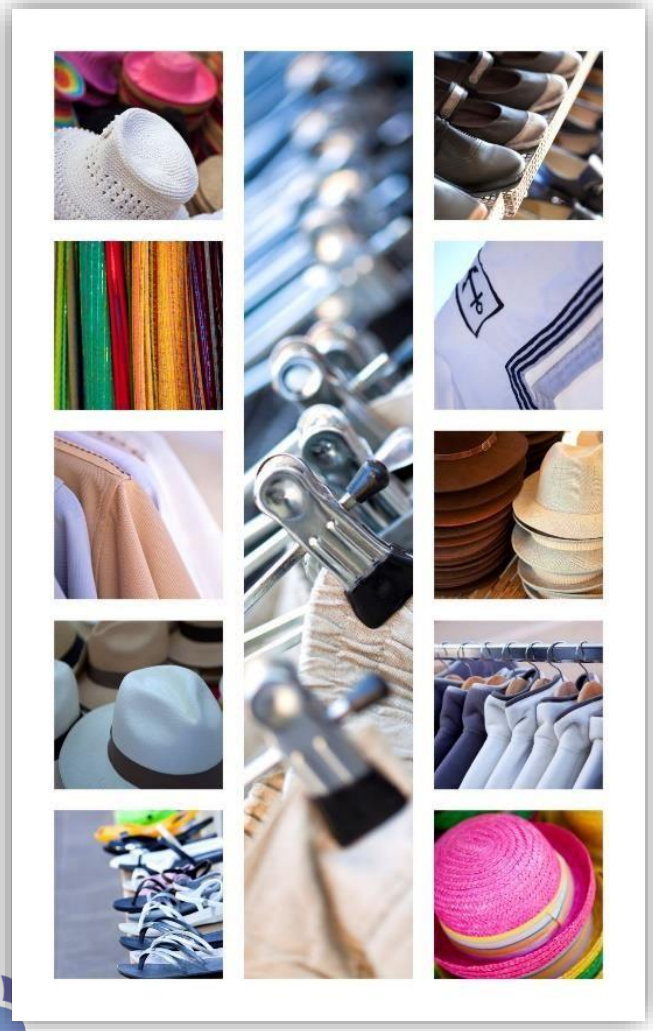
INGRESS RESOURCE

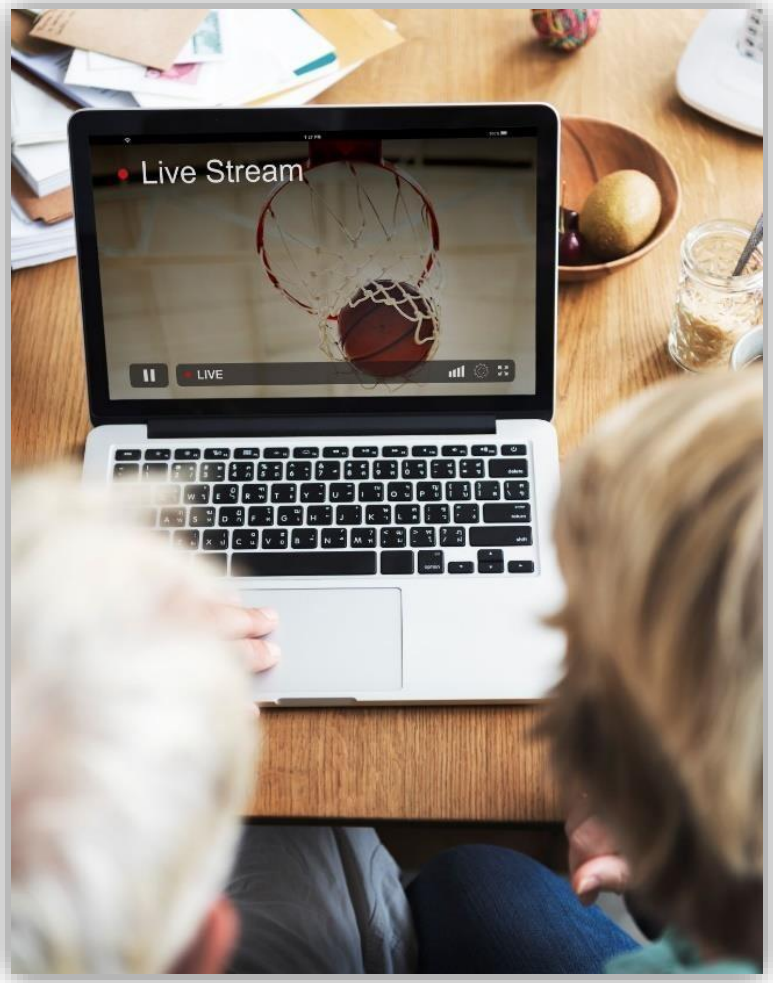
www.my-online-store.com/eat

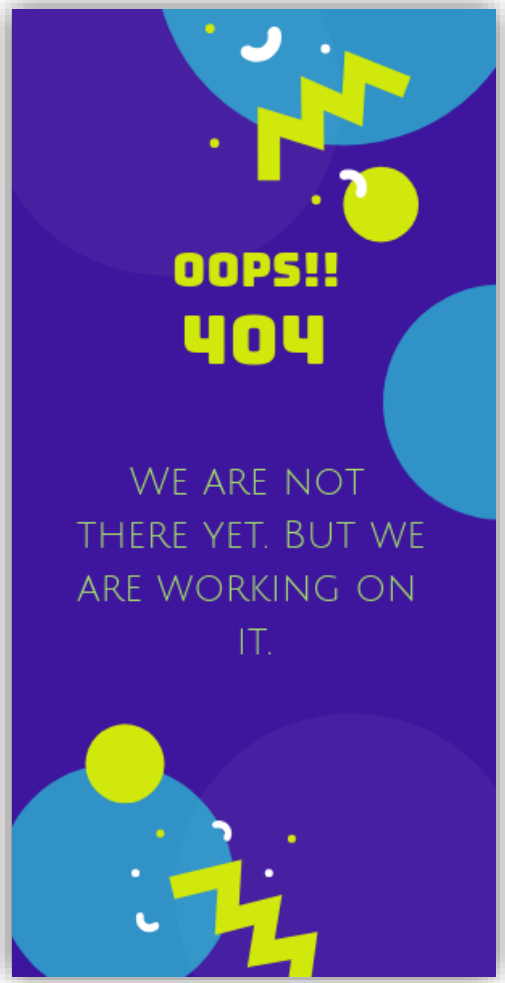
www.my-online-store.com/liste

www.my-online-store.com/wear

www.my-online-store.com www.my-online-store.com/watch







KODEKLOUD

INGRESS RESOURCE

Ingress-wear-watch.yaml

```yaml
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
  name: ingress-wear-watch
spec:

  rules:
  - host: wear.my-online-store.com
    http:
      paths:
      - backend:
          serviceName: wear-service
          servicePort: 80
  - host: watch.my-online-store.com
    http:
      paths:
      - backend:
          serviceName: watch-service
          servicePort: 80
```

KODEKLOUD

INGRESS RESOURCE

Ingress-wear-watch.yaml

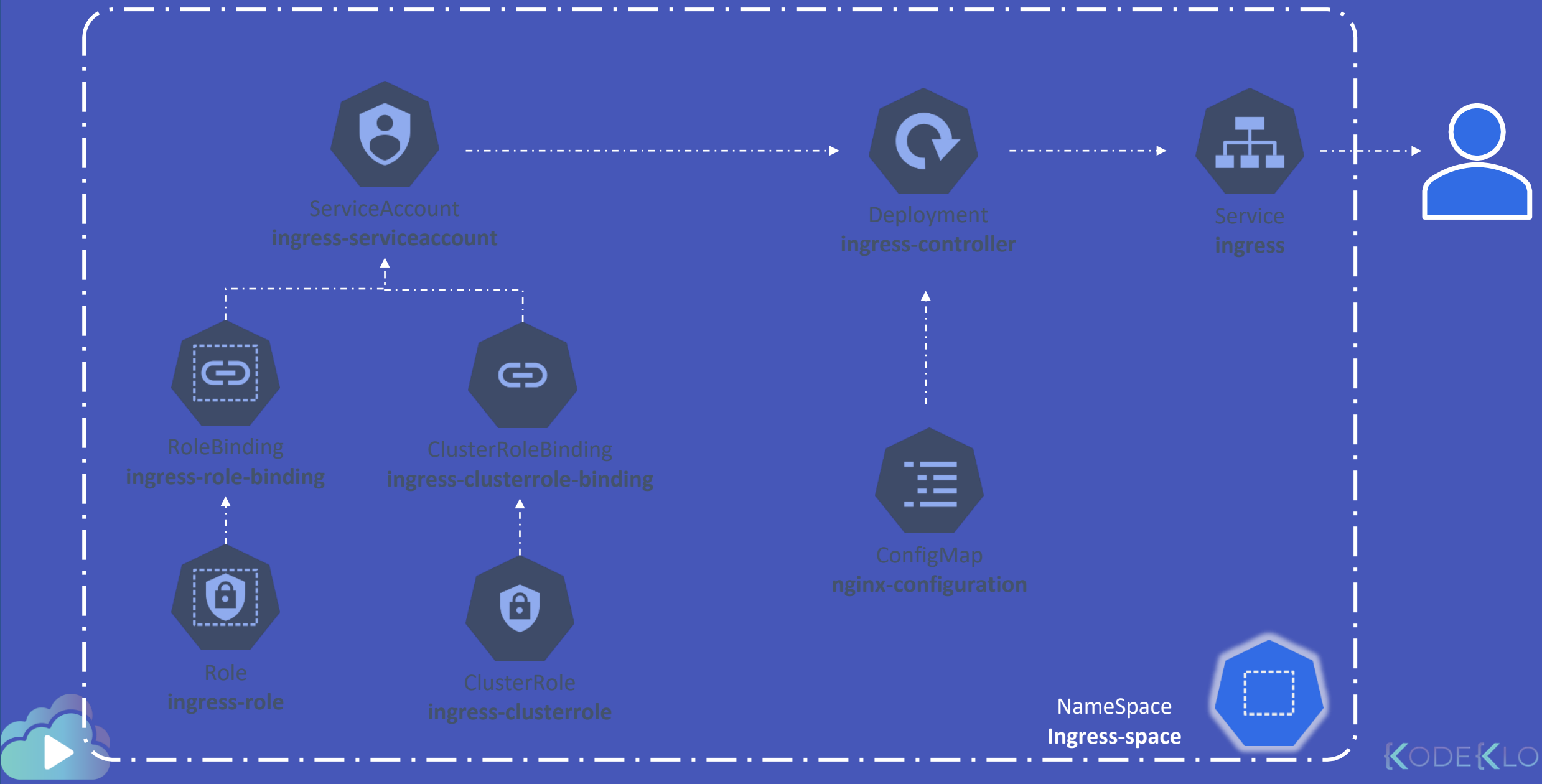
```yaml
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
  name: ingress-wear-watch
spec:
  rules:
  - http:
      paths:
      - path: /wear
        backend:
          serviceName: wear-service
          servicePort: 80
      - path: /watch
        backend:
          serviceName: watch-service
          servicePort: 80
```

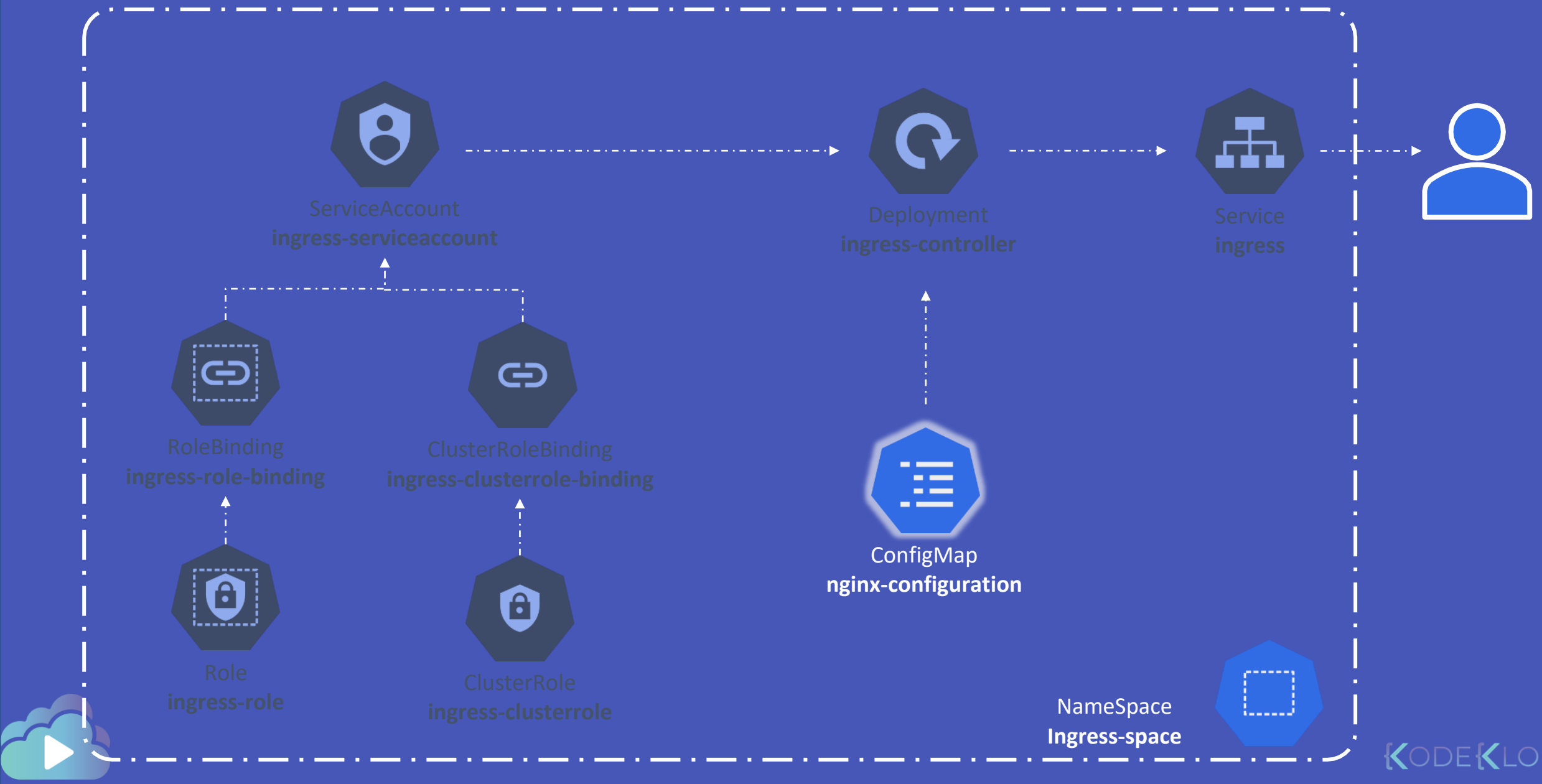
Ingress-wear-watch.yaml

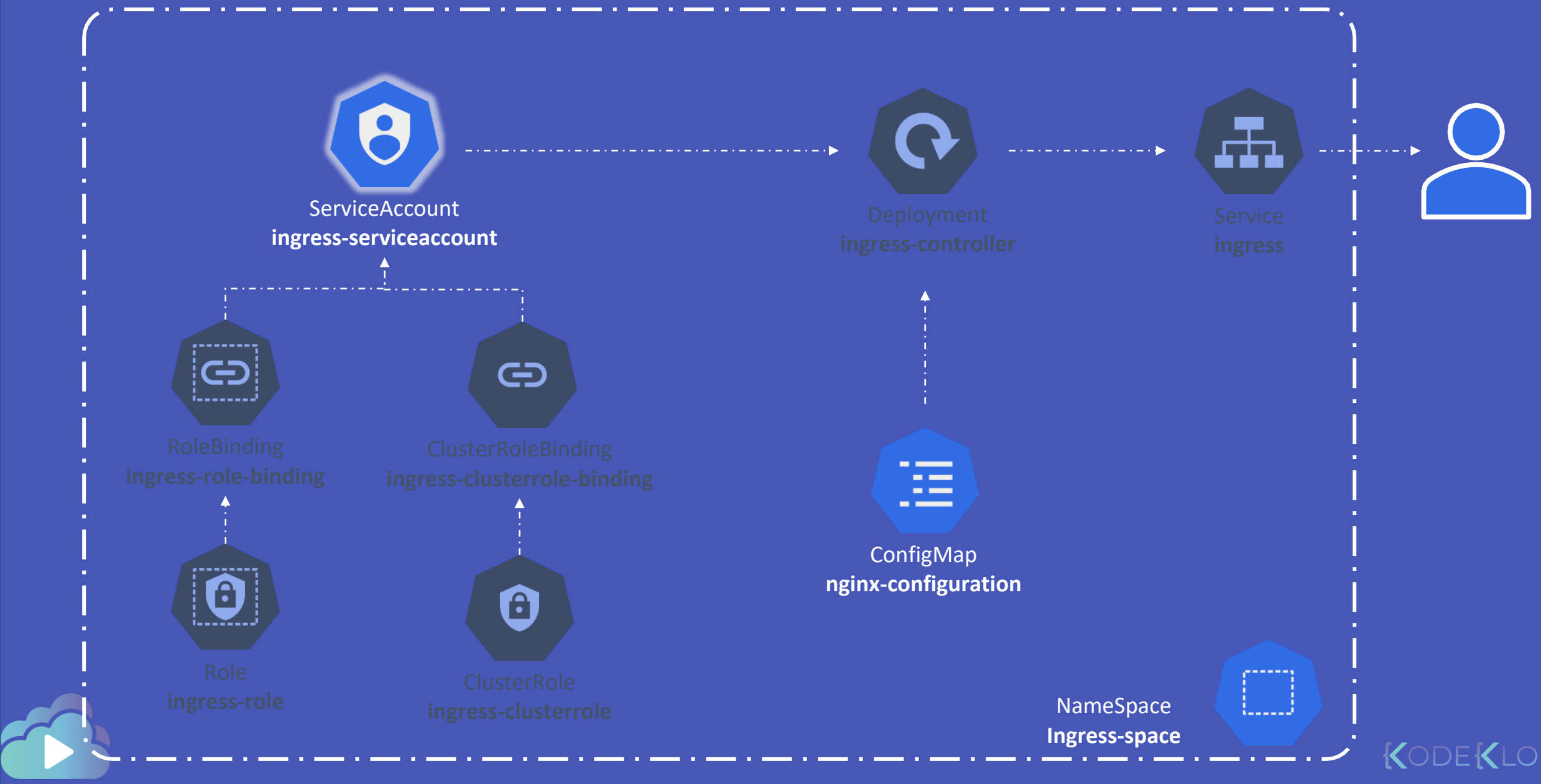
```yaml
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
  name: ingress-wear-watch
spec:

  rules:
  - host: wear.my-online-store.com
    http:
      paths:
      - backend:
          serviceName: wear-service
          servicePort: 80
  - host: watch.my-online-store.com
    http:
      paths:
      - backend:
          serviceName: watch-service
          servicePort: 80
```

KODEKLOUD

ServiceAccount
ingress-serviceaccount
Deployment
ingress-controller
Service
ingress
RoleBinding
ingress-role-binding
ClusterRoleBinding
ingress-clusterrole-binding
ConfigMap
nginx-configuration
Role
ingress-role
ClusterRole
ingress-clusterrole
NameSpace
Ingress-space

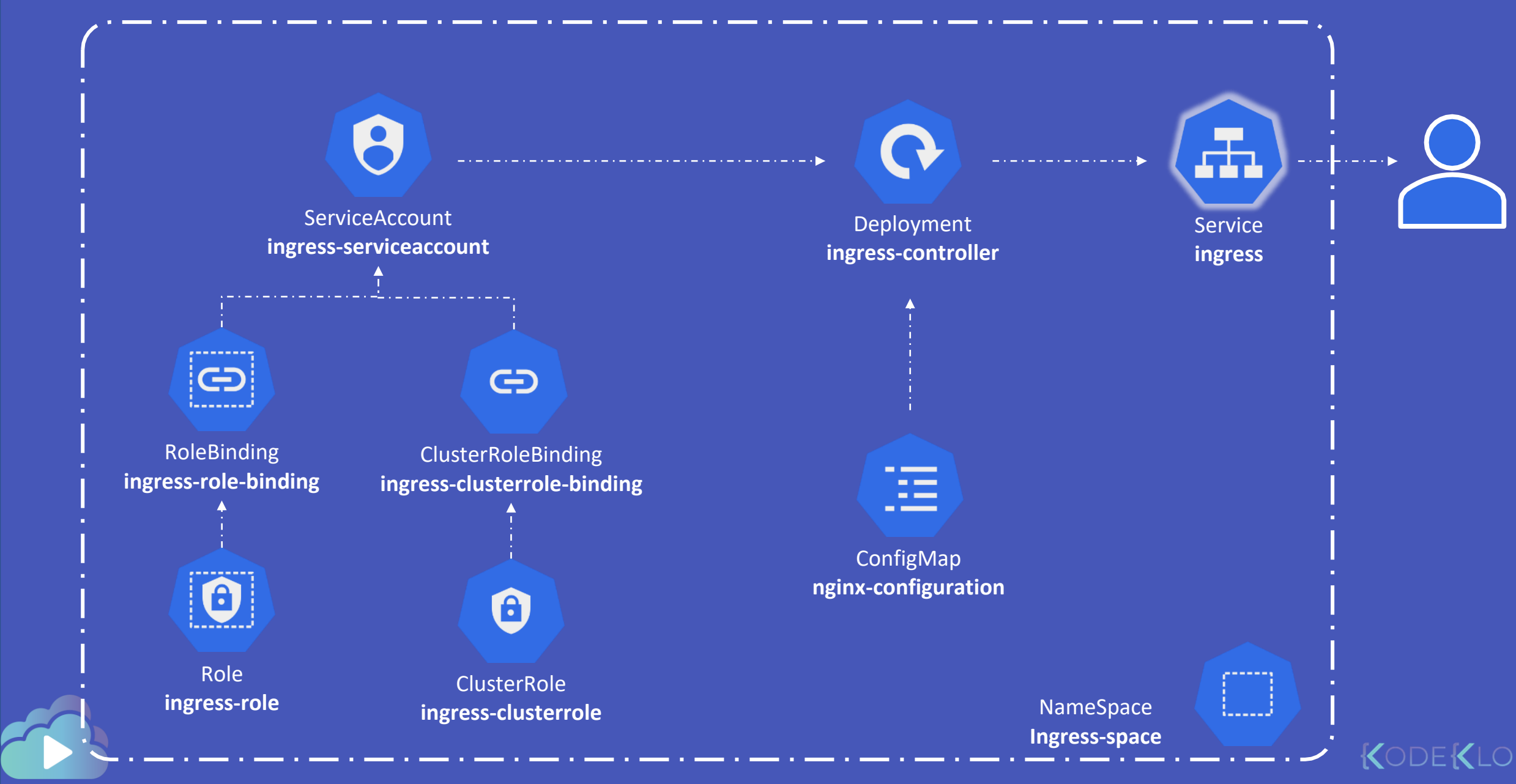

ServiceAccount
ingress-serviceaccount
Deployment
ingress-controller
Service
ingress
RoleBinding
ingress-role-binding
ClusterRoleBinding
ingress-clusterrole-binding
ConfigMap
nginx-configuration
Role
ingress-role
ClusterRole
ingress-clusterrole
NameSpace
Ingress-space


ServiceAccount
ingress-serviceaccount
Deployment
ingress-controller
Service
ingress
RoleBinding
ingress-role-binding
ClusterRoleBinding
ingress-clusterrole-binding
ConfigMap
nginx-configuration
Role
ingress-role
ClusterRole
ingress-clusterrole
NameSpace
Ingress-space


ServiceAccount
ingress-serviceaccount
Deployment
ingress-controller
Service
ingress
RoleBinding
ingress-role-binding
ClusterRoleBinding
ingress-clusterrole-binding
ConfigMap
nginx-configuration
Role
ingress-role
ClusterRole
ingress-clusterrole
NameSpace
Ingress-space

ServiceAccount
ingress-serviceaccount
Deployment
ingress-controller
Service
ingress
RoleBinding
ingress-role-binding
ClusterRoleBinding
ingress-clusterrole-binding
ConfigMap
nginx-configuration
Role
ingress-role
ClusterRole
ingress-clusterrole
NameSpace
Ingress-space

ServiceAccount
ingress-serviceaccount
Deployment
ingress-controller
Service
ingress
RoleBinding
ingress-role-binding
ClusterRoleBinding
ingress-clusterrole-binding
ConfigMap
nginx-configuration
Role
ingress-role
ClusterRole
ingress-clusterrole
NameSpace
Ingress-space


ServiceAccount
ingress-serviceaccount
Deployment
ingress-controller
Service
ingress
RoleBinding
ingress-role-binding
ClusterRoleBinding
ingress-clusterrole-binding
ConfigMap
nginx-configuration
Role
ingress-role
ClusterRole
ingress-clusterrole
NameSpace
Ingress-space

KODEKLOUD

Course Objectives

- Core Concepts
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Networking
- Installation, Configuration & Validation
  - Design a Kubernetes Cluster
  - Choose Kubernetes Infrastructure Config
  - Choose a Network Solution
  - HA Kubernetes Cluster
  - Provision Infrastructure
  - Secure Cluster Communication
  - Kubernetes Release Binaries
  - Install Kubernetes Master Nodes
  - Install Kubernetes Worker Nodes
  - TLS Bootstrapping a Node
  - Node end-to-end tests
  - Run & Analyze end-to-end tes
- Troubleshooting

KODEKLOUD

DESIGN A KUBERNETES CLUSTER

Objectives

- Node Considerations
- Resource Requirements
- Network Considerations

KODEKLOUD

Ask

- Purpose
  - Education
  - Development & Testing
  - Hosting Production Applications
- Cloud or OnPrem?
- Workloads
  - How many?
  - What kind?
    - Web
    - Big Data/Analytics
  - Application Resource Requirements
    - CPU Intensive
    - Memory Intensive
  - Traffic
    - Heavy traffic
    - Burst Traffic

KODEKLOUD

Purpose

- Education
  - Minikube
  - Single node cluster with kubeadm/GCP/AWS
- Development & Testing
  - Multi-node cluster with a Single Master and Multiple workers
  - Setup using kubeadm tool or quick provision on GCP or AWS or AKS
- Hosting Production Applications

KODEKLOUD

Hosting Production Applications

- High Availability Multi node cluster with multiple master nodes
- Kubeadm or GCP or Kops on AWS or other supported platforms
- Upto 5000 nodes
- Upto 150,000 PODs in the cluster
- Upto 300,000 Total Containers
- Upto 100 PODs per Node

Nodes	GCP		AWS	
1-5	N1-standard-1	1 vCPU 3.75 GB	M3.medium	1 vCPU 3.75 GB
6-10	N1-standard-2	2 vCPU 7.5 GB	M3.large	2 vCPU 7.5 GB
11-100	N1-standard-4	4 vCPU 15 GB	M3.xlarge	4 vCPU 15 GB
101-250	N1-standard-8	8 vCPU 30 GB	M3.2xlarge	8 vCPU 30 GB
251-500	N1-standard-16	16 vCPU 60 GB	C4.4xlarge	16 vCPU 30 GB
> 500	N1-standard-32	32 vCPU 120 GB	C4.8xlarge	36 vCPU 60 GB

Cloud or OnPrem?

- Use Kubeadm for on-prem
- GKE for GCP
- Kops for AWS
- Azure Kubernetes Service(AKS) for Azure

KODEKLOUD

Storage

- High Performance – SSD Backed Storage
- Multiple Concurrent connections – Network based storage
- Persistent shared volumes for shared access across multiple PODs
- Label nodes with specific disk types
- Use Node Selectors to assign applications to nodes with specific disk types

KODEKLOUD

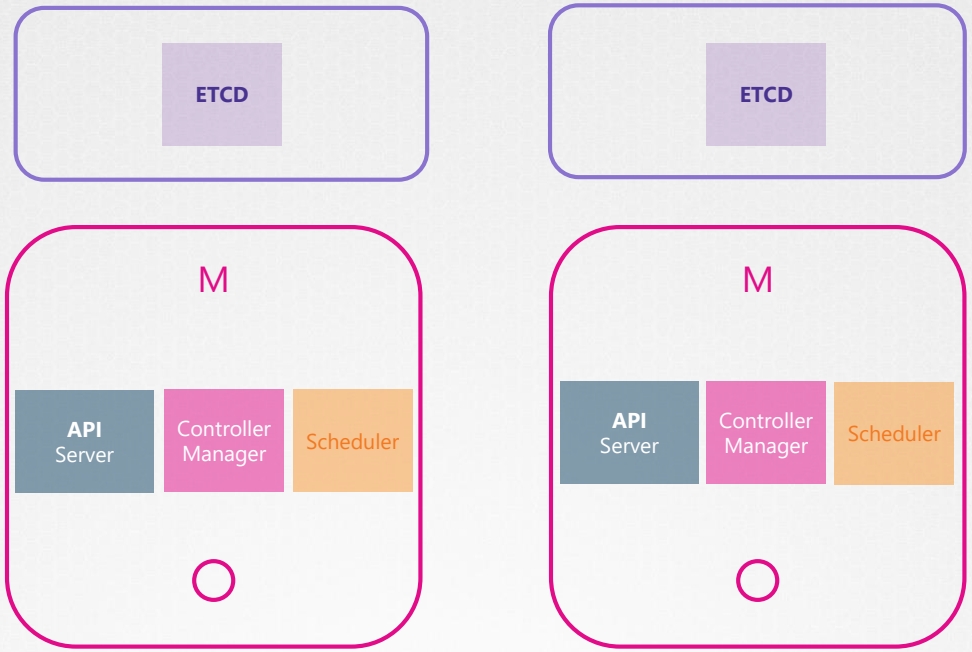
Nodes

- Virtual or Physical Machines
- Minimum of 4 Node Cluster (Size based on workload)
- Master vs Worker Nodes
- Linux X86_64 Architecture

- Master nodes can host workloads
- Best practice is to not host workloads on Master nodes

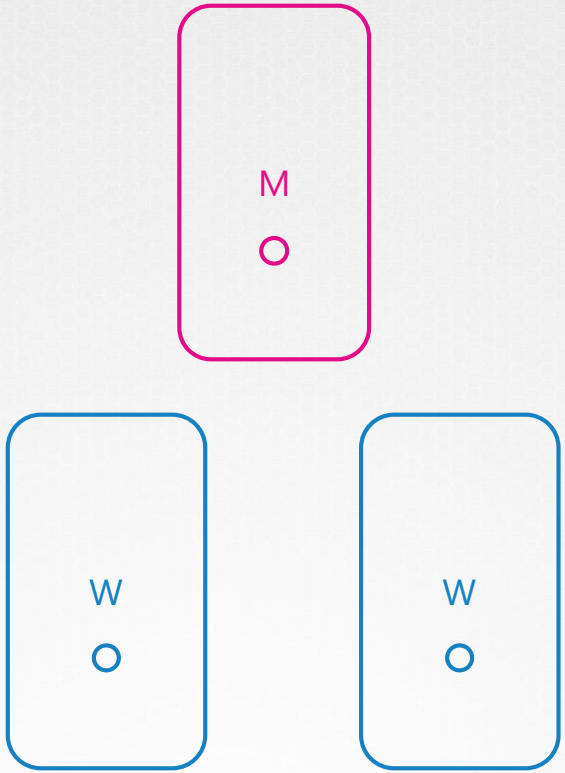
KODEKLOUD

Master Nodes



KODEKLOUD

Our Design



KODEKLOUD

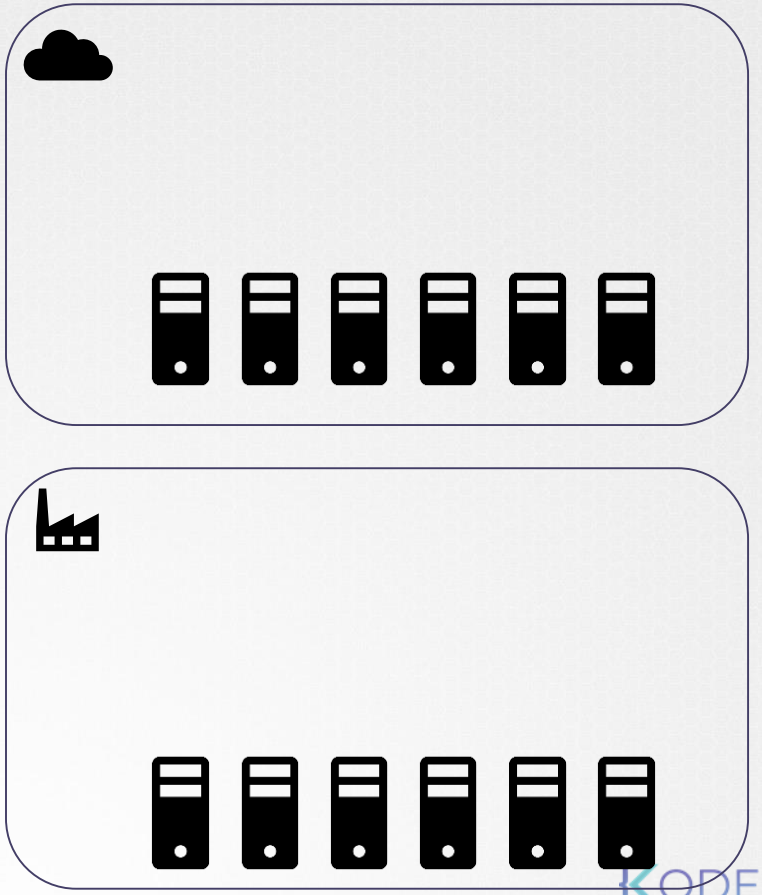
KODEKLOUD

Course Objectives

- Core Concepts
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Networking
- Installation, Configuration & Validation
  - Design a Kubernetes Cluster
  - Choose Kubernetes Infrastructure Config
  - Choose a Network Solution
  - HA Kubernetes Cluster
  - Provision Infrastructure
  - Secure Cluster Communication
  - Kubernetes Release Binaries
  - Install Kubernetes Master Nodes
  - Install Kubernetes Worker Nodes
  - TLS Bootstrapping a Node
  - Node end-to-end tests
  - Run & Analyze end-to-end tes
- Troubleshooting

KODEKLOUD

Choosing Kubernetes Infrastructure



KODEKLOUD

Linux

Windows

KODEKLOUD

Minikube

KUBEADM

Deploys VMs

Requires VMs to be ready

Singe Node Cluster

Singe/Multi Node Cluster

KODEKLOUD

Turnkey Solutions

- You Provision VMs
- You Configure VMs
- You Use Scripts to Deploy Cluster
- You Maintain VMs yourself
- Eg: Kubernetes on AWS using KOPS

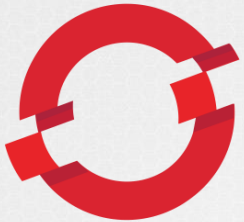
Hosted Solutions

(Managed Solutions)

- Kubernetes-As-A-Service
- Provider provisions VMs
- Provider installs Kubernetes
- Provider maintains VMs
- Eg: Google Container Engine (GKE)

KODEKLOUD

Turnkey Solutions

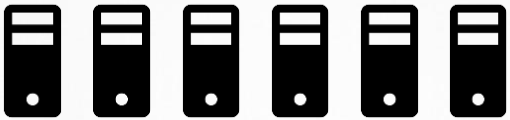


OpenShift

Cloud Foundry
Container Runtime

VMware Cloud
PKS

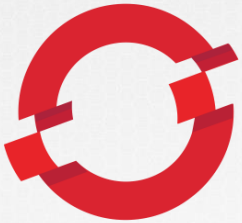
Vagrant



KUBERCLOUD

Hosted Solutions

Google Container Engine **(GKE)**



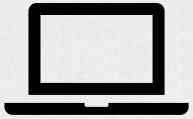
OpenShift Online

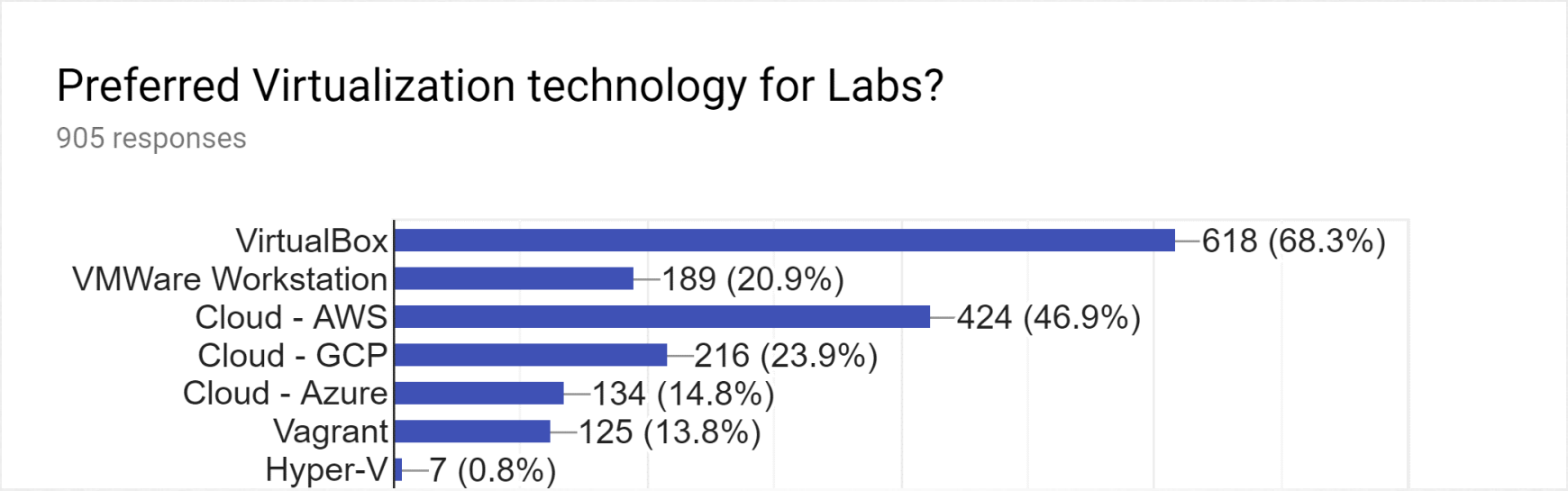
Azure Kubernetes Service

Amazon Elastic Container Service for Kubernetes (EKS)

KODEKLOUD

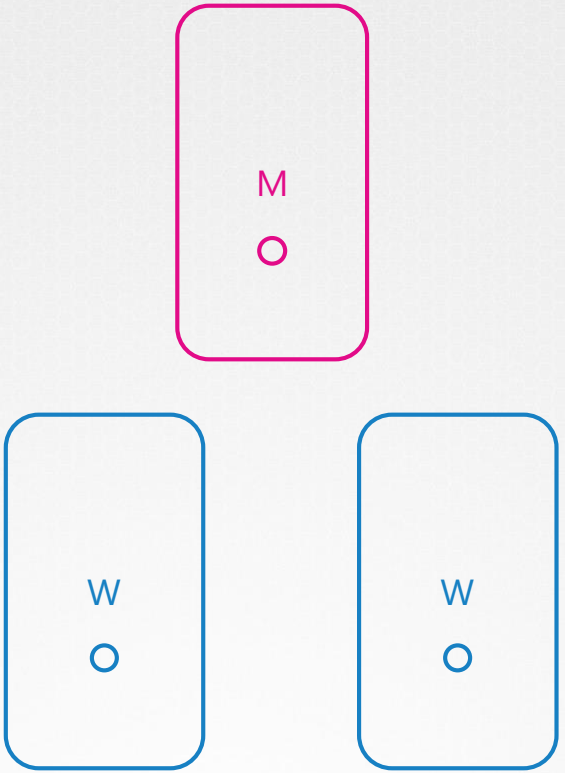
Our Choice





KODEKLOUD

Our Design



KODEKLOUD

KODEKLOUD

Course Objectives

- Core Concepts
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Networking
- Installation, Configuration & Validation
  - Design a Kubernetes Cluster
  - Choose Kubernetes Infrastructure Config
  - Choose a Network Solution
  - HA Kubernetes Cluster
  - Provision Infrastructure
  - Secure Cluster Communication
  - Kubernetes Release Binaries
  - Install Kubernetes Master Nodes
  - Install Kubernetes Worker Nodes
  - TLS Bootstrapping a Node
  - Node end-to-end tests
  - Run & Analyze end-to-end tes
- Troubleshooting

KODEKLOUD

Chose a Networking Solution

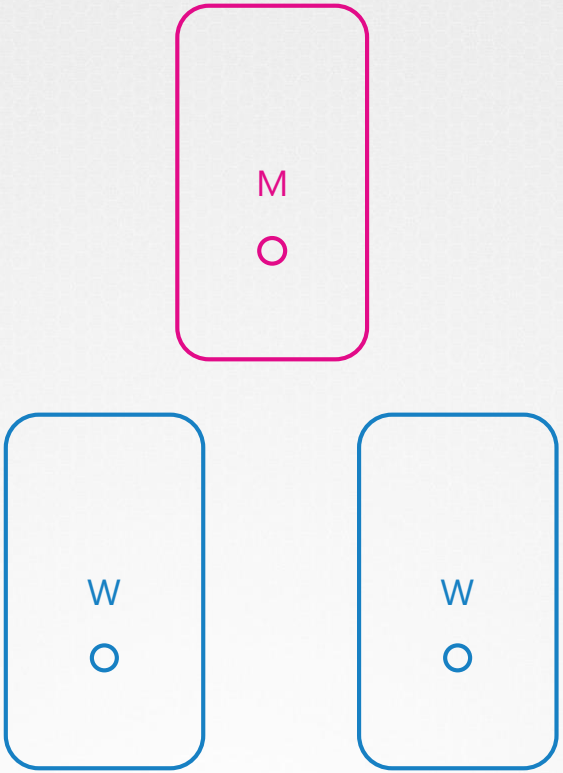
GCE

nuagenetworks

KODEKLOUD

Our Design

POD CIDR: 10.32.0.0/12
Service CIDR: 10.96.0.0/24



KODEKLOUD

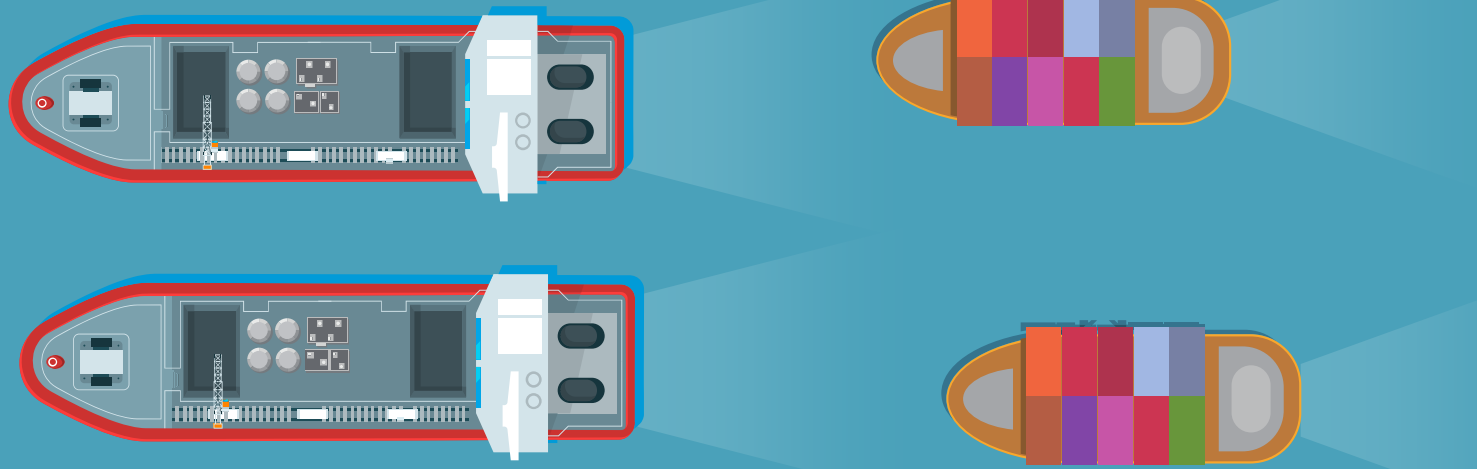
KODEKLOUD

Course Objectives

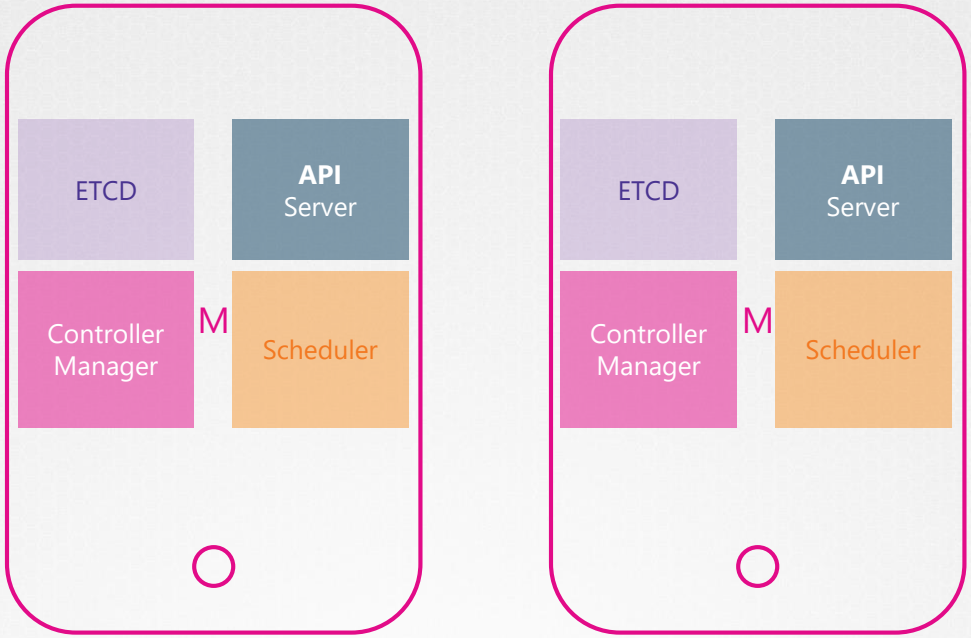
- Core Concepts
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Networking
- Installation, Configuration & Validation
  - Design a Kubernetes Cluster
  - Choose Kubernetes Infrastructure Config
  - Choose a Network Solution
  - HA Kubernetes Cluster
  - Provision Infrastructure
  - Secure Cluster Communication
  - Kubernetes Release Binaries
  - Install Kubernetes Master Nodes
  - Install Kubernetes Worker Nodes
  - TLS Bootstrapping a Node
  - Node end-to-end tests
  - Run & Analyze end-to-end tes
- Troubleshooting

KODEKLOUD

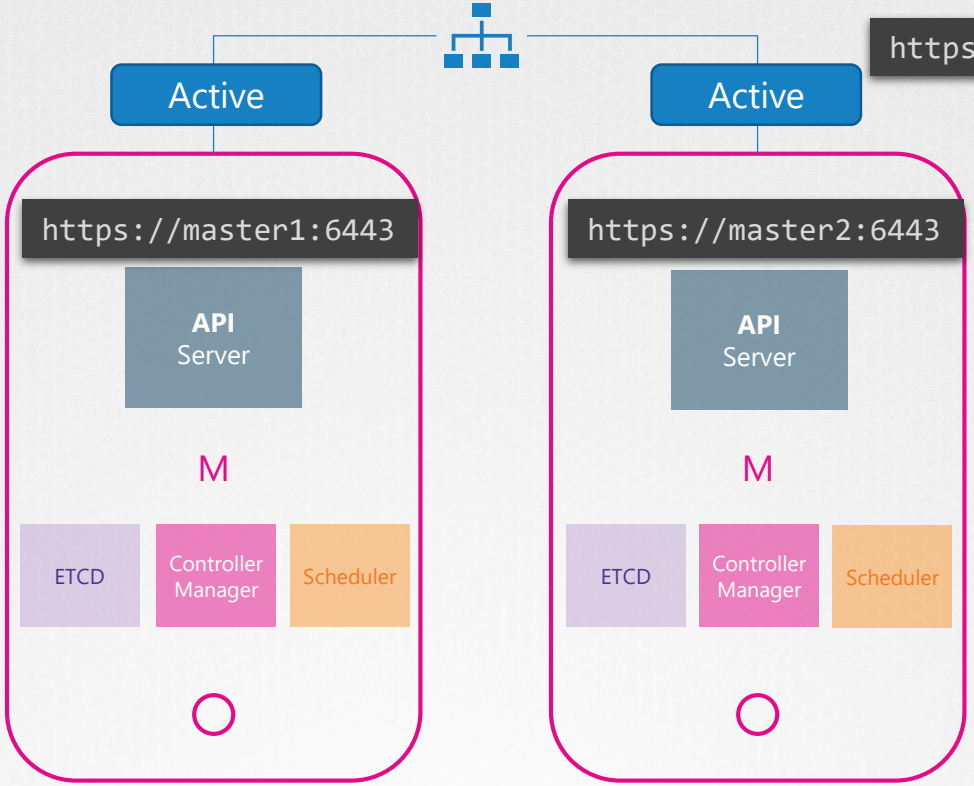
HA Kubernetes Cluster



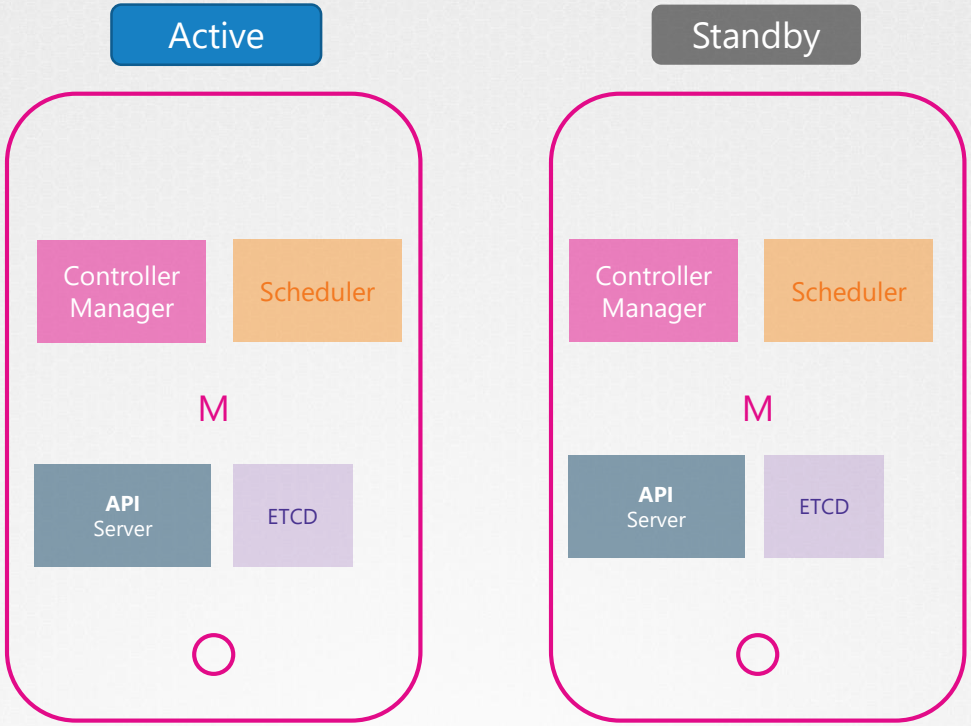
KODEKLOUD


ETCD
API Server
Controller Manager
M
Scheduler
ETCD
API Server
Controller Manager
M
Scheduler

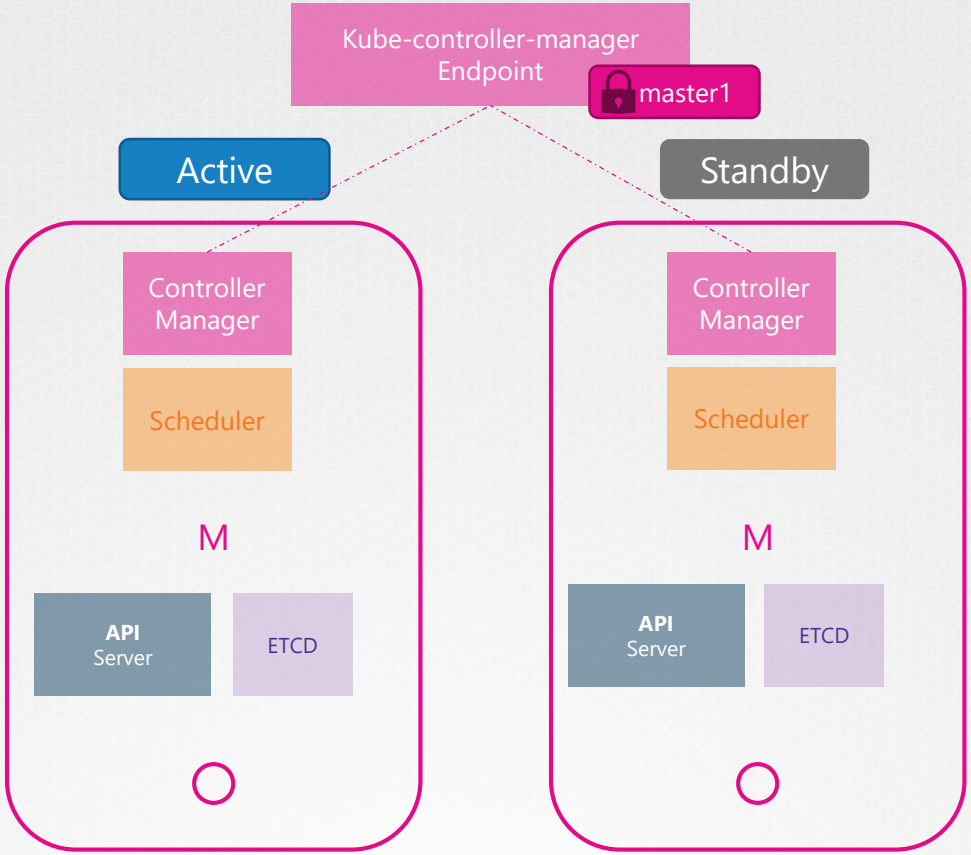
KODEKLOUD


https://load-balancer:6443
Active
Active
https://master1:6443
https://master2:6443
API
Server
API
Server
M
M
ETCD
Controller
Manager
Scheduler
ETCD
Controller
Manager
Scheduler

KODEKLOUD

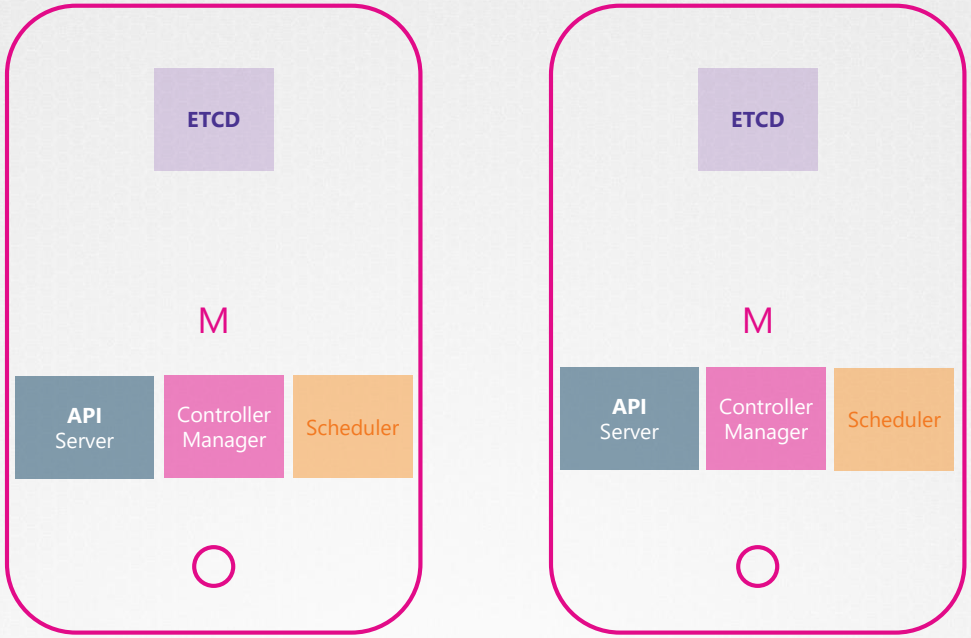

Active
Standby
Controller Manager
Scheduler
M
API Server
ETCD
Controller Manager
Scheduler
M
API Server
ETCD

KODEKLOUD




```
kube-controller-manager --leader-elect true  [other options]
                        --leader-elect-lease-duration 15s
                        --leader-elect-renew-deadline 10s
                        --leader-elect-retry-period 2s
```

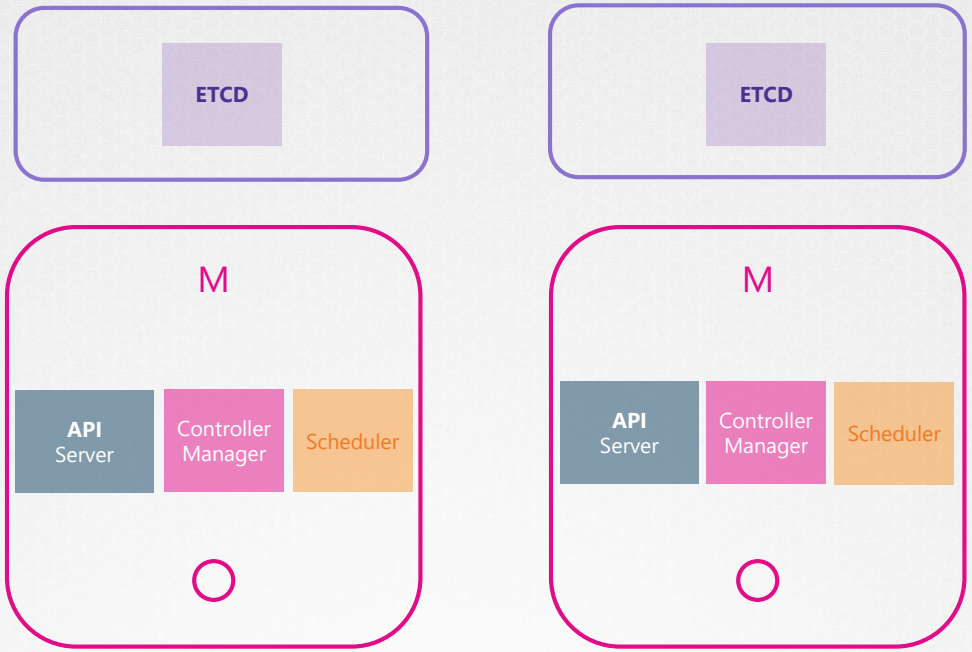
Stacked Topology



- ✓ Easier to setup
- ✓ Easier to manage
- ✓ Fewer Servers
- ❖ Risk during failures

KODEKLOUD

External ETCD Topology

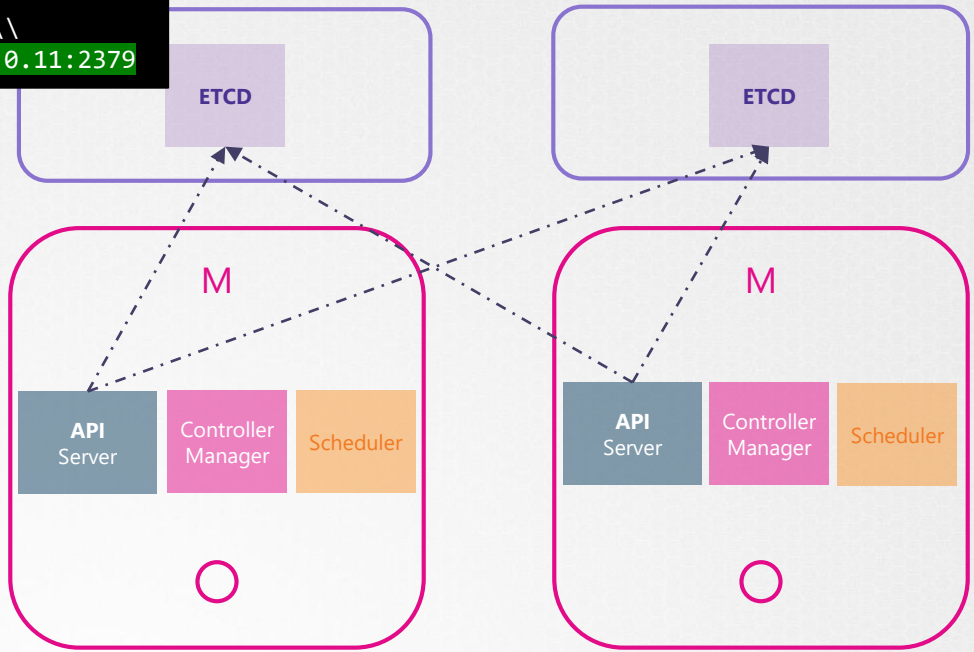


- ✓ Less Risky
- ❖ Harder to Setup
- ❖ More Servers

KODEKLOUD

```
cat /etc/systemd/system/kube-apiserver.service
```

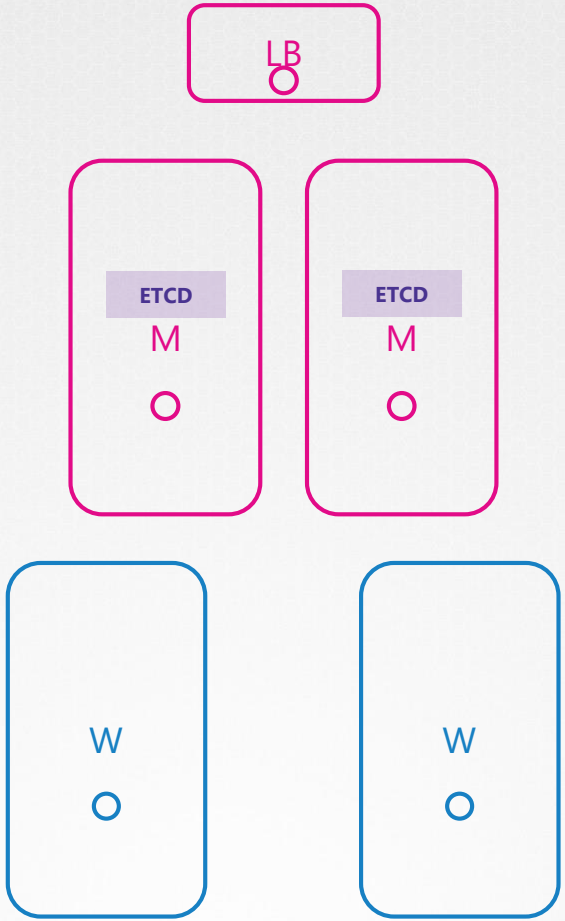
```
[Service]
ExecStart=/usr/local/bin/kube-apiserver \\
  --advertise-address=${INTERNAL_IP} \\
  --allow-privileged=true \\
  --apiserver-count=3 \\
  --etcd-cafile=/var/lib/kubernetes/ca.pem \\
  --etcd-certfile=/var/lib/kubernetes/kubernetes.pem \\
  --etcd-keyfile=/var/lib/kubernetes/kubernetes-key.pem \\
  --etcd-servers=https://10.240.0.10:2379,https://10.240.0.11:2379
```



KODEKLOUD

Our Design

weaveworks



KODEKLOUD

KODEKLOUD

Course Objectives

- Core Concepts
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Networking
- Installation, Configuration & Validation
  - Design a Kubernetes Cluster
  - Choose Kubernetes Infrastructure Config
  - Choose a Network Solution
  - HA Kubernetes Cluster
  - Provision Infrastructure
  - Secure Cluster Communication
  - Kubernetes Release Binaries
  - Install Kubernetes Master Nodes
  - Install Kubernetes Worker Nodes
  - TLS Bootstrapping a Node
  - Node end-to-end tests
  - Run & Analyze end-to-end tes
- Troubleshooting

KODEKLOUD

ETCD

In HA

KODEKLOUD

Objectives

- What is ETCD?
- What is a Key-Value Store?
- How to get started quickly?
- How to operate ETCD?
- What is a distributed system?
- How ETCD Operates
- RAFT Protocol
- Best practices on number of nodes

KODEKLOUD

ETCD is a distributed reliable key-value store that is Simple, Secure & Fast

key-value store

Tabular/Relational Databases

Name	Age	Location	Salary	Grade
John Doe	45	New York	5000	
Dave Smith	34	New York	4000	
Aryan Kumar	10	New York		A
Lauren Rob	13	Bangalore		C
Lily Oliver	15	Bangalore		B

KODEKLOUD

key-value store

Key	Value
Name	John Doe
Age	45
Location	New York
Salary	5000

Key	Value
Name	Dave Smith
Age	34
Location	New York
Salary	4000

Key	Value
Name	Aryan Kumar
Age	10
Location	New York
Grade	A

Key	Value
Name	Lauren Rob
Age	13
Location	Bangalore
Grade	C

Key	Value
Name	Lily Oliver
Age	15
Location	Bangalore
Grade	B

KODEKLOUD

key-value store

```
{
  "name": "John Doe",
  "age": 45,
  "location": "New York",
  "salary": 5000
}
```

```
{
  "name": "Dave Smith",
  "age": 34,
  "location": "New York",
  "salary": 4000,
  "organization": "ACME"
}
```

```
{
  "name": "Aryan Kumar",
  "age": 10,
  "location": "New York",
  "Grade": "A"
}
```

```
{
  "name": "Lily Oliver",
  "age": 15,
  "location": "Bangalore",
  "Grade": "B"
}
```

```
{
  "name": "Lauren Rob",
  "age": 13,
  "location": "Bangalore",
  "Grade": "C"
}
```

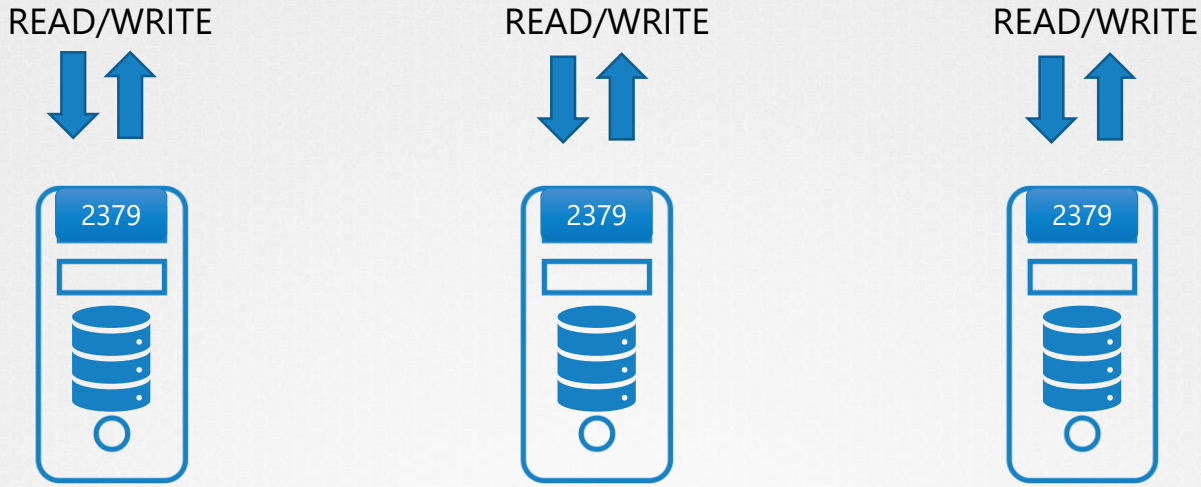
KODEKLOUD

ETCD is a distributed reliable key-value store that is Simple, Secure & Fast

distributed

KODEKLOUD

Consistent

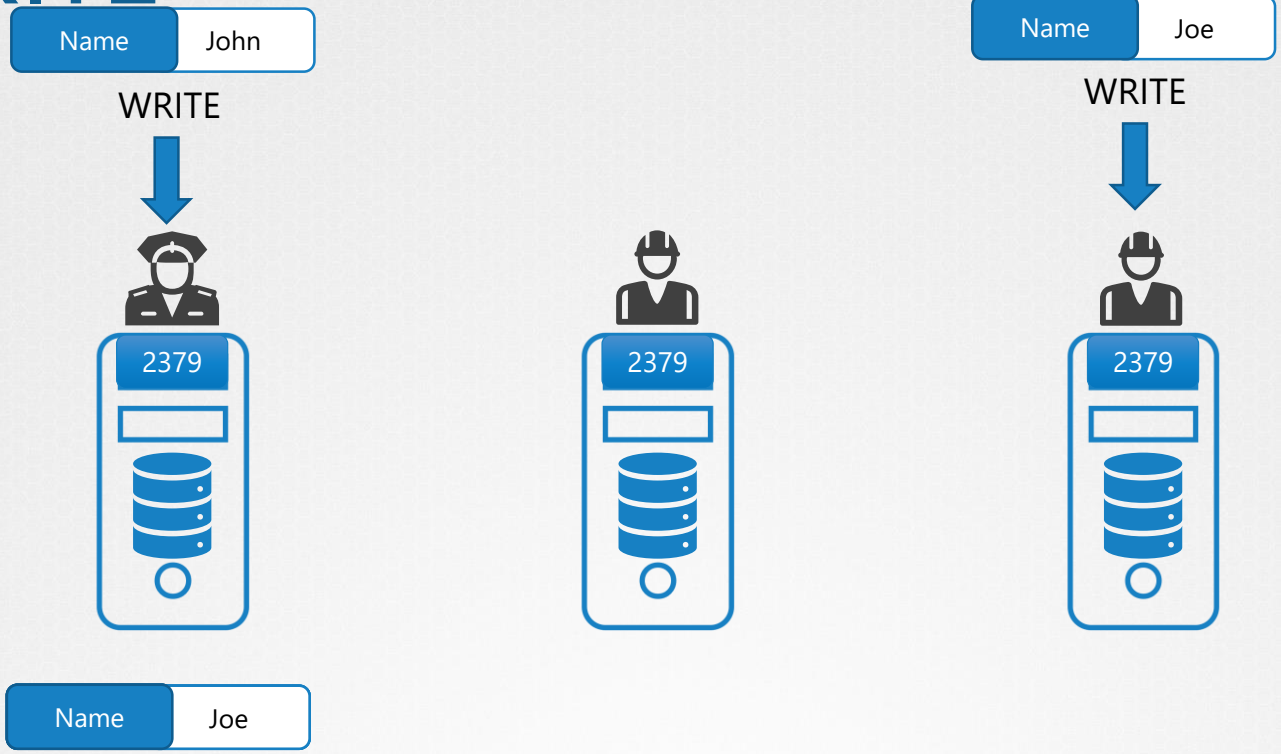


KODEKLOUD

READ

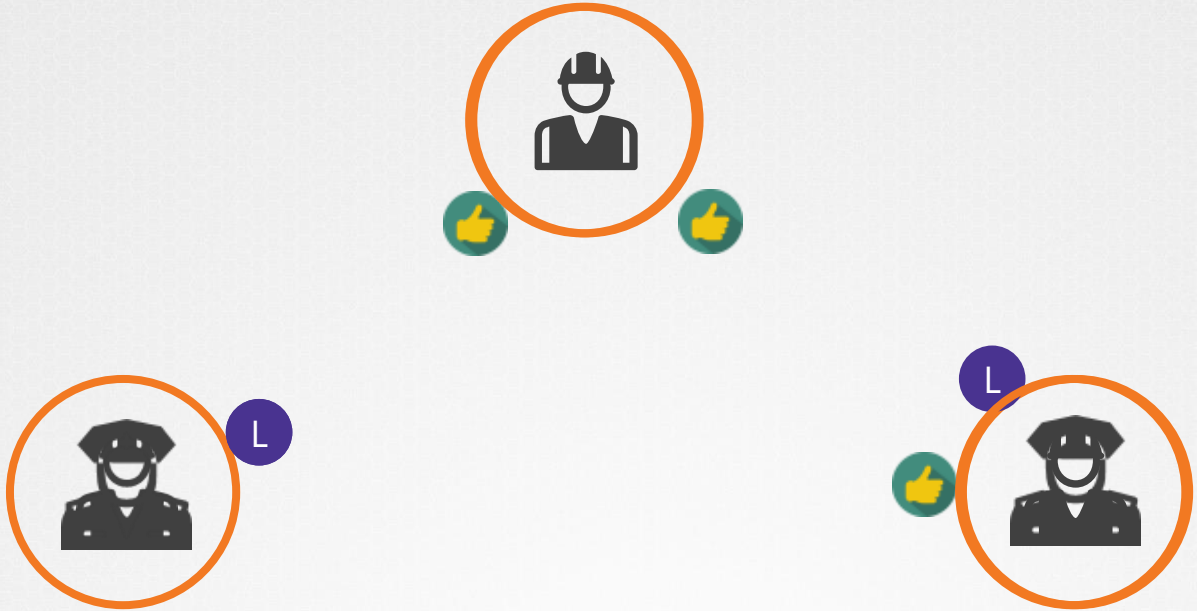
KODEKLOUD

WRITE

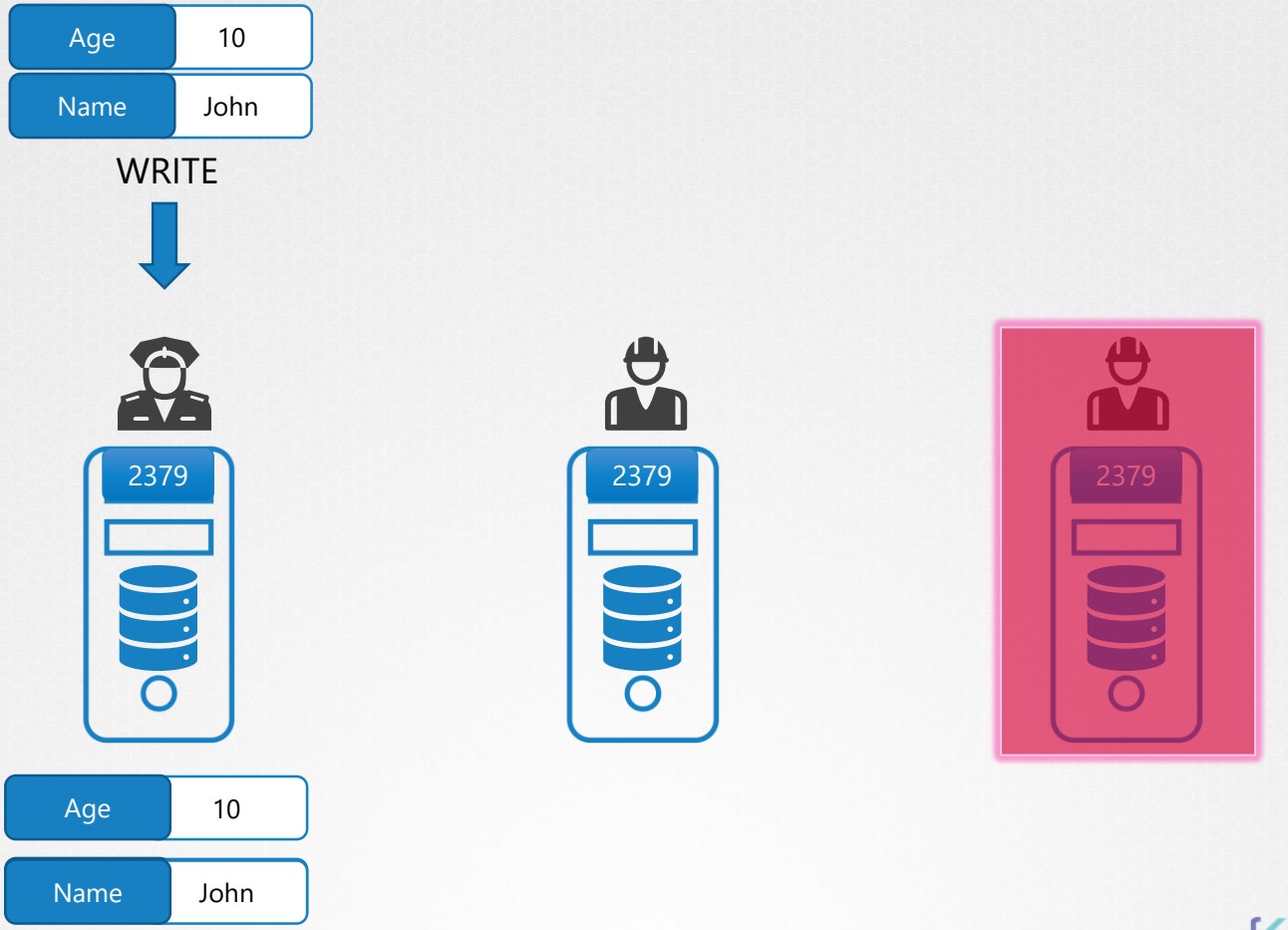


KODEKLOUD

Leader Election - RAFT



KODEKLOUD



KODEKLOUD

Instances	Quorum	Fault Tolerance
1	1	0
2	2	0
3	2	1
4	3	1
5	3	2
6	4	2
7	4	3

Majority = N/2 + 1

Quorum of 2 = 2/2 + 1 = 2

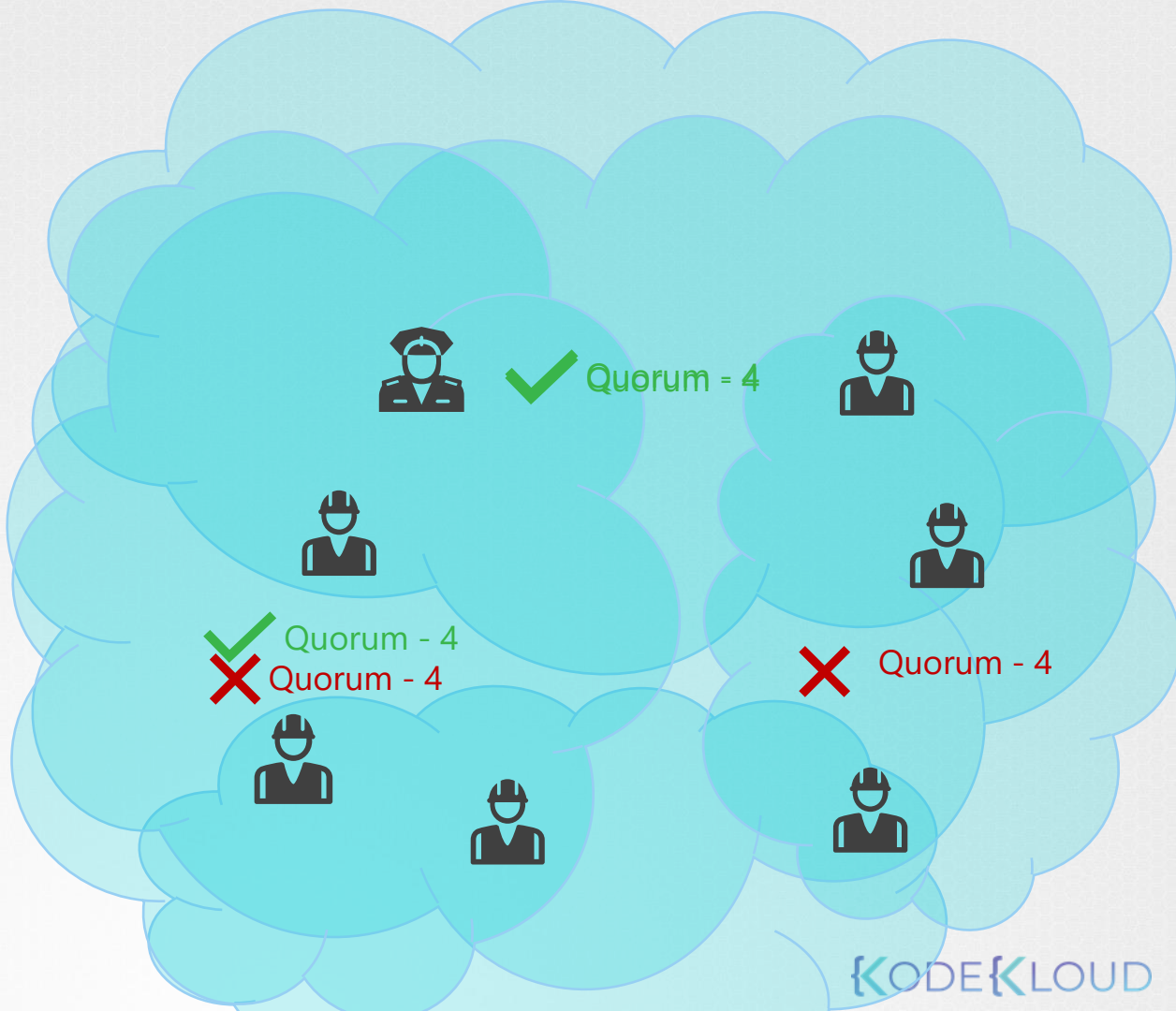
Quorum of 3 = 3/2 + 1 = 2.5 ~= 2

Quorum of 5 = 5/2 + 1 = 3.5 ~= 3

KODEKLOUD

Odd or even?

Managers	Majority	Fault Tolerance
1	1	0
2	2	0
3	2	1
4	3	1
5	3	2
6	4	2
7	4	3



KODEKLOUD

Getting Started

```
wget -q --https-only \
    "https://github.com/coreos/etcd/releases/download/v3.3.9/etcd-v3.3.9-linux-amd64.tar.gz"
```

```
tar -xvf etcd-v3.3.9-linux-amd64.tar.gz
```

```
mv etcd-v3.3.9-linux-amd64/etcd* /usr/local/bin/
```

```
mkdir -p /etc/etcd /var/lib/etcd
```

```
cp ca.pem kubernetes-key.pem kubernetes.pem /etc/etcd/
```

KODEKLOUD

etcd.service

```
ExecStart=/usr/local/bin/etcd \\
  --name ${ETCD_NAME} \\
  --cert-file=/etc/etcd/kubernetes.pem \\
  --key-file=/etc/etcd/kubernetes-key.pem \\
  --peer-cert-file=/etc/etcd/kubernetes.pem \\
  --peer-key-file=/etc/etcd/kubernetes-key.pem \\
  --trusted-ca-file=/etc/etcd/ca.pem \\
  --peer-trusted-ca-file=/etc/etcd/ca.pem \\
  --peer-client-cert-auth \\
  --client-cert-auth \\
  --initial-advertise-peer-urls https://${INTERNAL_IP}:2380 \\
  --listen-peer-urls https://${INTERNAL_IP}:2380 \\
  --listen-client-urls https://${INTERNAL_IP}:2379,https://127.0.0.1:2379 \\
  --advertise-client-urls https://${INTERNAL_IP}:2379 \\
  --initial-cluster-token etcd-cluster-0 \\
  --initial-cluster peer-1=https://${PEER1_IP}:2380,peer-2=https://${PEER2_IP}:2380 \\
  --initial-cluster-state new \\
  --data-dir=/var/lib/etcd
```

KODEKLOUD

ETCDCTL

```
export ETCDCTL_API=3
```

```
etcdctl put name john
```

```
etcdctl get name
```

```
name
john
```

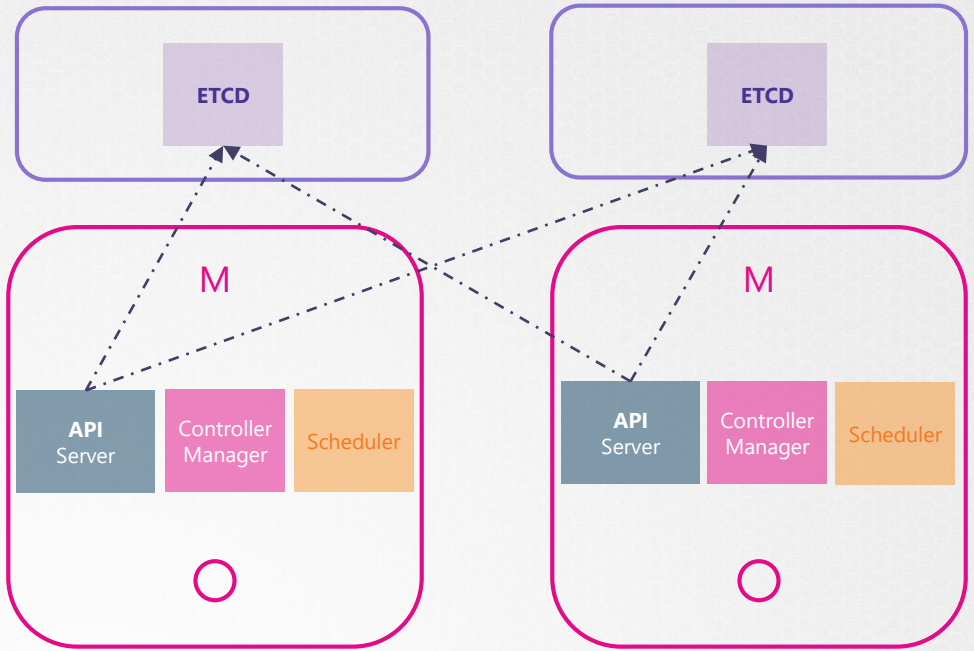
```
etcdctl get / --prefix --keys-only
```

```
name
```

KODEKLOUD

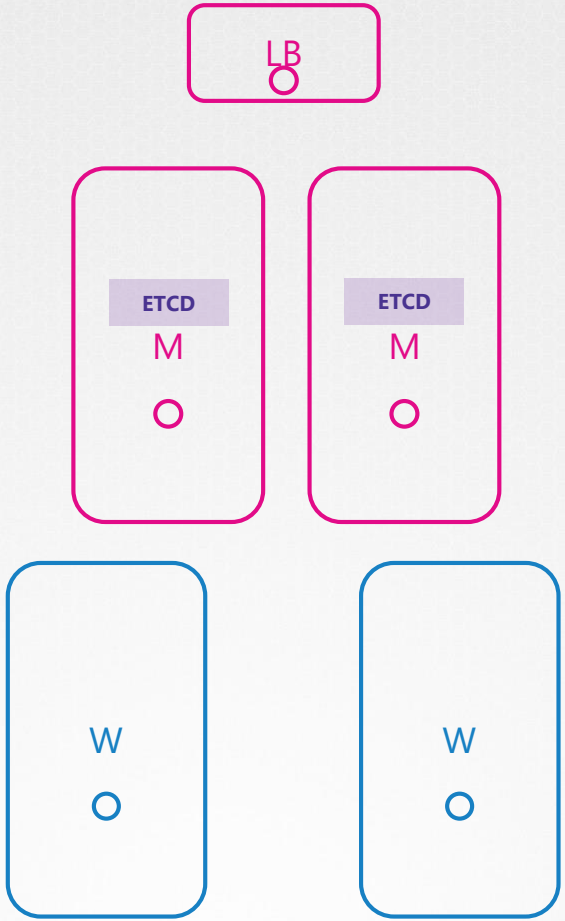
Number of Nodes

Instances	Quorum	Fault Tolerance
1	1	0
2	2	0
3	2	1
4	3	1
5	3	2
6	4	2
7	4	3



KODEKLOUD

Our Design



KODEKLOUD

KODEKLOUD

DEMO

Pre-Requisites

KODEKLOUD

Course Objectives

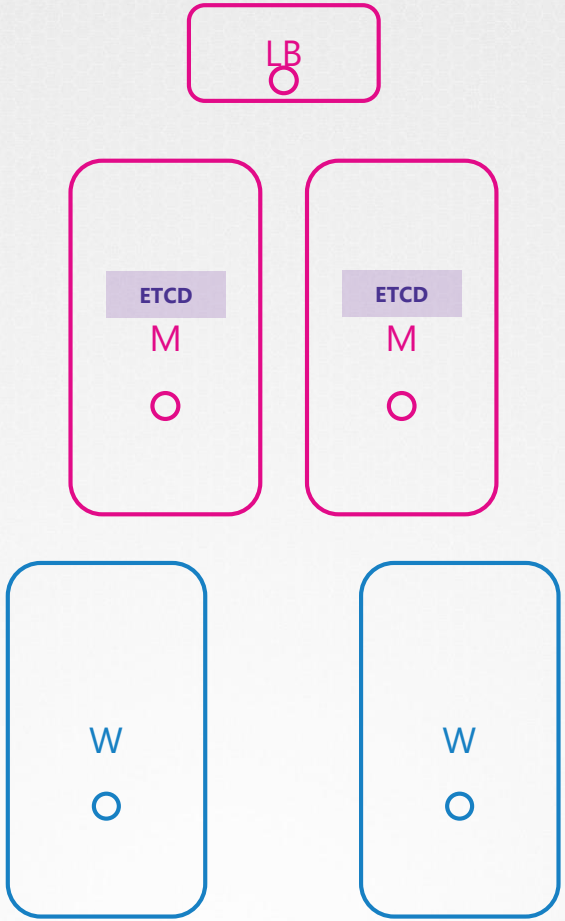
- Core Concepts
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Networking
- Installation, Configuration & Validation
  - Design a Kubernetes Cluster
  - Choose Kubernetes Infrastructure Config
  - Choose a Network Solution
  - HA Kubernetes Cluster
  - Provision Infrastructure
  - Secure Cluster Communication
  - Kubernetes Release Binaries
  - Install Kubernetes Master Nodes
  - Install Kubernetes Worker Nodes
  - TLS Bootstrapping a Node
  - Node end-to-end tests
  - Run & Analyze end-to-end tes
- Troubleshooting

KODEKLOUD

Provision Infrastructure

Our Design

weaveworks



KODEKLOUD

VAGRANT

KODEKLOUD

```
vagrant up
```

- Deploys 5 VMs - 2 Master, 2 Worker and 1 Loadbalancer with the name 'kubernetes-ha-* ‘
- Set's IP addresses in the range 192.168.5
- Add's a DNS entry to each of the nodes to access internet
- Install's Docker on the nodes

KODEKLOUD

DEMO

Provision Infrastructure

KODEKLOUD

DEMO

Install Client Tools

KODEKLOUD

Course Objectives

- Core Concepts
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Networking
- Installation, Configuration & Validation
  - Design a Kubernetes Cluster
  - Choose Kubernetes Infrastructure Config
  - Choose a Network Solution
  - HA Kubernetes Cluster
  - Provision Infrastructure
  - Secure Cluster Communication
  - Kubernetes Release Binaries
  - Install Kubernetes Master Nodes
  - Install Kubernetes Worker Nodes
  - TLS Bootstrapping a Node
  - Node end-to-end tests
  - Run & Analyze end-to-end tes
- Troubleshooting

DEMO

Secure Cluster Communication

KODEKLOUD

DEMO

Kube Config Files

KODEKLOUD

DEMO

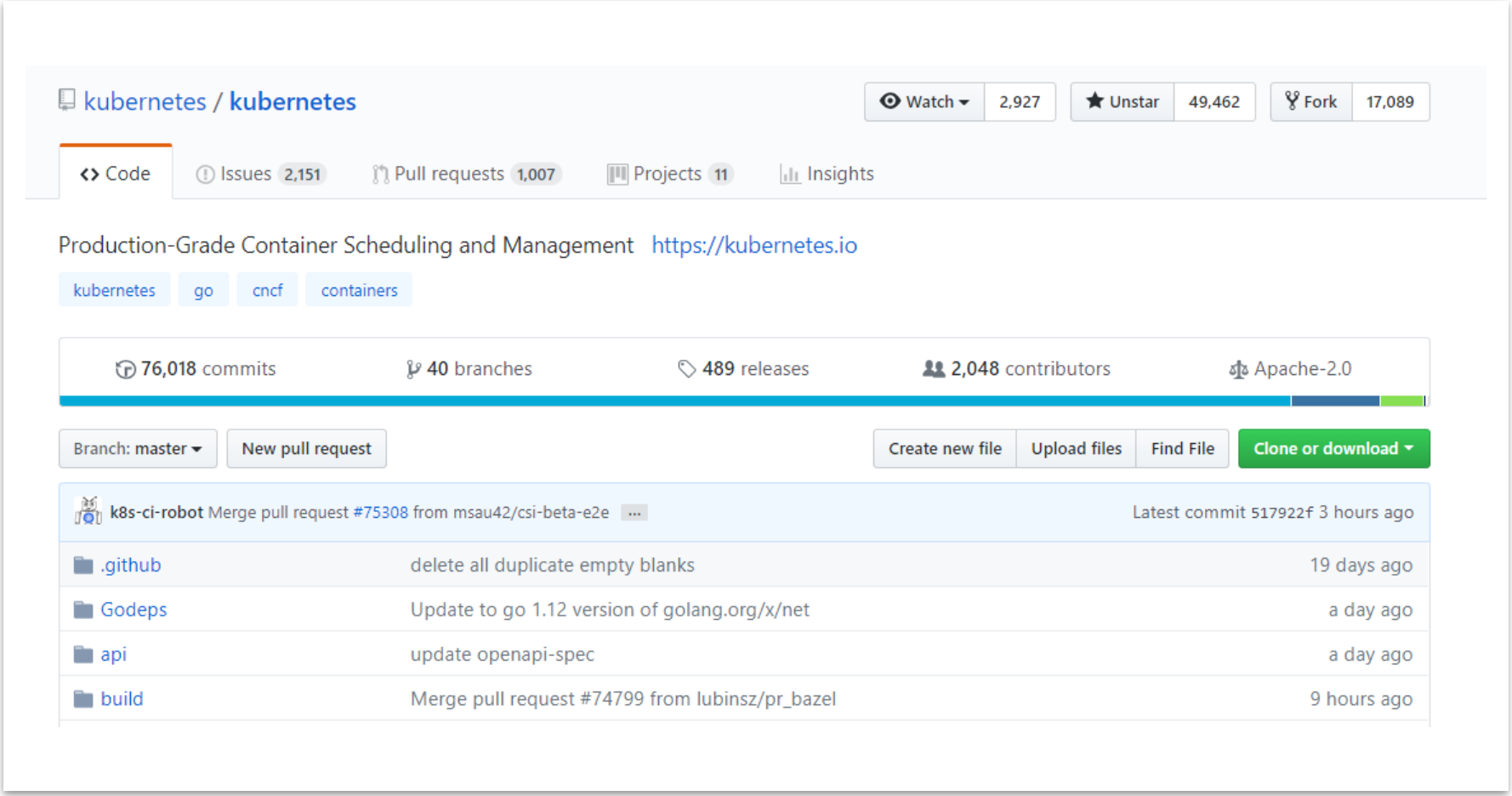
Data Encryption

KODEKLOUD

Course Objectives

- Core Concepts
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Networking
- Installation, Configuration & Validation
  - Design a Kubernetes Cluster
  - Choose Kubernetes Infrastructure Confi
  - Choose a Network Solution
  - HA Kubernetes Cluster
  - Provision Infrastructure
  - Secure Cluster Communication
  - Kubernetes Release Binaries
  - Install Kubernetes Master Nodes
  - Install Kubernetes Worker Nodes
  - TLS Bootstrapping a Node
  - Node end-to-end tests
  - Run & Analyze end-to-end tes
- Troubleshooting

Kubernetes Release Binaries


kubernetes / kubernetes
Watch 2,927
Unstar 49,462
Fork 17,089
Code
Issues 2,151
Pull requests 1,007
Projects 11
Insights
Production-Grade Container Scheduling and Management https://kubernetes.io
kubernetes
go
cncf
containers
76,018 commits
40 branches
489 releases
2,048 contributors
Apache-2.0
Branch: master
New pull request
Create new file
Upload files
Find File
Clone or download
k8s-ci-robot Merge pull request #75308 from msau42/csi-beta-e2e
Latest commit 517922f 3 hours ago
.github
delete all duplicate empty blanks
19 days ago
Godeps
Update to go 1.12 version of golang.org/x/net
a day ago
api
update openapi-spec
a day ago
build
Merge pull request #74799 from lubinsz/pr_bazel
9 hours ago

v1.13.3

721bfa7

v1.13.3

k8s-release-robot released this on Feb 2 · 64 commits to release-1.13 since this release

See kubernetes-announce@ and CHANGELOG-1.13.md for details.

SHA512 for `kubernetes.tar.gz`:

```
151af896b72c7fd09c05da1a7685e8b2f167c717adbe5776f80a264171e5f3359a948af93642856e0bfbabb49d3bf9c274085ea
cf6109c4b972ba5bc9d24b8a7
```

Additional binary downloads are linked in the CHANGELOG-1.13.md.

▼ **Assets** 3

kubernetes.tar.gz	1.87 MB
Source code (zip)	
Source code (tar.gz)	

```
wget https://github.com/kubernetes/kubernetes/releases/download/v1.13.3/kubernetes.tar.gz
```

```
kubernetes.tar.gz
```

```
tar -xzvf kubernetes.tar.gz
```

```
kubernetes
```

```
cd kubernetes; ls
```

```
client  cluster  docs  hack  LICENSES  platforms  README.md
server  version
```

```
cluster/get-kube-binaries.sh
```

```
client/kubernetes-client-linux-amd64.tar.gz
server/kubernetes-server-linux-amd64.tar.gz

Extracting /root/kubernetes/client/kubernetes-client-linux-amd64.tar.gz into /root/kubernetes/platforms/linux/amd64
Add '/root/kubernetes/client/bin' to your PATH to use newly-installed binaries.
```

```
cd server; tar –zxvf kubernetes-server-linux-amd64.tar.gz
```

```
Kubernetes/server
```

KODEKLOUD

```
cluster/get-kube-binaries.sh
```

```
client/kubernetes-client-linux-amd64.tar.gz
server/kubernetes-server-linux-amd64.tar.gz

Extracting /root/kubernetes/client/kubernetes-client-linux-amd64.tar.gz into /root/kubernetes/platforms/linux/amd64
Add '/root/kubernetes/client/bin' to your PATH to use newly-installed binaries.
```

```
cd server; tar –zxvf kubernetes-server-linux-amd64.tar.gz
```

```
Kubernetes/server
```

```
ls kubernetes/server/bin
```

```
apiextensions-apiserver          kubeadm                     kube-proxy.docker_tag          mounter
cloud-controller-manager         kube-apiserver              kube-controller-manager.tar    kube-proxy.tar
kubectl                          kube-scheduler              cloud-controller-manager.tar   kube-apiserver.tar
kubelet                          kube-scheduler.docker_tag
hyperkube                        kube-controller-manager     kube-proxy                     kube-scheduler.tar
```

KODEKLOUD

kubernetes

Setup

Setup

- Picking the Right Solution
- Downloading Kubernetes
  - Building a release
  - v1.13 Release Notes
- Bootstrapping Clusters with kubeadm
- Turnkey Cloud Solutions
- Custom Cloud Solutions
- On-Premises VMs
- Kubernetes Version and Version Skew Support Policy
- Building Large Clusters
- Running in Multiple Zones
- CRI installation
- Installing Kubernetes with Digital Rebar Provision (DRP) via KRIB
- PKI Certificates and Requirements
- Running Kubernetes Locally via Minikube
- Validate Node Setup

v1.13 Release Notes

v1.13.0
- Downloads for v1.13.0
  - Client Binaries
  - Server Binaries
  - Node Binaries

Kubernetes 1.13 Release Notes
- Security Content
- Urgent Upgrade Notes
  - (No, really, you MUST do this before you upgrade)
- Known Issues
- Deprecations
- Major Themes
  - SIG API Machinery
  - SIG Auth
  - SIG AWS
  - SIG Azure
  - SIG Big Data
  - SIG CLI
  - SIG Cloud Provider
  - SIG Cluster Lifecycle
  - SIG IBM Cloud

KLOUD

External Dependencies

- Default etcd server is unchanged at v3.2.24 since Kubernetes 1.12. (#68318)
- The list of validated docker versions remain unchanged at 1.11.1, 1.12.1, 1.13.1, 17.03, 17.06, 17.09, 18.06 since Kubernetes 1.12. (#68495)
- The default Go version was updated to 1.11.2. (#70665)
- The minimum supported Go version was updated to 1.11.2 (#69386)
- CNI is unchanged at v0.6.0 since Kubernetes 1.10 (#51250)
- CSI is updated to 1.0.0. Pre-1.0.0 API support is now deprecated. (#71020])
- The dashboard add-on has been updated to v1.10.0. (#68450)
- Heapster remains at v1.6.0-beta, but is now retired in Kubernetes 1.13 (#67074)
- Cluster Autoscaler has been upgraded to v1.13.0 (#71513)
- kube-dns is unchanged at v1.14.13 since Kubernetes 1.12 (#68900)
- Influxdb is unchanged at v1.3.3 since Kubernetes 1.10 (#53319)
- Grafana is unchanged at v4.4.3 since Kubernetes 1.10 (#53319)
- Kibana has been upgraded to v6.3.2. (#67582)
- CAdvisor has been updated to v0.32.0 (#70964)
- fluentd-gcp-scaler has been updated to v0.5.0 (#68837)
- Fluentd in fluentd-elasticsearch is unchanged at v1.2.4 since Kubernetes 1.11 (#67434)

LOUD

DEMO

Download Release Binaries

KODEKLOUD

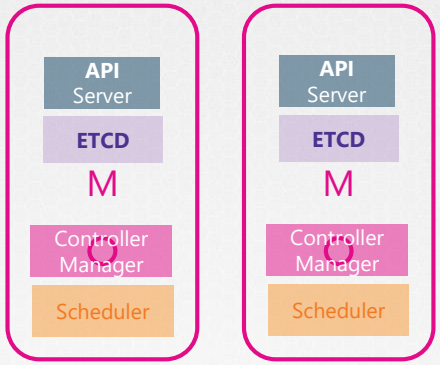
Course Objectives

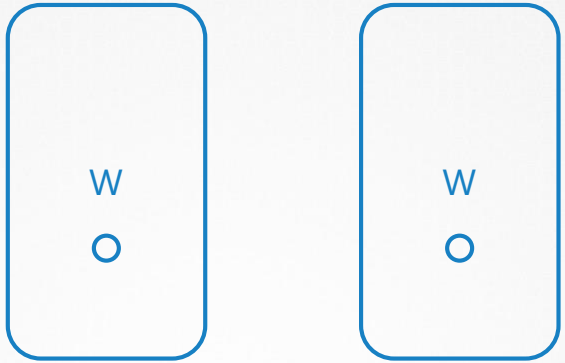
- Core Concepts
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Networking
- Installation, Configuration & Validation
  - Design a Kubernetes Cluster
  - Choose Kubernetes Infrastructure Config
  - Choose a Network Solution
  - HA Kubernetes Cluster
  - Provision Infrastructure
  - Secure Cluster Communication
  - Kubernetes Release Binaries
  - Install Kubernetes Master Nodes
  - Install Kubernetes Worker Nodes
  - TLS Bootstrapping a Node
  - Node end-to-end tests
  - Run & Analyze end-to-end tes
- Troubleshooting

KODEKLOUD

Install Master

Our Design





- ❑ Deploy ETCD Cluster
- ❑ Deploy Control Plane Components
- ❑ Network Loadbalancer

KODEKLOUD

KODEKLOUD

DEMO

Install ETCD Cluster

KODEKLOUD

DEMO

Install Control-plane Components

KODEKLOUD

DEMO

Install Load Balancer

KODEKLOUD

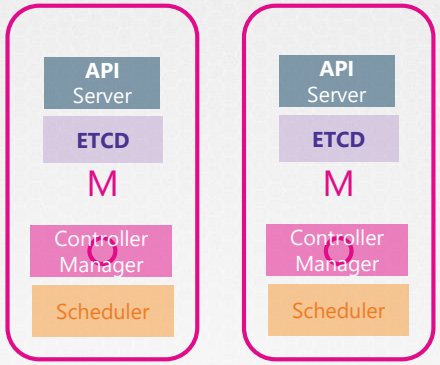
Course Objectives

- Core Concepts
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Networking
- Installation, Configuration & Validation
  - Design a Kubernetes Cluster
  - Choose Kubernetes Infrastructure Config
  - Choose a Network Solution
  - HA Kubernetes Cluster
  - Provision Infrastructure
  - Secure Cluster Communication
  - Kubernetes Release Binaries
  - Install Kubernetes Master Nodes
  - Install Kubernetes Worker Nodes
  - TLS Bootstrapping a Node
  - Node end-to-end tests
  - Run & Analyze end-to-end tes
- Troubleshooting

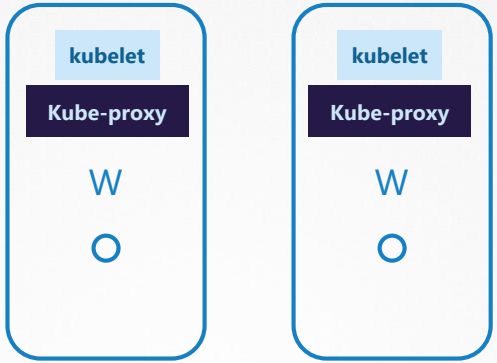
Install Worker Nodes

KODEKLOUD

Our Design



- ✓ Deploy ETCD Cluster
- ✓ Deploy Control Plane Components
- ✓ Network Loadbalancer



- ❑ Generate CERTs for Worker-1
- ❑ Configure Kubelet for Worker-1
- ❑ Renew Certificates
- ❑ Configure kube-proxy

TLS Bootstrap:

- ❑ Worker-2 to create and configure certificates by itself
- ❑ Configure Kubelet for Worker-2
- ❑ Worker-2 to renew certificates by itself
- ❑ Configure kube-proxy

KODEKLOUD

DEMO

Install Worker-1

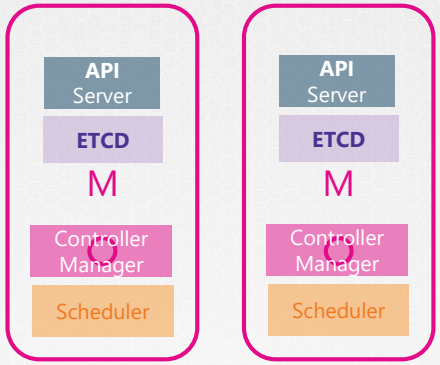
KODEKLOUD

Course Objectives

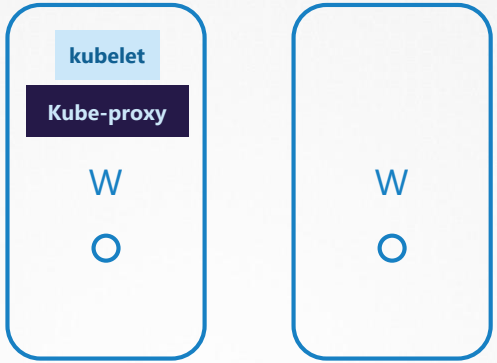
- Core Concepts
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Networking
- Installation, Configuration & Validation
  - Design a Kubernetes Cluster
  - Choose Kubernetes Infrastructure Config
  - Choose a Network Solution
  - HA Kubernetes Cluster
  - Provision Infrastructure
  - Secure Cluster Communication
  - Kubernetes Release Binaries
  - Install Kubernetes Master Nodes
  - Install Kubernetes Worker Nodes
  - TLS Bootstrapping a Node
  - Node end-to-end tests
  - Run & Analyze end-to-end tes
- Troubleshooting

TLS Bootstrap Kubelet

Our Design



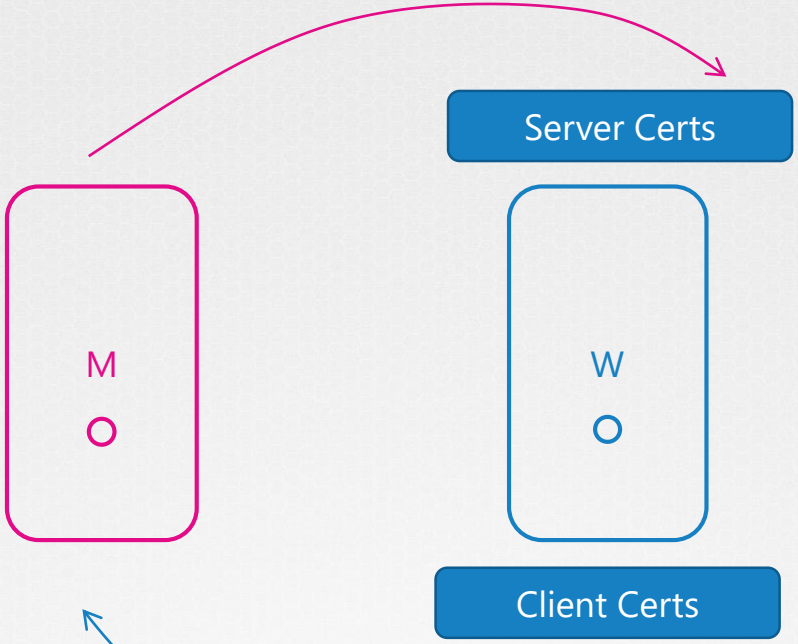
- ✓ Deploy ETCD Cluster
- ✓ Deploy Control Plane Components
- ✓ Network Loadbalancer



- ✓ Generate CERTs for Worker-1
- ✓ Configure Kubelet for Worker-1
- ✓ Renew Certificates
- ✓ Configure kube-proxy

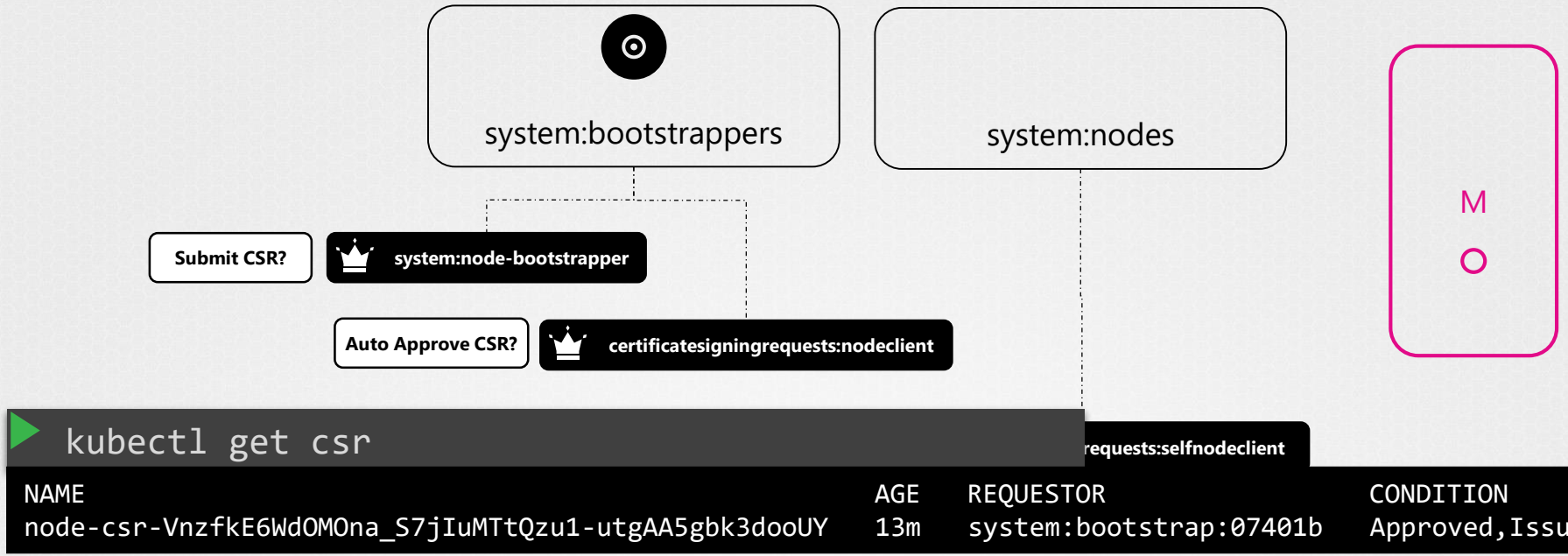
TLS Bootstrap:

- ❑ Worker-2 to create and configure certificates by itself
- ❑ Configure Kubelet for Worker-2
- ❑ Worker-2 to renew certificates by itself
- ❑ Configure kube-proxy



kubelet.service

```
ExecStart=/usr/local/bin/kubelet \\
  --config=/var/lib/kubelet/kubelet-config.yaml \\
  --image-pull-progress-deadline=2m \\
  --kubeconfig=/var/lib/kubelet/kubeconfig \\
  --tls-cert-file=/var/lib/kubelet/worker-1.crt \\
  --tls-private-key-file=/var/lib/kubelet/worker-1.key \\
  --network-plugin=cni \\
  --register-node=true \\
  --v=2
```




```
kubectl get csr
NAME                                                   AGE   REQUESTOR                 CONDITION
node-csr-VnzfkE6WdOMOna_S7jIuMTtQzu1-utgAA5gbk3dooUY   13m   system:bootstrap:07401b   Approved,Issued
```

1. Create Bootstrap Token and associate it to group **system:bootstrappers**
2. Assign Role – **system:node-bootstrapper** to group **system:bootstrappers**
3. Assign Role - **system:certificates.k8s.io:certificatesigningrequests:nodeclient** to group **system:bootstrappers**
4. Assign Role - **system:certificates.k8s.io:certificatesigningrequests:selfnodeclient** to group **system:nodes**

KODEKLOUD

bootstrap-kubeconfig

```
apiVersion: v1
clusters:
- cluster:
    certificate-authority: /var/lib/kubernetes/ca.crt
    server: https://192.168.5.30:6443
  name: bootstrap
contexts:
- context:
    cluster: bootstrap
    user: kubelet-bootstrap
  name: bootstrap
current-context: bootstrap
kind: Config
preferences: {}
users:
- name: kubelet-bootstrap
  user:
    token: 07401b.f395accd246ae52d
```

kubelet.service

```
ExecStart=/usr/local/bin/kubelet \\
  --config=/var/lib/kubelet/kubelet-config.yaml \\
  --image-pull-progress-deadline=2m \\
  --bootstrap-kubeconfig="/var/lib/kubelet/bootstrap-kubeconfig" \\
  --tls-cert-file=/var/lib/kubelet/worker-1.crt \\
  --tls-private-key-file=/var/lib/kubelet/worker-1.key \\
  --network-plugin=cni \\
  --register-node=true \\
  --v=2
```

bootstrap-kubeconfig

```
apiVersion: v1
clusters:
- cluster:
    certificate-authority: /var/lib/kubernetes/ca.crt
    server: https://192.168.5.30:6443
  name: bootstrap
contexts:
- context:
    cluster: bootstrap
    user: kubelet-bootstrap
  name: bootstrap
current-context: bootstrap
kind: Config
preferences: {}
users:
- name: kubelet-bootstrap
  user:
    token: 07401b.f395accd246ae52d
```

kubelet.service

```
ExecStart=/usr/local/bin/kubelet \\
  --config=/var/lib/kubelet/kubelet-config.yaml \\
  --image-pull-progress-deadline=2m \\
  --bootstrap-kubeconfig="/var/lib/kubelet/bootstrap-kubeconfig" \\
  --rotate-certificates=true \\
  --tls-cert-file=/var/lib/kubelet/worker-1.crt \\
  --tls-private-key-file=/var/lib/kubelet/worker-1.key \\
  --network-plugin=cni \\
  --register-node=true \\
  --v=2
```

bootstrap-kubeconfig

```
apiVersion: v1
clusters:
- cluster:
    certificate-authority: /var/lib/kubernetes/ca.crt
    server: https://192.168.5.30:6443
  name: bootstrap
contexts:
- context:
    cluster: bootstrap
    user: kubelet-bootstrap
  name: bootstrap
current-context: bootstrap
kind: Config
preferences: {}
users:
- name: kubelet-bootstrap
  user:
    token: 07401b.f395accd246ae52d
```

Client Certs

kubelet.service

```
ExecStart=/usr/local/bin/kubelet \\
  --config=/var/lib/kubelet/kubelet-config.yaml \\
  --image-pull-progress-deadline=2m \\
  --bootstrap-kubeconfig="/var/lib/kubelet/bootstrap-kubeconfig" \\
  --rotate-certificates=true \\
  --rotate-server-certificates=true \\
  --network-plugin=cni \\
  --register-node=true \\
  --v=2
```

Server Certs

CSR Approval
Manual

Client Certs

CSR Approval
Automatic

```
kubectl get csr
```

```
NAME                                                   AGE   REQUESTOR                 CONDITION
csr-x254z                                              13m   system:node:worker-2      Pending
node-csr-VnzfkE6WdOMOna_S7jIuMTtQzu1-utgAA5gbk3dooUY   13m   system:bootstrap:07401b   Approved,Issued
```

```
kubectl certificate approve csr-x254z
```

```
csr-x254z approved!
```

KODEKLOUD

DEMO

TLS Bootstrap Kubelet

KODEKLOUD

DEMO

Configure KubeConfig File

KODEKLOUD

DEMO

Provision Networking

KODEKLOUD

DEMO

KubeApi Server to Kubelet Connectivity

KODEKLOUD

DEMO

Deploy DNS - CoreDNS

KODEKLOUD

Course Objectives

- Core Concepts
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Networking
- Installation, Configuration & Validation
  - Design a Kubernetes Cluster
  - Choose Kubernetes Infrastructure Config
  - Choose a Network Solution
  - HA Kubernetes Cluster
  - Provision Infrastructure
  - Secure Cluster Communicatic
  - Kubernetes Release Binaries
  - Install Kubernetes Master Nodes
  - Install Kubernetes Worker Nodes
  - TLS Bootstrapping a Node
  - Node end-to-end tests
  - Run & Analyze end-to-end tes
- Troubleshooting

KODEKLOUD

Node end-to-end Tests

Test - Manual

```
kubectl get nodes
```

```
NAME       STATUS   ROLES    AGE   VERSION
worker-1   Ready    <none>   8d    v1.13.0
worker-2   Ready    <none>   8d    v1.13.0
```

```
kubectl get pods
```

```
NAMESPACE                   NAME                       READY   STATUS    RESTARTS   AGE
kube-system                 coredns-69cbb76ff8-9p45d   1/1     Running   1          8d
kube-system                 coredns-69cbb76ff8-rmhzt   1/1     Running   0          8d
kube-system                 weave-net-58j2j            2/2     Running   2          8d
kube-system                 weave-net-rr5dk            2/2     Running   2          8d
```

KODEKLOUD

Test - Manual

```
kubectl get pods -n kube-system
```

```
NAME                              READY     STATUS    RESTARTS   AGE
coredns-78fcdf6894-5dntv          1/1       Running   0          1h
coredns-78fcdf6894-knpzl          1/1       Running   0          1h
etcd-master                       1/1       Running   0          1h
kube-apiserver-master             1/1       Running   0          1h
kube-controller-manager-master    1/1       Running   0          1h
kube-proxy-fvbpj                  1/1       Running   0          1h
kube-proxy-v5r2t                  1/1       Running   0          1h
kube-scheduler-master             1/1       Running   0          1h
weave-net-7kd52                   2/2       Running   1          1h
weave-net-jtl5m                   2/2       Running   1          1h
```

KODEKLOUD

Test - Manual

```
service kube-apiserver status
```

```
● kube-apiserver.service - Kubernetes API Server
   Loaded: loaded (/etc/systemd/system/kube-apiserver.service; enabled; vendor preset: enabled)
   Active: active (running) since Wed 2019-03-20 07:57:25 UTC; 1 weeks 1 days ago
     Docs: https://github.com/kubernetes/kubernetes
 Main PID: 15767 (kube-apiserver)
    Tasks: 13 (limit: 2362)
```

```
service kube-controller-manager status
```

```
● kube-controller-manager.service - Kubernetes Controller Manager
   Loaded: loaded (/etc/systemd/system/kube-controller-manager.service; enabled; vendor preset: enabled)
   Active: active (running) since Wed 2019-03-20 07:57:25 UTC; 1 weeks 1 days ago
     Docs: https://github.com/kubernetes/kubernetes
 Main PID: 15771 (kube-controller)
    Tasks: 10 (limit: 2362)
```

```
service kube-scheduler status
```

```
● kube-scheduler.service - Kubernetes Scheduler
   Loaded: loaded (/etc/systemd/system/kube-scheduler.service; enabled; vendor preset: enabled)
   Active: active (running) since Fri 2019-03-29 01:45:32 UTC; 11min ago
     Docs: https://github.com/kubernetes/kubernetes
 Main PID: 28390 (kube-scheduler)
    Tasks: 10 (limit: 2362)
```

KODEKLOUD

Test - Manual

```
service kubelet status
```

```
● kubelet.service - Kubernetes Kubelet
   Loaded: loaded (/etc/systemd/system/kubelet.service; enabled; vendor preset: enabled)
   Active: active (running) since Wed 2019-03-20 14:22:06 UTC; 1 weeks 1 days ago
     Docs: https://github.com/kubernetes/kubernetes
 Main PID: 1281 (kubelet)
    Tasks: 24 (limit: 1152)
```

```
service kube-proxy status
```

```
● kube-proxy.service - Kubernetes Kube Proxy
   Loaded: loaded (/etc/systemd/system/kube-proxy.service; enabled; vendor preset: enabled)
   Active: active (running) since Wed 2019-03-20 14:21:54 UTC; 1 weeks 1 days ago
     Docs: https://github.com/kubernetes/kubernetes
 Main PID: 794 (kube-proxy)
    Tasks: 7 (limit: 1152)
```

KODEKLOUD

Test - Manual

```
kubectl run nginx
```

```
deployment.apps/nginx created
```

```
kubectl get pods
```

```
NAME                     READY   STATUS    RESTARTS   AGE
nginx-7cdbd8cdc9-g5q8d   1/1     Running   0          19s
```

```
kubectl scale --replicas=3 deploy/nginx
```

```
deployment.extensions/nginx scaled
```

```
kubectl get pods
```

```
NAME                     READY   STATUS    RESTARTS   AGE     IP          NODE       NOMINATED NODE   READINESS GATES
nginx-7cdbd8cdc9-djj6x   1/1     Running   0          74s     10.40.0.5   worker-2   <none>           <none>
nginx-7cdbd8cdc9-g5q8d   1/1     Running   0          3m29s   10.32.0.5   worker-1   <none>           <none>
nginx-7cdbd8cdc9-rsskt   1/1     Running   0          74s     10.32.0.6   worker-1   <none>           <none>
```

KODEKLOUD

Test - Manual

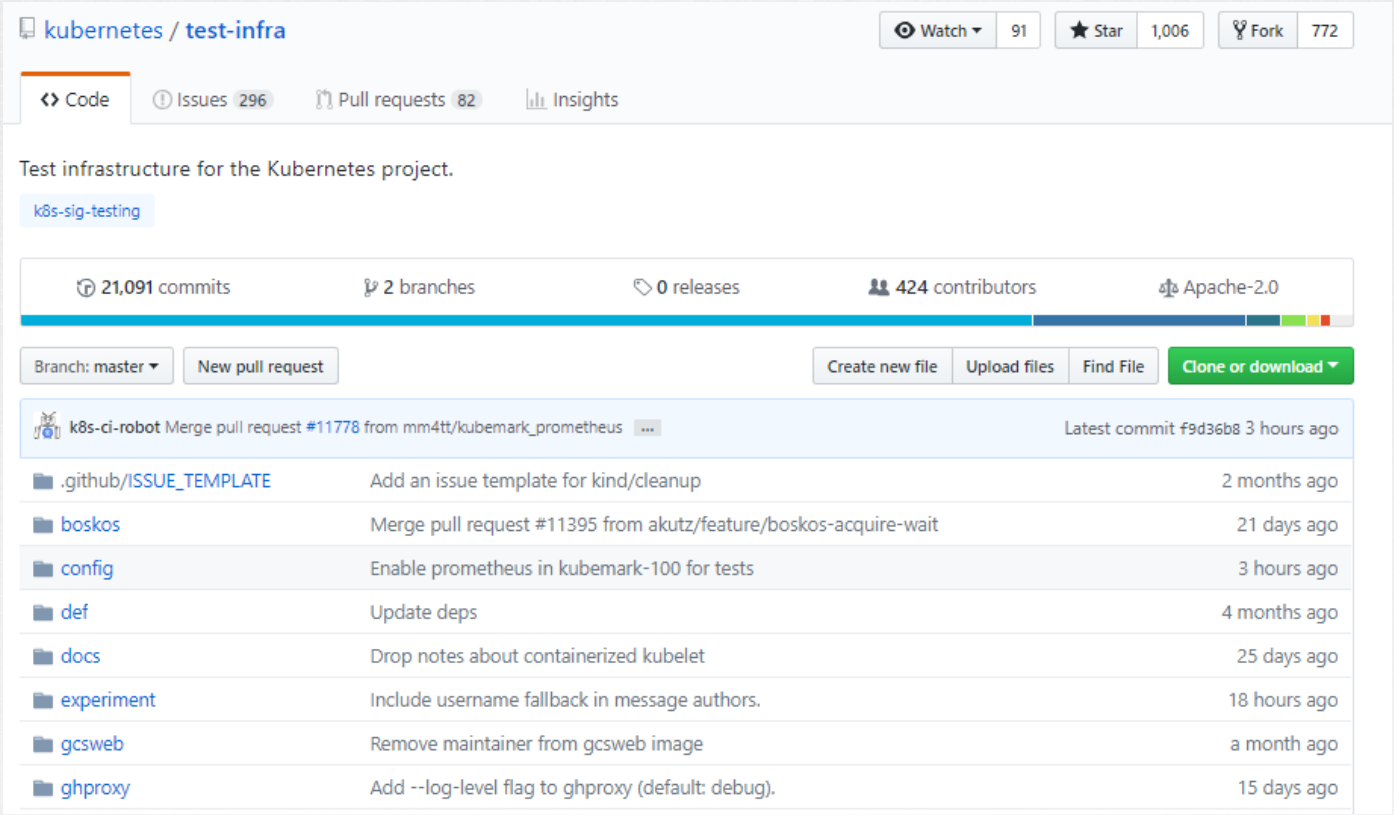
```
kubectl expose deployment nginx --port=80 --type=NodePort
service/nginx exposed
```

```
kubectl get service
NAME         TYPE        CLUSTER-IP    EXTERNAL-IP   PORT(S)        AGE
kubernetes   ClusterIP   10.96.0.1     <none>        443/TCP        8d
nginx        NodePort    10.96.0.88    <none>        80:31850/TCP   3s
```

```
curl http://worker-1:31850
...
<h1>Welcome to nginx!</h1>
...
```

KODEKLOUD

kubetest



KODEKLOUD

kubetest - Tests

e2e: ~1000

sig-api-machinery

sig-apps

sig-auth

sig-cli

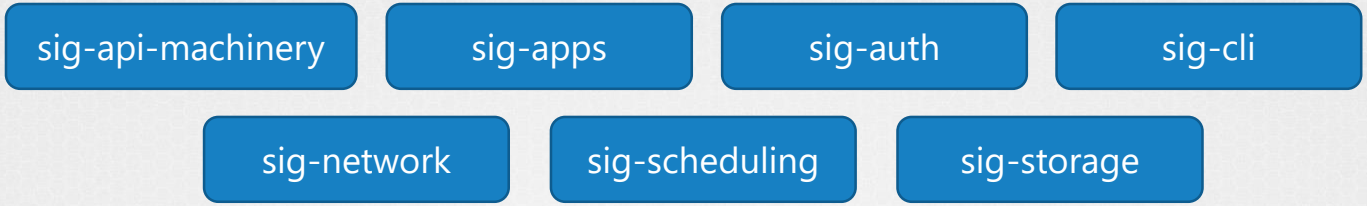
sig-network

sig-scheduling

sig-storage

KODEKLOUD

kubetest - Tests



- ✓ Networking should function for intra-pod communication (http)
- ✓ Services should serve a basic endpoint from pods
- ✓ Service endpoints latency should not be very high
- ✓ DNS should provide DNS for services

- ✓ Secrets should be consumable in multiple volumes in a pod
- ✓ Secrets should be consumable from pods in volume with mappings
- ✓ ConfigMap should be consumable from pods in volume

KODEKLOUD

kubetest - Tests

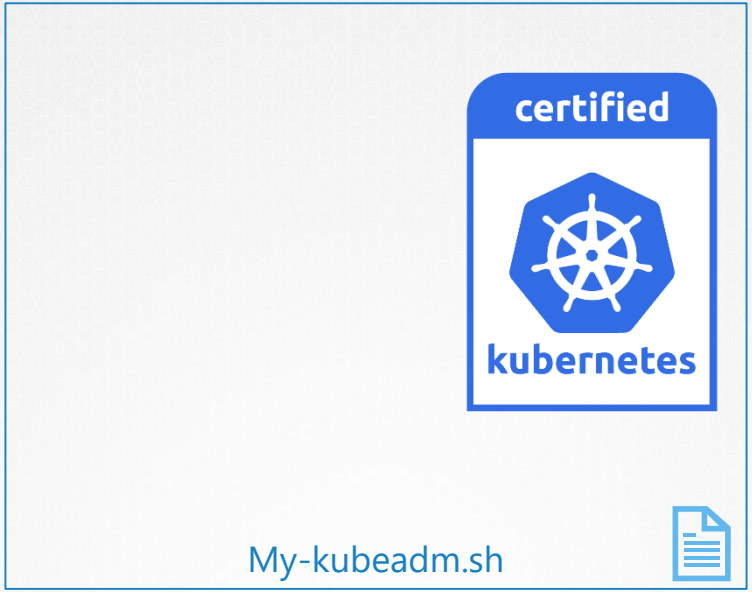
✓ Networking should function for intra-pod communication (http)

1. **Prepare:** Creates a namespace for this test
2. Creates Test Pod in this namespace
   Waits for the PODs to come up
3. **Test:** Executes curl on one pod to reach the IP of another over HTTP
4. Record the result
5. **Cleanup:** Delete the namespace

```
[sig-network] Networking Granular Checks: Pods
  should function for intra-pod communication: http [NodeConformance] [Confo
STEP: Building a namespace api object
STEP: Performing setup for networking test in namespace e2e-tests-pod-networ
Mar 14 11:35:19.315: INFO: Waiting up to 10m0s for all (but 0) nodes to be s
STEP: Creating test pods
Mar 14 11:35:39.522: INFO: ExecWithOptions {Command:[/bin/sh -c curl -g -q -
'http://10.32.0.8:8080/dial?request=hostName&protocol=http&host=10.32.0.7&po
drstd PodName:host-test-container-pod ContainerName:hostexec Stdin:<nil> Cap
PreserveWhitespace:false}
Mar 14 11:35:39.522: INFO: >>> kubeConfig: /root/.kube/config
Mar 14 11:35:39.656: INFO: Waiting for endpoints: map[]
Mar 14 11:35:39.660: INFO: ExecWithOptions {Command:[/bin/sh -c curl -g -q -
'http://10.32.0.8:8080/dial?request=hostName&protocol=http&host=10.38.0.12&p
drstd PodName:host-test-container-pod ContainerName:hostexec Stdin:<nil> Cap
PreserveWhitespace:false}
Mar 14 11:35:39.660: INFO: >>> kubeConfig: /root/.kube/config
STEP: Destroying namespace "e2e-tests-pod-network-test-drstd" for this suite
```

KODEKLOUD

Kubernetes Test-Infra



KODEKLOUD

Kubetest - Tests

e2e: ~1000

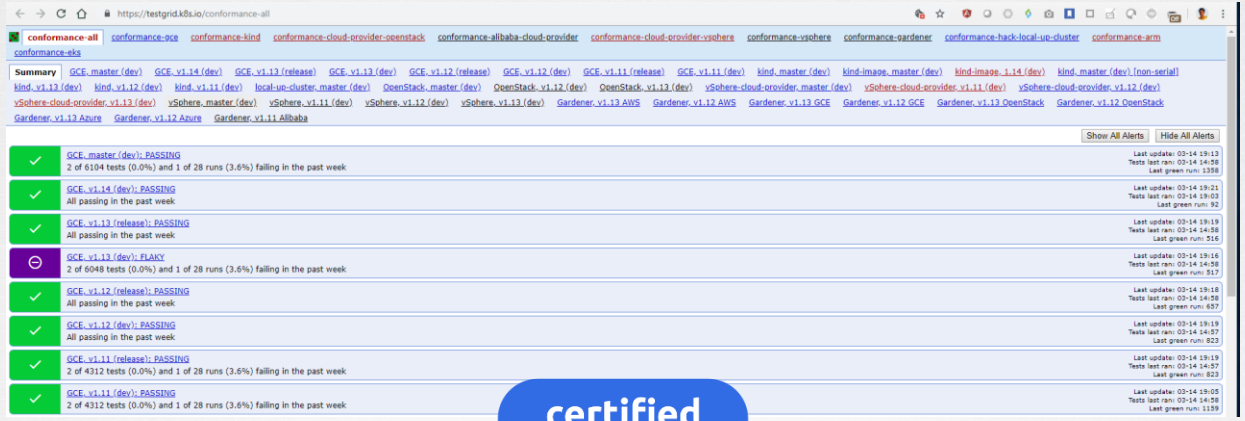
conformance: ~160

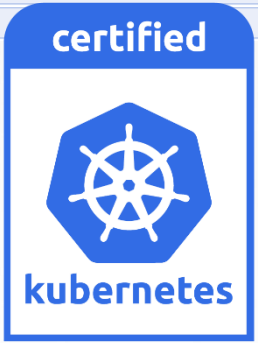
KODEKLOUD

Kubetest - Tests

e2e: ~1000

conformance: ~160





KODEKLOUD

Kubetest - Time

Full e2e = ~1000 Tests / 12 Hours

Conformance = 164 Tests / 1.5 hours

KODEKLOUD

SONOBUOY

KODEKLOUD

KODEKLOUD

Course Objectives

- Core Concepts
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Networking
- Installation, Configuration & Validation
  - Design a Kubernetes Cluster
  - Choose Kubernetes Infrastructure Config
  - Choose a Network Solution
  - HA Kubernetes Cluster
  - Provision Infrastructure
  - Secure Cluster Communication
  - Kubernetes Release Binaries
  - Install Kubernetes Master Nodes
  - Install Kubernetes Worker Nodes
  - TLS Bootstrapping a Node
  - Node end-to-end tests
  - Run & Analyze end-to-end tes
- Troubleshooting

KODEKLOUD

Run & Analyze E2E Tests

kubetest- Run

```
go get -u k8s.io/test-infra/kubetest
```

```
kubetest --extract=v1.11.3
kubernetes
```

```
cd kubernetes
```

```
export KUBE_MASTER_IP="192.168.26.10:6443"
```

```
export KUBE_MASTER=kube-master
```

```
kubetest --test --provider=skeleton > testout.txt
```

```
kubetest --test --provider=skeleton --test_args="--ginkgo.focus=Secrets" > testout.txt
```

```
kubetest --test --provider=skeleton --test_args="--ginkgo.focus=\[Conformance\]" > testout.txt
```

Note: Version must match the kubernetes server version

KODEKLOUD

kubetest- Run

```
kubetest --test --provider=skeleton --test_args="--ginkgo.focus=\[Conformance\]" > testout.txt
```

```
cat testout.txt
```

```
Client Version: version.Info{Major:"1", Minor:"11", GitVersion:"v1.11.8",
GitCommit:"4e209c9383fa00631d124c8adcc011d617339b3c", GitTreeState:"clean", BuildDate:"2019-02-28T18:49:34Z",
GoVersion:"go1.10.8", C
ompiler:"gc", Platform:"linux/amd64"}
Server Version: version.Info{Major:"1", Minor:"11", GitVersion:"v1.11.8",
GitCommit:"4e209c9383fa00631d124c8adcc011d617339b3c", GitTreeState:"clean", BuildDate:"2019-02-28T18:40:05Z",
GoVersion:"go1.10.8", C
ompiler:"gc", Platform:"linux/amd64"}
Setting up for KUBERNETES_PROVIDER="skeleton".
Mar 14 11:16:12.419: INFO: Overriding default scale value of zero to 1
Mar 14 11:16:12.419: INFO: Overriding default milliseconds value of zero to 5000
I0314 11:16:12.674596   20093 e2e.go:333] Starting e2e run "933b1eae-464a-11e9-81ea-02f0aa2d49f4" on Ginkgo node 1
Running Suite: Kubernetes e2e suite
===================================
Random Seed: 1552562172 - Will randomize all specs
Will run 167 of 1008 specs

Mar 14 11:16:12.731: INFO: >>> kubeConfig: /root/.kube/config
Mar 14 11:16:12.745: INFO: Waiting up to 30m0s for all (but 0) nodes to be schedulable
Mar 14 11:16:12.770: INFO: Waiting up to 10m0s for all pods (need at least 0) in namespace 'kube-system' to be running
and ready
Mar 14 11:16:12.831: INFO: 12 / 13 pods in namespace 'kube-system' are running and ready (0 seconds elapsed)
Mar 14 11:16:12.831: INFO: expected 2 pod replicas in namespace 'kube-system', 2 are Running and Ready
```

```
• [SLOW TEST:8.486 seconds]
[sig-storage] EmptyDir volumes
/workspace/anago-v1.11.8-
beta.0.41+4e209c9383fa00/src/k8s.io/kubernetes/_output/dockerized/go/src/k8s.io/kubernetes/test/e2e/common/empty_dir.g
o:40
  should support (root,0777,tmpfs) [NodeConformance] [Conformance]
  /workspace/anago-v1.11.8-
beta.0.41+4e209c9383fa00/src/k8s.io/kubernetes/_output/dockerized/go/src/k8s.io/kubernetes/test/e2e/framework/framewor
k.go:684
------------------------------
SSMar 14 13:01:15.397: INFO: Running AfterSuite actions on all node
Mar 14 13:01:15.397: INFO: Running AfterSuite actions on node 1


Summarizing 2 Failures:

[Fail] [sig-network] DNS [It] should provide DNS for services  [Conformance]
/workspace/anago-v1.11.8-
beta.0.41+4e209c9383fa00/src/k8s.io/kubernetes/_output/dockerized/go/src/k8s.io/kubernetes/test/e2e/network/dns_common
.go:497

[Fail] [sig-network] DNS [It] should provide DNS for the cluster  [Conformance]
/workspace/anago-v1.11.8-
beta.0.41+4e209c9383fa00/src/k8s.io/kubernetes/_output/dockerized/go/src/k8s.io/kubernetes/test/e2e/network/dns_common
.go:497

Ran 166 of 1008 Specs in 6302.670 seconds
FAIL! -- 164 Passed | 2 Failed | 0 Pending | 842 Skipped --- FAIL: TestE2E (6302.72s)
FAIL

Ginkgo ran 1 suite in 1h45m3.31433997s
Test Suite Failed
```

KODEKLOUD

DEMO

Run Smoke Test

KODEKLOUD

DEMO

Run End-to-End Tests

KODEKLOUD

Course Objectives

- Core Concepts
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Networking
- Installation, Configuration & Validation
- Troubleshooting
  - Application Failure
  - Control Plane Failure
  - Worker Node Failure
  - Networking

KODEKLOUD

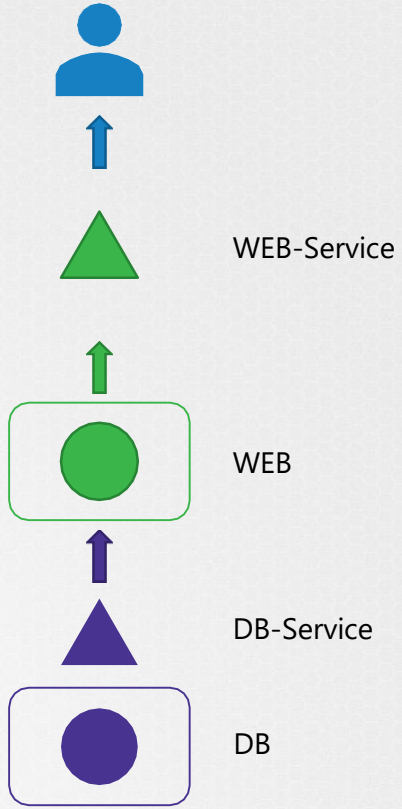
Application Failure

Check Service Status

```
curl http://web-service-ip:node-port
curl: (7) Failed to connect to web-service-ip port node-port: Connection timed out
```

```
kubectl describe service web-service
Name:                     web-service
Namespace:                default
Labels:                   <none>
Annotations:              <none>
Selector:                 name=webapp-mysql
Type:                     NodePort
IP:                       10.96.0.156
Port:                     <unset>  8080/TCP
TargetPort:               8080/TCP
NodePort:                 <unset>  31672/TCP
Endpoints:                10.32.0.6:8080
Session Affinity:         None
External Traffic Policy:  Cluster
Events:                   <none>
```

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: webapp-mysql
  labels:
    app: example-app
    name: webapp-mysql
spec:
  containers:
  - name: webapp-mysql
    image: simple-webapp-mysql
    ports:
    - containerPort: 8080
```



KODEKLOUD

Check Service

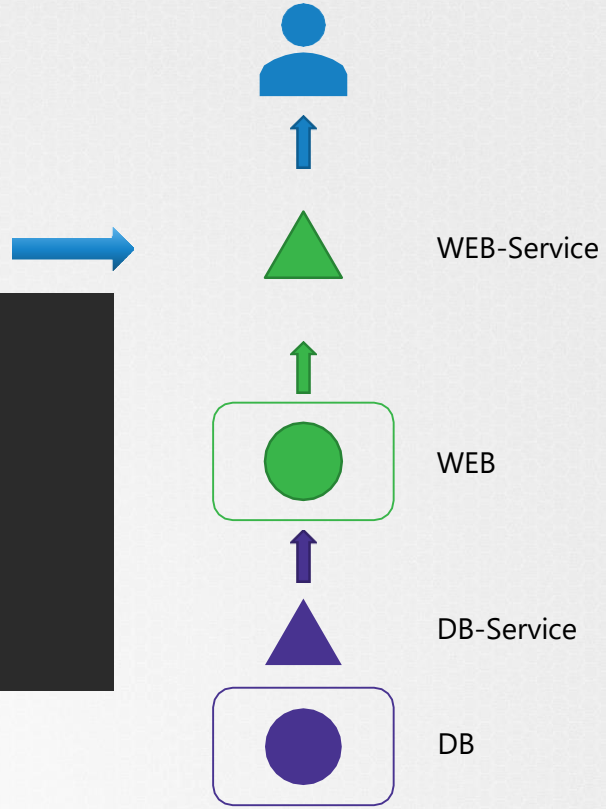
```
curl http://web-service-ip:node-port
```

```
curl: (7) Failed to connect to web-service-ip port node-port: Connection timed out
```

```
kubectl describe service web-service
```

```
Name:                     web-service
Namespace:                default
Labels:                   <none>
Annotations:              <none>
Selector:                 name=webapp-mysql
Type:                     NodePort
IP:                       10.96.0.156
Port:                     <unset>  8080/TCP
TargetPort:               8080/TCP
NodePort:                 <unset>  31672/TCP
Endpoints:                10.32.0.6:8080
Session Affinity:         None
External Traffic Policy:  Cluster
Events:                   <none>
```

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: webapp-mysql
  labels:
    app: example-app
    name: webapp-mysql
spec:
  containers:
  - name: webapp-mysql
    image: simple-webapp-mysql
    ports:
    - containerPort: 8080
```



KODEKLOUD

Check POD

```
kubectl get pod
```

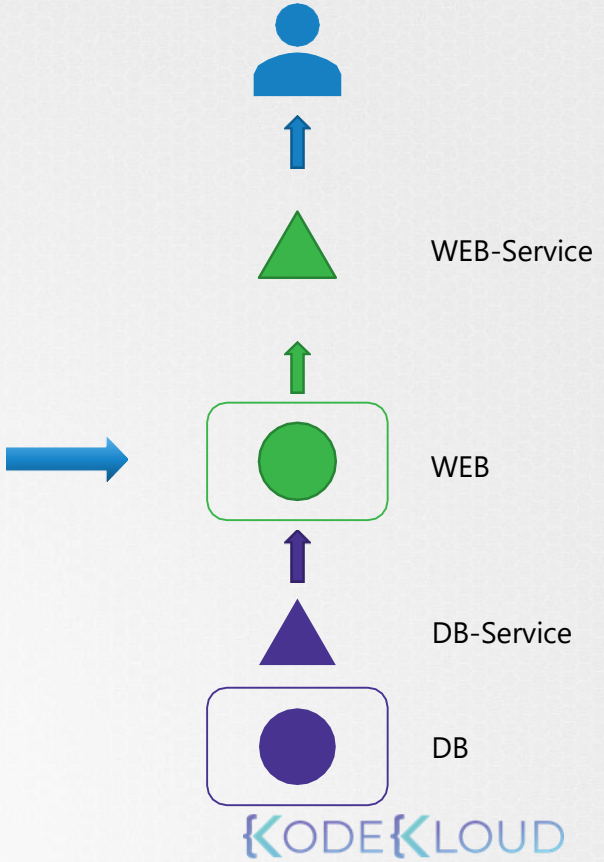
```
NAME            READY   STATUS    RESTARTS   AGE
Web             1/1     Running   5          50m
```

```
kubectl describe pod web
```

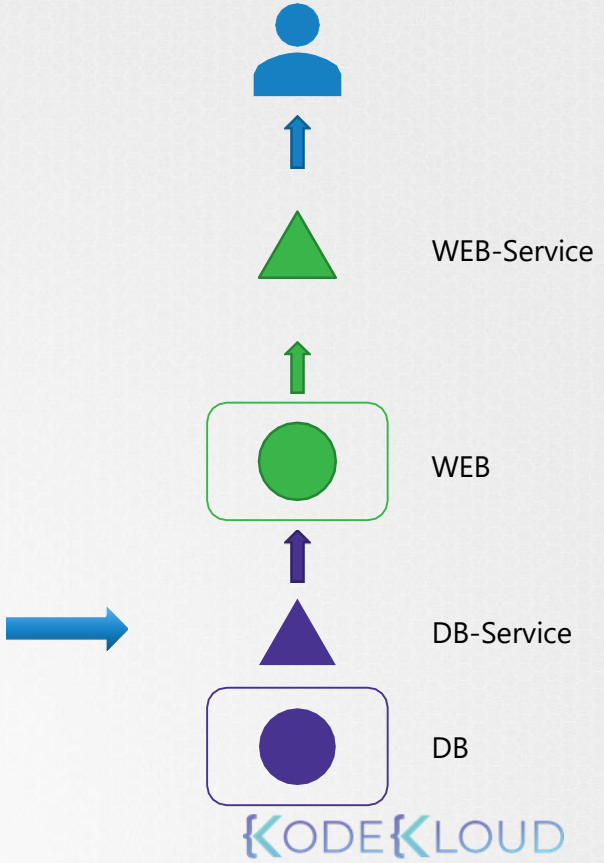
```
...

Events:
  Type    Reason     Age   From               Message
  ----    ------     ----  ----               -------
  Normal  Scheduled  52m   default-scheduler  Successfully assigned webapp-mysql to worker-1
  Normal  Pulling    52m   kubelet, worker-1  pulling image "simple-webapp-mysql"
  Normal  Pulled     52m   kubelet, worker-1  Successfully pulled image "simple-webapp-mysql"
  Normal  Created    52m   kubelet, worker-1  Created container
  Normal  Started    52m   kubelet, worker-1  Started container
```

```
kubectl logs web -f --previous
```

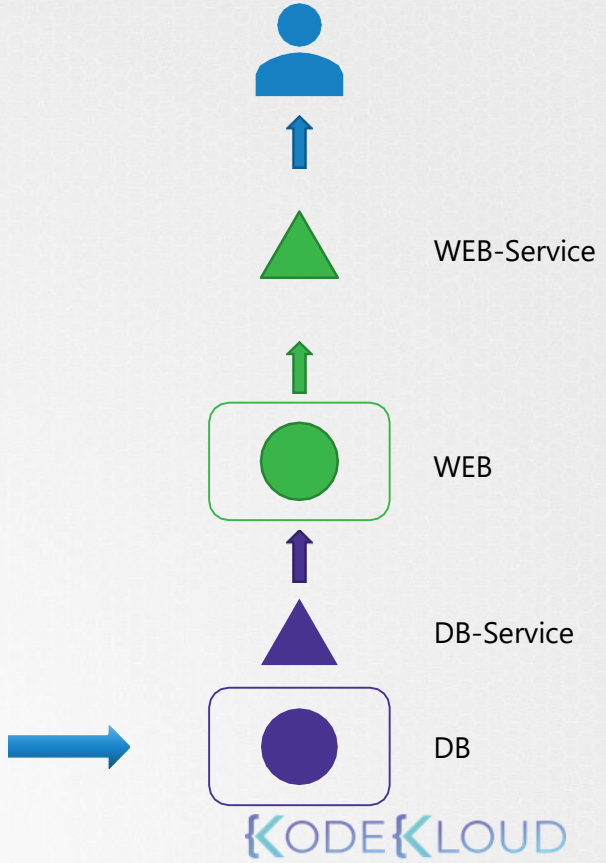
```
10.32.0.1 - - [01/Apr/2019 12:51:55] "GET / HTTP/1.1" 200 -
10.32.0.1 - - [01/Apr/2019 12:51:55] "GET /static/img/success.jpg HTTP/1.1" 200 -
10.32.0.1 - - [01/Apr/2019 12:51:55] "GET /favicon.ico HTTP/1.1" 404 -
10.32.0.1 - - [01/Apr/2019 12:51:57] "GET / HTTP/1.1" 200 -
10.32.0.1 - - [01/Apr/2019 12:51:57] "GET / HTTP/1.1" 200 -
10.32.0.1 - - [01/Apr/2019 12:51:58] "GET / HTTP/1.1" 200 -
10.32.0.1 - - [01/Apr/2019 12:51:58] "GET / HTTP/1.1" 200 -
10.32.0.1 - - [01/Apr/2019 12:51:58] "GET / HTTP/1.1" 400 - Some Database Error application exiting!
```



Check Dependent Service



Check Dependent Applications



KODEKLOUD

Course Objectives

- Core Concepts
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Networking
- Installation, Configuration & Validation
- Troubleshooting
  - Application Failure
  - Control Plane Failure
  - Worker Node Failure
  - Networking

KODEKLOUD

Control Plane Failure

Check Node Status

```
kubectl get nodes
NAME       STATUS   ROLES    AGE   VERSION
worker-1   Ready    <none>   8d    v1.13.0
worker-2   Ready    <none>   8d    v1.13.0
```

```
kubectl get pods
NAME           READY   STATUS    RESTARTS   AGE
mysql          1/1     Running   0          113m
webapp-mysql   1/1     Running   0          113m
```

KODEKLOUD

Check Controlplane Pods

```
kubectl get pods -n kube-system
```

NAME	READY	STATUS	RESTARTS	AGE
coredns-78fcdf6894-5dntv	1/1	Running	0	1h
coredns-78fcdf6894-knpzl	1/1	Running	0	1h
etcd-master	1/1	Running	0	1h
kube-apiserver-master	1/1	Running	0	1h
kube-controller-manager-master	1/1	Running	0	1h
kube-proxy-fvbpj	1/1	Running	0	1h
kube-proxy-v5r2t	1/1	Running	0	1h
kube-scheduler-master	1/1	Running	0	1h
weave-net-7kd52	2/2	Running	1	1h
weave-net-jtl5m	2/2	Running	1	1h

KODEKLOUD

Check Controlplane Services

```
service kube-apiserver status
```

```
● kube-apiserver.service - Kubernetes API Server
   Loaded: loaded (/etc/systemd/system/kube-apiserver.service; enabled; vendor preset: enabled)
   Active: active (running) since Wed 2019-03-20 07:57:25 UTC; 1 weeks 1 days ago
     Docs: https://github.com/kubernetes/kubernetes
 Main PID: 15767 (kube-apiserver)
    Tasks: 13 (limit: 2362)
```

```
service kube-controller-manager status
```

```
● kube-controller-manager.service - Kubernetes Controller Manager
   Loaded: loaded (/etc/systemd/system/kube-controller-manager.service; enabled; vendor preset: enabled)
   Active: active (running) since Wed 2019-03-20 07:57:25 UTC; 1 weeks 1 days ago
     Docs: https://github.com/kubernetes/kubernetes
 Main PID: 15771 (kube-controller)
    Tasks: 10 (limit: 2362)
```

```
service kube-scheduler status
```

```
● kube-scheduler.service - Kubernetes Scheduler
   Loaded: loaded (/etc/systemd/system/kube-scheduler.service; enabled; vendor preset: enabled)
   Active: active (running) since Fri 2019-03-29 01:45:32 UTC; 11min ago
     Docs: https://github.com/kubernetes/kubernetes
 Main PID: 28390 (kube-scheduler)
    Tasks: 10 (limit: 2362)
```

KODEKLOUD

Check Controlplane Services

```
service kubelet status
```

```
● kubelet.service - Kubernetes Kubelet
   Loaded: loaded (/etc/systemd/system/kubelet.service; enabled; vendor preset: enabled)
   Active: active (running) since Wed 2019-03-20 14:22:06 UTC; 1 weeks 1 days ago
     Docs: https://github.com/kubernetes/kubernetes
 Main PID: 1281 (kubelet)
    Tasks: 24 (limit: 1152)
```

```
service kube-proxy status
```

```
● kube-proxy.service - Kubernetes Kube Proxy
   Loaded: loaded (/etc/systemd/system/kube-proxy.service; enabled; vendor preset: enabled)
   Active: active (running) since Wed 2019-03-20 14:21:54 UTC; 1 weeks 1 days ago
     Docs: https://github.com/kubernetes/kubernetes
 Main PID: 794 (kube-proxy)
    Tasks: 7 (limit: 1152)
```

KODEKLOUD

Check Service Logs

```
kubectl logs kube-apiserver-master -n kube-system
```

```
I0401 13:45:38.190735       1 server.go:703] external host was not specified, using 172.17.0.117
I0401 13:45:38.194290       1 server.go:145] Version: v1.11.3
I0401 13:45:38.819705       1 plugins.go:158] Loaded 8 mutating admission controller(s) successfully in the following order:
NamespaceLifecycle,LimitRanger,ServiceAccount,NodeRestriction,Priority,DefaultTolerationSeconds,DefaultStorageClass,MutatingAdmissionWebhook.
I0401 13:45:38.819741       1 plugins.go:161] Loaded 6 validating admission controller(s) successfully in the following order:
LimitRanger,ServiceAccount,Priority,PersistentVolumeClaimResize,ValidatingAdmissionWebhook,ResourceQuota.
I0401 13:45:38.821372       1 plugins.go:158] Loaded 8 mutating admission controller(s) successfully in the following order:
NamespaceLifecycle,LimitRanger,ServiceAccount,NodeRestriction,Priority,DefaultTolerationSeconds,DefaultStorageClass,MutatingAdmissionWebhook.
I0401 13:45:38.821410       1 plugins.go:161] Loaded 6 validating admission controller(s) successfully in the following order:
LimitRanger,ServiceAccount,Priority,PersistentVolumeClaimResize,ValidatingAdmissionWebhook,ResourceQuota.
I0401 13:45:38.985453       1 master.go:234] Using reconciler: lease
W0401 13:45:40.900380       1 genericapiserver.go:319] Skipping API batch/v2alpha1 because it has no resources.
W0401 13:45:41.370677       1 genericapiserver.go:319] Skipping API rbac.authorization.k8s.io/v1alpha1 because it has no resources.
W0401 13:45:41.381736       1 genericapiserver.go:319] Skipping API scheduling.k8s.io/v1alpha1 because it has no resources.
```

```
sudo journalctl -u kube-apiserver
```

```
Mar 20 07:57:25 master-1 systemd[1]: Started Kubernetes API Server.
Mar 20 07:57:25 master-1 kube-apiserver[15767]: I0320 07:57:25.553377   15767 flags.go:33] FLAG: --address="127.0.0.1"
Mar 20 07:57:25 master-1 kube-apiserver[15767]: I0320 07:57:25.558273   15767 flags.go:33] FLAG: --admission-control="[]"
Mar 20 07:57:25 master-1 kube-apiserver[15767]: I0320 07:57:25.558325   15767 flags.go:33] FLAG: --admission-control-config-file=""
Mar 20 07:57:25 master-1 kube-apiserver[15767]: I0320 07:57:25.558339   15767 flags.go:33] FLAG: --advertise-address="192.168.5.11"
Mar 20 07:57:25 master-1 kube-apiserver[15767]: I0320 07:57:25.558353   15767 flags.go:33] FLAG: --allow-privileged="true"
Mar 20 07:57:25 master-1 kube-apiserver[15767]: I0320 07:57:25.558365   15767 flags.go:33] FLAG: --alsologtostderr="false"
Mar 20 07:57:25 master-1 kube-apiserver[15767]: I0320 07:57:25.558413   15767 flags.go:33] FLAG: --anonymous-auth="true"
Mar 20 07:57:25 master-1 kube-apiserver[15767]: I0320 07:57:25.558425   15767 flags.go:33] FLAG: --api-audiences="[]"
Mar 20 07:57:25 master-1 kube-apiserver[15767]: I0320 07:57:25.558442   15767 flags.go:33] FLAG: --apiserver-count="3"
Mar 20 07:57:25 master-1 kube-apiserver[15767]: I0320 07:57:25.558454   15767 flags.go:33] FLAG: --audit-dynamic-configuration="false"
Mar 20 07:57:25 master-1 kube-apiserver[15767]: I0320 07:57:25.558464   15767 flags.go:33] FLAG: --audit-log-batch-buffer-size="10000"
Mar 20 07:57:25 master-1 kube-apiserver[15767]: I0320 07:57:25.558474   15767 flags.go:33] FLAG: --audit-log-batch-max-size="1"
Mar 20 07:57:25 master-1 kube-apiserver[15767]: I0320 07:57:25.558484   15767 flags.go:33] FLAG: --audit-log-batch-max-wait="0s"
Mar 20 07:57:25 master-1 kube-apiserver[15767]: I0320 07:57:25.558495   15767 flags.go:33] FLAG: --audit-log-batch-throttle-burst="0"
Mar 20 07:57:25 master-1 kube-apiserver[15767]: I0320 07:57:25.558504   15767 flags.go:33] FLAG: --audit-log-batch-throttle-enable="false"
Mar 20 07:57:25 master-1 kube-apiserver[15767]: I0320 07:57:25.558514   15767 flags.go:33] FLAG: --audit-log-batch-throttle-qps="0"
Mar 20 07:57:25 master-1 kube-apiserver[15767]: I0320 07:57:25.558528   15767 flags.go:33] FLAG: --audit-log-format="json"
```

KODEKLOUD

Course Objectives

- Core Concepts
- Scheduling
- Logging Monitoring
- Application Lifecycle Management
- Cluster Maintenance
- Security
- Storage
- Networking
- Installation, Configuration & Validation
- Troubleshooting
  - Application Failure
  - Control Plane Failure
  - Worker Node Failure
  - Networking

KODEKLOUD

Worker Node Failure

Check Node Status

```
kubectl get nodes
```

```
NAME       STATUS    ROLES    AGE   VERSION
worker-1   Ready     <none>   8d    v1.13.0
worker-2   NotReady  <none>   8d    v1.13.0
```

```
kubectl describe node worker-1
```

```
...
Conditions:
  Type             Status  LastHeartbeatTime                 Reason                       Message
  ----             ------  -----------------                 ------                       -------
  OutOfDisk        False   Mon, 01 Apr 2019 14:30:33 +0000   KubeletHasSufficientDisk     kubelet has sufficient disk space available
  MemoryPressure   False   Mon, 01 Apr 2019 14:30:33 +0000   KubeletHasSufficientMemory   kubelet has sufficient memory available
  DiskPressure     False   Mon, 01 Apr 2019 14:30:33 +0000   KubeletHasNoDiskPressure     kubelet has no disk pressure
  PIDPressure      False   Mon, 01 Apr 2019 14:30:33 +0000   KubeletHasSufficientPID      kubelet has sufficient PID available
  Ready            True    Mon, 01 Apr 2019 14:30:33 +0000   KubeletReady                 kubelet is posting ready status. AppArmor enabled
```

```
kubectl describe node worker-1
```

```
...
Conditions:
  Type             Status    LastHeartbeatTime                 Reason                       Message
  ----             ------    -----------------                 ------                       -------
  OutOfDisk        Unknown   Mon, 01 Apr 2019 14:20:20 +0000   NodeStatusUnknown            Kubelet stopped posting node status.
  MemoryPressure   Unknown   Mon, 01 Apr 2019 14:20:20 +0000   NodeStatusUnknown            Kubelet stopped posting node status.
  DiskPressure     Unknown   Mon, 01 Apr 2019 14:20:20 +0000   NodeStatusUnknown            Kubelet stopped posting node status.
  PIDPressure      False     Mon, 01 Apr 2019 14:20:20 +0000   KubeletHasSufficientPID      kubelet has sufficient PID available
  Ready            Unknown   Mon, 01 Apr 2019 14:20:20 +0000   NodeStatusUnknown            Kubelet stopped posting node status.
```

Check Node

```
top
```

```
top - 14:43:56 up 3 days, 19:02,  1 user,  load average: 0.35, 0.29, 0.21
Tasks: 112 total,   1 running,  72 sleeping,   0 stopped,   0 zombie
%Cpu(s):  3.9 us,  1.7 sy,  0.1 ni, 94.3 id,  0.0 wa,  0.0 hi,  0.1 si,  0.0 st
KiB Mem :  1009112 total,    74144 free,   736608 used,   198360 buff/cache
KiB Swap:        0 total,        0 free,        0 used.   129244 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S %CPU %MEM     TIME+ COMMAND
   34 root      20   0       0      0      0 S  5.9  0.0   0:13.14 kswapd0
28826 999       20   0 1361320 383208   3596 S  5.9 38.0   0:46.95 mysqld
    1 root      20   0   78260   5924   3192 S  0.0  0.6   0:21.88 systemd
    2 root      20   0       0      0      0 S  0.0  0.0   0:00.02 kthreadd
    4 root       0 -20       0      0      0 I  0.0  0.0   0:00.00 kworker/0:0H
```

```
df -h
```

```
Filesystem      Size  Used Avail Use% Mounted on
udev            481M     0  481M   0% /dev
tmpfs            99M 1000K   98M   1% /run
/dev/sda1       9.7G  5.3G  4.5G  55% /
tmpfs           493M     0  493M   0% /dev/shm
tmpfs           5.0M     0  5.0M   0% /run/lock
tmpfs           493M     0  493M   0% /sys/fs/cgroup
tmpfs            99M     0   99M   0% /run/user/1000
```

KODEKLOUD

Check Kubelet Status

```
service kubelet status
```

```
● kubelet.service - Kubernetes Kubelet
   Loaded: loaded (/etc/systemd/system/kubelet.service; enabled; vendor preset: enabled)
   Active: active (running) since Wed 2019-03-20 14:22:06 UTC; 1 weeks 1 days ago
     Docs: https://github.com/kubernetes/kubernetes
 Main PID: 1281 (kubelet)
    Tasks: 24 (limit: 1152)
```

```
sudo journalctl –u kubelet
```

```
-- Logs begin at Wed 2019-03-20 05:30:37 UTC, end at Mon 2019-04-01 14:42:42 UTC. --
Mar 20 08:12:59 worker-1 systemd[1]: Started Kubernetes Kubelet.
Mar 20 08:12:59 worker-1 kubelet[18962]: Flag --tls-cert-file has been deprecated, This parameter should be set via the config file specified by
the Kubele
Mar 20 08:12:59 worker-1 kubelet[18962]: Flag --tls-private-key-file has been deprecated, This parameter should be set via the config file
specified by the
Mar 20 08:12:59 worker-1 kubelet[18962]: I0320 08:12:59.915179   18962 flags.go:33] FLAG: --address="0.0.0.0"
Mar 20 08:12:59 worker-1 kubelet[18962]: I0320 08:12:59.918149   18962 flags.go:33] FLAG: --allow-privileged="true"
Mar 20 08:12:59 worker-1 kubelet[18962]: I0320 08:12:59.918339   18962 flags.go:33] FLAG: --allowed-unsafe-sysctls="[]"
Mar 20 08:12:59 worker-1 kubelet[18962]: I0320 08:12:59.918502   18962 flags.go:33] FLAG: --alsologtostderr="false"
Mar 20 08:12:59 worker-1 kubelet[18962]: I0320 08:12:59.918648   18962 flags.go:33] FLAG: --anonymous-auth="true"
Mar 20 08:12:59 worker-1 kubelet[18962]: I0320 08:12:59.918841   18962 flags.go:33] FLAG: --application-metrics-count-limit="100"
Mar 20 08:12:59 worker-1 kubelet[18962]: I0320 08:12:59.918974   18962 flags.go:33] FLAG: --authentication-token-webhook="false"
Mar 20 08:12:59 worker-1 kubelet[18962]: I0320 08:12:59.919096   18962 flags.go:33] FLAG: --authentication-token-webhook-cache-ttl="2m0s"
Mar 20 08:12:59 worker-1 kubelet[18962]: I0320 08:12:59.919299   18962 flags.go:33] FLAG: --authorization-mode="AlwaysAllow"
Mar 20 08:12:59 worker-1 kubelet[18962]: I0320 08:12:59.919466   18962 flags.go:33] FLAG: --authorization-webhook-cache-authorized-ttl="5m0s"
Mar 20 08:12:59 worker-1 kubelet[18962]: I0320 08:12:59.919598   18962 flags.go:33] FLAG: --authorization-webhook-cache-unauthorized-ttl="30s"
Mar 20 08:12:59 worker-1 kubelet[18962]: I0320 08:12:59.919791   18962 flags.go:33] FLAG: --azure-container-registry-config=""
Mar 20 08:12:59 worker-1 kubelet[18962]: I0320 08:12:59.919971   18962 flags.go:33] FLAG: --boot-id-file="/proc/sys/kernel/random/boot_id"
Mar 20 08:12:59 worker-1 kubelet[18962]: I0320 08:12:59.920102   18962 flags.go:33] FLAG: --bootstrap-checkpoint-path=""
```

Check Certificates

```
openssl x509 -in /var/lib/kubelet/worker-1.crt  -text
```

```
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            ff:e0:23:9d:fc:78:03:35
    Signature Algorithm: sha256WithRSAEncryption
        Issuer: CN = KUBERNETES-CA
        Validity
            Not Before: Mar 20 08:09:29 2019 GMT
            Not After : Apr 19 08:09:29 2019 GMT
        Subject: CN = system:node:worker-1, O = system:nodes
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                Public-Key: (2048  bit)
                Modulus:
                    00:b4:28:0c:60:71:41:06:14:46:d9:97:58:2d:fe:
                    a9:c7:6d:51:cd:1c:98:b9:5e:e6:e4:02:d3:e3:71:
                    58:a1:60:fe:cb:e7:9b:4b:86:04:67:b5:4f:da:d6:
                    6c:08:3f:57:e9:70:59:57:48:6a:ce:e5:d4:f3:6e:
                    b2:fa:8a:18:7e:21:60:35:8f:44:f7:a9:39:57:16:
                    4f:4e:1e:b1:a3:77:32:c2:ef:d1:38:b4:82:20:8f:
                    11:0e:79:c4:d1:9b:f6:82:c4:08:84:84:68:d5:c3:
                    e2:15:a0:ce:23:3c:8d:9c:b8:dd:fc:3a:cd:42:ae:
                    5e:1b:80:2d:1b:e5:5d:1b:c1:fb:be:a3:9e:82:ff:
                    a1:27:c8:b6:0f:3c:cb:11:f9:1a:9b:d2:39:92:0e:
                    47:45:b8:8f:98:13:c6:4d:6a:18:75:a4:01:6f:73:
                    f6:f8:7f:eb:5d:59:94:46:d8:da:37:75:cf:27:0b:
                    39:7f:48:20:c5:fd:c7:a7:ce:22:9a:33:4a:30:1d:
                    95:ef:00:bd:fe:47:22:42:44:99:77:5a:c4:97:bb:
                    37:93:7c:33:64:f4:b8:3a:53:8c:f4:10:db:7f:5f:
                    2b:89:18:d6:0e:68:51:34:29:b1:f1:61:6b:4b:c6
```

KODEKLOUD

KODEKLOUD